GIÁO TRÌNH PHÁT TRIỂN PHẦN MỀM HƯỚNG DỊCH VỤ

Kiến trúc SOA & Microservices

Từ lý thuyết đến thực tiễn với case study KBM

Đặng Ngọc Hùng

Học viện Công nghệ Bưu chính Viễn thông (PTIT) · 2026

Kiến trúc SOA & Microservices — Từ lý thuyết đến thực tiễn với case study KBM
Đặng Ngọc Hùng
Phiên bản 1.2.7, 2026. Bản điện tử.

© 2026 Đặng Ngọc Hùng. Bảo lưu mọi quyền.

Bản điện tử được phát hành công khai để đọc và học tập, đồng thời dùng làm giáo trình tại Học viện Công nghệ Bưu chính Viễn thông (PTIT). Không sao chép, phân phối lại, dịch hoặc dùng cho mục đích thương mại khi chưa có sự đồng ý bằng văn bản của tác giả. Trích dẫn ngắn có ghi nguồn được phép theo thông lệ học thuật.

Trích dẫn: Đặng Ngọc Hùng (2026). *Kiến trúc SOA & Microservices*, phiên bản 1.2.7. Học viện Công nghệ Bưu chính Viễn thông (PTIT).

Tài liệu đi kèm (minh họa tương tác, hình gốc): hungdn1701.github.io/ptit-microservice-textbook
Liên hệ: hungdn@ptit.edu.vn

Lời nói đầu

Cuốn sách trình bày kiến trúc hướng dịch vụ (SOA) và microservices cho sinh viên, lập trình viên và kỹ sư phần mềm Việt Nam, qua một hệ thống thực tế được dùng xuyên suốt: KBM, hệ thống học tập và thực hành lập trình của một trường đại học. Các bài toán quen thuộc trong môi trường này — hệ thống đăng ký tín chỉ quá tải đầu học kỳ, hệ thống nộp bài nghẽn sát hạn chót — là điểm xuất phát cho từng quyết định kiến trúc trong sách.

Mục đích của sách

Đã có nhiều sách có giá trị về microservices, như *Microservices Patterns* (Chris Richardson), *Building Microservices* (Sam Newman) và *Designing Data-Intensive Applications* (Martin Kleppmann). Phần lớn viết bằng tiếng Anh và hướng đến người đã có kinh nghiệm. Cuốn sách này bổ sung ba điểm:

1. **Kế thừa nền tảng SOA.** Các nguyên lý hướng dịch vụ — hợp đồng dịch vụ (Service Contract), liên kết lỏng (Loose Coupling), trừu tượng hóa (Abstraction) — là cốt lõi của microservices. Sách trình bày microservices như bước tiến hóa tiếp theo của SOA, không tách rời khỏi SOA.
2. **Một case study xuyên suốt.** Cả 12 chương cùng phân tích KBM, một hệ thống LMS đang vận hành, thay vì các ví dụ rời rạc. KBM có nợ kỹ thuật (technical debt), có những đánh đổi (trade-off) và có thiết kế chưa tối ưu; sách dùng chính những điểm đó làm chất liệu: mỗi chương phân tích hiện trạng, chỉ ra khoảng trống và đề xuất lộ trình chuyển đổi.
3. **Bắt đầu từ bài toán.** Mỗi pattern được trình bày theo ba câu hỏi: vấn đề là gì, cách làm hiện có gặp giới hạn nào, và pattern giải quyết thế nào. Người đọc hiểu trade-off trước khi áp dụng kỹ thuật.

Đối tượng người đọc

- **Lập trình viên junior/mid-level**: đang làm việc với monolith, cần hiểu nguyên lý thiết kế dịch vụ và cách triển khai microservices. Đây là đối tượng chính khi cân nhắc độ sâu và nhịp trình bày.
- **Sinh viên CNTT năm 3–4**: học môn Phát triển phần mềm hướng dịch vụ hoặc cần nền tảng về kiến trúc phần mềm hiện đại.
- **Kỹ sư phần mềm**: cần tài liệu tham chiếu có hệ thống về các pattern, cách làm thực tế, và cơ sở để quyết định khi nào nên và không nên dùng microservices.

Sách không giảm độ sâu kỹ thuật để dễ đọc hơn. Ở lần đọc đầu, một số phần có thể cần tra cứu và đọc lại; hộp "Ôn lại kiến thức nền" (Chương 6), nhãn "Phần nâng cao (có thể lướt qua)" và Phụ lục A (Bảng thuật ngữ) được đặt ở những chỗ đó để hỗ trợ.

Kiến thức nền

- Java và Spring Boot ở mức đã tự viết một ứng dụng CRUD có JPA và REST controller.
- REST API, HTTP, SQL; cơ sở dữ liệu ở mức transaction và isolation level.
- Git và Docker ở mức cơ bản.
- Đọc được thuật ngữ tiếng Anh: sách giữ nhiều thuật ngữ gốc, định nghĩa ở Phụ lục A.

Về nội dung kiểm thử

Sách không có chương riêng về kiểm thử (testing); các phương pháp kiểm thử được trình bày trong ngữ cảnh sử dụng: kiểm thử hợp đồng (Contract Testing) ở Chương 3, Chaos Engineering ở Chương 11. Phụ lục G trình bày kiểm thử tích hợp, kiểm thử thành phần (Component Testing) và chiến lược kiểm thử chung; phần Bài tập có tình huống thiết kế kịch bản kiểm thử.

Quy ước trình bày

Nội dung bổ trợ trong các chương được đặt trong năm loại hộp, phân biệt bằng biểu tượng và màu (Bảng P.1). Tên trên nhãn mỗi hộp nêu chủ đề cụ thể của hộp đó; bảng ghi tên mặc định.

Bảng P.1: *Các loại hộp nội dung trong sách*

Hộp	Dùng cho
Nguyên tắc	Nguyên tắc thiết kế hoặc giải pháp khuyến nghị
Phân tích	Kịch bản, phân tích tình huống; đối chiếu hiện trạng KBM với giải pháp khuyến nghị
Sai lầm thường gặp	Anti-pattern, rủi ro và cách phòng tránh
Gợi ý	Mẹo thực hành, ẩn dụ, liên kết tới minh họa tương tác (Interactive Demo)
Lưu ý	Thông tin bổ sung, kiến thức nền, phần nâng cao có thể lướt qua

Trích dẫn trong văn bản có dạng **[mã, vị trí]** ví dụ [2a, Ch.3] là chương 3 của sách mã 2a. Mã của các sách tham khảo chính cho trong Bảng P.2; danh mục đầy đủ ở phần Tài liệu tham khảo.

Bảng P.2: *Mã tài liệu tham khảo chính dùng trong sách*

Mã	Tên sách	Tác giả	Ấn bản	Năm
[1]	*Service-Oriented Architecture: Analysis and Design for Services and Microservices*	Thomas Erl	2	2016
[2a]	*Microservices Patterns*	Chris Richardson	1	2018
[2b]	*Microservices Patterns*	Chris Richardson	2	2025
[3]	*Microservices: Up and Running*	Ronnie Mitra, Irakli Nadareishvili	1	2020
[4a]	*Building Microservices*	Sam Newman	1	2015
[4b]	*Monolith to Microservices*	Sam Newman	1	2019
[5]	*Practical Event-Driven Microservices Architecture*	Hugo Rocha	1	2022
[6]	*Domain-Driven Design*	Eric Evans	1	2003
[7]	*Designing Data-Intensive Applications*	Martin Kleppmann	1	2017

Lời cảm ơn

Cuốn sách được hình thành từ quá trình giảng dạy và phát triển hệ thống KBM. Tác giả trân trọng cảm ơn những người đã góp phần tạo nên nền tảng thực tế cho các ví dụ trong sách.

Xin cảm ơn các cộng tác viên đã tham gia phát triển, triển khai và vận hành KBM qua nhiều giai đoạn thay đổi yêu cầu và hạ tầng, cùng các thế hệ sinh viên từ D17 đến nay đã sử dụng hệ thống trong học tập, kiểm thử và phản hồi lỗi. Những kinh nghiệm và tình huống từ quá trình đó là chất liệu cho các thảo luận về phân rã dịch vụ, giao tiếp, dữ liệu phân tán, triển khai và vận hành trong sách. Danh sách người đóng góp có ở Phụ lục E của bản web.

Nội dung sách chịu ảnh hưởng từ công trình của Chris Richardson, Sam Newman, Martin Kleppmann, Thomas Erl, Martin Fowler và cộng đồng kiến trúc phần mềm; đó là những nguồn tham khảo chính khi hệ thống hóa nội dung về SOA và microservices cho bối cảnh đào tạo tại Việt Nam.

Mọi thiếu sót trong nội dung, diễn giải và lựa chọn ví dụ thuộc trách nhiệm của tác giả.

Giới thiệu

Phạm vi của sách

Sách không hướng dẫn xây dựng một hệ thống microservices từ đầu. Mục tiêu là giúp người đọc hiểu microservices: vì sao cần, khi nào nên dùng, phải đánh đổi những gì, và chuyển đổi từ monolith như thế nào. Lịch sử từ monolith, SOA đến microservices được trình bày ở Chương 1.

Mỗi chương đi theo cùng một trình tự: bài toán thực tế, pattern giải quyết, cách áp dụng cho KBM và lộ trình chuyển đổi.

Cấu trúc của sách

Sách gồm ba phần, mười hai chương, phần Kết luận và các phụ lục.

Phần I. Nền tảng (Chương 1–3)

Chương 1 Tổng quan SOA & Microservices: bài học từ Netflix, Amazon, Uber.
Chương 2 Phân tích hướng domain: Domain-Driven Design, Bounded Context, Context Map, Event Storming, tổ chức đội phát triển.
Chương 3 Thiết kế dịch vụ & API: REST, quản lý phiên bản (versioning), tiến hóa schema, tài liệu hóa API.

Phần II. Giao tiếp & dữ liệu (Chương 4–7)

Chương 4 Giao tiếp đồng bộ: REST, gRPC, OpenFeign và các resilience pattern.
Chương 5 Giao tiếp bất đồng bộ: Kafka, kiến trúc hướng sự kiện (event-driven), message broker.
Chương 6 Giao dịch phân tán: Saga Pattern, orchestration và choreography.
Chương 7 Quản lý dữ liệu: database-per-service, CQRS, Event Sourcing.

Phần III. Hạ tầng & vận hành (Chương 8–12)

Chương 8 API Gateway và BFF: routing, nghiệp vụ cắt ngang (cross-cutting concern), Spring Cloud Gateway.
Chương 9 Bảo mật Microservices: OAuth 2.0, JWT, RBAC, mTLS và Zero Trust.
Chương 10 Chuyển đổi thực tế: Strangler Fig, tách cơ sở dữ liệu, lộ trình chuyển đổi.
Chương 11 Quan sát hệ thống: Observability, log, metric, tracing, cảnh báo.
Chương 12 Triển khai & DevOps: Docker, CI/CD, chiến lược triển khai, IaC.

Phụ lục

Phụ lục A–D là tài liệu tra cứu: bảng thuật ngữ, công cụ, danh mục pattern và anti-pattern. Phụ lục E mô tả đầy đủ case study KBM. Phần Bài tập & case study, Phụ lục F (gợi ý hướng giải) và Phụ lục G (chiến lược kiểm thử) dùng cho luyện tập.

Cách sử dụng sách

Sách được thiết kế để đọc tuần tự: mỗi chương dựa trên kiến thức của chương trước. Khi cần tra cứu nhanh, Phụ lục C liệt kê 58 pattern kèm chương trình bày. Các minh họa tương tác (Interactive Demo) nêu trong từng chương nằm trên trang tài liệu đi kèm, địa chỉ ghi ở trang bản quyền.

Tùy mục tiêu, người đọc có thể chọn một lộ trình trong Bảng I.1.

Bảng I.1: *Lộ trình đọc theo nhóm người đọc*

Nhóm người đọc	Gợi ý lộ trình
Sinh viên năm 3–4 (lần đầu tiếp cận)	Đọc tuần tự từ Chương 1 đến Chương 12; đọc lướt các mục "Phần nâng cao" ở Chương 1–2 và một số chương sau để quay lại khi cần. Dùng Phụ lục A khi gặp thuật ngữ lạ. Nắm phần "Vấn đề" đầu mỗi chương trước khi đi vào pattern.
Lập trình viên junior/mid đang bảo trì monolith	Chương 1 (khi nào nên và không nên), Chương 2 (Bounded Context), Chương 10 (Strangler Fig, lộ trình chuyển đổi); quay lại Phần II khi cần kỹ thuật cụ thể.
Chuẩn bị phỏng vấn thiết kế hệ thống	Phần II (Chương 4–7) và Chương 9, sau đó luyện phần Bài tập (trade-off, ADR, gỡ lỗi).
Kỹ sư phần mềm cần ra quyết định kiến trúc	Chương 1 (khung ra quyết định), Chương 2 (Conway's Law), Chương 10 và Kết luận (bảng heuristic "khi nào dùng pattern nào"); phần lý thuyết chi tiết tra cứu khi cần.

Mục lục

Lời nói đầu i
Lời cảm ơn iii
Giới thiệu iv
Danh mục hình ảnh x
Danh mục bảng biểu xii

PHẦN I. NỀN TẢNG 1

1. Tổng quan SOA & Microservices 2
1.1. Kiến trúc phần mềm theo thời gian 3
1.2. SOA là gì? Nguyên lý cốt lõi 5
1.3. Microservices là gì? So sánh với SOA 6
1.4. Microservices trong thực tế — bài học từ các công ty công nghệ 8
1.5. Đo lường và đánh giá hiệu năng kiến trúc 10
1.6. Modular Monolith — lựa chọn hợp lệ 13
1.7. Các kịch bản áp dụng và hạn chế của Microservices 14
1.8. Case study: KBM 16
1.9. Tổng kết 21
1.10. Đọc thêm 21

2. Phân tích hướng domain — DDD & Bounded Contexts 23
2.1. Conway's Law — tổ chức quyết định kiến trúc 24
2.2. DDD cơ bản — ngôn ngữ chung và các building blocks 26
2.3. Strategic DDD — Bounded Context & Context Map 28
2.4. Case study: KBM 34
2.5. Shared Library và Shared Kernel — chia sẻ hay không chia sẻ? 37
2.6. Dark Energy & Dark Matter — 10 lực chi phối việc phân tách service 39
2.7. Assemblage Process — quy trình thiết kế service architecture 41
2.8. Tổng kết 41
2.9. Đọc thêm 42

3. Thiết kế dịch vụ & API 44
3.1. REST API Design — nguyên tắc cho Microservices 45
3.2. API Versioning & Schema Evolution 49
3.3. OpenAPI & API Documentation 51
3.4. DTO Pattern — tách Internal Model và External Contract 53
3.5. Hexagonal Architecture — tách biệt lõi nghiệp vụ khỏi hạ tầng 55
3.6. Contract Testing — kiểm thử hợp đồng API 56
3.7. GraphQL — khi REST không đủ linh hoạt 58
3.8. Ranh giới của chương này 61
3.9. Case study: KBM 61
3.10. Tổng kết 63
3.11. Đọc thêm 64

PHẦN II. GIAO TIẾP & DỮ LIỆU 65

4. Giao tiếp đồng bộ — REST, gRPC & Resilience . . . 66
4.1. Giao tiếp đồng bộ — khi nào nên, khi nào tránh? . . . 67
4.2. OpenFeign — Declarative REST Client . . . 71
4.3. Service Discovery & Load Balancing . . . 72
4.4. Resilience Patterns — sống sót trong Distributed System . . . 74
4.5. Case study: KBM . . . 82
4.6. Tổng kết . . . 86
4.7. Đọc thêm . . . 87

5. Giao tiếp bất đồng bộ — Kafka, Events & Messaging . . . 88
5.1. Động lực của giao tiếp bất đồng bộ . . . 89
5.2. Message Broker — hai trường phái thiết kế . . . 91
5.3. Apache Kafka — kiến trúc chi tiết . . . 95
5.4. Producer/Consumer Pattern trong KBM . . . 96
5.5. Delivery Guarantees & Idempotency . . . 99
5.6. Event Schema Design . . . 100
5.7. Kafka vs DB Queue — chọn broker theo bài toán . . . 102
5.8. WebSocket — Real-time Notifications . . . 102
5.9. Case study: KBM . . . 103
5.10. Tổng kết . . . 106
5.11. Đọc thêm . . . 106

6. Giao dịch phân tán — Saga Pattern . . . 108
6.1. Độ phức tạp của giao dịch phân tán . . . 109
6.2. Saga Pattern — chuỗi Local Transaction + Compensation . . . 112
6.3. Choreography vs Orchestration . . . 114
6.4. Giao dịch bù trừ (Compensating Transaction) & Isolation . . . 117
6.5. Eventual Consistency — chấp nhận và quản lý . . . 122
6.6. Case study: KBM . . . 123
6.7. Tổng kết . . . 128
6.8. Đọc thêm . . . 128

7. Quản lý dữ liệu — Database per Service & CQRS . . . 130
7.1. Database-per-Service — nguyên tắc Data Ownership . . . 131
7.2. Chiến lược tách database từ monolith . . . 137
7.3. Data Duplication vs Coupling — khi nào chấp nhận nhân bản dữ liệu? . . . 139
7.4. CQRS — Command Query Responsibility Segregation . . . 142
7.5. Event Sourcing — lưu event thay vì state . . . 146
7.6. Case study: KBM . . . 151
7.7. Tổng kết . . . 154
7.8. Đọc thêm . . . 155

PHẦN III. HẠ TẦNG & VẬN HÀNH 156

8. API Gateway & BFF Pattern . . . 157
8.1. Động lực sử dụng API Gateway . . . 158
8.2. Spring Cloud Gateway — Reactive Gateway . . . 161
8.3. Cấu hình route với Eureka . . . 162
8.4. Nghiệp vụ cắt ngang (cross-cutting concern) tại Gateway . . . 164
8.5. Case study: KBM . . . 172
8.6. Tổng kết . . . 175
8.7. Đọc thêm . . . 176

9. Bảo mật Microservices — OAuth 2.0, mTLS & Zero Trust . . . 177
9.1. Các thách thức bảo mật trong kiến trúc microservices . . . 178
9.2. JWT — cấu trúc và cơ chế . . . 181
9.3. Tiered Trust và xác minh theo tài nguyên . . . 186
9.4. OAuth2 — nhà cung cấp danh tính bên ngoài . . . 188
9.5. RBAC — Role-Based Access Control . . . 190
9.6. Case study: KBM . . . 192
9.7. Tổng kết . . . 197
9.8. Đọc thêm . . . 198

10. Chuyển đổi thực tế — từ Monolith đến Microservices . . . 200
10.1. Thời điểm thích hợp để chuyển đổi . . . 201
10.2. Strangler Fig Pattern — chuyển đổi tăng dần . . . 203
10.3. Tách database — thách thức lớn . . . 206
10.4. Anti-Corruption Layer và các pattern chuyển đổi . . . 209

10.5. Quá trình chuyển đổi kiến trúc tại KBM . . . 212
10.6. Sai lầm thường gặp khi chuyển đổi . . . 216
10.7. Tổng kết . . . 216
10.8. Đọc thêm . . . 217

11. Quan sát hệ thống — Observability & Distributed Tracing . . . 219
11.1. Ba trụ cột của observability . . . 220
11.2. Logging — từ console đến gom log tập trung . . . 222
11.3. Chi tiết về Distributed Tracing . . . 226
11.4. Metrics, SLI/SLO và chiến lược cảnh báo . . . 230
11.5. Chiến lược xử lý lỗi — nhất quán giữa các service . . . 233
11.6. Health check và Service Discovery . . . 235
11.7. User Activity Tracking — observability ở mức nghiệp vụ . . . 235
11.8. Chaos Engineering — kiểm tra khả năng phục hồi (resilience) chủ động . . . 236
11.9. Case study: KBM . . . 238
11.10. Tổng kết . . . 241
11.11. Đọc thêm . . . 241

12. Triển khai & DevOps . . . 243
12.1. Thách thức triển khai microservices . . . 244
12.2. Containerization với Docker . . . 245
12.3. Docker Compose — điều phối trên một máy . . . 248
12.4. Pipeline CI/CD cho microservices . . . 250
12.5. Các chiến lược triển khai . . . 257
12.6. Infrastructure as Code và nền tảng điều phối triển khai . . . 260
12.7. Case study: KBM . . . 268
12.8. Tổng kết . . . 271
12.9. Đọc thêm . . . 272

Kết luận . . . 273

PHỤ LỤC 276

A. Bảng thuật ngữ . . . 277

B. Công cụ & tài nguyên . . . 284
B.1. Framework & thư viện . . . 284
B.2. Messaging & dữ liệu . . . 285
B.3. Hạ tầng & DevOps . . . 285
B.4. Chuyển đổi & quản lý phát hành . . . 285
B.5. Observability . . . 285
B.6. Bảo mật & danh tính . . . 286
B.7. Công cụ phát triển . . . 286

C. Pattern Catalog . . . 287
C.1. Pattern giao tiếp . . . 287
C.2. Pattern resilience . . . 287
C.3. Pattern dữ liệu . . . 288
C.4. Pattern hạ tầng . . . 288
C.5. Pattern bảo mật . . . 289
C.6. Pattern observability . . . 289
C.7. Pattern triển khai . . . 289
C.8. Pattern chuyển đổi . . . 290
C.9. Pattern DDD . . . 290
C.10. Pattern kiểm thử . . . 290

D. Anti-pattern Catalog . . . 291
D.1. Kiến trúc & thiết kế . . . 291
D.2. Thiết kế API . . . 292
D.3. Giao tiếp . . . 292
D.4. Giao dịch & dữ liệu . . . 293
D.5. Bảo mật & Gateway . . . 294
D.6. Chuyển đổi & observability . . . 295
D.7. Triển khai & DevOps . . . 296

E. Case study KBM . . . 297
E.1. Bối cảnh bài toán . . . 297
E.2. Bốn trụ cột nghiệp vụ . . . 298
E.3. Quá trình tiến hóa — từ Monolith đến Microservices . . . 300
E.4. Kiến trúc hiện tại . . . 302

E.5. Trade-off và khoảng trống — bài học từ thực tế . . . 305
E.6. Sự cố thực tế và bài học . . . 306
E.7. Ánh xạ giữa case study và các chương . . . 308

Bài tập & case study . . . 309

F. Gợi ý hướng giải . . . 331

G. Chiến lược kiểm thử & Testcontainers . . . 337
G.1. Chiến lược kiểm thử trong microservices . . . 337
G.2. Tháp kiểm thử (Test Pyramid) trong hệ thống phân tán . . . 337
G.3. Thay thế mock bằng Testcontainers . . . 338
G.4. Case study 1: Kiểm thử idempotency với Kafka . . . 338
G.5. Case study 2: Kiểm thử chuyển trạng thái khi Saga gặp lỗi . . . 340
G.6. Consumer-Driven Contract Testing . . . 340

Tài liệu tham khảo . . . 342

Danh mục hình ảnh

Hình 1.1. Tiến hóa kiến trúc phần mềm doanh nghiệp từ thập niên 1990 đến 2020 4
Hình 1.2. Bốn kiểu kiến trúc SOA — từ dịch vụ đơn lẻ đến quy mô doanh nghiệp 6
Hình 1.3. Scale Cube — ba chiều mở rộng hệ thống 7
Hình 1.4. Cây quyết định chuyển đổi kiến trúc — từ monolith đến microservices 9
Hình 1.5. Fast Flow Success Triangle — ba yếu tố cần thiết cho fast flow 11
Hình 1.6. Con đường từ Patchwork Monolith đến Microservices 14
Hình 1.7. Khung ra quyết định (Decision Framework) — ba câu hỏi trước khi chọn microservices 15
Hình 1.8. Kiến trúc tổng quan KBM LMS (sơ đồ giản lược, xem đầy đủ ở Chương 2) 18
Hình 2.1. Conway's Law (thụ động) vs Inverse Conway Maneuver (chủ động) 24
Hình 2.2. Mô hình Squad/Tribe/Chapter/Guild của Spotify 25
Hình 2.3. Các building blocks trong DDD — Aggregate, Entity, Value Object, Repository 27
Hình 2.4. Cùng thuật ngữ "Submission" mang ý nghĩa khác nhau trong ba Bounded Context 29
Hình 2.5. Context Map của KBM — quan hệ giữa các Bounded Context 30
Hình 2.6. Event Storming — luồng từ Command đến Domain Event 31
Hình 2.7. Áp dụng Erl's Service-Oriented Analysis cho KBM — từ automation candidate đến service layer . 33
Hình 2.8. Bảy Bounded Context core của KBM 35
Hình 2.9. Hướng tách shared library — từ một khối sang nhiều module 38
Hình 3.1. Richardson Maturity Model — bốn cấp độ trưởng thành của REST API 45
Hình 3.2. Quy tắc schema evolution — thay đổi nào là breaking change 51
Hình 3.3. Pipeline từ code annotations đến Swagger UI và client SDK 52
Hình 3.4. DTO pattern — tách internal model và external contract 54
Hình 3.5. Hexagonal Architecture cho Core Service trong KBM 55
Hình 4.1. Giao tiếp đồng bộ — Core Service bị chặn (blocked) trong khi chờ Judge phản hồi 67
Hình 4.2. Cách Feign proxy tự động xử lý HTTP request, JWT, load balancing 72
Hình 4.3. Client-side discovery — service tự truy vấn Eureka và tự cân bằng tải 73
Hình 4.4. Server-side discovery — load balancer điều hướng request 73
Hình 4.5. Cascading failure — Service B chậm làm A, D, E đều bị ảnh hưởng 76
Hình 4.6. Circuit Breaker state machine — Closed, Open, Half-Open 77
Hình 4.7. Bulkhead pattern — thread pool cô lập theo từng chức năng 78
Hình 4.8. Bộ điều phối `kbm-judge-sql` — điều hướng SQL đến các worker `kbm-sandbox-*` theo DBMS 83
Hình 4.9. Lộ trình chuyển đổi cho SQL Judge — từ resilience đến bất đồng bộ 84
Hình 5.1. So sánh giao tiếp đồng bộ (coupled) và bất đồng bộ (decoupled) 89
Hình 5.2. Point-to-Point (Queue) vs Publish/Subscribe (Topic) 91
Hình 5.3. Ephemeral broker (message xóa sau ack) vs Durable broker (message vẫn còn) 92
Hình 5.4. Kiến trúc RabbitMQ — Exchange định tuyến message đến các queue 93
Hình 5.5. Kiến trúc Kafka — Topic, Partition và Consumer Group 96
Hình 5.6. Luồng hoàn chỉnh — từ lúc nộp bài qua Kafka đến kết quả qua WebSocket 98
Hình 5.7. Kiến trúc pipeline 2 Kafka topic chấm bài trong chế độ thi 104
Hình 6.1. Two-Phase Commit — Coordinator điều phối prepare và commit 110
Hình 6.2. KBM Submit Saga — chuỗi local transaction và compensating transaction 113
Hình 6.3. Choreography — các service tự phối hợp qua event 115
Hình 6.4. Orchestration — saga orchestrator điều phối toàn bộ luồng xử lý 116
Hình 6.5. Lost Update anomaly — hai saga ghi đè kết quả của nhau 118
Hình 6.6. Implicit Saga trong KBM — choreography qua sự kiện Kafka 124
Hình 7.1. Ba hậu quả của shared database trong microservices 132
Hình 7.2. Shared Database (đọc/ghi chung) vs Database-per-Service (dữ liệu qua API) 133
Hình 7.3. CAP Theorem — CP (chọn consistency) vs AP (chọn availability) khi xảy ra network partition . . 135
Hình 7.4. Các caching pattern — Cache-Aside, Read-Through/Write-Through, Write-Behind 136
Hình 7.5. Năm chiến lược tách database, theo thứ tự thường gặp khi chuyển đổi 138
Hình 7.6. Truy cập dữ liệu xuyên service bằng API Call 140

Hình 7.7. Truy cập dữ liệu xuyên service bằng Data Duplication qua event pipeline . . . 140
Hình 7.8. API Composition pattern — Gateway hoặc BFF gọi nhiều service rồi tổng hợp . . . 141
Hình 7.9. CQRS pattern — tách command model (write) và query model (read) . . . 143
Hình 7.10. Đường CQRS bất đồng bộ qua Kafka cho bảng xếp hạng . . . 146
Hình 7.11. Event Sourcing — lưu chuỗi sự kiện thay vì chỉ lưu trạng thái hiện tại . . . 147
Hình 7.12. Snapshot pattern — replay từ snapshot thay vì từ đầu . . . 148
Hình 7.13. Projection Rebuilds — dựng read model mới bằng replay, không sửa event store . . . 149
Hình 7.14. Kiến trúc dữ liệu hiện tại của KBM — shared database là nợ kỹ thuật chuyển tiếp . . . 151
Hình 8.1. Không có Gateway — client phải biết địa chỉ của từng service . . . 158
Hình 8.2. API Gateway — điểm vào duy nhất, định tuyến đến từng service . . . 159
Hình 8.3. API Gateway (một Gateway chung) và BFF (Gateway riêng cho từng client) . . . 160
Hình 8.4. Kiến trúc Spring Cloud Gateway — Route, Predicate, Filter . . . 162
Hình 8.5. Luồng `lb://` — Eureka lookup + load balance + route . . . 164
Hình 8.6. Luồng JWT tại Gateway — bằng chứng có chữ ký và context định danh dẫn xuất . . . 167
Hình 8.7. Kiến trúc tổng thể KBM — Gateway là điểm vào duy nhất . . . 172
Hình 9.1. Monolith (1 điểm bảo vệ) vs Microservices (N điểm bảo vệ) . . . 178
Hình 9.2. Phòng thủ chiều sâu — bảo vệ nhiều lớp từ tầng mạng đến tầng dữ liệu . . . 179
Hình 9.3. Cấu trúc JWT dạng compact JWS — Header, Payload, Signature . . . 181
Hình 9.4. Luồng JWT mục tiêu — kiểm tra ở ingress, bằng chứng có chữ ký và phân quyền cục bộ . . . 183
Hình 9.5. Luồng làm mới và luân phiên token — access token mới + refresh token mới . . . 184
Hình 9.6. Luồng định danh KBM — rủi ro được chấp nhận tại thời điểm khảo sát và mục tiêu Tier 1 . . . 187
Hình 9.7. Phân cấp vai trò — ADMIN kế thừa LECTURER, LECTURER kế thừa STUDENT . . . 191
Hình 9.8. Kiến trúc bảo mật tổng thể của KBM . . . 192
Hình 9.9. Vault Dynamic Secret — thông tin đăng nhập tạm thời, tự động luân phiên . . . 195
Hình 10.1. Strangler Fig Pattern — dần thay thế monolith qua 3 giai đoạn . . . 204
Hình 10.2. API Gateway làm điểm tiếp nối cho Strangler Fig — client không biết route nào đến monolith, route nào đến service mới . . . 204
Hình 10.3. Shared Database (coupled) vs Database-per-Service (decoupled) . . . 206
Hình 10.4. Thứ tự thực hiện 5 chiến lược tách database . . . 207
Hình 10.5. Cạm bẫy ghi kép — DB thành công nhưng event thất bại . . . 207
Hình 10.6. Outbox Pattern — sự kiện được ghi cùng giao dịch với dữ liệu nghiệp vụ . . . 208
Hình 10.7. Anti-Corruption Layer — lớp dịch giữa service mới và legacy . . . 210
Hình 10.8. Branch by Abstraction — chuyển đổi an toàn qua feature flag . . . 211
Hình 10.9. Parallel Run — chạy song song logic cũ và mới, so sánh kết quả . . . 211
Hình 10.10. Lộ trình chuyển đổi 4 giai đoạn — Quick Wins → Observability → Resilience → Decomposition . . . 213
Hình 10.11. Ma trận quyết định — ưu tiên theo tác động × nỗ lực . . . 215
Hình 11.1. Luồng request qua 5+ service — lỗi xảy ra ở đâu? . . . 220
Hình 11.2. Ba trụ cột của observability — Logs, Metrics, Traces . . . 221
Hình 11.3. Log phân tán (gỡ lỗi thủ công) vs Log tập trung — tìm kiếm tại một nơi . . . 223
Hình 11.4. Pipeline gom log tập trung — từ service đến nơi lưu trữ . . . 224
Hình 11.5. Truyền định danh xuyên suốt request HTTP và luồng sự kiện Kafka . . . 225
Hình 11.6. Biểu đồ Gantt của một trace — thời gian của từng span . . . 227
Hình 11.7. Mối quan hệ giữa SLI, SLO và SLA . . . 232
Hình 11.8. User Activity Tracking pipeline qua Kafka . . . 236
Hình 11.9. Đánh giá hiện trạng observability của KBM . . . 238
Hình 12.1. Triển khai thủ công và tự động . . . 245
Hình 12.2. So sánh kiến trúc máy ảo và container . . . 246
Hình 12.3. Pipeline CI/CD chuẩn cho microservices . . . 251
Hình 12.4. Quá trình Rolling Update . . . 257
Hình 12.5. Kiến trúc Blue/Green Deployment . . . 258
Hình 12.6. Kiến trúc Kubernetes — Control Plane và Worker Node . . . 262
Hình 12.7. Sidecar Pattern — proxy xử lý các nghiệp vụ cắt ngang (mTLS, tracing) . . . 267
Hình 12.8. Kiến trúc triển khai của KBM ở mức khái quát . . . 269

Danh mục bảng biểu

Bảng 1.1. Các biểu hiện khi monolith đạt đến giới hạn . . . 3
Bảng 1.2. Tám nguyên lý hướng dịch vụ — SOA truyền thống vs Microservices . . . 5
Bảng 1.3. So sánh SOA truyền thống và Microservices . . . 7
Bảng 1.4. DORA Metrics — so sánh Elite vs Low Performers . . . 12
Bảng 1.5. Lợi ích của kiến trúc microservices . . . 15
Bảng 1.6. Chi phí và thách thức của microservices . . . 15
Bảng 1.7. Ánh xạ bài toán kỹ thuật KBM LMS theo chương . . . 19
Bảng 1.8. Năm quyết định kiến trúc KBM muốn làm lại . . . 20
Bảng 2.1. Bốn kiểu team theo Team Topologies . . . 25
Bảng 2.2. Cấu trúc tổ chức Spotify — ý nghĩa cho microservices . . . 25
Bảng 2.3. Các kiểu quan hệ giữa các Bounded Context . . . 29
Bảng 2.4. Hai phương pháp phân tách service — Erl Service-Oriented Analysis vs DDD . . . 33
Bảng 2.5. Ánh xạ Bounded Context → service trong KBM . . . 36
Bảng 2.6. Phân tích shared library — cái gì nên dùng chung, cái gì không . . . 37
Bảng 2.7. Mười lực Dark Energy / Dark Matter . . . 40
Bảng 3.1. Các mã trạng thái HTTP thường dùng trong microservices . . . 46
Bảng 3.2. Tiêu chí lựa chọn — chọn chiến lược versioning nào? . . . 49
Bảng 3.3. So sánh Pact và Spring Cloud Contract . . . 57
Bảng 3.4. REST vs GraphQL — so sánh chi tiết . . . 59
Bảng 3.5. Khi nào chọn REST, khi nào chọn GraphQL . . . 60
Bảng 4.1. Ưu và nhược điểm của giao tiếp đồng bộ . . . 67
Bảng 4.2. Hai loại coupling trong microservices . . . 68
Bảng 4.3. REST vs gRPC — so sánh chi tiết . . . 69
Bảng 4.4. Các mức “nines” của tính sẵn sàng — downtime cho phép theo từng mục tiêu . . . 75
Bảng 4.5. Giá trị khuyến nghị cho từng môi trường . . . 82
Bảng 4.6. Phân tích vấn đề SQL Judge coordinator/worker . . . 83
Bảng 5.1. Khi nào chọn Kafka, khi nào chọn RabbitMQ . . . 94
Bảng 5.2. Các khái niệm cốt lõi của Apache Kafka . . . 96
Bảng 5.3. Bốn loại message trong kiến trúc hướng sự kiện . . . 101
Bảng 5.4. Chọn Kafka hay DB queue theo bài toán . . . 102
Bảng 5.5. Chi tiết các Kafka topic trong pipeline . . . 104
Bảng 5.6. Phân tích vấn đề Kafka pipeline trong KBM . . . 105
Bảng 7.1. Ba cấp độ tách database . . . 133
Bảng 7.2. CP vs AP cho từng luồng dữ liệu trong KBM . . . 135
Bảng 7.3. So sánh ba caching pattern . . . 137
Bảng 7.4. Lộ trình tách database cho KBM theo hướng schema trước (3 bước chọn lọc từ danh mục 5 chiến lược) . . . 139
Bảng 7.5. Khi nào chấp nhận nhân bản dữ liệu (data duplication) . . . 141
Bảng 7.6. Khi nào cần CQRS . . . 144
Bảng 7.7. Ưu và nhược điểm của Event Sourcing . . . 147
Bảng 7.8. Event Store — các công cụ phổ biến . . . 149
Bảng 7.9. Khi nào dùng Event Sourcing . . . 150
Bảng 7.10. Tổng hợp vấn đề quản lý dữ liệu trong KBM và chiến lược chuyển đổi . . . 152
Bảng 8.1. Phân tích cấu hình Gateway trong KBM . . . 173
Bảng 9.1. Năm thách thức bảo mật đặc thù khi chuyển từ monolith sang microservices . . . 178
Bảng 9.2. HS256 (đối xứng) vs RS256 (bất đối xứng) . . . 182
Bảng 9.3. Access token, refresh token và token family — thời hạn, nơi lưu trữ và lý do thiết kế . . . 186
Bảng 9.4. Bốn tầng tin cậy dùng xuyên suốt sách . . . 187
Bảng 9.5. Các phương thức xác thực trong KBM . . . 189
Bảng 9.6. Phân tích bảo mật KBM . . . 192

Bảng 9.7. Chiến lược Secret Management — từ cơ bản đến quy mô doanh nghiệp . . . 194
Bảng 10.1. Ma trận quyết định (Decision Matrix) — khi nào cân nhắc chuyển đổi . . . 201
Bảng 10.2. Các anti-pattern khi chuyển đổi sang microservices (minh họa với KBM) . . . 202
Bảng 10.3. 5 chiến lược tách database — từ góc độ chuyển đổi . . . 206
Bảng 10.4. Tổng hợp khoảng trống của KBM tại thời điểm khảo sát (Chương 2–9) và hai điều kiện tiên quyết xem trước (Chương 11–12) . . . 212
Bảng 10.5. Giai đoạn 1 — Quick Wins . . . 213
Bảng 10.6. Giai đoạn 2 — Nền tảng Observability . . . 214
Bảng 10.7. Giai đoạn 3 — Resilience & CI/CD . . . 214
Bảng 10.8. Giai đoạn 4 — Phân tách cơ sở dữ liệu . . . 215
Bảng 11.1. Vai trò của ba trụ cột observability . . . 221
Bảng 11.2. Phân biệt monitoring và observability . . . 221
Bảng 11.3. Mô hình trưởng thành observability . . . 222
Bảng 11.4. So sánh hai stack ELK và PLG . . . 224
Bảng 11.5. Các khái niệm cơ bản trong Distributed Tracing . . . 227
Bảng 11.6. Các chiến lược lấy mẫu cho tracing . . . 228
Bảng 11.7. Bốn observability pattern cần triển khai . . . 230
Bảng 11.8. SLO đề xuất cho KBM . . . 232
Bảng 11.9. Chiến lược cảnh báo theo mức độ nghiêm trọng . . . 233
Bảng 11.10. Phân loại hệ thống mã lỗi (`errorCode` dạng chuỗi có namespace) . . . 233
Bảng 11.11. Ba loại health check . . . 235
Bảng 11.12. Ứng dụng của User Activity Tracking . . . 236
Bảng 11.13. Phân loại kỹ thuật tiêm lỗi trong Chaos Engineering . . . 237
Bảng 11.14. Netflix Simian Army . . . 238
Bảng 11.15. Mức độ trưởng thành observability phân theo thành phần . . . 239
Bảng 11.16. Các chế độ hoạt động chính của LMS . . . 239
Bảng 12.1. So sánh chi tiết Docker Compose và Kubernetes . . . 264
Bảng 12.2. So sánh container (Docker/K8s) và serverless . . . 266
Bảng 12.3. Phân tích mức độ trưởng thành triển khai của KBM . . . 269
Bảng K.1. Quá trình chuyển đổi của KBM qua 12 chương — vấn đề và giải pháp . . . 273
Bảng K.2. Heuristic — khi nào áp dụng pattern nào . . . 274
Bảng B.1. Framework và thư viện . . . 284
Bảng B.2. Công cụ messaging và dữ liệu . . . 285
Bảng B.3. Công cụ hạ tầng và DevOps . . . 285
Bảng B.4. Công cụ chuyển đổi và quản lý phát hành . . . 285
Bảng B.5. Công cụ observability . . . 286
Bảng B.6. Công cụ bảo mật và định danh . . . 286
Bảng B.7. Công cụ phát triển . . . 286
Bảng C.1. Pattern giao tiếp . . . 287
Bảng C.2. Pattern resilience . . . 287
Bảng C.3. Pattern dữ liệu . . . 288
Bảng C.4. Pattern hạ tầng . . . 288
Bảng C.5. Pattern bảo mật . . . 289
Bảng C.6. Pattern observability . . . 289
Bảng C.7. Pattern triển khai . . . 289
Bảng C.8. Pattern chuyển đổi . . . 290
Bảng C.9. Pattern DDD . . . 290
Bảng C.10. Pattern kiểm thử . . . 290
Bảng D.1. Anti-pattern kiến trúc và thiết kế . . . 291
Bảng D.2. Anti-pattern thiết kế API . . . 292
Bảng D.3. Anti-pattern giao tiếp . . . 292
Bảng D.4. Anti-pattern giao dịch và dữ liệu . . . 293
Bảng D.5. Anti-pattern bảo mật và Gateway . . . 294
Bảng D.6. Anti-pattern chuyển đổi và observability . . . 295
Bảng D.7. Anti-pattern triển khai và DevOps . . . 296
Bảng E.1. Bốn trụ cột nghiệp vụ KBM và yêu cầu kiến trúc đặc trưng . . . 298
Bảng E.2. So sánh tech stack giữa LMS chính (Java) và DevOps Lab (Go) . . . 303
Bảng E.3. Danh sách dịch vụ KBM theo Bounded Context . . . 303
Bảng E.4. Ba kiểu tích hợp hệ thống bên ngoài trong KBM . . . 304
Bảng E.5. Trade-off mức nghiêm trọng — ảnh hưởng tính đúng đắn và bảo mật . . . 305
Bảng E.6. Trade-off mức quan trọng — ảnh hưởng khả năng mở rộng và bảo trì . . . 305
Bảng E.7. Trade-off mức nhỏ — cải thiện chất lượng . . . 306
Bảng E.8. Ánh xạ nghiệp vụ KBM ↔ chương sách ↔ sự cố thực tế . . . 308

PHẦN I

Nền tảng

Phần I đặt nền tảng: bức tranh chung về SOA và microservices, phân tích miền nghiệp vụ bằng Domain-Driven Design, và thiết kế dịch vụ cùng hợp đồng API. Sau phần này, người đọc có thể nhận diện ranh giới dịch vụ trong một hệ thống monolith.

Chương 1 Tổng quan SOA & Microservices
Chương 2 Phân tích hướng domain — DDD & Bounded Contexts
Chương 3 Thiết kế dịch vụ & API

CHƯƠNG 1

Tổng quan SOA & Microservices

> "Microservices are not the goal. You don't 'win' by having microservices."
> — Sam Newman, *Monolith to Microservices* [4b, Ch.2]

MỤC TIÊU HỌC TẬP

- Hiểu bối cảnh lịch sử dẫn đến kiến trúc hướng dịch vụ (SOA) và microservices
- Nắm được các nguyên lý cốt lõi của SOA và sự kế thừa trong microservices
- Phân biệt rõ SOA truyền thống, microservices, và modular monolith
- Tìm hiểu vì sao các công ty công nghệ lớn buộc phải chuyển đổi, và bài học rút ra
- Đánh giá khi nào nên — và khi nào *không* nên — áp dụng microservices
- Làm quen với case study của sách: KBM — một hệ thống LMS cho trường đại học

1.1. Kiến trúc phần mềm theo thời gian

Lịch sử phát triển phần mềm doanh nghiệp là câu chuyện về những lần vượt qua giới hạn: mỗi khi một hệ thống đạt đến quy mô mà kiến trúc hiện tại không còn đáp ứng được, cộng đồng lại tìm ra cách tiếp cận mới. Nhìn lại quá trình tiến hóa này giúp chúng ta hiểu *tại sao* microservices tồn tại, thay vì chỉ biết *nó là gì*.

1.1.1. Thời kỳ Monolith

Trong nhiều thập kỷ, hầu hết phần mềm được xây dựng theo kiến trúc **monolithic** (nguyên khối) — toàn bộ logic nghiệp vụ, giao diện người dùng, và truy cập dữ liệu nằm trong một ứng dụng duy nhất, triển khai như một đơn vị.

Kiến trúc monolith có những ưu điểm rõ ràng. Một codebase duy nhất giúp phát triển đơn giản: mở IDE, chạy ứng dụng, gỡ lỗi (debug) với các breakpoint — mọi thứ liền mạch. Việc triển khai (deploy) chỉ cần một artifact và một câu lệnh. Hiệu năng tốt vì gọi hàm cục bộ (in-process) luôn nhanh hơn gọi qua mạng đáng kể. Và quan trọng nhất, khi mọi thao tác dùng chung một database, transaction ACID cục bộ giúp giữ dữ liệu nhất quán đơn giản hơn nhiều — dù ràng buộc nghiệp vụ và mức cô lập vẫn phải được thiết kế đúng.

Tuy nhiên, khi hệ thống phát triển — thêm tính năng, thêm thành viên, thêm khách hàng — monolith bắt đầu bộc lộ những giới hạn nghiêm trọng. Thomas Erl gọi đây là kiến trúc dạng silo (*silo-based application architecture*) — một trạng thái mà mỗi ứng dụng trở thành "ốc đảo" với logic trùng lặp, khó tích hợp.

Monolith và tòa nhà đa năng

Chúng ta có thể hình dung kiến trúc monolith giống như một tòa nhà đa năng duy nhất của trường đại học, nơi chứa cả giảng đường, thư viện, nhà ăn và phòng thí nghiệm. Ở quy mô nhỏ, sinh viên chỉ cần đi vài bước là đến nơi, quản lý tòa nhà cũng dễ dàng. Nhưng khi số lượng sinh viên tăng lên hàng chục nghìn, việc sửa đường ống nước ở nhà ăn có thể phải đóng cửa cả thư viện và giảng đường, gây gián đoạn toàn bộ hoạt động học tập.

Các biểu hiện cụ thể:

Bảng 1.1: *Các biểu hiện khi monolith đạt đến giới hạn*

Biểu hiện	Mô tả	Hệ quả
Nút thắt phối hợp	Mỗi thay đổi nhỏ đòi hỏi phối hợp nhiều team	Tốc độ phát triển giảm dần theo kích thước team
Ràng buộc triển khai	Một tính năng mới đòi hỏi phải triển khai (deploy) lại toàn bộ	Chu kỳ phát hành kéo dài, rủi ro cao
Ràng buộc công nghệ	Toàn ứng dụng dùng chung stack	Không thể thử nghiệm công nghệ mới cho một phần
Hạn chế mở rộng cục bộ	Mở rộng = nhân bản toàn bộ ứng dụng	Chi phí cao, lãng phí tài nguyên
Quá tải độ phức tạp	Codebase quá lớn, ranh giới module mờ nhạt	Lập trình viên mới mất hàng tuần, hàng tháng để hiểu hệ thống

Khi monolith đạt đến trạng thái mà không ai hiểu toàn bộ, mỗi thay đổi đều tiềm ẩn hệ quả ngoài ý muốn (side effect), và chu kỳ phát hành (release) kéo dài từ tuần sang tháng — đó chính là trạng thái mà Chris Richardson gọi là **"monolithic hell"**.

1.1.2. Sự ra đời của SOA

Đầu thập niên 2000, ngành công nghiệp phần mềm nhận ra rằng vấn đề không nằm ở quy mô ứng dụng, mà ở *cách tổ chức* logic bên trong. *Service-Oriented Architecture* (SOA — kiến trúc hướng dịch vụ) ra đời với ý tưởng cốt lõi: **phân rã hệ thống thành các dịch vụ có thể tái sử dụng**, giao tiếp qua giao thức chuẩn.

SOA hướng tới ba mục tiêu lớn: tăng khả năng tái sử dụng logic nghiệp vụ trên toàn doanh nghiệp, giảm tổng lượng code trùng lặp giữa các hệ thống, và cải thiện khả năng tích hợp giữa các nền tảng khác nhau.

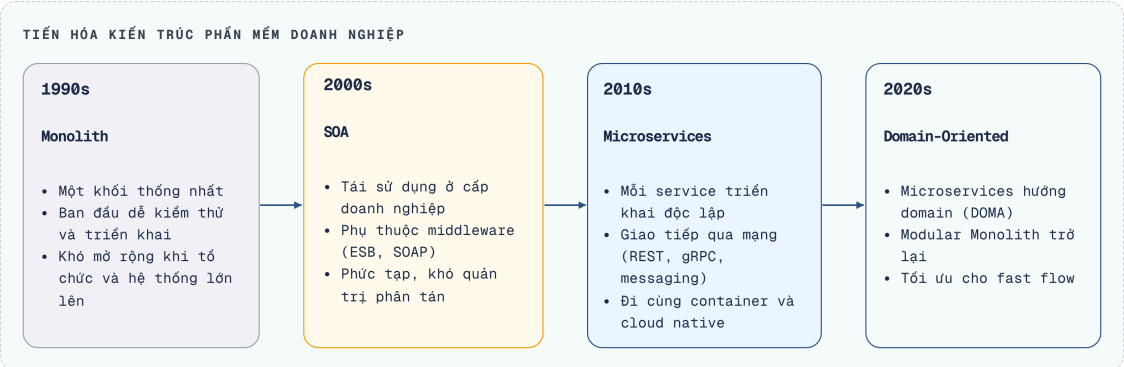
Tuy nhiên, SOA truyền thống thường đi kèm với Enterprise Service Bus (ESB) — một middleware trung tâm điều phối mọi giao tiếp. ESB nhanh chóng trở thành điểm nghẽn: phức tạp, nặng nề, và tạo ra một dạng coupling mới — *coupling vào middleware*. Đây là bài học mà microservices rút ra để thiết kế theo nguyên tắc ngược: "smart endpoints, dumb pipes".

ESB ≠ API Gateway

Đừng nhầm ESB với API Gateway hiện đại. Gateway xử lý các mối quan tâm hạ tầng — định tuyến, xác thực, rate limiting — theo đúng tinh thần "dumb pipes": không chứa business logic; còn ESB ôm cả business logic và orchestration phức tạp vào middleware. Dù vậy, vẫn cần cảnh giác: nếu lập trình viên dồn business logic vào Gateway, nó sẽ dần biến thành một "ESB kiểu mới" (Chương 8).

1.1.3. Từ SOA đến Microservices

Khoảng năm 2014, James Lewis và Martin Fowler đăng bài viết "Microservices" trên martinfowler.com, phổ biến rộng rãi thuật ngữ **microservices** để mô tả một phong cách kiến trúc đã tiến hóa từ SOA nhưng với mục tiêu khác biệt (thuật ngữ này được sử dụng trước đó tại các workshop phần mềm từ ~2011). Microservices không phải SOA phiên bản nhỏ — đây là sự chuyển dịch căn bản từ **tái sử dụng** (reuse) sang **tính độc lập** (independence) như ưu tiên số một.



Hình 1.1: *Tiến hóa kiến trúc phần mềm doanh nghiệp từ thập niên 1990 đến 2020*

1.2. SOA là gì? Nguyên lý cốt lõi

Dù microservices đã trở thành xu hướng chính, việc hiểu SOA vẫn quan trọng vì hai lý do: thứ nhất, nhiều doanh nghiệp lớn vẫn vận hành hệ thống SOA; thứ hai, các nguyên lý hướng dịch vụ của SOA là **nền tảng** mà microservices kế thừa và phát triển.

1.2.1. Bốn đặc trưng cốt lõi

Thomas Erl xác định SOA qua bốn đặc trưng cơ bản:

Business-Driven. Dịch vụ được thiết kế từ quy trình nghiệp vụ, không phải từ bảng cơ sở dữ liệu hay module kỹ thuật. Một dịch vụ "Đăng ký môn học" phản ánh quy trình đào tạo của sinh viên, không phải bảng `registrations` trong database. Nguyên tắc này vẫn giữ nguyên giá trị trong microservices — và trở thành nền tảng cho Domain-Driven Design (DDD) mà chúng ta sẽ khám phá trong Chương 2.

Vendor-Neutral. Dịch vụ giao tiếp qua giao thức chuẩn (HTTP, messaging), không phụ thuộc vào công nghệ cụ thể. Đây là điều kiện tiên quyết cho **technology diversity** — một trong những lợi thế lớn nhất của microservices.

Enterprise-Centric. SOA tư duy ở cấp toàn tổ chức: dịch vụ không phục vụ riêng một ứng dụng mà phục vụ toàn doanh nghiệp (giống như một "Phòng Đào tạo" chung phục vụ mọi khoa trong trường đại học). Microservices thu hẹp phạm vi này vào từng *Bounded Context* — nhưng vẫn giữ tư duy dịch vụ vượt ra ngoài một team duy nhất.

Composition-Centric. Dịch vụ được thiết kế để tổ hợp (compose) thành workflow phức tạp. Trong microservices, điều này thể hiện qua *Saga pattern* (giống như quy trình đăng ký tín chỉ: nếu hệ thống đã giữ chỗ lớp học và xuất hóa đơn, nhưng sinh viên không nộp học phí đúng hạn, hệ thống phải lần lượt gọi ngược lại các dịch vụ trước đó để 'hủy hóa đơn' và 'giải phóng chỗ ngồi') và *API Gateway* — các chủ đề của Chương 6 và 8.

1.2.2. Tám nguyên lý hướng dịch vụ

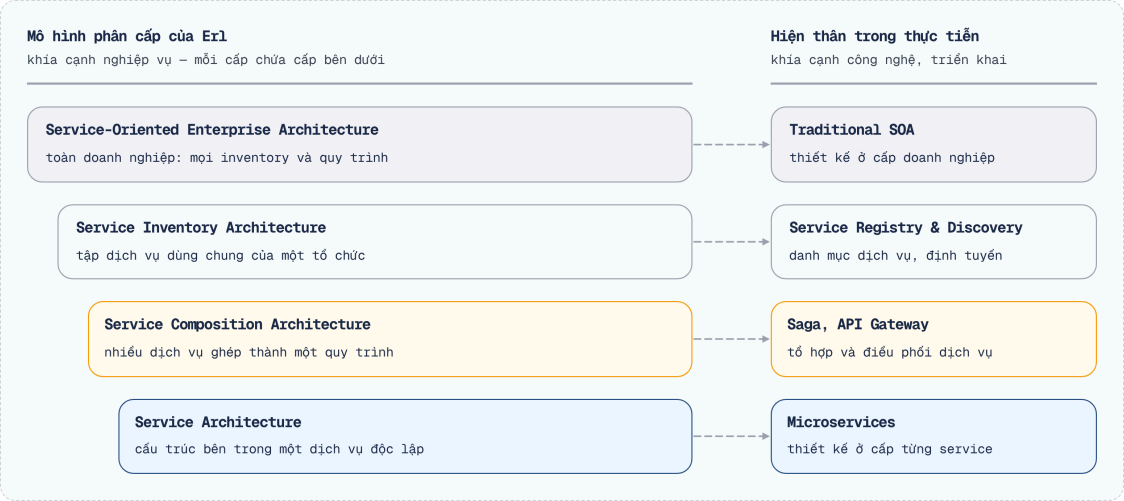
Erl đúc kết 8 nguyên lý nền tảng cho mọi dịch vụ. Bảng 1.2 cho thấy microservices kế thừa và chuyển hóa chúng như thế nào:

Bảng 1.2: *Tám nguyên lý hướng dịch vụ — SOA truyền thống vs Microservices*

#	Nguyên lý	Trong SOA truyền thống	Trong Microservices
1	**Standardized Contract**	WSDL, XML Schema	OpenAPI/Swagger, Protocol Buffers
2	**Loose Coupling**	ESB giảm coupling	**Ưu tiên số 1**: database-per-service, async messaging
3	**Abstraction**	Interface ẩn implementation	API là contract duy nhất, code là black box
4	**Reusability**	**Mục tiêu chính** — tái sử dụng ở cấp enterprise	Giảm ưu tiên — *independence* quan trọng hơn *reuse*
5	**Autonomy**	Hạn chế bởi shared infrastructure	**Mục tiêu chính** — own database, own lifecycle
6	**Statelessness**	Stateless design	RESTful by default, state externalized (Redis, DB)
7	**Discoverability**	UDDI registry	Service Discovery (Eureka, Consul, K8s DNS)
8	**Composability**	ESB orchestration	Saga, API Gateway, event choreography

Điểm mấu chốt: microservices không phủ nhận SOA — mà **chọn lọc và đẩy mạnh** những nguyên lý phù hợp (Loose Coupling, Autonomy) và giảm nhẹ những cơ chế dễ tạo liên kết chặt ở quy mô toàn doanh nghiệp (Reusability ở cấp enterprise, kho đặc tả tập trung kiểu UDDI và điều phối qua ESB).

Bốn kiểu kiến trúc SOA. SOA tồn tại ở nhiều cấp độ, từ nhỏ đến lớn:



__Hình 1.2:__ Bốn kiểu kiến trúc SOA — từ dịch vụ đơn lẻ đến quy mô doanh nghiệp

Hầu hết dự án microservices thực tế hoạt động ở tầng **Service Architecture** và **Service Composition Architecture**. Sự khác biệt cốt lõi: SOA truyền thống cố gắng quản lý một "Service Inventory" thống nhất cho toàn doanh nghiệp — trong khi microservices chấp nhận sự phân tán và trao quyền cho từng team.

1.3. Microservices là gì? So sánh với SOA

Có thể hình dung microservices qua cách một trường đại học tự tổ chức: thay vì một phòng ban duy nhất lo toàn bộ từ tuyển sinh, đào tạo đến cấp bằng, trường chia thành các khoa và phòng ban hoạt động độc lập — mỗi đơn vị tự quyết định cách vận hành và lưu trữ hồ sơ riêng.

1.3.1. Định nghĩa và đặc trưng

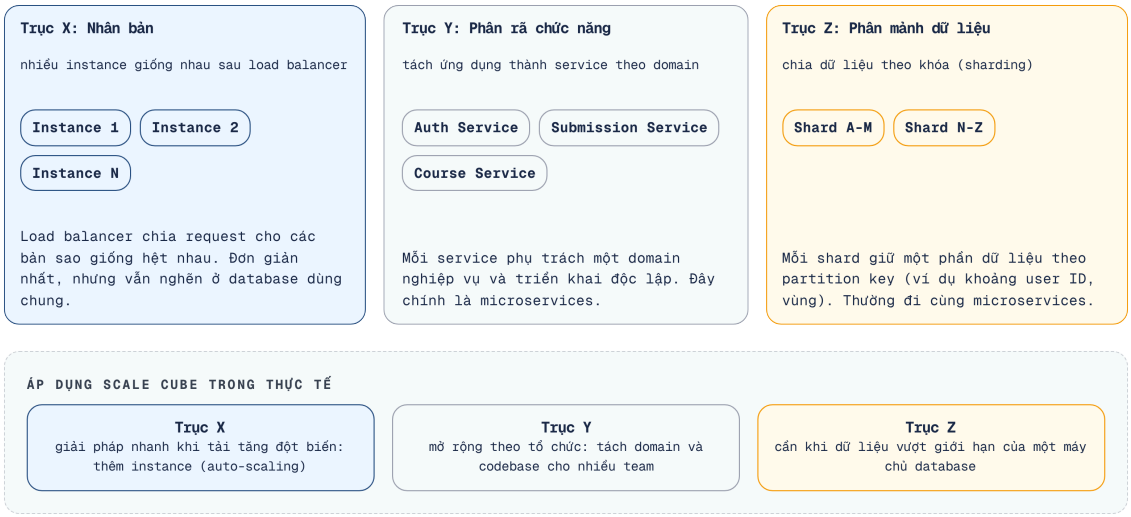
Microservices là phong cách kiến trúc trong đó ứng dụng được cấu thành từ **tập hợp các dịch vụ nhỏ, tự trị**. Mỗi dịch vụ thỏa mãn bốn điều kiện:

1. **Chạy trong tiến trình riêng** — deploy và restart độc lập
2. **Sở hữu dữ liệu riêng** — database-per-service, không chia sẻ bảng (lưu ý: có thể chỉ là các logical schemas riêng biệt trên cùng một DB server vật lý để tối ưu chi phí)
3. **Giao tiếp qua giao thức nhẹ** — HTTP/REST, gRPC, hoặc messaging (Kafka, RabbitMQ)
4. **Có thể triển khai độc lập** — thay đổi service A không yêu cầu triển khai lại service B

Đặc biệt, điều kiện thứ tư (*independent deployability*) chính là thước đo quan trọng nhất. Nếu không thể deploy một service mà không phải deploy các service khác — đó chưa phải microservices thực sự.

1.3.2. Scale Cube — ba chiều mở rộng

Để hiểu microservices trong bối cảnh scalability, chúng ta sử dụng mô hình **Scale Cube** do Martin L. Abbott và Michael T. Fisher giới thiệu trong cuốn "The Art of Scalability", sau đó được Chris Richardson phổ biến trong ngữ cảnh microservices. Mô hình này chia mở rộng hệ thống theo ba chiều:



Hình 1.3: Scale Cube — ba chiều mở rộng hệ thống

- **Trục X — Nhân bản**: chạy nhiều instance giống nhau phía sau load balancer. Đơn giản nhất, giới hạn bởi database bottleneck
- **Trục Y — Phân rã chức năng**: tách ứng dụng thành các service theo domain. **Đây là microservices**
- **Trục Z — Phân mảnh dữ liệu**: chia dữ liệu theo tiêu chí (region, user ID). Thường kết hợp với microservices

Scaling truyền thống dừng ở hai lựa chọn: nâng cấp phần cứng cho một máy (tăng RAM/CPU — *vertical scaling*, nằm ngoài Scale Cube) và nhân bản instance phía sau load balancer (trục X). Khi cả hai đạt giới hạn — database trở thành bottleneck, deployment trở nên rủi ro — đó là lúc cần xem xét trục Y.

1.3.3. So sánh SOA truyền thống vs Microservices

Bảng 1.3 so sánh hai cách tiếp cận theo tám khía cạnh, từ phạm vi và kích thước dịch vụ đến mô hình tổ chức đội.

Bảng 1.3: So sánh SOA truyền thống và Microservices

Khía cạnh	SOA truyền thống	Microservices
Phạm vi	Toàn doanh nghiệp	Một ứng dụng hoặc Bounded Context
Kích thước dịch vụ	Lớn, tái sử dụng tối đa	Nhỏ, focused, single responsibility
Giao tiếp	ESB — middleware trung tâm	"Smart endpoints, dumb pipes" — REST/messaging
Dữ liệu	Thường shared database	Database-per-service
Governance	Centralized, formal	Decentralized, team-owned
Trọng tâm thiết kế	Reuse-first	Independence-first

Khía cạnh	SOA truyền thống	Microservices
Deploy	Thường cùng deploy	Independent deployment
Team model	Chuyên môn theo layer (DB team, UI team)	Cross-functional, own the service

Cần lưu ý rằng SOA, microservices, và event-driven architecture (EDA) không phải các lựa chọn loại trừ nhau. Chúng là các lớp của một cách tiếp cận đang tiến hóa: SOA cung cấp nguyên lý nền tảng, microservices hiện thực hóa chúng ở mức độ phân rã chi tiết hơn (fine-grained), và EDA bổ sung khả năng giao tiếp bất đồng bộ cho các hệ thống phân tán phức tạp. Chúng ta sẽ khám phá EDA chi tiết trong Chương 5.

1.4. Microservices trong thực tế — bài học từ các công ty công nghệ

Lý thuyết về microservices chỉ thực sự trở nên thuyết phục khi nhìn vào *tại sao* các tổ chức lớn buộc phải chuyển đổi. Không phải vì microservices "thời thượng" — mà vì họ đối mặt với **những giới hạn mà kiến trúc truyền thống và cách mở rộng theo trục X không thể vượt qua**.

1.4.1. Netflix — khi downtime là không thể chấp nhận

Năm 2008, Netflix gặp sự cố database corruption nghiêm trọng, gây gián đoạn dịch vụ. Sự cố này là chất xúc tác cho quyết định chuyển toàn bộ hạ tầng lên cloud (AWS) và tái cấu trúc sang microservices. Theo Netflix Technology Blog [W3], Netflix hoàn thành chuyển đổi hạ tầng sang AWS vào đầu 2016 — đây là cột mốc của quá trình dịch chuyển (migration) lên đám mây, nhưng kiến trúc microservices tiếp tục tiến hóa qua nhiều năm sau đó. Tại thời điểm 2016, hệ thống vận hành hàng trăm microservices phục vụ gần 100 triệu người dùng (Netflix vượt mốc 100 triệu thuê bao vào giữa năm 2017).

Vấn đề cốt lõi là đòi hỏi khắt khe về tính sẵn sàng (uptime) và khả năng mở rộng (scale). Với lượng người dùng tăng theo cấp số nhân, một monolith duy nhất trở thành single point of failure. Vertical scaling (nâng cấp server) đạt giới hạn vật lý và chi phí. Netflix cần hai thứ mà monolith không thể cung cấp: khả năng mở rộng từng thành phần riêng biệt, và **fault isolation** — lỗi ở hệ thống recommendation không được phép ảnh hưởng đến streaming.

Fault Isolation — cách ly sự cố trong mùa thi

Tương tự như việc tổ chức thi học kỳ tập trung: Nếu tất cả sinh viên thi cùng một phòng lớn (monolith), một sự cố như mất điện hay hỏng máy chiếu sẽ ảnh hưởng tới toàn bộ thí sinh. Với microservices, sinh viên được chia vào nhiều phòng thi nhỏ biệt lập (fault isolation). Phòng thi số 1 có thể bị hỏng điều hòa, nhưng thí sinh ở các phòng khác vẫn làm bài bình thường. Hệ thống gợi ý phim (recommendation) của Netflix gặp sự cố thì người dùng vẫn xem phim (streaming) bình thường.

Netflix đóng góp ngược lại cho cộng đồng bằng các công cụ mã nguồn mở: **Eureka** (Service Discovery), **Zuul** (API Gateway), **Hystrix** (Circuit Breaker) — nhiều trong số đó trở thành nền tảng của Spring Cloud. Zuul 1 và Hystrix nay đã ở chế độ bảo trì, được thay bằng Spring Cloud Gateway (KBM đang dùng) và Resilience4j (nằm trong lộ trình của KBM, Chương 4 và 10); Eureka vẫn được dùng.

Sự ra đời của Docker vào năm 2013 cũng là một "chất xúc tác" (enabler) quan trọng giúp kiến trúc microservices phổ biến rộng rãi, vì nó chuẩn hóa cách đóng gói và vận hành hàng chục tiến trình độc lập.

1.4.2. Amazon — API Mandate và "two-pizza teams"

Khoảng 2001–2002, Amazon đối mặt với tình trạng monolith liên kết quá chặt chẽ (tightly coupled) — tầng giao diện (frontend) và tầng xử lý (backend) phụ thuộc vào nhau đến mức *bất kỳ thay đổi nào, dù nhỏ, đều ảnh hưởng toàn hệ thống*. Theo câu chuyện nổi tiếng được Steve Yegge kể lại trong "Google Platforms Rant" (2011) [W5], Jeff Bezos được cho là đã ra **"API mandate"** — một chỉ thị nội bộ: tất cả team phải giao tiếp với nhau *chỉ* qua service interfaces, không có ngoại lệ. Cùng giai đoạn, Amazon tổ chức thành các nhóm đủ nhỏ để hai chiếc pizza là đủ cho cả nhóm ăn (two-pizza teams), mỗi nhóm sở hữu trọn dịch vụ của mình.

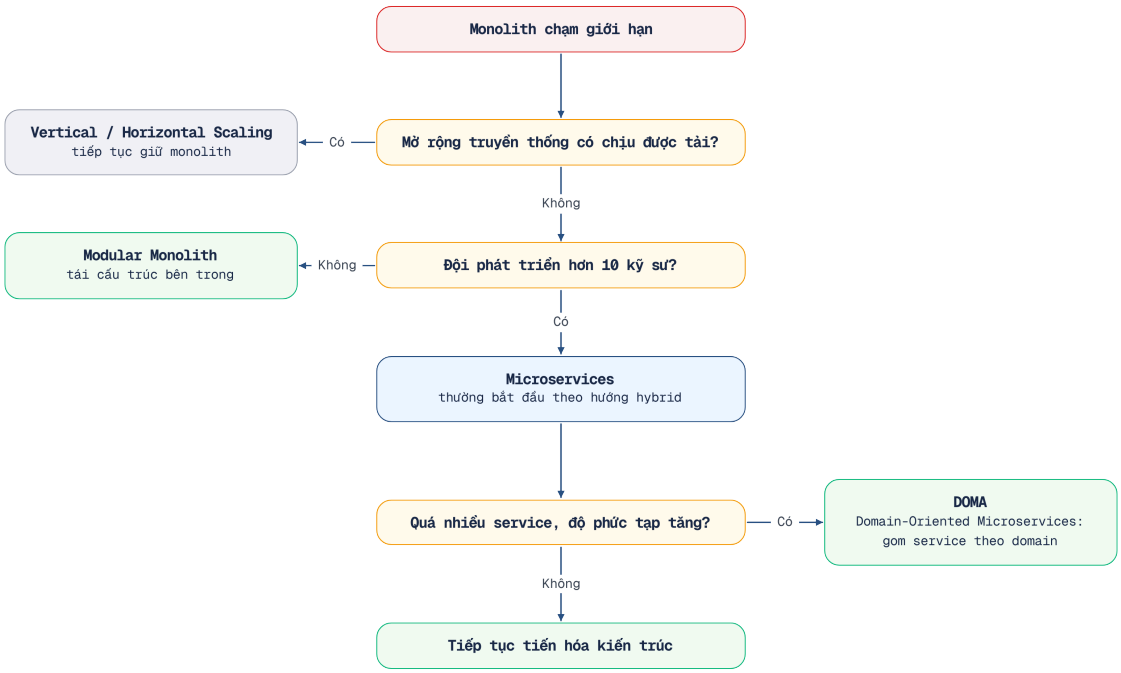
1.4.3. Uber — tổ chức lại microservices theo domain

Uber bắt đầu như một monolith Python đơn giản, tăng trưởng với tốc độ bùng nổ, và khoảng 2014–2016 chuyển sang microservices. Tuy nhiên, câu chuyện Uber đặc biệt đáng chú ý ở phần *hậu chuyển đổi*: đến khoảng 2020, với hơn 2.000 microservices, họ nhận ra hệ thống đang rơi vào *"microservice madness"* — quá nhiều service nhỏ tạo ra mạng lưới phụ thuộc phức tạp, khó debug và vận hành.

Uber phản ứng bằng cách giới thiệu **DOMA** (*Domain-Oriented Microservice Architecture*) — nhóm các microservices liên quan thành "domain" lớn hơn, giảm bề mặt tương tác. Uber không quay về monolith mà tổ chức lại: các microservice được gom theo domain, có quy tắc phụ thuộc giữa các tầng và gateway ở biên mỗi domain.

1.4.4. Bài học tổng hợp

Hình 1.4 tóm tắt ba câu chuyện trên thành một cây quyết định chuyển đổi kiến trúc.



Hình 1.4: *Cây quyết định chuyển đổi kiến trúc — từ monolith đến microservices*

Ba pattern chung nổi lên:

1. **Chuyển đổi vì bị buộc, không vì mong muốn.** Mỗi công ty trên đều chuyển sang microservices khi cách mở rộng truyền thống thất bại trước yêu cầu nghiêm ngặt — uptime 99.99%, hàng triệu request/giây, hàng trăm lập trình viên cần triển khai độc lập.
2. **Cấu trúc nhóm thay đổi trước (hoặc cùng) kiến trúc.** Amazon chia thành các nhóm nhỏ (two-pizza teams) trước khi tách service. Netflix tạo ra các "full-cycle developers" — mỗi lập trình viên chịu trách nhiệm từ khâu viết mã đến khi vận hành trên production. Kiến trúc phần mềm phản ánh cấu trúc tổ chức (Conway's Law). Ngành công nghiệp thậm chí còn áp dụng "Inverse Conway Maneuver" (Nghịch đảo định luật Conway) — chủ động thay đổi cấu trúc team để ép kiến trúc phần mềm phải thay đổi theo. Chúng ta sẽ phân tích sâu trong Chương 2.
3. **Kiến trúc còn tiếp tục tiến hóa sau chuyển đổi.** Microservices không phải đích cuối: Uber phải tổ chức lại hơn 2.000 service thành domain (DOMA), và nhiều tổ chức vẫn giữ monolith hoặc modular monolith cho những phần chưa cần tách — kết hợp như vậy là hợp lý, không phải thất bại.

Fast Flow

Động lực cốt lõi của kiến trúc microservices là giúp tổ chức đạt được *fast flow* — phân phối liên tục dòng thay đổi nhỏ lên môi trường vận hành với vòng phản hồi nhanh cho từng thay đổi.

— Theo Chris Richardson, Microservices Patterns, 2nd Ed. [2b, Ch.1]

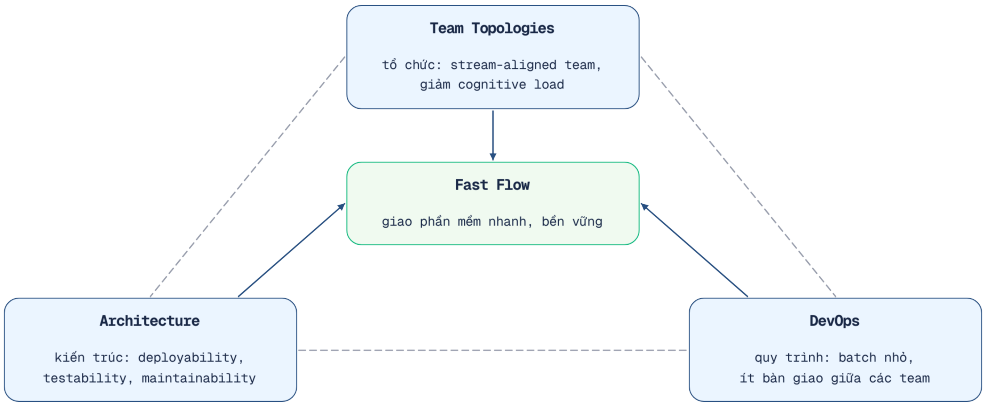
1.5. Đo lường và đánh giá hiệu năng kiến trúc

1.5.1. Fast Flow Success Triangle — DevOps + Team Topologies + Architecture

Phần nâng cao (có thể lướt qua)

Ba mục tiếp theo (*Fast Flow Success Triangle*, *DORA Metrics*, *Quality Attribute Scenarios*) là góc nhìn **đo lường và đánh giá** kiến trúc — hữu ích nhưng không bắt buộc để hiểu microservices là gì. Nếu đây là lần đọc đầu, người đọc có thể đọc lướt và quay lại sau: DORA Metrics sẽ gặp lại trong ngữ cảnh CI/CD ở **Chương 12**, còn Quality Attribute Scenarios sẽ phát huy tác dụng khi cân nhắc tách/gộp service ở **Chương 2**. Một số thuật ngữ xuất hiện sớm trong chương này (CAP, BASE, eventual consistency, Saga...) sẽ được giải thích kỹ ở các chương sau; có thể tra nhanh ở **Phụ lục A — Bảng thuật ngữ** bất cứ lúc nào.

Giống như việc nhà trường cần phối hợp nhịp nhàng giữa Phòng Đào tạo (quy trình), đội ngũ giảng viên (tổ chức) và cơ sở vật chất (kiến trúc) để một học kỳ diễn ra suôn sẻ, việc phát triển phần mềm cũng đòi hỏi sự đồng bộ tương tự. Richardson trong phiên bản thứ hai (2025) giới thiệu một framework quan trọng: **Fast Flow Success Triangle** — mô tả ba yếu tố cần thiết để đạt được fast flow:



Hình 1.5: *Fast Flow Success Triangle — ba yếu tố cần thiết cho fast flow*

DevOps (Quy trình) — Không phải tên team hay job title, mà là tập hợp nguyên tắc và thực hành để phân phối phần mềm nhanh và tin cậy. Hai nguyên tắc quan trọng nhất: (1) *chia nhỏ khối lượng mỗi lần cập nhật (batch size)* — commit và push các thay đổi nhỏ, lý tưởng ít nhất 1 lần/ngày; (2) *giảm các khâu bàn giao chéo giữa các team* — mỗi bước bàn giao đều gây ra độ trễ và rơi rớt thông tin.

Team Topologies (Tổ chức) — Cấu trúc tổ chức theo bốn kiểu team (stream-aligned, platform, enabling, complicated subsystem) với ba kiểu tương tác (X-aaS, collaboration, facilitating). Stream-aligned teams làm phần lớn công việc, các team khác hỗ trợ.

Architecture (Kiến trúc) — Dưới góc độ fast flow, vai trò của kiến trúc là **tạo tiền đề cho** DevOps và Team Topologies. Richardson trong [2b, Ch.5] phân tích các **architecture qualities** (thuộc tính kiến trúc) phục vụ fast flow; với bối cảnh KBM, chương này tập trung vào ba thuộc tính cốt lõi nhất:

- **Deployability (Khả năng triển khai)**: Mỗi service có thể được triển khai độc lập với pipeline riêng, không cần phối hợp với các service khác. Ví dụ tại KBM: Judge Service có thể được triển khai mà Core Service không cần biết.
- **Testability (Khả năng kiểm thử)**: Có thể kiểm thử một service hoàn toàn cô lập, không cần khởi chạy toàn bộ hệ thống. Ví dụ: kiểm thử logic của Judge mà không cần chạy Gateway, Auth, hay Kafka.
- **Maintainability (Khả năng bảo trì)**: Các nhóm có thể thay đổi mã nguồn thuận tiện, ít xung đột, với quyền sở hữu (ownership) rõ ràng. Giả sử khi mở rộng thành nhiều nhóm: Nhóm A sở hữu Judge, Nhóm B sở hữu Academic, cả hai có thể phát triển song song mà không chặn lẫn nhau.

Ba thuộc tính này là *điều kiện cần* cho fast flow — thiếu bất kỳ thuộc tính nào sẽ tạo điểm nghẽn (bottleneck). KBM hiện chưa đạt trọn vẹn cả ba: shared library thường kéo theo việc biên dịch lại nhiều service (deployability), shared database gây phụ thuộc khi kiểm thử tích hợp (testability), và các "god class" (lớp ôm đồm quá nhiều chức năng) hơn 1.000 dòng làm tăng nguy cơ xung đột mã nguồn và khó bảo trì (maintainability).

Tam giác chỉ đứng vững khi đủ cả ba chân. DevOps tốt và team tốt nhưng kiến trúc monolith coupling chặt thì fast flow vẫn khó đạt được. Ngược lại, microservices architecture tốt nhưng team vẫn tổ chức theo layer (DB team, UI team) thì lợi ích sẽ bị triệt tiêu bởi coordination overhead.

1.5.2. Đo lường Fast Flow — DORA Metrics

Làm sao biết chúng ta đã đạt fast flow? Nghiên cứu của Forsgren, Humble và Kim trong *Accelerate* (2018) định nghĩa bốn chỉ số DORA (DevOps Research and Assessment):

Bảng 1.4: *DORA Metrics — so sánh Elite vs Low Performers*

Metric	Elite Performers	Low Performers	Chênh lệch
Deployment Frequency	Theo nhu cầu (nhiều lần/ngày)	1 lần/1–6 tháng	208×
Lead Time (commit → production)	< 1 ngày	1–6 tháng	106×
Change Failure Rate	0–15%	46–60%	7×
Recovery Time (MTTR)	< 1 giờ	1 tuần–1 tháng	2.604×

Nguồn: DORA State of DevOps Report 2019 (bộ chỉ số định nghĩa trong Accelerate; con số cụ thể thay đổi qua từng năm)

Điều quan trọng hơn cả: **triển khai thường xuyên hơn lại *tin cậy* hơn**. Elite performers deploy với tần suất cao hơn 208 lần nhưng tỷ lệ lỗi thấp hơn 7 lần. Các thay đổi nhỏ sẽ dễ kiểm thử, debug và rollback hơn — nguyên lý "move fast and *don't* break things."

DORA Metrics cho LMS

KBM hiện tại:

- **Deployment Frequency**: Thủ công, khi cần; chưa đo tần suất nên chưa xếp được nhóm DORA
- **Lead Time**: Chưa đo, nhưng thiếu CI/CD tự động → ước tính ngày–tuần
- **Change Failure Rate**: Chưa theo dõi → chưa có số liệu
- **Recovery Time**: Khởi động lại thủ công → ước tính phút–giờ (mức chấp nhận được trong môi trường học thuật)

Lộ trình chuyển đổi (Migration path): Triển khai CI/CD pipeline (Chương 12) là bước đầu tiên để cải thiện DORA Metrics. Mục tiêu: deploy hàng tuần và lead time < 1 tháng — tương ứng nhóm Medium theo phân cụm DORA 2019; chạm mốc lead time < 1 tuần là đã sang ngưỡng High.

1.5.3. Quality Attribute Scenarios — chỉ định yêu cầu kiến trúc

DORA Metrics đo lường kết quả cuối cùng, nhưng làm sao chỉ định kiến trúc phải hỗ trợ kết quả đó ngay từ khâu thiết kế? Richardson trong [2b, Ch.5] khuyên dùng **Quality Attribute Scenarios** — kỹ thuật do Software Engineering Institute (SEI) phát triển, cho phép viết "spec" cho các thuộc tính phi chức năng một cách đo lường được.

Thay vì nói chung chung "hệ thống phải dễ deploy", một kịch bản chất lượng định nghĩa rõ ràng theo khuôn: **Source → Stimulus → Artifact → Environment → Response → Response Measure**

Ví dụ — kịch bản deployability cho LMS:

- **Source**: Lập trình viên (stream-aligned team)
- **Stimulus**: Commit code thay đổi logic chấm điểm SQL
- **Artifact**: Judge Service
- **Environment**: Deployment pipeline
- **Response**: Pipeline tự động chạy CI, build, test, và triển khai phiên bản mới lên production — mà không cần sự can thiệp của nhóm khác
- **Response Measure**: Lead time < 40 phút

Ví dụ — kịch bản scalability cho LMS:

- **Source**: 200 sinh viên thi cùng lúc (chế độ thi)
- **Stimulus**: Lượng bài nộp tăng vọt lên hàng trăm lượt mỗi phút trong những phút cuối giờ (sinh viên nộp lại nhiều lần)
- **Artifact**: Judge Service cluster
- **Environment**: Vận hành bình thường trong ca thi, cao điểm ở những phút cuối giờ
- **Response**: Tất cả bài nộp được xử lý thành công, không yêu cầu nào bị từ chối
- **Response Measure**: P99 thời gian *tiếp nhận* bài nộp (enqueue) < 500 ms; P99 thời gian *trả kết quả chấm* < 45 giây (bản thân việc chấm mất 5–30 giây, xem §4.1); 0% message loss

Bằng cách viết ra các kịch bản này, chúng ta có một dạng "unit test cho kiến trúc" — bất kỳ quyết định tách hay gộp service nào đều phải đối chiếu lại: *"Quyết định này có giúp chúng ta đạt kịch bản đó hay không?"*

1.6. Modular Monolith — lựa chọn hợp lệ

Trước khi nghĩ đến microservices, chúng ta cần dành thời gian nghiêm túc cho một lựa chọn thường bị bỏ qua: **Modular Monolith**. Đây có thể là bước đi hợp lý nhất, tùy ngữ cảnh.

1.6.1. Monolith không phải là xấu

Không phải mọi monolith đều giống nhau. Theo mức độ tổ chức nội bộ, có thể phân biệt hai loại:

- **Patchwork Monolith (Monolith chắp vá)**: Đặc trưng bởi mã nguồn lộn xộn, thiếu cấu trúc, mọi class phụ thuộc lẫn nhau, ranh giới mờ nhạt, thường được gọi là "big ball of mud". Hệ thống này cần được refactor nghiêm túc để xác định lại cả ranh giới bên trong lẫn bên ngoài.
- **Modular Monolith (Monolith theo module)**: Hệ thống có các module phân chia rõ ràng, giao tiếp với nhau qua các interface được định nghĩa sẵn. Mỗi module thậm chí có thể có schema riêng trong cùng một database. Mặc dù phải deploy cùng nhau, các module có thể được phát triển tương đối độc lập và hoàn toàn **có thể tách dần** ra thành các service riêng biệt khi quy mô đòi hỏi.

Monolith ≠ Bad

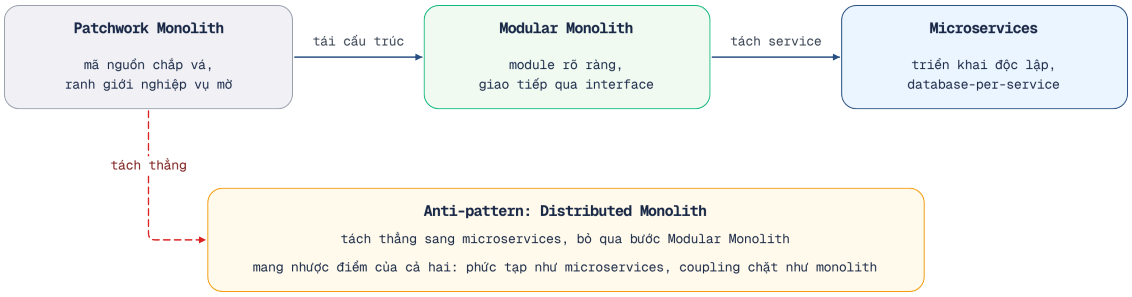
Monolith phản ánh kiến trúc đóng gói, không phản ánh chất lượng. Một modular monolith được thiết kế tốt hoàn toàn là lựa chọn kiến trúc hợp lệ — đặc biệt khi chi phí vận hành microservices vượt quá lợi ích mà nó mang lại.

1.6.2. Khi nào Modular Monolith phù hợp?

- **Team nhỏ**: dưới khoảng 5 người, overhead vận hành microservices (CI/CD per service, monitoring, distributed debugging) thường áp đảo lợi ích; với 5–10 người, modular monolith thường vẫn là lựa chọn hợp lý
- **Domain chưa rõ ràng**: Chia service quá sớm khi chưa hiểu domain rõ ràng dẫn đến chia sai. Gộp lại các service rất tốn kém và rủi ro so với refactor module trong monolith
- **Giai đoạn startup/MVP**: Cần tốc độ phát triển, đưa sản phẩm ra thị trường nhanh
- **Không cần polyglot**: Nếu toàn team dùng cùng ngôn ngữ/framework, lợi ích technology diversity không tồn tại

1.6.3. Con đường từ Monolith đến Microservices

Hình 1.6 mô tả con đường từ một patchwork monolith đến microservices, qua bước modular hóa.



Hình 1.6: Con đường từ Patchwork Monolith đến Microservices

Martin Fowler đề xuất chiến lược "**Monolith First**": bắt đầu với monolith, hiểu rõ domain qua vài iteration, rồi mới tách microservices nếu thực sự cần. Lịch sử cho thấy hầu hết hệ thống microservices thành công — Netflix, Amazon, Uber — đều **bắt đầu từ monolith**, không phải microservices ngay từ đầu.

Điều nguy hiểm nhất là cố tách một patchwork monolith trực tiếp thành microservices mà không trải qua bước modular hóa. Kết quả thường là **distributed monolith** — mang toàn bộ nhược điểm của cả monolith lẫn distributed system, không có ưu điểm nào.

Trong hệ sinh thái Spring, **Spring Modulith** (2023+) là extension chính thức của Spring Boot hỗ trợ kiến trúc *Modular Monolith*. Nó cung cấp: (1) kiểm chứng ranh giới module bằng structural verification (`ApplicationModules.verify()` — thường chạy trong test/CI), (2) integration testing từng module độc lập, và (3) event publication registry cho giao tiếp giữa các module — bước đệm tự nhiên trước khi tách microservice, đặc biệt phù hợp với chiến lược "Monolith First".

1.7. Các kịch bản áp dụng và hạn chế của Microservices

Việc chuyển đổi sang kiến trúc microservices giống như quyết định xây thêm một cơ sở đào tạo mới. Đừng mở thêm cơ sở nếu lượng sinh viên hiện tại vẫn vừa vặn trong một tòa nhà, bởi vì chi phí vận hành, bảo vệ và quản lý nhiều cơ sở cùng lúc sẽ cao hơn rất nhiều.

1.7.1. Quyết định kiến trúc không chỉ là kỹ thuật

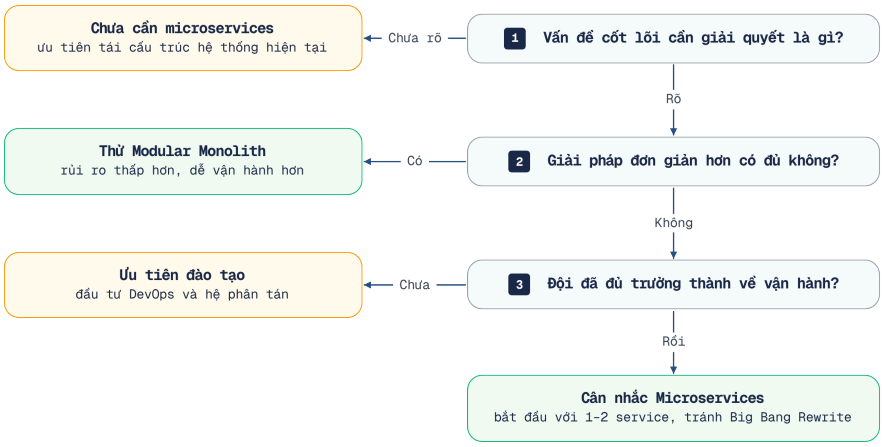
Điều quan trọng nhất cần nhấn mạnh trước khi đi vào chi tiết: **quyết định sử dụng microservices không phải thuần túy kỹ thuật**. Trong thực tế, các yếu tố phi kỹ thuật thường có ảnh hưởng lớn hơn cả yếu tố kỹ thuật:

- **Ngân sách**: Microservices đắt hơn để vận hành (infrastructure, monitoring, DevOps team)
- **Lịch trình**: Xây dựng microservices từ đầu chậm hơn monolith cho cùng feature set
- **Quy mô team**: team nhỏ thường chịu overhead quá lớn (ngưỡng ở mục Modular Monolith, §1.6)
- **Kỹ năng hiện có**: Distributed systems đòi hỏi kinh nghiệm khác biệt
- **Văn hóa tổ chức**: Các team làm việc biệt lập khó vận hành được microservices

Ngay cả Netflix (từ 2008) và Uber (từ 2014) cũng đi qua trạng thái hybrid trong lúc chuyển đổi — lựa chọn phù hợp nhất với bối cảnh lúc đó, không phải microservices thuần túy.

Trong cuốn sách này, chúng ta tập trung phân tích **khía cạnh kỹ thuật** — nhưng luôn ghi nhớ rằng đây chỉ là một chiều trong quyết định kiến trúc.

Khung ra quyết định. Ba câu hỏi cần trả lời trước khi chọn microservices:



Hình 1.7: *Khung ra quyết định (Decision Framework) — ba câu hỏi trước khi chọn microservices*

1.7.2. Phân tích lợi ích và chi phí

Nếu mô hình ra quyết định ở trên nghiêng về kiến trúc phân tán, bước tiếp theo là đặt lợi ích cạnh chi phí:

Bảng 1.5: *Lợi ích của kiến trúc microservices*

Lợi ích	Giải thích
Independent deployability	Deploy một service mà không ảnh hưởng các service khác — giảm rủi ro, tăng tần suất release
Team autonomy	Mỗi team làm chủ toàn bộ vòng đời dịch vụ — từ viết mã đến vận hành trên production
Technology diversity	Chọn công nghệ phù hợp nhất cho từng bài toán cụ thể
Selective scalability	Mở rộng riêng những service cần tài nguyên — tiết kiệm chi phí
Fault isolation	Lỗi ở một service *có thể* được cô lập, không kéo sập toàn hệ thống — với điều kiện có timeout, circuit breaker (Chương 4)

Đổi lại, những lợi ích trên không miễn phí:

Bảng 1.6: *Chi phí và thách thức của microservices*

Chi phí	Giải thích
Complexity tax	Hệ phân tán vốn phức tạp do độ trễ mạng, lỗi cục bộ và eventual consistency — minh chứng cho "8 Fallacies of Distributed Computing" (ví dụ: lầm tưởng rằng mạng luôn đáng tin, độ trễ bằng không)
Operational overhead	CI/CD pipeline, monitoring, logging, tracing, service discovery *cho mỗi service*
Data consistency	Transaction ACID của một database không bao phủ được nhiều service; giao dịch xuyên service thường dùng local transaction + Saga pattern, chấp nhận nhất quán kiểu BASE
Debugging difficulty	Một request đi qua 5 service → cần distributed tracing (Zipkin, Jaeger)
Integration testing	Test toàn hệ thống trở nên khó hơn nhiều so với monolith

Một số thuật ngữ trong bảng trên sẽ được phân tích sâu ở các chương sau. Hiểu ngắn gọn như sau:

- **Eventual consistency** — dữ liệu giữa các service cuối cùng sẽ đồng bộ, nhưng có một khoảng trễ không cam kết trước (Chương 6).
- **ACID → BASE** — thay vì transaction "tất cả hoặc không" trên một database, hệ phân tán chấp nhận nhất quán "mềm" hơn để đổi lấy tính sẵn sàng.

- **CAP Theorem** — khi xảy ra network partition, hệ buộc phải chọn giữa nhất quán (Consistency) và sẵn sàng (Availability) (Chương 7).
- **Saga** — chuỗi giao dịch cục bộ có bước bù trừ, thay cho transaction phân tán (Chương 6).

⚠ Distributed Monolith

Nếu chia hệ thống thành nhiều "service" nhưng chúng vẫn phải deploy cùng nhau, chia sẻ database, và thay đổi đồng loạt — kết quả là một **distributed monolith**: mang toàn bộ nhược điểm của monolith *và* hệ phân tán, không có ưu điểm nào. Đây là kết quả phổ biến nhất khi tách service mà chưa modular hóa.

1.8. Case study: KBM

Quay lại với KBM. Hệ thống không bắt đầu như một kiến trúc phân tán: nó lớn lên từ vài ứng dụng nguyên khối chạy cạnh nhau, và chính chi phí bảo trì các khối đó — chứ không phải xu hướng — là lý do tách service (xem mục "Điểm khởi đầu" dưới đây).

1.8.1. Bài toán nghiệp vụ

Xuyên suốt cuốn sách này, chúng ta sử dụng một case study thực tế: **KBM** — nền tảng học tập trực tuyến (LMS) chuyên biệt cho các môn tin học thực hành, được phát triển và vận hành tại một trường đại học kỹ thuật.

Vấn đề cần giải quyết. Tại trường đại học, các môn học liên quan đến Cơ sở dữ liệu, Lập trình mạng và Quản trị hệ thống/DevOps là bắt buộc cho hàng nghìn sinh viên mỗi năm. Giảng viên không thể thực thi và kiểm tra hàng trăm bài SQL, bài lập trình mạng hoặc bài thực hành Kubernetes mỗi tuần bằng tay. Sinh viên cần phản hồi nhanh để sửa lỗi ngay; giảng viên cần môi trường thi cử công bằng; hệ thống cần cô lập dữ liệu và môi trường thực hành để bài làm của sinh viên này không ảnh hưởng sinh viên khác.

Ba nhóm người dùng chính:

- **Sinh viên**: Luyện tập SQL, tham gia thi, nộp bài tập, làm đồ án. Quy mô: Hàng nghìn / học kỳ.
- **Giảng viên**: Soạn đề, tổ chức thi/kiểm tra, chấm điểm, theo dõi tiến độ. Quy mô: Hàng chục.
- **Quản trị viên**: Quản lý tài khoản, cấu hình hệ thống, giám sát vận hành. Quy mô: Vài người.

Bốn nhóm chức năng nghiệp vụ — mỗi nhóm đặt ra yêu cầu kiến trúc khác nhau:

- **SQL Lab**: Sinh viên viết SQL trên trình soạn thảo web, nộp bài và nhận kết quả trong chế độ luyện tập (Practice Mode) hoặc chế độ thi (Contest Mode). Chế độ luyện tập ưu tiên độ trễ thấp; chế độ thi ưu tiên throughput, tính công bằng, message queue và WebSocket thời gian thực.
- **Network Lab**: Sinh viên viết Java client kết nối đến judge server qua nhiều nhóm giao thức: TCP, UDP, RMI, SOAP, REST và gRPC. Đây là nơi KBM minh họa rất rõ technology diversity: không phải mọi service đều là Spring Boot REST API.
- **Course Management**: Giảng viên tạo khóa học, bài tập cá nhân/nhóm, MCQ, điểm danh, grade scheme, đồ án tốt nghiệp và tích hợp GitHub Classroom. Yêu cầu chính: data integrity, workflow nhiều bước, cross-service query.
- **DevOps Lab**: Sinh viên thực hành Docker/Kubernetes trong lab environment cô lập, truy cập qua terminal web, được chấm bằng validator script. Phần này dùng Go, k3s và Sysbox ở mức hạ tầng khái quát — đủ để phân tích kiến trúc, không cần công khai chi tiết vận hành cụ thể.

Ràng buộc thực tế — đây là điều làm cho case study có giá trị giảng dạy:

- **Team nhỏ**: 1–2 lập trình viên chính + sinh viên hỗ trợ theo đợt, không có đội ngũ DevOps hay kiểm thử (QA) riêng
- **Hạ tầng hạn chế**: hạ tầng nhỏ, LMS chính chạy Docker Compose; DevOps Lab dùng k3s/Sysbox cho bài toán lab isolation, không công khai sơ đồ máy chủ/domain cụ thể
- **Phát triển nhanh dưới áp lực**: Hệ thống phải phục vụ sinh viên ngay, nhiều quyết định kiến trúc ưu tiên *đưa vào dùng sớm*, chấp nhận trả giá về sau
- **Hệ thống đang chạy trên production**: Mọi cải tiến phải được thực hiện tăng dần — không thể tạm dừng hoạt động để tái cấu trúc (refactor) toàn bộ

Điểm quan trọng: KBM hiện tại **không phải là hình mẫu chuẩn mực** — và đó chính là lý do nó có giá trị làm case study. KBM tách service dù đội rất nhỏ; lý do (mục đích giáo dục, nhu cầu cô lập bộ chấm) và cái giá được phân tích ở §10.1. Giống như hầu hết dự án thực tế, đội ngũ phát triển hiếm khi xây dựng được hệ thống hoàn chỉnh ngay từ đầu. Cuốn sách sẽ sử dụng KBM như **bài toán chuyển đổi thực tế**: ở mỗi chương, chúng ta phân tích hệ thống đang ở đâu, vấn đề kỹ thuật là gì, giải pháp khuyến nghị là gì, và đề xuất lộ trình cải thiện.

1.8.2. Điểm khởi đầu: những monolith rời rạc, không phải microservices

Một chi tiết dễ bị bỏ qua nhưng quyết định cả tinh thần của case study: KBM hôm nay là một hệ **polyglot microservices** (Java + Go), nhưng nó *không khởi đầu* như vậy. "Before" của KBM không phải là một hệ thống duy nhất, mà là *vài* monolith rời rạc chạy cạnh nhau:

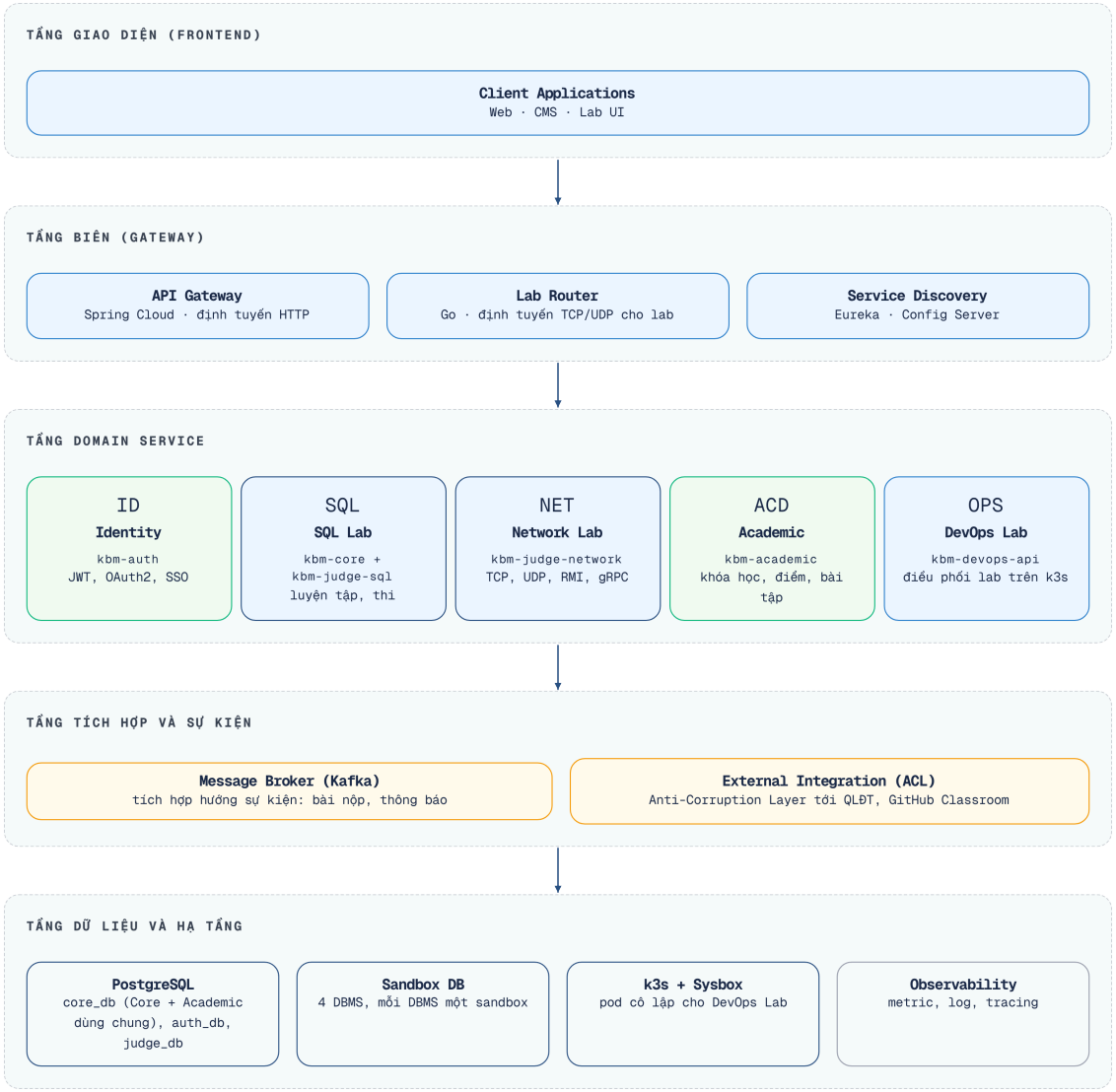
- **Quản lý đào tạo**: một ứng dụng **ASP.NET MVC** nguyên khối (`kbm-legacy-monolith`) quản lý khóa học, bài tập, điểm số, người dùng trên một database dùng chung.
- **Bộ chấm code (phiên bản đầu tiên)**: một monolith MVC **tách biệt**, do nhóm tự viết, hỗ trợ C/C++ và Java, chạy bài nộp trực tiếp trên máy chủ **không có sandbox**.
- **Bộ chấm mạng (phiên bản đầu tiên)**: lại là một monolith MVC riêng nữa cho lập trình mạng Java.

Mỗi hệ là một khối độc lập, không chia sẻ kiến trúc — nên chi phí bảo trì **nhân lên theo số hệ thống** chứ không hề được chia nhỏ. Đây mới là *động cơ thật* để tách service: một thay đổi nhỏ buộc build/deploy lại cả khối, bộ chấm không sandbox tạo rủi ro bảo mật (mã sinh viên chạy chung tiến trình máy chủ), và không thể mở rộng phần chấm độc lập với phần quản lý.

Quá trình chuyển đổi từ các monolith rời rạc sang các service `kbm-*` diễn ra **tăng dần** (Strangler Fig — xem Chương 10), không phải một lần viết lại. Bài học cốt lõi mở đầu cho cả cuốn sách: **microservices là điểm đến của một quá trình tiến hóa, không phải điểm khởi đầu** — đúng với tinh thần "Monolith First" ở §1.6. Có một "before" cụ thể như vậy giúp người đọc nhìn thấy lý do tách service, thay vì coi microservices là lựa chọn hiển nhiên.

1.8.3. Kiến trúc tổng quan

Hình 1.8 là sơ đồ giản lược kiến trúc KBM với năm nhóm dịch vụ tiêu biểu để mở đầu câu chuyện (cấu trúc đầy đủ gồm 7 Bounded Context core sẽ được phân tích ở Chương 2).



Hình 1.8: *Kiến trúc tổng quan KBM LMS (sơ đồ giản lược, xem đầy đủ ở Chương 2)*

Mô hình này không phải là trạng thái cuối cùng, mà là bối cảnh khởi đầu để chúng ta theo dõi quá trình chuyển đổi.

Sự tiến hóa của KBM qua 12 chương

KBM tiến hóa qua ba trạng thái được theo dõi xuyên suốt sách:

Trạng thái hiện tại (Chương 1–2): Từ ba khối monolith ban đầu (ASP.NET quản lý đào tạo cùng hai bộ chấm Code và Network dạng MVC), hệ thống đã tiến hóa thành 7 Bounded Context core (Identity & Access, SQL Practice, Network Practice, Code Practice — chuyển từ bộ chấm MVC tự viết sang DOMjudge, Academic Management, Examination & Proctoring — logic thi và giám sát hiện còn nằm chung trong `kbm-core` cùng ứng dụng desktop `kbm-exam-client`, và DevOps Practice) cùng 2 supporting context, nhưng vẫn còn shared database giữa SQL Practice và Academic Management — tạo coupling ngầm giữa hai dịch vụ.

Trạng thái đang chuyển đổi (Chương 3–11): Mỗi chương phân tích một khoảng trống cụ thể và đề xuất lộ trình chuyển đổi: tách database, thêm Saga pattern, hoàn thiện API Gateway hiện có, triển khai Circuit Breaker... Hệ thống dần tiến về kiến trúc microservices đầy đủ.

Trạng thái mục tiêu (Chương 12): mỗi service sở hữu dữ liệu của mình — tách schema, truy cập chéo qua API, tách hẳn DB server khi có nhu cầu mở rộng riêng (Chương 7); event-driven communication qua Kafka, CI/CD pipeline tự động, observability đầu cuối.

Các bài toán kỹ thuật chính. Mỗi chương sách sẽ quay lại KBM ở góc nhìn khác nhau. Bảng 1.7 ánh xạ các bài toán kỹ thuật — từ hiện trạng đến hướng chuyển đổi:

Bảng 1.7: *Ánh xạ bài toán kỹ thuật KBM LMS theo chương*

Bài toán kỹ thuật	Hiện trạng (khảo sát đầu năm 2026)	Vấn đề cần giải quyết	Chương
Domain decomposition	7 Bounded Context core + supporting services	Shared library và shared DB vẫn tạo coupling	Ch.2
API design	REST controllers + Network Lab đa giao thức	Naming/versioning chưa thống nhất; contract nhiều protocol	Ch.3
Sync communication	OpenFeign, bộ điều phối `kbm-judge-sql` + các worker `kbm-sandbox-*`, tích hợp ngoài	Cần resilience + Anti-Corruption Layer rõ ràng	Ch.4
Async communication	Kafka cho chế độ thi SQL; DB queue cho DevOps Lab	Cần idempotency, retry, dead letter và workload-based choice	Ch.5
Distributed data	**Shared database** giữa SQL Practice và Academic Management	Vi phạm database-per-service, coupling cao	Ch.6, Ch.7
API Gateway	Spring Cloud Gateway + Go reverse proxy cho lab routing	Path-based vs hostname-based gateway trade-off	Ch.8
Security	JWT HS256, OAuth2, SSO cross-platform	Secret management, token storage, phòng thủ chiều sâu	Ch.9
Migration strategy	10 năm tiến hóa, legacy QLĐT, GitHub Classroom	Strangler Fig, ACL, roadmap theo rủi ro	Ch.10
Observability	Actuator/Prometheus một phần, correlation ID đang mở rộng	Cần centralized log, tracing, metric đầu cuối	Ch.11
Deployment	Docker Compose cho LMS chính; k3s/ Sysbox cho DevOps Lab	CI/CD, rollback, resource isolation, tránh thiết kế quá mức (over-engineering)	Ch.12

Trong các bài toán trên, phân chia dữ liệu là thách thức dai dẳng nhất — nhìn ngay tại KBM:

KBM sử dụng shared database giữa SQL Practice (`kbm-core`) và Academic Management (`kbm-academic`) — đây là **anti-pattern** theo nguyên tắc database-per-service [2a, Ch.2]. Hậu quả: thay đổi schema ảnh hưởng đến cả hai service, không thể mở rộng hay deploy độc lập, và hai team nếu có sẽ phải phối hợp trong mọi đợt chuyển đổi schema. Lộ trình cải thiện: tách schema → API-based data access → database-per-service. Chi tiết sẽ phân tích tại Chương 7.

Những quyết định chúng ta muốn làm lại — Cautionary Tale. Richardson trong phiên bản 2 [2b, Ch.2] dành nguyên một chương kể câu chuyện cuộc chuyển đổi FTGO thất bại — cho thấy *những sai lầm* trước khi dạy pattern đúng. KBM cũng có những bài học tương tự — năm quyết định mà team sẽ chọn khác nếu làm lại.

Năm quyết định dưới đây là để nhận diện vấn đề — cơ chế chi tiết ở các chương sau; người đọc chưa quen với JWT HS256 hay idempotency có thể lướt qua trước.

Bảng 1.8: *Năm quyết định kiến trúc KBM muốn làm lại*

#	Quyết định	Tại sao chọn lúc đó	Rủi ro & hệ quả kiến trúc	Anti-pattern tương ứng
①	**Shared database** giữa SQL Practice và Academic Management	Nhanh hơn, không cần API giữa hai service	Thay đổi schema ở một dịch vụ có thể làm lỗi dịch vụ kia nếu thiếu quy ước sở hữu bảng; không thể tách chu kỳ chuyển đổi schema độc lập, buộc phải phối hợp triển khai đồng bộ	Richardson [2b]: *Shared Database anti-pattern*
②	**Shared library** chứa quá nhiều thứ (entity, security, SQL parser)	Tái sử dụng code, giảm duplication	Thay đổi trong thư viện dùng chung thường kéo theo việc biên dịch và triển khai lại nhiều dịch vụ cùng lúc, làm mất khả năng triển khai độc lập (independent deployability)	Richardson [2b]: *Lock-step deployment*
③	**JWT HS256** (khóa đối xứng chia sẻ giữa các service)	Cấu hình nhanh chóng chỉ mất vài phút	Vì khóa dùng để xác minh (verify) cũng chính là khóa dùng để ký, nên bất kỳ dịch vụ nào bị xâm phạm (compromise) cũng đều có thể làm giả JWT. Ở kiến trúc mục tiêu nhiều service, cần chuyển sang khóa bất đối xứng (RS256/JWKS) để chỉ Auth Service giữ quyền ký	Chi tiết tại Ch.9
④	**Không có idempotency** cho submit endpoint	"Không ai gửi yêu cầu trùng lặp đâu"	Trong chế độ thi: client tự động retry khi mạng không ổn định, dẫn đến nộp bài trùng lặp (duplicate submission) và chấm điểm sai	Chi tiết tại Ch.5, Ch.6
⑤	`docker logs` **thay cho observability**	"Quy mô nhóm nhỏ nên đọc log trực tiếp sẽ tiết kiệm thời gian hơn"	Monitoring truyền thống chỉ báo hệ thống lỗi, thiếu Observability để hiểu "tại sao". Chế độ thi: hàng trăm lượt nộp mỗi phút, Judge chậm, không xác định được bottleneck ở đâu, dẫn đến quá trình debug có thể mất hàng giờ	Chi tiết tại Ch.11

Mỗi quyết định trên đều *hợp lý tại thời điểm đó* — team nhỏ, cần phát hành sớm, ưu tiên tính năng hơn kiến trúc. Nhưng khi hệ thống mở rộng (chế độ thi, quản lý bài tập, quản lý khóa luận, Network Lab đa giao thức, DevOps Lab bằng Go), nợ kỹ thuật (technical debt) tích lũy nhanh hơn tốc độ phát triển tính năng. Đây là kiểu *penny wise and pound foolish* — tiết kiệm ở bước nhỏ, trả giá ở bước lớn. Nhưng đó cũng chính là giá trị sư phạm: **nếu hệ thống đã hoàn chỉnh từ đầu, sẽ không có gì để học** — mỗi anti-pattern là cơ hội để hiểu *tại sao* các giải pháp khuyến nghị tồn tại, không phải vì sách giáo khoa nói vậy, mà vì *hệ quả thực tế* đối với khả năng tiến hóa và vận hành hệ thống.

⚠ Sai lầm thường gặp

1. **Áp dụng microservices chỉ vì "chạy theo xu hướng" (trending)** — Nhiều team chọn microservices vì thấy Netflix, Amazon dùng, mà không tự hỏi: "Vấn đề cụ thể nào của chúng ta mà microservices giải quyết?" Hậu quả: tăng độ phức tạp mà không mang lại lợi ích. *Phòng tránh*: luôn bắt đầu từ vấn đề (Decision Framework §1.7), không từ giải pháp.
2. **Chuyển đổi đột ngột từ monolith sang microservices** — Tách patchwork monolith trực tiếp, bỏ qua bước modular hóa. Hậu quả: distributed monolith — hội tụ nhược điểm của cả hai. *Phòng tránh*: refactor ranh giới module rõ ràng *trước*, rồi mới tách service (§1.6).
3. **Tách service trước khi tổ chức team** — Chia 10 microservices nhưng vẫn 1 team duy nhất, deploy cùng nhau, review code chéo nhau. Hậu quả: overhead vận hành mà không có autonomy thực sự (Conway's Law). *Phòng tránh*: tổ chức team phù hợp *cùng lúc* hoặc *trước* khi tách service (chi tiết ở Chương 2).

Các pattern trong chương có minh họa tương tác đi kèm tại companion web:

Interactive Demo

- **So sánh Monolith và Microservices**: monolith-vs-microservices.html

1.9. Tổng kết

Kiến trúc phần mềm đã tiến hóa từ monolith, qua SOA, đến microservices với trọng tâm đặt tính độc lập lên hàng đầu.

Thực tế cho thấy, **mỗi bước tiến hóa đều giải quyết giới hạn của bước trước, nhưng đồng thời tạo ra thách thức mới**. SOA giải phóng khỏi silo nhưng tạo ra coupling vào middleware. Microservices trao quyền tự trị cho từng service nhưng đổi lại bằng complexity tax của distributed systems. Không có kiến trúc nào là "tốt nhất" — chỉ có kiến trúc phù hợp nhất cho ngữ cảnh.

Kinh nghiệm từ Netflix, Amazon, và Uber khẳng định rằng **chuyển đổi sang microservices là giải pháp trước giới hạn thực tế, không phải theo đuổi xu hướng**. Và trạng thái hybrid — trạng thái vận hành phổ biến trong thực tế — là hoàn toàn hợp lý.

Modular monolith không phải bước lùi mà là **bước đệm thông minh**: hiểu rõ domain trước khi chia tách. Quyết định kiến trúc cũng không chỉ là kỹ thuật — ngân sách, team, lịch trình, và văn hóa tổ chức nhiều khi còn nặng hơn yếu tố kỹ thuật.

Case study KBM — một hệ thống LMS với các ràng buộc điển hình (team nhỏ, hạ tầng hạn chế) — cung cấp bài học thực tiễn về những sai lầm kiến trúc ban đầu.

1.10. Đọc thêm

Sách tham khảo chính:

1. [1] Thomas Erl, *Service-Oriented Architecture: Analysis and Design for Services and Microservices*, 2nd Ed. — Ch.3–4: Service-Orientation Principles, Understanding SOA
2. [2a] Chris Richardson, *Microservices Patterns*, 1st Ed. — Ch.1: Escaping Monolithic Hell, Scale Cube
3. [2b] Chris Richardson, *Microservices Patterns*, 2nd Ed. — Ch.1: Fast Flow Success Triangle, DORA Metrics; Ch.5: Architectural Requirements for Fast Flow; Ch.6: Modular Monolith
4. [4b] Sam Newman, *Monolith to Microservices* — Ch.1: Just Enough Microservices, Should You Migrate?
5. [5] Hugo Rocha, *Practical Event-Driven Microservices Architecture* — Ch.1: SOA vs Microservices vs EDA

Sách bổ trợ:

6. Nicole Forsgren, Jez Humble, Gene Kim, *Accelerate* (2018) — DORA Metrics, nghiên cứu định lượng về software delivery performance
7. Gene Kim et al., *The DevOps Handbook*, 2nd Ed. (2021) — Three Ways of DevOps, deployment pipeline design

Nguồn trực tuyến:

- [W1] James Lewis & Martin Fowler, "Microservices" (2014) — martinfowler.com/articles/microservices.html

- [W2] Martin Fowler, “MonolithFirst” (2015) — martinfowler.com/bliki/MonolithFirst.html
- [W3] Netflix Technology Blog, “Completing the Netflix Cloud Migration” (2016)
- [W4] Uber Engineering, “Domain-Oriented Microservice Architecture” (2020)
- [W5] Steve Yegge, “Stevey’s Google Platforms Rant” (2011) — nguồn câu chuyện Bezos API Mandate
- DORA, “Accelerate State of DevOps Report 2019” (nguồn số liệu bảng DORA Metrics ở §1.5) & “State of DevOps Report 2024” — dora.dev/research

Bài tập: 1-0, 1-1, 1-2 ở phần Bài tập & case study cuối sách.

CHƯƠNG 2

Phân tích hướng domain — DDD & Bounded Contexts

> "Use the model as the backbone of a language. Commit the team to exercising that language relentlessly in all communication within the team and in the code."
> — Eric Evans, *Domain-Driven Design*

MỤC TIÊU HỌC TẬP

- Hiểu Conway's Law và tác động của cấu trúc tổ chức lên kiến trúc phần mềm
- Nắm vững các khái niệm DDD cốt lõi: Entity, Value Object, Aggregate, Repository
- Áp dụng Strategic DDD: Bounded Context, Context Map, Ubiquitous Language
- Thực hành xác định ranh giới dịch vụ từ domain — kỹ năng quan trọng nhất khi thiết kế microservices
- So sánh hai phương pháp phân tách: DDD (heuristic-based) vs Erl Service-Oriented Analysis (step-by-step)
- Phân tích case study KBM: 7 Bounded Context core cùng 2 supporting context; phân biệt Shared Library với Shared Kernel khi quyết định chia sẻ mã nguồn

2.1. Conway's Law — tổ chức quyết định kiến trúc

Một trong những quy luật nền tảng ảnh hưởng đến mọi quyết định kiến trúc — đặc biệt quan trọng khi áp dụng Domain-Driven Design — là: **kiến trúc phần mềm phản ánh cấu trúc tổ chức phát triển nó**.

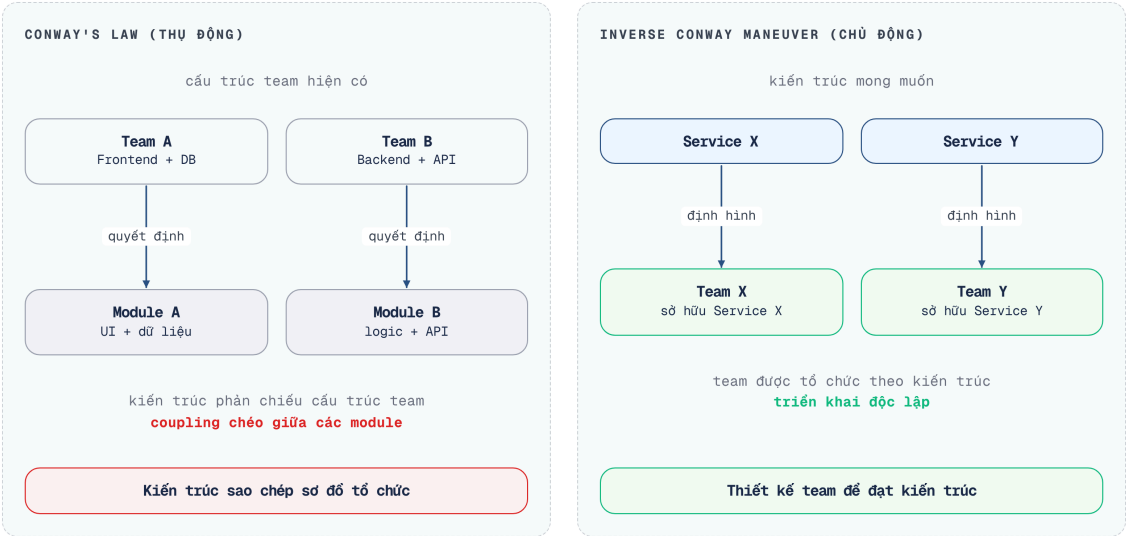
Quy luật Conway. Năm 1968, Melvin Conway quan sát rằng:

> **Conway's Law**
>
> "Organizations which design systems ... are constrained to produce designs which are copies of the communication structures of these organizations."
>
> — *Melvin Conway, 1968 (trích lại trong Richardson [2a, Ch.2])*

Nói cách khác: một tổ chức có 3 team sẽ có *xu hướng* tạo ra hệ thống xoay quanh 3 khối lớn (service hoặc module) — không phải ánh xạ 1:1 tuyệt đối, mà là ranh giới hệ thống phản chiếu ranh giới giao tiếp. Nếu hai nhóm phát triển giao tiếp với nhau tần suất cao, hai dịch vụ tương ứng cũng sẽ hình thành sự liên kết chặt chẽ (tight coupling). Đây không phải lý thuyết suông — nghiên cứu của MacCormack, Rusnak & Baldwin tại Harvard Business School (2008) đã cung cấp bằng chứng thực nghiệm hỗ trợ quy luật này trên các dự án mã nguồn mở.

Đối với lập trình viên và kiến trúc sư phần mềm, điều này có ý nghĩa thực tiễn rõ ràng: **không thể thiết kế microservices tốt nếu cấu trúc team không phù hợp**. Đây là lý do Amazon tái cấu trúc thành "two-pizza teams" *trước* khi phân rã hệ thống nguyên khối (monolithic system) (đã thảo luận ở Chương 1).

Inverse Conway Maneuver. Thay vì chấp nhận thụ động, nhiều tổ chức chủ động áp dụng Conway's Law theo hướng ngược lại: **thiết kế cấu trúc team để đạt được kiến trúc phần mềm mong muốn**. Chiến lược này gọi là *Inverse Conway Maneuver*.



Hình 2.1: *Conway's Law (thụ động) vs Inverse Conway Maneuver (chủ động)*

Theo mô hình này, một nhóm phát triển phụ trách một hoặc một vài dịch vụ liên quan, chịu trách nhiệm cho toàn bộ vòng đời (lifecycle) của phần mềm — từ lúc viết mã nguồn đến khi vận hành

trên production. Đây là mô hình "you build it, you run it" mà Amazon và Netflix đã chứng minh hiệu quả.

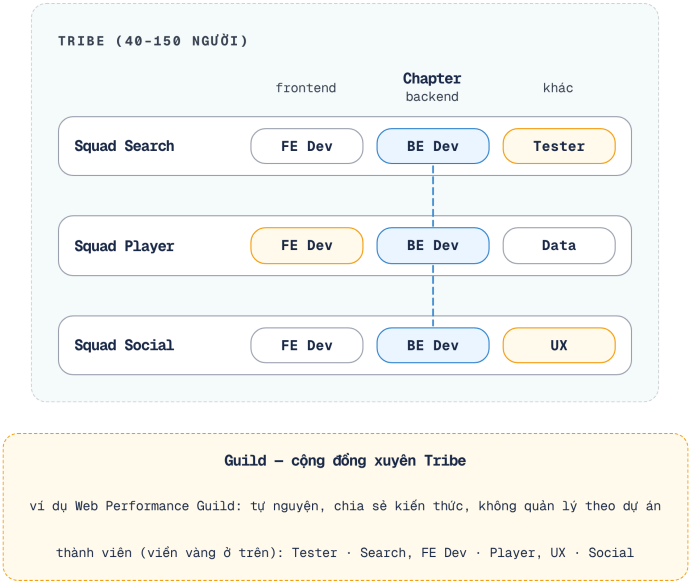
Team Topologies — Bốn kiểu team. Matthew Skelton và Manuel Pais đề xuất bốn kiểu team cơ bản cho tổ chức phần mềm hiện đại:

Bảng 2.1: Bốn kiểu team theo Team Topologies

Kiểu team	Vai trò	Nếu KBM có nhiều nhóm (giả định)
Stream-aligned	Phát triển theo luồng nghiệp vụ, sở hữu trọn luồng	Team "Bài tập & Chấm thi", Team "Quản lý người dùng"
Platform	Cung cấp nền tảng nội bộ giúp các nhóm stream-aligned phát triển nhanh hơn	Team DevOps (CI/CD, Docker, monitoring)
Enabling	Hỗ trợ các team khác nâng cao kỹ năng	Team kiến trúc (DDD coaching, code review)
Complicated subsystem	Sở hữu thành phần phức tạp đòi hỏi chuyên môn sâu	Team "SQL Judge" (sandbox isolation, multi-DBMS)

Với KBM — một dự án nhỏ (1–2 lập trình viên chính, cộng sinh viên hỗ trợ theo đợt) — không thể áp dụng đầy đủ 4 mô hình nhóm. Tuy nhiên, nguyên lý cốt lõi vẫn được giữ nguyên: lập trình viên nào am hiểu sâu sắc về một miền nghiệp vụ (domain) sẽ tự nhiên đảm nhận trách nhiệm chính cho dịch vụ tương ứng. Khi nhóm mở rộng, cấu trúc này sẽ trở nên tường minh hơn.

Case study thực tế: Spotify Squad Model. Spotify (2012) là case study nổi tiếng nhất về tổ chức team cho microservices. Henrik Kniberg và Anders Ivarsson mô tả mô hình gồm bốn thành phần:



Hình 2.2: Mô hình Squad/Tribe/Chapter/Guild của Spotify

Chi tiết về cấu trúc này và ý nghĩa của chúng đối với kiến trúc microservices được tóm tắt trong bảng sau.

Bảng 2.2: Cấu trúc tổ chức Spotify — ý nghĩa cho microservices

Cấu trúc	Mô tả	Ý nghĩa cho microservices
Squad (6–12 người)	Đơn vị tự trị, sở hữu trọn một tính năng	Tương ứng stream-aligned team, sở hữu một hoặc vài microservices

Cấu trúc	Mô tả	Ý nghĩa cho microservices
Tribe (thường ước lượng 40–150 người; dưới khoảng 100 người theo Kniberg & Ivarsson 2012)	Nhóm các squad cùng sứ mệnh (mission)	Đồng bộ mục tiêu mà không mất tính tự trị
Chapter	Nhóm người cùng năng lực chuyên môn xuyên các squad	Chia sẻ kinh nghiệm và chuẩn kỹ thuật chung (ví dụ: tất cả kỹ sư lập trình backend)
Guild	Cộng đồng quan tâm xuyên các tribe	Chia sẻ kiến thức (ví dụ: Web Performance Guild)

Bài học cho hệ thống nhỏ: Spotify áp dụng mô hình này khi đã có hơn 30 squad (Kniberg & Ivarsson, 2012) — quy mô đội KBM hiện nay rất nhỏ nên chưa cần mô hình squad/tribe. Nhưng nguyên tắc cốt lõi vẫn giữ nguyên giá trị: mỗi dịch vụ cần có người chịu trách nhiệm (owner) rõ ràng, và khi nhóm mở rộng, cần tổ chức nhân sự theo các Bounded Context (ranh giới nghiệp vụ độc lập — khái niệm trọng tâm sẽ được định nghĩa ở §2.3) thay vì theo technical layer.

Nguồn: Henrik Kniberg, Anders Ivarsson, "Scaling Agile @ Spotify," 2012. Lưu ý: Spotify đã tiến hóa/cải tiến mô hình này đáng kể từ năm 2012 — mô hình nguyên thủy chỉ là điểm khởi đầu, không phải là một khuôn mẫu cứng nhắc.

Team nhỏ và Conway's Law

Trong KBM, cùng một lập trình viên phát triển nhiều context (chẳng hạn SQL Practice và Academic Management). Điều này dẫn đến ranh giới giữa các service không rõ ràng — ví dụ: `spring.application.name` trùng nhau giữa `kbm-core` và `kbm-academic` (cả hai đều là `app`), shared database vẫn được duy trì vì tiện lợi. Richardson trong [2a] cho thấy việc định nghĩa rõ service ownership từ đầu giúp tránh sự phụ thuộc ngầm (implicit coupling). **Lộ trình chuyển đổi (Migration path)**: tách rõ naming, định nghĩa API contract giữa các service, và chuẩn bị cho việc mở rộng team.

2.2. DDD cơ bản — ngôn ngữ chung và các building blocks

Domain-Driven Design (DDD) là phương pháp thiết kế phần mềm đặt **domain** (miền nghiệp vụ) làm trung tâm. Eric Evans giới thiệu DDD năm 2003 trong cuốn sách cùng tên, và nó trở thành nền tảng cho việc phân tách microservices.

Đối với lập trình viên, DDD trả lời câu hỏi quan trọng nhất khi thiết kế microservices: **tách ở đâu?** Không phải tách theo layer kỹ thuật (UI team, DB team, API team) mà tách theo **ranh giới nghiệp vụ** — nơi mà sự thay đổi trong một domain không ảnh hưởng đến domain khác.

2.2.1. Ubiquitous Language — ngôn ngữ chung

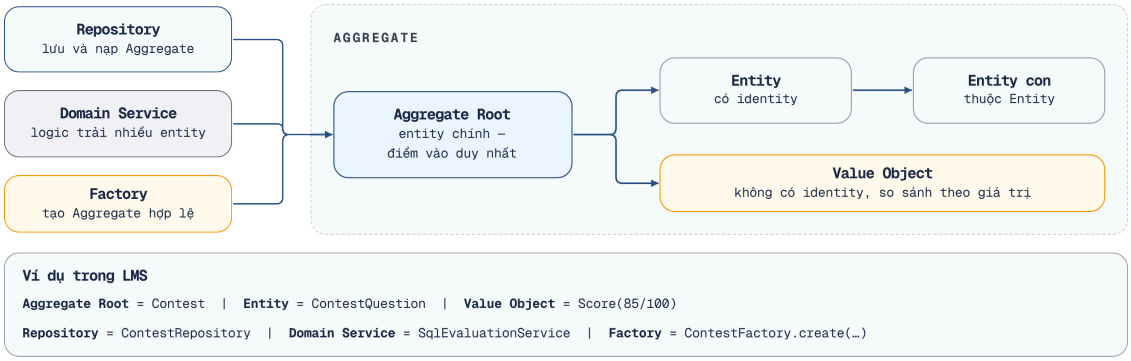
Bước đầu tiên và quan trọng nhất trong DDD là **xây dựng ngôn ngữ chung** (*Ubiquitous Language*) giữa lập trình viên và chuyên gia nghiệp vụ (domain expert).

Ubiquitous Language là tập hợp thuật ngữ mà *toàn bộ nhóm phát triển* — lập trình viên, kiểm thử viên, quản lý sản phẩm — sử dụng nhất quán trong mọi giao tiếp: email, các cuộc họp, mã nguồn, tài liệu. Nếu domain expert nói "bài nộp" (*submission*), code phải có class `Submission`, database có bảng `submissions`, API có endpoint `/submissions`. Không được có sự khác biệt.

Tại sao điều này quan trọng cho microservices? Vì khi hai nhóm phát triển bắt đầu dùng cùng một thuật ngữ nhưng với **ý nghĩa khác nhau**, đó là tín hiệu họ đang ở hai Bounded Context khác nhau; tách thành hai service triển khai riêng hay không còn tùy các lực đánh đổi ở §2.6.

2.2.2. Các building blocks

DDD định nghĩa một bộ building blocks để mô hình hóa domain:



Hình 2.3: Các building blocks trong DDD — Aggregate, Entity, Value Object, Repository

Entity — Đối tượng có danh tính (*identity*) duy nhất xuyên suốt vòng đời. Hai entity có cùng thuộc tính nhưng khác ID vẫn là hai entity khác nhau. Ví dụ trong KBM: `Student(id=1, name="Nguyen Van A")` và `Student(id=2, name="Nguyen Van A")` là hai sinh viên khác nhau dù trùng tên.

Value Object — Đối tượng được xác định bởi *giá trị*, không có identity riêng. Hai value object cùng giá trị là tương đương. Ví dụ: `Score(value=85, maxValue=100)` — không quan trọng "đây là điểm nào", chỉ quan trọng giá trị 85/100.

Aggregate — Cụm entity và value object được quản lý như một đơn vị nhất quán (*consistency boundary*). Mỗi aggregate có một **Aggregate Root** — entity duy nhất mà bên ngoài có thể tham chiếu. Trong LMS, Contest có thể là aggregate root bao gồm ContestQuestion, ContestParticipant — không một thành phần bên ngoài nào được phép *thay đổi dữ liệu* ContestQuestion mà không thông qua Aggregate Root là Contest. (Lưu ý: Có thể dùng SELECT trực tiếp để đọc dữ liệu, nhưng khi thao tác ghi (*Write*), mọi logic bắt buộc phải qua Root để tránh data corruption).

Aggregate = Transaction boundary

Trong microservices, aggregate chính là đơn vị transaction. Một transaction chỉ nên thao tác trên **một aggregate** duy nhất; khi quy trình nghiệp vụ chạm nhiều aggregate, hãy ưu tiên phối hợp qua domain event và chấp nhận eventual consistency. Khi nghiệp vụ trải qua *nhiều service* — mỗi service một database — transaction chung không còn khả thi: đó mới là lúc cần Saga pattern (Chương 6). Việc sử dụng `@Transactional` để cập nhật nhiều aggregate trong cùng một giao dịch cơ sở dữ liệu vẫn có thể chấp nhận được trong monolith, nhưng trong microservices, mở rộng một transaction xuyên hai service sẽ làm mất khả năng mở rộng (scale) độc lập và tạo nguy cơ khóa lẫn nhau giữa các dịch vụ.

Repository — Interface cho phép làm việc với aggregate như với một collection trong bộ nhớ. Lập trình viên làm việc với repository thay vì database trực tiếp. Trong Spring Boot: `@Repository interface ContestRepository extends JpaRepository<Contest, UUID>`.

Domain Service — Logic nghiệp vụ không thuộc về một entity hay value object cụ thể nào. Ví dụ: `SqlEvaluationService` so sánh kết quả SQL của sinh viên với đáp án — logic này không thuộc về `Submission` hay `Question` riêng lẻ.

2.2.3. DDD dành cho kỹ sư phần mềm: tư duy theo miền nghiệp vụ, không theo cơ sở dữ liệu

Sai lầm phổ biến nhất khi bắt đầu với DDD là **thiết kế entity từ bảng database**. DDD yêu cầu ngược lại: mô hình hóa domain trước, database sau.

Hai cách tiếp cận thiết kế phổ biến mang lại những kết quả hoàn toàn trái ngược nhau:

- **Database-first ×**: Bắt đầu từ bảng, cột và khóa ngoại (foreign key). Kết quả thường dẫn đến entity phản ánh nguyên vẹn cấu trúc bảng, dịch vụ trở thành các lớp bọc (wrapper) đơn thuần để thao tác dữ liệu (CRUD), và ranh giới nghiệp vụ giữa các thành phần bị mờ nhạt.
- **Domain-first ✓**: Bắt đầu từ quy trình nghiệp vụ và ngôn ngữ chuyên ngành (domain language). Kết quả là entity phản ánh chính xác các khái niệm nghiệp vụ, tạo ra những ranh giới tự nhiên và linh hoạt khi hệ thống cần mở rộng hoặc chia tách.

Trong thực tế, nhiều dự án (kể cả KBM) bắt đầu database-first. Ở nhiều hệ thống, khó khăn nghiêm trọng nhất của cách làm này là các bảng dữ liệu tập trung khổng lồ "God Table" (ví dụ minh họa: bảng `User` phình to thành 50 cột chứa hỗn hợp nhiều loại dữ liệu từ Auth, Billing đến Profile). Khi phân rã thành các microservice, các dịch vụ vẫn phải truy cập trực tiếp vào cùng một bảng `User` này, tạo ra sự phụ thuộc ngầm (coupling). Bước đầu tiên phải là *nghĩ lại* mô hình từ góc nhìn domain để phân rã (decompose) "God Table" đó ra.

Tư duy Database-first vs Domain-first

Có thể hình dung qua việc xây dựng Phòng Đào tạo. Với tư duy "Database-first", bước đầu tiên là mua các tủ hồ sơ, kệ sách, máy tính rồi sau đó mới nghĩ xem nhân viên sẽ làm việc thế nào với các trang thiết bị đó. Ngược lại, "Domain-first" yêu cầu vẽ ra quy trình trước: sinh viên nộp hồ sơ ở đâu, giáo vụ duyệt thế nào, trả kết quả ra sao, từ đó mới quyết định cần mua bao nhiêu tủ, sắp xếp bàn làm việc ra sao.

2.3. Strategic DDD — Bounded Context & Context Map

Nếu tactical DDD (Entity, Aggregate) giúp mô hình hóa *bên trong* một service, thì strategic DDD giúp trả lời câu hỏi lớn hơn: **hệ thống cần bao nhiêu service, và ranh giới ở đâu?** Để làm được điều này, DDD phân biệt rõ:

- **Problem Space (Không gian vấn đề)**: Chia domain lớn thành các Subdomain (subdomain cốt lõi, hỗ trợ hoặc chung).
- **Solution Space (Không gian giải pháp)**: Ánh xạ các Subdomain này thành các Bounded Context để cài đặt bằng code (lý tưởng là 1 Subdomain = 1 Bounded Context).

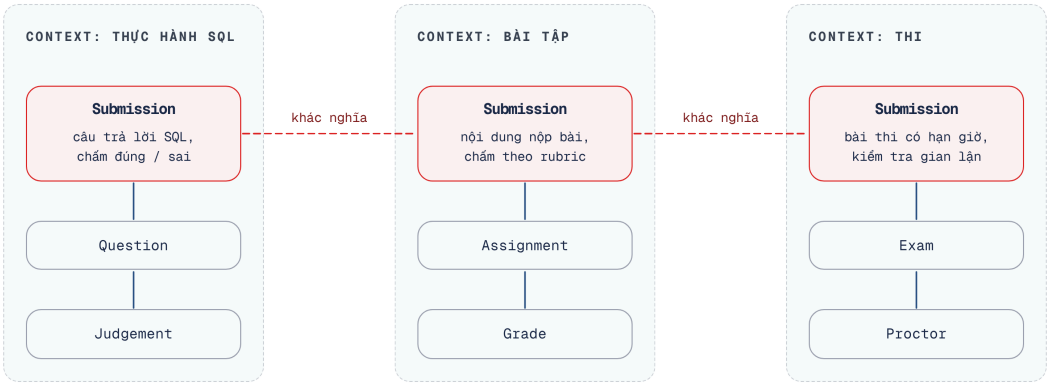
2.3.1. Bounded Context — ranh giới ngôn ngữ

Bounded Context là khái niệm quan trọng nhất trong DDD cho microservices. Một Bounded Context là phạm vi trong đó một mô hình domain cụ thể có hiệu lực — và **ngôn ngữ có ý nghĩa nhất quán**. Khái niệm này có thể được ví như một Khoa trong trường đại học quản lý độc lập sinh viên và điểm số của khoa đó, không cho phép khoa khác can thiệp trực tiếp dữ liệu.

Ví dụ thực tế: trong KBM, thuật ngữ "bài nộp" (*submission*) có ý nghĩa khác nhau tùy ngữ cảnh:

- Trong context **Thực hành SQL**: submission = câu trả lời SQL của sinh viên, cần chấm đúng/sai
- Trong context **Bài tập**: submission = file/nội dung nộp bài, cần chấm điểm theo rubric
- Trong context **Thi**: submission = bài thi với giới hạn thời gian, cần kiểm tra gian lận

Cùng một từ, ba ý nghĩa khác nhau → ba Bounded Context tiềm năng. Đây là heuristic hiệu quả nhất để xác định ranh giới: **khi cùng một thuật ngữ mang ý nghĩa khác nhau, chúng ta đang ở ranh giới giữa hai context**.



Hình 2.4: Cùng thuật ngữ "Submission" mang ý nghĩa khác nhau trong ba Bounded Context

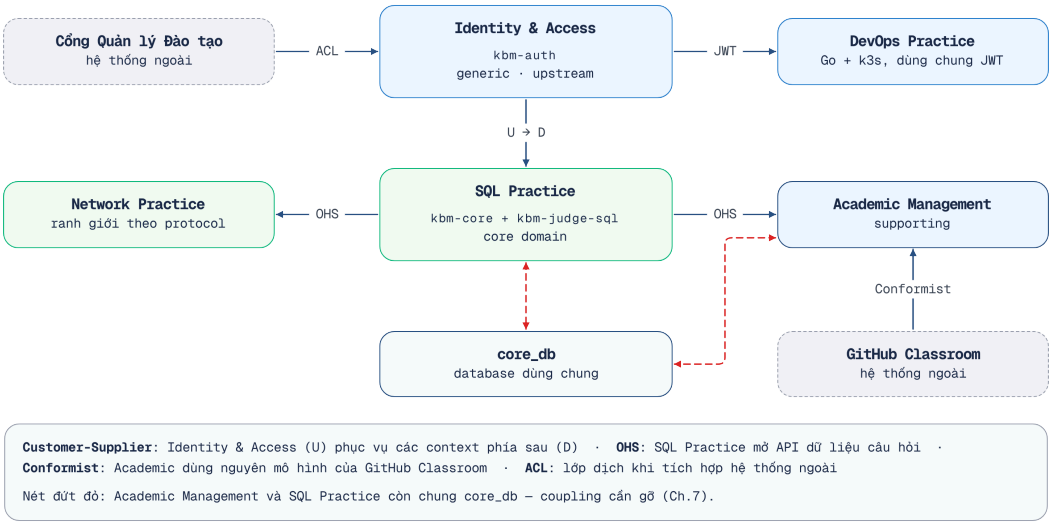
Context Map — Bản đồ quan hệ giữa các context. Khi đã xác định các Bounded Context, bước tiếp theo là vẽ **Context Map** — sơ đồ quan hệ giữa chúng. Context Map không chỉ cho thấy context nào giao tiếp với nhau, mà còn cho thấy **kiểu quan hệ**.

Các kiểu quan hệ phổ biến:

Bảng 2.3: Các kiểu quan hệ giữa các Bounded Context

Quan hệ	Mô tả	Ví dụ trong KBM
Partnership	Hai context phát triển đồng bộ, hợp tác chặt chẽ	Chưa có trong KBM (đội quá nhỏ); sẽ xuất hiện nếu hai nhóm cùng phát triển Core và Academic
Shared Kernel	Hai context cùng sở hữu và cùng quản trị một phần mô hình *miền*	Nhóm khái niệm miền dùng chung (vd: value object mã sinh viên, thang điểm). Phân biệt: thư viện kỹ thuật (base entities, JWT, exceptions) là Shared *Library*, không phải Shared Kernel
Customer-Supplier	Nhóm Upstream (U) ưu tiên phát triển theo yêu cầu nhóm downstream (D)	Identity & Access (U) → SQL Practice (D)
Conformist	(D) chấp nhận nguyên mô hình của (U) vì (U) không có động cơ đáp ứng — khác Customer-Supplier ở chỗ không thương lượng được	Academic Management dùng nguyên mô hình lớp/bài tập của GitHub Classroom: không thể yêu cầu bên ngoài đổi contract
Anti-Corruption Layer	(D) đặt lớp dịch trung gian ở biên để chuyển đổi mô hình của (U) sang mô hình nội bộ, ngăn mô hình ngoại lai xâm nhập domain	`kbm-auth` dựng lớp dịch khi tích hợp Cổng Quản lý Đào tạo (hệ thống ngoài) — model bên ngoài không xâm nhập domain KBM (Chương 4, Chương 10)
Open Host Service	(U) mở giao thức/API chuẩn hóa, có tài liệu rõ ràng để phục vụ đồng thời nhiều (D) tích hợp	`kbm-auth` cung cấp API xác thực/JWT chuẩn cho toàn hệ thống; hoặc `kbm-core` mở API dữ liệu câu hỏi cho các context khác khi tách DB (Ch.7)
Published Language	Ngôn ngữ trao đổi được chuẩn hóa (JSON, Protobuf)	Kafka messages với schema chuẩn

Áp dụng các mẫu quan hệ này vào KBM, chúng ta có Context Map của hệ thống ở Hình 2.5.



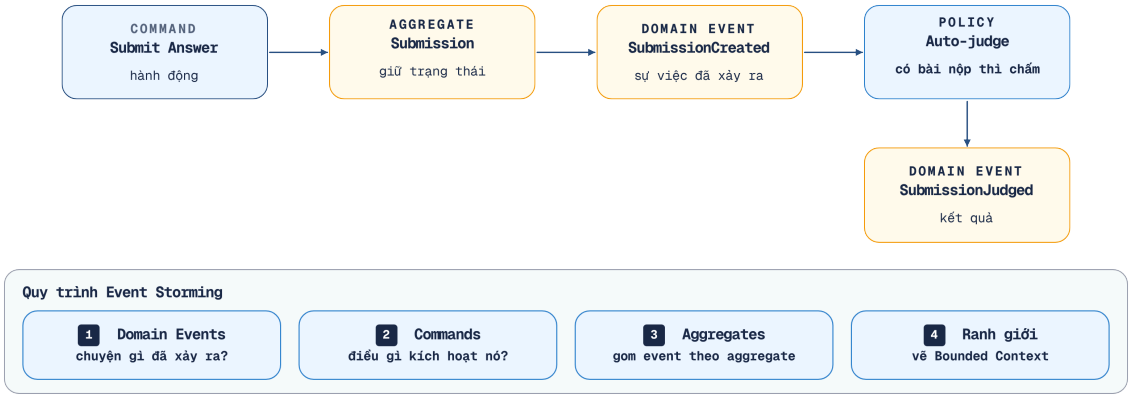
Hình 2.5: *Context Map của KBM — quan hệ giữa các Bounded Context*

2.3.2. Xác định Bounded Context từ yêu cầu nghiệp vụ

Làm thế nào để "tìm" Bounded Context trong một hệ thống? Không có công thức chính xác — đây là kỹ năng cần luyện tập. Tuy nhiên, có một số *heuristic* hữu ích (heuristic là quy tắc kinh nghiệm — lối suy luận thực dụng giúp ra quyết định nhanh khi không có công thức chính xác):

1. **Heuristic ngôn ngữ** — Khi cùng một thuật ngữ mang ý nghĩa khác nhau cho các nhóm người khác nhau, đó là ranh giới context. Ngược lại, khi hai thuật ngữ khác nhau chỉ cùng một thứ (ví dụ: "ref-des" và "component instance" trong câu chuyện PCB của Eric Evans), chúng thuộc cùng context.
2. **Heuristic thay đổi** — Tự hỏi: "Khi yêu cầu nghiệp vụ X thay đổi, những đoạn mã nguồn nào cần sửa?" Nếu thay đổi luôn ảnh hưởng đến cùng một nhóm tệp tin (files), chúng thuộc cùng context. Nếu thay đổi lan sang nhóm khác, đó là tín hiệu coupling cần xem xét.
3. **Heuristic team** — Bộ phận nào sẽ đảm nhận phát triển và bảo trì phần này? Nếu do cùng một nhóm phụ trách, các thành phần này *có thể* thuộc cùng một ngữ cảnh. Nếu do các nhóm khác nhau đảm nhận, cần *cân nhắc* việc phân tách ngữ cảnh (Conway's Law).
4. **Heuristic data** — Dữ liệu nào "thuộc về" nhau? Những entity nào phải cùng giữ một bất biến nghiệp vụ trong một transaction? Đó là ứng viên cho cùng một aggregate — chỉ thường được truy vấn cùng nhau thì chưa đủ để kết luận. Entity nào chỉ được tham chiếu qua ID? Đó có thể thuộc context khác.

Event Storming — Phương pháp khám phá domain. Event Storming là kỹ thuật workshop do Alberto Brandolini phát triển, giúp cả team cùng khám phá domain thông qua *domain events* — những sự kiện đã xảy ra trong hệ thống.



Hình 2.6: *Event Storming — luồng từ Command đến Domain Event*

Quy trình cơ bản:

1. **Liệt kê domain events** — giấy ghi chú (post-it) màu cam: "Submission Created", "SQL Executed", "Score Calculated"
2. **Tìm commands** — giấy ghi chú màu xanh: "Submit Answer", "Start Exam", "Create Assignment"
3. **Nhóm event theo aggregate** — giấy ghi chú màu vàng: những event nào cùng xảy ra trên một entity?
4. **Vẽ ranh giới** — nhóm các aggregate có liên quan chặt chẽ → Bounded Context

Kết quả Event Storming tự nhiên dẫn đến các Bounded Context. Nhóm event/aggregate nào tương tác nhiều nội bộ nhưng ít tương tác với nhóm khác → đó là một context.

2.3.3. Phân rã hướng dịch vụ theo Erl — cách tiếp cận step-by-step

DDD và Event Storming là phương pháp *khám phá* — bắt đầu từ ngôn ngữ domain, dần dần tìm ra ranh giới. Cách tiếp cận này mạnh nhưng đòi hỏi kinh nghiệm: các lập trình viên thiếu kinh nghiệm thường gặp khó khăn trong việc xác định điểm dừng hoặc tính đúng đắn của ranh giới.

Thomas Erl trong *Service-Oriented Architecture: Analysis and Design for Services and Microservices* [1] đề xuất một phương pháp bổ trợ: **Service-Oriented Analysis** — quy trình *prescriptive*, gồm các bước cụ thể từ yêu cầu nghiệp vụ đến danh mục dịch vụ. Giáo trình tóm lược tinh thần phương pháp này thành năm bước dễ dạy và dễ học, mỗi bước có input và output rõ ràng.

Năm bước phân rã dịch vụ (giáo trình tổng hợp theo tinh thần Erl [1]):

1. **Xác định automation candidates (ứng viên tự động hóa)** — Phân tích quy trình nghiệp vụ (business process) và xác định những bước nào *nên* được tự động hóa bằng phần mềm. Erl gọi đây là các *service candidate* — chưa phải service hoàn chỉnh, mà là ứng viên.

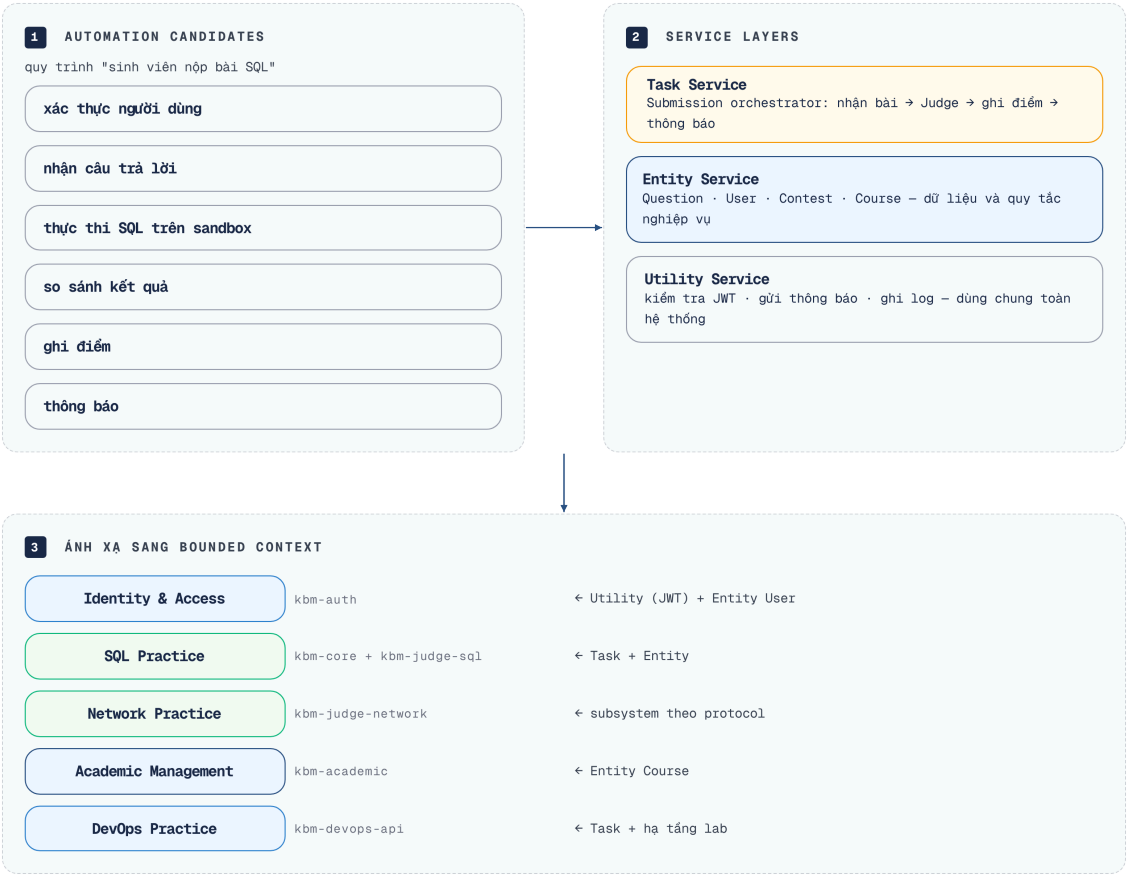
   Ví dụ KBM: quy trình "Sinh viên nộp bài SQL" có các bước: (1) xác thực người dùng, (2) nhận câu trả lời, (3) thực thi SQL trên sandbox, (4) so sánh kết quả, (5) ghi điểm, (6) thông báo. Mỗi bước là một automation candidate.
2. **Phân loại service layers** — Erl định nghĩa ba loại service:
   - **Task Service**: Chịu trách nhiệm điều phối quy trình nghiệp vụ (workflow) bằng cách gọi các service khác. Ví dụ trong KBM: Submission orchestrator thực hiện chuỗi xử lý từ nhận bài, gọi dịch vụ chấm điểm (Judge), ghi điểm cho đến thông báo.

- **Entity Service**: Chuyên quản lý dữ liệu nghiệp vụ cùng các thao tác CRUD và quy tắc nghiệp vụ (business rules). Ví dụ: Question Service, User Service, Contest Service.
- **Utility Service**: Cung cấp các chức năng kỹ thuật được dùng chung trên toàn hệ thống. Ví dụ: Dịch vụ kiểm tra JWT (validation), gửi thông báo (notification), hoặc ghi nhật ký hệ thống (logging).

Phân loại này giúp tránh sai lầm phổ biến: tạo ra các dịch vụ chỉ đơn thuần thực hiện CRUD (entity service không có business logic) hoặc gom toàn bộ logic vào một "god service" (thiếu task/utility decomposition).

3. **Tạo service inventory blueprint** — Vẽ bản đồ toàn bộ các service dự kiến, mỗi dịch vụ với các thao tác (operation) chính. Tương tự Context Map trong DDD, nhưng tập trung hơn vào các năng lực (*capabilities*) thay vì *language boundaries* (ranh giới ngôn ngữ).
4. **Xác định service boundaries + composition** — Kiểm tra từng service candidate: nó có thể hoạt động **độc lập** không? Nếu dịch vụ A *luôn* cần gọi dịch vụ B trước khi có thể xử lý bất kỳ yêu cầu (request) nào → cân nhắc việc gộp chung. Nếu dịch vụ A vẫn có thể hoạt động độc lập ngay cả khi dịch vụ B gặp sự cố (ngừng hoạt động) → ranh giới tốt.
5. **Kiểm tra theo 8 nguyên lý SOA** — Mỗi service phải thỏa mãn 8 nguyên lý hướng dịch vụ đã thảo luận ở Bảng 1.2 (Chương 1): Standardized Contract, Loose Coupling, Abstraction, Reusability, Autonomy, Statelessness, Discoverability, Composability. Đây là "checklist kiểm tra chất lượng" cho service design.

Áp dụng năm bước trên cho KBM cho kết quả như Hình 2.7, đi từ các automation candidate đến các lớp dịch vụ (service layer).



Hình 2.7: *Áp dụng Erl's Service-Oriented Analysis cho KBM — từ automation candidate đến service layer*

Kết quả ở bước minh họa này cho ra 5 nhóm dịch vụ tiêu biểu — trùng khớp với năm Bounded Context tương ứng trong phân tích DDD (§2.4): Identity & Access ≈ Utility Auth, SQL Practice ≈ Entity Question + Task Orchestrator, Network Practice ≈ subsystem giao thức chuyên biệt (khái niệm *complicated-subsystem* mượn từ Team Topologies — nằm ngoài taxonomy Erl), Academic Management ≈ Entity Course, và DevOps Practice ≈ Task Service + Infrastructure Automation. Hai Bounded Context core còn lại trong tổng số bảy context của KBM — Code Practice (tích hợp DOMjudge thay cho bộ chấm MVC thế hệ đầu) và Examination & Proctoring (hiện điều phối cùng `kbm-core`) — được trình bày đầy đủ ở §2.4.

So sánh: Erl vs DDD

Bảng 2.4 đặt hai phương pháp cạnh nhau theo từng tiêu chí.

Bảng 2.4: *Hai phương pháp phân tách service — Erl Service-Oriented Analysis vs DDD*

Tiêu chí	Erl Service-Oriented Analysis	DDD Bounded Context
Điểm bắt đầu	Quy trình nghiệp vụ (business process)	Ngôn ngữ domain (Ubiquitous Language)
Phong cách	Theo quy trình — 5 bước rõ ràng, input/output mỗi bước	Khám phá — heuristics, Event Storming workshop
Tiêu chí tách	Service layers (task/entity/utility) + 8 nguyên lý SOA	Ranh giới ngôn ngữ + aggregate consistency
Output	Service inventory blueprint	Context Map + Bounded Context diagram
Thế mạnh	Có hệ thống, dễ dạy, hợp nhóm mới	Sâu về domain, phát hiện ranh giới ẩn

Tiêu chí	Erl Service-Oriented Analysis	DDD Bounded Context
Hạn chế	Có thể tạo ranh giới quá kỹ thuật (thiếu domain insight)	Cần kinh nghiệm, kết quả phụ thuộc người phân tích
Phù hợp khi	Team ít kinh nghiệm DDD, cần quy trình chuẩn	Team hiểu domain sâu, cần phát hiện complexity ẩn

Dù tiếp cận theo cách nào, mục tiêu cuối cùng vẫn là tìm ra ranh giới hợp lý nhất cho hệ thống. Thay vì coi đây là hai trường phái đối lập, hãy xem chúng là các công cụ hỗ trợ lẫn nhau.

Bổ trợ, không thay thế

Erl cung cấp *quy trình* (process) — đặc biệt hữu ích khi chúng ta chưa biết bắt đầu từ đâu. DDD cung cấp *insight* — phát hiện những ranh giới mà quy trình cơ học bỏ sót. Trong thực tế: dùng Erl để tạo bản phác thảo ban đầu (service inventory), sau đó dùng DDD (Event Storming, ngôn ngữ chung) để *tinh chỉnh* ranh giới. Richardson's Assemblage Process (§2.7) kết hợp cả hai hướng.

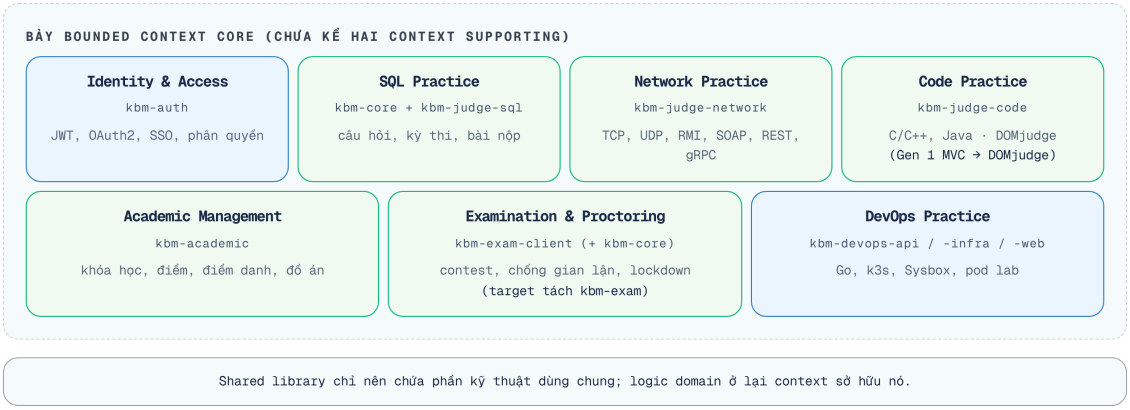
2.4. Case study: KBM

Trở lại với bài toán hệ thống LMS của trường đại học, chúng ta không thể giao toàn bộ việc quản lý điểm số, bài tập và kỳ thi cho cùng một bộ phận. Việc chia nhỏ thành các Bounded Context giúp định hình rõ quyền hạn và trách nhiệm của từng phòng ban.

2.4.1. Phân tích domain

KBM phục vụ các hoạt động giảng dạy, thực hành, đánh giá và vận hành lab thực hành. Từ góc nhìn DDD, chúng ta phân tách Problem Space thành các Subdomain: **SQL Practice** là Core Domain (cốt lõi tạo ra giá trị); **Identity & Access** là Generic Subdomain (đã có chuẩn chung ngoài thị trường); các mảng **Network Practice**, **Code Practice**, **Academic Management**, **Examination & Proctoring**, **DevOps Practice** và **Communication** (thông báo, email) là các Supporting Subdomain (cần thiết để hệ thống vận hành trọn vẹn nhưng không tạo lợi thế cạnh tranh cốt lõi). Còn các service như Gateway (`kbm-gateway`), Registry (`kbm-registry`) và thư viện dùng chung (`kbm-shared`) thuộc lớp **Platform / Infrastructure** (hạ tầng kỹ thuật), cần phân biệt rõ chúng với Subdomain nghiệp vụ.

Bước tiếp theo là chuyển sang *không gian giải pháp*: bảy Subdomain trực tiếp phục vụ bài toán giảng dạy và thi cử được hiện thực hóa thành **bảy Bounded Context core**, kết hợp với **hai supporting context** lo phần nền tảng dùng chung cho cả hệ thống (**Communication** — `kbm-notification`, và **Platform / Infrastructure** — `kbm-gateway`, `kbm-registry`, `kbm-shared`), tạo thành chín context trong bản đồ kiến trúc KBM. Cần đọc cặp "core / supporting" ở đây đúng tầng: nó nói về *vai trò kiến trúc* của context (nghiệp vụ đối lập với nền tảng dùng chung), không trùng với `Core Domain` hay `Supporting Subdomain` ở không gian bài toán vừa nhắc ở trên. Theo Evans, một hệ thống có thể có nhiều context core nhưng chỉ nên có *một* Core Domain — với KBM, đó là SQL Practice.



Hình 2.8: Bảy Bounded Context core của KBM

2.4.2. Bảy Bounded Context core

1. Identity & Access (`kbm-auth`)

Domain rõ ràng nhất — quản lý danh tính người dùng, xác thực, phân quyền. Entities: `User`, `Role`, `Token`. Đây là context upstream — mọi context khác đều phụ thuộc vào kết quả authentication.

Trong KBM, context này được tách thành service riêng sớm nhất (`kbm-auth`), vì ranh giới rất rõ: thay đổi cơ chế xác thực (tài khoản nội bộ, Google/Microsoft OAuth2, email verification, SSO từ hệ thống đào tạo) không ảnh hưởng logic bài tập hay chấm thi.

2. SQL Practice (`kbm-core` + `kbm-judge-sql` — *Core Domain*)

Đây là **Core Domain** ban đầu — phần tạo ra giá trị chính cho hệ thống SQL Judge. Entities: `Question`, `Submission`, `Contest`, `UserContest`, `Statistic`. Toàn bộ quy trình từ "sinh viên xem câu hỏi → viết SQL → nộp bài → nhận kết quả" nằm trong context này.

Đây là context lớn nhất và phức tạp nhất. Bên trong nó có kiến trúc chấm SQL gồm một service điều phối `kbm-judge-sql` và các worker `kbm-sandbox-*` theo từng DBMS (`kbm-sandbox-mysql`, `kbm-sandbox-postgres`, `kbm-sandbox-sqlserver`, ...). Đây mới là nơi cần phân tích kỹ về các vấn đề định tuyến (routing), khả năng phục hồi (resilience), idempotency và quyền sở hữu (ownership) giữa thành phần điều phối (coordinator) và thành phần xử lý (worker); không nên nhầm với Network Practice.

3. Network Practice (`kbm-judge-network`)

Domain chuyên biệt cho bài thực hành lập trình mạng Java: TCP, UDP, RMI, SOAP, REST và gRPC. Khác với SQL Practice, sinh viên viết client trên IDE cá nhân và kết nối trực tiếp đến judge server. Context này minh họa ranh giới domain đến từ **protocol semantics**, không chỉ từ entity/database.

Network Practice được thiết kế tương đối độc lập với các service còn lại: nó phục vụ mục tiêu đánh giá giao tiếp qua mạng, có protocol contract riêng và không cần chia sẻ pipeline chấm SQL. Trong sách, context này chủ yếu minh họa **technology diversity** và boundary theo protocol, không phải một khoảng trống kiến trúc cần sửa như SQL Judge.

4. Academic Management (`kbm-academic`)

Quản lý khóa học, bài tập dạng rubric, đăng ký môn, điểm số, điểm danh, MCQ daily, syllabus/CLO, GitHub Classroom và đồ án tốt nghiệp. Entities: `Course`, `Assignment`, `Enrollment`, `Grade`, `AttendanceSession`, `Thesis`.

Context này được tách ra từ Core Service — nhưng *vẫn sử dụng chung cơ sở dữ liệu* (`core_db`). Đây là trade-off quan trọng mà chúng ta sẽ phân tích trong phần tiếp theo.

5. DevOps Practice (`kbm-devops-api`, `kbm-devops-infra`, `kbm-devops-web`)

Domain mới phục vụ bài thực hành Docker/Kubernetes: tạo lab environment cô lập, terminal web, validator script và progress tracking. Context này dùng Go, k3s và Sysbox, sử dụng stack công nghệ khác với LMS chính. Đây là ví dụ tốt cho polyglot microservices: dùng công nghệ khác khi domain và hạ tầng yêu cầu, nhưng vẫn chia sẻ danh tính (identity) qua JWT của KBM.

6. Code Practice (`kbm-judge-code`)

Domain chấm bài lập trình thuật toán đa ngôn ngữ (C/C++, Java, Python...) theo mô hình ICPC: sinh viên nộp mã nguồn, hệ thống biên dịch, chạy với bộ test, đối chiếu đầu ra (output) và trả về kết quả (Verdict). Trong giai đoạn đầu, hệ thống sử dụng một bộ chấm điểm *tự phát triển nội bộ* (chỉ C/C++ và Java, không sandbox); về sau được thay bằng `kbm-judge-code` tích hợp **DOMjudge** (hệ thống chấm bài trực tuyến mã nguồn mở chuẩn thi đấu). Context này tách khỏi SQL Practice vì *mô hình chấm khác hẳn*: SQL so sánh result set trên CSDL thật, còn Code Practice biên dịch–chạy–so sánh kết quả đầu ra (output) trong sandbox cô lập. Đây cũng là minh họa kinh điển cho quyết định **"xây hay mua" (build vs buy)** (sẽ phân tích ở Chương 4 và Chương 10).

7. Examination & Proctoring (*logic Contest/Anti-cheat hiện trong* `kbm-core`*, mục tiêu tách* `kbm-exam`*;* `kbm-exam-client`)

Domain thi cử và giám sát: vòng đời Contest (Scheduled → Open → Closed), bảng xếp hạng thời gian thực, chống gian lận (chuyển thẻ trình duyệt (tab), sao chép-dán (copy-paste), đổi IP/User-Agent) và đặc biệt là kỳ thi **trên phòng máy** qua ứng dụng desktop `kbm-exam-client` (lockdown toàn màn hình, whitelist IP phòng thi, giám sát từ phía máy khách (client-side proctoring)). Về mặt nghiệp vụ đây là một Bounded Context riêng, dù *hiện tại* phần điều phối thi vẫn nằm chung trong `kbm-core` (một biểu hiện của siêu dịch vụ ôm đồm — god-service) — mục tiêu chuyển đổi (migration target) hệ thống là phân tách thành `kbm-exam`. Context này minh họa nguyên tắc **một backend phục vụ nhiều loại client** (web luyện tập + desktop thi) và **phòng thủ chiều sâu (defense in depth)** cho chống gian lận (Ch.9).

Từ Bounded Context đến service

Bảng 2.5 ánh xạ bảy Bounded Context core sang các service triển khai, kèm lý do tách của từng context.

Bảng 2.5: *Ánh xạ Bounded Context → service trong KBM*

Bounded Context	Service	Lý do tách
Identity & Access	`kbm-auth`	Ranh giới rõ ràng, thay đổi auth ≠ thay đổi logic học tập
SQL Practice	`kbm-core`, `kbm-judge-sql`, sandbox DB services	Core Domain, cần sandbox isolation và chấm tự động
Network Practice	`kbm-judge-network`	Protocol đa dạng, state/routing khác REST API thông thường
Code Practice	`kbm-judge-code`	Mô hình chấm compile–run–compare khác SQL; tích hợp DOMjudge ("xây hay mua" — build vs buy)
Academic Management	`kbm-academic`	Domain học vụ, grade scheme, attendance, thesis
Examination & Proctoring	`kbm-exam-client` (+ logic trong `kbm-core`, target `kbm-exam`)	Thi cử, lockdown phòng máy, anti-cheat; nhiều loại client một backend
DevOps Practice	`kbm-devops-api`, `kbm-devops-infra`, `kbm-devops-web`	Domain lab infrastructure, Go/k3s/Sysbox, validator script

Khi đối chiếu bảng phân rã này, một điểm quan trọng cần lưu ý là mối quan hệ giữa ranh giới nghiệp vụ và cấu trúc triển khai vật lý.

Bounded Context ≠ service

Bounded Context là ranh giới logic. Một ngữ cảnh có thể được hiện thực hóa (implement) bởi nhiều dịch vụ (như SQL Practice = Core + Judge + sandbox DB services). Ngược lại, một kho mã nguồn (repository) cũng có thể tạo ra nhiều tệp thực thi (binary) phục vụ cùng context (như DevOps Practice = API + router + grader trong một codebase Go) — miễn là ranh giới domain rõ ràng.

2.5. Shared Library và Shared Kernel — chia sẻ hay không chia sẻ?

Trong môi trường đại học, có những thông tin bắt buộc phải dùng chung như mã sinh viên hay quy chế đào tạo cốt lõi. Tương tự, **Shared Kernel** là phần *mô hình miền* mà nhiều Bounded Context buộc phải đồng thuận chia sẻ, dù điều này làm giảm tính độc lập — khác với **Shared Library**, nơi chỉ gom mã hạ tầng kỹ thuật dùng chung.

2.5.1. Vấn đề thực tế

Trong KBM, tồn tại một **shared library** chứa 23 tệp Java: các lớp thực thể cơ sở (base entity), DTO, tiện ích JWT, xử lý ngoại lệ và tiện ích kiểm tra hợp lệ (validation). Nhiều service Java phụ thuộc vào library này.

Đúng tên gọi, `kbm-shared` là một **Shared Library**: nó gom mã nguồn hạ tầng dùng chung cho nhiều dịch vụ. Kiểu chia sẻ này giảm trùng lặp nhưng sinh ra phụ thuộc — mọi thay đổi trong thư viện đều *có thể* ảnh hưởng đến tất cả dịch vụ đang dùng nó.

Shared Kernel trong DDD là khái niệm khác hẳn. Thuật ngữ này chỉ dành cho phần **Domain Model** mà nhiều Bounded Context cố ý đồng sở hữu và cùng quản trị — chẳng hạn value object `StudentId` hay thang điểm `GradingScale` mà cả Academic Management lẫn Examination đều phải tuân theo. Vì vậy KBM chấp nhận Shared Library cho hạ tầng, nhưng hạn chế tối đa Shared Kernel trong domain.

Cấu trúc thư mục hiện tại của `kbm-shared`:

```
kbm-shared/
├── common/         BaseEntity, BaseMapper, BaseRequest, BaseResponse, BaseService
├── config/         AuditConfig, CorsConfig, TimeZoneConfig
├── enumerate/      AnswerStatus, ErrorCode, TypeQuestion
├── exception/      GlobalExceptionHandler
├── securityConfig/ JwtConfig, UserDetailCustom
└── utils/          CompareUtil, JwtUtil, SqlUtil, TableDependencyResolver
```

Phân tích: cái gì nên shared, cái gì không?

Bảng 2.6 phân loại nội dung của `kbm-shared` theo câu hỏi có nên dùng chung hay không.

__Bảng 2.6:__ Phân tích shared library — cái gì nên dùng chung, cái gì không

Loại	Ví dụ	Nên dùng chung?	Lý do
Cross-cutting concerns	JWT validation, exception handling, CORS config	✓ Có	Nhất quán hành vi giữa các dịch vụ, ít thay đổi

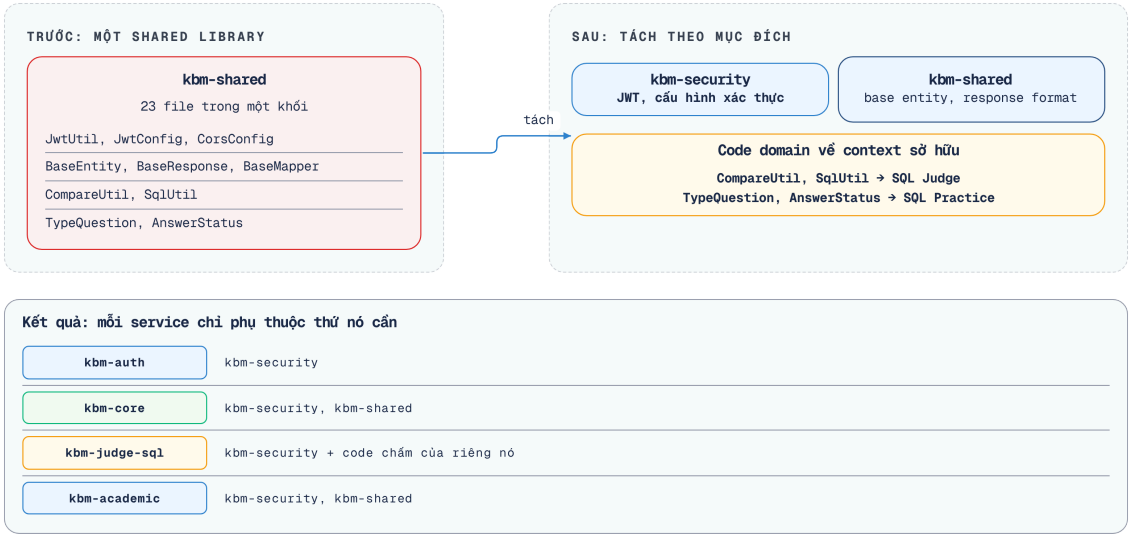
Loại	Ví dụ	Nên dùng chung?	Lý do
Base abstractions	BaseEntity, BaseMapper, BaseResponse	! Cân nhắc	Mang lại sự tiện lợi nhưng tạo ra sự phụ thuộc (coupling) cứng nhắc vào các quy ước chung (convention)
Domain-specific logic	`CompareUtil` (SQL comparison), `SqlUtil`	× Không	Chỉ Judge context cần — không phải shared concern
Domain enums	`TypeQuestion`, `AnswerStatus`	× Không	Thuộc về SQL Practice context, không universal

2.5.2. Hướng cải thiện

Nếu team phát triển lớn hơn và cần giảm coupling, shared library có thể được tách thành:

1. `kbm-security` — JWT, cấu hình xác thực (mọi dịch vụ đều cần)
2. `kbm-shared` — Base entities, response format (convention)
3. **Domain-specific code** → chuyển vào service tương ứng (thuộc SQL Practice, Code Practice, v.v.)

Sơ đồ dưới đây minh họa hướng tách này — mục tiêu là giảm phụ thuộc không cần thiết giữa các module:



Hình 2.9: Hướng tách shared library — từ một khối sang nhiều module

Chia sẻ mã nguồn là thỏa thuận có chủ đích

Dù là Shared Library hay Shared Kernel, việc chia sẻ đều phải là *thỏa thuận có chủ đích*, không phải sự tiện lợi. Mỗi file trong thư viện dùng chung phải trả lời được: "Thay đổi file này ảnh hưởng tới những service nào, và các service đó có *buộc phải* cập nhật cùng lúc không?" File chỉ phục vụ nghiệp vụ của một context thì thuộc về context đó; mã hạ tầng ổn định có thể dùng chung, miễn các service nâng phiên bản độc lập được.

Hai loại chia sẻ có tiêu chí khác nhau: **Shared Library** chỉ nên giữ mã hạ tầng cross-cutting đã ổn định (JWT, CORS, base response); **Shared Kernel** chỉ dành cho phần Domain Model mà nhiều Bounded Context cố ý đồng sở hữu và cùng quản trị — loại này càng ít càng tốt.

Đối chiếu nguyên tắc này với `kbm-shared` hiện tại cho thấy những phần cần tách:

Shared library cần tách

Với 23 tệp trong `kbm-shared`, KBM đang tạo ra sự ràng buộc (coupling) không cần thiết: mã nguồn đặc thù miền nghiệp vụ (domain-specific code) như `CompareUtil` nằm chung với các nghiệp vụ cắt ngang (cross-cutting concern) như JWT hay `GlobalExceptionHandler`. Theo nguyên lý tách biệt thiết kế trong [2a, Ch.2; 2b, Ch.4], chỉ các phần hạ tầng thực sự ổn định và dùng chung mới nên nằm trong thư viện nội bộ, còn hợp đồng API và định nghĩa sự kiện (event schema) cần trao đổi qua đặc tả độc lập ngôn ngữ (OpenAPI, JSON Schema — Chương 3, Chương 5) chứ không đóng gói chung vào một shared library dùng chéo giữa mọi dịch vụ. **Lộ trình chuyển đổi**:

1. Tách domain code về service riêng.
2. Chỉ giữ security và base response trong thư viện dùng chung.
3. Đưa domain enum (`TypeQuestion`, `AnswerStatus`) về context sở hữu.

Tuy nhiên, cần tránh những lầm tưởng kinh điển sau:

Sai lầm thường gặp

1. **Phân rã theo schema database** — Thiết kế service dựa trên các bảng CSDL thay vì nghiệp vụ. Hậu quả: tạo ra các CRUD API mỏng. *Phòng tránh*: Bắt đầu từ domain process (Event Storming §2.3), không từ schema.
2. **Chia quá nhỏ (nano-services)** — Tách mỗi entity thành 1 service riêng (UserService, RoleService, TokenService) khi chúng luôn thay đổi cùng nhau. Hậu quả: mỗi yêu cầu (request) cần gọi chéo 3–4 dịch vụ, dẫn đến độ trễ (latency) tăng cao và quá trình gỡ lỗi (debugging) trở nên phức tạp. *Phòng tránh*: tách theo Bounded Context, không theo entity. Nếu hai entity luôn thay đổi cùng nhau, chúng thuộc cùng service.
3. **Chia sẻ mã nguồn vì tiện, không vì thiết kế** — Đưa toàn bộ mã nguồn chung vào thư viện dùng chung mà không phân biệt cross-cutting concern và domain logic. Hậu quả: mỗi thay đổi trong thư viện dùng chung sẽ ảnh hưởng đến tất cả các dịch vụ, tạo ra sự phụ thuộc ngầm khi triển khai (deployment coupling). *Phòng tránh*: mỗi file trong thư viện dùng chung phải có phạm vi sử dụng và người chịu trách nhiệm rõ ràng, và các service dùng nó phải nâng phiên bản độc lập được; domain logic thì trả về context sở hữu (§2.5).

2.6. Dark Energy & Dark Matter — 10 lực chi phối việc phân tách service

Phần nâng cao (có thể lướt qua)

Người đọc mới có thể đọc lướt mục này ở lần đầu. Cốt lõi cần nhớ là cặp "lực đẩy / lực hút"; chi tiết 10 lực phát huy tác dụng khi thực sự phải cân nhắc tách hoặc gộp một service cụ thể.

Richardson trong phiên bản thứ hai (2025) giới thiệu một framework phân tích trade-off cho việc service decomposition, mượn ẩn dụ từ vật lý thiên văn:

- **Dark Energy — lực đẩy (repulsive)**: kéo các thành phần (component) ra xa nhau → nên phân tách thành dịch vụ riêng biệt.
- **Dark Matter — lực hút (attractive)**: kéo các thành phần lại gần nhau → nên giữ cùng service.

Bảng 2.7 liệt kê mười lực cụ thể và cách mỗi lực thể hiện trong KBM.

Bảng 2.7: *Mười lực Dark Energy / Dark Matter*

Lực	Loại	Mô tả	Ví dụ KBM
Simple interactions	Matter (hút)	Các thành phần tương tác phức tạp → giữ cùng service	`ContestQuestion` ↔ `Contest` luôn query chung → hợp lý ở cùng `kbm-core`
Efficient interactions	Matter (hút)	Yêu cầu độ trễ (latency) thấp → việc gọi hàm nội bộ (in-process) luôn nhanh hơn gọi qua mạng	Submission → Score cập nhật cần nhanh → hiện cùng service
Prefer ACID	Matter (hút)	Cần ACID transaction → cùng DB dễ hơn saga	Create contest + add questions = 1 transaction → cùng service
Minimize runtime coupling	Matter (hút)	Giảm thiểu sự phụ thuộc trong thời gian chạy (runtime dependency) dẫn đến giảm số lượng lệnh gọi qua mạng	Nếu tách phần tính điểm khỏi Core, mỗi lần nộp bài phải chờ thêm một lời gọi mạng tới service đó
Minimize design-time coupling	Matter (hút)	Giảm phụ thuộc lúc thiết kế (API/ schema chung): nếu hai bên tách ra mà vẫn phải sửa cùng nhau, coupling chỉ đổi hình thức	`kbm-shared` buộc các service Java rebuild đồng loạt — một dạng design-time coupling cần thu hẹp (§2.5)
Simple components	Energy (đẩy)	Mỗi service nhỏ, dễ hiểu	Mỗi service một codebase riêng, chỉ chứa code của context mình — dễ nắm hơn một khối gộp tất cả
Team autonomy	Energy (đẩy)	Các nhóm triển khai (deploy) độc lập, không cần phối hợp chéo (coordination)	Identity & Access và SQL Practice có thể triển khai độc lập; khi có nhiều nhóm, mỗi nhóm không phải chờ nhau
Fast deployment pipeline	Energy (đẩy)	Dịch vụ nhỏ gọn giúp quá trình biên dịch (build) và kiểm thử (test) diễn ra nhanh hơn	Build một service nhỏ nhanh hơn nhiều so với build cả khối
Support multiple tech stacks	Energy (đẩy)	Các dịch vụ khác nhau có thể sử dụng nền tảng công nghệ (tech stack) khác nhau	DevOps Practice viết bằng Go, phần còn lại của LMS bằng Java (§2.4)
Segregate by characteristics	Energy (đẩy)	Tách các phần có đặc tính vận hành khác biệt: nhu cầu mở rộng, tính sẵn sàng, yêu cầu bảo mật, dữ liệu sở hữu riêng…	Các dịch vụ Judge cần thêm tài nguyên khi có kỳ thi và cần sandbox bảo mật riêng — khác hẳn Identity & Access

Cách sử dụng: Khi phân vân "có nên tách component X thành service riêng?", liệt kê các lực tác động. Nếu Dark Energy (đẩy) chiếm ưu thế → tách. Nếu Dark Matter (hút) mạnh hơn → giữ chung. Đây không phải công thức chính xác mà là **framework tư duy** giúp cấu trúc cuộc thảo luận.

Dark Energy/Matter cho quyết định tách SQL Practice & Academic Management

KBM hiện có SQL Practice (`kbm-core`) và Academic Management (`kbm-academic`) dùng chung cơ sở dữ liệu. Phân tích:

- **Dark Matter (giữ chung)**: cơ sở dữ liệu dùng chung giúp đảm bảo ACID dễ dàng, bảng `assignment_questions` tham chiếu tới `questions.id` (tương tác đơn giản), nhóm phát triển rất nhỏ (1–2 lập trình viên chính)
- **Dark Energy (tách)**: domain logic khác biệt (bài tập vs kỳ thi), data ownership rõ ràng, tiềm năng triển khai độc lập

Kết luận: Dark Matter hiện mạnh hơn (do team nhỏ, shared DB). Trạng thái cân bằng sẽ dịch chuyển khi các tín hiệu quan sát được xuất hiện: nhóm phát triển tách thành nhiều team có ownership riêng, các thay đổi lockstep bắt đầu làm nghẽn pipeline, hoặc nhu cầu scale của hai domain khác hẳn nhau → lúc đó việc phân tách cơ sở dữ liệu và dịch vụ sẽ trở nên hợp lý hơn.

Đây chính là bài học thực tế của nguyên tắc không phân tách quá sớm (*premature decomposition* [4a, Ch.3; 4b, Ch.2]) khi các lực hút Dark Matter vẫn chiếm ưu thế (Richardson [2b, Ch.7]).

2.7. Assemblage Process — quy trình thiết kế service architecture

Phần nâng cao (có thể lướt qua)

Mục này tổng hợp các kỹ thuật phía trên thành một quy trình thiết kế hoàn chỉnh. Người đọc mới nên nắm chắc Bounded Context và Context Map trước; có thể quay lại quy trình tổng hợp này khi bắt tay thiết kế một hệ thống thật.

Richardson trong [2b, Ch.20] trình bày **Assemblage Process** — quy trình step-by-step tổng hợp các kỹ thuật đã học (Bounded Context, Context Map, Dark Energy/Matter) thành chuỗi bước đi từ yêu cầu nghiệp vụ đến kiến trúc microservices cụ thể:

1. **Khám phá System Operations** — Liệt kê các thao tác chính mà hệ thống phải hỗ trợ (ví dụ: `submitAnswer()`, `createContest()`, `gradeSubmission()`).
2. **Xác định Subdomains** — Phân vùng *không gian bài toán* thành các subdomain từ nghiệp vụ và system operations. Với KBM, các subdomain này về sau được ánh xạ thành bảy Bounded Context core (Identity & Access, SQL Practice, Network Practice, Code Practice, Academic Management, Examination & Proctoring, DevOps Practice) cùng các supporting context (Communication, Platform) trong không gian giải pháp (§2.4).
3. **Thiết kế Services & Collaborations** — Gán mỗi operation vào subdomain sở hữu logic của nó, nhóm các subdomain thành service bằng cách cân 10 lực Dark Energy/Dark Matter, rồi chọn mô hình giao tiếp giữa các service: đồng bộ REST/gRPC (Ch.4) hay bất đồng bộ Kafka (Ch.5).
4. **Đánh giá kiến trúc** — Đối chiếu kết quả với các quality attribute scenario (Ch.1): kiến trúc có đạt deployability, testability, các yêu cầu scale đã đặt ra không?
5. **Tái cấu trúc** — Quy trình không chạy một lần. Khi kiến thức domain sâu hơn hoặc phép đánh giá phát hiện lệch, ranh giới service được điều chỉnh và vòng lặp tiếp tục.

Assemblage process biến quá trình thiết kế kiến trúc — vốn mang tính nghệ thuật và chủ quan — thành chuỗi các quyết định kỹ thuật có thể giải thích được. Thay vì nói "tách Judge Service vì có vẻ hợp lý", chúng ta có thể lập luận: *"Judge Service được tách vì các lực đẩy Dark Energy (Segregate by characteristics — cần cô lập sandbox và mở rộng độc lập khi thi, cùng Simple components và Fast deployment pipeline) vượt trội so với lực hút Dark Matter (Efficient interactions) trong ngữ cảnh này."*

Các pattern trong chương có minh họa tương tác đi kèm tại companion web:

Interactive Demo

- **Bản đồ Bounded Context (Context Map)**: context-map.html

2.8. Tổng kết

Quy trình phân tách hệ thống trải dài từ cấp vĩ mô (Conway's Law) đến vi mô (Entity, Aggregate, Repository).

Conway's Law nhắc nhở rằng kiến trúc phần mềm phản ánh cấu trúc tổ chức và chịu ràng buộc từ cấu trúc đó. Inverse Conway Maneuver cho phép chúng ta chủ động thiết kế team để đạt kiến trúc mong muốn.

Chúng ta có hai phương pháp bổ trợ để xác định ranh giới service. **DDD** cung cấp ngôn ngữ và heuristics — Bounded Context, Context Map, Event Storming — phù hợp khi team hiểu domain sâu. **Erl's Service-Oriented Analysis** cung cấp quy trình step-by-step — từ automation candidates đến service layers — phù hợp khi team cần một framework có cấu trúc. Cả hai đều dẫn đến ranh giới tương đồng (như minh họa với KBM qua năm nhóm dịch vụ tiêu biểu ở bước tổng hợp của Erl), nhưng xuất phát từ góc nhìn khác nhau. Dark Energy/Dark Matter forces làm tường minh các yếu tố ảnh hưởng đến quyết định tách/gộp, và Assemblage Process (§2.7) tổng hợp tất cả thành quy trình thiết kế có hệ thống.

Phân tích KBM cho thấy 7 Bounded Context core rõ ràng (Identity & Access, SQL Practice, Network Practice, Code Practice, Academic Management, Examination & Proctoring, DevOps Practice) cùng 2 supporting context (Communication và Platform / Infrastructure), mỗi context tương ứng với một nhóm service hoặc binary (dù về mặt triển khai vật lý, phần điều phối thi của Examination & Proctoring hiện vẫn nằm trong `kbm-core` với mục tiêu tách thành `kbm-exam`). Thư viện dùng chung của KBM — dù tiện lợi cho team nhỏ — cần được quản lý có chủ đích, phân biệt rõ giữa cross-cutting concern của Shared Library (nên share) và domain logic (chỉ share qua Shared Kernel khi các context thực sự đồng sở hữu).

2.9. Đọc thêm

Sách tham khảo chính:

1. [1] Thomas Erl, *Service-Oriented Architecture: Analysis and Design for Services and Microservices*, 2nd Ed. — Service-Oriented Analysis, Service Layers (task/entity/utility), Service Inventory Blueprint
2. [6] Eric Evans, *Domain-Driven Design* — Ch.1–5: Knowledge Crunching, Ubiquitous Language, Building Blocks; Ch.14: Bounded Context, Context Map
3. [2a] Chris Richardson, *Microservices Patterns*, 1st Ed. — Ch.2: Decomposition Strategies
4. [2b] Chris Richardson, *Microservices Patterns*, 2nd Ed. — Ch.4: Loose Coupling, Dark Energy/Dark Matter Forces; Ch.7: Microservice Architecture; Ch.20: Assemblage Process
5. [4a] Sam Newman, *Building Microservices* — Ch.3: How to Model Services; Ch.10: Conway's Law
6. [4b] Sam Newman, *Monolith to Microservices* — Ch.2: Planning a Migration, Decomposition by Domain
7. [5] Hugo Rocha, *Practical Event-Driven Microservices Architecture* — Ch.3: Service Boundaries, DDD/Bounded Context

Sách bổ trợ:

8. Vaughn Vernon, *Implementing Domain-Driven Design* (2013) — Ch.2: Strategic Design with Bounded Contexts
9. Matthew Skelton & Manuel Pais, *Team Topologies* (2019) — Four Team Types, Cognitive Load, Conway's Law
10. Alberto Brandolini, *Introducing EventStorming* (2013, Leanpub, cập nhật liên tục) — Domain discovery workshops

Nguồn trực tuyến:

- MacCormack, Rusnak & Baldwin, *"Exploring the Duality between Product and Organizational Architectures"* (2008) — Harvard Business School Working Paper

Bài tập: 2-0, 2-1, 2-2 ở phần Bài tập & case study cuối sách.

CHƯƠNG 3

Thiết kế dịch vụ & API

> "Public APIs, like diamonds, are forever. You have one chance to get it right so give it your best."
> — Joshua Bloch, "How to Design a Good API and Why it Matters" (2006)

MỤC TIÊU HỌC TẬP

- Nắm vững nguyên tắc thiết kế REST API cho microservices
- Hiểu các chiến lược API versioning và schema evolution
- Sử dụng OpenAPI/Swagger để tạo tài liệu API tự động
- Áp dụng DTO pattern và Hexagonal Architecture (Ports & Adapters) để tách biệt lõi nghiệp vụ khỏi hợp đồng và hạ tầng bên ngoài
- Hiểu Contract Testing (Pact, Spring Cloud Contract) để ngăn breaking change
- So sánh REST và GraphQL, nhận diện bài toán phù hợp cho từng công nghệ
- Phân tích bài học thực tế từ API design trong KBM

3.1. REST API Design — nguyên tắc cho Microservices

Hãy tưởng tượng REST API như những mẫu đơn chuẩn hóa của phòng đào tạo. Nếu mỗi khoa yêu cầu một biểu mẫu khác nhau, sinh viên sẽ gặp khó khăn. Trong hệ thống phân tán, một thiết kế API chuẩn mực là yêu cầu tiên quyết để các dịch vụ giao tiếp.

3.1.1. Tại sao API design quan trọng hơn trong microservices?

Trong monolith, giao tiếp giữa các module là gọi hàm nội bộ (in-process call) — nếu interface thay đổi, trình biên dịch (compiler) sẽ báo lỗi ngay. Trong microservices, giao tiếp diễn ra qua mạng (HTTP, gRPC, messaging) — và **trình biên dịch của từng service không tự phát hiện được thay đổi phá vỡ hợp đồng với các service triển khai độc lập**. Một breaking change trong API có thể gây gián đoạn cho các dịch vụ khác mà không ai hay biết, cho đến khi hệ thống trên production gặp sự cố.

Do đó, API trong microservices cần được thiết kế **cẩn thận hơn**, **ổn định hơn**, và **có chiến lược evolution rõ ràng** — đây không phải tùy chọn, mà là điều kiện tiên quyết.

Richardson Maturity Model. Leonard Richardson[1] đề xuất mô hình 4 cấp độ trưởng thành cho REST API [2a, Ch.3]:

Hình 3.1: Richardson Maturity Model — bốn cấp độ trưởng thành của REST API

Trong bốn cấp độ này, hầu hết microservices thực tế (bao gồm KBM LMS) dừng lại ở **Level 2** — sử dụng đúng resource và HTTP verb, chưa cần đến hypermedia.

Level 3 (HATEOAS) — Khi nào cần? HATEOAS (*Hypermedia as the Engine of Application State*) yêu cầu phản hồi (response) chứa các liên kết (link) điều hướng — máy khách không cần biết trước cấu trúc URL, chỉ cần đi theo các liên kết từ phản hồi. Ví dụ: `GET /questions/123` trả về `{..., "_links": {"submissions": "/questions/123/submissions", "edit": "/questions/123"}}`. Ưu điểm: API có khả năng tự mô tả (self-documenting), máy khách (client) không phụ thuộc vào cấu trúc URL. Nhược điểm: làm tăng payload size, làm phức tạp response format, hầu hết frontend framework (React, Angular) không sử dụng tối đa HATEOAS. Trong thực tế, HATEOAS hiếm khi được áp dụng cho các API nội bộ (internal API) [4a, Ch.4] — phù hợp hơn cho các API công khai (public API) khi provider không kiểm soát consumer — nhưng ngay cả GitHub API v3 cũng sử dụng cơ chế đánh

[1] Cần lưu ý phân biệt Leonard Richardson (người đề xuất mô hình này) với Chris Richardson (tác giả của cuốn sách "Microservices Patterns" được trích dẫn nhiều trong tài liệu này).

phiên bản tường minh (explicit versioning) thay vì HATEOAS thuần túy. HATEOAS vẫn là ngoại lệ trong thực tế, không phải chuẩn phổ biến.

3.1.2. Nguyên tắc thiết kế REST API

1. Resource-oriented URLs — URL đại diện cho resource (danh từ), không phải hành động (động từ):

```
ĐÚNG: GET  /questions/{id}          → Lấy câu hỏi
ĐÚNG: POST /questions               → Tạo câu hỏi mới
ĐÚNG: GET  /contests/{id}/questions  → Câu hỏi trong kỳ thi

SAI:  GET  /getQuestion?id=123      → Dùng verb, query string
SAI:  POST /createQuestion           → URL chứa action
```

Tên phòng ban và danh từ

Hãy tưởng tượng hệ thống biển báo trong trường đại học. Chúng ta sẽ thấy biển báo chỉ đường đến "Phòng Đào tạo", "Thư viện" (danh từ — resource) chứ không phải biển báo "Đăng ký môn học", "Mượn sách" (động từ — action). REST API cũng vậy: URL nên định danh "nơi chứa tài nguyên" thay vì "hành động sẽ thực hiện".

2. HTTP methods đúng ngữ nghĩa:

Các phương thức HTTP cốt lõi và ý nghĩa tương ứng bao gồm:

- `GET`: Được sử dụng để đọc hoặc lấy thông tin tài nguyên (ví dụ: `GET /questions/123`). Theo RFC 9110, `GET` vừa mang tính **safe** (chỉ đọc, không làm thay đổi trạng thái tài nguyên trên server) vừa mang tính **idempotent** (gọi lặp lại nhiều lần có tác động lên trạng thái hệ thống giống hệt gọi một lần — mọi phương thức safe đều tự động idempotent, trong khi `PUT` và `DELETE` ở dưới là idempotent nhưng không safe).
- `POST`: Dùng để tạo ra một tài nguyên hoàn toàn mới trên server (ví dụ: `POST /submissions`). Đặc điểm của `POST` là mặc định không mang tính idempotent.
- `PUT`: Có nhiệm vụ cập nhật hoặc ghi đè toàn bộ thông tin của một tài nguyên (ví dụ: `PUT /questions/123`). Quá trình này được thiết kế để luôn có tính idempotent.
- `PATCH`: Cho phép cập nhật một phần nhỏ dữ liệu của tài nguyên thay vì toàn bộ (ví dụ: `PATCH /users/123`). Theo tiêu chuẩn RFC 5789, `PATCH` mặc định là *non-idempotent*, mặc dù trong thực tế các lập trình viên thường chủ động thiết kế và triển khai nó một cách idempotent.
- `DELETE`: Có chức năng loại bỏ hoặc xóa một tài nguyên cụ thể (ví dụ: `DELETE /questions/123`). Dù thao tác xóa diễn ra thành công ở lần đầu hay báo lỗi không tìm thấy ở các lần sau, trạng thái dữ liệu vẫn nhất quán nên nó mang tính idempotent.

3. Response codes có ý nghĩa:

__Bảng 3.1:__ Các mã trạng thái HTTP thường dùng trong microservices

Code	Khi nào dùng	Ví dụ
`200 OK`	Request thành công	`GET /questions/123` trả về question
`201 Created`	Tạo resource mới thành công	`POST /questions` trả về question vừa tạo
`202 Accepted`	Đã nhận, xử lý sau	`POST /submissions` qua Kafka (Ch.5)
`204 No Content`	Thành công nhưng không có body	`DELETE /questions/123`
`400 Bad Request`	Request sai cú pháp	JSON hỏng, sai kiểu tham số

Code	Khi nào dùng	Ví dụ
`401 Unauthorized`	Chưa xác thực	Token hết hạn
`403 Forbidden`	Không có quyền	Student truy cập admin API
`404 Not Found`	Resource không tồn tại	`GET /questions/99999`
`409 Conflict`	Conflict trạng thái	Submission đã được chấm
`422 Unprocessable Content`	Đúng cú pháp, sai ràng buộc	Thiếu field bắt buộc
`500 Internal Server Error`	Lỗi server	Database connection failed

4. Plural nouns, consistent naming — Luôn dùng số nhiều cho collection: `/questions`, không phải `/question`. Chọn một quy ước (kebab-case, camelCase, snake_case) và giữ nhất quán xuyên suốt.

5. Pagination, filtering, sorting — Mọi endpoint trả về collection phải hỗ trợ phân trang. Có hai chiến lược chính:

Offset-based (ví dụ: `?page=2&size=20`). Chiến lược này rất đơn giản và cho phép người dùng nhảy trực tiếp đến một trang cụ thể (jump to page). Tuy nhiên, hiệu năng sẽ giảm đáng kể ở các trang có số thứ tự lớn do cơ sở dữ liệu phải bỏ qua một lượng lớn bản ghi (skip N rows). Hơn nữa, kết quả có thể không ổn định nếu dữ liệu bị thêm hoặc xóa trong quá trình người dùng đang duyệt qua các trang.

Cursor-based (ví dụ: `?after=abc123&limit=20`). Với thứ tự sắp xếp xác định và index phù hợp, chi phí mỗi trang gần như không đổi dù trang sâu hay nông (không phải bỏ qua nhiều bản ghi như offset), và không bị lặp hay sót bản ghi khi có dữ liệu mới chèn vào giữa hai lần đọc — dù cursor không tự bảo đảm một snapshot nhất quán. Đổi lại, chiến lược này không hỗ trợ việc nhảy trang tự do và việc triển khai cũng phức tạp hơn nhiều so với offset-based.

KBM sử dụng **offset-based** (Spring Data `Pageable`) — hợp lý cho danh sách câu hỏi quy mô vừa. Với các tập dữ liệu (dataset) lớn (lịch sử nộp bài, nhật ký hệ thống), cursor-based pagination hiệu quả hơn khi đo thấy các trang sâu (offset lớn) làm truy vấn chậm rõ rệt.

Phản hồi phân trang (response pagination) nên bao gồm siêu dữ liệu (metadata) để máy khách nhận biết còn dữ liệu tiếp theo hay không:

Listing 3.1: *Phân trang response kèm metadata*

```
GET /questions?page=0&size=20&sort=createdAt,desc&difficulty=HARD

// Response envelope
{
  "data": [...],
  "pagination": {
    "page": 0,
    "size": 20,
    "totalElements": 157,
    "totalPages": 8,
    "hasNext": true
  }
}
```

6. Standardized Error Response — Các phản hồi lỗi (error response) cần có định dạng nhất quán để máy khách có thể xử lý tự động qua mã nguồn (programmatically). Lấy cảm hứng từ chuẩn Problem Details for HTTP APIs (RFC 9457 — bản thay thế RFC 7807), một error response nên bao gồm các trường sau. Lưu ý theo chuẩn gốc, các trường này đều tùy chọn ở mức từng response

(`type` mặc định là `about:blank`); mức "Bắt buộc" dưới đây là *quy ước nội bộ KBM* áp đặt để client xử lý thống nhất:

- `type` **(Bắt buộc)**: URI định danh loại lỗi.
- `title` **(Bắt buộc)**: Mô tả ngắn gọn về loại lỗi.
- `status` **(Bắt buộc)**: Mã trạng thái HTTP chuẩn (HTTP status code).
- `detail` **(Khuyến nghị)**: Mô tả chi tiết nguyên nhân gây ra lỗi cụ thể này.
- `instance` **(Khuyến nghị)**: URI định danh lần xảy ra lỗi cụ thể; không nhất thiết là URL của request.
- `errorCode` **(Bắt buộc — extension KBM)**: mã máy đọc được, ổn định, dạng chuỗi, ví dụ `VALIDATION_FAILED`, `JUDGE_UNAVAILABLE`. Không dùng số trần thiếu ngữ nghĩa.
- `traceId` **(Bắt buộc khi tracing hoạt động — extension KBM)**: W3C trace ID của occurrence; giúp client báo cáo cho đội ngũ hỗ trợ.
- `errors` **(Validation-only extension KBM — tùy chọn)**: member mở rộng do KBM tự định nghĩa theo cơ chế extension của RFC 9457; chứa danh sách các trường bị lỗi trong quá trình kiểm tra tính hợp lệ của dữ liệu (validation error), mỗi item dùng `pointer` (JSON Pointer), `detail`, và `code` tùy chọn.

Dưới đây là một ví dụ minh họa cho cấu trúc phản hồi lỗi theo chuẩn RFC 9457, giúp việc xử lý lỗi phía client trở nên đồng nhất hơn:

Listing 3.2: *Response lỗi chuẩn hóa (RFC 9457 Problem Details)*

```
HTTP/1.1 422 Unprocessable Content
Content-Type: application/problem+json

{
  "type": "urn:kbm:problem:validation-failed",
  "title": "Input validation failed",
  "status": 422,
  "detail": "The request contains invalid fields.",
  "instance": "urn:kbm:problem-instance:req-7f7bb1",
  "errorCode": "VALIDATION_FAILED",
  "traceId": "0af7651916cd43dd8448eb211c80319c",
  "errors": [
    {"pointer": "#/title", "detail": "must not be blank", "code": "NOT_BLANK"},
    {"pointer": "#/difficulty", "detail": "must be EASY, MEDIUM, or HARD",
     "code": "INVALID_ENUM"}
  ]
}
```

Mọi dịch vụ trong hệ thống nên trả về lỗi theo cùng một định dạng — máy khách chỉ cần xây dựng **một** bộ xử lý lỗi (error handler) duy nhất. Trong Spring Boot, `@ControllerAdvice` + `GlobalExceptionHandler` giải quyết: mọi exception được bắt tại một điểm duy nhất và chuyển đổi thành định dạng phản hồi (response format) chuẩn (xem Bài học rút ra, §3.9.2).

7. Idempotency — Một số thao tác (operation) cần đảm bảo tính **idempotent**: thực hiện nhiều lần có cùng tác động lên trạng thái hệ thống như thực hiện một lần. GET, PUT, DELETE vốn dĩ đã mang tính idempotent. POST thì không — nếu máy khách gọi `POST /submissions` hai lần (do request bị gửi lại khi gặp lỗi mạng — network retry), hệ thống có thể tạo ra hai bản ghi submission.

Giải pháp: **Idempotency Key** — máy khách gửi kèm header `Idempotency-Key: <UUID>`. Máy chủ kiểm tra khóa (key) đã được xử lý chưa: nếu rồi, trả kết quả cũ; nếu chưa, xử lý và lưu khóa. Stripe,

PayPal đều dùng mẫu thiết kế (pattern) này cho các API thanh toán — điều này đặc biệt quan trọng khi liên quan đến tài chính hoặc các thao tác nghiệp vụ cốt lõi.

3.2. API Versioning & Schema Evolution

Tại sao cần versioning? Trong monolith, thay đổi internal API chỉ yêu cầu refactor mã nguồn và chạy lại các bài kiểm thử. Trong microservices, service A thay đổi định dạng phản hồi (response format) → service B (consumer) *có thể* gặp lỗi nghiêm trọng (crash) nếu không xử lý. Đây là bài toán **schema evolution** — API versioning cùng các quy tắc tương thích (backward/forward) trình bày trong mục này là những giải pháp bổ trợ cho nhau [7, Ch.4].

3.2.1. Các chiến lược versioning

Ba chiến lược phiên bản hóa (versioning) nền tảng thường được áp dụng trong microservices:

- **URL path**: ví dụ `/api/v1/questions`, `/api/v2/questions`. Cách tiếp cận này đơn giản và rất rõ ràng cho người sử dụng. Tuy nhiên, nó dẫn đến việc phải duy trì nhiều URL khác nhau, làm cho cấu hình routing trên hệ thống trở nên phức tạp hơn.
- **Query parameter**: ví dụ `/questions?version=2`. Cách này giúp hạn chế việc phải thay đổi cấu trúc URL cốt lõi. Mặc dù vậy, tham số này rất dễ bị bỏ sót trong quá trình tích hợp nếu người dùng không đọc kỹ tài liệu.
- **Header**: ví dụ custom header `X-API-Version: 2`. Cách này giữ cho URL sạch sẽ (clean URL). Đổi lại, việc gọi thử nghiệm và debug trở nên khó khăn hơn vì yêu cầu cấu hình header thủ công, không trực quan như việc nhìn trực tiếp vào URL.

Theo Sam Newman, **URL path versioning** phổ biến nhất vì đơn giản và thân thiện với lập trình viên [4a, Ch.4]. Đây cũng là cách mà nhiều API public lớn (ví dụ Twitter) sử dụng. Ngoài ra còn một biến thể của header versioning đáng chú ý: *date-based versioning* — thay vì `v1`/`v2`, consumer ghim (pin) một ngày phát hành cụ thể qua header (Stripe dùng `Stripe-Version: 2024-12-18`); chúng ta sẽ gặp lại ví dụ Stripe ở phần phân tích KBM bên dưới.

Chiến lược thứ tư — **Media Type versioning** (hay **Content Negotiation versioning**) (`Accept: application/vnd.kbm.question.v2+json`) — từng được GitHub REST API v3 dùng: nhúng phiên bản vào header `Accept` (khác với custom header ở trên), cho phép version từng resource riêng lẻ thay vì toàn bộ API. Cần làm rõ rằng header `Accept` được dùng cho quá trình đàm phán nội dung (content negotiation — định dạng mà client kỳ vọng nhận về), trong khi `Content-Type` được dùng để mô tả định dạng của phần thân (body) trong request. Tuy nhiên, phương pháp này phức tạp hơn cho cả provider và consumer.

Bảng 3.2: *Tiêu chí lựa chọn — chọn chiến lược versioning nào?*

Tiêu chí	URL path	Query param	Header	Media Type
Dễ implement	★★★	★★★	★★	★
Cache-friendly	✓ (URL khác nhau)	✓ (URL khác nhau)	× (cần Vary header)	×
API Gateway routing	✓ Dễ (path-based)	! Trung bình	! Trung bình	× Khó
Granularity	Toàn bộ API	Toàn bộ API	Toàn bộ API	Per resource
Dùng bởi	Twitter	Azure (`api-version`)	Stripe (`Stripe-Version`)	GitHub API v3 (cũ)

Chính sách versioning — nguyên tắc vận hành:
Chọn chiến lược chưa đủ — cần **chính sách** rõ ràng cho cách version API:

1. **Khi nào cần nâng cấp phiên bản (version bump)?** — Chỉ khi có **breaking change** (xóa trường dữ liệu, đổi kiểu dữ liệu, đổi hành vi). Thêm một trường dữ liệu (field) không bắt buộc → *không* cần phiên bản mới (tương thích ngược, backward compatible)
2. **Deprecation timeline** — Thông báo deprecation trước ít nhất **1 release cycle** qua các response header theo đúng định dạng chuẩn: `Deprecation: @1764547200` (Unix timestamp, RFC 9745 — ở đây là 2025-12-01, sớm hơn Sunset sáu tháng), `Sunset: Mon, 01 Jun 2026 00:00:00 GMT` (HTTP-date, RFC 8594), kèm `Link: <...>; rel="deprecation"` trỏ tới hướng dẫn chuyển đổi
3. **Chạy song song** — Phiên bản cũ và mới hoạt động đồng thời trong giai đoạn chuyển đổi (transition). Sử dụng Gateway routing để chuyển hướng dần lưu lượng truy cập (traffic) (Strangler Fig pattern — xem Chương 10)

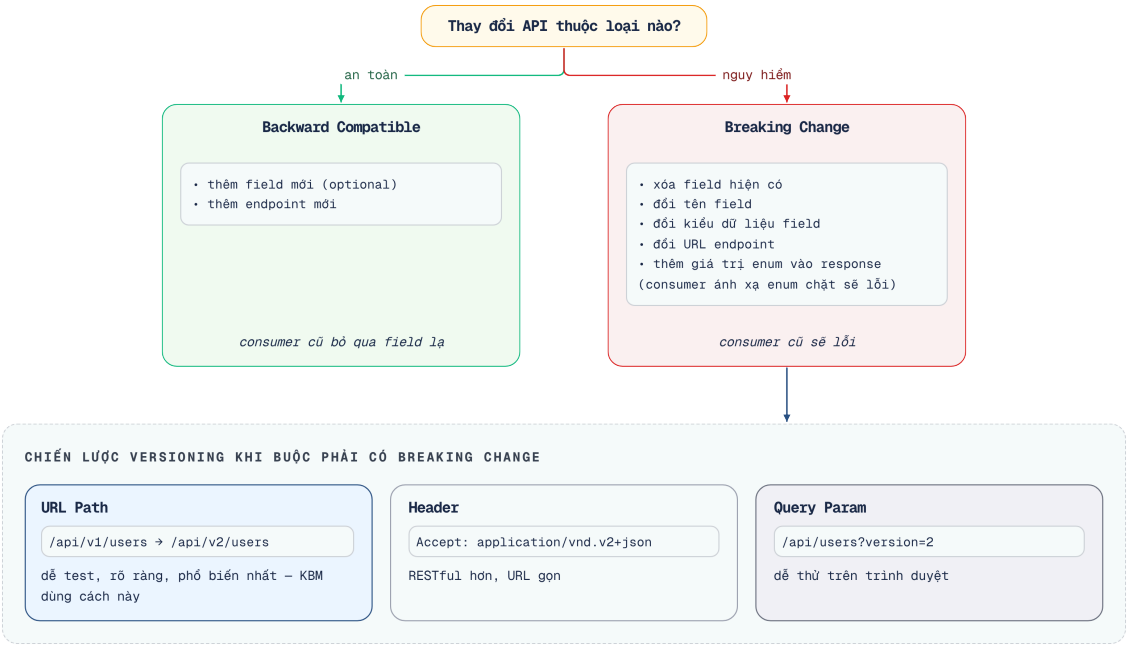
Chính sách versioning của KBM

Với KBM (LMS học thuật, team nhỏ), chiến lược đơn giản nhất: (1) **URL path versioning** (`/api/v1/questions`), (2) giữ **backward compatibility** bằng cách chỉ thêm trường, không xóa, (3) khi breaking change bắt buộc → tạo `/api/v2/` → chạy song song 1 học kỳ (semester) trước khi loại bỏ (sunset) v1. Phức tạp hơn không cần thiết cho hệ thống giáo dục nội bộ.

Schema Evolution — Thay đổi mà không breaking. Martin Kleppmann phân tích compatibility [7, Ch.4]. Trong ví dụ API dưới đây, hai khái niệm cùng xét client cũ làm việc với API mới, chỉ khác góc nhìn: backward nhìn từ provider, forward nhìn từ consumer (hộp Tolerant Reader ở §3.9.2 làm rõ điều này):

- **Tương thích ngược (Backward compatible)**: code (reader) phiên bản **mới** đọc được dữ liệu do phiên bản **cũ** tạo ra — ví dụ trường mới được thêm dưới dạng không bắt buộc (optional field) nên dữ liệu cũ thiếu trường đó vẫn hợp lệ.
- **Tương thích xuôi (Forward compatible)**: code (reader) phiên bản **cũ** đọc được dữ liệu do phiên bản **mới** tạo ra — ví dụ client cũ bỏ qua field lạ trong response mới thay vì báo lỗi. Đây chính là điều kiện để Tolerant Reader (§3.9.2) hoạt động. Vì vậy, để API mới không làm gián đoạn client cũ cần cả hai chiều: client cũ tương thích xuôi với response mới, và provider mới tương thích ngược với request cũ.

Quy tắc vàng khi thay đổi API:



Hình 3.2: *Quy tắc schema evolution — thay đổi nào là breaking change*

KBM thiếu API versioning

KBM hiện chưa chuẩn hóa API versioning — nhiều endpoint nằm tại `/api/*` không có version prefix. Ở quy mô một consumer duy nhất, đây là **nợ kỹ thuật chấp nhận được (xem Bài học rút ra ở cuối chương) — nhưng phải trả trước khi mở rộng**: khi hệ thống thêm mobile app hoặc third-party integration, mỗi thay đổi API sẽ là breaking change cho consumer không kịp cập nhật. Stripe API — một trong những API được thiết kế tốt nhất — sử dụng date-based versioning (`2024-12-18`), cho phép consumer chỉ định cố định (pin) một version cụ thể. **Lộ trình chuyển đổi (Migration path)**: thêm tiền tố `/api/v1/` cho tất cả route hiện tại, cấu hình định tuyến trên Gateway, và lập tài liệu chính sách đánh phiên bản.

3.3. OpenAPI & API Documentation

Tại sao tài liệu (documentation) quan trọng? Chris Richardson nhấn mạnh rằng trong microservices, tài liệu API không phải là một "tùy chọn có thì tốt" (nice-to-have) mà là một **hợp đồng** (*contract*) giữa provider và consumer [2a, Ch.3]. Nếu tài liệu sai hoặc thiếu, lập trình viên phải đọc mã nguồn (source code) để hiểu API — điều này phá vỡ nguyên lý *Service Abstraction* [1].

OpenAPI Specification (Swagger). OpenAPI (trước đây gọi là Swagger) là chuẩn mô tả REST API dưới dạng YAML/JSON. Từ spec này, có thể tự động generate:

- Documentation UI (Swagger UI, ReDoc)
- Bộ thư viện phát triển máy khách (Client SDK: Java, TypeScript, Python)
- Mã khung máy chủ (Server stubs)
- Mã kiểm thử (Tests)

Trong Spring Boot (như LMS), sử dụng **SpringDoc OpenAPI** để tự động tạo spec từ code:

Listing 3.3: Spring Boot controller với OpenAPI annotations

```
// Annotations trên controller → auto-generate OpenAPI spec
@RestController
@RequestMapping("/api/questions")
@Tag(name = "Questions", description = "Quản lý câu hỏi SQL")
public class QuestionController {

    @GetMapping("/{id}")
    @Operation(summary = "Lấy chi tiết câu hỏi theo ID")
    @ApiResponse(responseCode = "200", description = "Thành công")
    @ApiResponse(responseCode = "404", description = "Không tìm thấy")
    public QuestionResponse getById(@PathVariable UUID id) {
        // ...
    }
}
```

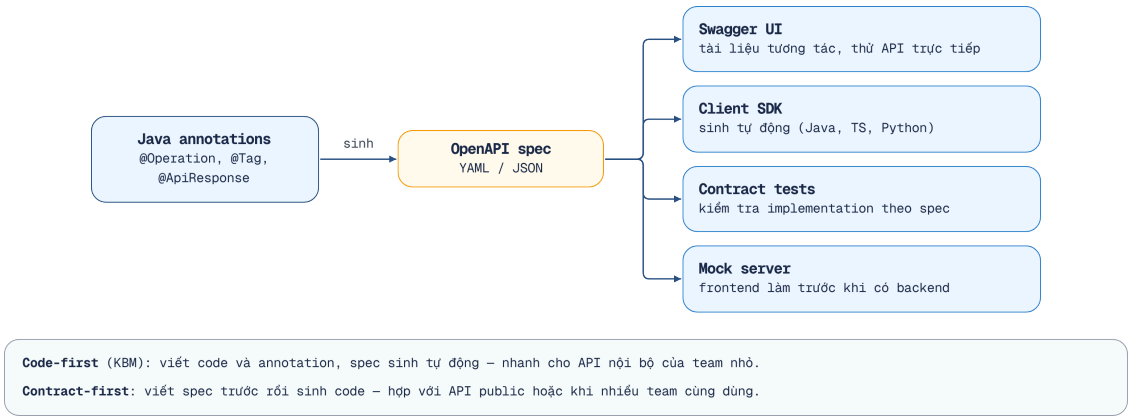
Khi ứng dụng đã thêm dependency `org.springdoc:springdoc-openapi-starter-webmvc-ui`, truy cập `/swagger-ui.html` để dùng giao diện tương tác thử nghiệm API trực tiếp.

Một quyết định quy trình quan trọng còn lại: viết spec trước hay viết code trước?

Contract-First vs Code-First

Có hai cách tiếp cận thiết kế API: **Contract-first** (viết OpenAPI spec trước, generate code sau) và **Code-first** (viết code trước, generate spec sau). Contract-first tốt hơn cho API public hoặc khi giao tiếp giữa nhiều nhóm phát triển (cross-team). Code-first nhanh hơn cho API internal trong các nhóm phát triển quy mô nhỏ. KBM sử dụng code-first (Spring Boot annotations → auto-generated docs) — hợp lý cho team nhỏ [1, Ch.7].

Hình 3.3 cho thấy toàn bộ chuỗi theo cách code-first: từ annotation trong code, qua OpenAPI spec, đến Swagger UI, client SDK, contract test và mock server.



Hình 3.3: Pipeline từ code annotations đến Swagger UI và client SDK

Cách làm này không chỉ giảm thiểu sức lao động thủ công mà còn giúp duy trì một single source of truth, đảm bảo tài liệu luôn đồng bộ với mã nguồn thực tế.

Documentation-as-Code

Coi API documentation như code: version control, review, automate. Nếu documentation nằm ngoài code (wiki, Confluence), nó sẽ nhanh chóng lỗi thời. SpringDoc/OpenAPI giải quyết vấn đề này bằng cách generate docs trực tiếp từ code.

3.4. DTO Pattern — tách Internal Model và External Contract

Vấn đề: Entity ≠ API Response. Sai lầm phổ biến trong microservices (và monolith) là trả về trực tiếp thực thể (entity) qua API:

Listing 3.4: Anti-pattern — trả entity trực tiếp qua API

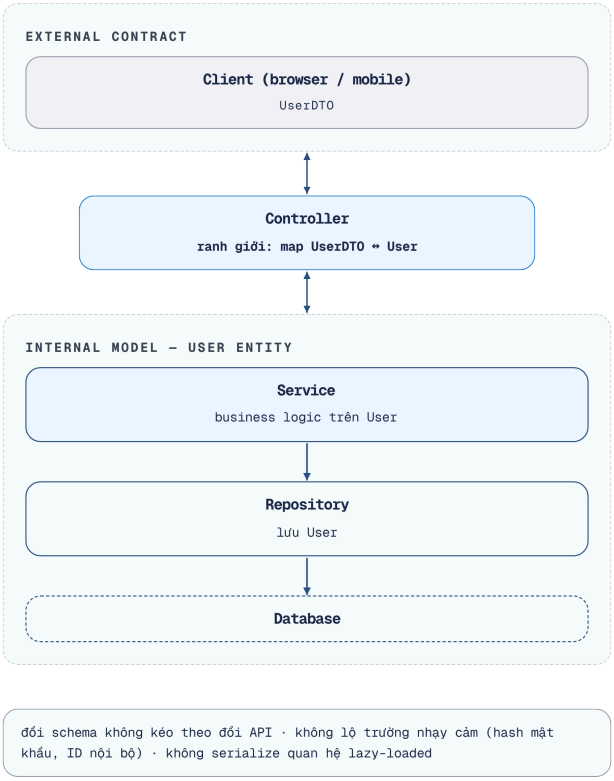
```
// SAI: Trả entity trực tiếp --- expose internal structure
@GetMapping("/questions/{id}")
public Question getById(@PathVariable UUID id) {
    return questionRepository.findById(id).orElseThrow();
}
```

Vấn đề:

- **Sự phụ thuộc (Coupling)**: Thay đổi lược đồ cơ sở dữ liệu (database schema) đồng nghĩa với việc thay đổi phản hồi API
- **Bảo mật (Security)**: Có nguy cơ để lộ (expose) các trường nhạy cảm (chuỗi băm mật khẩu, định danh nội bộ)
- **Hiệu năng (Performance)**: Thực thể có thể chứa các quan hệ tải lười (lazy-loaded) → dẫn đến lỗi N+1 truy vấn hoặc lỗi tuần tự hóa (serialization error)

3.4.1. DTO (Data Transfer Object)

DTO pattern tạo lớp đệm giữa internal model và external contract [2a], như minh họa ở Hình 3.4.



Hình 3.4: DTO pattern — tách internal model và external contract

Trong mã nguồn, lớp đệm này gồm DTO cho request, DTO cho response và một mapper chuyển đổi giữa entity và DTO (Listing 3.5):

Listing 3.5: Request DTO, Response DTO và Mapper

```
// ĐÚNG: Request DTO --- chỉ chứa field client cần gửi
public record CreateQuestionRequest(
    String title,
    String description,
    String solution,
    String difficulty
) {}

// ĐÚNG: Response DTO --- chỉ chứa field client cần nhận
public record QuestionResponse(
    UUID id,
    String title,
    String difficulty,
    int submitCount,
    double acceptRate,
    LocalDateTime createdAt
) {}

// Mapper chuyển đổi Entity ↔ DTO
@Mapper(componentModel = "spring")
public interface QuestionMapper {
    QuestionResponse toResponse(Question entity);
```

```
    Question toEntity(CreateQuestionRequest request);
}
```

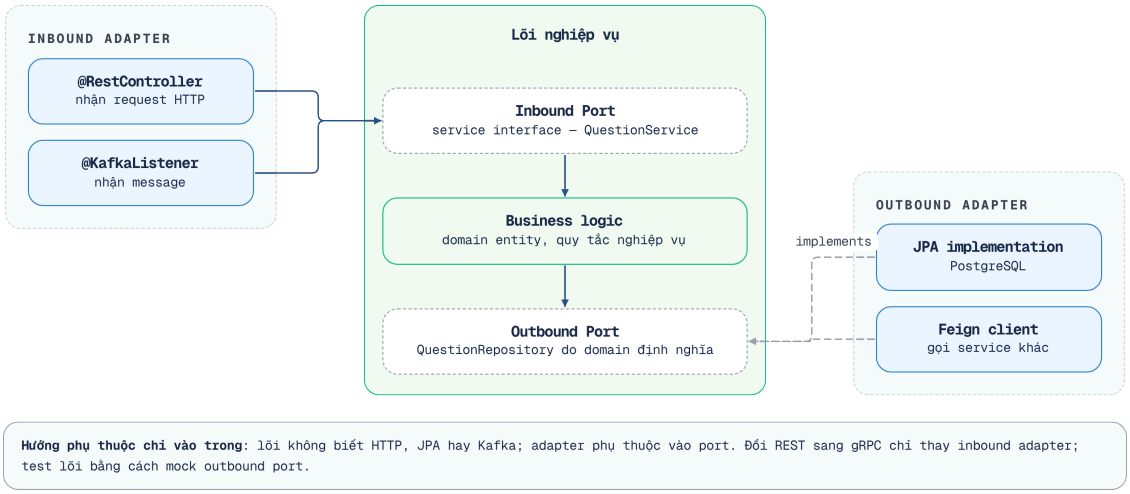
3.4.2. Lợi ích thực tế

- **Decoupling**: Thay đổi cấu trúc của entity (ví dụ: thêm cột trong cơ sở dữ liệu) sẽ không tự động làm thay đổi cấu trúc API trả về cho client.
- **Security**: Tránh được việc vô tình làm rò rỉ các trường dữ liệu nhạy cảm như mật khẩu hay các trường nội bộ (internal field) ra bên ngoài.
- **Performance**: Chỉ phản hồi những trường dữ liệu thực sự cần thiết, ngăn chặn tình trạng hệ thống tự động tải toàn bộ đồ thị đối tượng (lazy-load) không mong muốn.
- **Versioning**: Cho phép tạo ra các phiên bản response mới như `QuestionResponseV2` một cách độc lập mà không cần phải can thiệp hay chỉnh sửa entity gốc.
- **Validation**: Đưa các quy tắc kiểm tra tính hợp lệ (`@Valid`) lên mức DTO, giữ cho entity hoàn toàn tập trung vào biểu diễn dữ liệu cốt lõi.

Trong KBM, pattern này được áp dụng qua các package `dto/request/` và `dto/response/`, với MapStruct (`BaseMapper`) để tự động chuyển đổi. Các lớp cơ sở `BaseRequest` và `BaseResponse` trong shared library cung cấp format phản hồi chuẩn (`status`, `message`, `data` wrapper) cho luồng thành công, tách biệt với định dạng lỗi đích theo chuẩn RFC 9457 Problem Details qua `GlobalExceptionHandler` đã nêu ở §3.1 (hiện trạng lỗi không đồng nhất giữa các thành phần được phân tích ở §3.9.1).

3.5. Hexagonal Architecture — tách biệt lõi nghiệp vụ khỏi hạ tầng

DTO pattern ở trên là một phần của kiến trúc rộng hơn mà Richardson mô tả chi tiết trong [2b, Ch.3 & Ch.13]: **Hexagonal Architecture** (còn gọi là Ports & Adapters, do Alistair Cockburn đề xuất). Đây là kiến trúc được đề xuất cho mỗi service trong microservices:



Hình 3.5: Hexagonal Architecture cho Core Service trong KBM

Nguyên tắc cốt lõi: Business logic **không phụ thuộc** vào các adapter — ngược lại, các adapter phụ thuộc vào business logic. Điều này có nghĩa:

- Nếu đổi từ REST sang gRPC, nhóm phát triển chỉ cần thay inbound adapter mà không chạm tới business logic.

- Nếu đổi từ MySQL sang PostgreSQL, nhóm phát triển chỉ cần thay outbound adapter.
- Khi kiểm thử business logic, chỉ cần mock các outbound port rồi gọi trực tiếp vào inbound port.

Các khái niệm của Spring Boot có thể được ánh xạ vào Hexagonal Architecture như sau:

- **Inbound Adapter**: Thành phần chịu trách nhiệm nhận request từ bên ngoài (ví dụ: `@RestController`, `@KafkaListener`).
- **Inbound Port**: API giao tiếp của business logic, thường là các service interface (ví dụ: `QuestionService`).
- **Business Logic**: Trái tim của hệ thống, bao gồm các domain entity và lớp hiện thực service (service implementation) xử lý nghiệp vụ cốt lõi.
- **Outbound Port**: Cách mà business logic định nghĩa yêu cầu giao tiếp với thế giới bên ngoài, là interface do chính lõi nghiệp vụ định nghĩa và chỉ nói bằng ngôn ngữ domain (ví dụ `QuestionRepository` tự viết). Lưu ý `JpaRepository` **không phải** outbound port — nó thuộc về adapter; để port sạch, domain không import bất kỳ type nào của Spring Data.
- **Outbound Adapter**: Các thành phần thực hiện kết nối thực tế ra hệ thống bên ngoài, như JPA implementation, Feign client, hay Kafka producer.

Iceberg Principle

Hexagonal Architecture là hiện thực hóa của **Iceberg Principle** (Richardson [2b, Ch.4]): Inbound Ports (API) là phần nổi — nhỏ, ổn định. Phần chìm bao gồm Business Logic (cốt lõi nghiệp vụ) và Outbound Adapters (hạ tầng kỹ thuật). Hexagonal tách biệt chúng để hạ tầng linh hoạt thay thế mà không ảnh hưởng lõi nghiệp vụ. DTO pattern là cơ chế đảm bảo entity (phần chìm) không bị để lộ (expose) qua API (phần nổi).

Ví dụ minh họa sự khác biệt giữa Port (Interface) và Adapter (Implementation):

```
// Outbound Port (Business Logic)
public interface QuestionRepository {
    Question save(Question q); // Domain model
}

// Outbound Adapter (Infrastructure)
@Repository
public class JpaQuestionAdapter implements QuestionRepository {
    @Override
    public Question save(Question q) {
        // map ngược về domain model
        return EntityMapper.toDomain(springDataRepo.save(EntityMapper.toJpa(q)));
    }
}
```

3.6. Contract Testing — kiểm thử hợp đồng API

3.6.1. Vấn đề: API thay đổi làm consumer gặp lỗi

Tài liệu OpenAPI mô tả diện mạo của API — nhưng không đảm bảo consumer và provider thực sự đồng ý về ý nghĩa của từng trường dữ liệu, mã trạng thái, và các trường hợp biên (edge case). Trong microservices, mỗi service là một independent deployment unit: khi `kbm-core` thay đổi cấu trúc phản hồi (response schema), `kbm-academic` (consumer) có thể gặp sự cố trong quá trình thực

thi (runtime crash) — và nhóm phát triển sẽ chỉ phát hiện sau khi deploy lên môi trường staging hoặc production.

Hợp đồng API và lịch học

Sự thay đổi API không báo trước giống như việc Phòng Đào tạo (Provider) tự ý đổi giờ học môn Cơ sở dữ liệu từ sáng sang chiều mà không báo cho sinh viên (Consumer). Dù môn học vẫn diễn ra (các bài kiểm thử đơn vị vẫn đạt), nhưng sinh viên không thể tham gia vì kẹt lịch khác (máy khách bị lỗi/crash). Contract Testing giúp Phòng Đào tạo phải "hỏi" sinh viên xem có ai bị kẹt lịch không trước khi chính thức ban hành thay đổi.

Các bài kiểm thử đơn vị (unit test) và kiểm thử tích hợp (integration test) truyền thống không giải quyết được vấn đề này vì chúng kiểm thử từng dịch vụ độc lập. Cần một cơ chế để **provider biết trước khi deploy liệu thay đổi của mình có làm hỏng consumer nào không**.

3.6.2. Consumer-Driven Contract Testing

Consumer-Driven Contract Testing (CDCT) giải quyết bằng cách để consumer viết "contract" — tập hợp kỳ vọng cụ thể về API của provider. Provider sau đó verify contract này trong test suite của mình trước khi deploy.

Với **Pact**, luồng hoạt động gồm năm bước:

1. **Consumer team** viết kiểm thử mô tả chi tiết request gửi đi và response kỳ vọng nhận lại từ provider.
2. **Framework** (Pact) chạy kiểm thử phía consumer với mock server và tự động sinh ra *contract file* (Pact JSON).
3. Thông qua **CI pipeline**, contract file được phát hành (publish) lên kho lưu trữ hợp đồng trung tâm (*Pact Broker*).
4. CI của **provider** tự động kéo các contract từ Broker về và phát lại (replay) request vào provider thực để xác minh phản hồi.
5. Nếu thay đổi của provider làm vỡ bất kỳ hợp đồng nào đang hiệu lực, **CI pipeline** lập tức chặn deploy trước khi lỗi lọt lên production.

Với **Spring Cloud Contract**, chiều thực thi hơi khác dù cùng mục tiêu bảo vệ consumer: hợp đồng được viết bằng Groovy DSL hoặc YAML (do consumer đề xuất qua pull request vào kho của provider hoặc kho hợp đồng chung); plugin trên phía provider tự sinh bộ kiểm thử JUnit (`@SpringBootTest`) để xác minh API thật, đồng thời đóng gói WireMock stub JAR phát hành lên kho artifact (Maven/Nexus) cho consumer tải về chạy kiểm thử tích hợp.

Hai framework phổ biến trong Java/Spring ecosystem:

Bảng 3.3: *So sánh Pact và Spring Cloud Contract*

Khía cạnh	Pact	Spring Cloud Contract
Ngôn ngữ	Polyglot (Java, JS, Go, Python…)	Chủ yếu Java/Spring; ngoài JVM dùng qua Docker image và contract YAML
Contract format	Pact JSON	Groovy DSL / YAML
Broker	Pact Broker (self-hosted/cloud)	Artifact repo (Maven/Nexus)
Tích hợp Spring Boot	Pact JVM + MockMvc	Native (`@SpringBootTest`)
Phù hợp khi	Multi-language, nhiều consumer	Stack thuần Java/Spring

3.6.3. Áp dụng cho KBM

Trong KBM, `kbm-academic` là consumer chính của `kbm-core` (truy vấn câu hỏi, vd. `GET /api/questions/batch`). Nếu nhóm phát triển thêm hoặc đổi tên trường dữ liệu trong phản hồi của `kbm-core`, sẽ không có cảnh báo nào xuất hiện cho đến khi `kbm-academic` nhận được phản hồi không đúng cấu trúc tại thời điểm thực thi (runtime).

Kịch bản minh họa: `kbm-core` đổi `"title"` → `"name"` trong phản hồi câu hỏi. `kbm-academic` đọc trường `title` → nhận `null` → trang bài tập hiện câu hỏi không có tên (hoặc NullPointerException nếu code không kiểm tra null). Không có lỗi biên dịch (compile error), các bài kiểm thử đơn vị (unit test) vẫn vượt qua thành công.

Với Contract Testing (Spring Cloud Contract):

```
// contract định nghĩa bởi kbm-academic (consumer)
Contract.make {
  request {
    method GET()
    url "/api/questions/550e8400-e29b-41d4-a716-446655440000"
  }
  response {
    status 200
    // consumer expect field title
    body([id: "550e8400-e29b-41d4-a716-446655440000",
          title: "Truy vấn JOIN cơ bản", difficulty: "EASY"])
    headers { contentType(applicationJson()) }
  }
}
```

Khi `kbm-core` chạy xác minh hợp đồng này bằng trường dữ liệu `name` thay vì `title` → kiểm thử thất bại (test fail) → hệ thống CI chặn đứng quá trình deploy → nhóm phát triển phát hiện lỗi trước khi đưa lên production.

Khi nào dùng Contract Testing?

Contract Testing mang lại giá trị khi: (1) có từ 2 nhóm phát triển trở lên sở hữu các service giao tiếp với nhau, (2) chu kỳ triển khai khác biệt (consumer và provider deploy độc lập), (3) đã có unit test và integration test cơ bản. Với nhóm phát triển quy mô nhỏ và deploy cùng lúc, chi phí thiết lập (setup) Pact Broker có thể không mang lại hiệu quả tương xứng — ưu tiên OpenAPI contract + integration test trước.

3.7. GraphQL — khi REST không đủ linh hoạt

REST là chuẩn phổ biến nhất cho các API trong microservices, nhưng không phải lựa chọn duy nhất. **GraphQL** (Facebook, 2012; open-sourced 2015) giải quyết một số hạn chế của REST bằng cách cho phép client **chọn chính xác data cần lấy**.

Vấn đề mà GraphQL giải quyết. Với REST, mỗi endpoint trả về *fixed structure*. Client lấy nhiều hơn hoặc ít hơn data cần thiết. Cụ thể, GraphQL nhắm vào ba vấn đề kinh điển:

- **Over-fetching**: API trả về quá nhiều dữ liệu không cần thiết. Ví dụ, `GET /questions/1` trả về 20 trường, nhưng mobile chỉ cần `title` và `difficulty`. Với GraphQL, client chỉ yêu cầu đúng những trường cần thiết: `{ question(id:1) { title difficulty } }`.

- **Under-fetching**: API không trả đủ dữ liệu, buộc phải gọi nhiều lần. Ví dụ, dashboard cần thông tin question, submissions và user, dẫn đến 3 lệnh gọi API. GraphQL cho phép lấy tất cả trong 1 query: `{ question(id: "123") { title submissions { score } createdBy { name } } }`.
- **Multiple round trips**: Việc gọi nhiều endpoint như trên gây ra nhiều request qua mạng (đặc biệt tốn kém về độ trễ trên mạng di động). GraphQL gộp thành 1 request và 1 response duy nhất, giảm rõ rệt số vòng khứ hồi qua mạng.

3.7.1. REST vs GraphQL — so sánh

Bảng 3.4 so sánh hai cách tiếp cận theo bảy tiêu chí.

__Bảng 3.4:__ REST vs GraphQL — so sánh chi tiết

Tiêu chí	REST	GraphQL
Endpoint	Nhiều endpoint (`/questions`, `/submissions`, `/users`)	Một endpoint (`/graphql`)
Cấu trúc dữ liệu trả về (Data shape)	Máy chủ quyết định cấu trúc (structure) trả về	Máy khách quyết định các trường cần lấy
Versioning	URL versioning (`/v1/`, `/v2/`) hoặc header	Schema evolution — thêm trường mới, đánh dấu ngừng dùng (deprecate) trường cũ
Caching	HTTP caching tự nhiên (GET, ETag, Cache-Control)	Phức tạp hơn (dùng POST request, cần Apollo Cache hoặc tương tự)
Tooling	Swagger/OpenAPI — trưởng thành, phổ biến rộng rãi	GraphiQL, Apollo DevTools — mạnh mẽ nhưng hệ sinh thái nhỏ hơn
Learning curve	Thấp (HTTP verbs, status codes)	Trung bình (schema language, resolvers, N+1 problem)
Xử lý lỗi	Mã trạng thái HTTP (400, 404, 500)	Operation đã thực thi trả `200 OK` kèm mảng `errors` (kể cả khi có field lỗi); lỗi transport/ request hỏng/xác thực vẫn dùng 4xx/5xx — ngữ nghĩa status code mờ hơn REST

3.7.2. N+1 Problem — thách thức chính của GraphQL

Với GraphQL, một truy vấn lồng nhau có thể khiến server thực thi N+1 câu SQL:

```
# Query: lấy tất cả questions và submissions cho mỗi question
{ questions { title submissions { score } } }

# Server thực thi:
# 1. SELECT * FROM questions (N questions)
# 2. Với MỖI question: SELECT * FROM submissions WHERE question_id = ?
# → N+1 queries! (1 cho questions + N cho submissions)
```

Giải pháp: DataLoader pattern — gom N truy vấn thành 1: `SELECT * FROM submissions WHERE question_id IN (1, 2, ..., N)`. Facebook open-sourced DataLoader library cho mục đích này. Điểm khác biệt với REST: trong REST, server quyết định trước query structure nên lập trình viên có thể tối ưu sẵn từng endpoint (dù N+1 vẫn xảy ra ở tầng ORM nếu bất cẩn — xem §3.4); trong GraphQL, truy vấn do client quyết định nên server bắt buộc phải xử lý N+1 một cách tường minh.

Schema-First Approach. GraphQL server nào cũng cần schema ở runtime, nhưng quy trình phát triển có thể theo hướng *schema-first* (viết schema trước rồi implement resolver) hoặc *code-first* (sinh schema từ code/annotation). Sách chọn schema-first cho ví dụ KBM — tương tự contract-first cho REST (OpenAPI spec trước code), có lợi cho governance và review hợp đồng giữa các team:

Listing 3.6: GraphQL schema definition cho KBM

```
type Question {
  id: ID!
  title: String!
  difficulty: Difficulty!
  submissions: [Submission!]!
  createdBy: User!
}

type Query {
  question(id: ID!): Question
  questions(difficulty: Difficulty, limit: Int = 20): [Question!]!
}

enum Difficulty { EASY MEDIUM HARD }
```

Lược đồ (schema) này vừa là tài liệu, vừa là hợp đồng (contract), vừa là hệ thống kiểu dữ liệu (type system) — tính năng GraphQL introspection cho phép các công cụ (tool) tự động sinh ra mã nguồn máy khách (client code) và tài liệu.

3.7.3. Khi nào chọn REST, khi nào chọn GraphQL?

Bảng 3.5 gợi ý lựa chọn theo từng kịch bản sử dụng.

Bảng 3.5: Khi nào chọn REST, khi nào chọn GraphQL

Kịch bản	Khuyến nghị	Lý do
Service-to-service (backend)	REST hoặc gRPC	Đơn giản, dễ lưu đệm (cacheable), các nhóm kiểm soát cả hai phía
Backend-for-Frontend (BFF) (mobile)	GraphQL	Ứng dụng di động cần các truy vấn linh hoạt (flexible query), giảm số vòng giao tiếp qua mạng (round trip)
Public API	REST	HTTP standards, caching, documentation ecosystem
Dashboard aggregation	GraphQL	1 truy vấn thay vì 5–10 lệnh gọi REST
CRUD đơn giản	REST	Chi phí bổ sung (overhead) của GraphQL không tương xứng với lợi ích mang lại
Real-time (subscriptions)	GraphQL + WebSocket	Native support cho subscriptions

KBM hiện tại: REST vẫn là lựa chọn phù hợp cho LMS core vì đơn giản, dễ debug và được OpenAPI hỗ trợ tốt. Tuy nhiên, KBM đã có hệ sinh thái API phức tạp hơn: Network Lab dùng nhiều protocol để phục vụ bài học tích hợp hệ thống; DevOps Lab dùng Go/Chi cho router và API vận hành; các luồng chấm bài có thể đi qua Kafka thay vì HTTP đồng bộ. GraphQL sẽ phát huy giá trị nếu KBM bổ sung: (1) **mobile app** (ứng dụng di động) — đòi hỏi các truy vấn linh hoạt cho nhiều kích thước màn hình khác nhau, (2) **teacher dashboard** (trang quản trị của giảng viên) — cần tổng hợp dữ liệu (aggregate data) từ nhiều dịch vụ chỉ thông qua một yêu cầu, (3) **third-party API** (API dành cho bên thứ ba) — người dùng bên ngoài cần kiểm soát chính xác lượng dữ liệu họ nhận được.

Điều quan trọng cần nhớ: **GraphQL không phải là sự thay thế cho REST** — chúng giải quyết vấn đề khác nhau. REST tối ưu cho giao tiếp giữa các service (đơn giản, cache được, theo chuẩn); GraphQL tối ưu cho các API hướng máy khách (client-facing API — linh hoạt, một endpoint, có kiểu dữ liệu). Nhiều hệ thống dùng cả hai: REST giữa các service, GraphQL cho tầng BFF (Richardson [2b, Ch.8]).

⚠ Sai lầm thường gặp

1. **Trả entity trực tiếp qua API** — Không dùng DTO, expose toàn bộ internal structure (password hash, internal IDs, lazy-loaded relations). Hậu quả: thay đổi schema = breaking change cho mọi consumer, rủi ro bảo mật. *Phòng tránh*: luôn dùng DTO pattern (§3.4) — tách internal model khỏi external contract.
2. **Không thống nhất naming convention từ đầu** — Trộn lẫn giữa danh từ số ít và số nhiều, hoặc mix kebab-case với camelCase trong cùng hệ thống. Hậu quả: khi có từ 3 consumer trở lên (mobile, third-party, internal), việc thay đổi quy tắc đặt tên (naming) sẽ dẫn đến hàng loạt breaking change. *Phòng tránh*: lựa chọn quy ước (ví dụ: plural + kebab-case) và bắt buộc tuân thủ (enforce) ngay từ các Pull Request đầu tiên.
3. **Bỏ qua error format chuẩn** — Mỗi dịch vụ trả về lỗi theo định dạng riêng, mỗi lập trình viên tự viết logic xử lý lỗi. Hậu quả: consumer phải xử lý (handle) nhiều định dạng khác nhau, khiến quá trình gỡ lỗi (debugging) khó khăn. *Phòng tránh*: dùng `GlobalExceptionHandler` với `@ControllerAdvice` và error response format thống nhất cho toàn hệ thống (§3.1).

Các pattern trong chương có minh họa tương tác đi kèm tại companion web:

Interactive Demo

- **Thiết kế REST API**: rest-api-explorer.html
- **GraphQL và REST**: graphql-vs-rest.html

3.8. Ranh giới của chương này

Chương 3 dừng ở *hợp đồng API*: tài nguyên được đặt tên ra sao, lỗi được phát biểu thế nào, và hợp đồng tiến hóa mà không phá client bằng cách nào. Hai câu hỏi tự nhiên nảy ra từ đây lại thuộc về chương khác, và sách trả lời chúng ở đúng chỗ có đủ ngữ cảnh:

- **"REST có phải lựa chọn duy nhất cho giao tiếp nội bộ không?"** — gRPC, Protobuf và tiêu chí chọn giữa hai bên được trình bày ở **Chương 4**, kèm bảng so sánh chi tiết và phân tích vì sao lõi KBM vẫn giữ REST/JSON.
- **"Frontend làm sao gọi tới hàng chục service?"** — API Gateway và biến thể BFF được trình bày ở **Chương 8**, nơi chúng đi cùng routing, xác thực tại biên và rate limiting.

3.9. Case study: KBM

Thiết kế API của KBM lớn lên theo nhu cầu từng đợt, nên có chỗ làm tốt và có chỗ chưa nhất quán — đúng như phần lớn hệ thống thật.

3.9.1. Hiện trạng

Phân tích source code KBM cho thấy một số quyết định API design thực tế — một số tốt, một số là *anti-pattern* do phát triển hữu cơ (organic growth):

Điểm mạnh:

- DTO pattern được áp dụng nhất quán (request/response separation)
- Swagger/OpenAPI tích hợp sẵn

- JSON response format chuẩn (`BaseResponse<T>` wrapper)
- KBM giữ REST/OpenAPI làm contract chính cho LMS core, đồng thời mở rộng Network Lab bằng TCP/UDP/RMI/SOAP/REST/gRPC và DevOps Lab bằng Go/Chi router. Vì vậy, case study hiện tại là **multi-protocol có chủ đích**, không còn là một hệ thống REST thuần.

Điểm cần cải thiện (từ khảo sát mã nguồn KBM):

Inconsistent naming. Hiện tại, hệ thống đang trộn lẫn giữa `/question` (danh từ số ít) với `/users` (số nhiều), đồng thời dùng `/submit-history` (cụm động–danh từ số ít thay vì danh từ số nhiều như `/submissions`). Cần chuẩn hóa toàn bộ để nhất quán dùng danh từ số nhiều kết hợp với quy ước kebab-case.

Không có API versioning. Tất cả các endpoint đang được đặt chung tại `/api/*`. Giải pháp khắc phục là bổ sung thêm tiền tố phiên bản như `/api/v1/*` để dễ dàng nâng cấp trong tương lai.

Error response không nhất quán. Mã lỗi trả về đang không đồng nhất, điển hình là sự khác biệt giữa `ServerResponse.description` và `JwtUtil.message`. Nên chuẩn hóa một định dạng phản hồi lỗi duy nhất thông qua `GlobalExceptionHandler`.

Hard delete only. Dữ liệu khi xóa đang bị xóa vĩnh viễn khỏi hệ thống thay vì soft-delete. Việc bổ sung cơ chế soft-delete sẽ giúp lưu vết (audit trail) tốt hơn cho mục đích tra cứu sau này.

SQL Judge worker contract chưa chuẩn hóa. Dịch vụ điều phối `kbm-judge-sql` chuyển yêu cầu tới các sandbox worker như `kbm-sandbox-mysql`, `kbm-sandbox-sqlserver` nhưng cấu trúc `JudgeRequest` và `JudgeResult` chưa có hợp đồng (contract) rõ ràng. Cần xây dựng DTO và OpenAPI chuẩn dùng chung cho tất cả các `kbm-sandbox-*`, đi kèm với cơ chế đánh phiên bản và idempotency key.

3.9.2. Bài học rút ra

Những sự không nhất quán này không phải "lỗi" — chúng là kết quả tự nhiên khi hệ thống phát triển nhanh với team nhỏ, qua nhiều vòng lặp phát triển. Bài học quan trọng:

1. **Quy ước đặt tên (naming convention) cần thống nhất sớm** — chi phí sửa sau này tăng theo số lượng consumer. Khi API chỉ có một consumer (frontend), việc đổi tên rất dễ dàng. Tuy nhiên, khi phục vụ từ ba consumer trở lên (ứng dụng di động, bên thứ ba, dịch vụ nội bộ), việc đổi tên đồng nghĩa với một breaking change.
2. **Error format chuẩn hóa từ đầu** — `GlobalExceptionHandler` với `@ControllerAdvice` giải quyết phần lớn các trường hợp lỗi phổ biến. Trong KBM, handler tồn tại nhưng Gateway xử lý lỗi JWT riêng (vì được phát triển độc lập) → hai format khác nhau.
3. **API versioning chỉ cần khi có nhiều consumer** — với team sở hữu cả provider và consumer, implicit versioning (deploy cùng nhau) là đủ. Không nên phức tạp hóa hay thiết kế quá mức.

Ở phía tiêu thụ, cách client xử lý dữ liệu trả về cũng quyết định độ ổn định của hệ thống.

Tolerant Reader

Martin Fowler đề xuất pattern "Tolerant Reader": consumer nên bỏ qua các trường không biết và chỉ đọc những trường cần thiết. Nhờ consumer chịu được phiên bản mới của provider (forward-compatible), provider có thể thêm trường mới mà không gây lỗi (breaking) cho các consumer — nhìn từ phía provider, đó chính là **tính tương thích ngược (backward compatibility) tự nhiên** [W6].

3.9.3. Phục vụ nhiều loại client và tích hợp judge ngoài — bài học mở rộng

Phần case study ở trên tập trung vào API giữa các service nội bộ. Nhưng KBM còn đặt ra hai câu hỏi API design mà nhiều hệ thống thực tế phải trả lời: khi backend phục vụ *nhiều loại client khác stack*, và khi nên *tự thiết kế API* hay *chấp nhận hợp đồng của một dịch vụ ngoài*.

Một backend, nhiều loại client

KBM có hai client tiêu thụ **cùng một backend và cùng một hệ định danh** nhưng phục vụ hai ngữ cảnh khác nhau:

- `kbm-web` **(React/TypeScript)**: client luyện tập hàng ngày, chạy trên trình duyệt bất kỳ, nộp bài ở chế độ luyện tập.
- `kbm-exam-client` **(desktop thick client)**: client thi trên phòng máy, siết chặt môi trường thi (lockdown), chỉ kết nối từ IP phòng thi.

Cả hai dùng chung REST API và cùng JWT do `kbm-auth` cấp (SSO) — *không* có "API riêng cho desktop". Đây là một nguyên tắc thiết kế quan trọng: API là hợp đồng **trung lập với client**. Khi contract ổn định và tách khỏi chi tiết client (nhờ DTO, chính sách versioning, error format chuẩn ở §3.1–3.4), việc thêm một loại client mới — desktop, mobile, hay third-party — không buộc phải thiết kế lại backend. Khác biệt giữa web và desktop (lockdown, ràng IP, tín hiệu proctoring) nằm ở *client* và ở phần kiểm soát truy cập phía server, không phải ở hình dạng API.

API trung lập với client

Một API tốt không "biết" ai đang gọi nó. `kbm-exam-client` (desktop) và `kbm-web` (React) gửi cùng request, cùng JWT, nhận cùng response — backend chỉ thấy một định danh hợp lệ và một hành động. Khi các máy khách không đồng nhất (heterogeneous client) chia sẻ chung một hợp đồng và hệ định danh (identity), chi phí thêm máy khách mới gần như chỉ là chi phí viết máy khách đó. Ngược lại, nếu mỗi máy khách có một API riêng, số hợp đồng (contract) phải bảo trì tăng theo số loại máy khách.

API bộ chấm code: tự thiết kế vs tích hợp hợp đồng OSS

Module chấm bài lập trình đa ngôn ngữ (`kbm-judge-code`) minh họa một lựa chọn API *ngược* với phần còn lại của chương. Thế hệ đầu (Gen 1) là bộ chấm tự viết cho C/C++ và Java — nhóm tự định nghĩa toàn bộ API surface: `JudgeRequest`/`JudgeResult` riêng, format verdict riêng, hợp đồng do nhóm phát triển tự kiểm soát hoàn toàn. Thế hệ hiện tại (Gen 2) tích hợp **DOMjudge** — một online judge mã nguồn mở chuẩn thi ICPC.

Điểm mấu chốt về API design: khi áp dụng DOMjudge, KBM **không còn tự thiết kế API chấm code** mà phải tiêu thụ hợp đồng API *đã có sẵn* của DOMjudge (các endpoint REST submit/judging, định dạng verdict, kênh trả kết quả). Bài toán đổi từ "thiết kế một API tốt" sang "tích hợp API của người khác cho tốt": tiến hành ánh xạ (map) hợp đồng từ hệ thống bên ngoài về các DTO nội bộ, đặt một adapter ở ranh giới (boundary) để che giấu chi tiết DOMjudge khỏi phần còn lại của hệ thống (Anti-Corruption Layer — xem §2.3 và Chương 10), và đảm bảo consumer nội bộ chỉ nhận được một `JudgeResult` chuẩn hóa. Đây là một quyết định **"xây hay mua" (build vs buy)** điển hình; khía cạnh giao tiếp đồng bộ và bảo mật/sandbox được phân tích tiếp ở Chương 4 và Chương 9.

3.10. Tổng kết

Thiết kế API chuẩn mực đảm bảo sự ổn định cho toàn bộ kiến trúc microservices.

API versioning và schema evolution là bài toán không thể tránh khỏi khi hệ thống phát triển. Bốn chiến lược versioning (URL path, query param, header, media type) mỗi cách có trade-off riêng — URL path phổ biến nhất vì thân thiện với lập trình viên. Quy tắc tương thích ngược giúp thay đổi API mà không làm consumer gặp lỗi.

OpenAPI/Swagger biến tài liệu thành mã nguồn: tự động, luôn cập nhật, và thử được trực tiếp. DTO pattern tách internal model khỏi external contract — điều kiện cần để quá trình tiến hóa API không gắn chặt vào lược đồ cơ sở dữ liệu. Mở rộng nguyên tắc tách biệt đó lên toàn service, Hexagonal Architecture (Ports & Adapters) cô lập lõi nghiệp vụ khỏi chi tiết giao thức và lưu trữ bên ngoài.

Contract Testing (Pact, Spring Cloud Contract) đưa khái niệm "API là hợp đồng" lên một tầng cao hơn: consumer định nghĩa kỳ vọng, provider xác minh trước khi deploy — giúp phát hiện sớm các breaking change ở CI thay vì trên production. Bên cạnh REST cho giao tiếp liên dịch vụ và API chuẩn hóa, GraphQL cung cấp lựa chọn bổ trợ hiệu quả cho tầng hướng máy khách (BFF, dashboard tổng hợp) khi cần truy vấn linh hoạt, miễn là kiểm soát tốt bài toán N+1 bằng DataLoader.

Phân tích KBM cho thấy: thiết kế API "đủ tốt" là hoàn toàn khả thi cho nhóm nhỏ, nhưng cần ý thức chuẩn hóa từ sớm (quy tắc đặt tên, định dạng lỗi, chính sách phiên bản) để tránh tích lũy nợ kỹ thuật.

3.11. Đọc thêm

Sách tham khảo chính:

1. [1] Thomas Erl, *Service-Oriented Architecture: Analysis and Design for Services and Microservices*, 2nd Ed. — Ch.7: Service API Design, Service Contract
2. [2a] Chris Richardson, *Microservices Patterns*, 1st Ed. — Ch.3: Interprocess Communication, REST API Design
3. [2b] Chris Richardson, *Microservices Patterns*, 2nd Ed. — Ch.3: Architecture Fundamentals, Hexagonal Architecture; Ch.13: Service Design with Ports & Adapters
4. [4a] Sam Newman, *Building Microservices* — Ch.4: Integration, REST best practices
5. [7] Martin Kleppmann, *Designing Data-Intensive Applications* — Ch.4: Encoding and Evolution, Schema Evolution

Nguồn trực tuyến:

- [W6] Martin Fowler, "TolerantReader" — martinfowler.com/bliki/TolerantReader.html
- Leonard Richardson, "Richardson Maturity Model" — martinfowler.com/articles/richardsonMaturityModel.html
- OpenAPI Specification 3.1 — spec.openapis.org
- Pact documentation — docs.pact.io (Consumer-Driven Contract Testing)
- Spring Cloud Contract — docs.spring.io/spring-cloud-contract

Bài tập: 3-0, 3-1, 3-2 ở phần Bài tập & case study cuối sách.

PHẦN II

Giao tiếp & dữ liệu

Khi hệ thống đã tách thành nhiều dịch vụ, các dịch vụ phải giao tiếp và chia sẻ dữ liệu với nhau. Phần II trình bày giao tiếp đồng bộ và bất đồng bộ, giao dịch phân tán bằng Saga, và quản lý dữ liệu theo database-per-service và CQRS.

Chương 4 Giao tiếp đồng bộ — REST, gRPC & Resilience
Chương 5 Giao tiếp bất đồng bộ — Kafka, Events & Messaging
Chương 6 Giao dịch phân tán — Saga Pattern
Chương 7 Quản lý dữ liệu — Database per Service & CQRS

CHƯƠNG 4

Giao tiếp đồng bộ — REST, gRPC & Resilience

> "Smart endpoints and dumb pipes."
> — James Lewis & Martin Fowler, *Microservices* (2014)

MỤC TIÊU HỌC TẬP

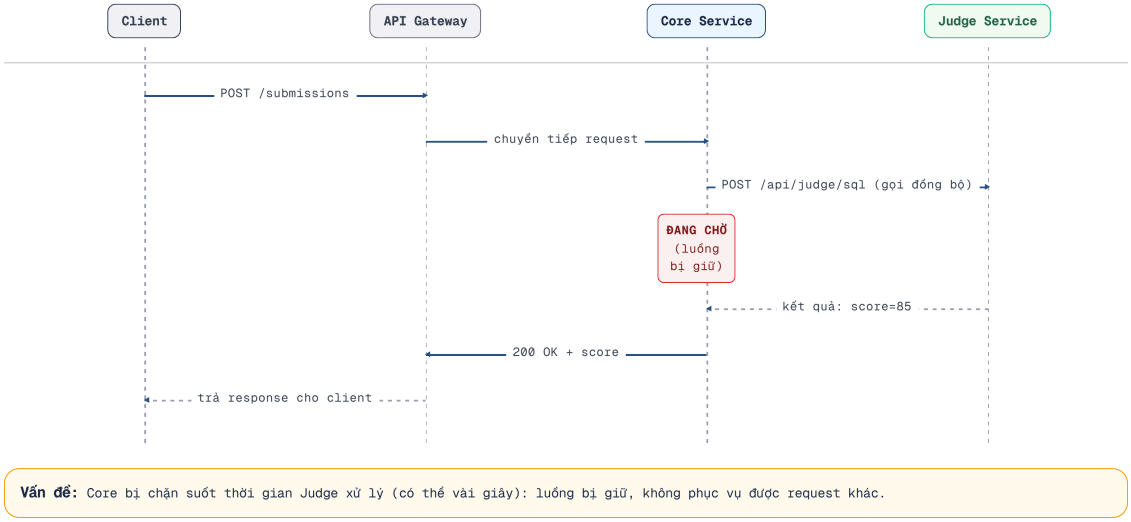
- Hiểu các kiểu tương tác (interaction styles) trong microservices
- So sánh REST và gRPC — khi nào dùng cái nào
- Sử dụng OpenFeign để thực hiện các cuộc gọi liên dịch vụ (service-to-service) theo mô hình khai báo (declarative)
- Nắm vững Service Discovery và Load Balancing với Eureka
- Áp dụng các resilience pattern: Circuit Breaker, Retry, Timeout, Bulkhead
- Phân tích bài toán dispatch SQL và tích hợp ngoài trong KBM

4.1. Giao tiếp đồng bộ — khi nào nên, khi nào tránh?

Trước khi phân tích chi tiết, hãy hình dung giao tiếp đồng bộ giống như việc sinh viên lên thẳng Phòng Đào tạo để nộp đơn và phải đứng đợi cho đến khi chuyên viên xử lý xong mới được ra về. Sự chờ đợi này đảm bảo tính chắc chắn, nhưng lại gây tắc nghẽn khi có quá đông người.

4.1.1. Request-Response: mô hình quen thuộc

Giao tiếp đồng bộ (*synchronous communication*) là mô hình tương tác cơ bản và quen thuộc nhất đối với hầu hết các nhà phát triển phần mềm. Trong mô hình này, khi dịch vụ A cần thông tin hoặc yêu cầu xử lý từ dịch vụ B, nó sẽ gửi một yêu cầu (request) qua mạng, bị chặn luồng (block) và phải **chờ đợi** cho đến khi nhận được phản hồi (response) trả về. Chỉ khi có kết quả từ dịch vụ B, dịch vụ A mới tiếp tục thực thi các logic nghiệp vụ tiếp theo của nó. HTTP, thường theo phong cách REST, hiện là cách phổ biến nhất để hiện thực hóa mô hình này nhờ tính đơn giản, dễ tích hợp và khả năng khai thác hệ sinh thái công cụ phong phú. Tuy nhiên, đằng sau sự đơn giản đó là những chi phí ngầm về độ trễ và sự phụ thuộc chặt chẽ về mặt thời gian giữa các thành phần hệ thống.



Hình 4.1: *Giao tiếp đồng bộ — Core Service bị chặn (blocked) trong khi chờ Judge phản hồi*

4.1.2. Ưu và nhược điểm

Bảng dưới đây tổng hợp các ưu và nhược điểm cốt lõi của giao tiếp đồng bộ:

Bảng 4.1: *Ưu và nhược điểm của giao tiếp đồng bộ*

Khía cạnh	Ưu điểm	Nhược điểm
Đơn giản	Mô hình request-response quen thuộc, dễ gỡ lỗi (debug)	Bên gọi (caller) bị chặn (**block**) hoặc đình trệ cho đến khi nhận được phản hồi (response)
Nhất quán	Nhận kết quả trực tiếp khi có phản hồi; timeout có thể để kết quả nghiệp vụ chưa xác định	**Temporal coupling**: cả hai dịch vụ phải khả dụng (available) cùng lúc
Tooling	HTTP tooling phong phú (Postman, curl, browser)	**Cascading failure**: service B ngừng hoạt động → A cũng ngừng hoạt động
Tracing	Ngữ cảnh truy vết (`traceparent` / `traceId`) truyền tuần tự qua HTTP header theo luồng lời gọi trực tiếp (xem Chương 11)	**Độ trễ cộng dồn**: A→B→C = tổng độ trễ

Richardson phân tích trong [2a, Ch.3] rằng giao tiếp đồng bộ làm giảm tính sẵn sàng: dịch vụ gọi không phản hồi được nếu dịch vụ được gọi ngừng hoạt động. Hiện tượng này thường được gọi là **temporal coupling** (cả hai dịch vụ phải sẵn sàng cùng lúc); bản thứ hai của sách gọi là **runtime coupling** [2b, Ch.4]. Đây là lý do kiến trúc microservices thường ưu tiên giao tiếp bất đồng bộ (async) cho các luồng xử lý không yêu cầu phản hồi tức thời.

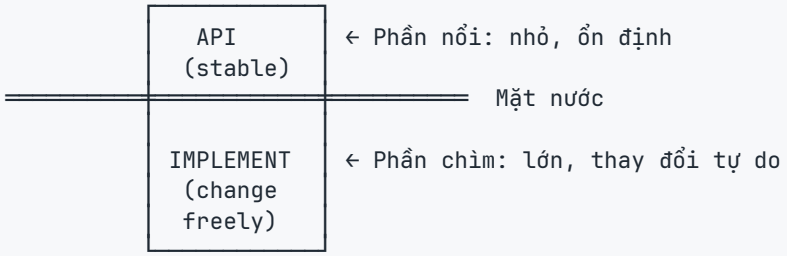
4.1.3. Design-time Coupling vs Runtime Coupling

Richardson trong phiên bản thứ hai [2b, Ch.4] phân biệt bốn loại coupling: design-time, build-time, runtime và infrastructure. Khi chọn kiểu giao tiếp, hai loại cần tách bạch nhất là design-time và runtime; hiểu sai sự khác biệt giữa chúng dễ dẫn đến thiết kế microservices kém:

__Bảng 4.2:__ Hai loại coupling trong microservices

	Design-time Coupling	Runtime Coupling
Định nghĩa	Thay đổi dịch vụ A → **phải** thay đổi dịch vụ B	Dịch vụ A **cần** B đang chạy để xử lý yêu cầu (request)
Khi nào xảy ra	Khi thiết kế và sửa mã	Khi hệ thống đang chạy (runtime)
Ảnh hưởng	Giảm tính tự chủ của nhóm (team autonomy), tăng sự phụ thuộc/phối hợp	Giảm tính sẵn sàng, tăng độ trễ
Ví dụ KBM	Core Service đổi DTO câu hỏi → Academic Service phải sửa phần đọc	Core Service gọi sync Judge → Judge ngừng hoạt động → Core trả lỗi
Giải pháp	API versioning (§3.2), backward compatible changes	Async messaging (Ch.5), Circuit Breaker (§4.4)

Iceberg Principle — Richardson gọi đây là nguyên tắc nền tảng cho loose coupling [2b, Ch.4]:



API (phần nổi) phải nhỏ nhất có thể — chỉ bộc lộ (expose) những gì thành phần gọi/sử dụng dịch vụ (consumer) thực sự cần. Implementation (phần chìm) có thể lớn và thay đổi tự do mà không làm ảnh hưởng đến các consumer này. **Đây cũng là lý do dùng DTO pattern** (§3.4) thay vì trả entity trực tiếp — entity là "phần chìm" không nên lộ ra.

Nghịch lý DRY và Coupling

Trong monolith, DRY (Don't Repeat Yourself) gần như luôn đúng. Nhưng trong microservices, DRY có thể tạo coupling không mong muốn. Ví dụ: Lớp `ScoreCalculator` (quy tắc tính điểm) được sử dụng chung thông qua một thư viện chia sẻ (shared library). Khi đó, mỗi lần cập nhật quy tắc nghiệp vụ (business rule), tất cả các dịch vụ phụ thuộc vào thư viện này đều phải được nâng cấp đồng loạt (lock-step deployment). Đôi khi **duplication có chủ đích** tốt hơn coupling — đặc biệt khi logic ở mỗi context có lý do để đi theo hướng riêng. Richardson trong [2b, Ch.4] khuyên: chỉ áp dụng nguyên tắc DRY khi việc các bản triển khai khác nhau thực sự gây ra lỗi (**bug**), không nên áp dụng một cách khiên cưỡng trong mọi trường hợp.

4.1.4. Khi nào dùng sync?

Giao tiếp đồng bộ phù hợp khi:

- **Cần phản hồi ngay**: query data (GET), validation, authentication
- **Flow đơn giản**: chuỗi gọi dịch vụ ngắn (A→B), không phải chuỗi dài (A→B→C→D)
- **Các thao tác nghiêng về truy xuất dữ liệu (Read-heavy operations)**: lấy thông tin, không phải thay đổi trạng thái phức tạp

Không phù hợp khi:

- **Các tác vụ thực thi kéo dài (Long-running operations)**: chấm bài SQL mất 5–30 giây
- **Tác vụ gửi-và-quên (Fire-and-forget)**: gửi thông báo (notification), ghi nhận nhật ký hệ thống (log event)
- **Giao dịch liên dịch vụ (Cross-service transactions)**: cần saga pattern (Chương 6)

Một nguyên tắc kinh nghiệm đơn giản: nếu bên gọi (caller) *phải biết* kết quả để tiếp tục xử lý → chọn giao tiếp đồng bộ; nếu bên gọi chỉ cần *kích hoạt* hành động và không cần đợi kết quả → chọn giao tiếp bất đồng bộ (Chương 5). Trong hệ thống KBM, việc truy vấn trạng thái hoạt động (health query) của Judge sử dụng cơ chế đồng bộ, nhưng việc gửi yêu cầu chấm bài (submit) lại sử dụng cơ chế bất đồng bộ thông qua Kafka — phản ánh hai nhu cầu hoàn toàn khác biệt.

4.1.5. Interaction Styles — phân loại kiểu tương tác

Richardson phân loại các kiểu giao tiếp trong microservices dựa trên hai không gian chính: số lượng bên tiếp nhận (one-to-one so với one-to-many) và tính đồng bộ (synchronous so với asynchronous) [2a, Ch.3]. Trong khi giao tiếp đồng bộ thường bị giới hạn ở kiểu một-một (như Request/Response quen thuộc), thì giao tiếp bất đồng bộ cung cấp tính linh hoạt cao hơn. Nó không chỉ hỗ trợ gọi một-một (như thông báo một chiều hay request/response bất đồng bộ), mà còn đặc biệt hiệu quả ở mô hình một-nhiều (như Publish/Subscribe hay Publish/Async responses) nơi một thông điệp có thể kích hoạt hàng loạt dịch vụ cùng lúc.

Hầu hết service dùng kết hợp nhiều kiểu. Ví dụ: KBM Core Service dùng *request/response* (Feign) để truy vấn hệ thống chấm điểm (Judge), *publish/subscribe* (Kafka) để gửi bài nộp (submission), và *one-way notification* (WebSocket) để đẩy/trả kết quả.

4.1.6. REST vs gRPC — hai lựa chọn cho sync

Khi quyết định sử dụng giao tiếp đồng bộ, REST (dựa trên HTTP và JSON) thường là lựa chọn mặc định. Bên cạnh REST, **gRPC** — một framework RPC mã nguồn mở hiệu năng cao do Google phát triển — là giải pháp phổ biến cho giao tiếp nội bộ giữa các microservice đòi hỏi tốc độ cao, độ trễ thấp và băng thông được tối ưu bằng định dạng nhị phân. Bảng sau so sánh chi tiết hai công nghệ này:

Bảng 4.3: *REST vs gRPC — so sánh chi tiết*

Tiêu chí	REST (HTTP/JSON)	gRPC (HTTP/2 + Protobuf)
Format	JSON (text, human-readable)	Protocol Buffers (binary, compact)
Performance	Chậm hơn (text parsing)	Thường nhanh hơn đáng kể, nhất là với khối lượng dữ liệu (payload) lớn / cần nhiều lượt đi-về (round-trip) (binary + HTTP/2 multiplexing); mức cải thiện tùy thuộc vào tải hệ thống (workload)
Contract	OpenAPI spec (optional)	`.proto` file (bắt buộc)
Code gen	Optional (OpenAPI generators)	Built-in (protoc generates client/server)

Tiêu chí	REST (HTTP/JSON)	gRPC (HTTP/2 + Protobuf)
Browser support	Native	Cần gRPC-Web proxy
Streaming	Không native (phải dùng SSE/WebSocket)	Bi-directional streaming native
Debugging	Dễ dàng (hỗ trợ curl, Postman, trình duyệt)	Phức tạp (cần gRPC tools)

gRPC Architecture — Tại sao nhanh hơn? gRPC khác REST ở ba yếu tố kiến trúc; hai yếu tố đầu giải thích vì sao nó thường nhanh hơn, yếu tố thứ ba mở thêm kiểu giao tiếp:

1. **HTTP/2 multiplexing** — Nhiều yêu cầu (request) chạy đồng thời trên cùng một kết nối TCP, không phải chờ nhau như trên HTTP/1.1. Quan trọng khi dịch vụ A gọi dịch vụ B hàng trăm lần/giây.
2. **Protocol Buffers** — Schema-first, binary encoding. Payload thường nhỏ hơn JSON đáng kể — tỉ lệ phụ thuộc dữ liệu: chênh nhiều với payload nhiều số/enum, thu hẹp với payload chủ yếu là chuỗi văn bản hoặc khi JSON đã nén gzip. Schema evolution built-in (backward/forward compatible) — `.proto` file *là* contract.
3. **Four communication patterns**: Unary (request-response, như REST), Server streaming (server gửi stream), Client streaming (client gửi stream), **Bi-directional streaming** (cả hai gửi stream đồng thời). **Lưu ý quan trọng:** Bi-directional streaming rất hiệu quả cho backend-to-backend. Trình duyệt (thông qua gRPC-Web) có hỗ trợ Server-Streaming, nhưng chưa hỗ trợ Bi-directional streaming. Dù có thể dùng Server-streaming để trả kết quả (ví dụ: stream kết quả từng test case cho trình duyệt của sinh viên), WebSocket hoặc SSE (Server-Sent Events) vẫn là chuẩn phổ biến hơn cho real-time frontend.

Lưu ý về Load Balancing: Vì gRPC duy trì kết nối HTTP/2 dài hạn (persistent connection), các Load Balancer L4 thông thường (phân phối theo kết nối TCP) sẽ chia tải bị lệch. Để cân bằng tải trên từng request, hãy dùng L7 Load Balancer (như Envoy, HAProxy) hoặc Service Mesh; một lựa chọn khác là client-side load balancing qua service discovery khi client nội bộ đáng tin cậy. L4 vẫn dùng được cho các thiết lập đơn giản, nhưng phải chấp nhận rủi ro lệch tải theo kết nối.

Khi nào dùng gRPC thay REST?

- **Giao tiếp nội bộ (Internal service-to-service)**: Khi hiệu năng là ưu tiên hàng đầu và không yêu cầu tương tác trực tiếp từ trình duyệt → chọn gRPC
- **API công khai / Giao diện người dùng (Frontend)**: Khi cần sự hỗ trợ từ trình duyệt và dễ dàng gỡ lỗi thủ công → chọn REST
- **Truyền phát dữ liệu (Streaming)**: Khi yêu cầu giao tiếp hai chiều thời gian thực (real-time bi-directional) → chọn gRPC (hoặc WebSocket)
- **Môi trường Polyglot**: đội ngũ phát triển sử dụng nhiều ngôn ngữ lập trình khác nhau → gRPC (code gen đa ngôn ngữ)

KBM không còn là hệ thống REST thuần

LMS core của KBM vẫn ưu tiên REST/JSON vì các trình duyệt (browser), bảng điều khiển (dashboard) và bộ công cụ OpenAPI yêu cầu khả năng gỡ lỗi dễ dàng. Nhưng Network Lab chủ động dùng TCP/UDP/RMI/SOAP/REST/gRPC để sinh viên thấy được sự khác biệt giữa các giao thức (protocol) trong cùng một lĩnh vực (domain) bài tập. Vì vậy, chuyển sang gRPC không phải mục tiêu "đổi toàn bộ REST"; cách đúng hơn là **chọn protocol theo boundary**: Sử dụng REST cho các API hướng người dùng (client-facing), gRPC cho các giao tiếp nội bộ đòi hỏi thông lượng cao hoặc truyền phát dữ liệu (high-throughput/streaming) khi cần thiết, và các hàng đợi lô/

công việc (batch/job queue) cho các tác vụ chấm điểm (judge) kéo dài hoặc mang tính trạng thái (stateful).

4.2. OpenFeign — Declarative REST Client

Thay vì bắt sinh viên tự cầm hồ sơ gõ cửa từng phòng ban, hệ thống một cửa OpenFeign tự động chuẩn hóa mọi thủ tục giao tiếp. Cách tiếp cận này giúp mã gọi dịch vụ nội bộ gọn hơn và ít lặp lại hơn.

4.2.1. Vấn đề: gọi service khác không đơn giản như gọi hàm

Trong kiến trúc nguyên khối (monolith), gọi module khác chỉ là một dòng mã: `judgeService.execute(request)`. Trong microservices, cùng logic đó trở thành: xây dựng HTTP request, tuần tự hóa dữ liệu (serialize payload), gắn header (JWT token), gửi qua mạng, xử lý mã phản hồi (response code), giải tuần tự hóa kết quả (deserialize result) và xử lý lỗi hoặc quá hạn (handle timeout/error). Và điều nguy hiểm nhất: khi gọi hàm nội bộ, nếu có lỗi, chương trình sẽ ném exception ngay lập tức. Nhưng khi gọi qua mạng, hệ thống có thể bị đình trệ vô thời hạn mà không hề ném ra exception nào. Trong hệ thống phân tán, *một phản hồi chậm (slow response) còn nguy hiểm hơn rất nhiều so với một lỗi trả về ngay lập tức (fail-fast)*.

4.2.2. OpenFeign: gọi REST như gọi hàm

Để giải quyết sự cồng kềnh của các HTTP client truyền thống, hệ sinh thái Spring cung cấp **OpenFeign** (Spring Cloud OpenFeign). Công cụ này mang đến một cách tiếp cận mang tính khai báo (declarative), cho phép các lập trình viên chỉ cần định nghĩa các lời gọi API dưới dạng một Java interface thông thường. Framework tự sinh mã thực thi (implementation) lúc runtime cho các tác vụ như thiết lập kết nối, quản lý header và chuyển đổi dữ liệu. Từ Spring Cloud 2022.0.0, OpenFeign chỉ còn được sửa lỗi; tài liệu chính thức gợi ý mã mới dùng Spring HTTP Service Clients (`@HttpExchange`). Sách vẫn dùng OpenFeign vì KBM đang dùng, và cơ chế client khai báo ở hai công cụ là như nhau.

Listing 4.1: *So sánh RestTemplate (viết dài dòng) và OpenFeign (dạng khai báo)*

```
// Cách thủ công: dựng HTTP request và xử lý response
RestTemplate restTemplate = new RestTemplate();
HttpHeaders headers = new HttpHeaders();
headers.setBearerAuth(jwtToken);
HttpEntity<JudgeRequest> entity = new HttpEntity<>(request, headers);
ResponseEntity<JudgeResult> response = restTemplate.exchange(
    url, HttpMethod.POST, entity, JudgeResult.class
);

// Cách khai báo: mô tả API bằng Feign interface
@FeignClient(name = "kbm-judge-sql")
public interface JudgeClient {
    @PostMapping("/api/judge/sql")
    // Gọi như hàm bình thường!
    JudgeResult submitSql(@RequestBody JudgeRequest request);
}
```

Mô hình khai báo của OpenFeign giúp mã nguồn gọn hơn hẳn lối viết thủ công bằng `RestTemplate` và loại bỏ nhiều lỗi tiềm ẩn do cấu hình thiếu sót. Tuy nhiên, sự trừu tượng hóa này cũng có thể vô tình che khuất một số thiết lập cấu hình mạng quan trọng ở tầng dưới, đặc biệt là cách quản lý

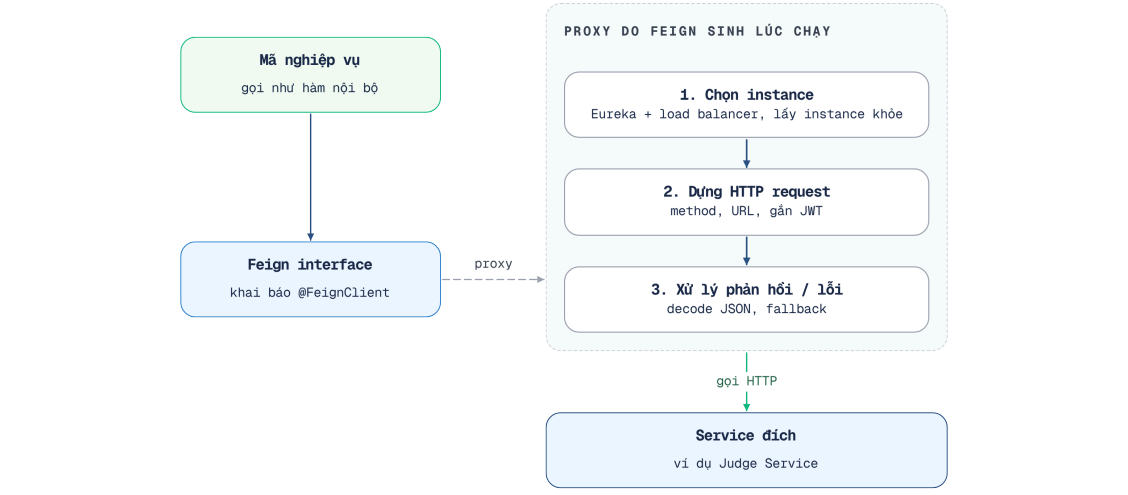
vòng đời của các kết nối TCP mà nếu không chú ý sẽ nhanh chóng trở thành nút thắt cổ chai về hiệu năng của toàn hệ thống.

Cấu hình Connection Pooling cho OpenFeign

Mặc định, khi classpath không có HTTP client khác, OpenFeign sử dụng `HttpURLConnection` của Java — không cung cấp một connection pool được quản lý đúng nghĩa (dù có keep-alive cơ bản, nó không gộp và tái sử dụng kết nối hiệu quả dưới tải cao), dẫn đến suy giảm hiệu năng do chi phí thiết lập kết nối lặp lại. Trên production, đội ngũ phát triển nên cấu hình OpenFeign sử dụng `Apache HttpClient 5` hoặc `OkHttp` để khai thác cơ chế quản lý connection pool, giúp giảm thiểu độ trễ mạng.

4.2.3. Cách hoạt động

Hình dưới đây cho thấy các bước Feign làm thay lập trình viên.




```java
@FeignClient(name = "kbm-judge-sql")
public interface JudgeClient {
  @PostMapping("/api/judge/sql")
  JudgeResult submitSql(@RequestBody JudgeRequest req);
}
```

Hình 4.2: Cách Feign proxy tự động xử lý HTTP request, JWT, load balancing

Feign tự động xử lý: serialization/deserialization JSON sang Java, header injection (ví dụ JWT token qua `RequestInterceptor`), service discovery (tra cứu Eureka thay vì gán cứng URL), và error decoding (chuyển đổi response code thành exception).

4.3. Service Discovery & Load Balancing

Khi quy mô hệ thống trường học mở rộng, việc ghi nhớ vị trí chính xác của từng phòng ban trở nên bất khả thi. Service Discovery hoạt động như một bản đồ điện tử động của toàn trường, giúp các ứng dụng luôn tìm thấy nhau mà không cần cấu hình cứng.

4.3.1. Vấn đề: URL cứng trong distributed system

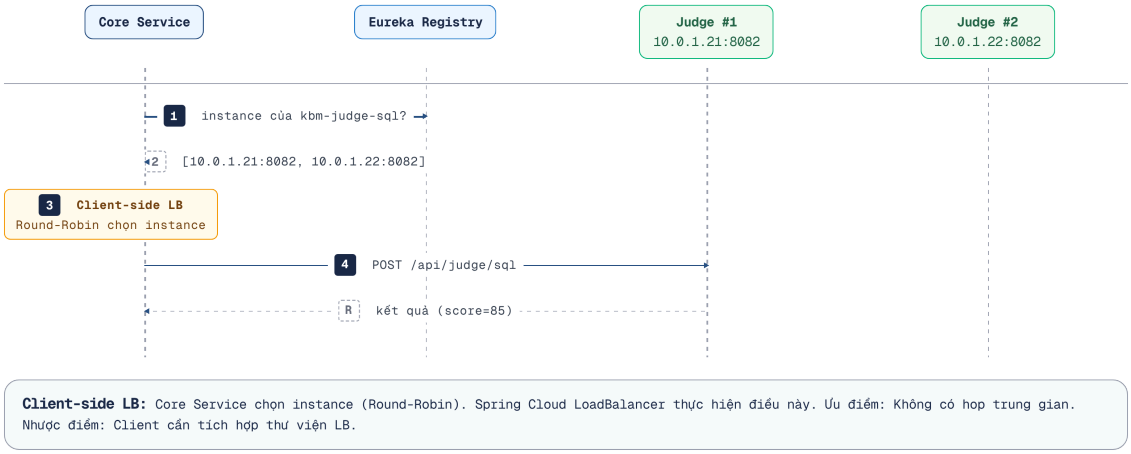
Trong monolith, gọi module khác là gọi hàm — không cần biết "module ở đâu". Trong microservices, mỗi dịch vụ là một tiến trình (process) độc lập, chạy trên các địa chỉ và cổng mạng (host:port) khác nhau. Câu hỏi: **dịch vụ A tìm dịch vụ B ở đâu?**

Cách đơn giản nhất: gán cứng (hardcode) URL trong cấu hình. Nhưng khi dịch vụ mở rộng quy mô (scale) thành 3 instance, hệ thống nên sử dụng URL nào? Khi dịch vụ được triển khai (deploy) sang container mới, địa chỉ IP thay đổi — cơ chế nào sẽ chịu trách nhiệm cập nhật?

4.3.2. Service Discovery patterns

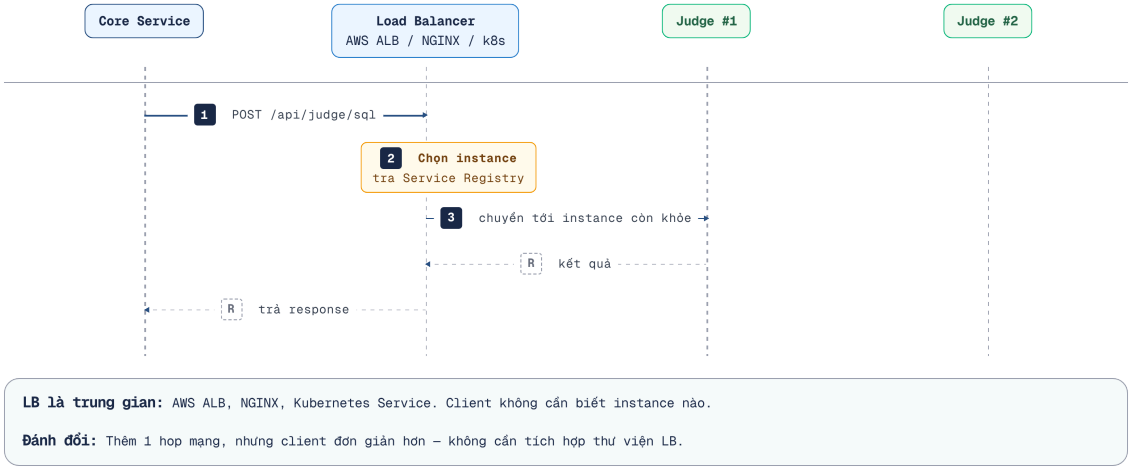
Có hai pattern chính [2a, Ch.3]:

Client-side discovery — Client (dịch vụ gọi) tự truy vấn (query) service registry để lấy danh sách instance, sau đó tự phân tải (load balance):



Hình 4.3: Client-side discovery — service tự truy vấn Eureka và tự cân bằng tải

Server-side discovery — Load balancer (hoặc API Gateway) hoạt động như một thành phần trung gian, client chỉ cần biết URL của load balancer:



Hình 4.4: Server-side discovery — load balancer điều hướng request

4.3.3. Netflix Eureka trong KBM

KBM sử dụng **Netflix Eureka** cho cụm các service Java/Spring — client-side discovery pattern:

Khi microservice khởi động, nó đăng ký tại Eureka server với tên (`spring.application.name`) và địa chỉ. Mặc định, Eureka client chỉ gửi nhịp tim định kỳ (mỗi 30 giây) để báo tiến trình còn sống; muốn đồng bộ trạng thái kiểm tra sức khỏe sâu của Spring Boot Actuator (như kết nối CSDL, ổ đĩa) lên registry thì cần bật tường minh `eureka.client.healthcheck.enabled=true` (xem §11.6). Gateway và các dịch vụ khác sẽ truy vấn (query) Eureka để tìm các instance đang hoạt động. Trong cấu hình (config) Gateway: `uri: lb://kbm-core` nghĩa là tra cứu Eureka tìm tất cả instance đăng ký dưới tên `kbm-core`, rồi phân tải (load balance) lưu lượng giữa chúng. Client giữ bản sao danh sách instance trong bộ nhớ và làm mới định kỳ (mặc định 30 giây), nên khi registry tạm ngừng, các lời gọi đang có vẫn chạy theo danh sách đã lưu; điều này không bảo đảm cho client mới chưa có bản sao, hoặc instance đã đổi địa chỉ.

Service naming không nhất quán

Trong snapshot cũ của KBM, `kbm-core` và `kbm-academic` từng dùng `spring.application.name = "app"` — vi phạm nguyên tắc unique service identity [2a]. Khi Eureka nhận hai service cùng tên, nó coi chúng là các instance của *cùng một dịch vụ* → dẫn đến định tuyến sai. Đây là hậu quả của việc `kbm-academic` tách ra từ Core nhưng không đổi service name. **Lộ trình chuyển đổi (Migration path)**: đổi thành `kbm-core` và `kbm-academic`, cập nhật Eureka config và Gateway routes.

Cần lưu ý rằng Eureka là một *application-level* service discovery — service tự đăng ký và tự tra cứu trong code (qua Spring Cloud). Nếu hệ thống được triển khai trên **Kubernetes**, phần lớn vai trò này đã có sẵn ở tầng hạ tầng: mỗi service là một `Service` object, Kubernetes sẽ đảm nhiệm việc đăng ký, DNS nội bộ (`http://kbm-core`) và load balancing — thường *không cần* Eureka nữa. Nói gọn: dùng Eureka khi chạy ngoài Kubernetes (VM, bare-metal, Spring Cloud thuần); cân nhắc bỏ Eureka khi đã lên Kubernetes để tránh trùng lặp hai lớp discovery. KBM hiện dùng Eureka vì cụm Java/Spring chạy bằng Docker Compose, chưa lên Kubernetes; lộ trình lên k3s (Chương 12) sẽ mở ra lựa chọn chuyển sang discovery Kubernetes-native.

4.4. Resilience Patterns — sống sót trong Distributed System

Môi trường mạng không đáng tin cậy cũng giống như việc mất điện hay rớt mạng cục bộ trong ký túc xá. Các resilience pattern giúp KBM không sụp đổ dây chuyền khi điều đó xảy ra.

4.4.1. Resilience Metrics — đo lường độ tin cậy

Để đánh giá độ tin cậy của một hệ thống có thể khôi phục (repairable system), giới kỹ thuật thường sử dụng bốn thước đo cốt lõi gắn kết chặt chẽ với nhau. Khoảng thời gian trung bình hệ thống vận hành trơn tru kể từ khi sẵn sàng cho đến lúc phát sinh sự cố tiếp theo được gọi là **MTTF (Mean Time To Failure)**. Khi sự cố xảy ra, tốc độ khắc phục được đo bằng **MTTR (Mean Time To Recovery)** — khoảng thời gian trung bình từ lúc hệ thống ngưng trệ đến khi khôi phục hoàn toàn. Tổng thời gian của một chu kỳ đầy đủ giữa hai sự cố liên tiếp chính là **MTBF (Mean Time Between Failures)**, với quan hệ `MTBF = MTTF + MTTR` (ví dụ: một service trung bình mỗi tuần gặp sự cố một lần thì MTBF là 168 giờ). Từ ba đại lượng trên, **Availability (Tính sẵn sàng)** được tính bằng tỷ lệ phần trăm thời gian hệ thống thực sự phục vụ người dùng: `Availability = MTTF / (MTTF + MTTR) = MTTF / MTBF`. Ví dụ: với MTBF 168 giờ và MTTR 1 giờ, Availability = 167 / 168 ≈ 99.4%.

Thuật ngữ "nines" (các số 9) thường được sử dụng trong ngành công nghiệp phần mềm để mô tả nhanh các mức độ khả dụng mục tiêu của một hệ thống. Mỗi "số 9" được thêm vào đại diện cho những thách thức kỹ thuật lớn, đi kèm với sự phức tạp và chi phí vận hành cao hơn. Bảng 4.4 minh họa quy đổi từ các tỷ lệ phần trăm này sang thời gian ngừng hoạt động tối đa cho phép, giúp chúng ta hình dung rõ ràng hơn về mức độ khắt khe của từng cấp độ cam kết dịch vụ.

Bảng 4.4: *Các mức "nines" của tính sẵn sàng — downtime cho phép theo từng mục tiêu*

Mục tiêu	Downtime/năm	Downtime/tháng	Phù hợp cho
99% (two nines)	3.65 ngày	7.3 giờ	Internal tools, dev environments
99.9% (three nines)	8.76 giờ	43.8 phút	**KBM** — mục tiêu cho chế độ thi (tính trên khung giờ từng ca thi, xem Chương 11)
99.99% (four nines)	52.6 phút	4.38 phút	E-commerce, banking
99.999% (five nines)	5.26 phút	26.3 giây	Critical infrastructure

Trong thực tế, các mục tiêu khả dụng này định hình trực tiếp quyết định thiết kế và cách vận hành hằng ngày. Để chúng trở thành cơ chế điều tiết rủi ro và nhịp phát hành phần mềm thay vì chỉ là con số lý thuyết, giới kỹ thuật áp dụng khái niệm **ngân sách lỗi** (error budget).

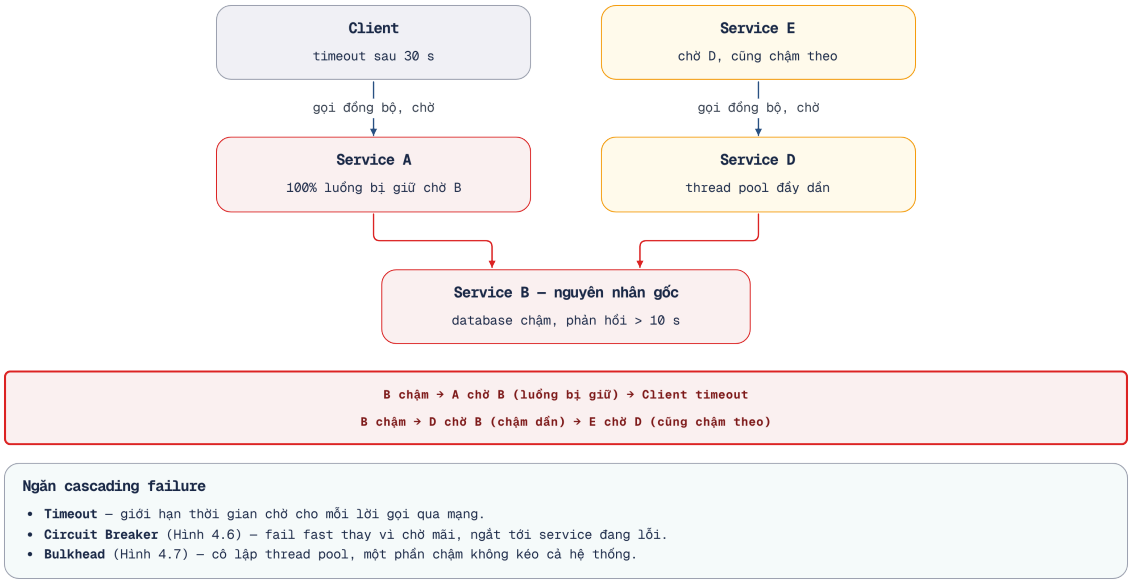
Error Budgets

Google SRE (Site Reliability Engineering) đề xuất: nếu mục tiêu khả dụng đạt 99.9%, hệ thống có **error budget = 0.1%** downtime/tháng (43.8 phút). Ngân sách lỗi này sẽ được dùng để đánh đổi cho các hoạt động như: triển khai phiên bản mới (deployments), thử nghiệm tính năng (experimentation) hoặc bảo trì có kiểm soát (controlled maintenance). Khi cạn kiệt ngân sách lỗi, đội ngũ vận hành sẽ đóng băng việc triển khai (freeze deployments) và phải tập trung hoàn toàn vào việc duy trì tính ổn định của hệ thống. Đây là cách biến mục tiêu khả dụng từ khái niệm mơ hồ thành **nguyên tắc vận hành cụ thể** (xem thêm SLI/SLO/SLA ở Chương 11).

4.4.2. Tại sao cần resilience?

Trong kiến trúc nguyên khối (monolith), nếu cơ sở dữ liệu chậm, *toàn bộ* ứng dụng sẽ chậm theo — nhưng ít nhất lỗi được thể hiện rõ ràng. Trong microservices, khi dịch vụ B chậm, dịch vụ A *vẫn tiếp tục chạy* nhưng các luồng xử lý (thread) bị chặn lại để chờ B → các yêu cầu (request) gửi đến A cũng bị chậm lại → client gọi A cũng bị chậm → tạo thành lỗi dây chuyền (cascading failure) lan khắp hệ thống [5, Ch.7]. Hiện tượng này được gọi là **Thread Pool Exhaustion** (Cạn kiệt luồng). Hãy tưởng tượng Web Server (như Tomcat) là một Phòng Giáo vụ chỉ có giới hạn 200 chuyên viên (200 worker threads). Nếu một dịch vụ phía sau (dịch vụ B) bị chậm, toàn bộ 200 chuyên viên này sẽ phải đứng chờ, dẫn đến việc các sinh viên mới (request mới) bước vào Phòng Giáo vụ sẽ không có ai phục vụ, khiến hệ thống tê liệt hoàn toàn.

Hugo Rocha trong [5] nhấn mạnh: trong distributed system, *lỗi không phải ngoại lệ — lỗi là trạng thái bình thường*. Mạng sẽ quá hạn kết nối (timeout), các dịch vụ sẽ ngưng hoạt động đột ngột (crash), và cơ sở dữ liệu sẽ phản hồi chậm. Câu hỏi không phải "nếu lỗi xảy ra" mà là "khi lỗi xảy ra".



Hình 4.5: *Cascading failure — Service B chậm làm A, D, E đều bị ảnh hưởng*

4.4.3. Bốn resilience pattern cốt lõi

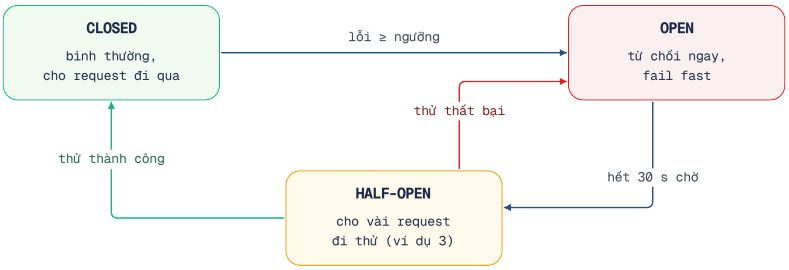
Để ngăn chặn lỗi dây chuyền (cascading failure), hệ thống phân tán áp dụng bốn cơ chế bảo vệ (resilience pattern) kinh điển: **Timeout** cắt những kết nối quá hạn để giải phóng tài nguyên; **Retry** thử lại trước lỗi mạng chập chờn tạm thời; **Circuit Breaker** ngắt mạch khi dịch vụ phía sau lỗi kéo dài; và **Bulkhead** giam rủi ro cạn kiệt tài nguyên vào những khu vực (thread pool) biệt lập để lỗi không lan rộng.

Timeout — giới hạn thời gian chờ

Timeout là tuyến phòng thủ đầu tiên và bắt buộc cho mọi lời gọi qua mạng: nếu không đặt ngưỡng chờ tối đa, một dịch vụ hạ nguồn bị treo sẽ giữ chặt luồng của bên gọi cho đến khi cạn kiệt tài nguyên. Khi cấu hình HTTP client, cần phân biệt hai loại ngưỡng: **connect timeout** (thời gian tối đa để thiết lập kết nối TCP, thường ngắn — vài trăm mili-giây đến 1 giây) và **read/response timeout** (thời gian chờ dịch vụ đích xử lý và trả dữ liệu về, đặt dựa trên phân vị độ trễ P99 đo được). Một bất biến kiến trúc cần nhớ là *timeout ở phía gọi chỉ giải phóng luồng chờ của caller, không tự động hủy tác vụ đang chạy ở phía downstream* — nếu downstream vẫn tiếp tục ghi dữ liệu rồi bên gọi lại retry, hệ thống rất dễ sinh bản ghi trùng nếu thiếu idempotency key. Trong chuỗi gọi nhiều tầng, timeout của tầng ngoài phải lớn hơn timeout của tầng trong; nếu ngược lại, tầng ngoài bỏ cuộc trong khi tầng trong vẫn đang làm, và kết quả đó bị bỏ phí.

Circuit Breaker — chi tiết state machine

Trọng tâm của cơ chế Circuit Breaker là một mô hình trạng thái (state machine), liên tục quan sát và chuyển đổi để quyết định có nên cho phép một yêu cầu mạng được gửi đi hay không.



Cấu hình Resilience4j (Listing 4.3)

`slidingWindowSize` = 10 | `failureRateThreshold` = 50% | `waitDurationInOpenState` = 30s | `permittedNumberOfCallsInHalfOpenState` = 3

KBM: `@CircuitBreaker` trên `JudgeCallService` khi Core gọi Judge (Listing 4.2)

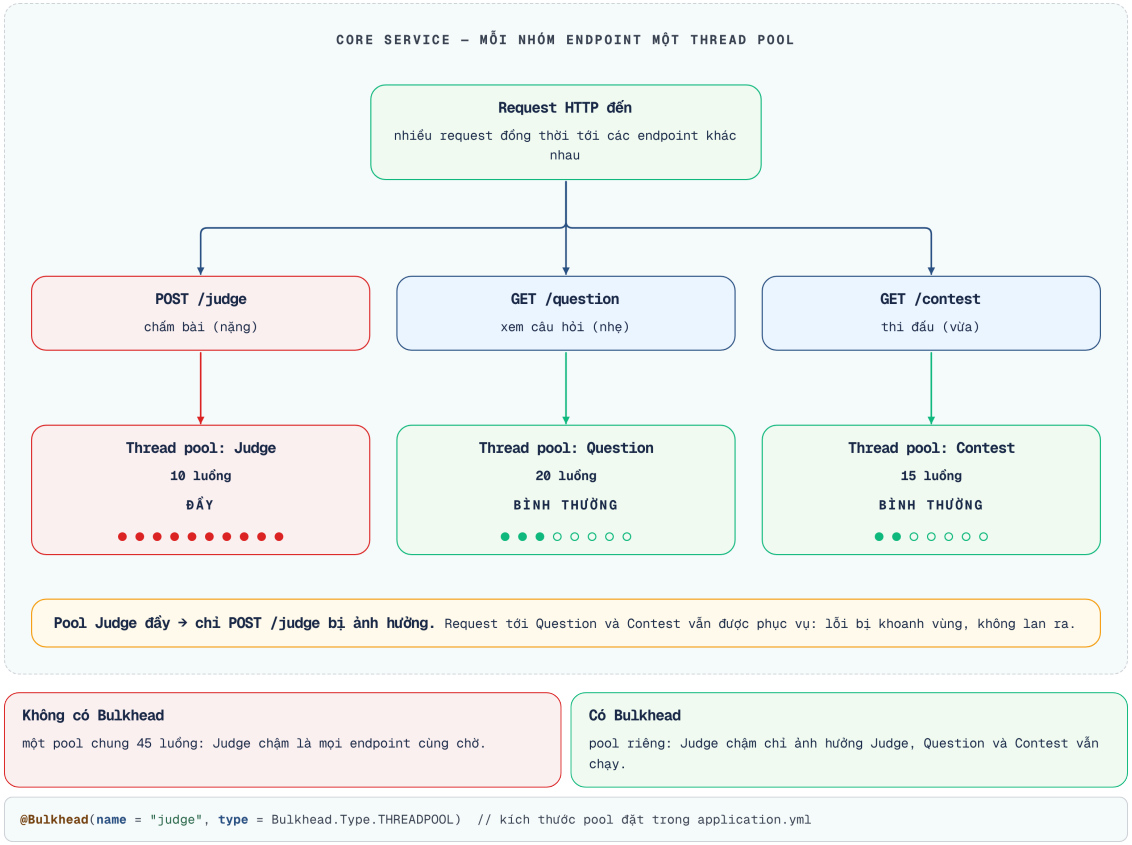
Hình 4.6: *Circuit Breaker state machine — Closed, Open, Half-Open*

Khi mạch ở trạng thái **Open**, mọi yêu cầu bị từ chối (reject) *ngay lập tức* và không được gửi tới dịch vụ đích (downstream). Cơ chế này bảo vệ cả hệ thống gọi (caller) khỏi tình trạng bị chặn luồng (thread blocking), và hệ thống được gọi (callee) khỏi sự quá tải (overwhelm) trong quá trình phục hồi (recovery).

Nhưng làm sao biết khi nào hệ thống đã phục hồi? Đó là lúc trạng thái **Half-Open** (mở một phần) phát huy tác dụng. Sau một khoảng thời gian chờ, Circuit Breaker sẽ chuyển sang Half-Open và cho phép một số lượng request giới hạn (ví dụ: 3 request) lọt qua để "thử nghiệm lưu lượng". Nếu các request này thành công, Circuit Breaker tự động đóng lại (Closed) và hệ thống hoạt động bình thường. Nếu thất bại, nó lập tức ngắt lại (Open) để tiếp tục chờ đợi.

Bulkhead — cô lập tài nguyên

Cơ chế tiếp theo trong hệ thống bảo vệ là Bulkhead, chuyên tập trung vào việc cô lập phạm vi thiệt hại khi một dịch vụ trong hệ thống bắt đầu bộc lộ sự chậm trễ bất thường. Nó thực hiện việc cô lập này bằng cách phân chia tài nguyên hệ thống — cụ thể là cấp phát các thread pool hoặc giới hạn quota (số lượng request đồng thời) riêng biệt cho từng nhóm nghiệp vụ hoặc từng API gọi ra ngoài. Thay vì sử dụng chung một thread pool khổng lồ cho mọi request, Bulkhead đảm bảo rằng nếu một dịch vụ phản hồi chậm và chiếm dụng toàn bộ các luồng xử lý (hiện tượng thread pool exhaustion), thì rủi ro cục bộ này sẽ chỉ bị khoanh vùng trong chính thread pool được cấp cho nó. Các thread pool khác vẫn còn nguyên tài nguyên để phục vụ những chức năng không liên quan — lỗi được khoanh vùng thay vì lan ra toàn hệ thống.



Hình 4.7: Bulkhead pattern — thread pool cô lập theo từng chức năng

Tên "Bulkhead" lấy từ thiết kế tàu thủy: khoang tàu bị nước tràn vào, các khoang khác vẫn khô — tàu không chìm [5, Ch.7]. Michael Nygard giới thiệu pattern này trong *Release It!* (2007).

Retry — thử lại có kiểm soát

Chỉ thử lại (retry) các thao tác bảo đảm idempotency (GET, PUT, DELETE). Việc thử lại các thao tác POST có thể tạo ra dữ liệu trùng lặp (duplicate data). Muốn retry POST thì request phải mang idempotency key để bên nhận nhận ra và bỏ qua lần gửi lặp. Ngoài ra, không bao giờ thử lại tức thì liên tiếp: phải giãn cách thời gian chờ theo cấp số nhân (**exponential backoff**, ví dụ 200 ms → 400 ms → 800 ms) kết hợp độ lệch ngẫu nhiên (**jitter**) để hàng trăm client không đồng loạt dội lại yêu cầu vào đúng một thời điểm (hiện tượng *retry storm* / *thundering herd*).

Kết hợp các pattern

Trong thực tế, các pattern thường kết hợp theo thứ tự:

```
Request → Retry (3 lần, backoff) → Circuit Breaker → Timeout (5s)
        → Bulkhead → Actual Call
```

Chuỗi kết hợp trên tạo ra một quy trình kiểm soát lỗi đa tầng. Khi một yêu cầu mạng được gửi đi, nó phải vượt qua từng chốt kiểm tra này trước khi thực sự chạm đến hệ thống đích.

Spring Cloud Circuit Breaker với Resilience4j là lựa chọn phổ biến cho Java microservices: logic retry và circuit breaker được khai báo bằng annotation và cấu hình, không phải viết tay.

4.4.4. Triển khai thực tế với Resilience4j — mã ví dụ cho LMS

Đoạn mã dưới đây minh họa cách áp dụng cho Core Service của KBM — đây là đích của Giai đoạn 3 trong lộ trình Chương 10, chưa phải mã đang chạy. Spring Cloud Circuit Breaker kết hợp cùng Resilience4j cung cấp một bộ công cụ cho phép tích hợp cả bốn cơ chế bảo vệ thông qua các annotation. Cách tiếp cận này tách biệt logic nghiệp vụ khỏi logic xử lý lỗi và quản lý độ tin cậy.

Listing 4.2: *Resilience4j annotations cho Judge Feign Client*

```
// Core Service — gọi Judge với đầy đủ các resilience pattern
@Service
public class JudgeCallService {
    private static final Logger log =
            LoggerFactory.getLogger(JudgeCallService.class);
    private final JudgeClient judgeClient;
    private final JudgeRetryQueue judgeRetryQueue;

    public JudgeCallService(JudgeClient judgeClient,
                            JudgeRetryQueue judgeRetryQueue) {
        this.judgeClient = judgeClient;
        this.judgeRetryQueue = judgeRetryQueue;
    }

    // Thứ tự khai báo annotation không ảnh hưởng đến luồng thực thi (execution)
    // — Spring AOP sử dụng @Order của aspect
    // Thứ tự thực thi thực tế:
    // Retry → CircuitBreaker → TimeLimiter → Bulkhead → call
    // Retry an toàn dù đây là POST: JudgeRequest mang submissionId
    // làm idempotency key (§4.4)
    @Retry(name = "judge", fallbackMethod = "judgeFallback")
    @CircuitBreaker(name = "judge")
    @TimeLimiter(name = "judge")
    @Bulkhead(name = "judge")
    public CompletableFuture<JudgeResult> executeSQL(JudgeRequest request) {
        return CompletableFuture.supplyAsync(() →
            judgeClient.submitSql(request)
        );
    }

    // Phương thức dự phòng (Fallback) khi mạch bị ngắt (OPEN)
    // hoặc vượt quá số lần thử lại (retries)
    private CompletableFuture<JudgeResult> judgeFallback(
            JudgeRequest request, Throwable t) {
        if (t instanceof BusinessException) {
            // Lỗi nghiệp vụ: không đưa vào hàng đợi
            return CompletableFuture.failedFuture(t);
        }
        log.warn("Judge unavailable: {}, submissionId={}",
                 t.getMessage(), request.getSubmissionId());
        // Phải ghi command vào durable queue/outbox TRƯỚC khi hứa "sẽ chấm sau"
        // (idempotency key = submissionId); nếu enqueue thất bại thì trả lỗi
        // tạm thời (503 + Retry-After) thay vì một lời hứa không ai thực hiện
        judgeRetryQueue.enqueue(request);
        return CompletableFuture.completedFuture(
            JudgeResult.pending("Sandbox đang bận, bài sẽ được chấm khi sẵn sàng")
        );
    }
}
```

Về phân tầng Circuit Breaker: instance `judge` trong ví dụ trên bảo vệ Core khỏi *cụm Judge như một tổng thể* — đúng vai của Core, vì Core chỉ nói chuyện với bộ điều phối `kbm-judge-sql` (Giai đoạn 1, §4.5). Còn lớp bảo vệ chi tiết theo từng DBMS worker (`kbm-sandbox-mysql`, `kbm-sandbox-sqlserver`...) thuộc về chính coordinator: mỗi HTTP client tới một worker cần Circuit Breaker riêng (Giai đoạn 2, §4.5), để `kbm-sandbox-mysql` quá tải không kéo sập đường chấm của `kbm-sandbox-sqlserver`. Hai tầng này bổ trợ nhau, không thay thế nhau.

Đoạn mã trên minh họa cách phối hợp nhiều lớp bảo vệ trên cùng một phương thức khai báo, tạo thành một cơ chế phòng vệ trước các rủi ro sụp đổ dây chuyền. Dù vậy, bao bọc các lời gọi mạng chậm bằng `CompletableFuture` có thể làm cạn kiệt tài nguyên luồng, đặc biệt nếu lập trình viên không nắm rõ cơ chế cấp phát thread pool mặc định bên dưới.

⚠ Cấp Executor cho CompletableFuture

Đoạn `supplyAsync(...)` ở trên dùng overload *một tham số*, nên task chạy trên **common ForkJoinPool** dùng chung toàn JVM. Khi kết hợp với `@TimeLimiter` và `@Bulkhead`, các lệnh gọi kéo dài (chờ phản hồi từ Judge) có thể chiếm dụng hết nhóm luồng dùng chung (thread pool exhaustion), khiến các tác vụ khác trong JVM rơi vào trạng thái đói luồng (thread starvation) và đình trệ dây chuyền — vì mọi `CompletableFuture` không chỉ định executor và cả các parallel stream trong cùng JVM đều xếp hàng chờ trên chính pool này. Trên production, nên truyền một `Executor` riêng (`supplyAsync(() → ..., judgeExecutor)`) hoặc dùng `@Bulkhead(type = Bulkhead.Type.THREADPOOL)` để Resilience4j quản lý thread pool tách biệt cho nhóm call này (khi đó gọi `judgeClient.submitSql(request)` ngay trong method rồi trả `CompletableFuture.completedFuture(result)`, không gọi `supplyAsync` nữa). Đồng thời, `@TimeLimiter` chỉ hủy chờ trên `Future` chứ không tự đóng socket HTTP đang bị chặn bên dưới — bắt buộc phải cấu hình kèm `connectTimeout` và `readTimeout` cho chính OpenFeign client (`spring.cloud.openfeign.client.config.<name>.*`, xem Listing 4.3).

Từ Java 21 LTS, **Virtual Threads** (Project Loom) thay đổi đáng kể bài toán khan hiếm luồng ở trên: mỗi request có thể chạy trên một luồng ảo gần như "miễn phí" (JVM quản lý hàng triệu luồng ảo trên vài luồng hệ điều hành), nên phần lớn các lệnh gọi blocking I/O không còn chiếm giữ luồng nền tảng đắt đỏ (trên Java 21, luồng ảo vẫn bị ghim khi chặn trong khối `synchronized`; JDK 24 mới gỡ hạn chế này). Spring Boot 3.2+ chỉ cần bật `spring.threads.virtual.enabled=true` để áp dụng cho mô hình thread-per-request. Tuy nhiên, virtual threads chỉ giải quyết chi phí *chờ đợi I/O* — chúng không thay thế được Timeout, Retry, Circuit Breaker hay Bulkhead, vì dịch vụ phía sau chậm thì vẫn chậm; các resilience pattern trong chương này vẫn nguyên giá trị.

Ngoài khai báo annotation trong mã nguồn, Resilience4j cho phép tinh chỉnh các ngưỡng bảo vệ qua tệp cấu hình bên ngoài. Khi tách khỏi mã nguồn, các ngưỡng này có thể thay đổi mà không cần biên dịch lại — thông thường chỉ cần khởi động lại service để nạp giá trị mới. Muốn thay đổi "nóng" thật sự lúc runtime, cần bổ sung cơ chế chuyên dụng như Spring Cloud Config kết hợp `@RefreshScope`/actuator refresh endpoint.

Listing 4.3: *Cấu hình Resilience4j (YAML) cho LMS*

```yaml
# application.yml — Resilience4j configuration
resilience4j:
  circuitbreaker:
    instances:
      judge:
        slidingWindowSize: 10              # Đánh giá trên 10 requests gần nhất
```

```
        failureRateThreshold: 50           # Mở circuit khi 50% requests lỗi
        waitDurationInOpenState: 30s       # Chờ 30s trước khi thử lại (half-open)
        permittedNumberOfCallsInHalfOpenState: 3  # Thử 3 calls khi half-open
        recordExceptions:                  # Chỉ count những exceptions này
          - java.io.IOException
          - java.util.concurrent.TimeoutException
          - feign.RetryableException
          - feign.FeignException$ServiceUnavailable

  retry:
    instances:
      judge:
        maxAttempts: 3                     # Tối đa 3 lần thử
        waitDuration: 1s                   # Chờ 1s giữa các lần
        enableExponentialBackoff: true     # Kích hoạt backoff
        exponentialBackoffMultiplier: 2    # 1s → 2s (3 lần thử = 2 lần chờ)
        enableRandomizedWait: true         # Thêm jitter để tránh retry storm
        randomizedWaitFactor: 0.5          # Hệ số ví dụ, cần tinh chỉnh theo tải
        retryExceptions:
          - java.io.IOException            # Chỉ retry lỗi tạm thời
          # Bắt buộc để Retry bắt được timeout từ TimeLimiter
          - java.util.concurrent.TimeoutException
          # Lỗi mạng mà Feign bọc lại — thiếu dòng này Retry sẽ bỏ qua
          - feign.RetryableException
          # 503 từ downstream cũng là lỗi tạm thời đáng retry
          - feign.FeignException$ServiceUnavailable
        ignoreExceptions:
          - com.kbm.exception.BusinessException  # KHÔNG retry lỗi logic

  timelimiter:
    instances:
      judge:
        timeoutDuration: 5s               # Timeout 5s cho mỗi lần thử (attempt)

  bulkhead:
    instances:
      judge:
        maxConcurrentCalls: 10            # Tối đa 10 calls đồng thời đến Judge
        maxWaitDuration: 500ms            # Chờ tối đa 500ms nếu bulkhead đầy

spring:
  cloud:
    openfeign:
      client:
        config:
          kbm-judge-sql:
            connectTimeout: 1000          # 1s thiết lập kết nối TCP
            # connectTimeout + readTimeout = 4,5s, nhỏ hơn timeoutDuration (5s)
            readTimeout: 3500
```

Ngưỡng 5 giây ở đây chỉ hợp với lượt chấm truy vấn đơn giản trên đường đồng bộ; các lượt chấm kéo dài 5–30 giây (§4.1) đi đường bất đồng bộ ở Giai đoạn 4 (§4.5), không đi qua lời gọi này.

Các cấu hình YAML phía trên đã thiết lập các ngưỡng an toàn cho quá trình tương tác giữa hệ thống Core và hệ thống chấm điểm Judge. Tuy nhiên, các tham số này không phải giá trị tĩnh dùng chung cho mọi hệ thống. Tùy theo môi trường và mức tải dự kiến, kỹ sư cần giám sát và tinh chỉnh liên tục để đạt điểm cân bằng: đủ nhạy để ngắt mạch khi có lỗi thật, nhưng đủ bao dung

với những chập chờn mạng tạm thời. Bảng 4.5 dưới đây cung cấp một số giá trị tham khảo cốt lõi được khuyến nghị cho các kịch bản triển khai khác nhau.

Bảng 4.5: Giá trị khuyến nghị cho từng môi trường

Tham số	Phát triển	Vận hành (LMS)	Tải cao
`slidingWindowSize`	5	10	20
`failureRateThreshold`	50%	50%	30%
`waitDurationInOpenState`	10s	30s	60s
`retry.maxAttempts`	1	3	3
`timelimiter.timeoutDuration`	10s	5s	3s
`bulkhead.maxConcurrentCalls`	5	10	25

Bên cạnh các thông số ngưỡng, còn một yếu tố thường bị cấu hình sai và gây ra lỗi khó phát hiện: thứ tự áp dụng các annotation decorator trên cùng một phương thức. Theo tài liệu Resilience4j, các decorator được lồng theo thứ tự mặc định **từ ngoài vào trong**: `Retry( CircuitBreaker( TimeLimiter( Bulkhead( call ))))`. Nghĩa là Retry nằm ở lớp ngoài cùng — mỗi lần retry sẽ chạy lại toàn bộ chuỗi CircuitBreaker → TimeLimiter → Bulkhead. Thứ tự này quan trọng: nếu TimeLimiter bao bọc bên ngoài Retry, một timeout exception đầu tiên sẽ hủy bỏ toàn bộ chuỗi thử lại, làm vô hiệu hóa hoàn toàn mục đích của cơ chế Retry. Cấu hình mặc định của Resilience4j đã xử lý đúng thứ tự này. Hơn nữa, vì `TimeLimiter` ném ra `TimeoutException` khi hết hạn, chúng ta **bắt buộc phải khai báo** `java.util.concurrent.TimeoutException` trong danh sách `retryExceptions` của Retry — nếu thiếu, khi timeout xảy ra, Retry sẽ không kích hoạt và báo lỗi ngay lập tức. Cũng vì Retry bọc ngoài TimeLimiter, tổng thời gian chờ tối đa mà bên gọi có thể phải chịu bằng tổng timeout của các lần thử cộng thời gian backoff giữa chúng (ví dụ với `maxAttempts: 3`, `timeoutDuration: 5s` và backoff `1s → 2s`, trường hợp xấu nhất có thể kéo dài khoảng `3 × 5s + 1s + 2s ≈ 18s` (jitter làm thời gian chờ dao động quanh mức này)); do đó cần đặt `timeoutDuration` của mỗi lần thử đủ ngắn hoặc giới hạn tổng thời gian chờ ở tầng gọi.

Design for Failure

Trong hệ thống phân tán, lỗi không phải là ngoại lệ — lỗi là trạng thái bình thường. Hãy thiết kế mọi lời gọi liên dịch vụ với giả định rằng nó sẽ thất bại. Timeout, Retry, Circuit Breaker, Bulkhead — đây không phải những lựa chọn tùy ý, mà là điều kiện tiên quyết.

— Tổng hợp từ Hugo Rocha [5, Ch.7] và Michael Nygard, Release It!

4.5. Case study: KBM

Để hiểu rõ sự phối hợp giữa các thành phần, hãy xem xét hệ thống chấm điểm SQL của KBM như một hội đồng thi. Trong đó, Coordinator hoạt động như thư ký hội đồng điều phối đề thi, còn các worker DBMS là các giảng viên chuyên trách từng môn học khác nhau.

4.5.1. Bài toán

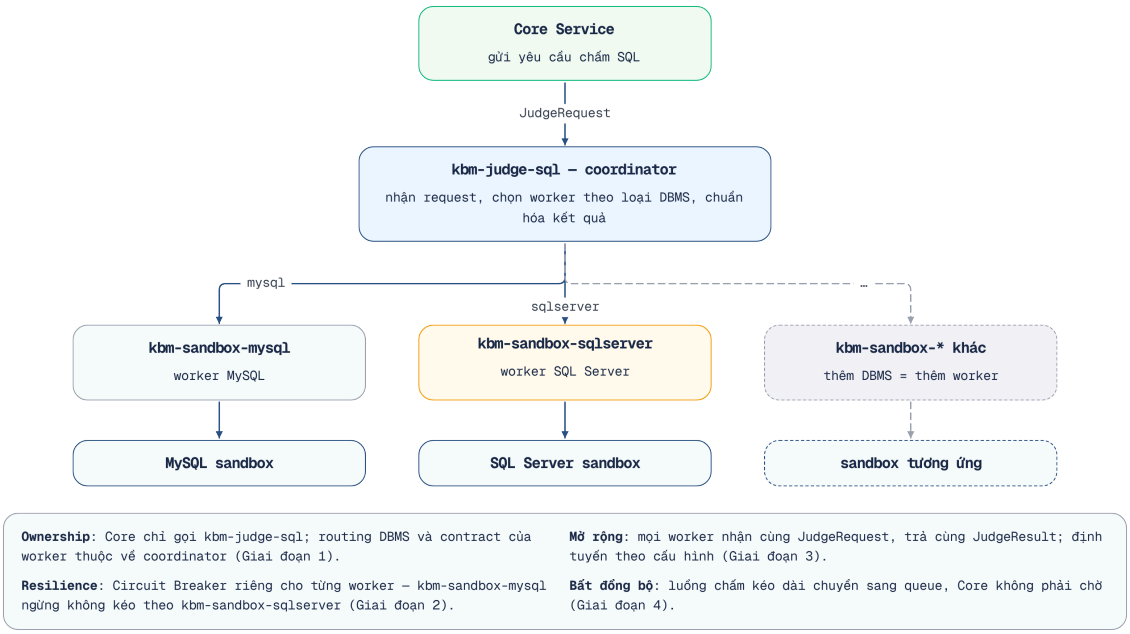
KBM hỗ trợ sinh viên thực hành SQL trên nhiều DBMS. Mỗi DBMS chạy trong sandbox container riêng biệt. Khi sinh viên nộp bài, hệ thống cần:

1. Xác định DBMS nào (dựa vào câu hỏi)
2. Gửi SQL đến đúng sandbox service
3. So sánh kết quả với đáp án (SHA-256 hash)

4. Trả về kết quả

4.5.2. Hiện trạng: Strategy Pattern + OpenFeign

Với yêu cầu hỗ trợ đa dạng các hệ quản trị cơ sở dữ liệu (DBMS), kiến trúc hiện tại của hệ thống KBM sử dụng mẫu thiết kế Strategy kết hợp cùng OpenFeign. Trong mô hình phối hợp này, một dịch vụ điều phối trung tâm `kbm-judge-sql` được xây dựng để hoạt động như một thư ký hội đồng, chịu trách nhiệm tiếp nhận toàn bộ các yêu cầu chấm bài từ hệ thống Core. Sau đó, dựa trên loại cơ sở dữ liệu được chỉ định, nó sẽ phân tích và điều hướng (dispatch) các đoạn mã SQL này đến đúng các worker chuyên trách tương ứng, đảm bảo sự phân tách trách nhiệm rõ ràng.



Hình 4.8: *Bộ điều phối `kbm-judge-sql` — điều hướng SQL đến các worker `kbm-sandbox-*` theo DBMS*

Core Service không nên biết chi tiết MySQL hay SQL Server được xử lý ở worker nào. Nó gửi yêu cầu chấm đến `kbm-judge-sql`; `kbm-judge-sql` làm nhiệm vụ coordinator: nhận database type, chọn worker tương ứng (`kbm-sandbox-mysql`, `kbm-sandbox-sqlserver`, ...), gọi worker qua HTTP/Feign hoặc queue, rồi chuẩn hóa kết quả trả về Core. Đây là pattern phổ biến: một coordinator giữ contract nghiệp vụ ổn định, các worker phía sau chuyên môn hóa theo runtime/DBMS.

4.5.3. Phân tích các vấn đề

Mặc dù kiến trúc hiện hành với thiết kế Strategy Pattern đã giúp tách biệt các trình điều khiển cơ sở dữ liệu một cách trực quan, nó vẫn bộc lộ các điểm yếu khi số worker và lượng bài nộp tăng, hoặc khi gặp lỗi mạng. Bảng 4.6 dưới đây trình bày chi tiết năm vấn đề cốt lõi của thiết kế hiện tại cùng với các phương pháp giải quyết tối ưu.

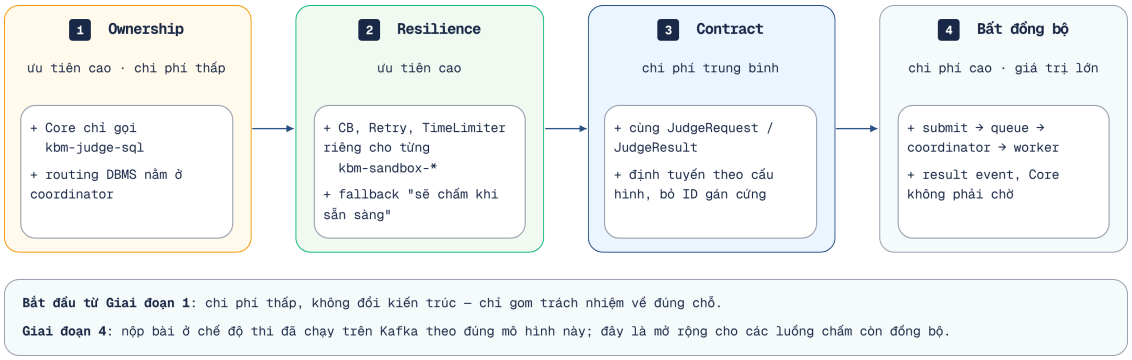
Bảng 4.6: *Phân tích vấn đề SQL Judge coordinator/worker*

#	Vấn đề	Mô tả	Giải pháp khuyến nghị
1	**Routing ownership chưa rõ**	Nếu Core cũng chứa logic chọn DBMS, routing bị rò rỉ ra ngoài Judge boundary	Single ownership: `kbm-judge-sql` là coordinator duy nhất

#	Vấn đề	Mô tả	Giải pháp khuyến nghị
2	**Định tuyến cứng (hardcoded routing) dựa trên các ID bí ẩn (Magic ID)**	Database type xác định bằng hardcoded UUID/string	Enum hoặc configuration-driven registry
3	**Tính chống chịu (resilience) theo từng worker chưa rõ ràng**	`kbm-sandbox-mysql` ngừng hoạt động không được phép làm gián đoạn toàn bộ dịch vụ `kbm-judge-sql` hay các worker khác	Circuit Breaker/Timeout riêng cho từng worker
4	**Cơ chế dự phòng (fallback) chưa nhất quán**	Worker của DBMS gặp sự cố → người dùng nhận lỗi kỹ thuật hoặc tiến trình nộp bài bị kẹt (stuck)	Phản hồi dự phòng (fallback), thử lại/hàng đợi, trạng thái rõ ràng
5	**Contract của worker chưa được chuẩn hóa**	Các `kbm-sandbox-*` dễ trả về định dạng hoặc kết quả khác nhau	Chuẩn hóa `JudgeRequest`/`JudgeResult` contract

4.5.4. Lộ trình chuyển đổi đề xuất

Để giải quyết các rủi ro thiết kế đã phân tích và chuẩn bị cho sự gia tăng khối lượng tải, hệ thống chấm điểm SQL cần một lộ trình tái cấu trúc gồm bốn giai đoạn. Lộ trình này bắt đầu từ việc củng cố sự ổn định cục bộ thông qua các mẫu thiết kế bảo vệ (resilience), cho đến việc thay đổi mô hình tương tác sang dạng bất đồng bộ hướng sự kiện (event-driven).



Hình 4.9: Lộ trình chuyển đổi cho SQL Judge — từ resilience đến bất đồng bộ

- **Giai đoạn 1 — Khóa ownership**: Ưu tiên cao, chi phí triển khai thấp. Giữ nguyên tắc Core chỉ gọi `kbm-judge-sql`; toàn bộ logic định tuyến (routing) hệ quản trị cơ sở dữ liệu phải được quản lý bởi bộ điều phối `kbm-judge-sql`. Không để logic định tuyến hệ quản trị cơ sở dữ liệu rò rỉ sang Core hoặc các service không sở hữu nghiệp vụ chấm SQL.
- **Giai đoạn 2 — Thêm resilience theo worker**: Ưu tiên cao. `@CircuitBreaker`, `@Retry`, `@TimeLimiter` riêng cho từng `kbm-sandbox-*`. Khi `kbm-sandbox-mysql` ngừng hoạt động → hệ thống sẽ kích hoạt phản hồi dự phòng (fallback): "Sandbox MySQL đang bận, bài sẽ được chấm khi sẵn sàng", không ảnh hưởng `kbm-sandbox-sqlserver`.
- **Giai đoạn 3 — Chuẩn hóa worker contract**: Chi phí triển khai trung bình. Tất cả worker nhận cùng `JudgeRequest`, trả cùng `JudgeResult`, định tuyến dựa trên cấu hình (config-driven) thay vì gán cứng ID.
- **Giai đoạn 4 — Chuyển các luồng xử lý kéo dài sang mô hình bất đồng bộ**: Chi phí triển khai cao, giá trị lớn. Submit → Kafka/queue → `kbm-judge-sql` → `kbm-sandbox-*` worker → Result event → Core cập nhật. Người dùng không phải chờ đồng bộ — sẽ nhận được thông báo (notification) khi có kết quả. Luồng nộp bài chế độ thi SQL đã chạy trên Kafka theo đúng mô hình này

(Chương 5); Giai đoạn 4 là việc mở rộng nó cho các luồng chấm kéo dài còn lại đang đi đường đồng bộ.

4.5.5. Tích hợp ngoài: QLĐT, GitHub Classroom và Network Practice

Không phải mọi tích hợp trong KBM đều nên đi qua cùng một protocol. Với QLĐT hoặc GitHub Classroom, KBM nên đặt **Anti-Corruption Layer** ở boundary: adapter dịch các contract bên ngoài thành lệnh/sự kiện nội bộ, che giấu định dạng cấu trúc dữ liệu, các lỗi và giới hạn lưu lượng (rate limit) của hệ thống đối tác. Với Network Practice, điểm đáng học là ngược lại: context này được thiết kế độc lập để đánh giá giao tiếp mạng, không phụ thuộc vào service chấm SQL. Vì vậy, nó là ví dụ tốt cho Bounded Context theo protocol, còn vấn đề coordinator/worker cần phân tích nằm ở SQL Judge.

ⓘ Xây hay mua --- tự viết hay tích hợp DOMjudge cho bộ chấm code

Cùng tinh thần Anti-Corruption Layer, module chấm code đa ngôn ngữ (`kbm-judge-code`) minh họa một quyết định **'xây hay mua' (build-vs-buy)** điển hình. Thế hệ đầu (Gen 1) là bộ chấm đồng bộ *tự viết* (C/C++ và Java) — kiểm soát tối đa nhưng nhóm phát triển phải tự đảm đương và duy trì toàn bộ trình biên dịch (compiler), môi trường thực thi (runner) cũng như các cơ chế bảo mật. Thế hệ thứ hai (Gen 2) chuyển sang tích hợp **DOMjudge** — một dịch vụ OSS đã được kiểm chứng qua nhiều kỳ thi ICPC. Bằng việc chấp nhận một thành phần phụ thuộc (dependency) bên ngoài, KBM loại bỏ cả một lớp rủi ro tự xây (sandbox, đa ngôn ngữ, bảo trì compiler). Bài học sync communication: với một service đồng bộ phức tạp và nhạy cảm bảo mật, *tích hợp một giải pháp đã kiểm chứng* thường thắng *tự duy trì* — miễn đặt adapter rõ ràng ở boundary để cô lập contract ngoài. (Khía cạnh cô lập/sandbox bảo mật: Chương 9.)

Trên thực tế, nhiều hệ thống bắt đầu với sync (đơn giản, dễ gỡ lỗi) rồi chuyển dần sang async khi cần mở rộng. Richardson trong [2a] minh họa quá trình này: ban đầu là các lệnh gọi REST giữa các dịch vụ, sau đó chuyển sang mẫu thiết kế Saga khi phát sinh nhu cầu giao dịch phân tán (distributed transaction). Hệ thống KBM đang ở giữa tiến trình chuyển đổi này: một số luồng xử lý (flow) đã chuyển sang mô hình bất đồng bộ (sử dụng Kafka để nộp bài), một số luồng vẫn duy trì đồng bộ (sử dụng Feign để truy vấn), và phân hệ SQL Judge cần được tách bạch rõ ràng giữa coordinator và worker trước khi tiến hành mở rộng theo chiều ngang (horizontal scaling) một cách an toàn. Chương 5 sẽ tiếp tục phần async.

⚠ Sai lầm thường gặp khi dùng giao tiếp đồng bộ

Khi triển khai các cuộc gọi API đồng bộ, các nhóm phát triển thường mắc phải một số lỗi kinh điển dẫn đến sự sụp đổ dây chuyền của hệ thống (cascading failure). Nhận diện và phòng tránh những cái bẫy này là nền tảng để xây dựng hệ thống bền bỉ.

- **Không đặt timeout cho inter-service calls (Tin vào timeout mặc định)**: Nhiều HTTP client (kể cả Feign nếu không cấu hình) có timeout mặc định rất dài hoặc chờ vô hạn. Giống như việc sinh viên đứng chờ mãi ở phòng giáo vụ mà không biết người phụ trách đã nghỉ ốm. Hậu quả: khi dịch vụ phía sau (downstream) phản hồi chậm hoặc bị treo, luồng xử lý của bên gọi bị giữ lại vô thời hạn → dẫn đến cạn kiệt thread pool → gây lỗi dây chuyền (cascading failure). *Phòng tránh*: Luôn cấu hình thời gian chờ kết nối/đọc (connect/read timeout) một cách tường minh (1–3 giây là điểm khởi đầu hợp lý cho lời gọi nội bộ — tham

chiếu ngưỡng mặc định 1 s của Hystrix; con số cuối cùng phải lấy từ P99 đo được của chính downstream).

- **Tin rằng mạng luôn ổn định**: Viết code như thể mọi HTTP call đều thành công. Hậu quả: lỗi đầu tiên trên production mới phát hiện không có retry, không fallback. *Phòng tránh*: Áp dụng các resilience pattern từ đầu (Circuit Breaker + Retry + Timeout) — ngay cả khi chưa cần scale (§4.4).
- **Thử lại mà không có độ trễ lùi dần (backoff) và không kiểm tra idempotency**: Thử lại ngay lập tức một thao tác POST (có thể tạo ra dữ liệu trùng lặp, duplicate data). Giống như sinh viên bấm nút nộp bài thi liên tục khi mạng chậm, dẫn đến nộp 10 bản sao. Hậu quả: vừa khuếch đại tải lên dịch vụ đang quá tải/suy yếu (tạo retry storm), dữ liệu sai, người dùng bị tính điểm trùng. *Phòng tránh*: Chỉ retry thao tác idempotent (GET, PUT, DELETE) hoặc dùng idempotency key cho POST, dùng exponential backoff + jitter, giới hạn số lần.
- **Circuit Breaker không có cơ chế dự phòng (fallback) hợp lý**: Ngắt mạch (open circuit) nhưng vẫn trả về unhandled exception cho người dùng. *Phòng tránh*: Định nghĩa fallback trả về giá trị mặc định/cache hoặc thông báo "Hệ thống đang bận, sẽ xử lý sau" để đảm bảo hệ thống suy giảm chất lượng một cách có kiểm soát (graceful degradation).
- **Chuỗi gọi đồng bộ quá dài**: A→B→C→D đồng bộ. Hậu quả: độ trễ cộng dồn, và sự chậm trễ của một mắt xích sẽ làm đình trệ toàn bộ chuỗi giao tiếp — chính là distributed monolith. Việc bắt một sinh viên phải tự chạy qua 4 phòng ban khác nhau để xin đủ 4 chữ ký trong cùng một buổi sáng là không khả thi. *Phòng tránh*: Rút ngắn chuỗi, cân nhắc async (Chương 5) cho các bước không cần kết quả ngay.

Các pattern trong chương có minh họa tương tác đi kèm tại companion web:

Interactive Demo

- **Service Discovery**: service-discovery.html
- **Circuit Breaker**: circuit-breaker.html
- **Load Shedding**: load-shedding.html

4.6. Tổng kết

Giao tiếp đồng bộ là mô hình đơn giản nhất nhưng tiềm ẩn rủi ro lớn nhất trong hệ thống phân tán (distributed system). Temporal coupling, cascading failure, và độ trễ cộng dồn là ba vấn đề cốt lõi mà các kỹ sư phần mềm phải đối mặt. Chọn kiểu tương tác và giao thức theo từng ranh giới: REST cho API hướng người dùng, gRPC khi giao tiếp nội bộ cần thông lượng cao hoặc streaming. Bài toán SQL Judge của KBM cho thấy một coordinator giữ contract ổn định giúp các worker phía sau thay đổi độc lập.

OpenFeign đơn giản hóa các lệnh gọi liên dịch vụ thành các khai báo interface trong Java, loại bỏ các đoạn mã HTTP lặp lại. Service Discovery (Eureka) giải quyết bài toán "tìm dịch vụ ở đâu" — một vấn đề không tồn tại trong monolith nhưng đặc biệt quan trọng trong microservices.

Các resilience pattern — Timeout, Retry, Circuit Breaker, Bulkhead — không phải là các tùy chọn có hay không cũng được. Chúng là điều kiện tiên quyết để hệ thống microservices hoạt động ổn định trên production. Phân tích SQL Judge của KBM cho thấy resilience theo từng worker và cơ chế dự phòng là những điểm cần củng cố trước khi mở rộng (§4.5).

Ở Chương 5, chúng ta sẽ khám phá **giao tiếp bất đồng bộ** — giải pháp cho những giới hạn của giao tiếp đồng bộ (sync): Apache Kafka, kiến trúc hướng sự kiện (event-driven architecture), và cách KBM ứng dụng cơ chế truyền tin (messaging) cho luồng xử lý nộp bài (submission pipeline).

4.7. Đọc thêm

Sách tham khảo chính:

1. [2a] Chris Richardson, *Microservices Patterns*, 1st Ed. — Ch.3: Interprocess Communication, Service Discovery
2. [4a] Sam Newman, *Building Microservices* — Ch.4: Integration, Smart Endpoints and Dumb Pipes
3. [5] Hugo Rocha, *Practical Event-Driven Microservices Architecture* — Ch.7: Resilience & Reliability

Sách bổ trợ:

4. [2b] Chris Richardson, *Microservices Patterns*, 2nd Ed. — Ch.4: Coupling Taxonomy (design-time, build-time, runtime, infrastructure), Iceberg Principle, DRY in distributed systems; Ch.8: IPC patterns
5. Michael Nygard, *Release It!*, 2nd Ed. (2018) — Circuit Breaker, Bulkhead, Timeout patterns

Nguồn trực tuyến:

- [W7] Netflix Technology Blog — “Making the Netflix API More Resilient” (2011)
- Resilience4j documentation — resilience4j.readme.io

Bài tập: 4-0, 4-1, 4-2 ở phần Bài tập & case study cuối sách.

CHƯƠNG 5

Giao tiếp bất đồng bộ — Kafka, Events & Messaging

> "Event streams become the heart of data sharing throughout the company. Data no longer sits solely on a database and is accessible only through synchronous interfaces; instead, it is shared in event streams readily available for every current and future consumer."
>
> — Hugo Rocha, *Practical Event-Driven Microservices Architecture* [5]

MỤC TIÊU HỌC TẬP

- Hiểu tại sao giao tiếp bất đồng bộ giải quyết các giới hạn của giao tiếp đồng bộ
- Phân biệt hai loại message broker: durable (Kafka) vs ephemeral (RabbitMQ)
- Nắm vững kiến trúc Apache Kafka: topic, partition, consumer group
- Hiểu kiến trúc RabbitMQ: exchange, queue, binding
- Thiết kế event schema: 4 loại message (Command, Event, Document, Query)
- So sánh các cấp độ đảm bảo giao nhận: at-most-once, at-least-once, exactly-once
- Sử dụng WebSocket cho thông báo thời gian thực
- Phân tích Kafka pipeline và DB queue trong KBM

5.1. Động lực của giao tiếp bất đồng bộ

Giống như việc sinh viên nộp đơn vào hòm thư thay vì đứng chờ trực tiếp trước phòng giáo vụ, giao tiếp bất đồng bộ cho phép hệ thống tiếp nhận yêu cầu và xử lý sau, giải phóng các dịch vụ khỏi sự chờ đợi chéo nhau gây tắc nghẽn.

5.1.1. Vấn đề mà sync không giải quyết được

Chương 4 đã phân tích ba vấn đề cốt lõi của giao tiếp đồng bộ: sự phụ thuộc về mặt thời gian (temporal coupling), lỗi dây chuyền (cascading failure), và độ trễ cộng dồn. Giao tiếp bất đồng bộ (*asynchronous messaging*) làm nhẹ cả ba bằng cách **tách rời** (*decouple*) producer và consumer qua một trung gian — message broker.

TIÊU CHÍ	ĐỒNG BỘ (REST / GRPC)	BẤT ĐỒNG BỘ (KAFKA / RABBITMQ)
Độ trễ	thấp – phản hồi trực tiếp	cao hơn – đi qua broker
Coupling	cao – A phụ thuộc trực tiếp B	thấp – A không cần biết B
Khi có lỗi	lan dây chuyền	khoanh vùng tại B
Thông lượng	giới hạn bởi tốc độ của B	broker đệm – mở rộng độc lập
Độ phức tạp	đơn giản – request/response rõ ràng	phức tạp hơn (idempotency, thứ tự)

Hình 5.1: *So sánh giao tiếp đồng bộ (coupled) và bất đồng bộ (decoupled)*

Thay vì bị khóa cứng về mặt thời gian, giao tiếp bất đồng bộ làm nhẹ phần lớn những điểm yếu này. Với bài toán **temporal coupling**, producer chỉ cần gửi thông điệp vào broker rồi tiếp tục công việc khác, trong khi consumer có thể xử lý sau khi đã sẵn sàng. Với **cascading failure**, phạm vi ảnh hưởng được thu hẹp: nếu một service tạm ngưng hoạt động, thông điệp vẫn nằm tại broker chờ đến khi service đó phục hồi, thay vì đẩy lỗi ngược lên phía gọi. Cần lưu ý rằng rủi ro không biến mất mà dịch chuyển — hàng đợi có thể tồn đọng và chính broker trở thành thành phần phải được bảo vệ. Về **độ trễ**, thay vì bắt người dùng phải chờ đợi hệ thống xử lý xong toàn bộ chuỗi tác vụ, ứng dụng có thể trả về phản hồi ngay lập tức và gửi kết quả thực tế qua cơ chế thông báo hoặc polling sau đó.

Broker thay vì Async HTTP

Nhiều lập trình viên mới thắc mắc: *"Tại sao không gửi một HTTP request và không cần chờ kết quả (fire-and-forget)?"*

Câu trả lời là: Giao tiếp HTTP bất đồng bộ giống như một cuộc gọi điện thoại — dù người gọi truyền đạt thông tin rất nhanh rồi ngắt kết nối, người nhận vẫn bắt buộc phải đang trong trạng

thái sẵn sàng tiếp nhận (online) ngay tại thời điểm đó. Còn Message Broker giống như việc *sinh viên nộp đơn phúc khảo vào hộp thư Phòng Đào tạo*. Người gửi bỏ đơn vào hộp (Broker), hệ thống cam kết sẽ giữ đơn và chuyển đến tay chuyên viên, kể cả khi máy chủ người nhận đang gặp sự cố hay mất điện.

Richardson trong [2a, Ch.3] phân tích: async messaging cải thiện **tính sẵn sàng (availability)** vì các dịch vụ không phụ thuộc lẫn nhau tại runtime. Tuy nhiên, hệ thống phải đánh đổi bằng **độ phức tạp (complexity)** — tính nhất quán cuối cùng (eventual consistency), thứ tự thông điệp (message ordering) và xử lý trùng lặp (duplicate handling) — đây là những chi phí không thể tránh khỏi.

Nguyên tắc dưới đây tóm tắt sự đánh đổi đó.

Đổi tính nhất quán lấy tính sẵn sàng

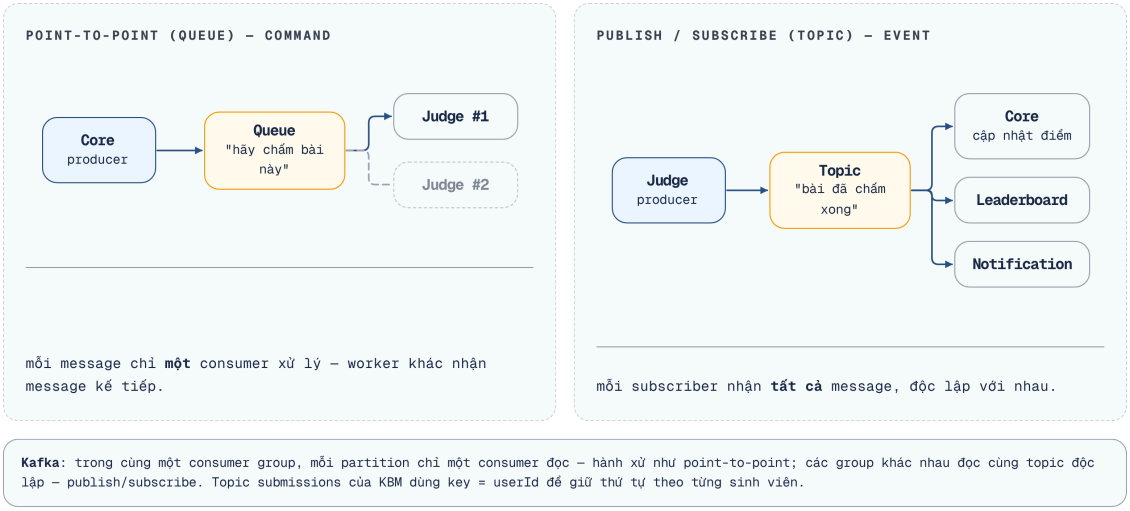
Async messaging cải thiện tính sẵn sàng và giảm temporal coupling — các dịch vụ không cần online cùng lúc — nhưng phải trả giá bằng tính nhất quán cuối cùng, khử trùng lặp (deduplication) và xử lý dead letter. Đây là sự đánh đổi có chủ đích, không phải hệ quả ngẫu nhiên. Trước khi chọn async, câu hỏi bắt buộc: "Khoảng thời gian chờ nhất quán (consistency window) bao lâu là chấp nhận được cho bài toán thực tế này?"

5.1.2. Messaging patterns

Có ba messaging pattern cơ bản [2a, Ch.3]:

1. **Point-to-point (Queue)** — Một message, một consumer. Giống như giảng viên giao đề tài riêng cho một sinh viên trên hệ thống LMS (sinh viên này đã nhận và xử lý thì người khác không cần làm nữa). Phù hợp cho command: "hãy chấm bài này".
2. **Publish/Subscribe (Topic)** — Một message, nhiều consumer. Giống như Phòng Đào tạo gửi thông báo toàn trường (bất kỳ phòng ban nào quan tâm đều tiếp nhận được thông tin nguyên bản độc lập với nhau). Phù hợp cho event: "bài đã được chấm xong".
3. **Request/Async Response** — Gửi request qua messaging, nhận response qua message khác (có correlation ID). Kết hợp lợi ích của cả hai.

Hình 5.2 so sánh hai mô hình phân phối đầu tiên.



Hình 5.2: Point-to-Point (Queue) vs Publish/Subscribe (Topic)

Command vs Event

Phân biệt rõ **command** (yêu cầu hành động: "ChấmBài", "GửiEmail") và **event** (thông báo sự kiện đã xảy ra: "BàiĐãĐượcChấm", "ĐiểmĐãCậpNhật"). Các lệnh (command) thường được phân phối theo mô hình point-to-point, trong khi các sự kiện (event) thường tuân theo mô hình pub/sub. Sự phân biệt này ảnh hưởng đến thiết kế schema và xử lý lỗi [5, Ch.8].

5.2. Message Broker — hai trường phái thiết kế

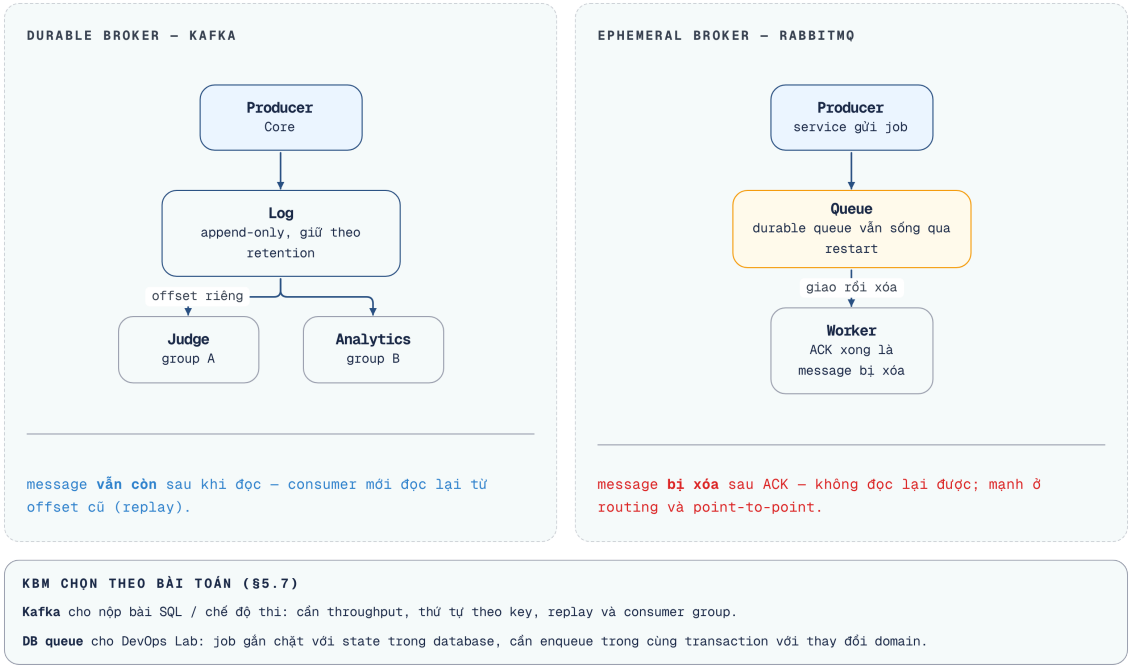
Message Broker hoạt động giống như trung tâm bưu điện của trường đại học. Tùy thuộc vào cách thức lưu trữ và phân phát thư tín, các thiết kế được chia thành hai trường phái trái ngược, quyết định trực tiếp đến sức chịu tải của toàn bộ hệ thống.

5.2.1. Durable vs Ephemeral Brokers

Rocha trong [5, §3.1.3] phân loại message broker thành hai trường phái cơ bản dựa trên vòng đời của message:

- **Ephemeral (tạm thời)**: Thông điệp bị xóa sau khi consumer gửi phản hồi xác nhận (acknowledge). Đại diện: **RabbitMQ**, ActiveMQ, Amazon SQS.
- **Durable (bền vững)**: Thông điệp được lưu trữ trên ổ cứng (disk) và vẫn sẵn sàng (available) sau khi được tiêu thụ (consume). Đại diện: **Apache Kafka**, Apache Pulsar, Amazon Kinesis. Hệ sinh thái này cũng xuất hiện các lựa chọn mới đáng chú ý: **Redpanda** (tương thích Kafka API nhưng viết bằng C++, một binary duy nhất, không cần JVM — gọn nhẹ cho self-hosted) và **NATS JetStream** (nhẹ, độ trễ thấp, phù hợp hệ thống nhỏ). Nội dung chương dùng Kafka làm đại diện; các khái niệm partition, offset, consumer group áp dụng gần như nguyên vẹn cho Redpanda.

Hình 5.3 minh họa khác biệt giữa hai trường phái: message bị xóa sau khi được xác nhận, hay vẫn được giữ lại để đọc lại.



Hình 5.3: Ephemeral broker (message xóa sau ack) vs Durable broker (message vẫn còn)

Ở đây, "ephemeral" chỉ việc message bị xóa sau khi được xác nhận tiêu thụ (destructive consumption). Thuật ngữ này không có nghĩa RabbitMQ thiếu độ bền: durable queue và persistent message vẫn sống sót qua lần khởi động lại. Ngược lại, "durable" theo nghĩa của Kafka là message được giữ lại theo chính sách lưu giữ (retention) để đọc lại (replay), chứ không mặc nhiên vĩnh viễn.

Sự khác biệt này **không chỉ là kỹ thuật** — nó ảnh hưởng đến cách thiết kế hệ thống. Các broker lưu trữ bền vững (durable broker) cho phép phát lại (replay), lưu vết sự kiện (event sourcing), và hỗ trợ nhiều bên tiêu thụ (consumer) cùng đọc một luồng dữ liệu. Trong khi đó, các broker trung chuyển (ephemeral broker) tập trung vào định tuyến thông minh (smart routing), hàng đợi ưu tiên (priority queue), và giao tiếp điểm-điểm (point-to-point communication).

Chọn broker theo bài toán thực tế

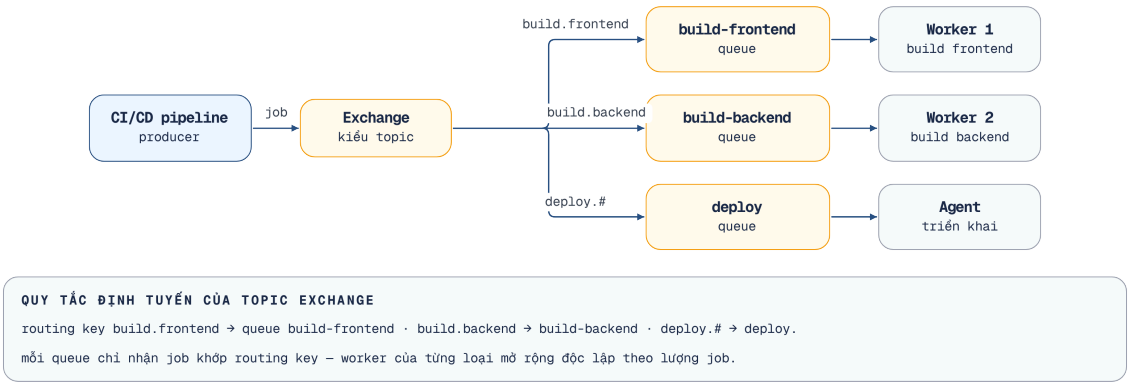
Durable broker (Kafka) phù hợp khi cần replay và nhiều consumer cùng đọc (Kafka có thể làm kho sự kiện cho event sourcing, kèm các giới hạn ở Chương 7). Ephemeral broker (RabbitMQ) phù hợp khi cần smart routing và point-to-point delivery. Chọn broker dựa trên **thói quen của đội ngũ phát triển** thay vì đặc điểm kỹ thuật sẽ dẫn đến sự lệch pha (mismatch) kiến trúc khó sửa chữa về sau.

Việc lựa chọn broker giống như việc thiết lập kênh thông báo trong trường đại học. Nếu cần lưu trữ toàn bộ lịch sử điểm số để sinh viên có thể tra cứu lại bất cứ lúc nào (lưu vết sự kiện), Kafka là hệ thống sổ cái đào tạo phù hợp nhất. Ngược lại, nếu chỉ cần phân luồng đơn từ phúc khảo đến đúng phòng ban xử lý và có thể bỏ đi sau khi giải quyết xong, RabbitMQ hoạt động như một hòm thư tiếp nhận thông minh.

5.2.2. RabbitMQ — Smart Routing & Flexible Messaging

RabbitMQ dựa trên **AMQP** (Advanced Message Queuing Protocol), với kiến trúc phong phú hơn Kafka về routing:

Trong môi trường đại học, không phải mọi thông báo đều được phát loa toàn trường. RabbitMQ cung cấp một cơ chế định tuyến thông minh (smart routing), giống như một nhân viên phân loại thư tín tại Phòng Hành chính, đảm bảo mỗi đơn từ hay yêu cầu nghiệp vụ đều được chuyển phát chính xác đến tay người cần xử lý.



Hình 5.4: Kiến trúc RabbitMQ — Exchange định tuyến message đến các queue

Khác với mô hình append-only log đơn giản của Kafka, message trong RabbitMQ đi qua exchange trước khi vào queue.

Dưới đây là các khái niệm cốt lõi của RabbitMQ:

- **Exchange**: Nhận message và định tuyến (route) đến các queue theo các quy tắc thiết lập trước.
- **Queue**: Lưu trữ message trong khi chờ consumer tiêu thụ (hoạt động theo nguyên tắc FIFO).
- **Binding**: Quy tắc kết nối từ exchange đến queue (thông qua routing key, header).
- **Ack/Nack**: Cơ chế để Consumer xác nhận đã xử lý xong (ack) hoặc từ chối (nack → requeue/dead letter).
- **Exchange types**: Direct, Fanout, Topic, Headers — mỗi loại cung cấp một logic định tuyến riêng biệt.

Trong khi Kafka chú trọng vào khả năng lưu trữ bền vững và thông lượng cực lớn bằng cách phân vùng dữ liệu đơn giản theo partition, thì RabbitMQ lại có thế mạnh ở năng lực **smart routing**. Một thông điệp trong RabbitMQ có thể được nhân bản, lọc hoặc định tuyến phức tạp đến đúng các hàng đợi mục tiêu dựa trên routing key, cấu trúc header matching, hoặc broadcast tới nhiều thành phần của hệ thống.

5.2.3. So sánh chi tiết

Dưới đây là so sánh chi tiết giữa hai hệ thống này theo từng tiêu chí cụ thể:

- **Loại hệ thống**: Kafka là một durable event log, lưu trữ lâu dài. RabbitMQ là ephemeral message queue, xóa thông điệp sau khi xử lý.
- **Mô hình dữ liệu**: Kafka sử dụng cấu trúc append-only log, nơi consumer theo dõi qua offset. RabbitMQ sử dụng hàng đợi FIFO, thông điệp biến mất sau khi được xác nhận (ack).
- **Throughput (Thông lượng)**: Kafka đạt mức rất cao (hàng triệu thông điệp mỗi giây). RabbitMQ với cấu hình truyền thống ở mức hàng chục nghìn thông điệp mỗi giây (Quorum

Queues/Streams hiện đại có thể cao hơn đáng kể, nhưng thường vẫn không phải lựa chọn tối ưu cho extreme throughput).

- **Đảm bảo thứ tự (Ordering)**: Kafka đảm bảo thứ tự trên từng partition. RabbitMQ giữ thứ tự FIFO trong một hàng đợi khi chỉ có một consumer hoạt động (*Single Active Consumer*); nếu nhiều competing consumer xử lý song song hoặc đẩy trả message lỗi về hàng đợi (`requeue`), thứ tự hoàn tất thực tế có thể bị đảo (hoặc phải dùng RabbitMQ Streams).
- **Khả năng Replay**: Kafka hỗ trợ chạy lại (replay) từ bất kỳ offset nào. RabbitMQ mặc định không thể replay (trừ khi dùng RabbitMQ Streams bản 3.9+).
- **Cơ chế định tuyến (Routing)**: Kafka định tuyến đơn giản qua topic và partition key. RabbitMQ định tuyến linh hoạt bằng 4 loại exchange và routing key.
- **Hàng đợi ưu tiên (Priority)**: Kafka không hỗ trợ mức ưu tiên. RabbitMQ hỗ trợ tốt hàng đợi ưu tiên (priority queue).
- **Hẹn giờ thông điệp (Delayed message)**: Kafka không có tính năng nguyên bản. RabbitMQ giải quyết qua tổ hợp Dead Letter Exchange + TTL hoặc plugin cộng đồng `rabbitmq_delayed_message_exchange`.
- **Giao thức (Protocol)**: Kafka dùng custom binary protocol tối ưu riêng. RabbitMQ hỗ trợ đa dạng (AMQP, STOMP, MQTT).
- **Bài toán phù hợp**: Kafka dành cho event streaming, stream processing, audit log (ví dụ: lịch sử thao tác của sinh viên). RabbitMQ phù hợp cho task queue, tác vụ hẹn giờ (delayed job), định tuyến phức tạp (ví dụ: phân luồng các lệnh in ấn bảng điểm, gửi email).

Sự khác biệt này quyết định môi trường áp dụng của mỗi broker. Chọn sai công cụ cho bài toán thường dẫn tới việc thiếu đúng tính năng cần dùng, hoặc tắc nghẽn khi tải tăng.

RabbitMQ trong High-throughput

Rocha chia sẻ kinh nghiệm từ một nền tảng thương mại điện tử lớn [5, §3.1.3]: Đội ngũ đã sử dụng RabbitMQ nhiều năm trong môi trường production thông lượng cao. Vấn đề: khi tải đạt đỉnh và thông điệp tích tụ, **toàn bộ cluster bị ảnh hưởng** — các dịch vụ không liên quan cũng bị chậm. Nhìn lại, kịch bản sử dụng thực tế của họ (event streaming, high throughput) phù hợp với durable broker hơn. Bài học: **chọn broker phải dựa trên bài toán thực tế**, không phải theo thói quen.

Lưu ý khi dùng RabbitMQ: Để các worker thực sự chia đều công việc (mô hình Competing Consumers), cần cấu hình `prefetch` nhỏ (ví dụ `prefetch=1`) cho tác vụ kéo dài. `prefetch` giới hạn số message chưa ACK được giao trước cho mỗi consumer: không có giới hạn này, RabbitMQ dồn giao trước (round-robin) hàng loạt message cho worker đang bận trong khi worker khác đã rảnh. Đổi lại, `prefetch` quá nhỏ làm giảm thông lượng với tác vụ ngắn — đây là tham số tuning theo độ dài tác vụ, không phải hằng số bắt buộc.

5.2.4. Khi nào dùng gì?

Bảng 5.1 tóm tắt lựa chọn broker theo từng nhu cầu.

Bảng 5.1: *Khi nào chọn Kafka, khi nào chọn RabbitMQ*

Nhu cầu	Chọn
Event streaming, audit trail, replay	**Kafka**
High throughput (100K+ msg/sec)	**Kafka**
Stream processing (KStreams, ksqlDB)	**Kafka**

Nhu cầu	Chọn
Task/job queue, background worker	**RabbitMQ**
Complex routing (header, topic pattern)	**RabbitMQ**
Priority queue, delayed message	**RabbitMQ**
Cả hai	Kafka cho event backbone + RabbitMQ cho task queue

KBM chọn Kafka cho pipeline xử lý bài nộp (submission) vì cần khả năng phát lại (**replay**) — cho phép chấm lại bài khi logic hệ thống thay đổi, đảm bảo thứ tự (ordering) theo người dùng, và hỗ trợ khả năng mở rộng khi một kỳ thi có nhiều lượt nộp bài cùng lúc [5, Ch.3]. Phát lại message cũ vẫn phải giữ idempotency; khi chủ động chấm lại bằng logic mới, cần tạo lượt chấm mới với `judgeRunId` mới và chống trùng theo lượt chấm đó. Tuy nhiên, Kafka không phải câu trả lời mặc định cho mọi queue: DevOps Lab dùng **DB queue** — bảng database với status field (`PENDING`, `PROCESSING`, `DONE`) và worker poll định kỳ — cho job vận hành vì cần transactional enqueue đơn giản, dễ quan sát cùng dữ liệu domain và không cần replay event log toàn hệ thống.

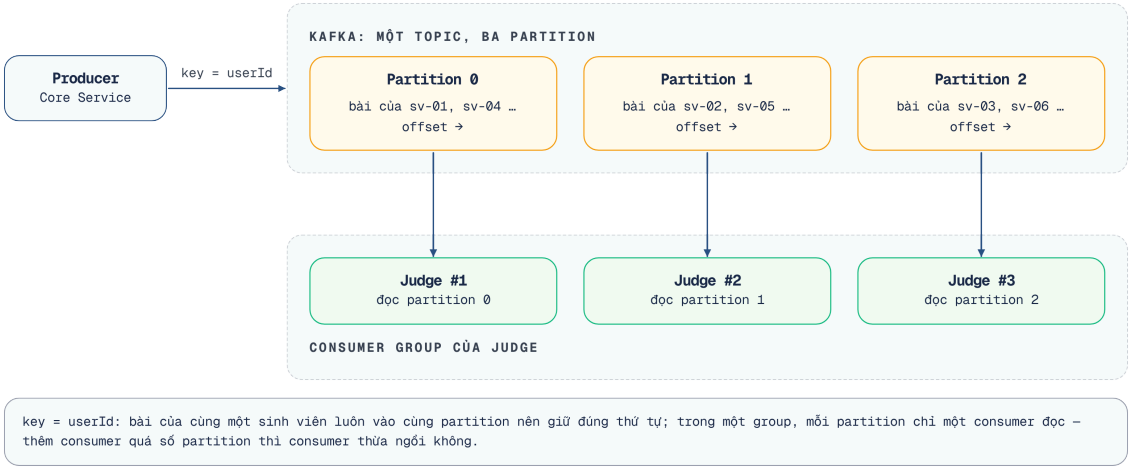
KBM: hai công nghệ, hai bài toán khác nhau

KBM sử dụng Kafka cho luồng xử lý bài nộp (submission pipeline) và hàng đợi cơ sở dữ liệu cho các tác vụ của DevOps Lab — đây là lựa chọn hợp lý xét về khả năng phát lại (replayability) và thông lượng (throughput). Tuy nhiên, việc áp dụng Kafka cho submission (một long-running task) cũng mang rủi ro Head-of-Line blocking; cách giảm rủi ro (thread pool sau consumer hoặc Confluent Parallel Consumer) được bàn ở §5.4 và nằm trong Giai đoạn 3 của lộ trình. Job vận hành cần transactional enqueue, dễ giám sát bằng SQL, và không cần mở rộng lớn → hàng đợi cơ sở dữ liệu là đủ và đơn giản hơn nhiều. **Việc chuẩn hóa cực đoan (over-standardizing) trên Kafka** cho mọi hàng đợi sẽ tạo ra gánh nặng vận hành (operational overhead) không cần thiết.

5.3. Apache Kafka — kiến trúc chi tiết

5.3.1. Các khái niệm cốt lõi

Hình 5.5 cho thấy quan hệ giữa topic, partition và consumer group (minh họa với 3 partition; hiện trạng KBM ở §5.9); từng khái niệm được giải thích ngay sau hình.



Hình 5.5: Kiến trúc Kafka — Topic, Partition và Consumer Group

Hệ sinh thái Kafka được cấu thành từ các khái niệm nền tảng sau:

Bảng 5.2: Các khái niệm cốt lõi của Apache Kafka

Khái niệm	Mô tả	Ví dụ KBM
Topic	Luồng message theo category	`submissions`, `judge-results`
Partition	Các luồng nhật ký chỉ ghi nối tiếp (append-only log) độc lập trong một topic, cho phép xử lý song song và bảo toàn thứ tự trong từng partition	1 partition cho `submissions` (hiện tại); đề xuất tăng lên 3 khi cần mở rộng
Consumer Group	Nhóm các consumer chia nhau các partition	`judge-group` — hiện chạy 1 instance trên 1 partition (ví dụ mở rộng: nếu chạy 3 instance mà topic vẫn chỉ có 1 partition thì 2 instance sẽ đứng chờ)
Offset	Vị trí của một record trong partition. Consumer lưu riêng vị trí đã commit, giống **dấu kẹp sách** đặt *ngay sau* message cuối cùng đã xử lý thành công	Consumer 1 xử lý xong record có offset 42 → commit position 43; sau restart đọc tiếp từ offset 43
Retention	Thời gian giữ message	7 ngày (default) hoặc vĩnh viễn

Partition key quyết định message vào partition nào. Các thông điệp có cùng khóa (key) sẽ được phân bổ vào cùng một partition, qua đó đảm bảo được thứ tự xử lý. Trong KBM, topic submissions dùng partition key = `userId` (hoặc `submissionId` cho luồng kết quả) để đảm bảo ordering ở đúng phạm vi nghiệp vụ; cần tránh chọn khóa quá thô như ID kỳ thi vì sẽ dồn toàn bộ lưu lượng kỳ thi vào một partition.

Kleppmann trong [7, Ch.11] giải thích: Kafka kết hợp ưu điểm của database (durable storage, replay) với ưu điểm của messaging (real-time, decoupling) — đây là mô hình **log-based messaging** khác biệt cơ bản với queue-based messaging.

5.4. Producer/Consumer Pattern trong KBM

Trong hệ thống LMS, Producer hoạt động như cổng tiếp nhận bài thi của sinh viên để đẩy vào đường ống xử lý. Ngược lại, Consumer giống như bộ phận chấm thi, lần lượt lấy bài từ hàng đợi ra để chấm.

5.4.1. Kafka Producer

Trong KBM, các bài nộp (submission) được gửi qua Kafka khi sinh viên nộp bài:

Listing 5.1: Kafka Producer — Core Service gửi bài nộp

```
// Producer: Core Service gửi submission vào Kafka
@Service
public class SubmitProducer extends BaseProducerService<SubmitMessage> {

    private static final String TOPIC = "submissions";

    public void sendSubmission(Submission submission) {
        SubmitMessage message = SubmitMessage.builder()
            .submissionId(submission.getId())
            .questionId(submission.getQuestionId())
            .userId(submission.getUserId())
            .sqlContent(submission.getSqlContent())
            .databaseType(submission.getDatabaseType())
            .build();

        // key = userId → đảm bảo ordering per user
        // Rút gọn: mã thật cần kiểm kết quả send;
        // ghi DB và gửi Kafka nguyên tử thì dùng Outbox (Chương 10)
        kafkaTemplate.send(TOPIC, submission.getUserId().toString(), message);
    }
}
```

5.4.2. Kafka Consumer

Phần nâng cao (có thể lướt qua)

Đến hết §5.4, chương phân tích sâu các "bẫy" vận hành Kafka trong production: Head-of-Line Blocking và rebalance. Từ §5.5 trở lại là nội dung cốt lõi. Người đọc mới chỉ cần nắm nguyên lý cốt lõi: consumer đọc message từ topic theo group và xử lý. Các nội dung chuyên sâu về rủi ro vận hành bên dưới có thể quay lại tra cứu khi trực tiếp triển khai hệ thống thực tế.

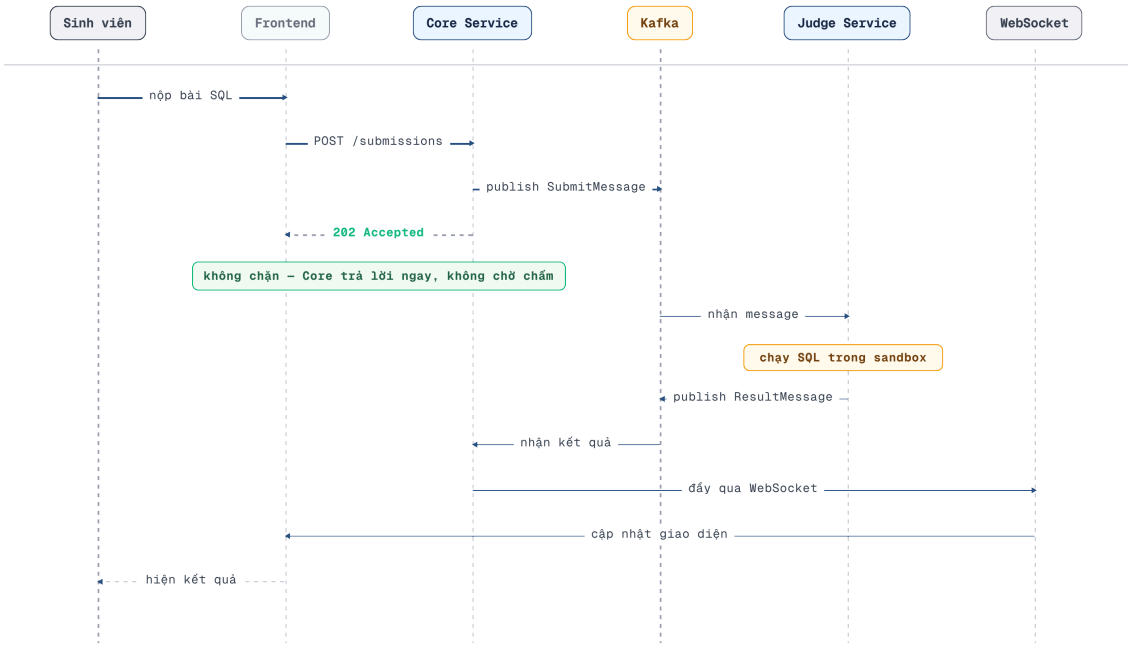
Judge Service tiêu thụ các bài nộp (submission) và trả kết quả:

Listing 5.2: Kafka Consumer — cách cài đặt đồng bộ đơn giản (naive) gây Head-of-Line Blocking

```
// Consumer: Judge Service nhận và xử lý (Naive, Blocking)
@KafkaListener(
    topics = "submissions",
    groupId = "judge-group",
    containerFactory = "kafkaListenerContainerFactory"
)
public void processSubmission(SubmitMessage message) {
    // CẢNH BÁO THIẾT KẾ (ANTI-PATTERN): Việc gọi execute() đồng bộ ở đây sẽ chặn
    // thread của Kafka consumer. Nếu task SQL Judge chờ quá lâu và vi phạm cấu hình
    // max.poll.interval.ms, consumer bị coi là đã hỏng và tự rời nhóm, gây ra
    // rebalance loop (Rebalance Storm), làm gián đoạn toàn hệ thống.
    JudgeResult result = sqlExecutorService.execute(message);

    // Gửi kết quả ngược lại qua topic khác
    resultProducer.send("judge-results", message.getSubmissionId(), result);
}
```

5.4.3. Luồng hoàn chỉnh

Sau khi đã xem xét từng thành phần (Producer, Kafka, Consumer) riêng lẻ, sơ đồ dưới đây ghép nối chúng thành trọn luồng: từ lúc sinh viên bấm "Nộp bài" đến khi nhận kết quả chấm trên giao diện.



Hình 5.6: Luồng hoàn chỉnh — từ lúc nộp bài qua Kafka đến kết quả qua WebSocket

Lưu ý: response là **202 Accepted** (không phải 200 OK) — nghĩa là "request đã nhận, đang xử lý". Người dùng nhận kết quả qua thông báo WebSocket, không phải HTTP response.

Thiếu xử lý lỗi trong Kafka pipeline

Một số pipeline của KBM vẫn cần được gia cố (hardening) để đối phó với tình trạng lỗi xử lý thông điệp (message failure): nếu dịch vụ chấm điểm (Judge Service) ngừng hoạt động đột ngột (crash), thông điệp có thể bị mất hoặc bị xử lý lặp tùy thuộc vào cơ chế commit offset. **Giải pháp khuyến nghị** theo [2a, Ch.3]: dùng manual offset commit kết hợp với dead letter topic cho các giao dịch thất bại. **Lộ trình chuyển đổi (Migration path)**: (1) chuyển sang manual commit, (2) áp dụng `@RetryableTopic` cùng Dead Letter Topic (DLT), (3) triển khai consumer lag monitoring (giám sát độ trễ tiêu thụ). Riêng lỗi giải tuần tự hóa (như JSON hỏng xảy ra trước khi vào `@KafkaListener`), cần bọc `ErrorHandlingDeserializer` để đẩy thẳng thông điệp hỏng sang DLT thay vì để consumer lặp vô hạn. Lưu ý: retry topic không giữ thứ tự xử lý ban đầu; nếu luồng chấm cần thứ tự theo user thì phải dùng cơ chế retry giữ thứ tự hoặc kiểm soát sequence tại nơi xử lý.

Ngoài lỗi xử lý thông điệp, cách tiêu thụ submission trên Kafka còn một rủi ro về hiệu năng:

Head-of-Line Blocking trong Kafka

Cần phân biệt rõ: dùng Kafka để *nhận* và *đệm* submission (enqueue nhanh, throughput cao, replay) là hợp lý — đó chính là lý do KBM chọn Kafka. Vấn đề chỉ phát sinh khi *xử lý đồng bộ* tác

vụ chấm dài (hàng chục giây) *ngay trên consumer thread*: lúc đó nó trở thành anti-pattern nghiêm trọng. Kafka hoạt động theo cơ chế partition, một thông điệp xử lý chậm sẽ chặn toàn bộ các thông điệp phía sau trong cùng partition (hiện tượng **Head-of-Line Blocking**). Với các tác vụ chạy dài không yêu cầu thứ tự nghiêm ngặt (strict ordering), mô hình Competing Consumers của **RabbitMQ** tỏ ra hiệu quả hơn. Nếu bắt buộc phải dùng Kafka, hệ thống cần triển khai Thread Pool phía sau consumer để tránh nghẽn. (Lưu ý: Nếu tự xây dựng Thread Pool, việc quản lý manual offset commit sẽ rất phức tạp vì các task hoàn thành không theo thứ tự, dễ gây mất dữ liệu khi service crash. Do đó, nên ưu tiên dùng các thư viện chuyên dụng đã xử lý sẵn bài toán này như **Confluent Parallel Consumer**. Từ Kafka 4.2, share group (KIP-932) cũng là một lựa chọn: nhiều consumer xử lý đồng thời trên cùng partition và xác nhận từng record, đổi lại không còn thứ tự theo partition).

Lưu ý (Rebalance Storm): Khi consumer xử lý một batch các long-running task, tổng thời gian rất dễ vượt quá giới hạn `max.poll.interval.ms` (mặc định là 5 phút). Khi đó, consumer bị coi là đã hỏng và tự rời nhóm, nhóm phải rebalance, trong khi tác vụ thực tế vẫn đang chạy. Điều này dẫn đến vòng lặp rebalance vô tận (Endless Rebalance). Để khắc phục, cần tuning cấu hình: tăng `max.poll.interval.ms` hoặc giảm `max.poll.records` (số lượng message mỗi lần poll) để đảm bảo thời gian xử lý batch luôn nằm trong ngưỡng an toàn.

5.5. Delivery Guarantees & Idempotency

Quản lý thông điệp giống việc thu bài thi: mỗi bài phải được ghi nhận đúng một lần, không sót, không chấm hai lần.

5.5.1. Ba cấp độ đảm bảo

Đây là một trong những khái niệm quan trọng nhất trong messaging — và cũng dễ bị hiểu sai nhất [7, Ch.11]. Trong hệ thống phân tán, cơ chế giao nhận thông điệp có ba cấp độ đảm bảo:

- **At-most-once (gửi tối đa một lần)**: Ưu tiên tốc độ, đổi lại là rủi ro mất thông điệp.
- **At-least-once (gửi ít nhất một lần)**: Đảm bảo độ tin cậy, nhưng có thể sinh ra thông điệp trùng lặp — consumer buộc phải xử lý idempotent.
- **Exactly-once (xử lý đúng một lần)**: Cấp độ cao nhất, đổi lại là chi phí hiệu năng đắt đỏ.

Kleppmann trong [7, Ch.11] phân tích: **exactly-once semantics** trong thực tế là *effectively-once* — đạt được bằng cách kết hợp at-least-once delivery + idempotent processing. Kafka transactions (từ bản 0.11) hỗ trợ exactly-once **trong Kafka**, nhưng không xuyên hệ thống (Kafka → DB).

5.5.2. Idempotency — thiết kế để chịu được duplicate

Khi sử dụng chiến lược at-least-once (phổ biến nhất), consumer *có thể* nhận cùng một thông điệp nhiều lần. Mã nguồn phải đảm bảo idempotency — xử lý nhiều lần cho cùng kết quả [5, Ch.8]:

Listing 5.3: *Idempotent consumer — kiểm tra trước khi xử lý*

```
// SAI: Không idempotent — chấm trùng = điểm sai
@KafkaListener(topics = "submissions")
public void process(SubmitMessage msg) {
    JudgeResult result = judge(msg);
    submissionRepository.updateScore(msg.getSubmissionId(), result.getScore());
    // Nếu message trùng → score bị cộng dồn hoặc ghi đè không đúng state
}
```

```
// ĐÚNG: Idempotent — check trước khi xử lý
@KafkaListener(topics = "submissions")
public void process(SubmitMessage msg) {
    if (submissionRepository.isAlreadyJudged(msg.getSubmissionId())) {
        log.info("Submission {} already judged, skipping", msg.getSubmissionId());
        return; // Skip duplicate
    }
    JudgeResult result = judge(msg);
    submissionRepository.updateScore(msg.getSubmissionId(), result.getScore());
}
```

Rocha trong [5, Ch.8] nhấn mạnh: event consumption phải **idempotent** — xử lý nhiều lần vẫn cho cùng kết quả — và, khi domain cho phép, nên thiết kế thêm tính giao hoán/kết hợp (commutative/associative) để chịu được thứ tự xáo trộn. Với các luồng bắt buộc đúng thứ tự, hệ thống nên dựa vào ordering theo partition key (mục trên) và version/sequence của aggregate để từ chối hoặc trì hoãn event sai thứ tự, thay vì cố ép mọi phép xử lý phải có tính giao hoán.

Race Window trong Check-then-write

Mẫu `isAlreadyJudged()` rồi `updateScore()` ở trên đủ để minh họa ý tưởng, nhưng *chưa* an toàn tuyệt đối khi hai bản sao consumer xử lý cùng một message song song: cả hai có thể cùng vượt qua bước kiểm tra trước khi bất kỳ bên nào kịp ghi. Để idempotency chắc chắn, cần dựa vào **ràng buộc ở tầng database** — ví dụ `UNIQUE constraint` trên `submissionId`, thao tác `UPSERT`/`INSERT ... ON CONFLICT DO NOTHING`, hoặc một bảng "đã xử lý" (processed-message table) trong cùng transaction với việc ghi kết quả.

Trong KBM, idempotency đặc biệt quan trọng ở SQL Judge: submission có thể được retry qua Kafka/queue, và bộ điều phối `kbm-judge-sql` có thể gọi lại cùng một worker `kbm-sandbox-*` sau timeout. Mỗi command nên có `submissionId` hoặc `judgeRunId` ổn định; consumer/worker kiểm tra trạng thái trước khi ghi kết quả. Nếu đã chấm xong, hệ thống trả về kết quả hiện có hoặc bỏ qua message, thay vì tạo thêm một lần chấm mới.

5.6. Event Schema Design

Mọi thông báo gửi đi trong trường học đều cần tuân thủ một biểu mẫu chuẩn. Trong kiến trúc sự kiện, việc thống nhất cấu trúc dữ liệu chung này là yêu cầu bắt buộc, giúp các dịch vụ đọc hiểu nhau dễ dàng mà không vấp phải lỗi tương thích.

5.6.1. Cấu trúc một event

Theo tinh thần thiết kế của Rocha [5, Ch.8], mỗi event nên tách thành hai phần rõ rệt:

Listing 5.4: *Cấu trúc event với metadata và payload*

```
{
  "metadata": {
    "eventId": "e7d3f2a1-...",
    "eventType": "SubmissionJudged",
    "schemaVersion": "1.0",
    "occurredAt": "2026-03-10T14:30:00Z",
    "source": "kbm-judge-sql",
    "correlationId": "sub-456",
```

```
    "causationId": "cmd-sub-456"
  },
  "payload": {
    "submissionId": "sub-456",
    "questionId": "q-789",
    "userId": "user-001",
    "result": "CORRECT",
    "executionTimeMs": 245
  }
}
```

Một event tiêu chuẩn cần tách biệt rõ ràng giữa hai phần: **metadata** cung cấp siêu dữ liệu phục vụ chống trùng lặp (deduplication), quản lý phiên bản (versioning) và đối chiếu ngữ cảnh qua bộ định danh chuẩn — `eventId` (định danh bất biến của thông điệp), `correlationId` (gom nhóm toàn bộ tiến trình nghiệp vụ, ví dụ một lượt nộp bài), và `causationId` (ID của lệnh hoặc sự kiện trực tiếp sinh ra thông điệp này); ngữ cảnh truy vết (`traceparent`/`tracestate` chuẩn W3C) không nằm trong envelope mà đi trong Kafka header, do instrumentation tự truyền ở tầng transport (xem Chương 11); trong khi đó, **payload** chứa dữ liệu nghiệp vụ cụ thể của domain. Trong hệ sinh thái Cloud Native, cấu trúc tách biệt giữa metadata và payload này đã được chuẩn hóa thành tiêu chuẩn công nghiệp **CloudEvents** (do CNCF quản lý), giúp dễ tích hợp với các công cụ trung gian.

5.6.2. Bốn loại message

Rocha phân loại message thành 4 loại, mỗi loại có mục đích và naming convention riêng [5, §3.1.4]:

Bảng 5.3: *Bốn loại message trong kiến trúc hướng sự kiện*

Loại	Mô tả	Cách đặt tên	Ví dụ KBM	Phân phối
Command	Yêu cầu thực hiện hành động — *có thể bị từ chối*	Verb imperative	`JudgeSubmission`, `SendNotification`	Point-to-point
Event	Thông báo sự kiện đã xảy ra — *sự thật, không thể reject*	Past participle	`SubmissionJudged`, `ContestStarted`	Pub/sub
Document	Snapshot toàn bộ entity khi thay đổi — *full state, hiện thân của Event-Carried State Transfer Pattern*	Noun	`SubmissionDocument`, `UserDocument`	Pub/sub
Query	Yêu cầu thông tin, không thay đổi state	Noun + request	Thường qua HTTP (broker có thể dùng Request-Reply Pattern nhưng ít phổ biến)	Sync (hoặc RPC)

5.6.3. Schema evolution

Quá trình tiến hóa cấu trúc dữ liệu (Schema evolution) diễn ra tương tự như việc nâng cấp API versioning đã đề cập ở Chương 3. Việc thay đổi event schema cũng đòi hỏi một chiến lược tiến hóa nhất quán [7, Ch.4]. Hãy hình dung biểu mẫu đăng ký học phần của sinh viên qua các năm:

- **Thêm field optional (Tương thích ngược)**: Bổ sung một cột "Nguyện vọng 2" không bắt buộc. Biểu mẫu cũ vẫn dùng được, biểu mẫu mới mang thêm thông tin — một thay đổi an toàn (✓).
- **Bỏ field mà consumer còn dùng (Breaking change)**: Xóa bỏ cột "Mã số sinh viên". Các hệ thống (consumer) cũ vẫn phụ thuộc vào thông tin này để xử lý hồ sơ, dẫn đến lỗi toàn hệ thống (×).
- **Đổi field type (Breaking change)**: Thay đổi mã môn học từ dạng số sang chuỗi ký tự dài. Các hệ thống cũ vốn lưu mã môn là số nguyên sẽ lập tức bị gián đoạn (×).

Giải pháp được khuyến nghị là triển khai một **Schema Registry** (như Confluent Schema Registry hoặc AWS Glue) hoạt động như một kho schema trung tâm. Giống như việc Phòng Đào tạo phải kiểm duyệt và đóng dấu mọi thay đổi trên mẫu đơn trước khi phát hành, mỗi sự thay đổi về schema đều được hệ thống tự động kiểm chứng mức độ tương thích trước khi cho phép ban hành (publish).

KBM thiếu event schema chuẩn

Các thông điệp trong KBM là các đối tượng Java thuần (POJO) được tuần tự hóa (serialize) dưới định dạng JSON — không có metadata chuẩn (eventId, occurredAt, correlationId), không có schema registry, không có version. Khi Judge Service thay đổi định dạng thông điệp, Core Service có thể gặp sự cố (crash) nếu chưa được cập nhật. **Lộ trình chuyển đổi**: (1) thêm metadata wrapper cho tất cả thông điệp Kafka, (2) sử dụng `eventId` để kiểm tra idempotency, (3) cân nhắc Avro + Schema Registry khi quy mô đội ngũ phát triển mở rộng.

5.7. Kafka vs DB Queue — chọn broker theo bài toán

Không phải mọi luồng xử lý bất đồng bộ đều cần Kafka. Việc chọn đúng cơ chế hàng đợi phụ thuộc vào yêu cầu cụ thể của từng bài toán trong hệ thống.

Bảng 5.4: Chọn Kafka hay DB queue theo bài toán

Bài toán	Lựa chọn	Lý do
Nộp bài SQL / chế độ thi	Kafka	Cần throughput, ordering theo key, replay và consumer group
SQL Judge worker dispatch (`kbm-judge-sql` → `kbm-sandbox-*`)	Kafka hoặc DB queue + worker	Mỗi tác vụ (job) mang một định danh (`submissionId`/`judgeRunId`) ổn định, thuận tiện cho việc thử lại (retry) và đảm bảo idempotency; các worker theo từng DBMS (`kbm-sandbox-mysql`, `kbm-sandbox-sqlserver`…) xử lý độc lập
DevOps Lab operations	DB queue	Job gắn chặt với state trong database, cần transaction enqueue cùng thay đổi domain
Audit/learning analytics	Kafka	Nhiều consumer có thể đọc cùng event stream

Việc sử dụng DB queue không làm giảm đi tính chất microservices của hệ thống so với Kafka. Nó là lựa chọn hợp lý khi lưu lượng vừa phải, số lượng consumer ít, và yêu cầu quan trọng nhất là tính nguyên tử (atomicity) giữa trạng thái nghiệp vụ và bản ghi tác vụ. Kafka tỏ ra vượt trội hơn khi hệ thống cần phân tán thông điệp (fan-out), phát lại dữ liệu (replay), phân vùng (partitioning) và xử lý luồng (stream processing).

5.8. WebSocket — Real-time Notifications

Thay vì để sinh viên phải liên tục làm mới trang web để xem điểm thi, hệ thống cần một kênh liên lạc chủ động. WebSocket mở một kết nối hai chiều, cho phép kết quả hiện lên ngay khi Judge Service chấm xong bài.

5.8.1. Bài toán

Sau khi Judge Service chấm xong, kết quả cần đến tay sinh viên. Có bốn phương án:

- **Polling**: Client hỏi lại định kỳ. Dễ cài đặt, nhưng lãng phí băng thông và độ trễ cao.
- **Long polling**: Client giữ kết nối chờ đến khi server có dữ liệu mới. Khắc phục được độ trễ, nhưng khó mở rộng.

- **Server-Sent Events (SSE)**: Kết nối một chiều từ server tới client trên HTTP. Đơn giản hơn WebSocket khi không cần chiều ngược lại.
- **WebSocket**: Kết nối hai chiều, server chủ động đẩy dữ liệu thời gian thực. Đổi lại là chi phí quản lý trạng thái kết nối.

KBM sử dụng **STOMP over WebSocket** — sub-protocol messaging chạy trên WebSocket, triển khai bằng Spring WebSocket (có thể bật thêm SockJS fallback qua `.withSockJS()` khi cần hỗ trợ trình duyệt cũ; khi đó client phải dùng `webSocketFactory` thay cho `brokerURL`):

Listing 5.5: WebSocket server — đẩy kết quả chấm bài qua STOMP

```
// Server-side: push kết quả chấm bài
@Service
public class NotificationService {
    private final SimpMessagingTemplate messagingTemplate;

    public NotificationService(SimpMessagingTemplate messagingTemplate) {
        this.messagingTemplate = messagingTemplate;
    }

    public void notifyJudgeResult(UUID userId, JudgeResult result) {
        messagingTemplate.convertAndSendToUser(
            userId.toString(),
            "/queue/notifications",
            new JudgeNotification(result)
        );
    }
}
```

Phía trình duyệt, client đăng ký (subscribe) vào hàng đợi riêng của người dùng để nhận thông báo (Listing 5.6):

Listing 5.6: WebSocket client — đăng ký (subscribe) nhận kết quả theo thời gian thực

```
// Client-side: subscribe nhận kết quả
const stompClient = new StompJs.Client({
    brokerURL: 'wss://<gateway-host>/ws'
});

stompClient.onConnect = () => {
    stompClient.subscribe('/user/queue/notifications', (message) => {
        const result = JSON.parse(message.body);
        showNotification(`Kết quả: ${result.status}`);
    });
};
stompClient.activate();
```

WebSocket phù hợp cho **thông báo ở cuối pipeline** (kết quả chấm bài, cập nhật bảng xếp hạng trong kỳ thi) nhưng không thay thế Kafka cho **messaging giữa các service** — hai vai trò khác nhau. Khi service chạy nhiều instance, kết nối của sinh viên chỉ nằm trên một instance, nên thông báo phải đi qua một broker chung để tới đúng nơi.

5.9. Case study: KBM

Kỳ thi tính giờ trên KBM là phép thử rõ nhất cho đường ống bất đồng bộ. Tải không lớn nếu tính trung bình, nhưng dồn cục vào vài phút cuối giờ làm bài.

5.9.1. Bài toán kỳ thi

Trong chế độ thi, khoảng 200 sinh viên đồng thời nộp bài trong thời gian giới hạn (60–120 phút). Yêu cầu:

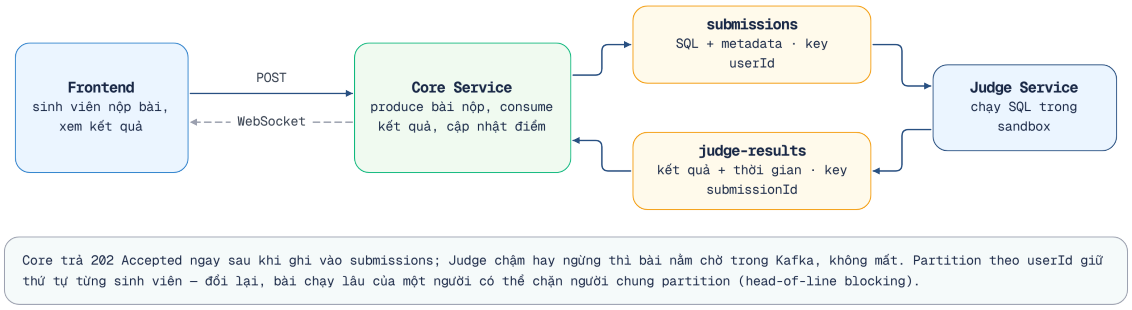
- Thông lượng cao: Xử lý hàng trăm lượt nộp bài mỗi phút vào cao điểm cuối giờ (kịch bản chất lượng §1.5)
- Thứ tự công bằng: Đảm bảo thứ tự xử lý cho từng user
- Phản hồi thời gian thực: Sinh viên thấy kết quả ngay
- Cập nhật bảng xếp hạng: Bảng xếp hạng cập nhật liên tục

⚠ Cạm bẫy: key co-location

Yêu cầu "thứ tự công bằng" (partition theo `userId`) lại có mặt trái: khi nhiều user rơi vào cùng một partition (key co-location — hệ quả bình thường của partition theo key, không phải hash collision), bài nộp chậm của người này chặn người kia. Sinh viên A nộp một truy vấn chạy chậm (ví dụ JOIN không điều kiện trên bảng lớn) mất 30 giây để xử lý sẽ chặn đứng bài nộp chỉ cần 1 giây của sinh viên B nếu cả hai thông điệp rơi vào cùng một phân vùng (partition). Đây chính là Head-of-Line Blocking đã phân tích ở trên.

5.9.2. Kiến trúc pipeline 2 Kafka topic

Hình 5.7 mô tả pipeline chấm bài trong chế độ thi.



Hình 5.7: *Kiến trúc pipeline 2 Kafka topic chấm bài trong chế độ thi*

Pipeline chấm bài dùng hai Kafka topic: `submissions` đưa bài nộp tới Judge, `judge-results` đưa kết quả về Core; chặng cuối đẩy thông báo theo `userId` tới trình duyệt đi qua WebSocket (§5.8), không tạo thêm Kafka topic thứ ba. Trong pipeline hai topic này, lớp `NotificationService` đẩy WebSocket có thể vận hành trực tiếp trong Core Service hoặc tách thành dịch vụ riêng (`kbm-notification`) khi mở rộng mà không làm thay đổi luồng sự kiện cốt lõi.

Bảng 5.5: *Chi tiết các Kafka topic trong pipeline*

Topic	Producer	Consumer	Message	Key
`submissions`	Core Service	Judge Service	SQL + metadata	`userId`
`judge-results`	Judge Service	Core Service	Result + timing	`submissionId`

5.9.3. Phân tích các vấn đề

Đối chiếu với các giải pháp khuyến nghị, cấu hình hiện tại còn năm điểm yếu:

Bảng 5.6: Phân tích vấn đề Kafka pipeline trong KBM

#	Vấn đề	Hiện trạng	Giải pháp khuyến nghị [2a]
1	**Commit offset chưa gắn với xử lý**	Offset có thể được commit trước khi xử lý xong, nhất là khi xử lý ở luồng khác → message loss	Manual commit sau khi xử lý thành công
2	**Không có DLT**	Thông điệp lỗi bị thử lại (retry) vô hạn hoặc bị mất	Cấu hình Dead Letter Topic cho các thông điệp lỗi
3	**Không idempotent**	Thông điệp trùng lặp (duplicate) có thể tạo ra kết quả sai	Kiểm tra `submissionId` trước khi chấm điểm (judge)
4	**Thiếu monitoring**	Không nắm được độ trễ của phía tiêu thụ (consumer lag)	Thiết lập Kafka consumer lag metric và cảnh báo
5	**Single partition**	Tất cả bài nộp (submission) bị đẩy vào 1 partition dẫn đến tắc nghẽn (bottleneck)	Tăng số partition và giữ nguyên `key = userId` đang dùng để phân tán tải theo sinh viên

5.9.4. Lộ trình củng cố pipeline

Quá trình chuyển đổi và củng cố pipeline cần được chia thành các giai đoạn rõ ràng. Trong **Giai đoạn 1 — Độ tin cậy** (ưu tiên cao), hệ thống cần đảm bảo không mất mát dữ liệu bằng cách sử dụng manual offset commit kết hợp `@RetryableTopic` (với 3 lần thử lại) và cấu hình Dead Letter Topic. Đồng thời, phải bổ sung bước kiểm tra idempotency thông qua `submissionId`.

Chuyển sang **Giai đoạn 2 — Observability** (cũng ưu tiên cao), trọng tâm là khả năng giám sát. Hệ thống cần triển khai Kafka consumer lag monitoring sử dụng Kafka Exporter và Prometheus để theo dõi độ trễ. Việc truyền `traceId` và `correlationId` xuyên suốt pipeline (từ submission đến judge, result, và notification) cũng bắt buộc để dễ dàng truy vết (tracing).

Cuối cùng, ở **Giai đoạn 3 — Hiệu năng** (khi cần mở rộng quy mô), hệ thống bắt buộc phải tách biệt (decouple) Consumer Thread và Execution Thread. Consumer chỉ nên chịu trách nhiệm lấy dữ liệu rồi đẩy vào Local Thread Pool để xử lý thực tế (kèm cơ chế quản lý offset cẩn trọng để tránh mất dữ liệu do out-of-order execution), hoặc ưu tiên sử dụng thư viện chuyên dụng như Confluent Parallel Consumer. Chỉ tăng số lượng partition (ví dụ lên 3) sẽ không giải quyết được tắc nghẽn nếu nhiều bài nộp chạy chậm cùng lúc. Muốn tăng thông lượng, còn phải thêm các instance Judge vào consumer group — với điều kiện số partition được tăng tương ứng: số consumer hoạt động đồng thời trong một group không thể vượt quá số partition của topic, instance dư ra sẽ chỉ đứng chờ (idle). Tăng partition cũng đổi ánh xạ key → partition cho message ghi sau đó; nếu cần giữ thứ tự theo user, phải kiểm soát thời điểm chuyển đổi, chẳng hạn tạm dừng producer và xử lý hết backlog trước. Số partition lớn hơn còn làm rebalance chậm hơn và tốn tài nguyên broker hơn.

⚠ Sai lầm thường gặp khi làm việc với Message Broker

- **Auto-commit offset khi xử lý chạy ở luồng khác**: Consumer xác nhận (acknowledge) thông điệp trước khi xử lý xong. Giống như việc chuyên viên đào tạo đóng dấu "đã xử lý" lên đơn đăng ký tín chỉ trước khi thực sự cập nhật vào hệ thống; nếu hệ thống tắt nguồn đột ngột lúc đó, đơn sẽ bị quên lãng vĩnh viễn. *Phòng tránh*: Dùng manual offset commit — chỉ commit *sau khi* xử lý thành công.
- **Không có Dead Letter Topic**: Thông điệp lỗi bị thử lại (retry) vô hạn hoặc làm tắc nghẽn (block) toàn bộ pipeline. Một đơn xin bảo lưu bị lỗi định dạng có thể chặn đứng mọi đơn từ khác xếp sau nó (hiện tượng poison pill). *Phòng tránh*: Cấu hình DLT (`@RetryableTopic` trong

Spring Kafka) — chuyển các message lỗi sau N lần thử sang một topic riêng (DLT) để xử lý thủ công.
- **Không thiết kế idempotent consumer**: Giả định mỗi message chỉ đến đúng một lần. Khi Kafka rebalance, một bài tập có thể bị chấm hai lần, khiến sinh viên nhận được hai kết quả khác nhau. *Phòng tránh*: Kiểm tra trạng thái trước khi xử lý, nếu đã chấm rồi thì bỏ qua (§5.5).
- **Chọn broker vì quen thuộc, không vì bài toán**: Sử dụng RabbitMQ cho luồng sự kiện (event streaming) chỉ vì đội ngũ phát triển đã quen thuộc với công nghệ này. Hậu quả thường không phải quá tải mà là thiếu thứ cần dùng: RabbitMQ xóa thông điệp sau khi consumer xác nhận nên không phát lại được luồng sự kiện để chấm lại hay cho consumer mới. *Phòng tránh*: Đánh giá bài toán trước (§5.2) — chọn Kafka cho event streaming, RabbitMQ cho task queue.

Các pattern trong chương có minh họa tương tác đi kèm tại companion web:

Interactive Demo

- **Cơ chế hoạt động của Message Broker**: message-broker.html
- **Idempotency và retry**: idempotency-retry.html

5.10. Tổng kết

Giao tiếp bất đồng bộ làm nhẹ ba giới hạn cốt lõi của sync: temporal coupling, cascading failure, và độ trễ. Hai trường phái message broker — durable (Kafka) và ephemeral (RabbitMQ) — phục vụ các bài toán khác nhau, và nhiều hệ thống dùng cả hai.

Kafka với mô hình event log (durable, replayable, high-throughput) trở thành nền tảng phổ biến nhất cho event-driven architecture. RabbitMQ với smart routing và flexible messaging phù hợp cho task queue, delayed job, và các mẫu định tuyến phức tạp. Partition, consumer group, và exchange type cho phép mỗi broker mở rộng theo cách riêng.

Delivery guarantees (at-most/at-least/exactly-once) và idempotency là hai khái niệm không thể tách rời. Thiết kế event schema chuẩn — với metadata, versioning, và correlation — là đầu tư cần thiết giúp hệ thống dễ dàng gỡ lỗi và tiến hóa.

Phân tích KBM cho thấy pipeline Kafka hoạt động nhưng thiếu nhiều giải pháp khuyến nghị quan trọng: xử lý lỗi, idempotency, giám sát. Đồng thời, DevOps Lab nhắc lại một bài học thực tế: có những job queue nên nằm gần database thay vì ép qua Kafka. Đây là nợ kỹ thuật (technical debt) và sự đánh đổi phổ biến khi các đội ngũ nhỏ ưu tiên tính năng trước độ tin cậy (reliability).

Ở Chương 6, chúng ta sẽ đối mặt với một bài toán khó của hệ phân tán: **giao dịch phân tán (distributed transaction)**. Khi dữ liệu nằm ở nhiều service khác nhau, làm thế nào để bảo đảm tính nhất quán? Saga pattern sẽ là câu trả lời.

5.11. Đọc thêm

Sách tham khảo chính:

1. [5] Hugo Rocha, *Practical Event-Driven Microservices Architecture* — Ch.1: Why EDA; Ch.3: Kafka; Ch.8: Event Schema Design

2. [2a] Chris Richardson, *Microservices Patterns*, 1st Ed. — Ch.3: Async Messaging, Transactional Outbox
3. [7] Martin Kleppmann, *Designing Data-Intensive Applications* — Ch.11: Stream Processing, Event Logs

Sách bổ trợ:

4. [3] Mitra & Nadareishvili, *Microservices: Up and Running* — Ch.4: Event-Driven Communication

Nguồn trực tuyến:

- Apache Kafka documentation — kafka.apache.org/documentation
- Confluent Schema Registry — docs.confluent.io/platform/current/schema-registry
- Spring Kafka reference — docs.spring.io/spring-kafka

Bài tập: 5-0, 5-1, 5-2 ở phần Bài tập & case study cuối sách.

CHƯƠNG 6

Giao dịch phân tán — Saga Pattern

> "A saga is a sequence of local transactions. Each local transaction updates data within a single service using the familiar ACID transaction frameworks and libraries..."
> — Chris Richardson, *Microservices Patterns* [2a, Ch.4]

MỤC TIÊU HỌC TẬP

- Hiểu tại sao giao dịch phân tán (2PC) không phù hợp với microservices
- Nắm vững Saga pattern: định nghĩa, cấu trúc, và giao dịch bù trừ (compensating transaction)
- So sánh hai cơ chế điều phối: Choreography vs Orchestration
- Áp dụng các biện pháp đối phó (countermeasure) để xử lý việc thiếu isolation giữa các saga
- Hiểu eventual consistency và cách quản lý từ góc nhìn business
- Phân tích luồng nộp bài trong KBM — implicit saga và lộ trình cải tiến

Chuyển sang microservices mang lại khả năng mở rộng độc lập và triển khai linh hoạt, nhưng đồng thời phá vỡ nền tảng nhất quán dữ liệu truyền thống: transaction không còn gói gọn trong một database. Chương này giải quyết bài toán giao dịch phân tán bằng Saga pattern — từ nguyên lý đến cách KBM áp dụng cho luồng nộp bài.

Ôn lại kiến thức nền

Chương này dựa nhiều vào một số khái niệm cơ sở dữ liệu. Nếu chưa chắc, hãy nắm nhanh bốn ý sau (ví dụ trừ tiền trong ví e-learning của thẻ sinh viên để nạp quota in ấn ở thư viện):

- **Transaction**: một nhóm thao tác "tất cả hoặc không có gì". Nạp quota phải *trừ* tiền thẻ sinh viên *và cộng* quota in ấn cùng lúc; không thể trừ tiền xong rồi dừng giữa chừng.
- **ACID**: bốn đảm bảo của transaction trong một database (Atomicity, Consistency, Isolation, Durability). Chữ **I — Isolation** nghĩa là các giao dịch chạy song song không gây xung đột hay can thiệp lẫn nhau.
- **Anomaly (bất thường)**: khi thiếu isolation, hai thao tác đồng thời có thể cho kết quả sai: ví dụ hai luồng cùng đọc số dư thẻ 100.000 đồng, cùng trừ 80.000 đồng → số dư cuối sai (*lost update*).
- **Idempotency**: một thao tác chạy lại nhiều lần vẫn cho cùng kết quả như chạy một lần. "Ghi số dư = 20.000 đồng" là idempotent; "cộng thêm 20.000 đồng" thì *không* (chạy 2 lần thành 40.000 đồng).

Vấn đề trung tâm của chương: khi một "transaction" trải trên **nhiều service với database riêng**, chúng ta *không còn* ACID xuyên suốt — và Saga là cách sống chung với điều đó.

6.1. Độ phức tạp của giao dịch phân tán

6.1.1. Khi một giao dịch trải rộng trên nhiều dịch vụ

Nếu toàn bộ luồng nộp bài nằm trong một monolith, nó gói gọn trong một database transaction (minh họa):

Listing 6.1: *Transaction trong monolith — ACID đảm bảo tất cả cùng thành công hoặc cùng rollback*

```
BEGIN TRANSACTION;
  INSERT INTO submissions (id, user_id, sql_content) VALUES (...);
  INSERT INTO judge_queue (submission_id, status) VALUES (..., 'PENDING');
  UPDATE user_stats SET total_submissions = total_submissions + 1
    WHERE user_id = ...;
COMMIT;
-- Tất cả thành công hoặc tất cả rollback — ACID đảm bảo
```

Khi KBM chuyển sang microservices, dữ liệu không còn nằm chung một chỗ: Core Service lưu bài nộp trong database của mình, Judge Service ghi việc chấm vào database riêng (`judge_db`) và chạy bài trong sandbox, còn thông báo được đẩy tới sinh viên (qua `NotificationService` nội bộ hoặc dịch vụ `kbm-notification` riêng, ngoài phạm vi giao dịch). Không có "shared transaction" giữa chúng — Core không biết Judge đã chấm xong hay chưa nếu không có thông điệp báo về.

6.1.2. Tại sao 2PC không phù hợp?

Để hiểu lý do Saga Pattern ra đời, trước tiên chúng ta cần nhìn lại cách các hệ thống truyền thống xử lý giao dịch phân tán. Từ nhiều thập kỷ qua, **Two-Phase Commit (2PC)** là cơ chế tiêu chuẩn để

giải quyết bài toán đồng bộ hóa dữ liệu trên nhiều nút mạng. Ý tưởng cốt lõi của 2PC được chia thành hai giai đoạn dưới sự dẫn dắt của một thành phần trung tâm gọi là Coordinator.

Trong giai đoạn đầu (Prepare Phase), Coordinator sẽ gửi yêu cầu tới tất cả các cơ sở dữ liệu thành phần (participant), hỏi xem chúng đã sẵn sàng để ghi dữ liệu hay chưa. Chỉ khi tất cả các bên đều phản hồi "Sẵn sàng", đã ghi bền vững phần chuẩn bị và giữ khóa (lock) trên dữ liệu liên quan, Coordinator mới ghi lại quyết định commit rồi chuyển sang giai đoạn thứ hai (Commit Phase), gửi quyết định đó tới từng bên. Sự phối hợp này bảo đảm mọi bên cùng commit hoặc cùng abort; các thuộc tính còn lại của ACID, như isolation, vẫn do cơ chế giao dịch của từng cơ sở dữ liệu quyết định.

Dưới góc nhìn lý thuyết theo Kleppmann [7, Ch.9], 2PC dường như là giải pháp an toàn để bảo toàn sự nhất quán dữ liệu ở mức độ cao nhất. Tuy nhiên, khi áp dụng cho hệ thống phân mảnh cao và yêu cầu xử lý lớn, cơ chế này bộc lộ những hạn chế lớn về hiệu năng và khả năng chịu lỗi.

Hình 6.1: *Two-Phase Commit — Coordinator điều phối prepare và commit*

Richardson trong [2a, Ch.4] liệt kê các lý do chính khiến 2PC (giao dịch XA — một chuẩn kỹ thuật phổ biến cài đặt 2PC) không phù hợp cho microservices, đặc biệt khi áp dụng vào bối cảnh hệ thống thực tế như KBM:

- **Synchronous blocking**: Bộ điều phối (Coordinator) bị chặn (block) để chờ phản hồi từ tất cả các thành phần tham gia (participant); đồng thời, mỗi thành phần tham gia phải duy trì khóa giao dịch trên tài nguyên của nó cho đến khi nhận lệnh commit/abort. Trong KBM, Judge Service xử lý một bài nộp SQL có thể mất từ 5–30 giây. Điều này đồng nghĩa với việc thành phần tham gia sẽ giữ row lock và bộ điều phối bị chặn chờ trong toàn bộ khoảng thời gian đó, khiến luồng xử lý của Core Service bị giữ suốt thời gian đó, và thread pool sẽ cạn khi nhiều bài nộp cùng chờ.

- **Single point of failure**: Nếu bộ điều phối trung tâm gặp sự cố ngừng hoạt động (crash), tất cả các thành phần tham gia sẽ bị đình trệ ở trạng thái chờ. Với KBM, nếu bộ điều phối gặp sự cố ngắt kết nối ngay trong lúc Judge đang chạy, toàn bộ tiến trình nộp bài sẽ bị đình trệ vô thời hạn.
- **Reduced availability**: Giao thức yêu cầu tất cả các thành phần tham gia giao dịch phải trực tuyến cùng lúc — độ khả dụng tổng thể xấp xỉ tích độ khả dụng của từng mắt xích, nếu coi lỗi của chúng là độc lập. Chẳng hạn, nếu một giao dịch nộp bài cập nhật điểm dùng 2PC trải qua `kbm-core`, `kbm-academic` và `kbm-notification`, chỉ cần `kbm-notification` tạm ngưng bảo trì là toàn bộ luồng nộp bài cũng bị từ chối theo.
- **NoSQL incompatible**: Nhiều công nghệ lưu trữ hiện đại không hỗ trợ giao dịch XA. Ví dụ, Kafka — hệ thống truyền tin cốt lõi (messaging backbone) của KBM — hoàn toàn không hỗ trợ giao thức XA, khiến việc áp dụng 2PC là bất khả thi.
- **Performance bottleneck**: Việc giữ lock trong thời gian dài dẫn đến mức độ contention (cạnh tranh tài nguyên) rất cao. Trong chế độ thi của KBM — 200 sinh viên cùng ca thi, cao điểm cuối giờ hàng trăm lượt nộp mỗi phút theo kịch bản chất lượng ở §1.5 — cơ chế khóa này sẽ tạo ra một nút thắt cổ chai (bottleneck) làm suy giảm hiệu năng toàn hệ thống.

Kleppmann trong [7, Ch.9] bổ sung: 2PC là "blocking protocol" — nếu bộ điều phối gặp sự cố sau giai đoạn một, các participant phải chờ vô hạn. Trên production, trong chế độ thi, đây là một rủi ro không thể chấp nhận được.

2PC mâu thuẫn với Microservices

Two-Phase Commit yêu cầu tất cả participant blocking và đồng bộ — điều này trực tiếp vi phạm mục tiêu triển khai độc lập (independent deployment) và tính sẵn sàng cao (high availability) của hệ thống microservices. Khi coordinator gặp sự cố, các giao dịch đang dở bị treo và giữ khóa cho đến khi coordinator phục hồi. Có thể giảm rủi ro này bằng cách nhân bản coordinator, nhưng đổi lại là thêm độ trễ và độ phức tạp, trái với mục tiêu các service tự chủ.

Sự khác biệt này đòi hỏi thay đổi tư duy thiết kế khi chuyển sang Microservices. Trong kiến trúc monolith, các cơ chế khóa đồng bộ (synchronous locking) đảm bảo tính toàn vẹn một cách tự nhiên. Khi một giao dịch bắt đầu, cơ sở dữ liệu sẽ giữ chặt các bản ghi liên quan, ngăn cản bất kỳ sự can thiệp nào từ bên ngoài cho đến khi giao dịch hoàn tất.

Tuy nhiên, trên hệ thống phân tán như KBM, nơi mạng không đáng tin cậy, việc khóa tài nguyên bằng 2PC trở thành điểm nghẽn (bottleneck). Khi quy mô tăng, xác suất tắc nghẽn cục bộ hoặc lỗi dây chuyền càng cao. Do đó, thiết kế hệ thống phải chấp nhận tính nhất quán cuối cùng (eventual consistency) thay vì nhất quán tức thời (strong consistency).

Hệ thống phân tán hiện đại ưu tiên khả năng phục hồi (resilience) hơn là khóa tài nguyên: lỗi không bị ngăn hoàn toàn mà được xử lý qua các giao dịch bù trừ (compensating transaction). Đó là nền tảng của Saga Pattern.

Compensation thay vì Locking

Nghĩ về quy trình nộp bài thực tế: sinh viên nộp SQL → hệ thống nhận bài → Judge chấm → trả kết quả. Nếu Judge gặp lỗi, chúng ta không "hoàn tác" việc nhận bài — chúng ta *cập nhật trạng thái* thành ERROR và thông báo sinh viên thử lại. Đây chính là tư duy compensation — hành động

nghiệp vụ ngược thay vì database rollback. Richardson trong [2a, Ch.4] đề cập đến quan sát của Gregor Hohpe: "Not even Starbucks uses two-phase commit" [xem Đọc thêm]. Khi khách gọi cà phê, thu ngân nhận tiền rồi đưa biên lai, sau đó khách ra góc chờ Barista pha cà phê (bất đồng bộ). Nếu Starbucks dùng 2PC, thu ngân sẽ yêu cầu khách hàng chờ đợi, đồng thời phong tỏa trạng thái của cả thu ngân và nhân viên pha chế cho đến khi ly cà phê được hoàn thành rồi mới thực hiện thanh toán. Saga hoạt động theo cùng nguyên tắc này.

6.2. Saga Pattern — chuỗi Local Transaction + Compensation

Saga hoạt động giống như quy trình xét duyệt đồ án qua nhiều phòng ban. Mỗi bước hoàn tất sẽ tự động kích hoạt bước tiếp theo. Nếu xảy ra lỗi giữa chừng, hệ thống sẽ chạy các nghiệp vụ bù trừ để hoàn tác lại những thay đổi trước đó.

6.2.1. Định nghĩa

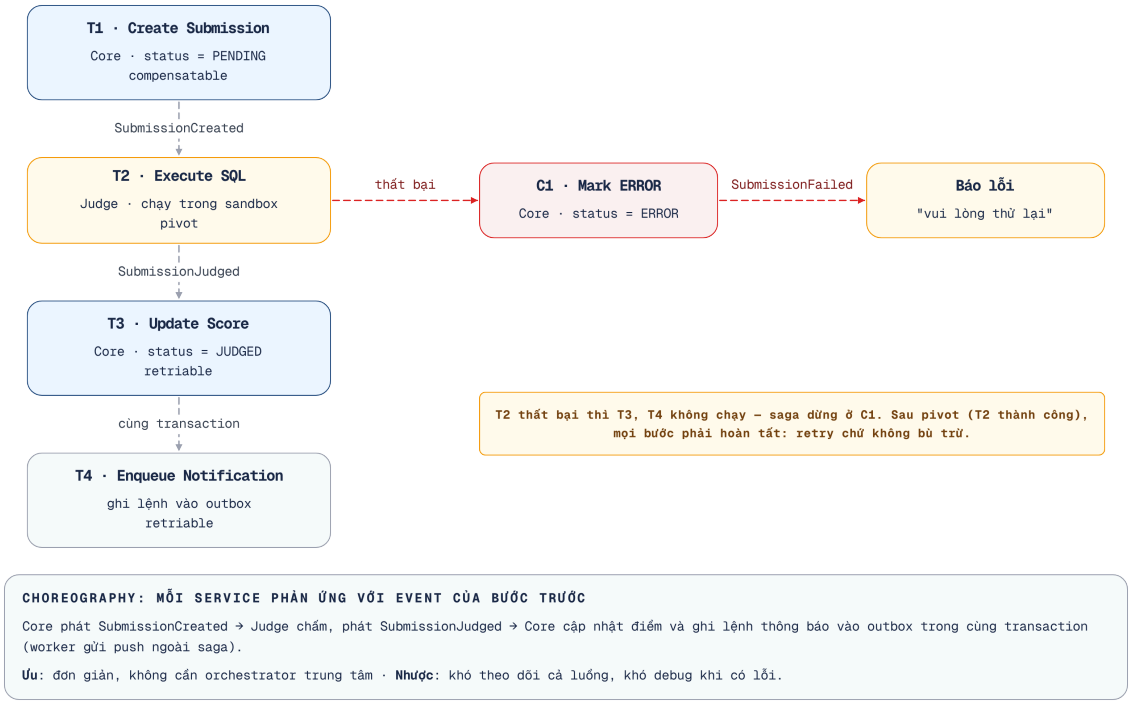
Saga là chuỗi **giao dịch cục bộ (local transaction)**, mỗi transaction cập nhật database của một service và phát đi các sự kiện hoặc thông điệp để kích hoạt giao dịch tiếp theo. Nếu một transaction thất bại, saga thực hiện **giao dịch bù trừ (compensating transaction)** để hoàn tác các thay đổi trước đó [2a, Ch.4].

6.2.2. KBM Submit Saga

Để làm rõ cách thức hoạt động của Saga, hãy xét luồng nghiệp vụ cốt lõi của KBM: quy trình nộp và chấm bài tập SQL. Khi sinh viên nộp bài, thay vì tạo ra một giao dịch phân tán khóa toàn bộ các bảng dữ liệu liên quan, hệ thống phân rã tiến trình này thành một chuỗi các giao dịch cục bộ độc lập (local transaction).

Mỗi dịch vụ trong hệ thống sẽ tự chịu trách nhiệm cập nhật kho dữ liệu riêng biệt của dịch vụ đó, sau đó sử dụng các sự kiện (event) thông qua hệ thống message broker như Kafka để truyền tín hiệu báo hiệu cho dịch vụ tiếp theo trong chuỗi. Tuy nhiên, sự phân tách này để lại một vấn đề bắt buộc phải xử lý: khi một bước giữa chuỗi thất bại, những bước đã hoàn tất trước đó vẫn nằm lại trong cơ sở dữ liệu của từng dịch vụ.

Lúc này, cơ chế giao dịch bù trừ (compensating transaction) được kích hoạt. Nếu Core Service đã ghi nhận bài nộp nhưng Judge Service lại gặp lỗi khi khởi tạo sandbox, hệ thống không thể rollback database của Core Service thông qua ACID. Thay vào đó, một sự kiện bù trừ được phát đi, yêu cầu Core Service cập nhật trạng thái bài nộp thành "Lỗi" và thông báo cho sinh viên. Chuỗi các giao dịch cục bộ và giao dịch bù trừ này chính là một Saga.



Hình 6.2: KBM Submit Saga — chuỗi local transaction và compensating transaction

Richardson [2a, Ch.4] minh họa một saga điển hình qua luồng đặt hàng thương mại điện tử, trong đó bước cấp phép thanh toán thẻ tín dụng là một giao dịch bản lề (pivot transaction) quyết định kết quả của toàn chuỗi. Đối chiếu với KBM, luồng nộp bài (Submit Saga) có ít dịch vụ tham gia hơn (Core Service, Judge Service) nhưng bước chấm chạy lâu nên việc quản lý trạng thái vẫn cần cẩn thận.

Điểm khác biệt lớn nhất nằm ở bản chất của quá trình thực thi chấm bài (Judge execution) — một tác vụ mang tính chất **long-running** (chạy nền trong thời gian dài). Trong khi việc cấp phép thanh toán thẻ thường xong trong khoảng một giây, một tiến trình chạy sandbox để biên dịch, khởi tạo dữ liệu mẫu và chạy thử các câu truy vấn SQL của sinh viên có thể kéo dài từ vài giây cho đến vài chục giây.

Thời gian thực thi dài đồng nghĩa với một asynchronous window rộng hơn hẳn — tức là khoảng thời gian mà các thao tác đồng thời hoặc lỗi mạng có thể chen ngang cũng dài theo. Hệ quả thiết kế là: không thể đối xử với mọi bước trong saga như nhau, mà phải phân loại chúng theo mức độ rủi ro, khả năng hoàn tác và tính bắt buộc phải hoàn thành.

6.2.3. Ba loại transaction trong saga

Richardson phân loại mỗi bước trong một chuỗi saga thành ba loại giao dịch riêng biệt, giúp định hình rõ ràng ranh giới và cách xử lý lỗi tại từng thời điểm. Áp dụng cơ sở lý thuyết này vào KBM Submit Saga, chúng ta có:

- **Compensatable Transaction (Giao dịch có thể bù trừ)**: Đây là những bước nằm ở phần đầu của saga, có thể bị hoàn tác (undo) một cách an toàn thông qua các giao dịch bù trừ (compensating transaction). Ở ví dụ KBM, đó là T1 (Create Submission). Nếu có lỗi xảy ra ở bước sau, chúng ta chạy C1 (Mark ERROR) để bù trừ lại.

- **Pivot Transaction (Giao dịch bản lề)**: Đây là "điểm không thể quay đầu", tương tự như thời điểm khoản thanh toán thẻ tín dụng của khách hàng được cấp phép. Sau bước này, toàn bộ saga bắt buộc phải được hoàn thành và không được phép bù trừ nữa. Bản thân pivot không có bù trừ. Trong KBM, T2 (Execute SQL) chính là bước pivot: thao tác chạy mã SQL trong sandbox sẽ thành công hoặc thất bại, quyết định số phận của toàn bộ luồng.
- **Retriable Transaction (Giao dịch có thể thử lại)**: Những bước đi theo sau điểm pivot. Chúng mang đặc tính là *bắt buộc* phải thành công và không cần giao dịch bù trừ. Nếu gặp lỗi kết nối hay timeout, hệ thống sẽ thực hiện thử lại (retry) cho đến khi hoàn tất. KBM có hai bước retriable là T3 (Update Score) và T4 (Notify Student).

Lưu ý thực tế về T4: "retry đến khi thành công" giả định bước đó chắc chắn có thể thành công (như ghi điểm vào database nội bộ). Vì việc gửi email/push thực tế có thể thất bại vĩnh viễn (địa chỉ hỏng), bước retriable đúng nghĩa của T4 nên là *ghi bền vững lệnh thông báo* — enqueue vào outbox (bảng "hộp thư đi" ghi trong cùng transaction nội bộ nên luôn có thể thành công; chi tiết pattern này ở Chương 10). Còn việc phát thông báo thực tế chạy *ngoài* saga theo kiểu best-effort với retry giới hạn, backoff và Dead Letter Queue — một thông báo hỏng địa chỉ không được phép giữ saga "treo" vô hạn hay tạo vòng lặp poison pill.

Việc cấu trúc đúng ba phần này là yêu cầu bắt buộc khi thiết kế Saga. Trình tự thực thi phải luôn là: bắt đầu với các bước **Compensatable**, vượt qua điểm **Pivot**, và kết thúc bằng các bước **Retriable**. Nếu đặt một bước có khả năng thất bại (nhưng không thể bù trừ) vào cuối chuỗi, hệ thống sẽ rơi vào trạng thái bất nhất. Trong luồng nộp bài của KBM, T2 (Execute SQL) là điểm Pivot. Một khi T2 thành công, saga chắc chắn phải đi đến đích. Các bước tiếp theo như T3 (Update Score) và T4 (Notify Student) là Retriable. Nếu các bước này gặp lỗi mạng hay timeout, hệ thống sẽ kiên trì thử lại cho đến khi thành công (với backoff; phần phát thông báo thực tế được tách ra ngoài saga như đã lưu ý ở trên), và tuyệt đối không kích hoạt giao dịch bù trừ để hủy bỏ kết quả T2.

Hiểu đúng nghĩa 'Pivot'

T2 là pivot theo nghĩa: *nếu thành công*, saga cam kết tiến tới và không quay đầu. Cần phân biệt rõ trường hợp T2 *thất bại*: đó không phải là "compensation cho pivot" (pivot không có compensation), mà là tín hiệu để rollback các bước **compensatable** đứng trước nó — ở đây là chạy C1 để đánh dấu submission ERROR. Nói cách khác, một bước pivot thất bại sẽ kích hoạt compensation của T1, chứ bản thân pivot không bị undo.

6.3. Choreography vs Orchestration

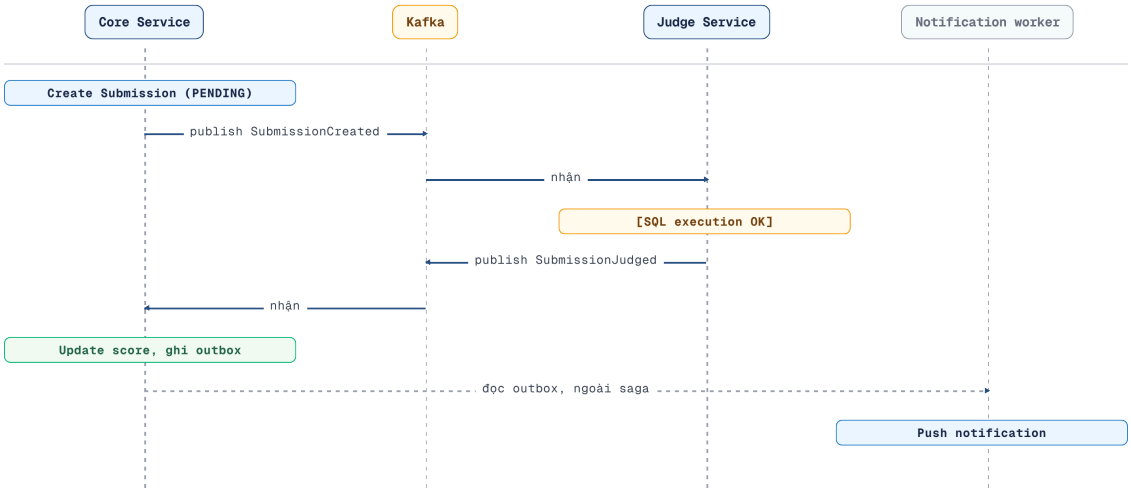
Để các dịch vụ phối hợp nhịp nhàng, hệ thống cần một nhạc trưởng hoặc các quy tắc tự quản. Việc chọn mô hình điều phối giống như quyết định cho sinh viên tự do đăng ký tín chỉ hay bắt buộc phải thông qua sự duyệt xét của cố vấn.

6.3.1. Choreography — phi tập trung, event-driven

Khi đã định hình rõ rệt các mảnh ghép tạo nên một Saga, vấn đề tiếp theo là làm thế nào để kết nối chúng. Có hai mô hình chính để giải quyết bài toán điều phối này. Mô hình đầu tiên là **Choreography**, trong đó hệ thống hoạt động hoàn toàn phi tập trung và **không có coordinator** (bộ điều phối trung tâm).

Hình ảnh so sánh gần gũi nhất là một nhóm sinh viên tổ chức sự kiện: Mỗi nhóm sinh viên trong ban tổ chức tự biết nhiệm vụ của mình; nhóm hậu cần hoàn tất công việc sẽ tự thông báo để nhóm truyền thông bắt đầu, không cần một trưởng ban (coordinator) điều phối từng bước.

Đặt vào ngữ cảnh kỹ thuật, mỗi service trong hệ thống sẽ âm thầm lắng nghe các sự kiện (event) được phát ra trên một kênh truyền thông điệp chung (như Kafka), và tự động đưa ra quyết định thực thi các hành động tiếp nối dựa trên logic nội tại của chính nó [5, §4.2]. Luồng nộp bài của KBM hiện cũng chạy theo kiểu này.



Hình 6.3: *Choreography — các service tự phối hợp qua event*

Mô hình Choreography rất **đơn giản** trong giai đoạn đầu vì không yêu cầu một thành phần điều phối tập trung. Các dịch vụ hoạt động hoàn toàn **loosely coupled** (ràng buộc lỏng lẻo); chúng không cần biết đến sự tồn tại của nhau mà chỉ phản ứng dựa trên các sự kiện (event).

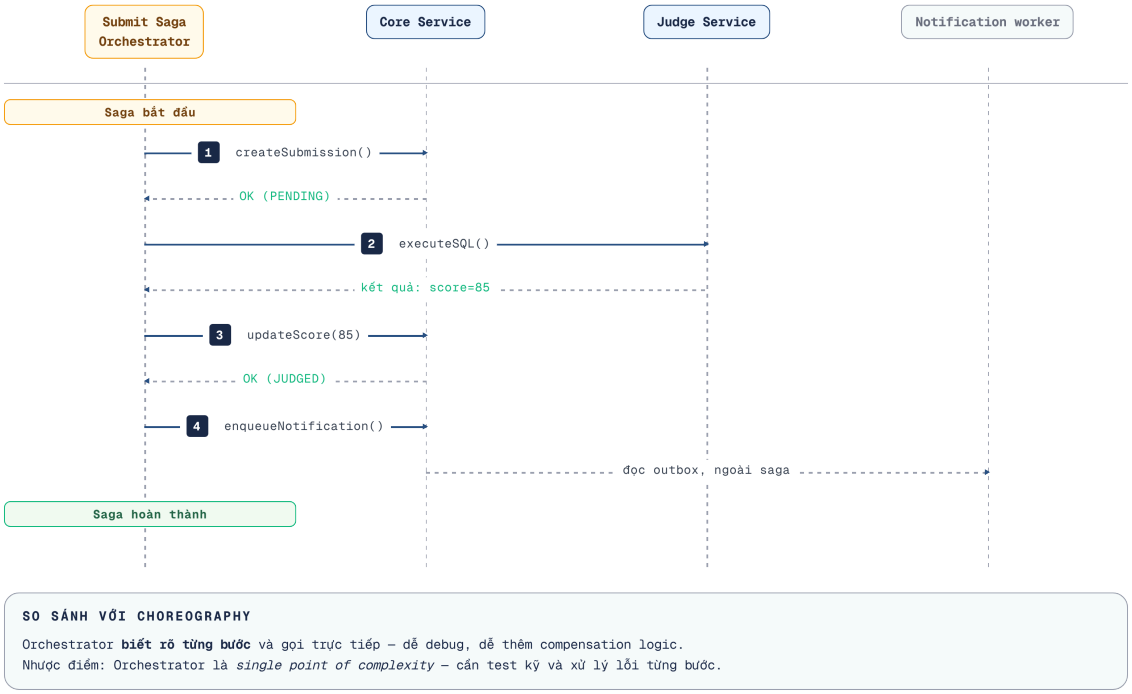
Sự độc lập này giúp quá trình **thêm mới thành phần tham gia** trở nên dễ dàng: một dịch vụ mới chỉ cần đăng ký (subscribe) vào luồng sự kiện tương ứng mà không làm ảnh hưởng dịch vụ cũ. Tuy nhiên, độ phức tạp sẽ tăng cao khi hệ thống mở rộng.

Luồng xử lý tổng thể dần trở nên **khó theo dõi** do logic bị phân tán. Nếu thiết kế không tốt, hệ thống dễ rơi vào **cyclic dependencies** (phụ thuộc vòng) — service A đăng ký sự kiện của B, rồi B lại đăng ký sự kiện của A. Hơn nữa, việc **kiểm thử (testing)** toàn bộ luồng đòi hỏi phải chạy đồng thời tất cả dịch vụ tham gia.

6.3.2. Orchestration — tập trung, điều khiển bằng lệnh

Khác với Choreography, mô hình **Orchestration** (Điều phối) sử dụng một thành phần trung tâm gọi là **Saga Orchestrator**. Orchestrator hoạt động như một lớp trưởng hoặc trưởng ban tổ chức, nắm quyền kiểm soát và điều phối toàn bộ luồng thực thi của Saga bằng cách trực tiếp giao việc cho từng thành viên và chờ báo cáo.

Orchestrator không chờ đợi các sự kiện xảy ra một cách thụ động, mà nó chủ động phát đi các mệnh lệnh (command) rõ ràng đến từng dịch vụ con và cẩn thận chờ đợi phản hồi (reply) trả về để quyết định bước đi tiếp theo [2a, Ch.4]. Nếu áp dụng mô hình orchestrator tập trung vào luồng nộp bài (Submit Saga) của KBM, kiến trúc sẽ thay đổi rõ rệt.



Hình 6.4: Orchestration — saga orchestrator điều phối toàn bộ luồng xử lý

Ưu điểm lớn nhất của Orchestration là tính **dễ hiểu** và sự rõ ràng của luồng xử lý. Toàn bộ logic điều hướng được tập trung tại một nơi, giúp kỹ sư dễ dàng theo dõi trạng thái saga. Kiến trúc này loại bỏ rủi ro phụ thuộc vòng (**không có cyclic dependencies**), do biểu đồ gọi dịch vụ luôn đi theo một chiều từ orchestrator xuống các dịch vụ con.

Về mặt đảm bảo chất lượng, khâu **kiểm thử (testing) cũng trở nên đơn giản hơn**. Các lập trình viên có thể tạo ra các bản giả lập (mock) cho orchestrator để kiểm thử độc lập tính đúng đắn của từng dịch vụ con mà không cần phải dựng cả một hệ sinh thái cồng kềnh. Đổi lại những lợi ích quản trị, mô hình này phải chấp nhận **centralization risk**: orchestrator có thể trở thành single point of failure nếu chỉ chạy một instance và không lưu trạng thái saga bền vững.

Ngoài ra, nếu không phân chia trách nhiệm rõ ràng, hệ thống dễ rơi vào anti-pattern **smart orchestrator**: business logic bị đẩy hết vào orchestrator, biến nó thành một monolith, còn các dịch vụ con thoái hóa thành **anemic service** — chỉ còn lại thao tác đọc/ghi cơ bản, không nắm giữ nghiệp vụ nào.

6.3.3. Khi nào dùng gì?

Rocha trong [5, §4.4] đề xuất một góc nhìn đa chiều để kết hợp cả hai cơ chế, và đây được xem là cách tiếp cận thực tế nhất:

- **Saga đơn giản (ít bước, tuần tự, không phân nhánh)**: Nên ưu tiên **Choreography**. Ví dụ thực tế chính là luồng nộp bài (Submit flow) hiện tại của KBM (4 bước T1–T4, 2 dịch vụ, không phân nhánh): áp dụng choreography là vừa đủ, không gây phức tạp hóa hệ thống.
- **Saga phức tạp (nhiều hơn 4 bước hoặc có phân nhánh)**: **Orchestration** là giải pháp phù hợp/cần thiết. Đối với KBM, nếu trong tương lai luồng nộp bài mở rộng, cần tích hợp thêm các dịch vụ như kiểm tra đạo văn (plagiarism check) hoặc chấm điểm dựa trên rubric, số lượng nhánh rẽ tăng lên sẽ đòi hỏi một orchestrator để kiểm soát mạch lạc.

- **Giao tiếp giữa các Bounded Context**: Nên sử dụng **Choreography** để giữ sự độc lập và lỏng lẻo (loose coupling) giữa các ranh giới ngữ cảnh. Giao tiếp giữa Core Service và Judge Service trong KBM là minh chứng điển hình cho việc kết nối hai domain khác biệt.
- **Nhóm phát triển chưa quen với EDA (Event-Driven Architecture)**: **Orchestration** thường dễ nắm bắt và dễ gỡ lỗi (debug) hơn, phù hợp với một đội ngũ đang trong quá trình mở rộng hoặc chưa có nhiều kinh nghiệm về luồng sự kiện phi tập trung.

Điểm mấu chốt không phải chọn phe, mà là nhận ra đúng thời điểm luồng nghiệp vụ đã đủ phức tạp để cần một bộ điều phối tường minh.

Với luồng nộp bài hiện tại (4 bước, 2 dịch vụ, không phân nhánh), **choreography là đủ** — việc thêm bộ điều phối sẽ làm thiết kế phức tạp hơn mức cần thiết. Tuy nhiên, nếu tương lai luồng nghiệp vụ mở rộng (thêm plagiarism detection service, thêm grading rubric service, thêm analytics service), orchestration trở nên cần thiết. Đây là quyết định kiến trúc mở — ghi nhận để xem xét lại khi yêu cầu bài toán thay đổi.

6.4. Giao dịch bù trừ (Compensating Transaction) & Isolation

Bù trừ dữ liệu chính là các nghiệp vụ thực tế. Khi phòng đào tạo phát hiện sai sót lịch học, họ không thể quay ngược thời gian để xóa đi, mà phải phát hành thông báo đính chính và chủ động sắp xếp lại một lịch học mới cho sinh viên.

6.4.1. Thiết kế compensation trong KBM

Compensation không phải là một lệnh "undo" mù quáng ở mức cơ sở dữ liệu. Về bản chất, nó là một **hành động nghiệp vụ ngược** có ý nghĩa thực tiễn [2a, Ch.4]. Việc thiết kế các hành động này phải gắn liền với logic của từng dịch vụ. Trong ngữ cảnh KBM, cơ chế này được phân rã như sau:

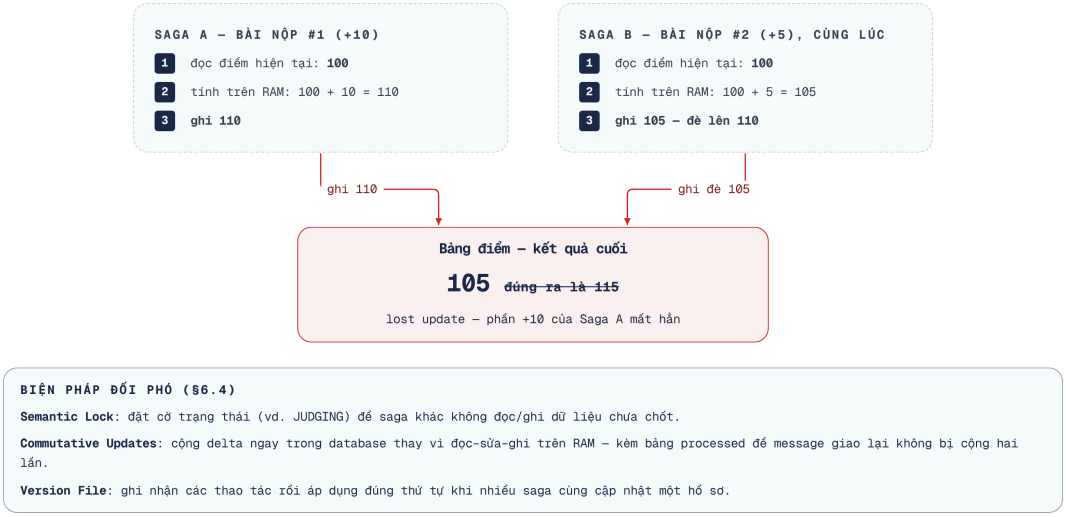
- **T1: Create Submission (Trạng thái PENDING)**: Hành động bù trừ tương ứng là C1 (Mark ERROR). Thay vì xóa bỏ (DELETE) bản ghi submission trong cơ sở dữ liệu một cách trực tiếp, hệ thống chỉ cập nhật trạng thái của nó thành ERROR. Việc này giữ lại lịch sử để hỗ trợ đối soát sau này.
- **T2: Execute SQL**: Đây là điểm Pivot nên không có hành động compensation. Nếu tiến trình gặp lỗi, Judge Service sẽ tự động dọn dẹp (cleanup) môi trường sandbox tạm thời của nó.
- **T3: Update Score và T4: Enqueue Notification Command**: Đây là các bước Retriable, đồng nghĩa với việc chúng không bao giờ kích hoạt compensation hay rollback. Cả hai đều là thao tác *nội bộ* (ghi điểm vào database, ghi lệnh thông báo vào outbox) nên luôn có thể thử lại đến khi thành công. Việc gửi email/push thực tế nằm *ngoài* saga: notification worker đọc outbox và gửi theo kiểu best-effort với retry giới hạn, backoff và Dead Letter Queue.

Nguyên tắc cốt lõi: Mỗi compensation phải được thiết kế sao cho nó **idempotent** — nghĩa là có thể được gọi hàng chục lần nhưng vẫn chỉ cho ra duy nhất một kết quả như lần gọi đầu tiên. Với KBM, `markError(submissionId)` cần được viết sao cho gọi lặp do lỗi mạng vẫn chỉ ghi ERROR một lần, không sinh side effect (tác dụng phụ) nào. Lưu ý quan trọng: sau khi vượt qua Pivot transaction (T2), tất cả Retriable transaction (T3, T4) **bắt buộc** phải được thực thi đến cùng — điều này khả thi vì chúng đều là thao tác nội bộ. Chúng không được phép rollback hay kích hoạt compensate, vì lúc này Saga đã được xem là hoàn tất ở cấp độ nghiệp vụ. Ngược lại, một bản ghi nằm trong DLQ nghĩa là việc gửi thông báo *chưa* thành công — DLQ là nơi chờ đối soát, không phải bằng chứng bước đó đã hoàn tất; đó cũng chính là lý do việc gửi thực tế được tách ra khỏi saga.

6.4.2. Vấn đề isolation — Anomalies

Do Saga là chuỗi các giao dịch cục bộ, kết quả trung gian chưa hoàn tất của một saga có thể bị đọc bởi các saga chạy đồng thời khác. Richardson [2a, Ch.4] gọi đây là hiện tượng **lack of isolation**. Sự thiếu vắng isolation gây ra ba loại bất thường (anomaly) về dữ liệu, cần xử lý bằng **các biện pháp đối phó (countermeasure)** thủ công:

1. **Lost Update (Mất dữ liệu cập nhật)** — Xảy ra khi Saga A ghi đè lên thay đổi mà Saga B vừa lưu thành công, trong khi Saga A không biết bản ghi đó đã bị thay đổi kể từ lúc đọc dữ liệu ban đầu. Đây là hệ quả khi các tiến trình song song cùng tranh giành quyền cập nhật một trạng thái dùng chung.



Hình 6.5: Lost Update anomaly — hai saga ghi đè kết quả của nhau

Ví dụ trong KBM: một sinh viên nộp hai bài liên tiếp, bài A được cộng 10 điểm và bài B được cộng 5 điểm, và hai bài được xử lý đồng thời. Cả hai luồng xử lý đều đọc bảng điểm hiện tại và cùng thấy con số gốc là 100 điểm. Luồng A cập nhật cơ sở dữ liệu thành 110, trong khi luồng B chậm hơn một chút nên đã ghi đè thành 105. Kết quả là hệ thống ghi nhận 105 điểm thay vì 115: điểm của lượt nộp A đã mất.

2. **Dirty Read (Đọc dữ liệu rác/chưa hoàn tất)** — Xảy ra khi Saga A đọc và sử dụng dữ liệu trung gian mà Saga B vừa ghi xuống nhưng chưa hoàn tất (và có thể bị rollback sau đó).

   Ví dụ về Dirty Read: Nếu saga chấm bài cộng điểm cho sinh viên ngay khi nộp bài, tiến trình đọc của Leaderboard lập tức thấy số điểm này và hiển thị. Tuy nhiên, nếu Judge Service sau đó gặp lỗi và saga kích hoạt bù trừ (trừ điểm lại), Leaderboard đã vô tình hiển thị mức điểm chưa chính thức. KBM tránh được lỗi này bằng cách đặt "Update Score" ở cuối dưới dạng giao dịch Retriable, nghĩa là điểm chỉ được cập nhật sau khi chấm xong hoàn toàn.

3. **Non-repeatable/Fuzzy Read (Đọc dữ liệu không nhất quán)** — Xuất hiện khi một tập dữ liệu thay đổi giá trị giữa hai lần đọc trong khuôn khổ của cùng một saga duy nhất.

   Trở lại với hệ thống KBM: hãy hình dung saga "chốt kết quả kỳ thi". Ở bước đầu, saga đọc bảng điểm (truy vấn đọc lần một) để xác định top 3 chuẩn bị trao giải, rồi lần lượt gọi các service khác để khóa kỳ thi và sinh chứng nhận. Đến bước cuối, khi đọc lại bảng điểm (truy vấn đọc lần hai) để ghi thứ hạng chính thức, dữ liệu đã khác đi — vài saga chấm bài nộp sát

giờ vừa hoàn tất trong lúc saga này đang chạy. Hai lần đọc *trong cùng một saga* cho hai kết quả khác nhau — đó chính là Fuzzy Read.

ACD thay vì ACID

Richardson trong [2a, Ch.4] chỉ ra: Saga chỉ đảm bảo **ACD** (Atomicity, Consistency, Durability) — *không có Isolation*. Atomicity đạt được nhờ compensating transaction (rollback logic). Consistency bảo toàn bởi quy tắc nghiệp vụ (business rule) trong mỗi local transaction. Durability do mỗi database đảm bảo. Nhưng Isolation phải được xử lý riêng — bằng **các biện pháp đối phó**.

6.4.3. Các biện pháp đối phó (countermeasure)

Richardson đề xuất các biện pháp đối phó [2a, Ch.4], áp dụng cho KBM:

Semantic Lock — Đặt cờ "đang xử lý" trên record, ngăn saga khác đọc/ghi data chưa final:

Listing 6.2: *Semantic Lock — đặt cờ "JUDGING" để chặn thao tác trên dữ liệu chưa chốt*

```
// Chặn nộp đúp ngay ở tầng DB bằng partial unique index
// (tránh lỗi TOCTOU của check-then-insert):
// CREATE UNIQUE INDEX uq_active_submission ON submissions (user_id, question_id)
//   WHERE status IN ('PENDING', 'JUDGING', 'CANCELLING');
try {
    submissionRepository.saveAndFlush(Submission.pending(userId, questionId, sql));
} catch (DataIntegrityViolationException ex) {
    // Ném runtime exception để Spring rollback transaction
    // và trả về HTTP 409 Conflict
    throw new ConflictException("SUBMISSION_IN_PROGRESS",
        "Đang có bài chờ chấm cho câu hỏi này.");
}

// Khi saga bắt đầu — dùng Atomic SQL Update để giữ khóa nghiệp vụ:
// UPDATE submissions SET status = 'JUDGING' WHERE id = ? AND status = 'PENDING'
int rows = submissionRepository.markAsJudging(submissionId);
if (rows == 0) {
    throw new ConflictException("SUBMISSION_ALREADY_LOCKED",
        "Submission đang được xử lý hoặc không còn ở trạng thái PENDING.");
}

// Khi saga hoàn thành — giải phóng khóa
// (UPDATE ... WHERE id = ? AND status = 'JUDGING')
submissionRepository.markAsJudged(submissionId);
```

Lưu ý việc chặn nộp đúp không dùng `existsBy...` ở tầng ứng dụng rồi mới `INSERT`, vì hai yêu cầu đến đồng thời vẫn lọt qua khe hở kiểm tra–ghi (TOCTOU). Ràng buộc `UNIQUE` có điều kiện (`WHERE status IN ('PENDING', 'JUDGING', 'CANCELLING')`) ép cơ sở dữ liệu phân xử nguyên tử và trả về mã `409 Conflict` thay vì `400 Bad Request` (dữ liệu đầu vào hợp lệ nhưng xung đột với trạng thái hiện hành của tài nguyên, xem §3.1).

Commutative Updates — Thiết kế thao tác cập nhật không phụ thuộc thứ tự — giải quyết **lost update**:

Listing 6.3: *Commutative Updates — cập nhật theo lượng thay đổi (delta) thay vì giá trị tuyệt đối*

```
// SAI: Không commutative: set absolute value — thứ tự quan trọng
score.setValue(150);

// VẪN SAI: delta update nhưng là read-modify-write trên RAM,
// khi flush vẫn sinh "SET value = <giá trị tuyệt đối>" → Lost Update y nguyên.
// Ngoài ra không idempotent: event gửi lại (retry) sẽ cộng 20, 30 điểm.
score.incrementValue(+10);

// ĐÚNG: Commutative + Idempotent (trong cùng @Transactional):
// 1) INSERT INTO processed_events(event_id, processed_at)
//    VALUES (:eventId, now()) ON CONFLICT (event_id) DO NOTHING
if (processedEventRepository.tryMarkProcessed(eventId) == 0) {
    return; // Đã xử lý event này, thoát an toàn mà không làm hỏng transaction
}
// 2) UPDATE scores SET value = value + :delta WHERE user_id = :userId
scoreRepository.incrementScoreAtomically(userId, 10);
```

Chiến thuật Commutative Updates (Cập nhật giao hoán) là một giải pháp hiệu quả cho vấn đề Lost Update. Thay vì ghi đè một giá trị tuyệt đối (ví dụ: gán điểm số = 150), hệ thống áp dụng một phép toán thay đổi tương đối (delta, ví dụ: +10 điểm). Nhờ tính chất giao hoán của phép cộng, dù các sự kiện cập nhật điểm đến không theo thứ tự, kết quả cuối cùng vẫn chính xác. Cơ chế này rất phổ biến trong các hệ thống tính điểm, kế toán hoặc ví điện tử. Tuy nhiên, việc áp dụng cập nhật tương đối khiến hệ thống mất đi idempotency tự nhiên. Nếu một sự kiện (+10 điểm) bị broker phát lại hai lần, dữ liệu sẽ bị cộng dồn sai lệch.

⚠ Lỗ hổng Idempotency với Commutative Updates

Tuy phép cộng có tính giao hoán, nhưng nó *không* idempotent. Vì bước Update Score là một *Retriable transaction* (được retry liên tục nếu lỗi mạng), nếu thao tác cộng điểm nguyên tử (`+10`) bị gọi lại 3 lần mà không kiểm tra trùng, sinh viên sẽ bị cộng lặp thành 30 điểm (Double-counting)! Bắt buộc phải kết hợp với khóa chống trùng lặp (ví dụ: ghi `eventId` vào bảng `processed_events` bằng `INSERT ... ON CONFLICT DO NOTHING` trong cùng transaction hoặc dùng Inbox pattern) để đảm bảo không bị cộng dồn sai lệch.

Ba biện pháp còn lại nhắm vào dirty read, non-repeatable read và những lost update còn sót:

Pessimistic View (Góc nhìn bi quan) — Sắp xếp lại thứ tự các bước trong saga để giảm thiểu nguy cơ dirty read. Các bước không thể đảo ngược hoặc cập nhật tích lũy (như update score) sẽ được đặt *sau* pivot transaction. KBM tuân thủ nguyên tắc này khi chỉ cộng điểm sinh viên sau khi Judge ra phán quyết cuối cùng.

Re-read Value (Đọc lại giá trị) — Đây là hình thức áp dụng optimistic locking: thay vì dùng dữ liệu đã truy xuất từ trước, saga thực hiện đọc lại dữ liệu (re-read) ngay trước khi ghi đè để kiểm tra xem có thay đổi nào xảy ra không.

Listing 6.4: *Re-read Value — kiểm tra lại trước khi quyết định*

```
// Trước khi update score, không nên sử dụng câu lệnh `if` để kiểm tra
// trên bộ nhớ ứng dụng vì dễ mắc lỗi TOCTOU
// (Time-of-Check to Time-of-Use). Phải đẩy kiểm tra xuống tầng DB:
// UPDATE scores SET value = value + 10
// WHERE user_id = ? AND NOT EXISTS (
```

```
//   SELECT 1 FROM submissions WHERE id = ? AND status IN ('CANCELLING', 'TIMEOUT',
'ERROR')
// )
// Lưu ý: subquery chéo bảng này chạy được vì scores và submissions
// hiện còn chung database (hiện trạng KBM). Sau khi tách database,
// Score Service phải kiểm tra trạng thái qua API/event thay vì SQL.
scoreRepository.incrementIfSubmissionNotCancelled(
    userId, 10, submissionId
);
```

Đẩy khối kiểm tra điều kiện (condition check) trực tiếp xuống lớp cơ sở dữ liệu thay vì xử lý trong bộ nhớ của ứng dụng giúp thu hẹp khe hở Time-of-Check to Time-of-Use (TOCTOU), nhưng chưa đóng hẳn: ở mức READ COMMITTED, subquery không khóa dòng `submissions`. Muốn đóng hẳn, khóa dòng đó bằng `SELECT status FROM submissions WHERE id = ? FOR SHARE` trong cùng transaction rồi mới cập nhật, hoặc chạy ở mức SERIALIZABLE.

Version File (Tệp phiên bản) — Để quản lý chặt chẽ những tình huống mà nhiều luồng saga tranh chấp nhau cập nhật cùng một hồ sơ, kỹ thuật cuối cùng được nhắc tới là lưu trữ lại một cuốn sổ cái ghi nhận thứ tự xuất hiện của mọi tác vụ. Từ đó cho phép hệ thống sắp xếp lại (reorder) các thao tác này theo một trật tự mong muốn trước khi áp dụng vào dữ liệu thực.

Giả sử Saga A và Saga B cùng cập nhật điểm số, Version File cho phép hệ thống ghi nhận rồi sắp xếp lại các thao tác về đúng trật tự trước khi áp dụng. Với KBM, hướng đơn giản hơn là *tuần tự hóa* thông điệp điểm số trên cùng một partition (key theo sinh viên) ngay từ đầu vào, khiến xung đột ghi đè đồng thời không còn cơ hội phát sinh — tức là *phòng ngừa* out-of-order từ gốc, thay vì lưu lại rồi reorder như Version File nguyên bản. Hiện `judge-results` dùng key `submissionId` nên KBM chưa có đảm bảo này.

6.4.4. Tổng hợp: anomaly và biện pháp đối phó

Để giải quyết ba anomaly trên, KBM cần các biện pháp đối phó tương ứng (phần lớn nằm trong lộ trình ở §6.6). Với tình trạng **Lost update**, thay vì ghi đè giá trị tuyệt đối, hệ thống cần chuyển sang **Commutative updates** — delta được áp *nguyên tử ngay ở tầng database* (`UPDATE scores SET value = value + ?`), không phải tăng giá trị trên object trong bộ nhớ — để thứ tự ghi nhận điểm số không làm sai lệch kết quả cuối; khi các thao tác không có tính giao hoán, có thể kết hợp **Version file** (hoặc tuần tự hóa thông điệp điểm số theo `userId` trên cùng một Kafka partition) để chống ghi đè lệch thứ tự. Nhằm chặn hiện tượng **Dirty read**, KBM cần **Semantic lock** (bật cờ trạng thái `JUDGING`) kết hợp **Pessimistic view** (chỉ cập nhật điểm sau bước pivot) để ngăn tiến trình khác đọc điểm số chưa chốt. Cuối cùng, trước cả **Non-repeatable read** lẫn xung đột ghi đồng thời, **Re-read value** đóng vai trò chốt chặn kiểm tra lại ngay trong câu lệnh ghi nhằm phát hiện trạng thái đã đổi giữa chừng (chẳng hạn bài nộp vừa chuyển sang `CANCELLING` hoặc `TIMEOUT` khi quét quá hạn, hoặc bảng điểm vừa cập nhật trước khi chốt kết quả thi).

Các kỹ thuật trên bù đắp cho sự thiếu hụt của tính cô lập (Isolation) trong hệ thống phân tán. Khi gánh nặng duy trì tính toàn vẹn chuyển từ cơ sở dữ liệu sang tầng ứng dụng, thiết kế phải bám sát quy trình nghiệp vụ thực tế (business process); các hành động bù trừ (compensation) vì thế cần phản ánh đúng cách nghiệp vụ xử lý lỗi, thay vì chỉ rollback dữ liệu cơ học.

Compensating Transaction là hành động nghiệp vụ

Compensation trong Saga không phải là `DELETE` hay `ROLLBACK` — đó là **hành động nghiệp vụ có ý nghĩa** phản ánh quy trình thực tế: "đánh dấu đã hủy", "hoàn tiền", "gửi thông báo lỗi tới user". Điều quan trọng: mỗi compensating transaction phải được **thiết kế cùng lúc** với forward transaction, không phải thêm vào sau. Và tất cả compensation phải idempotent — gọi nhiều lần cho cùng kết quả.

6.5. Eventual Consistency — chấp nhận và quản lý

6.5.1. Định lý CAP và mô hình BASE

Quyết định từ bỏ 2PC để chuyển sang Saga thực chất là một sự đánh đổi có chủ đích liên quan đến định lý CAP. CAP buộc hệ thống phân tán phải chọn giữa Consistency (Nhất quán) và Availability (Sẵn sàng) *khi xảy ra network partition*. Luồng nộp bài của KBM ưu tiên phía **Availability** — vẫn tiếp nhận bài nộp khi một dịch vụ phía sau gián đoạn — và chấp nhận nới lỏng **C** (nhất quán tức thời). Bản thân Saga không phải là "lựa chọn AP": nó là kỹ thuật điều phối giúp hệ thống hội tụ về trạng thái đúng (eventual consistency) sau khi đã chấp nhận đánh đổi đó; từng service vẫn có thể từ chối ghi khi cần bảo toàn bất biến nghiệp vụ (phân tích kỹ hơn ở Chương 7).

Trong thiết kế từ bỏ 2PC như của KBM, các giao dịch *xuyên dịch vụ* không còn giữ được trọn vẹn bảo đảm ACID mà dịch chuyển sang mô hình **BASE** [7, Ch.7] — lưu ý quan trọng: mỗi local transaction bên trong từng service *vẫn là ACID*. Hệ thống BASE hy sinh tính cô lập (Isolation) toàn cục để đổi lấy tính sẵn sàng (Availability) và tính nhất quán cuối cùng (Eventual Consistency), trong khi Saga tạo ra tính nguyên tử ở cấp nghiệp vụ: hoặc toàn bộ chuỗi hoàn tất, hoặc các bước đã thực hiện được bù trừ. Đây không phải rollback theo nghĩa ACID, vì bên khác có thể đã kịp quan sát trạng thái trung gian. Sự chuyển dịch này thể hiện rõ ràng qua từng đặc tính trong thiết kế của KBM:

- **Basically Available (Sẵn sàng cơ bản)**: Hệ thống ưu tiên sự khả dụng cao: luôn trong trạng thái sẵn sàng tiếp nhận các bài nộp mới từ sinh viên, bất chấp việc một vài dịch vụ phía sau có thể đang bị gián đoạn.
- **Soft state (Trạng thái mềm)**: Dữ liệu điểm số có thể tạm thời chưa được cập nhật đầy đủ và vẫn đang thay đổi trong hệ thống, hệ thống không khóa cứng các bản ghi.
- **Eventual consistency (Nhất quán cuối cùng)**: Chấp nhận việc bảng xếp hạng (leaderboard) có độ trễ tạm thời, nhưng cam kết rằng *cuối cùng* mọi dữ liệu sẽ đồng bộ và chính xác.

Lưu ý về **tính bền vững (Durability — chữ D của ACID)**: việc BASE không có chữ `D` không có nghĩa là hệ thống từ bỏ độ bền dữ liệu. Cả hai mô hình đều giữ đặc tính này ở mức local commit — dữ liệu một khi đã commit xuống cơ sở dữ liệu cục bộ vẫn sống sót qua sự cố theo đúng cam kết của tầng lưu trữ; còn retention, replication và backup là những thiết kế vận hành riêng, không tự nhiên mà có.

6.5.2. Consistency window trong KBM

Khi KBM chọn Saga thay cho giao dịch phân tán, hệ thống phải chấp nhận một hệ quả trực tiếp: dữ liệu giữa các dịch vụ có độ trễ. Trong một giao dịch nguyên khối, mọi bảng dữ liệu được đồng bộ tại đúng thời điểm commit; với saga, giữa các local transaction luôn tồn tại khoảng quá độ mà thông tin ở các dịch vụ chưa khớp nhau. Nhiệm vụ của thiết kế không phải là xóa bỏ khoảng trễ đó, mà là kiểm soát nó ở mức người dùng không bị ảnh hưởng.

Consistency Window

Rocha trong [5, Ch.5] định nghĩa **consistency window** (cửa sổ nhất quán): khoảng thời gian từ lúc một sự kiện xảy ra cho đến khi tất cả các dịch vụ liên quan phản ánh trạng thái mới đó. Mục tiêu của người thiết kế không phải là triệt tiêu khoảng thời gian này, mà là giữ độ rộng của consistency window ở mức **đủ nhỏ** để người dùng cuối không nhận ra sự chậm trễ.

Trong quy trình của KBM: sau khi sinh viên nhấn nút nộp bài, consistency window mở ra và kéo dài cho tới khi có kết quả. Độ rộng của cửa sổ này phụ thuộc vào thời gian thực thi mã nguồn và tải của Judge Service. Trong khoảng thời gian chờ đó, thiết kế đề xuất cho KBM dùng các cơ chế sau (semantic lock thuộc Giai đoạn 1 ở §6.6):

- `submission.status = JUDGING` (semantic lock)
- Bảng xếp hạng (Leaderboard) tạm thời hiển thị điểm số cũ chưa được cập nhật
- Giao diện người dùng (UI) hiển thị trạng thái "Đang chấm..." nhằm cung cấp phản hồi trực quan để người dùng an tâm chờ đợi

6.5.3. Chiến thuật cho giao diện và nghiệp vụ

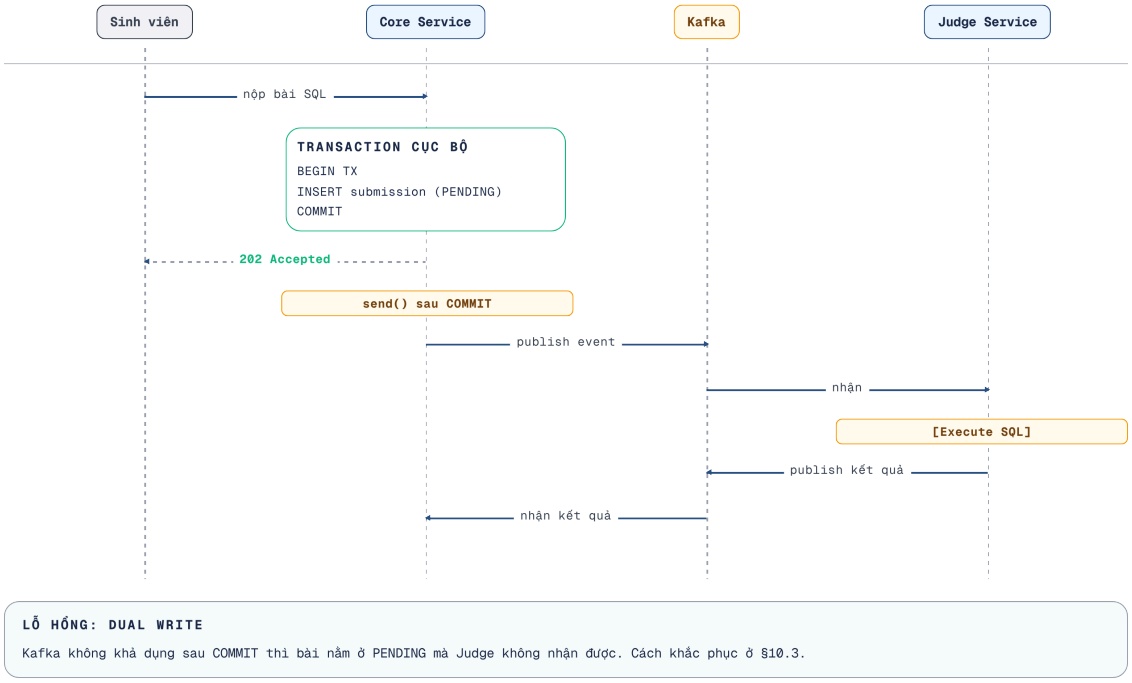
Để lấp khoảng trống của "consistency window", frontend thường kết hợp bốn chiến thuật UI. **Optimistic UI** hiển thị ngay "Bài đã gửi thành công" sau thao tác, dù server phía sau vẫn đang chấm. **progress indicator** cập nhật các trạng thái trung gian ("Đang chấm...", "Đang so sánh...") để người dùng biết hệ thống không bị treo. Khi xử lý hoàn tất, **push notification** (qua WebSocket) truyền kết quả cuối cùng lên màn hình. Nếu Judge Service gặp lỗi và saga phải bù trừ, **compensation notification** hiển thị thông báo "Bài không chấm được, vui lòng thử lại" — đóng vòng phản hồi một cách an toàn. Về phía nghiệp vụ, cần chốt với người dùng khoảng chờ chấp nhận được (ví dụ kết quả chấm trong 45 giây ở kịch bản chất lượng §1.5) và cách xử lý khi quá hạn: chấm lại tự động hay yêu cầu nộp lại.

6.6. Case study: KBM

Quá trình sinh viên nộp bài thực chất là một chuỗi hành động trải dài trên nhiều dịch vụ. Việc phân tích luồng xử lý này giúp nhận ra các chỗ thiếu trong thiết kế hiện tại.

6.6.1. Hiện trạng: Implicit Saga (Choreography)

KBM hiện có một implicit saga — choreography qua sự kiện Kafka, nhưng *không* được định nghĩa rõ ràng như saga:



Hình 6.6: Implicit Saga trong KBM — choreography qua sự kiện Kafka

Richardson trong [2a, Ch.4] mô tả hai cách điều phối saga đều hợp lệ: **choreography** (các service tự phản ứng theo event) và **orchestration** (bộ điều phối tường minh quản lý state machine kèm compensation). Dù chọn cách nào, một saga đúng nghĩa vẫn cần trạng thái, timeout và compensation *tường minh*. Khi xem xét luồng nộp bài (Submit flow) hiện tại của KBM, thiết kế choreography ngầm định đang thiếu chính các cơ chế an toàn thiết yếu đó (safety mechanism). Việc thiếu saga tường minh khiến chuỗi tiến trình khó kiểm soát. Thiếu timeout và compensation, một lỗi mạng hay Judge dừng đột ngột có thể để bài nộp treo ở `PENDING` mà không ai xử lý — đúng như rủi ro kiến trúc phân tích dưới đây.

Saga không có timeout tại KBM

Trong thiết kế Choreography hiện tại của KBM cho luồng chấm bài, `kbm-core` lưu trạng thái `PENDING` rồi đẩy thông điệp nộp bài vào topic `submissions`. Tuy nhiên, nếu tiến trình gửi thông điệp gặp sự cố hoặc `kbm-judge-sql` dừng đột ngột (crash), sinh viên sẽ thấy bài nộp rơi vào trạng thái chờ đợi (loading) vô thời hạn do không có dịch vụ nào phản hồi và cập nhật trạng thái.

Cách khắc phục được đề xuất là mẫu **Timeout & Compensation** (Giai đoạn 1 ở dưới). Tuy nhiên, nếu chỉ sử dụng một tác vụ theo lịch trình (`@Scheduled` job) để chuyển đổi trạng thái thành `TIMEOUT` một cách máy móc khi quá hạn 5 phút, hệ thống sẽ gặp race condition: Judge Service có thể hoàn thành việc chấm đúng 1 giây sau đó và ghi đè kết quả. Để giải quyết, Core Service (khi đóng vai điều phối cho luồng quá hạn) phải gửi một lệnh "Cancellation/Probe Command" tới Judge Service. Chỉ khi nhận được phản hồi xác nhận hủy (hoặc xác nhận chưa từng chạy) từ Judge, Core Service mới an toàn đổi trạng thái thành `TIMEOUT` và thực hiện các bước Compensation (đánh dấu bài nộp để sinh viên nộp lại). Đây là minh chứng rõ nét: trong hệ thống phân tán, *luôn phải thiết kế cho trường hợp thành phần khác không bao giờ trả lời*.

Vùng xám giữa Choreography và Orchestration. Việc Core Service chủ động gửi "Cancellation Command" (lệnh) tới Judge và lắng nghe "ACK" đã khiến nó hành xử giống như một **Lightweight Orchestrator** thay vì Choreography thuần túy (vốn chỉ phát event trung lập). Sự chuyển dịch từ Event-driven sang Command-driven này là rất phổ biến để giải quyết các luồng Timeout hoặc Compensation phức tạp.

6.6.2. Phân tích các vấn đề và lộ trình cải tiến

Đối chiếu với các tiêu chuẩn thiết kế phân tán (Giải pháp khuyến nghị theo Richardson [2a, Ch.4]), luồng Submit hiện tại của KBM bộc lộ bốn lỗ hổng kiến trúc:

- **Implicit saga (Saga ngầm định)**: Luồng xử lý nằm rải rác trong mã nguồn, không có định nghĩa state machine tập trung — lập trình viên khó bao quát toàn bộ quy trình.
- **Thiếu compensation và timeout**: Nếu Judge Service treo hoặc không phản hồi do nghẽn mạng, trạng thái PENDING của bài nộp bị đình trệ vô thời hạn.
- **Không có semantic lock**: Sinh viên có thể nộp bài mới khi hệ thống vẫn đang xử lý bài cũ, vô tình hoặc cố ý.
- **Thiếu cơ chế theo dõi Saga**: Hệ thống không theo dõi được tiến độ hiện tại của chuỗi (PENDING → JUDGING → CANCELLING → JUDGED / ERROR / TIMEOUT).

Để khắc phục, lộ trình nâng cấp kiến trúc được chia làm ba giai đoạn. **Giai đoạn 1 — Semantic Lock + Timeout** mang tính cấp bách nhất, tập trung vào việc áp dụng khóa nghiệp vụ và cơ chế bù trừ khi quá hạn nhằm tránh race condition giữa timeout và kết quả chấm.

Listing 6.5: Semantic Lock + Probe/Timeout compensation để tránh race condition

```
// Scheduled job kiểm tra timeout
// (2 cấp: Probe mềm sau 5 phút → Hard Timeout sau 10 phút)
@Scheduled(fixedRate = 60000)
public void checkSubmissionTimeouts() {
    // Cấp 1 (quá 5 phút ở PENDING hoặc JUDGING):
    // chuyển sang CANCELLING rồi gửi lệnh hủy/thăm dò
    List<Submission> stuck =
            submissionRepository.findStuckInPendingOrJudging(cutoff5m);
    for (Submission s : stuck) {
        // UPDATE submissions SET status = 'CANCELLING'
        // WHERE id = :id AND status IN ('PENDING', 'JUDGING')
        if (submissionRepository.markAsCancelling(s.getId()) > 0) {
            // Thực tế nên ghi qua Outbox (§10.3)
            judgeCommandProducer.sendCancelCommand(s.getId(), s.getJudgeRunId());
        }
    }

    // Cấp 2 — Hard Timeout
    // (quá 10 phút vẫn kẹt ở CANCELLING vì Judge chết, không có ACK):
    // dùng chung luồng bù trừ để sinh viên vẫn được thông báo
    // và thử lại / đưa vào DLQ
    List<String> deadIds = submissionRepository.findStuckInCancelling(cutoff10m);
    deadIds.forEach(compensator::compensateTimedOutSubmission);
}

// Listener nhận ACK hủy từ kbm-judge-sql
// (trường hợp Judge còn sống và xác nhận đã dừng)
@KafkaListener(topics = "judge-cancel-ack")
public void handleJudgeCancelAck(String submissionId) {
```

```
    compensator.compensateTimedOutSubmission(submissionId);
}

@Transactional
public void compensateTimedOutSubmission(String submissionId) {
    // Nằm trong bean riêng (SubmissionCompensator)
    // được inject vào job và listener ở trên:
    // gọi qua proxy của Spring thì @Transactional mới có hiệu lực;
    // gọi nội bộ (this::...) sẽ bỏ qua giao dịch.
    // Atomic transition: UPDATE submissions SET status = 'TIMEOUT'
    // WHERE id = :id AND status = 'CANCELLING'
    // (chống chạy đúp giữa ACK đến muộn và Hard Timeout)
    if (submissionRepository.markAsTimeoutFromCancelling(submissionId) == 0) {
        return;
    }
    Submission s = submissionRepository.findById(submissionId).orElseThrow();
    // Ba tác dụng phụ ghi vào outbox trong cùng transaction (§10.3):
    // transaction rollback thì không thông báo hay message nào lọt ra ngoài
    if (s.getRetryCount() < MAX_RETRIES) {
        // Lưu bản thử lại (status = PENDING, retryCount + 1, judgeRunId mới)
        Submission retry = submissionRepository.save(s.cloneForRetry());
        outbox.enqueueNotification(s.getUserId(),
                "Judge quá hạn — hệ thống đang chấm lại tự động");
        outbox.enqueueSubmission(retry);
    } else {
        outbox.enqueueNotification(s.getUserId(),
                "Không thể chấm bài lúc này — vui lòng nộp lại sau");
        outbox.enqueueDeadLetter(s); // Vượt ngưỡng retry → DLQ để đối soát
    }
}
```

Fencing cho kết quả đến muộn

Hard timeout là biện pháp cuối cùng khi Judge không phản hồi ACK, nhưng nó mở ra một rủi ro đối xứng: kết quả *hợp lệ* vẫn có thể đến sau khi hệ thống đã ép TIMEOUT (Judge chỉ nghẽn mạng chứ không chết), hoặc kết quả của lần chạy cũ đến sau khi đã có bản retry mới.

Vì vậy mọi handler ghi trạng thái — kể cả handler nhận kết quả chấm — đều phải là transition có điều kiện trên state machine (`UPDATE ... WHERE status = ...`). Mỗi lần chấm cũng nên mang một `judgeRunId` riêng: chỉ kết quả khớp lần chạy đang hiệu lực mới được chấp nhận, còn kết quả "mồ côi" bị bỏ qua và ghi log để đối soát.

Giai đoạn 2 — Định nghĩa Saga tường minh (đòi hỏi mức độ nỗ lực trung bình):

- Định nghĩa `SubmitSaga` class với state machine: PENDING → JUDGING → CANCELLING → JUDGED / ERROR / TIMEOUT
- Mỗi transition có điều kiện và xử lý lỗi rõ ràng; bước compensatable có compensation, còn các bước sau pivot có cơ chế retry
- Ghi log mọi chuyển đổi trạng thái saga để kiểm toán (audit) và gỡ lỗi

Giai đoạn 3 — Saga Orchestrator (đòi hỏi nhiều nguồn lực, khi cần mở rộng quy mô):

- Cân nhắc khi luồng nộp bài mở rộng (thêm plagiarism check, grading rubric)
- Hiện tại choreography vẫn đủ — luồng xử lý 4 bước, không phân nhánh
- Ngưỡng: khi luồng nghiệp vụ > 4 bước hoặc có phân nhánh phức tạp → chuyển sang orchestration

6.6.3. Workflow học vụ cũng là saga

KBM không chỉ có luồng nộp bài. Các luồng như duyệt bài tập, phê duyệt đề tài/đồ án, công bố điểm hoặc cập nhật CLO cũng có bản chất saga: nhiều bước, nhiều người tham gia, có trạng thái trung gian và có hành động bù trừ khi bị từ chối hoặc quá hạn.

Nhiều quy trình học vụ ánh xạ được vào ba loại bước: Compensatable, Pivot và Retriable:

- **Quy trình phê duyệt bài tập (Assignment approval)**: Giai đoạn đầu tiên (Compensatable) là khi giảng viên tạo bản nháp (Draft assignment) — lúc này mọi thông tin vẫn có thể hủy hoặc sửa dễ dàng. Điểm Pivot chính là khoảnh khắc giảng viên hoặc ban quản lý quyết định bấm nút phê duyệt (Approve). Một khi đã phê duyệt, hệ thống tiến vào chuỗi Retriable bắt buộc phải thành công: xuất bản bài tập ra khóa học và tự động gửi thông báo đến toàn bộ sinh viên.
- **Quy trình xét duyệt đồ án/khóa luận (Thesis/project review)**: Sinh viên khởi tạo đề xuất đề tài (Submit proposal), một hành động Compensatable vì hoàn toàn có thể rút lại. Điểm Pivot rơi vào quyết định chính thức của hội đồng chuyên môn (Committee decision). Sau quyết định này, hệ thống sẽ thực thi các tác vụ Retriable như cập nhật trạng thái phê duyệt cuối cùng và thông báo kết quả cho cả sinh viên lẫn giảng viên hướng dẫn.
- **Quy trình công bố điểm (Grade publication)**: Hệ thống bắt đầu bằng việc tính toán điểm số dự kiến (Compute provisional grade) — một bước Compensatable vì có thể bị tính lại nhiều lần. Pivot transaction xuất hiện khi giảng viên xác nhận điểm lần cuối (Lecturer confirms). Ngay sau đó, quá trình Retriable sẽ phụ trách việc công bố bảng điểm chính thức và đồng bộ dữ liệu vào mô hình đọc (read model) để phục vụ cho công tác báo cáo thống kê.

Ở cả ba workflow, bước "thông báo" nằm trong chuỗi Retriable phải được hiểu theo đúng ranh giới đã phân tích ở mục T4: thứ bắt buộc thành công là việc *ghi bền vững lệnh thông báo* còn việc gửi email/push đến người nhận chạy ngoài saga theo kiểu best-effort.

Các workflow này thường nên bắt đầu bằng **explicit state machine** hơn là orchestrator riêng. Khi số bước và nhánh tăng, orchestration mới đáng cân nhắc.

⚠ Sai lầm thường gặp khi triển khai Saga

- **Ép nhất quán mạnh bằng 2PC**: Thói quen ACID từ kiến trúc nguyên khối khiến nhiều kỹ sư cố giữ strong consistency xuyên các dịch vụ, đổi lại là khóa tài nguyên chặn luồng và giảm tính sẵn sàng. *Phòng tránh*: Chấp nhận eventual consistency ở nơi nghiệp vụ cho phép và dùng saga.
- **Chỉ thiết kế cho happy path**: Quên hành động bù trừ (compensation) cho các bước có thể hoàn tác, dẫn đến hệ thống kẹt ở trạng thái trung gian với dữ liệu mâu thuẫn. *Phòng tránh*: Mỗi bước compensatable đều phải có compensation tường minh và đã được kiểm thử.
- **Không kiểm soát timeout**: Saga bắt đầu nhưng không ai theo dõi thời gian thực thi sẽ treo vĩnh viễn khi mạng chập chờn hoặc dịch vụ dừng đột ngột. *Phòng tránh*: Dùng tác vụ chạy ngầm quét saga quá hạn và tự kích hoạt bù trừ.
- **Bỏ qua semantic lock**: Không khóa dữ liệu đang xử lý cho phép người dùng thao tác đúp (sinh viên nộp lại bài khi hệ thống đang chấm), sinh ra race condition và dữ liệu rác. *Phòng tránh*: Áp dụng semantic lock ngay từ bước đầu của saga (§6.4.3).
- **Compensation không idempotent**: Khi bước bù trừ được thực thi lại, hệ thống bù trừ hai lần và làm sai lệch dữ liệu. *Phòng tránh*: Thiết kế mọi hành động bù trừ bảo đảm idempotency (§6.4.1).

- **Lạm dụng choreography cho saga nhiều nhánh**: Tạo ra ma trận sự kiện khó theo dõi và dễ phụ thuộc vòng. *Phòng tránh*: Khi quy trình vượt quá bốn bước hoặc có phân nhánh phức tạp, chuyển sang orchestration (§6.3.3).

Các pattern trong chương có minh họa tương tác đi kèm tại companion web:

Interactive Demo

- **Saga Pattern (Orchestration)**: saga-orchestration.html

6.7. Tổng kết

Giao dịch phân tán (distributed transaction) là một trong những bài toán khó nhất khi chuyển từ monolith sang microservices. Two-Phase Commit — giải pháp truyền thống — không phù hợp vì blocking, single point of failure, và giảm availability.

Saga pattern giải quyết bằng cách thay một ACID transaction bằng chuỗi local transaction + compensating transaction. Ba loại transaction (compensatable, pivot, retriable) tạo cấu trúc rõ ràng cho mỗi saga. Choreography (phi tập trung) phù hợp cho các saga đơn giản, orchestration (tập trung) cho các saga phức tạp.

Thiếu isolation là thách thức lớn nhất. Semantic lock, commutative updates, pessimistic view, re-read value và version file là năm biện pháp đối phó giúp giảm thiểu các anomaly — tất cả đều áp dụng được cho KBM.

Eventual consistency là trade-off có chủ đích cho tính sẵn sàng và khả năng mở rộng. Trong KBM, consistency window ngắn là chấp nhận được — sinh viên quen với việc chờ kết quả chấm bài. Quản lý consistency window bằng semantic lock và real-time notification là đủ cho bài toán hiện tại.

Phân tích KBM cho thấy hệ thống đang dùng implicit saga (choreography) và cần làm rõ compensation, semantic lock, timeout handling. Bài nộp hoặc các luồng quy trình học vụ có thể bị đình trệ ở trạng thái trung gian — đây là nợ kỹ thuật nghiêm trọng cần xử lý ưu tiên (Giai đoạn 1).

Ở Chương 7, chúng ta sẽ giải quyết bài toán **quản lý dữ liệu** — database-per-service, CQRS, Event Sourcing — và phân tích sâu hơn vấn đề cơ sở dữ liệu dùng chung trong KBM.

6.8. Đọc thêm

Sách tham khảo chính:

1. [2a] Chris Richardson, *Microservices Patterns*, 1st Ed. — Ch.4: Managing Transactions with Sagas
2. [5] Hugo Rocha, *Practical Event-Driven Microservices Architecture* — Ch.4: Sagas (Choreography, Orchestration); Ch.5: Eventual Consistency
3. [7] Martin Kleppmann, *Designing Data-Intensive Applications* — Ch.7: Transactions; Ch.9: Consistency and Consensus

Nguồn trực tuyến:

- Gregor Hohpe, "Starbucks Does Not Use Two-Phase Commit" (2004) — enterpriseintegrationpatterns.com
- Caitie McCaffrey, "Applying the Saga Pattern" (GOTO Chicago 2015) — youtube.com

- Eventuate Tram Sagas framework — eventuate.io

Bài tập: 6-0, 6-1, 6-2 ở phần Bài tập & case study cuối sách.

CHƯƠNG 7

Quản lý dữ liệu — Database per Service & CQRS

> "By hiding the database that backs our service, we also ensure we reduce coupling."
> — Sam Newman, *Monolith to Microservices* [4b, Ch.1]

MỤC TIÊU HỌC TẬP

- Hiểu nguyên tắc **database-per-service** và tại sao quyền sở hữu dữ liệu là nền tảng của microservices
- Nắm được các chiến lược tách database từ monolith — từ shared database đến database-per-service
- Phân biệt khi nào data duplication là chấp nhận được và khi nào nó là vấn đề
- Hiểu CQRS (Command Query Responsibility Segregation) và khi nào nó cần thiết
- Nắm tổng quan Event Sourcing — ưu/nhược điểm và khi nào nên áp dụng
- Hiểu đúng CAP Theorem và chọn nhất quán hay sẵn sàng theo từng luồng dữ liệu
- Phân tích bài toán truy vấn xuyên service (cross-service query) trong KBM

Bản đồ khái niệm: Chương 5 → Chương 6 → Chương 7

Ba chương cuối Phần II nối với nhau theo một mạch: **Chương 5** trình bày cách các service giao tiếp bất đồng bộ qua message/event; **Chương 6** dùng chính các message đó để giữ tính nhất quán khi một nghiệp vụ trải trên nhiều service (Saga); và **Chương 7** (chương này) trả lời câu hỏi sâu hơn: nếu mỗi service *sở hữu database riêng*, thì lưu trữ, truy vấn và đồng bộ dữ liệu giữa chúng ra sao? Nói gọn: Chương 5 = "gửi tin", Chương 6 = "phối hợp giao dịch", Chương 7 = "quản lý dữ liệu của từng service". Nếu khối lượng kiến thức quá lớn, độc giả chỉ cần ghi nhớ một nguyên lý cốt lõi cho chương này: **mỗi service quản lý dữ liệu độc lập, và chỉ chia sẻ qua API hoặc event — không bao giờ chung bảng.**

7.1. Database-per-Service — nguyên tắc Data Ownership

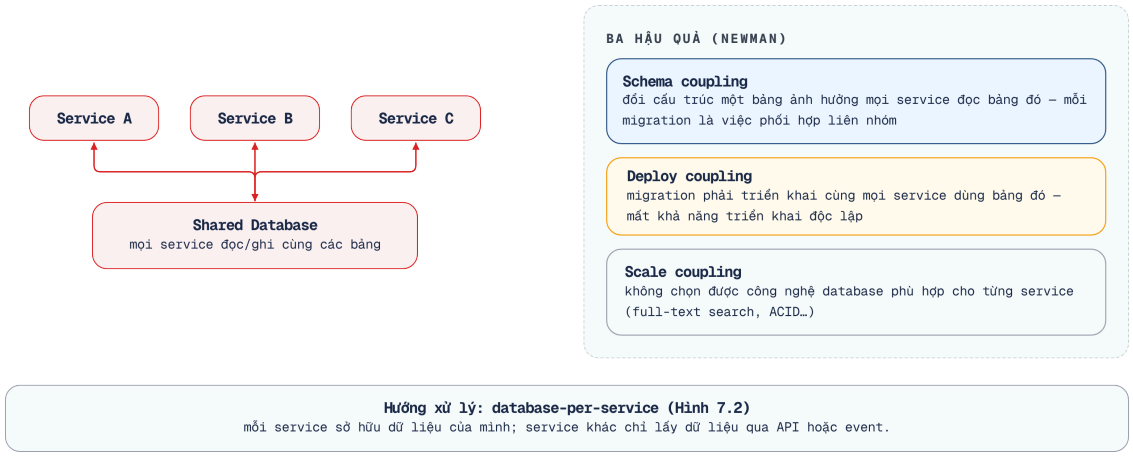
Để đạt được tính tự chủ, mỗi dịch vụ cần quản lý một không gian lưu trữ độc lập, tương tự như việc các khoa trong trường đại học tự quản lý hồ sơ sinh viên của khoa đó.

7.1.1. Vấn đề: shared database phá vỡ tính độc lập của dịch vụ

Trong monolith, toàn bộ ứng dụng truy cập một database duy nhất — đơn giản, nhất quán, và transaction ACID hoạt động tự nhiên. Khi chuyển sang kiến trúc microservices, nhiều nhóm phát triển vẫn giữ nguyên cơ sở dữ liệu dùng chung vì sự tiện lợi — mọi service vẫn đọc/ghi cùng bảng. Thoạt nhìn, đây có vẻ là giải pháp tốt: không cần thay đổi tầng dữ liệu, không cần xử lý eventual consistency.

Tuy nhiên, bài toán dữ liệu trong hệ thống phân tán bị ràng buộc bởi một giới hạn lý thuyết quan trọng — **CAP Theorem** (Consistency, Availability, Partition tolerance): trong hệ thống phân tán, khi network partition (phân vùng mạng) xảy ra, hệ thống chỉ có thể chọn *hoặc* consistency *hoặc* availability, không thể có cả hai [7, Ch.9]. Newman trong [4a, Ch.11] giải thích: đây là lý do microservices *thường* chấp nhận eventual consistency cho các bản sao dữ liệu xuyên service. Tuy vậy, mỗi thao tác vẫn phải tự chọn hành vi khi xảy ra network partition: chấp nhận trả lời dữ liệu có thể cũ, hay từ chối phục vụ để giữ nhất quán. Đó cũng là lý do thiết kế quản lý dữ liệu cần cân nhắc kỹ trade-off giữa consistency và availability cho *từng bài toán cụ thể*.

Shared database **phá vỡ nguyên lý cốt lõi nhất** của microservices: khả năng triển khai độc lập. Newman trong [4b, Ch.4] chỉ ra vấn đề gốc: các service mất quyền quyết định phần nào được chia sẻ, phần nào được ẩn, và không còn rõ ai "kiểm soát" dữ liệu. Từ đó kéo theo ba hậu quả:



Hình 7.1: Ba hậu quả của shared database trong microservices

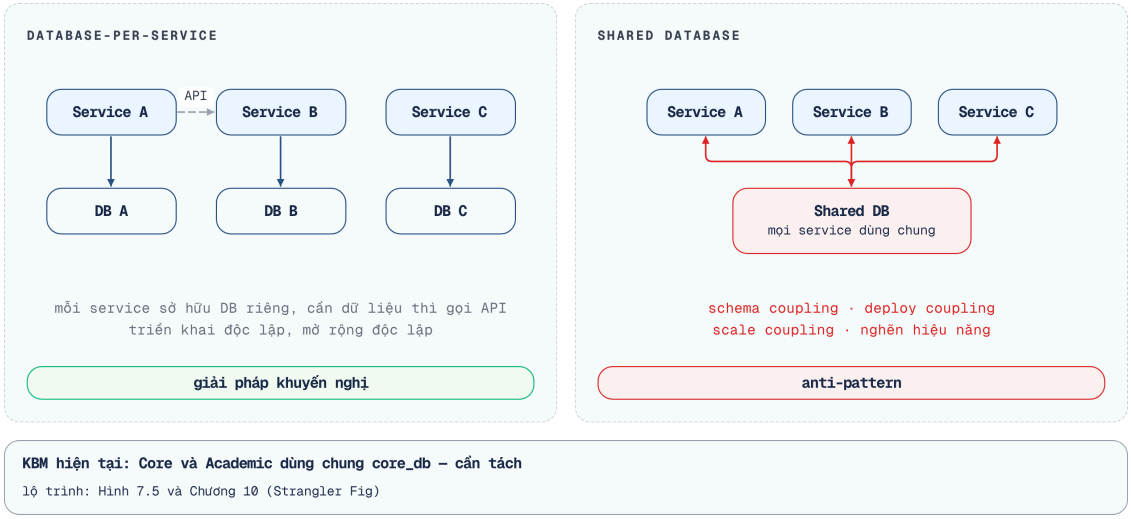
1. Schema coupling — Khi hai service đọc cùng bảng `users`, thay đổi schema (đổi tên cột, thêm constraint) ảnh hưởng cả hai. Mỗi lần cập nhật cấu trúc cơ sở dữ liệu (database migration) lại trở thành một bài toán phối hợp liên nhóm phức tạp — đây chính xác là vấn đề mà kiến trúc microservices muốn loại bỏ.

2. Phụ thuộc khi triển khai — Database migration phải được deploy cùng với tất cả service sử dụng bảng đó. Nếu Service A đổi tên một cột trong `users`, Service B phải cập nhật mã nguồn và triển khai cùng lúc — mặc dù Service B không hề liên quan đến lý do đổi tên đó.

3. Phụ thuộc khi mở rộng — Không thể mở rộng database cho riêng một service: một service tải nặng làm chậm mọi service dùng chung. Cũng không chọn được công nghệ phù hợp cho từng service (Elasticsearch cho tìm kiếm, PostgreSQL cho ACID transaction).

7.1.2. Database-per-Service pattern

Nguyên tắc **database-per-service** yêu cầu: mỗi service sở hữu dữ liệu riêng, và **service khác không bao giờ truy cập trực tiếp vào database đó** — dữ liệu chỉ được cung cấp ra ngoài qua API hoặc event contract do service sở hữu kiểm soát [2a, Ch.1].



Hình 7.2: *Shared Database (đọc/ghi chung) vs Database-per-Service (dữ liệu qua API)*

"Database riêng" không nhất thiết là database server riêng. Có ba cấp độ tách:

Bảng 7.1: *Ba cấp độ tách database*

Cấp độ	Mô tả	Mức cô lập	Chi phí
Separate schema	Cùng DB server, khác schema/ namespace	Thấp (vẫn có thể truy vấn xuyên schema)	Thấp
Separate database	Cùng DB server, khác database	Trung bình	Thấp
Separate server	Khác DB server hoàn toàn	Cao (cho phép đa công nghệ lưu trữ, polyglot persistence)	Cao

Cấp độ nào phù hợp tùy thuộc vào yêu cầu cô lập. Với KBM hiện tại, **separate schema** là bước đầu tiên hợp lý — chi phí thấp nhưng ngăn được schema coupling ngầm.

Dữ liệu có chủ sở hữu, không dùng chung

"If one service wants to access data held by another service, then it should go and ask that service for the data it needs. This gives the service the ability to decide what is shared and what is hidden."

— *Sam Newman, Monolith to Microservices [4b, Ch.1]*

Đối chiếu với nguyên tắc này, việc dùng chung cơ sở dữ liệu giữa Core và Academic trong KBM là một khoản nợ kỹ thuật chuyển tiếp cần được quản lý rõ ranh giới:

KBM shared database

KBM hiện sử dụng **shared database** (`core_db`) giữa Core Service và Academic Service (`kbm-academic`) — cả hai service đọc/ghi cùng PostgreSQL instance, có khả năng truy cập chéo bảng. Đây là hệ quả chuyển tiếp của việc Academic được tách ra từ Core (cả hai ban đầu thuộc cùng monolith). Với đội ngũ 1–2 lập trình viên chính, đây là **nợ kỹ thuật chuyển tiếp được chấp nhận có điều kiện** (chưa bắt buộc tách máy chủ vật lý ngay), nhưng nếu thiếu ranh giới sở hữu rõ

ràng, thay đổi schema của Academic có thể ảnh hưởng Core và ngược lại — cản trở khả năng triển khai độc lập.

Lộ trình chuyển đổi: (1) kiểm kê bảng, xác định một chủ sở hữu ghi duy nhất cho từng bảng (dựa vào phân tích Bounded Context ở Chương 2) và chỉ giữ một pipeline migration duy nhất (tắt `JPA auto-DDL` cạnh tranh), (2) tách schema riêng kèm database role riêng trong cùng PostgreSQL, (3) thay thế các truy vấn chéo bảng (cross-table query) bằng lời gọi API hoặc bản sao qua sự kiện tùy bài toán, (4) dài hạn: chỉ tách database server vật lý khi phát sinh nhu cầu mở rộng độc lập hoặc đa công nghệ lưu trữ.

7.1.3. Case study thực tế: Uber — Domain-Oriented Microservice Architecture (DOMA)

Uber (2020) công bố bài học từ việc quản lý **khoảng 2.200 microservice** — trong đó quản trị phụ thuộc và quyền sở hữu dữ liệu xuyên hàng nghìn service là những thách thức trung tâm. Adam Gluck mô tả cách Uber chuyển từ "microservices spaghetti" sang **DOMA** (Domain-Oriented Microservice Architecture):

Kiến trúc của Uber đi qua ba chặng. Ban đầu, việc chuyển đổi từ monolith sang microservices diễn ra tự do, dẫn đến số lượng service gia tăng nhanh chóng và đồ thị phụ thuộc ngày càng phức tạp.

Chặng tiếp theo, hệ thống rơi vào trạng thái "microservices spaghetti" với hàng nghìn dịch vụ giao tiếp và truy cập dữ liệu chéo nhau, phá vỡ tính tự chủ.

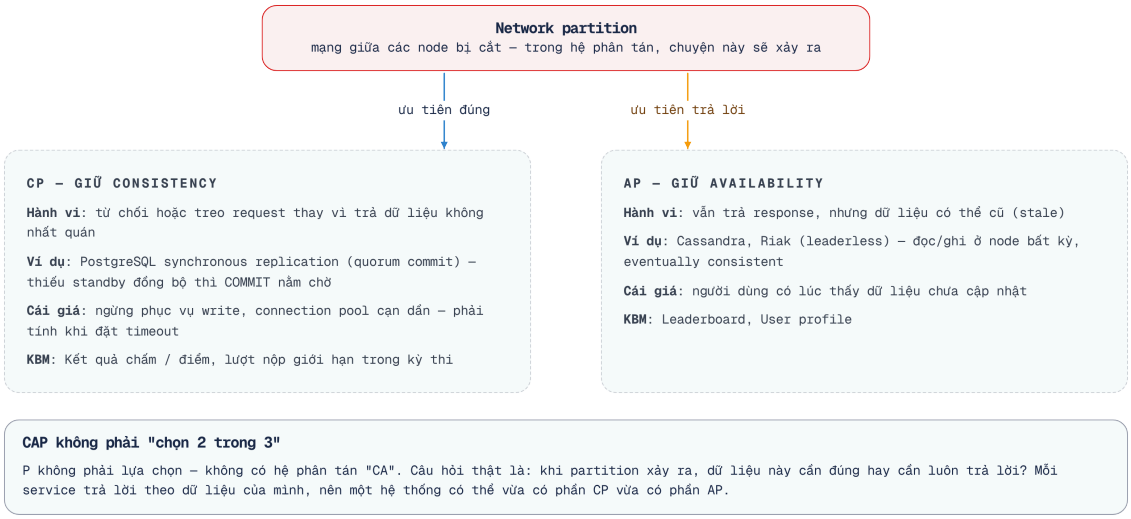
Sau đó, Uber áp dụng DOMA bằng cách gom nhóm các dịch vụ thành những miền (domain) chức năng. Mọi giao tiếp từ bên ngoài bắt buộc đi qua một domain gateway duy nhất, qua đó giảm coupling về dữ liệu.

Bài học rút ra cho quản lý dữ liệu (giáo trình tổng hợp từ case study trên): (1) Database-per-service là cần thiết nhưng *chưa đủ* — cần thêm *domain-level data encapsulation*. (2) Khi hệ thống lớn, nhóm các service thành domain giúp kìm đà tăng của số phụ thuộc chéo. (3) Data duplication giữa các domain là chấp nhận được — domain gateway kiểm soát dữ liệu nào được cung cấp ra ngoài.

Nguồn: Adam Gluck, "Introducing Domain-Oriented Microservice Architecture," Uber Engineering Blog, July 2020 (www.uber.com/blog/microservice-architecture/). Xem thêm: Uber Technology Day 2019 talks.

7.1.4. CAP Theorem — hiểu đúng giới hạn

CAP Theorem được đề cập ngắn gọn ở trên, nhưng hiểu *đúng* CAP rất quan trọng khi thiết kế quản lý dữ liệu. Kleppmann trong [7, Ch.9] phân tích kỹ: CAP không phải "chọn 2 trong 3" như thường hiểu sai — mà là: **khi network partition xảy ra** (và nó *sẽ* xảy ra trong hệ thống phân tán), hệ thống phải chọn giữa consistency và availability.



Hình 7.3: CAP Theorem — CP (chọn consistency) vs AP (chọn availability) khi xảy ra network partition

Hệ thống CP (Consistency + Partition tolerance): khi network partition xảy ra, hệ thống ưu tiên consistency — từ chối request thay vì trả dữ liệu không nhất quán. Ví dụ: PostgreSQL với synchronous replication (quorum commit) đặt consistency lên trên — khi không đủ standby đồng bộ, primary **giữ COMMIT ở trạng thái chờ** thay vì xác nhận một thao tác ghi có thể mất. Hệ quả vận hành: dịch vụ ngừng phục vụ thao tác ghi (và connection pool cạn dần) chứ không trả về lỗi nhanh — cần tính đến khi đặt timeout.

Hệ thống AP (Availability + Partition tolerance): khi network partition xảy ra, hệ thống ưu tiên availability — vẫn trả response nhưng dữ liệu có thể đã cũ. Ví dụ: Cassandra, Riak (kiến trúc Leaderless) với mức nhất quán thấp như ONE cho phép đọc/ghi ở bất kỳ node nào còn truy cập được — dữ liệu *eventually* consistent. Cassandra chọn consistency level theo từng thao tác: QUORUM hoặc ALL có thể không đáp ứng được khi network partition làm thiếu replica.

Bảng 7.2: CP vs AP cho từng luồng dữ liệu trong KBM

Luồng dữ liệu / Nghiệp vụ	Loại dữ liệu	Nên CP hay AP?	Lý do
Kết quả chấm / điểm	Transactional	**CP**	Điểm phải đúng: thà tạm từ chối ghi còn hơn ghi điểm sai, dẫn đến khiếu nại. (Việc *tiếp nhận* bài nộp vẫn ưu tiên sẵn sàng — Chương 6.)
Leaderboard	Derived/read	**AP**	Chấp nhận eventual consistency. Người dùng thà thấy bảng xếp hạng cập nhật chậm vài giây còn hơn là không thể truy cập dịch vụ.
User profile	Reference	**AP**	Tên sinh viên rất hiếm khi thay đổi, do đó việc tồn tại dữ liệu cũ (stale data) trong vài phút không gây ra tác động nghiêm trọng
Lượt nộp giới hạn trong kỳ thi	Transactional	**CP**	Số lượt nộp tối đa của thí sinh phải nhất quán chặt để đảm bảo công bằng; không cho phép ghi vượt hạn mức ở hai phía của network partition

Một lưu ý quan trọng: CAP Theorem *chỉ* phát biểu về sự đánh đổi giữa Consistency và Availability khi network partition thực sự xảy ra. Trong điều kiện bình thường (không có network partition), hệ thống không bị buộc phải từ chối phục vụ như khi đứt mạng, nhưng theo cách phát biểu mở rộng **PACELC** (Daniel Abadi), hệ thống có nhân bản dữ liệu vẫn phải đánh đổi giữa **độ trễ** (chữ *L*) và **tính nhất quán** (chữ *C*) — tức chờ xác nhận đồng bộ từ các bản sao hay trả lời ngay trên node cục bộ. Hiểu đúng cả hai vế giúp tránh thiết kế quá phòng thủ: không mặc định chấp nhận eventual consistency ở những nơi một cơ sở dữ liệu chính hoặc một quorum nội bộ có độ trễ thấp đã đáp ứng tốt.

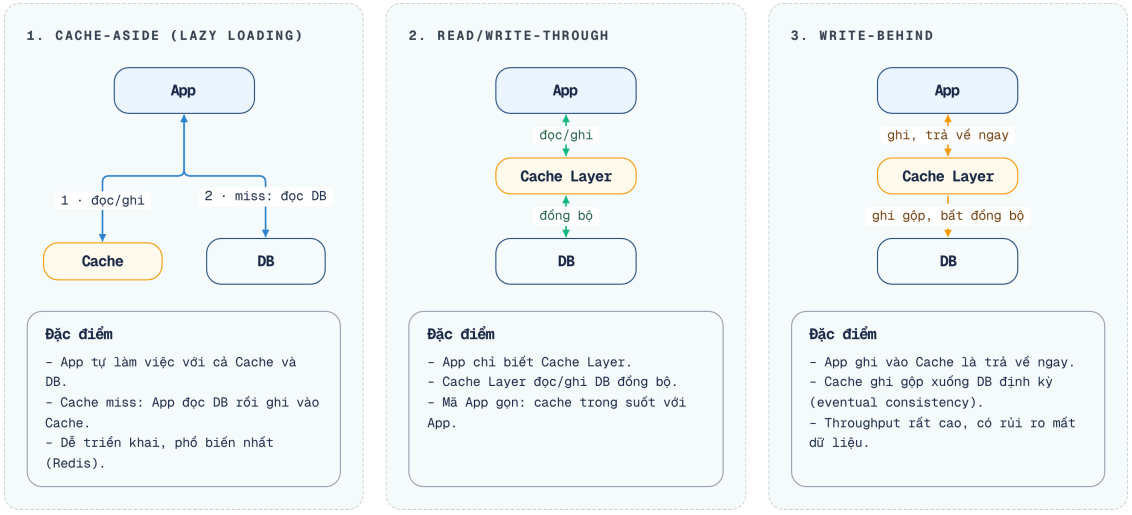
Quyết định CAP theo từng service

Không phải toàn bộ hệ thống "là CP" hay "là AP". Mỗi service, thậm chí mỗi *bài toán*, chọn trade-off riêng. Cách áp dụng CAP vào thực tế là phân tích *từng luồng dữ liệu* — dữ liệu nào cần strong consistency (mọi lần đọc thấy bản ghi mới nhất), dữ liệu nào chấp nhận eventual consistency. Lưu ý phân biệt: chữ C của CAP nói về tính nhất quán giữa các bản sao khi đọc/ghi, khác với chữ C trong ACID (transaction bảo toàn ràng buộc nghiệp vụ) — hai khái niệm trùng tên nhưng không trùng nghĩa.

7.1.5. Chiến lược caching — giảm tải và độ trễ

Khi cơ sở dữ liệu đã được phân tách, các truy vấn xuyên service sẽ có độ trễ cao hơn do phải thực hiện qua mạng. Sử dụng bộ nhớ đệm (caching) là cách phổ biến để giảm độ trễ này, nhưng đòi hỏi chiến lược phù hợp để tránh tình trạng dữ liệu cũ.

Các pattern chính:



Hình 7.4: *Các caching pattern — Cache-Aside, Read-Through/Write-Through, Write-Behind*

Bảng 7.3 so sánh ưu, nhược điểm của các pattern này và khả năng áp dụng cho KBM.

Bảng 7.3: So sánh ba caching pattern

Pattern	Ưu điểm	Nhược điểm	Có thể áp dụng cho KBM
Cache-Aside	Đơn giản, ứng dụng kiểm soát	Khi không tìm thấy dữ liệu trong bộ đệm (cache miss), hệ thống phải thực hiện hai luồng truy vấn (tới cache và database) làm tăng độ trễ	Danh sách câu hỏi, user profile
Read-Through / Write-Through	Trong suốt (transparent) đối với ứng dụng	Tầng cache phức tạp hơn	—
Write-Behind	Ghi nhanh (bất đồng bộ)	Nguy cơ mất dữ liệu nếu cache gặp sự cố	(ví dụ chung) bộ đếm lượt xem, ghi dồn theo lô

Cache Invalidation — "There are only two hard things in Computer Science: cache invalidation and naming things." (Phil Karlton).

Chiến lược cache invalidation xoay quanh ba phương pháp chính:

- **TTL (Time-to-Live)**: Dữ liệu tự hết hạn sau một khoảng thời gian định trước. Đơn giản nhất, phù hợp với dữ liệu tham chiếu ít thay đổi như danh sách câu hỏi.
- **Event-driven invalidation**: Cache chỉ bị xóa khi hệ thống gốc phát sự kiện thay đổi. Hiệu quả hơn cho dữ liệu biến động liên tục như điểm số.
- **Versioned keys**: Gắn số phiên bản vào khóa cache. Thường dùng để dọn hàng loạt khi cấu trúc dữ liệu thay đổi lớn.

Trong KBM, bên cạnh rate limiting theo token bucket ở Gateway (Chương 8), kiến trúc mục tiêu mở rộng vai trò của Redis sang caching và bảng xếp hạng trực tiếp (mục sau) — tận dụng TTL tự động và cơ chế pub/sub cho các sự kiện làm mới bộ đệm (invalidation event).

7.2. Chiến lược tách database từ monolith

Chia nhỏ cơ sở dữ liệu thường gặp khó khăn tương tự như khi trường đại học tách phòng ban: tách nhân sự (code) thì nhanh chóng, nhưng bóc tách hồ sơ giấy tờ (dữ liệu) lại phức tạp hơn nhiều.

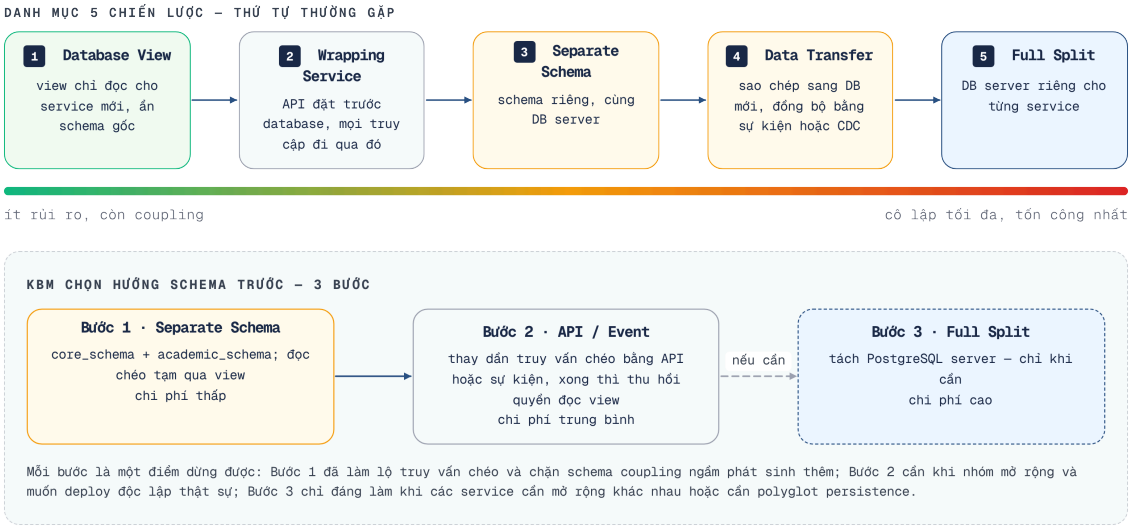
7.2.1. Vấn đề: tách service dễ, tách dữ liệu khó

Nhiều nhóm phát triển tách microservices ở tầng ứng dụng (deploy riêng, repo riêng) nhưng vẫn **trỏ vào cùng database**. Đây là bước chuyển đổi mới đi được nửa đường. Service đã "tách" nhưng dữ liệu vẫn bị gắn chặt — lợi ích triển khai độc lập gần như bằng 0.

Tách database là **một trong những bước khó nhất** trong quá trình chuyển đổi từ monolith sang microservices. Kleppmann trong [7, Ch.5–6] phân tích: khi dữ liệu nằm ở nhiều nơi, mọi vấn đề của hệ thống phân tán xuất hiện — replication lag, partitioning, consistency trade-off. Đây là chi phí không thể tránh.

7.2.2. Năm chiến lược tách database

Từ các pattern của Newman [4b, Ch.4], giáo trình rút ra năm chiến lược theo thứ tự thường gặp khi chuyển đổi. Thứ tự này nhìn chung đi từ ít đến nhiều rủi ro, nhưng không tuyệt đối: tách schema thường ít rủi ro hơn bước thay truy vấn chéo bằng API (bảng chiến lược ở Chương 10). Đây là danh mục tổng quát; KBM chỉ chọn lọc 3 chiến lược phù hợp, xem bảng dưới.



Hình 7.5: *Năm chiến lược tách database, theo thứ tự thường gặp khi chuyển đổi*

1. Database View — Tạo view chỉ đọc cho service mới, ẩn schema gốc. Ưu điểm: không thay đổi dữ liệu, service cũ không bị ảnh hưởng. Nhược điểm: chỉ đọc được, vẫn phụ thuộc vào schema.

2. Database Wrapping Service — Đặt một API đơn giản phía trước database, mọi truy cập đi qua API này. Ưu điểm: bắt đầu tách coupling mà không di chuyển dữ liệu. Nhược điểm: làm phát sinh thêm một dịch vụ cần được bảo trì.

3. Separate Schema — Tách bảng vào schema/namespace riêng trong cùng database. Ưu điểm: sự cô lập rõ ràng, không yêu cầu thiết lập hạ tầng mới. Nhược điểm: cùng DB server → vẫn dùng chung tài nguyên.

4. Data Transfer — Sao chép dữ liệu sang database mới, giữ đồng bộ bằng event hoặc CDC (Change Data Capture). Ưu điểm: service mới có dữ liệu riêng. Nhược điểm: cần cơ chế đồng bộ, eventual consistency.

5. Full Database Split — Mỗi service có database server hoàn toàn riêng. Ưu điểm: cô lập tối đa, cho phép đa công nghệ lưu trữ (polyglot persistence). Nhược điểm: đòi hỏi nhiều công sức triển khai và phải xử lý phức tạp các truy cập dữ liệu xuyên service.

7.2.3. Áp dụng cho KBM

Danh mục năm chiến lược cho biết *có những bước nào* câu hỏi còn lại là *bước nào đi trước*. Với hai service đang dùng chung `core_db`, có hai hướng hợp lý. Chúng dùng cùng các chiến lược và đi đến cùng một đích — chỉ khác ở chỗ dựng ranh giới dữ liệu trước hay gỡ truy vấn chéo trước.

- **Hướng A — schema trước**: tách `core_db` thành `core_schema` và `academic_schema` ngay trong cùng PostgreSQL, mỗi service một database role. Những truy vấn chéo còn lại tạm đọc qua view được cấp quyền rõ ràng, nên Academic chưa phải đổi code. Từ đây mọi truy cập chéo phải gọi đích danh schema, nên đếm và theo dõi được; lần lượt thay chúng bằng API, rồi mới thu hồi quyền đọc chéo. Được: chi phí thấp, ranh giới thấy được ngay, và biết chính xác cần viết những API nào. Trả giá: trong giai đoạn đầu ranh giới còn "mềm" — vẫn dựa vào quyền đọc tạm thời và kỷ luật của nhóm.
- **Hướng B — API trước**: đây là thứ tự tổng quát trình bày ở Chương 10 (bảng năm chiến lược theo góc độ chuyển đổi): thay truy vấn chéo bằng API (hoặc view) trước; khi không còn truy

vấn chéo nào, việc tách schema gần như không còn rủi ro. Được: bước tách schema an toàn, mỗi bước đều rollback được. Trả giá: lâu mới có ranh giới ở tầng dữ liệu, và phải đoán trước truy vấn chéo nào cần API khi chưa có số liệu.

Hướng B an toàn hơn khi nhiều service và nhiều nhóm cùng chạm vào một database: không ai phải chịu một đợt đổi schema lớn khi truy vấn chéo còn nằm rải rác. Hướng A hợp khi chỉ có vài service, một DB server và một nhóm nhỏ: dựng ranh giới rẻ, và số liệu truy vấn chéo thu được giúp tránh viết API thừa.

KBM chọn **hướng A**. Chỉ Core và Academic dùng chung `core_db`, trên cùng một PostgreSQL, với đội phát triển một đến hai người. Rủi ro lớn nhất hiện nay là schema coupling ngầm — chưa ai biết chắc Academic đang đọc những bảng nào của Core — nên bước có giá trị nhất là một ranh giới rẻ làm lộ ra các truy vấn đó:

__Bảng 7.4:__ Lộ trình tách database cho KBM theo hướng schema trước (3 bước chọn lọc từ danh mục 5 chiến lược)

Bước	Chiến lược	Mô tả	Chi phí
1	**Separate Schema**	Kiểm kê chủ sở hữu bảng, tắt auto-DDL cạnh tranh; tách `core_db` thành `core_schema` và `academic_schema` với role riêng, các truy vấn chéo tạm đọc qua view	Thấp
2	**API / Event theo bài toán**	Thay dần truy vấn chéo bằng API hoặc bản sao qua sự kiện tùy luồng đọc/ghi; hết truy vấn chéo thì thu hồi quyền đọc view	Trung bình
3	**Full Split** (nếu cần)	Tách PostgreSQL server vật lý khi phát sinh nhu cầu mở rộng độc lập	Cao

Khi nào đủ?

Không phải hệ thống nào cũng cần đến Bước 3. Với KBM, Bước 1 (separate schema) đủ để làm lộ toàn bộ truy vấn chéo và chặn schema coupling ngầm phát sinh thêm. Bước 2 (API / Event theo bài toán) cần khi nhóm mở rộng và cần deploy độc lập thực sự. Bước 3 chỉ cần khi có yêu cầu mở rộng khác nhau hoặc polyglot persistence.

Chi tiết lộ trình chuyển đổi, Outbox Pattern (reliable messaging khi tách DB), và thứ tự triển khai → xem Chương 10.

7.3. Data Duplication vs Coupling — khi nào chấp nhận nhân bản dữ liệu?

Trong kiến trúc microservices, việc nhân bản dữ liệu thường được áp dụng để đảm bảo tính độc lập. Cách làm này tương tự như việc Thư viện lưu bản sao thông tin thẻ sinh viên từ Phòng Đào tạo để tra cứu nhanh mà không cần đối chiếu chéo liên tục.

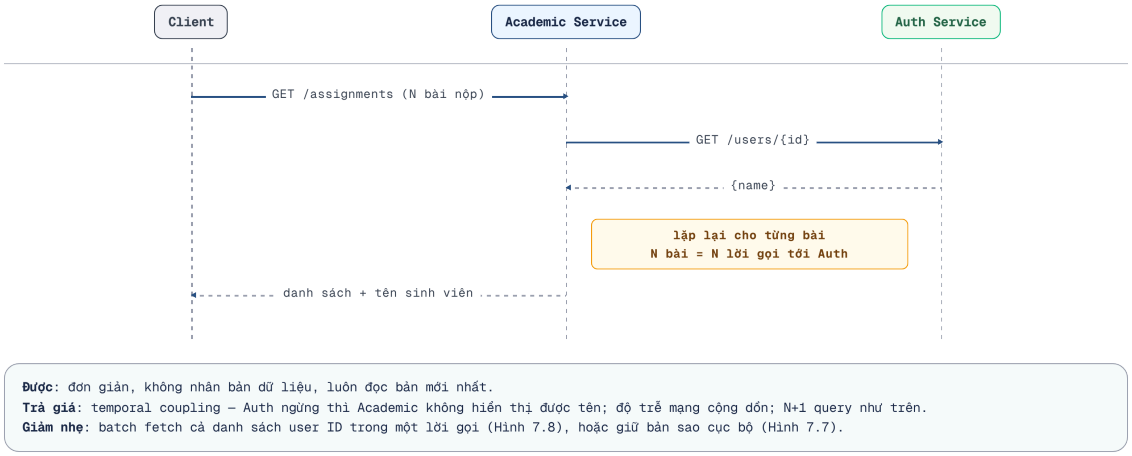
7.3.1. Vấn đề: cần dữ liệu từ service khác

Sau khi tách database, vấn đề tiếp theo lập tức xuất hiện: **service A cần dữ liệu mà service B sở hữu**. Ví dụ trong KBM: Academic Service cần hiển thị tên sinh viên — nhưng `users` thuộc về Auth Service. Gọi Auth API mỗi lần cần tên? Lưu bản sao tên sinh viên tại Academic?

Có ba cách tiếp cận, mỗi cách có trade-off riêng; cách thứ nhất tương ứng Data Delegate pattern của Mitra [3, Ch.5]:

7.3.2. Ba chiến lược truy cập dữ liệu xuyên service

1. API Call (Runtime dependency)
Cách đơn giản nhất là gọi API của service sở hữu dữ liệu mỗi khi cần (Hình 7.6).

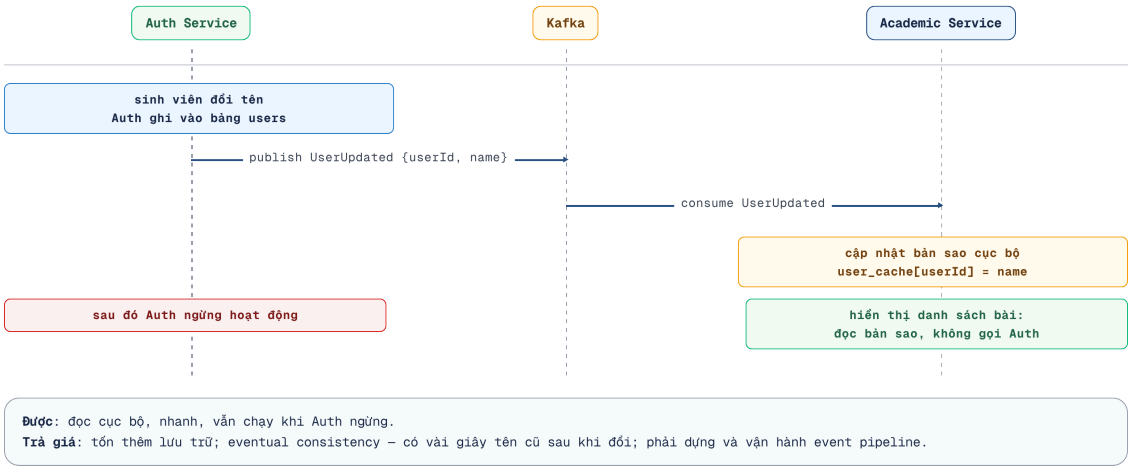


Hình 7.6: *Truy cập dữ liệu xuyên service bằng API Call*

API Call được chọn vì đơn giản khi triển khai, không nhân bản dữ liệu và luôn đọc được thông tin mới nhất. Cái giá phải trả gồm temporal coupling (dịch vụ gọi bị gián đoạn theo dịch vụ đích: Auth ngừng hoạt động thì Academic không hiển thị được tên người dùng), độ trễ mạng cộng dồn và việc luồng đọc dễ rơi vào bài toán N+1 truy vấn (N+1 query).

2. Data Duplication (Local copy)

Cách thứ hai là giữ một bản sao cục bộ, được cập nhật qua event pipeline từ service sở hữu (Hình 7.7).

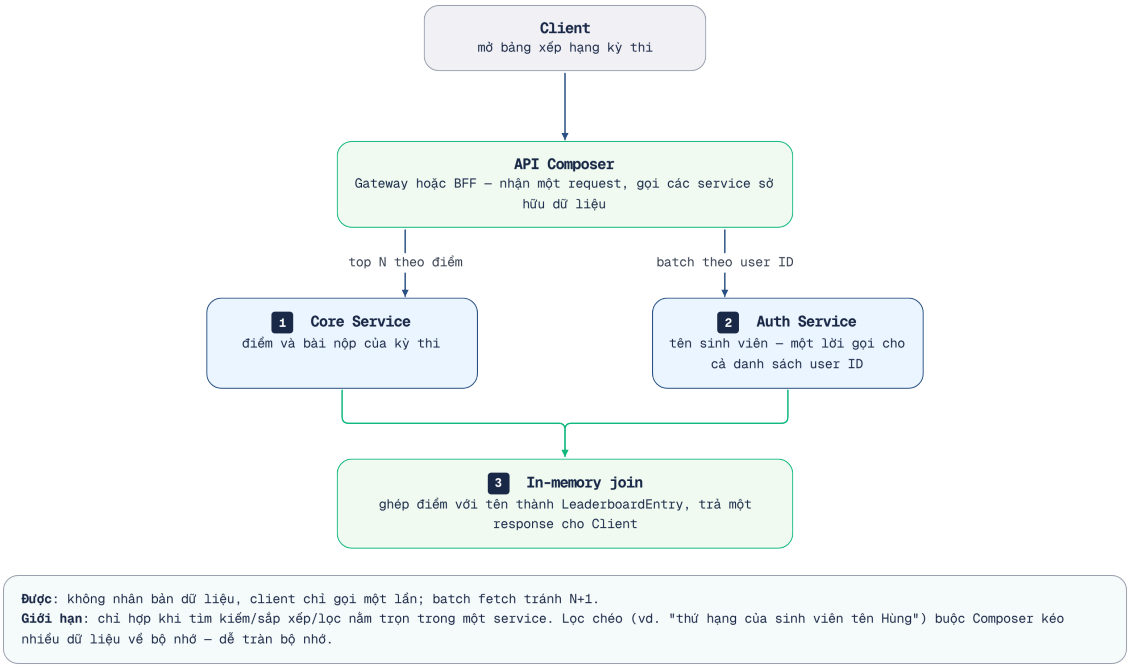


Hình 7.7: *Truy cập dữ liệu xuyên service bằng Data Duplication qua event pipeline*

Ngược lại, Data Duplication khắc phục điểm yếu của API Call bằng cách loại bỏ runtime dependency và cho phép truy vấn dữ liệu cục bộ với tốc độ cao ngay cả khi các dịch vụ khác đang gặp sự cố. Đổi lại, hệ thống phải chấp nhận tốn thêm dung lượng lưu trữ, đối mặt với eventual consistency và tình trạng dữ liệu cũ tạm thời, và tốn công sức xây dựng các event pipeline phức tạp.

3. API Composition / Data Aggregation

Dùng khi cần ghép dữ liệu từ nhiều service. Một service (hoặc API Gateway) gọi nhiều service rồi tổng hợp. Richardson trong [2a, Ch.7] gọi đây là **API Composition pattern**:



Hình 7.8: API Composition pattern — Gateway hoặc BFF gọi nhiều service rồi tổng hợp

Ví dụ: hiển thị bảng xếp hạng kỳ thi cần dữ liệu từ Core (bài nộp) và Auth (tên người dùng). API Composer gọi cả hai service rồi thực hiện **in-memory join**: truy vấn theo lô (batch fetch) danh sách user ID để tránh lỗi N+1 truy vấn, sau đó tổng hợp dữ liệu trong bộ nhớ để tạo ra `LeaderboardEntry`.

⚠ Tràn bộ nhớ (Out-of-Memory) khi lọc chéo trên in-memory join

Phương pháp in-memory join hoạt động rất hiệu quả nếu chỉ lấy dữ liệu hiển thị. Tuy nhiên, nếu phát sinh yêu cầu **tìm kiếm/lọc chéo** (ví dụ: "Tìm thứ hạng của sinh viên tên Hùng"), API Composer sẽ phải truy vấn toàn bộ dữ liệu từ Auth (tất cả user tên Hùng) rồi chuyển cho Core để lọc, có thể gây tràn bộ nhớ (Out-of-Memory). API Composition phù hợp khi tập dữ liệu cần ghép có kích thước kiểm soát được; truy vấn lọc, sắp xếp hoặc phân trang xuyên service trên tập lớn thì cần read model riêng.

7.3.3. Khi nào chấp nhận nhân bản dữ liệu?

Newman trong [4a, Ch.4] nhận định ở tầng mã nguồn: giữa các service, tác hại của coupling quá chặt lớn hơn nhiều so với vấn đề do lặp mã gây ra; với dữ liệu, cùng lập luận đó ủng hộ việc giữ bản sao. Tuy nhiên, điều này không đồng nghĩa với việc cho phép nhân bản mọi loại dữ liệu một cách tùy tiện. Quy tắc:

Bảng 7.5: Khi nào chấp nhận nhân bản dữ liệu (data duplication)

Loại dữ liệu	Nên nhân bản?	Lý do
Reference data (tên user, tên khóa học)	✓ Có	Ít thay đổi, cần hiển thị ở nhiều nơi

Loại dữ liệu	Nên nhân bản?	Lý do
Transactional data (điểm số, trạng thái bài nộp)	× Không (Master-Master)	Không nhân bản 2 chiều. Nếu tách read model thì sao chép một chiều từ write model
Configuration (loại database, danh mục)	✓ Có	Gần như tĩnh, cache tốt
Audit data (lịch sử thay đổi)	× Không (đa nguồn ghi)	Chỉ một nơi được ghi; bản sao chỉ đọc đưa về Data Warehouse qua ETL/Data Streaming (một chiều) là hợp lệ

Nguyên tắc quan trọng nhất ở đây là phải luôn biết rõ ai đang là người làm chủ nguồn dữ liệu được sao chép.

Single Source of Truth

Nhân bản dữ liệu (duplication) chỉ chấp nhận được khi tồn tại một **single source of truth** (nguồn sự thật duy nhất) rõ ràng: Service A sở hữu dữ liệu, Service B giữ bản sao. Khi dữ liệu thay đổi, thay đổi chảy theo chiều A → B (qua event), không bao giờ theo chiều ngược lại. Nếu không thể xác định rõ ràng quyền sở hữu đối với một tập dữ liệu, đó là dấu hiệu cho thấy Bounded Context được thiết kế chưa tốt. Trong trường hợp này, kiến trúc sư cần xem xét lại quá trình phân tích ở Chương 2.

Hiện KBM vẫn dựa vào chiến lược thứ nhất (API call) cho các truy vấn xuyên service:

Truy vấn xuyên service trong KBM

KBM hiện dùng **Feign call** để truy vấn dữ liệu xuyên service — ví dụ: Academic Service gọi Core Service để lấy danh sách câu hỏi. Đây là API call pattern (chiến lược 1). Với quy mô học thuật vừa phải, cách này chấp nhận được. Tuy nhiên, trong chế độ thi hoặc trên các bảng điều khiển (dashboard) chứa nhiều thông tin, mỗi lần tải trang có thể kích hoạt hàng loạt các lệnh gọi Feign liên tiếp, dẫn đến độ trễ của hệ thống tăng lên đáng kể.

Lộ trình chuyển đổi: (1) xác định dữ liệu nào Academic cần *hiển thị* từ Core (chủ yếu reference data: tên câu hỏi, tên kỳ thi), (2) nhân bản dữ liệu tham chiếu (reference data) thông qua các sự kiện Kafka (`UserUpdated`, `QuestionUpdated`), (3) giữ Feign call cho các truy vấn giao dịch (transactional query) (kiểm tra điểm theo thời gian thực).

7.4. CQRS — Command Query Responsibility Segregation

Hệ thống phân tán có thể tách biệt thao tác ghi và đọc dữ liệu để tối ưu hiệu suất, tương tự như việc trường học tách riêng cổng nộp hồ sơ nhập học (ghi) và bảng tin xem kết quả (đọc).

7.4.1. Từ CQS đến CQRS

Trước khi nói về CQRS, cần hiểu nguồn gốc. Bertrand Meyer đề xuất **CQS** (Command Query Separation) như một nguyên tắc thiết kế *ở cấp độ method*: mỗi method hoặc trả về dữ liệu (query, không thay đổi state) hoặc thay đổi state (command, không trả về dữ liệu) — nhưng không làm cả hai. Rocha trong [5, §4.5.1] phân biệt rõ: **CQS là nguyên tắc ở cấp một service, một class**, còn **CQRS mở rộng nguyên tắc này lên cấp kiến trúc hệ thống** — tách hẳn *model* (thậm chí *database*) cho đọc và ghi. CQRS không bắt buộc phải có CQS, nhưng tinh thần tương tự: tách biệt trách nhiệm đọc và ghi.

7.4.2. Vấn đề: cùng model cho đọc và ghi là không đủ

Trong monolith, cùng một entity `Submission` phục vụ cả hai mục đích:

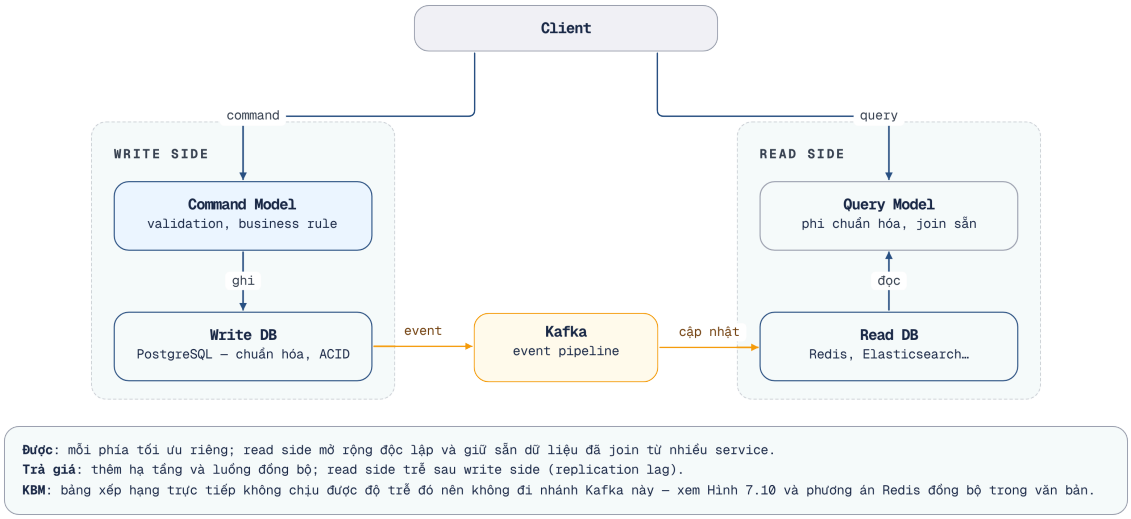
- **Write**: nộp bài (INSERT), cập nhật kết quả (UPDATE)
- **Read**: hiển thị lịch sử (SELECT nhiều cột, JOIN nhiều bảng, pagination, filtering)

Yêu cầu của các thao tác đọc và ghi là hoàn toàn khác biệt. Quá trình ghi đòi hỏi tính nhất quán, khả năng kiểm tra tính hợp lệ (validation) và phải tuân thủ chặt chẽ các quy tắc nghiệp vụ (business rule). Trong khi đó, quá trình đọc lại ưu tiên hiệu năng, thường sử dụng cấu trúc dữ liệu phi chuẩn hóa (denormalized data) để hỗ trợ các truy vấn linh hoạt. Khi hệ thống phức tạp, dùng chung một mô hình dữ liệu duy nhất sẽ buộc phải dung hòa cả hai yêu cầu này, khiến không thao tác nào đạt mức tối ưu [2a, Ch.7].

Richardson trong [2a, Ch.7] mô tả vấn đề này đặc biệt nghiêm trọng khi cần **truy vấn xuyên service**: *không thể* hiển thị bảng xếp hạng kỳ thi (dữ liệu từ Core + Auth) bằng một truy vấn duy nhất khi dữ liệu nằm ở nhiều database.

7.4.3. CQRS pattern

CQRS (Command Query Responsibility Segregation) tách model thành hai phần riêng biệt [2a, Ch.7]:



Hình 7.9: CQRS pattern — tách command model (write) và query model (read)

Command side (write):

- Mô hình dữ liệu chuẩn hóa (3NF)
- Kiểm tra dữ liệu chặt, áp business rule
- Database: PostgreSQL (ACID transaction)

Query side (read):

- View phi chuẩn hóa, join sẵn
- Tối ưu cho từng kiểu truy vấn
- Database: có thể khác công nghệ lưu trữ (Elasticsearch cho tìm kiếm, Redis cho bảng xếp hạng, PostgreSQL materialized view cho báo cáo)

7.4.4. Khi nào cần CQRS?

CQRS làm tăng độ phức tạp đáng kể — **không sử dụng khi không thực sự cần thiết** [5, §4.5]:

Bảng 7.6: Khi nào cần CQRS

Kịch bản	Cần CQRS?	Lý do
Tỉ lệ đọc/ghi cân bằng, cùng model	× Không	CRUD đơn giản là đủ
Đọc phức tạp (join xuyên service)	✓ Có	Query model phi chuẩn hóa giải quyết join
Lượng đọc >> lượng ghi	! Tùy	Thử read replica và cache trước; dùng CQRS khi read model cần cấu trúc khác hẳn write model
Giao diện cần cập nhật theo thời gian thực (leaderboard)	✓ Có	Pre-computed view nhanh hơn
Nhóm phát triển quy mô nhỏ, dự án ở giai đoạn thử nghiệm (MVP)	× Không	Độ phức tạp phát sinh lớn hơn nhiều so với lợi ích thu được.

7.4.5. Ví dụ: bảng xếp hạng trực tiếp trong chế độ thi — vì sao CQRS bất đồng bộ không phù hợp

Trong KBM, bảng xếp hạng trong chế độ thi cần dữ liệu từ Auth và Core — SQL JOIN truyền thống không dùng được khi database đã tách.

Cách tiếp cận đơn giản nhất là áp dụng CQRS qua Kafka: write side phát `ScoreUpdatedEvent`, read side tiêu thụ và cập nhật `LeaderboardEntry` phi chuẩn hóa. Tuy nhiên, nếu kỳ thi đang diễn ra yêu cầu người nộp đọc lại thứ hạng ngay sau khi ghi thành công, CQRS bất đồng bộ đơn thuần **không bảo đảm read-after-write** vì eventual consistency tạo ra replication lag. Nếu cho phép trễ vài giây (như bảng CAP ở §7.1) thì vẫn dùng được, kèm hiển thị trạng thái đang cập nhật. Hơn nữa, việc áp dụng "Optimistic UI" (tự đoán thứ hạng) là **không khả thi** vì thứ hạng của thí sinh phụ thuộc vào điểm của nhiều thí sinh khác đang nộp bài cùng lúc.

Giải pháp kiến trúc đề xuất: Sử dụng **Synchronous Materialized View** qua Redis kết hợp với **WebSocket/SSE** thay vì CQRS bất đồng bộ (qua Kafka). Trong kiến trúc mục tiêu của KBM, Redis Sorted Set đóng vai trò read model trong bộ nhớ do chính `kbm-core` (hoặc `kbm-exam` khi tách phân hệ thi ở §2.4) sở hữu; ngay trên đường ghi chính, dịch vụ cập nhật điểm số vào Redis bằng lệnh `ZADD` với kỹ thuật **Composite Score** (ví dụ: `score = (real_score * 10^13) - submit_timestamp_ms`, trong đó `submit_timestamp_ms` là mốc thời gian Unix tính bằng *mili-giây*). (Nếu ở một hệ thống quy mô lớn tách hẳn một dịch vụ đọc bảng xếp hạng độc lập, dịch vụ chấm điểm sẽ gọi **gRPC/REST API đồng bộ** sang dịch vụ đọc đó thay vì chạm trực tiếp vào Redis của nó.)

Composite Score giải quyết bài toán "bằng điểm" (tie-breaker), tránh việc Redis tự động xếp hạng theo bảng chữ cái của `userId`; còn tên hiển thị của sinh viên (`userId → fullName`) được ghép từ bản sao tham chiếu cục bộ/Redis hash được đồng bộ qua sự kiện `UserUpdated` từ `Auth Service`.

Redis cung cấp truy vấn thứ hạng nhanh qua `ZREVRANK` (chính xác, `O(log N)`, in-memory), loại bỏ replication lag trên đường ghi chính (fast path) cho người nộp bài. Tuy nhiên, để những người xem bảng xếp hạng khác thấy được sự thay đổi ngay lập tức, hệ thống vẫn cần tích hợp cơ chế đẩy từ máy chủ xuống trình duyệt (như WebSocket hoặc Server-Sent Events) để phát thông báo thay đổi thứ hạng ra toàn bộ client.

Phần nâng cao (có thể lướt qua)

Từ đây đến hết hộp Dual-Write là chi tiết cài đặt. Người đọc mới chỉ cần nắm: bảng xếp hạng cập nhật đồng bộ trên đường ghi chính, còn Outbox bảo đảm không mất cập nhật.

Lưu ý về giới hạn số học: Redis lưu score của Sorted Set bằng số thực dấu phẩy động 64-bit, nên chỉ biểu diễn chính xác các số nguyên trong khoảng `±2^53`, xấp xỉ `9 × 10^15`. Công thức trên vì vậy đòi hỏi một điều kiện bất biến (invariant): `real_score` là số nguyên đã chuẩn hóa theo thang 0–100. Khi đó giá trị composite tối đa khoảng `10^15`, còn cách trần an toàn khoảng chín lần. Nếu thang điểm được nới rộng vượt quá 900, phần mili-giây bắt đầu bị làm tròn và tie-breaker mất tác dụng — hai bài nộp cách nhau 1 ms có thể nhận cùng một score. Ngoài ra, hệ số `10^13` chỉ lớn hơn mốc thời gian Unix mili-giây cho đến năm 2286, và leaderboard cần quy ước rõ giữ *thời điểm sớm nhất* khi một người dùng đạt lại cùng mức điểm.

⚠ Dual-Write (ghi kép đồng bộ)

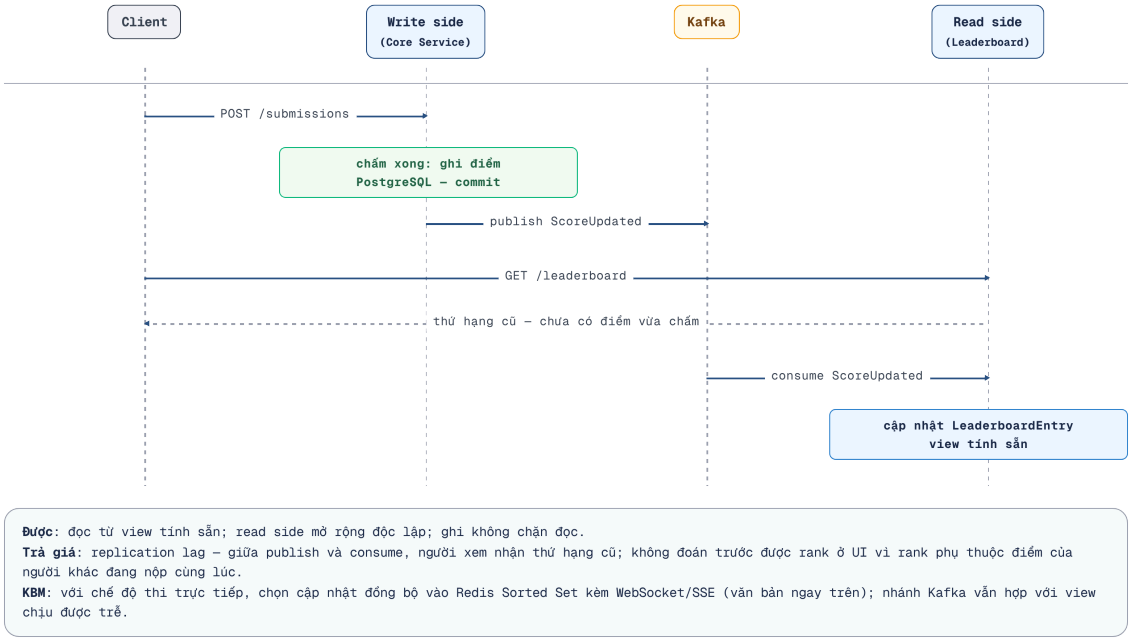
Khi bảng xếp hạng được cập nhật đồng bộ sang một kho lưu trữ thứ hai, thao tác vừa commit PostgreSQL vừa cập nhật Redis luôn tiềm ẩn rủi ro **partial failure** (lỗi mạng hoặc tiến trình sập giữa chừng gây lệch dữ liệu). Kho thứ hai có thể là Redis nội bộ của `kbm-core`/`kbm-exam`, hoặc một dịch vụ đọc tách rời; với trường hợp sau, dịch vụ ghi **không** được chạm trực tiếp vào Redis của dịch vụ đọc (giữ data encapsulation).

Để khép lại rủi ro này, ghi qua outbox phải là **mặc định** chứ không phải phương án dự phòng: mọi thay đổi điểm được ghi vào **Transactional Outbox (Chương 10) trong cùng local transaction** với PostgreSQL, kèm một `eventId` ổn định. Nhờ đó không tồn tại cửa sổ mất dữ liệu nào, kể cả khi dịch vụ ghi sập ngay sau commit.

Bước cập nhật đồng bộ vào Redis khi đó chỉ là *fast path* để xóa độ trễ ở điều kiện bình thường. Nó mang chính `eventId` nói trên và thực thi idempotent, nên bản relay từ outbox đến sau không tạo cập nhật trùng — kể cả khi lệnh cập nhật đồng bộ bị timeout nhưng thực chất đã thành công. Lưu ý `eventId` chỉ chặn trùng lặp của *cùng một event*, không chặn sai thứ tự giữa *các event khác nhau*. Vì vậy event phải mang thêm `scoreVersion` tăng dần, và Lua script chỉ ghi vào Sorted Set khi `scoreVersion` mới hơn version đang lưu kèm (ví dụ trong một hash), so sánh và ghi nguyên tử trong một lần gọi. Nếu không, bản relay trễ của event cũ có thể ghi đè điểm mới.

Một biến thể cần cảnh giác là "thử sync trước, thất bại mới ghi outbox". Cách này **không** đóng được cửa sổ dual-write: nếu dịch vụ sập sau khi commit database mà chưa kịp ghi outbox, bản cập nhật bị mất vĩnh viễn.

Hình 7.10 vẽ đường CQRS bất đồng bộ qua Kafka — chính luồng sinh ra replication lag vừa nói; phương án Redis đồng bộ không có trong hình.



Hình 7.10: *Đường CQRS bất đồng bộ qua Kafka cho bảng xếp hạng*

CQRS ≠ Event Sourcing

CQRS và Event Sourcing thường được nhắc đến cùng nhau, nhưng chúng là **hai pattern độc lập**. CQRS tách read/write model — có thể dùng mà không cần Event Sourcing. Event Sourcing lưu event thay vì state — có thể dùng mà không cần CQRS. Kết hợp cả hai mang lại nhiều lợi ích nhưng cũng *rất phức tạp*. Cách làm an toàn là bắt đầu với CQRS đơn giản trước, thêm Event Sourcing *khi có nhu cầu cụ thể* (audit trail, temporal query).

7.5. Event Sourcing — lưu event thay vì state

Nếu một bảng điểm chỉ ghi nhận kết quả cuối cùng mà không lưu lại quá trình học tập, thi lại hay phúc khảo, hệ thống đã đánh mất phần lịch sử quan trọng nhất. Event Sourcing giữ lại đúng phần đó: nó lưu chuỗi biến động thay vì trạng thái sau cùng, giống một cuốn sổ học vụ ghi đủ mọi lần thay đổi.

7.5.1. Vấn đề: mất lịch sử khi chỉ lưu state

Với database truyền thống, chúng ta lưu **trạng thái hiện tại** (*current state*): `submission.status = JUDGED, score = 85`. Mọi thay đổi trước đó bị ghi đè — không biết bài nộp đã qua những trạng thái nào, ai thay đổi, khi nào.

Trong nhiều domain (tài chính, kiểm toán, pháp lý), lịch sử thay đổi có giá trị ngang bằng hoặc hơn trạng thái hiện tại. Ngay cả trong KBM: "sinh viên nộp bài 3 lần, lần đầu sai, lần 2 đúng một phần, lần 3 đúng hoàn toàn" — thông tin này có giá trị cho phân tích học tập, nhưng nếu chỉ lưu `status = CORRECT`, lịch sử bị mất.

7.5.2. Event Sourcing pattern

Event Sourcing thay đổi cách lưu trữ: thay vì lưu state, lưu **chuỗi sự kiện** đã xảy ra. State hiện tại được **suy ra** bằng cách phát lại (replay) tất cả event [2a, Ch.6]:

Hình 7.11: *Event Sourcing — lưu chuỗi sự kiện thay vì chỉ lưu trạng thái hiện tại*

Kleppmann trong [7, Ch.11] giải thích: Event Sourcing coi event log là **nguồn sự thật** (*source of truth*), còn các khung nhìn trạng thái (bảng cơ sở dữ liệu, materialized view) là **derived data** — có thể dựng lại bất kỳ lúc nào bằng cách replay event.

Lịch sử biến động và bảng điểm tổng kết

Các kỹ sư phần mềm đã quen thuộc với mô hình CRUD và câu lệnh `UPDATE` thường gặp khó khăn khi làm quen với Event Sourcing. Hãy nghĩ đơn giản:

- **Event Store**: giống như **lịch sử biến động điểm/nhật ký chấm bài** (ghi nhận mọi giao dịch cộng điểm, trừ điểm). Không thể dùng bút xóa sửa lại quá khứ, chỉ có thể ghi thêm sự kiện mới. Nhưng việc lấy nhật ký chấm bài ra để cộng lại xem hiện tại sinh viên đang có bao nhiêu điểm là rất chậm. Hơn nữa, truy vấn kiểu `SELECT * FROM nhat_ky WHERE so_diem_hien_tai > 10` không thực hiện được ngay trên nhật ký — `so_diem_hien_tai` không nằm trong bất kỳ dòng nhật ký nào mà là kết quả *cộng dồn* của cả chuỗi sự kiện (điều này đúng kể cả khi event store được cài trên chính một relational database).
- **Projection (Materialized View)**: giống như **bảng điểm tổng kết**. Hệ thống tổng hợp toàn bộ dữ liệu sự kiện trong quá khứ, tính toán và xuất ra kết quả "Điểm hiện tại". Khi UI cần hiển thị, nó chỉ cần đọc bảng điểm này. Trong thực tế, projection gần như luôn cần thiết cho các truy vấn *xuyên aggregate* và cho UI; còn để dựng trạng thái của *một* thực thể, hệ thống chỉ cần replay stream sự kiện của chính thực thể đó.

Bảng 7.7 tóm tắt ưu và nhược điểm của Event Sourcing.

Bảng 7.7: *Ưu và nhược điểm của Event Sourcing*

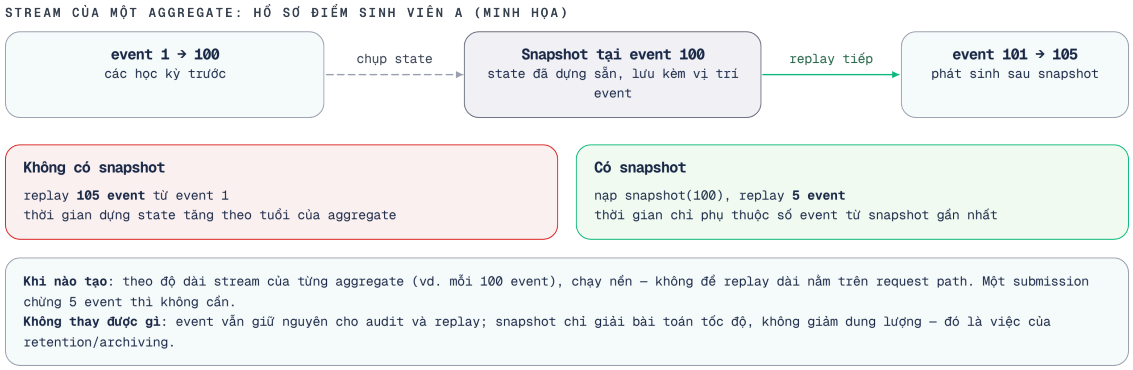
Ưu điểm	Nhược điểm
Full audit trail — mọi thay đổi đều được ghi	**Độ phức tạp** — rebuild state cần replay, vấn đề về hiệu năng

Ưu điểm	Nhược điểm
Truy vấn theo thời gian (temporal query) — "state tại thời điểm T?"	**Event schema evolution** — event cũ có format khác event mới
Hỗ trợ gỡ lỗi (debug) thuận tiện — có thể replay các sự kiện để tái hiện lỗi	**Eventual consistency** — khi projection cập nhật bất đồng bộ, read model "chậm hơn" event stream và cần xử lý khoảng trễ
Replay — rebuild state, tạo view mới	**Dung lượng lưu trữ** — event log tăng theo thời gian, cần retention/ archiving phù hợp (snapshot chỉ giải bài toán tốc độ replay)
Kết hợp CQRS — event là nguồn cấp tự nhiên cho read model	**Khó tiếp cận** — đòi hỏi sự thay đổi tư duy hoàn toàn so với CRUD

Tuy nhiên, hệ thống Event Sourcing không thể replay toàn bộ sự kiện mỗi khi cần truy vấn trạng thái hiện tại. Điều này tương tự như việc phòng giáo vụ không thể lật lại từng bài kiểm tra từ năm nhất chỉ để tính điểm tích lũy. Giải pháp ở đây là tạo ra các **snapshot** — bản chụp trạng thái tại một mốc sự kiện.

7.5.3. Snapshot pattern — giải quyết vấn đề hiệu năng của event replay

Khi stream của *một aggregate* dài ra (ví dụ hồ sơ điểm của một sinh viên tích lũy hàng nghìn event sau nhiều học kỳ), thời gian dựng lại trạng thái bằng cách replay từ event đầu tiên sẽ **tăng dần theo tuổi đời của aggregate**. **Snapshot pattern** giải quyết vấn đề này:



Hình 7.12: *Snapshot pattern — replay từ snapshot thay vì từ đầu*

Chiến lược tạo snapshot trong Event Sourcing áp dụng ở cấp độ **từng aggregate**, ví dụ cứ sau 100 event của một aggregate thì tạo một bản snapshot mới (**count-based**).

⚠ Snapshot trong production

Tiêu chí quyết định snapshot là **độ dài stream của từng aggregate**, không phải tổng số event toàn hệ thống. Một bài nộp chỉ có khoảng 5 event thì không bao giờ cần snapshot; nhưng một aggregate sống lâu (ví dụ hồ sơ điểm tích lũy của một sinh viên qua nhiều học kỳ) sẽ khiến mỗi lần dựng lại trạng thái chậm dần theo thời gian. Hãy quyết định dựa trên thời gian dựng lại trạng thái (rehydration) đo được so với SLO — và lưu ý snapshot không giải quyết bài toán dung lượng lưu trữ; đó là việc của retention/archiving.

Điều cần tránh tuyệt đối là để việc replay không giới hạn nằm trên *request path*: chờ người dùng gọi API GET mới bắt đầu replay hàng trăm nghìn event sẽ làm độ trễ tăng vọt. Ngoài nguyên tắc đó, count-based (mỗi N event của một aggregate), time-based (theo lịch, ngoài giờ cao điểm — cách LMAX từng dùng) hay on-demand có kiểm soát đều là lựa chọn hợp lệ — chọn theo độ

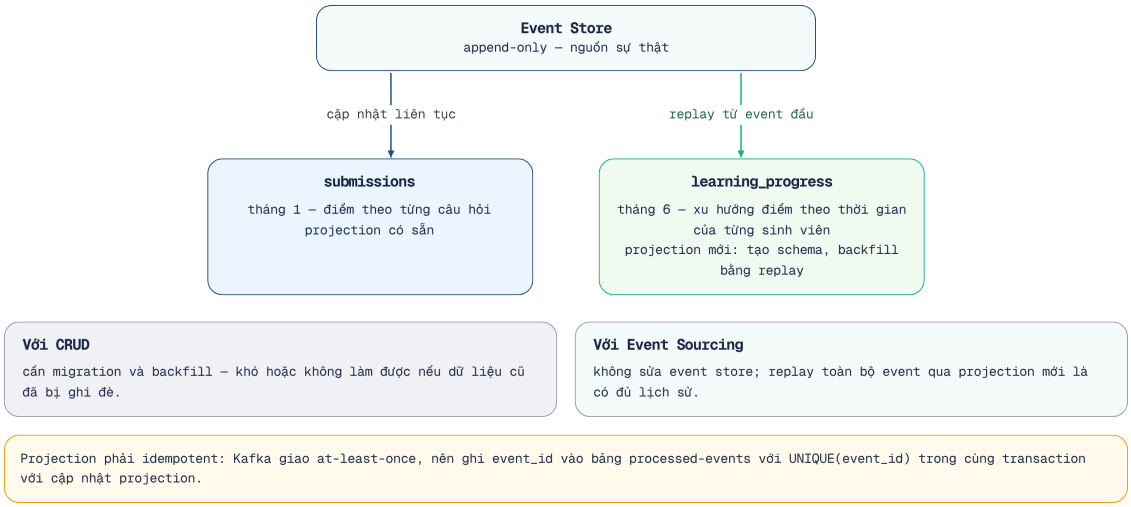
dài stream, tốc độ ghi và SLO cho thời gian dựng lại trạng thái. Với KBM, count-based chạy nền theo từng aggregate là điểm khởi đầu đơn giản và an toàn nhất.

7.5.4. Projection Rebuilds — tạo lại view từ event

Một trong những lợi ích lớn nhất của Event Sourcing: **tạo view mới mà không phải sửa event store hay suy diễn lại lịch sử từ current state** (read model mới vẫn cần được tạo schema rồi backfill bằng replay). Ví dụ giả định với KBM:

1. Tháng 1: chỉ cần bảng `submissions` (điểm theo câu hỏi)
2. Tháng 6: cần thêm bảng `learning_progress` (xu hướng điểm theo thời gian của từng sinh viên)
3. Với CRUD: cần database migration + backfill dữ liệu (khó hoặc không thể nếu dữ liệu cũ đã bị ghi đè)
4. Với Event Sourcing: hệ thống chỉ cần replay toàn bộ sự kiện qua một projection mới, từ đó bảng `learning_progress` sẽ tự động được dựng lại cùng với toàn bộ dữ liệu lịch sử

Hình 7.13 minh họa kịch bản trên: projection mới được dựng lại bằng cách replay event store.



Hình 7.13: *Projection Rebuilds — dựng read model mới bằng replay, không sửa event store*

Khi CQRS tiêu thụ event để dựng read model (projection), cần đặc biệt chú ý đến **idempotency**: message broker (như Kafka) thường chỉ đảm bảo *at-least-once delivery*, nghĩa là một event có thể bị gửi lặp lại. Do đó, hàm cập nhật projection bắt buộc phải có tính idempotent. Cách làm chắc chắn là dùng một bảng inbox/processed-events với ràng buộc `UNIQUE(event_id)`, được ghi *trong cùng transaction* với cập nhật projection — event trùng sẽ vi phạm UNIQUE và bị bỏ qua, tránh cộng dồn sai lệch. Nếu cần chống cả event *sai thứ tự*, so sánh thêm version/sequence của aggregate; đừng dựa vào một mốc `last_processed_event_id` duy nhất, vì `eventId` là định danh duy nhất nhưng không mang thứ tự.

Bảng 7.8: *Event Store — các công cụ phổ biến*

Event Store	Mô tả	Phù hợp khi
KurrentDB (tên cũ EventStoreDB)	Database chuyên dụng cho Event Sourcing, do Greg Young khởi xướng	Cần đủ tính năng Event Sourcing: subscription, projection, temporal query

Event Store	Mô tả	Phù hợp khi
Axon Framework	Framework Java cho CQRS + Event Sourcing, tích hợp Spring	Nhóm Java/Spring, cần framework có sẵn quy ước (opinionated)
Kafka (kèm lưu ý)	Message broker với retention vô hạn	Đã dùng Kafka, chỉ cần event streaming (không cần truy vấn theo aggregate)
PostgreSQL + custom	Relational DB với bảng sự kiện	Nhóm phát triển quy mô nhỏ, ưu tiên sự đơn giản và muốn sử dụng hệ quản trị cơ sở dữ liệu sẵn có

Trong các lựa chọn trên, Kafka thường bị nhầm lẫn với một event store chuyên dụng.

Kafka ≈ Event Store?

Kafka với retention vô hạn có thể hoạt động như event store. KBM đã dùng Kafka cho pipeline xử lý bài nộp (Chương 5). Về lý thuyết, các thông điệp trên topic `submissions` và `judge-results` *chính là* chuỗi event. Tuy nhiên, Kafka được thiết kế là message broker, không phải event store chuyên dụng — truy vấn event theo entity ID khó hơn so với KurrentDB hoặc Axon. Kleppmann trong [7, Ch.11] lưu ý rằng Event Sourcing không gắn với công cụ cụ thể: cơ sở dữ liệu thông thường hay log-based message broker đều dùng được; event store chuyên dụng chỉ làm việc truy vấn event theo aggregate dễ hơn.

7.5.5. Tiến hóa schema của event (event schema evolution)

Event, giống API (Chương 3), cần **schema evolution** khi nghiệp vụ thay đổi. Nhưng khác API: các event đã lưu *không thể sửa* — event store là append-only. Hai chiến lược:

1. Upcasting — Khi đọc các event cũ, chuyển đổi sang định dạng mới trước khi áp dụng:

Listing 7.1: *Upcasting — chuyển event cũ sang định dạng mới*

```
// Event v1: { type: "SubmissionCreated", userId: "123", sql: "SELECT ..." }
// Event v2: { type: "SubmissionCreated", userId: "123", queryStr: "SELECT ..." }
// Upcaster: xử lý Breaking Change: đổi tên field 'sql' thành 'queryStr'
```

2. Weak schema — Event chứa các trường bổ sung mà consumer bỏ qua nếu không hiểu (Tolerant Reader — Chương 3). Reader cũ bỏ qua trường mới; muốn reader mới đọc được event cũ, trường mới phải là tùy chọn, có giá trị mặc định, hoặc được bổ sung bằng upcaster.

Event là điều đã xảy ra, không phải lệnh

Event mô tả *điều đã xảy ra* (past tense: `SubmissionJudged`), không phải *yêu cầu* (`JudgeSubmission`). Event có tính bất biến (immutable) — không bao giờ sửa/xóa event đã lưu. Nếu cần "undo", thêm event mới: `SubmissionResultCorrected`. Tên event là *quyết định nghiệp vụ*, không phải *quyết định kỹ thuật* — chuyên gia nghiệp vụ nên đặt tên cho các event, như trong event storming [2a, Ch.5].

7.5.6. Khi nào dùng Event Sourcing?

Bảng 7.9 đánh giá mức phù hợp của Event Sourcing theo từng kịch bản.

Bảng 7.9: *Khi nào dùng Event Sourcing*

Kịch bản	Phù hợp?	Lý do
Yêu cầu kiểm toán (tài chính, tuân thủ)	✓ Rất phù hợp	Cần lịch sử đầy đủ, không thể xóa

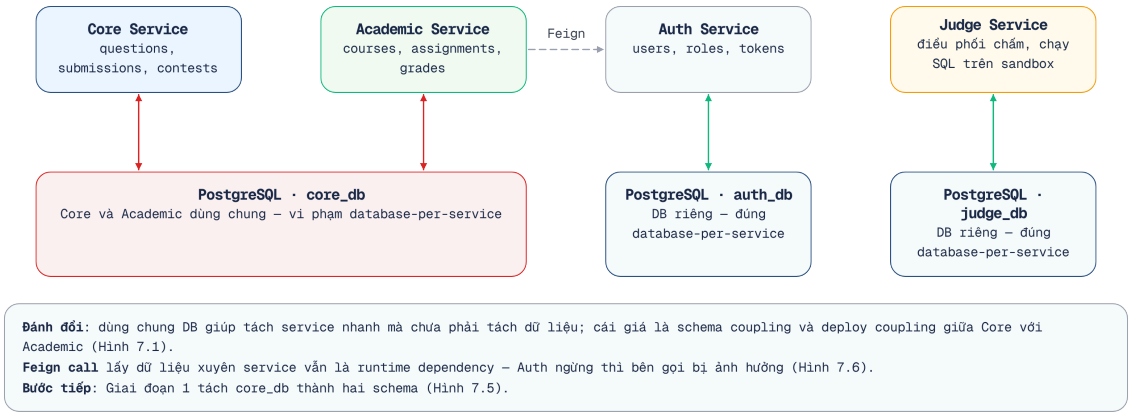
Kịch bản	Phù hợp?	Lý do
Temporal analysis (phân tích xu hướng)	✓ Phù hợp	Hỗ trợ tự nhiên truy vấn "trạng thái tại thời điểm X"
CRUD đơn giản (quản lý user, cấu hình)	× Không	Chi phí vận hành và độ phức tạp phát sinh quá lớn so với lợi ích mang lại
Domain phức tạp (đơn hàng, đặt chỗ)	✓ Phù hợp	Sự kiện nghiệp vụ ánh xạ trực tiếp vào domain event
Theo dõi bài nộp trong KBM	! Tùy	Có giá trị cho phân tích học tập, nhưng hiện tại nhóm phát triển nhỏ → không ưu tiên

7.6. Case study: KBM

Phần này áp dụng các nguyên tắc quản lý dữ liệu phân tán đã trình bày vào KBM: nhận diện kiến trúc dữ liệu hiện tại, chỉ ra khoảng trống so với giải pháp khuyến nghị, và đề xuất lộ trình chuyển đổi theo giai đoạn.

7.6.1. Hiện trạng

Phân tích mã nguồn KBM cho thấy kiến trúc dữ liệu hiện tại:



Hình 7.14: Kiến trúc dữ liệu hiện tại của KBM — shared database là nợ kỹ thuật chuyển tiếp

Quan sát chính:

1. **Auth Service** — đã tách database riêng (`auth_db`) → ✓ đúng database-per-service
2. **Core + Academic** — chia sẻ `core_db` → ! nợ kỹ thuật chuyển tiếp có điều kiện (cần xác lập ranh giới sở hữu bảng trước khi tách)
3. **Judge Service** — database riêng (`judge_db`: dữ liệu test case, bảng đã tạo trong sandbox, bài nộp phía Judge Service) → ✓ đúng database-per-service
4. **Cross-service data** — dùng Feign call → ! runtime dependency

Phân tích query pattern: KBM sử dụng hai pattern khi truy vấn dữ liệu vượt ra ngoài một entity:

- **Interface Projections** (Spring Data JPA) — tối ưu truy vấn *nội bộ* (kể cả cross-schema trên shared database hiện tại): trả về lightweight projection thay vì full entity, giảm lượng dữ liệu truyền tải. Đây không phải cơ chế vượt ranh giới dịch vụ.
- **Feign call** — pattern cross-service thực sự: gọi API của service khác khi cần dữ liệu ngoài ranh giới.

7.6.2. Từ hiện trạng đến kiến trúc mục tiêu

Bảng 7.10 tổng hợp các vấn đề dữ liệu của KBM và chiến lược chuyển đổi cho từng vấn đề.

__Bảng 7.10:__ Tổng hợp vấn đề quản lý dữ liệu trong KBM và chiến lược chuyển đổi

#	Vấn đề	Hiện trạng KBM	Chiến lược chuyển đổi
1	**Shared database**	Core + Academic cùng `core_db`	Separate schema + phân quyền role (tách vật lý khi cần mở rộng)
2	**Cross-service join**	Cross-table query trực tiếp	Thay bằng Feign hoặc bản sao qua event tùy luồng
3	**Leaderboard query**	Truy vấn tổng hợp trên `core_db` kết hợp lấy thông tin người dùng qua API	CQRS — pre-computed read model
4	**Lịch sử bài nộp**	CRUD (chỉ lưu current state)	(Tùy chọn, chưa ưu tiên) Event Sourcing hoặc bảng lịch sử trạng thái cho phân tích
5	**User reference data**	Feign call mỗi khi cần	Bản sao cục bộ qua event
6	**Grade Scheme / Attendance / MCQ Daily / CLO**	Nhiều read model học vụ cần tổng hợp dữ liệu khóa học, lớp, bài làm, chuẩn đầu ra	Xác định chủ sở hữu rõ ràng, nhân bản chỉ đọc qua event hoặc materialized view
7	**Schema migration không đồng nhất**	Các service Java thiên về JPA auto-DDL; các service Go trong DevOps Lab phù hợp hơn với golang-migrate	Chuẩn hóa migration bằng versioned script, không dựa vào auto-DDL ở production

7.6.3. Lộ trình chuyển đổi đề xuất

Lộ trình dưới đây mở rộng 3 bước tách database ở Bảng 7.4 thành kế hoạch quản lý dữ liệu đầy đủ: hai giai đoạn đầu dựng ranh giới schema và chọn đường truy cập dữ liệu theo bài toán; Full Split (bước 3 của bảng) không nằm trong lộ trình ngắn hạn và chỉ thực hiện khi phát sinh nhu cầu mở rộng độc lập; các giai đoạn 3–5 bổ sung phần nhân bản dữ liệu tham chiếu, bảng xếp hạng và read model học vụ.

Giai đoạn 1 — Kiểm kê sở hữu và tách schema (ưu tiên cao, chi phí thực hiện thấp): Kiểm kê toàn bộ bảng trong `core_db`, khóa một chủ sở hữu ghi duy nhất cho từng bảng và tắt `JPA auto-DDL` cạnh tranh (chỉ một pipeline migration quản lý schema của mỗi dịch vụ). Sau đó tách `core_db` thành schema `core_schema` và `academic_schema`, mỗi service một database role riêng. Truy vấn chéo còn lại chỉ được đọc qua các view được cấp quyền rõ ràng (`GRANT SELECT` trên view, không trên bảng) — danh sách view này chính là danh sách việc của bước kế tiếp. Chưa thu hồi quyền đọc chéo ở bước này để không làm hỏng chức năng đang chạy.

Giai đoạn 2 — Truy cập dữ liệu qua API cho luồng cần dữ liệu tức thời (chi phí thực hiện trung bình): Với các thao tác ghi hoặc luồng đọc ít nhưng đòi hỏi dữ liệu mới tức thời (ví dụ kiểm tra điều kiện nộp bài, trạng thái bài thi), thay truy vấn xuyên schema bằng Feign call — Academic gọi API của Core (như `GET /api/questions/batch`) thay vì đọc trực tiếp `core_schema.questions`. Khi các view đọc chéo lần lượt được thay bằng API (hoặc bản sao qua sự kiện ở Giai đoạn 3) và không còn ai dùng, gỡ view và chặn cứng ranh giới bằng `REVOKE ALL ON SCHEMA core_schema FROM academic_role;`.

Giai đoạn 3 — Nhân bản dữ liệu tham chiếu qua sự kiện (chi phí thực hiện trung bình): Thay vì gọi API đồng bộ lặp đi lặp lại trên mỗi lần tải trang, với dữ liệu tham chiếu đọc nhiều và ít thay đổi, Core/Auth phát sự kiện (`QuestionUpdated`, `UserUpdated`) khi dữ liệu gốc đổi; Academic tiêu thụ và lưu bản sao cục bộ chỉ đọc (tiêu đề, độ khó câu hỏi, họ tên người dùng) để giảm runtime dependency.

Lưu ý về khởi tạo dữ liệu ban đầu (bootstrapping): Khi áp dụng Data Duplication qua sự kiện, cần thiết kế thêm cơ chế "Bulk Export/Sync" hoặc "Reconciliation" (đối soát định kỳ) để khởi tạo

dữ liệu ban đầu khi triển khai hoặc dựng môi trường mới — nếu không, service tiêu thụ sẽ khởi động với bản sao rỗng và chỉ có dữ liệu từ thời điểm bật event trở đi. Việc phát sự kiện cũng phải đảm bảo tính nguyên tử (dùng Transactional Outbox Pattern ở Chương 10) để đề phòng lỗi mạng.

Giai đoạn 4 — Leaderboard thời gian thực với Redis (chi phí thực hiện cao, giá trị lớn cho chế độ thi): Xây dựng bảng xếp hạng thành read model trong bộ nhớ bằng Redis Sorted Set ngay tại `kbm-core` (hoặc `kbm-exam` khi tách phân hệ thi), cập nhật đồng bộ với **Composite Score** trên đường ghi chính kết hợp Transactional Outbox bền vững, đồng thời ghép họ tên sinh viên từ bản sao tham chiếu `UserUpdated`. Truy vấn `ZREVRANK` chạy in-memory với độ phức tạp `O(log N)` — đủ nhanh cho hiển thị thời gian thực. Khi fast path thành công, người nộp bài đọc lại được ngay điểm vừa ghi (read-after-write trên primary); khi fast path lỗi, bản relay từ outbox vẫn cập nhật bất đồng bộ — tức độ trễ chỉ được loại bỏ trên đường ghi chính, không phải "tuyệt đối".

Giai đoạn 5 — Read model cho phân tích học tập (khi mở rộng học vụ): Grade Scheme, Attendance, MCQ Daily và CLO nên được xem như các read model học vụ. Chủ sở hữu dữ liệu gốc vẫn nằm trong Bounded Context tương ứng; bảng tổng hợp chỉ phục vụ truy vấn nhanh cho giảng viên và báo cáo, không trở thành nguồn sự thật thứ hai.

⚠ Sai lầm thường gặp khi quản lý dữ liệu phân tán

- **Chia database quá sớm**: Tách database trước khi hiểu rõ các mẫu truy cập dữ liệu (data access pattern). Hậu quả: phát hiện hai service cần JOIN dữ liệu liên tục, phải xây dựng event pipeline phức tạp cho một tác vụ mà trước đây chỉ cần một câu lệnh SQL JOIN đơn giản. *Phòng tránh*: Bắt đầu với separate schema (cùng DB server), theo dõi các truy vấn xuyên schema trong 2–4 tuần, rồi mới quyết định tách hoàn toàn.
- **Dùng CQRS cho mọi thứ**: Áp dụng CQRS/Event Sourcing cho các chức năng CRUD cơ bản. Hậu quả: độ phức tạp của hệ thống tăng vọt, đội ngũ phát triển lãng phí thời gian bảo trì luồng sự kiện cho những dữ liệu rất ít khi thay đổi. *Phòng tránh*: Chỉ áp dụng CQRS cho các luồng truy vấn phức tạp (yêu cầu nối dữ liệu xuyên service, lưu lượng đọc lớn, cần giao diện theo thời gian thực). Kiến trúc CRUD cơ bản đã đáp ứng đủ cho phần lớn các bài toán thông thường.
- **Nhân bản dữ liệu nhưng không xác định source of truth**: Sao chép dữ liệu giữa các service mà không rõ "ai sở hữu dữ liệu này?". Hậu quả: hai service cùng sửa dữ liệu → xung đột, không biết bản nào đúng. *Phòng tránh*: Mỗi thực thể dữ liệu chỉ có **duy nhất một service sở hữu** — các bản sao khác là bản sao chỉ đọc (read-only replica), chỉ cập nhật qua event từ service sở hữu.
- **Lạm dụng eventual consistency cho giao diện theo thời gian thực**: Áp dụng CQRS bất đồng bộ (như qua Kafka) cho tính năng cần cập nhật gần như tức thời như bảng xếp hạng trực tiếp. Hậu quả: người dùng nộp bài thành công nhưng bảng xếp hạng chưa cập nhật; Optimistic UI không thể áp dụng ở đây vì client không thể tự tính thứ hạng khi nó phụ thuộc vào điểm số của những người dùng khác. *Phòng tránh*: Với các bài toán yêu cầu cập nhật bảng xếp hạng theo thời gian thực (như trong chế độ thi), hãy dùng cấu trúc dữ liệu trong bộ nhớ cập nhật đồng bộ trên primary (như **Redis Sorted Set** — cho read-after-write ngay trên đường ghi chính) thay vì nhân bản bất đồng bộ qua event.

Các pattern trong chương có minh họa tương tác đi kèm tại companion web:

Interactive Demo

- **Mô hình CQRS và Event Sourcing**: cqrs-event-sourcing.html
- **Event-Carried State Transfer (nhân bản dữ liệu qua event)**: event-carried-state-transfer.html

7.7. Tổng kết

Quản lý dữ liệu là một trong những bài toán phức tạp nhất khi chuyển từ monolith sang microservices — phức tạp hơn cả việc tách mã ứng dụng. Database-per-service không chỉ là nguyên tắc kỹ thuật mà là **điều kiện tiên quyết** cho khả năng triển khai độc lập — mục tiêu cốt lõi của microservices. CAP Theorem (cùng mở rộng PACELC) nhắc nhở: trong hệ thống phân tán, tính nhất quán, tính sẵn sàng và độ trễ luôn đòi hỏi sự đánh đổi có chủ đích theo từng luồng dữ liệu.

Năm chiến lược tách database (view → wrapping service → separate schema → data transfer → full split) cho phép chuyển đổi **tăng dần** — không cần "big bang". Với nhóm phát triển nhỏ như KBM, kiểm kê chủ sở hữu bảng và bắt đầu từ separate schema là bước đi thực tế nhất.

Data duplication không phải anti-pattern — đây là **trade-off có chủ đích** để giảm runtime coupling. Nguyên tắc: mỗi tập dữ liệu chỉ có một source of truth, các bản sao chỉ phục vụ đọc và được cập nhật qua event. Lập luận này suy rộng từ nhận định của Newman ở tầng mã nguồn: tác hại của coupling quá chặt lớn hơn nhiều so với vấn đề do lặp mã gây ra.

CQRS (mở rộng từ nguyên tắc CQS của Meyer) tách read model và write model — giải quyết bài toán truy vấn xuyên service và lưu lượng đọc lớn. Event Sourcing bổ sung bằng cách lưu lịch sử đầy đủ thay vì chỉ trạng thái hiện tại. Cả hai pattern này mang lại nhiều lợi ích nhưng rất phức tạp — chỉ áp dụng khi có nhu cầu cụ thể, không phải mặc định.

Một bài toán liên quan chặt chẽ là **giao dịch phân tán (distributed transaction)** — khi một nghiệp vụ cần thay đổi dữ liệu ở nhiều service. Saga pattern (đã thảo luận chi tiết ở Chương 6) là giải pháp: chuỗi local transaction + compensating action thay vì distributed ACID transaction. Saga và quản lý dữ liệu bổ trợ nhau: database-per-service tạo ra nhu cầu Saga; còn Saga có thể dùng chung hạ tầng domain-event/outbox với các projection CQRS hay hệ Event Sourcing — nhưng không phụ thuộc vào hai pattern đó.

Phân tích KBM cho thấy nợ kỹ thuật lớn nhất ở tầng dữ liệu là shared database giữa Core và Academic. Lộ trình chuyển đổi đi theo từng giai đoạn: khóa sở hữu và tách schema → chọn API hoặc bản sao qua sự kiện tùy bài toán → CQRS với Redis Sorted Set cho bảng xếp hạng trực tiếp và read model cho phân tích học tập. Mỗi giai đoạn độc lập, có thể dừng ở bất kỳ điểm nào khi đã đáp ứng đủ nhu cầu của đội ngũ.

Ở Chương 8, chúng ta sẽ chuyển sang **API Gateway** — điểm vào tập trung cho hệ thống microservices: định tuyến (routing), xác thực, giới hạn lưu lượng (rate limiting), và cách KBM sử dụng Spring Cloud Gateway.

7.8. Đọc thêm

Sách tham khảo chính:

1. [4b] Sam Newman, *Monolith to Microservices* — Ch.4: Decomposing the Database, Database-per-service patterns
2. [2a] Chris Richardson, *Microservices Patterns*, 1st Ed. — Ch.6: Event Sourcing; Ch.7: Implementing Queries (API Composition, CQRS)
3. [7] Martin Kleppmann, *Designing Data-Intensive Applications* — Ch.5: Replication; Ch.6: Partitioning; Ch.7,9: Transactions & CAP; Ch.11: Stream Processing, Event Sourcing

Sách bổ trợ:

4. [3] Ronnie Mitra & Irakli Nadareishvili, *Microservices: Up and Running* — Ch.5: Data Delegate Pattern, Event Sourcing in practice
5. [5] Hugo Rocha, *Practical Event-Driven Microservices Architecture* — §4.5: CQS vs CQRS, Command Sourcing; Ch.5: CAP in real world, eventual consistency
6. [6] Eric Evans, *Domain-Driven Design* — Aggregate design → data ownership per Bounded Context

Chương liên quan:

- **Chương 6 — Saga Pattern**: Giao dịch phân tán, orchestration vs choreography, compensating action. Saga giải quyết bài toán "thay đổi dữ liệu ở nhiều service trong một giao dịch nghiệp vụ" — bổ trợ trực tiếp cho database-per-service.

Nguồn trực tuyến:

- Martin Fowler, "CQRS" — martinfowler.com/bliki/CQRS.html
- Greg Young, "CQRS Documents" — cqrs.files.wordpress.com/2010/11/cqrs_documents.pdf
- Confluent, "Event Sourcing with Kafka" — developer.confluent.io

Bài tập: 7-0, 7-1, 7-2 ở phần Bài tập & case study cuối sách.

PHẦN III

Hạ tầng & vận hành

Phần III đưa hệ thống vào vận hành: API Gateway, bảo mật, chuyển đổi từ monolith, observability và triển khai. Chương 8–9 và 11–12 có thể đọc độc lập; Chương 10 nên đọc sau Phần I–II vì áp dụng lại các pattern đã học vào một hệ thống đang chạy.

Chương 8 API Gateway & BFF Pattern
Chương 9 Bảo mật Microservices — OAuth 2.0, mTLS & Zero Trust
Chương 10 Chuyển đổi thực tế — từ Monolith đến Microservices
Chương 11 Quan sát hệ thống — Observability & Distributed Tracing
Chương 12 Triển khai & DevOps

CHƯƠNG 8

API Gateway & BFF Pattern

> "A much better approach is for a client to make a single request to an API gateway, a service that serves as the single entry point for API requests into an application from outside the firewall."
> — Chris Richardson, *Microservices Patterns* [2a, Ch.8]

MỤC TIÊU HỌC TẬP

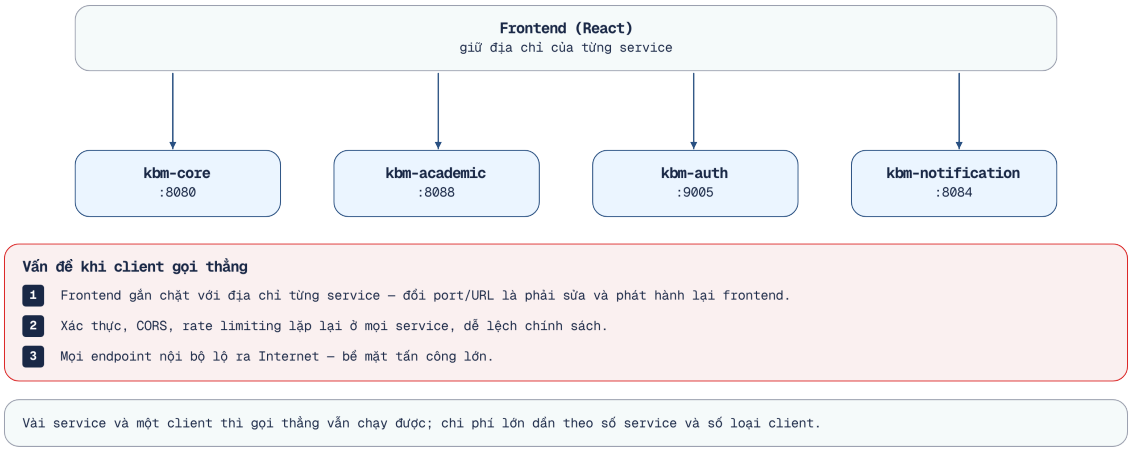
- Hiểu tại sao microservices cần API Gateway và các vấn đề nó giải quyết
- Nắm vững API Gateway pattern và Backend for Frontend (BFF) pattern
- Sử dụng Spring Cloud Gateway (WebFlux) để triển khai gateway
- Cấu hình route với Eureka service discovery (load-balanced URI)
- Thiết kế nghiệp vụ cắt ngang (cross-cutting concern) tại Gateway: xác thực, CORS, rate limiting
- Phân tích kiến trúc gateway trong KBM

8.1. Động lực sử dụng API Gateway

Trong một khuôn viên trường đại học rộng lớn, nếu không có một cổng thông tin duy nhất, việc tìm kiếm và truy cập các dịch vụ sẽ rối rắm. Hệ thống microservices cũng đối mặt với một thách thức tương tự ở phía client.

8.1.1. Vấn đề: client giao tiếp trực tiếp với nhiều service

Khi không có Gateway, client (web, mobile) phải biết địa chỉ của *từng* microservice và gọi trực tiếp. Với KBM gồm nhiều service và module lab, mỗi trang web có thể cần gọi nhiều API khác nhau:



Hình 8.1: *Không có Gateway — client phải biết địa chỉ của từng service*

Việc bắt client (đặc biệt là giao diện người dùng) phải tự ghi nhớ và gọi đến từng service riêng lẻ chẳng khác nào yêu cầu tân sinh viên tự cầm bản đồ đi tìm từng phòng ban để hoàn tất thủ tục nhập học thay vì chỉ đến bộ phận một cửa duy nhất.

Cổng bảo vệ trường đại học

Hãy tưởng tượng hệ thống microservices như một khuôn viên trường đại học với hàng chục tòa nhà (Khoa CNTT, Thư viện, Ký túc xá). Nếu không có cổng chính (API Gateway), một sinh viên sẽ phải tự tìm kiếm và di chuyển đến từng tòa nhà, gõ cửa hỏi đường, và tòa nhà nào cũng phải bố trí bảo vệ riêng để kiểm tra thẻ. **API Gateway** hoạt động như bộ phận một cửa của trường đại học. Sinh viên chỉ cần đến một địa chỉ duy nhất (một URL), nhân viên trực sẽ kiểm tra thẻ sinh viên (authentication) và chuyển yêu cầu đến đúng phòng ban chức năng (routing). Tuy nhiên, phòng ban đích vẫn phải kiểm tra con dấu hoặc giấy giới thiệu theo mức độ rủi ro, rồi tự quyết định sinh viên có quyền thực hiện nghiệp vụ cụ thể hay không (phân quyền, authorization).

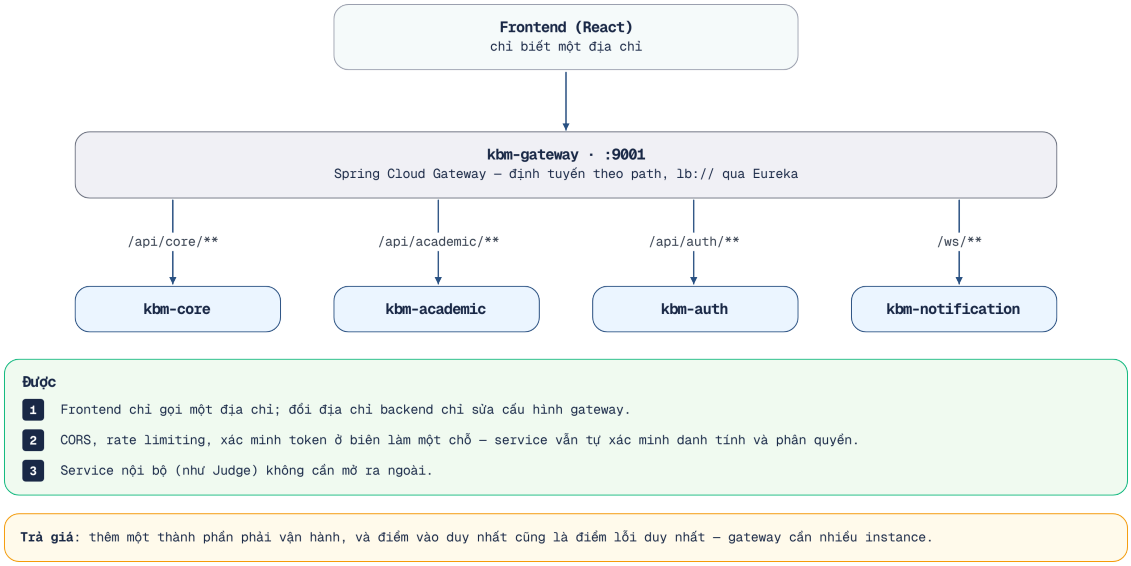
Việc ứng dụng client gọi trực tiếp đến từng microservice gây ra năm vấn đề. Richardson trong [2a, Ch.8] nêu các hạn chế về số lượng request, tính đóng gói và giao thức; phần còn lại là các vấn đề vận hành thường gặp:

1. **Nhiều endpoint:** Client phải quản lý và ghi nhớ URL của mọi dịch vụ riêng lẻ, dẫn đến sự liên kết chặt chẽ (tight coupling) giữa frontend và backend.
2. **Giao thức khác nhau:** Các dịch vụ có thể sử dụng đa dạng giao thức (REST, gRPC, WebSocket), buộc frontend phải trang bị nhiều thư viện hỗ trợ tương ứng, làm tăng độ phức tạp của mã nguồn.

3. **Nghiệp vụ cắt ngang phân tán:** Mỗi dịch vụ phải tự triển khai các nghiệp vụ cắt ngang như xác thực, CORS và rate limiting, gây ra sự trùng lặp và nguy cơ thiếu nhất quán trong các chính sách hệ thống.
4. **Mạng không an toàn:** Việc phơi bày trực tiếp toàn bộ API endpoint nội bộ ra môi trường Internet làm gia tăng bề mặt tấn công (attack surface).
5. **API không phù hợp:** Các giao diện lập trình nội bộ thường được thiết kế cho quá trình tương tác giữa máy chủ với máy chủ, trả về lượng dữ liệu dư thừa hoặc yêu cầu quá nhiều vòng gọi mạng, trở thành điểm nghẽn đối với các nền tảng di động có băng thông hạn hẹp.

8.1.2. API Gateway pattern

API Gateway là **điểm vào duy nhất** (single entry point) — toàn bộ yêu cầu (request) từ client sẽ đi qua Gateway và được định tuyến (route) đến đúng dịch vụ tương ứng:



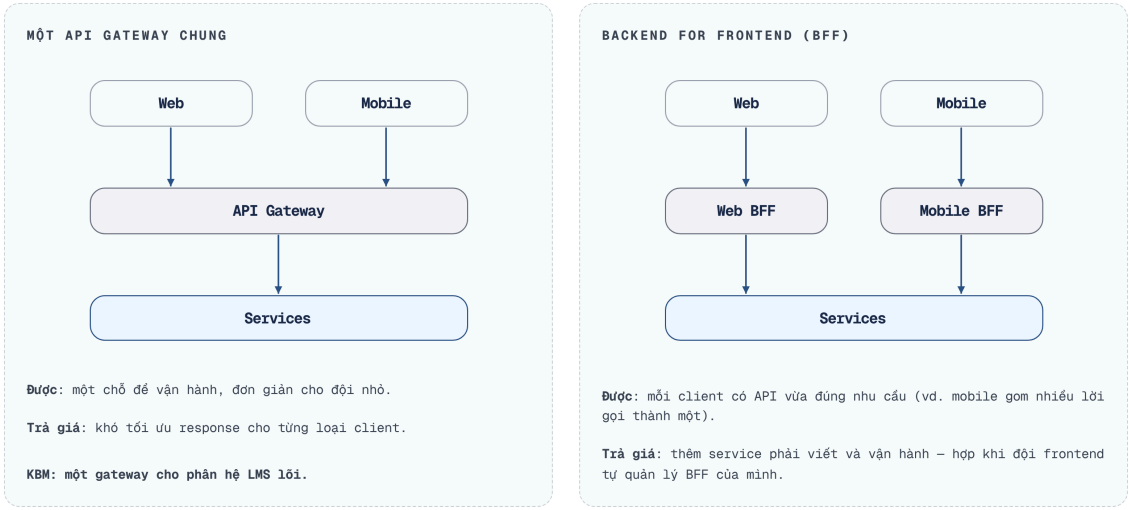
Hình 8.2: *API Gateway — điểm vào duy nhất, định tuyến đến từng service*

Gateway xử lý tập trung các nghiệp vụ cắt ngang (**cross-cutting concern**), bao gồm: xác thực ở biên (edge authentication), CORS, giới hạn lưu lượng (rate limiting), logging và kết thúc SSL (SSL termination). Các dịch vụ phía sau không cần lặp lại CORS hay rate limiting, nhưng vẫn phải xác minh bằng chứng danh tính theo tầng tin cậy (trust tier) và thực hiện phân quyền cục bộ (local authorization) cho tài nguyên mình sở hữu.

Gateway thường được ví như một "smart edge" (điểm biên thông minh) chuyên xử lý nghiệp vụ cắt ngang. Tuy nhiên, theo đúng tinh thần của nguyên lý **"Smart endpoints and dumb pipes"** do James Lewis và Martin Fowler nêu ra (2014), Gateway không nên chứa business logic. Đối với dữ liệu nghiệp vụ, Gateway vẫn phải duy trì vai trò của một "dumb pipe" (chỉ định tuyến cơ bản) để tránh lặp lại vấn đề của ESB.

8.1.3. API Gateway và BFF (Backend for Frontend)

Richardson trong [2a, Ch.8] phân biệt hai biến thể:



Hình 8.3: API Gateway (một Gateway chung) và BFF (Gateway riêng cho từng client)

Biến thể phổ biến nhất là **API Gateway**, nơi một Gateway duy nhất phục vụ mọi loại client. Phương pháp này đặc biệt phù hợp với các đội ngũ nhỏ hoặc khi các ứng dụng client (như web admin và web sinh viên) có nhu cầu truy xuất dữ liệu tương đồng nhau. Tuy nhiên, khi hệ thống mở rộng, biến thể **BFF (Backend for Frontend)** phù hợp hơn: mỗi loại client có một Gateway riêng. BFF thường cần thiết khi ứng dụng di động (mobile app) đòi hỏi định dạng API khác biệt so với nền tảng web, ví dụ như cần gộp nhiều lời gọi (batching) và tối giản hóa dữ liệu trả về để tiết kiệm băng thông.

KBM sử dụng **một API Gateway duy nhất** cho LMS core — điều này hoàn toàn phù hợp vì các web frontend chính đều có chung những yêu cầu API tương tự nhau. Với DevOps Lab, KBM bổ sung một lớp router riêng theo hostname để điều hướng workspace/lab runtime; đây là gateway chuyên biệt cho một Bounded Context, không thay thế Spring Cloud Gateway của LMS core.

Khi nào cần BFF? BFF trở nên cần thiết khi các client khác nhau có nhu cầu API hoàn toàn khác biệt về bản chất — không chỉ là "lọc bớt các trường dữ liệu":

Nếu hệ thống chỉ phục vụ web và web admin với nhu cầu dữ liệu tương tự nhau, một API Gateway duy nhất là đủ — áp dụng BFF lúc này chỉ gây dư thừa kỹ thuật (over-engineering).

Khi có thêm ứng dụng di động, một API Gateway duy nhất buộc client mobile gọi nhiều API rời rạc chỉ để dựng một màn hình; một Mobile BFF chuyên gom dữ liệu (aggregate) sẽ cắt đáng kể số vòng gọi mạng.

Ở kịch bản phức tạp nhất — thêm thiết bị IoT (như hệ thống điểm danh khuôn mặt ở giảng đường) và bên thứ ba (như hệ thống quản lý đào tạo của trường cần đồng bộ điểm) — một Gateway duy nhất sẽ phình to, khó bảo trì; đây là lúc nên chia thành **nhiều BFF độc lập** phục vụ riêng từng nhóm client.

Ví dụ: nếu KBM phát triển thêm **mobile app** cho sinh viên, nền tảng này sẽ yêu cầu: (1) **batched API** — giao diện "Bảng điều khiển" (Dashboard) chỉ cần một lời gọi để lấy cả thông tin cá nhân, bài nộp gần đây và thứ hạng (thay vì phải gọi ba lần, nhờ đó giảm ảnh hưởng của mạng di động chậm); (2) **reduced payload** — thiết bị di động không cần dữ liệu định dạng sẵn cho HTML mà chỉ

cần dữ liệu thô gọn nhẹ; (3) **tích hợp thông báo đẩy (push notification)** — Gateway dành riêng cho di động sẽ quản lý các mã định danh thiết bị (device token).

Newman trong [4a, Ch.4] khuyến nghị: **đội ngũ phát triển giao diện (frontend)** nên quản lý BFF — đội ngũ phát triển ứng dụng di động chịu trách nhiệm về Mobile BFF, đội ngũ phát triển web đảm nhận Web BFF. Mỗi BFF hoạt động như một lớp xử lý mỏng (thin layer): tiếp nhận yêu cầu từ client, gọi các dịch vụ nội bộ (downstream service), tiến hành gom nhóm và biến đổi dữ liệu (aggregate và transform), sau đó trả về định dạng phù hợp cho từng loại client.

⚠ Aggregation vs Orchestration

BFF được phép thực hiện *Data Aggregation* (gom nhiều lời gọi API lại để giao diện người dùng (UI) hiển thị nhanh hơn). Tuy nhiên, cần đặc biệt tránh việc sử dụng BFF hay Gateway để thực hiện *Business Orchestration* (điều phối nghiệp vụ, ví dụ: tạo bài nộp rồi cập nhật điểm số). Mọi logic điều phối giao dịch phải nằm ở tầng dịch vụ miền (sử dụng Saga/Process Manager như ở Chương 6).

8.2. Spring Cloud Gateway — Reactive Gateway

Khi xây dựng API Gateway, việc lựa chọn công nghệ lõi ảnh hưởng trực tiếp đến khả năng mở rộng của hệ thống.

8.2.1. Vấn đề: chọn công nghệ cho Gateway

Trong hệ sinh thái Spring, lập trình viên thường cân nhắc giữa hai lựa chọn chính khi xây dựng Gateway tập trung:

- **Mô hình hoạt động**: Spring Cloud Gateway sử dụng mô hình phản ứng (Reactive, WebFlux, non-blocking), cho phép xử lý số lượng lớn các kết nối đồng thời với rất ít luồng (thread). Trong khi đó, Netflix Zuul (phiên bản 1.x) dựa trên mô hình Servlet truyền thống (blocking, thread-per-request), dễ trở thành điểm nghẽn khi tải lớn.
- **Hiệu năng và WebSocket**: Nhờ kiến trúc WebFlux, Spring Cloud Gateway giữ được nhiều kết nối và chờ I/O với ít thread hơn, đồng thời hỗ trợ sẵn WebSocket (native support) — một tính năng cốt lõi cho các ứng dụng thời gian thực. Ngược lại, Zuul 1.x có hiệu năng thấp hơn do phụ thuộc vào giới hạn của thread pool và hoàn toàn không hỗ trợ WebSocket.
- **Hệ sinh thái và tình trạng**: Spring Cloud Gateway hiện là dự án đang được tích cực phát triển (active), được Spring chính thức khuyên dùng và gắn kết chặt chẽ vào hệ sinh thái Spring Cloud. Trái lại, module Zuul 1.x của Spring Cloud Netflix đã chuyển sang chế độ bảo trì từ năm 2018 và bị gỡ khỏi Spring Cloud từ bản 2020.0, nên hiện chỉ được xem như một công nghệ cũ (legacy).

Dựa trên các ưu điểm này, LMS core sử dụng **Spring Cloud Gateway**. Quyết định này nhằm đáp ứng yêu cầu hỗ trợ giao thức WebSocket cho tính năng đẩy thông báo (notification push) liên tục, đồng thời đảm bảo hiệu năng cao cho hệ thống.

8.2.2. Kiến trúc Spring Cloud Gateway

Khác với các gateway nguyên khối thế hệ trước, Spring Cloud Gateway chia quá trình xử lý request thành những thành phần độc lập và dễ kết hợp. Hình 8.4 minh họa luồng đi của một request: nó lần lượt được đối chiếu với các **Predicate** (điều kiện định tuyến) và đi qua chuỗi **Filter** (bộ lọc) trước khi được chuyển tới service đích.

Hình 8.4: Kiến trúc Spring Cloud Gateway — Route, Predicate, Filter

Kiến trúc của Spring Cloud Gateway xoay quanh ba khái niệm cốt lõi:

- **Route**: Quy tắc định tuyến từ một điều kiện đầu vào đến URI đích — ví dụ chuyển toàn bộ `/api/core/**` sang `lb://kbm-core`.
- **Predicate**: Phép thử boolean quyết định request có khớp Route hay không, dựa trên các thuộc tính như `Path` hay `Method`.
- **Filter**: Điểm can thiệp vào vòng đời request/response, chạy trước hoặc sau bước định tuyến để xử lý các nghiệp vụ cắt ngang — xác minh JWT qua `JwtAuthGlobalFilter`, gắn thêm header, ghi log.

Về dependency: Gateway sử dụng thư viện `spring-cloud-starter-gateway` (WebFlux; từ Spring Cloud 2025.0 đổi tên thành `spring-cloud-starter-gateway-server-webflux`, và tiền tố cấu hình `spring.cloud.gateway.` thành `spring.cloud.gateway.server.webflux.`) và `spring-cloud-starter-netflix-eureka-client`. **Lưu ý**: Gateway hoạt động dựa trên WebFlux (mô hình phản ứng, không chặn luồng), nên *không* tương thích với `spring-boot-starter-web` (mô hình servlet, chặn luồng). Khai báo đồng thời cả hai thư viện này trong dự án Gateway sẽ gây xung đột (conflict) và khiến ứng dụng không thể khởi động.

8.3. Cấu hình route với Eureka

8.3.1. Vấn đề: route cố định hay động?

Phương pháp đơn giản nhất là gắn cố định (hardcode) URL cho từng dịch vụ trong tệp cấu hình của Gateway. Tuy nhiên, khi các dịch vụ mở rộng quy mô hoặc được triển khai sang container mới, các URL sẽ thay đổi. Khi đó, việc cập nhật thủ công cấu hình Gateway sẽ khó khăn và dễ phát sinh lỗi.

Giải pháp: kết hợp Gateway routing với **Eureka service discovery** (đã học ở Chương 4). Gateway không cần lưu trữ địa chỉ IP và cổng cụ thể của dịch vụ — nó chỉ cần tên dịch vụ: Eureka cung cấp danh sách instance đang hoạt động, Spring Cloud LoadBalancer chọn một instance trong danh sách đó, và Gateway chuyển tiếp request tới đích đã chọn.

8.3.2. Cấu hình route cho LMS Gateway

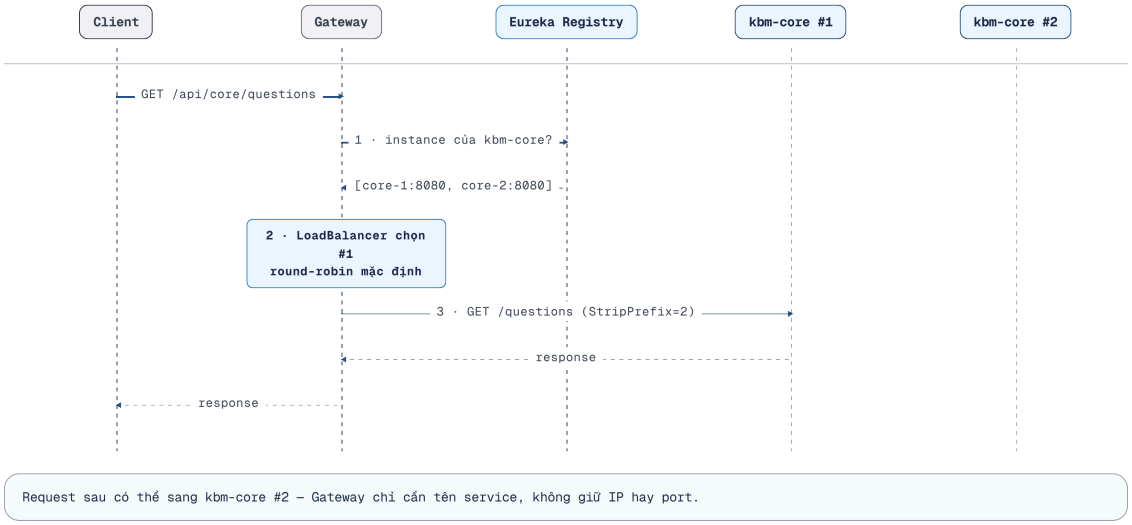
Listing 8.1 là cấu hình route của LMS Gateway: mỗi route trỏ tới tên service trong Eureka thay vì địa chỉ cố định.

Listing 8.1: *Cấu hình route của LMS Gateway (YAML + Eureka)*

```yaml
# application-lb.yml — LMS Gateway routing configuration
spring:
  cloud:
    gateway:
      routes:
        # Core Service — questions, submissions, contests
        - id: kbm-core
          uri: lb://kbm-core          # lb:// = Eureka lookup + load balance
          predicates:
            - Path=/api/core/**
          filters:
            - StripPrefix=2               # /api/core/questions → /questions

        # Academic Service (kbm-academic) — courses, grades
        - id: kbm-academic
          uri: lb://kbm-academic
          predicates:
            - Path=/api/academic/**
          filters:
            - StripPrefix=2

        # Auth Service — login, register
        - id: kbm-auth
          uri: lb://kbm-auth
          predicates:
            - Path=/api/auth/**
          filters:
            - StripPrefix=2

        # Notification — WebSocket endpoint
        - id: kbm-notification-ws
          uri: lb://kbm-notification
          predicates:
            - Path=/ws/**
```

8.3.3. Cách `lb://` hoạt động

Hình 8.5 minh họa luồng phân giải URI có tiền tố `lb://`: request được định tuyến đến một tên service trong Eureka thay vì một IP tĩnh:



Hình 8.5: Luồng `lb://` — Eureka lookup + load balance + route

`lb://kbm-core` thực hiện ba bước tự động:

1. **Lookup**: truy vấn Eureka để lấy danh sách toàn bộ instance đang đăng ký dưới tên `kbm-core`
2. **Load balance**: lựa chọn một instance thông qua Spring Cloud LoadBalancer (round-robin mặc định)
3. **Forward**: chuyển tiếp request đến instance đã được chọn

`StripPrefix=2` loại bỏ hai phần đầu tiên của đường dẫn: `/api/core/questions` → `/questions`. Nhờ đó, dịch vụ không cần quan tâm đến đường dẫn gốc (mount path) của nó tại Gateway — dịch vụ chỉ cần xử lý đường dẫn nội bộ của chính nó.

Service không biết Gateway

Các dịch vụ phía sau *không nên biết* về sự tồn tại của Gateway. Mỗi dịch vụ tự xử lý các đường dẫn nội bộ tương ứng (`/questions`, `/users`), trong khi Gateway đảm nhiệm việc xử lý tiền tố (prefix) và định tuyến. Điều này đảm bảo: (1) các dịch vụ có thể được kiểm thử độc lập (standalone) mà không phụ thuộc vào Gateway; (2) việc cấu trúc lại định tuyến tại Gateway không ảnh hưởng đến mã nguồn của dịch vụ; (3) dịch vụ dễ chuyển sang môi trường khác (portable) — có thể triển khai sau các gateway khác (như NGINX, Kong) mà không cần chỉnh sửa mã nguồn.

8.4. Nghiệp vụ cắt ngang (cross-cutting concern) tại Gateway

Triển khai các nghiệp vụ cắt ngang ở mỗi dịch vụ giống như từng lớp học tự đặt ra quy định kiểm tra thẻ sinh viên riêng lẻ thay vì giao cho bảo vệ cổng trường: vừa dư thừa, vừa không thống nhất và khó quản lý.

8.4.1. Vấn đề: mỗi service triển khai xác thực, CORS và logging theo cách khác nhau

Nếu mỗi dịch vụ tự viết một bộ phân tích JWT, tự cấu hình CORS và tự triển khai rate limiting theo cách riêng, mã nguồn sẽ bị trùng lặp và chính sách dễ trôi lệch. Gateway giải quyết phần chính sách ở biên: tiếp nhận thông tin xác thực (credential) từ client, loại bỏ dữ liệu định danh có thể bị giả mạo, kiểm tra token và áp dụng các biện pháp bảo vệ lưu lượng. Điều đó không miễn trách

nhiệm cho resource service: service đích vẫn xác minh bằng chứng đã ký theo hợp đồng chung và tự thực hiện phân quyền nghiệp vụ.

8.4.2. Xác thực — kiểm tra JWT tại Gateway

Hạn chế của Session Cookie

Trong monolith, trường đại học chỉ có một cô thủ thư duy nhất. Khi một sinh viên vào thư viện, cô nhận ra mặt và nhớ đó là sinh viên (Session Cookie). Nhưng khi trường mở rộng ra 10 chi nhánh thư viện (microservices), sinh viên sang chi nhánh khác, cô thủ thư mới sẽ không biết người đó là ai. Các chi nhánh phải dùng chung một sổ đăng ký trung tâm (Shared Redis), và mọi lượt kiểm tra đều phụ thuộc vào sổ đó. Đó là lý do chúng ta chuyển sang **JWT (JSON Web Token)** — hãy coi nó như **thẻ sinh viên điện tử**. Trên thẻ JWT đã dán ảnh, ghi mã sinh viên và có "con dấu đỏ" chữ ký điện tử của Phòng Đào tạo. Các chi nhánh (downstream service) hoặc cổng bảo vệ (Gateway) chỉ cần nhìn con dấu là biết thẻ thật, hoàn toàn không cần gọi điện hỏi lại Phòng Đào tạo (xác minh không trạng thái, stateless verification).

Do hạn chế của Session Cookie trong kiến trúc phân tán, KBM lựa chọn cơ chế xác thực không trạng thái (stateless) với JSON Web Token (JWT). Gateway là chốt kiểm tra đầu tiên, nhưng kết quả kiểm tra tại đó không phải bằng chứng duy nhất cho mọi trust boundary. Ở đây chỉ cần theo dõi trách nhiệm kiểm tra ở Gateway và service đích; cấu trúc JWT, issuer/audience và các trust tier được giải thích ở §9.2–9.3. Đoạn mã dưới đây mô tả **thiết kế chuyển tiếp** của `JwtAuthGlobalFilter`: loại bỏ các header định danh do client tự khai, xác minh access token tại biên và giữ lại `Authorization` để các service đang xác minh token không bị mất bằng chứng trước khi cơ chế token nội bộ có chữ ký được triển khai:

Listing 8.2: `GlobalFilter` xác minh JWT tại Gateway

```
// Gateway JWT Filter — validate token trước khi route
@Component
public class JwtAuthGlobalFilter implements GlobalFilter, Ordered {

    private final JwtUtil jwtUtil;

    public JwtAuthGlobalFilter(JwtUtil jwtUtil) {
        this.jwtUtil = jwtUtil;
    }

    @Override
    public Mono<Void> filter(ServerWebExchange exchange, GatewayFilterChain chain) {
        ServerHttpRequest request = exchange.getRequest();
        String path = request.getURI().getPath();

        // Bắt buộc XÓA identity headers do client tự gửi (chống Header Spoofing).
        // KHÔNG xóa Authorization trước khi có signed downstream token thay thế.
        ServerHttpRequest.Builder requestBuilder = request.mutate()
            .headers(headers -> {
                // Gỡ mọi X-User-* do client gửi, không phân biệt hoa thường
                headers.keySet().removeIf(
                        name -> name.regionMatches(true, 0, "X-User-", 0, 7));
            });

        // Bỏ qua xác thực cho các endpoint công khai
```

```
        if (isPublicEndpoint(path)) {
            return chain.filter(
                exchange.mutate().request(requestBuilder.build()).build()
            );
        }

        // Xác minh alg + signature + iss + aud + exp + nbf cho audience của Gateway
        String token = extractToken(request);
        if (token == null || !jwtUtil.validateAccessToken(token, "kbm-gateway")) {
            exchange.getResponse().setStatusCode(HttpStatus.UNAUTHORIZED);
            return exchange.getResponse().setComplete();
        }

        // Context dẫn xuất: chỉ dùng cho routing/rate limit.
        // Giữ Authorization cho downstream verifier.
        requestBuilder.headers(headers -> {
            headers.set("X-User-Id", jwtUtil.getUserId(token));
            headers.set("X-User-Roles", jwtUtil.getRoles(token));
        });

        return chain.filter(
            exchange.mutate().request(requestBuilder.build()).build()
        );
    }

    @Override
    public int getOrder() {
        return -1; // Chạy TRƯỚC RequestRateLimiter
    }
}
```

Sau khi Gateway xác minh JWT, thông tin định danh (claim) cần được truyền đến các dịch vụ phía sau (downstream) mà không biến một header thuần thành bằng chứng danh tính. Hình 8.6 mô tả luồng chuyển tiếp: token đã ký được giữ lại hoặc được đổi thành token nội bộ đúng audience; các header dẫn xuất chỉ là context tiện dụng cho routing, logging và rate limiting:

__Hình 8.6:__ Luồng JWT tại Gateway — bằng chứng có chữ ký và context định danh dẫn xuất

Tại thời điểm khảo sát (đầu năm 2026), Gateway của KBM chèn `X-User-Id` và `X-User-Roles`; một số service tin các header này, trong khi một số service khác tự parse JWT bằng thư viện và phiên bản khác nhau. Đây là sự trôi lệch triển khai (implementation drift), không phải phòng thủ chiều sâu (defense in depth) có chủ đích. Việc tin header thuần chỉ được ghi nhận là **rủi ro được chấp nhận** (accepted risk) tạm thời khi service không thể bị truy cập ngoài đường đi đã kiểm soát; đó không phải kiến trúc mục tiêu.

Ở **mục tiêu Tier 1** (§9.3), Gateway xác minh token ngoài rồi truyền bằng chứng có chữ ký xuống service đích: giữ access token trong giai đoạn chuyển tiếp, hoặc dùng token exchange để đổi sang token nội bộ đúng audience. Resource service xác minh issuer, audience, signature và freshness trước khi dùng claim, sau đó tự kiểm tra phân quyền nghiệp vụ. `X-User-*` có thể tồn tại như dữ liệu dẫn xuất, nhưng không được dùng một mình để ra quyết định cấp quyền.

⚠ Giả định tin cậy mạng

Chính sách mạng (quy tắc tường lửa, Kubernetes NetworkPolicy) phải chặn truy cập trực tiếp vào service, còn Gateway phải loại bỏ mọi `X-User-*` từ Internet trước khi tự ghi đè context đã dẫn xuất. Đây là điều kiện tối thiểu để chấp nhận rủi ro nói trên, nhưng chỉ chứng minh *đường đi của request*, không chứng minh nội dung claim.

Cần nhớ giới hạn của mTLS: nó bảo vệ kênh truyền và xác thực *workload/actor* ở hai đầu, không xác thực *user subject* trong `X-User-Id`. Một workload nội bộ bị chiếm quyền vẫn có thể tạo header giả. Vì vậy, request nhạy cảm cần đồng thời có bằng chứng danh tính có chữ ký (signed identity proof) đúng audience, phân quyền cục bộ và — nếu service gọi thay mặt người dùng — ngữ nghĩa delegation tách biệt subject khỏi actor.

8.4.3. CORS — Cross-Origin Resource Sharing

Gateway là **nơi duy nhất** quản lý cấu hình CORS: các origin, method và header được chấp nhận. Cấu hình `globalcors` trong Spring Cloud Gateway cho phép khai báo `allowedOrigins` (các origin hợp lệ), `allowedMethods`, `allowedHeaders`, `allowCredentials` — các dịch vụ phía sau không cần thiết lập cấu hình CORS vì Gateway đã đảm nhận chức năng này.

__Listing 8.3:__ Cấu hình CORS an toàn tại Gateway

```yaml
spring:
  cloud:
    gateway:
      globalcors:
        cors-configurations:
          '[/**]':
            allowed-origins:
              - "https://kbm.example.edu"
              - "https://admin.kbm.example.edu"
            allowed-methods: "GET, POST, PUT, DELETE, OPTIONS"
            # BẮT BUỘC — thiếu dòng này thì allowedHeaders = null
            # → checkHeaders() trả null → preflight bị từ chối
            # → mọi lời gọi API cross-origin kèm JWT bị chặn ngay
            allowed-headers:
              - "Authorization"
              - "Content-Type"
              - "X-Request-ID"
            # chỉ đổi thành true khi xác thực bằng cookie
            # và đã có biện pháp chống CSRF
            allow-credentials: false
```

Cấu hình CORS ở cấp độ Gateway giúp hệ thống tránh phải thiết lập lại các quy tắc tương tự ở từng dịch vụ. Bằng cách khai báo tập trung các origin hợp lệ (`allowedOrigins`), method được phép (`allowedMethods`), danh sách header được chấp nhận (`allowedHeaders`) và việc có cho phép gửi kèm credential hay không (`allowCredentials`), Gateway kiểm soát nhất quán các yêu cầu cross-origin. **Lưu ý quan trọng**: Spring Cloud Gateway *không* tự động cho phép mọi header khi khai báo `globalcors` qua YAML — nếu thiếu `allowed-headers`, `CorsConfiguration.checkHeaders()` sẽ trả về `null` (từ chối), khiến mọi preflight request chứa `Authorization` thất bại và toàn bộ lời gọi API cross-origin kèm JWT bị chặn ngay ở bước OPTIONS.

CORS trong KBM

Tại thời điểm khảo sát, KBM dùng cấu hình `allowedOrigins: "*"` — cho phép *mọi* origin truy xuất API. Trong giai đoạn phát triển (development), đây là giải pháp tiện lợi để tránh lỗi CORS. Tuy nhiên, trên production, đây là một chính sách **quá rộng** so với chủ đích của KBM (API không nhằm phục vụ truy cập cross-origin công khai): mọi trang web đều có thể gọi và *đọc* phản hồi không kèm credential. Cần tách bạch hai điều thường bị nhầm: (1) với chế độ gửi kèm credential, trình duyệt *từ chối* tổ hợp `*` + `allowCredentials: true` — muốn cookie hoạt động xuyên origin bắt buộc phải allowlist origin cụ thể; (2) CORS chỉ kiểm soát việc JavaScript *đọc* response, nó **không phải** cơ chế chống CSRF — xác thực dựa trên cookie vẫn cần `SameSite`, CSRF token hoặc kiểm tra Origin riêng (Chương 9). **Lộ trình chuyển đổi (Migration path)**: (1) lập allowlist origin riêng cho từng môi trường — sách không công bố domain thật của hệ thống; (2) chỉ bật `allowCredentials: true` khi dùng xác thực cookie và đã áp dụng các biện pháp chống CSRF nói trên; (3) kiểm thử riêng với ứng dụng di động, nếu có.

8.4.4. Rate Limiting

Cơ chế giới hạn lưu lượng (rate limiting) là kỹ thuật để ngăn chặn tình trạng một client gửi quá nhiều yêu cầu trong thời gian ngắn, qua đó bảo vệ các dịch vụ cốt lõi khỏi nguy cơ quá tải hoặc bị tấn công từ chối dịch vụ (DoS). Spring Cloud Gateway hỗ trợ sẵn bộ lọc `RequestRateLimiter` kết hợp với Redis để hiện thực hóa tính năng này. Bằng cách tùy chỉnh các thông số như tốc độ bổ sung token `replenishRate` (số request mỗi giây), dung lượng tối đa của bucket `burstCapacity` (số request tối đa xử lý tức thời) và thành phần xác định khóa `KeyResolver`, hệ thống có thể kiểm soát lưu lượng truy cập dựa trên nhiều chính sách khác nhau. Các chiến lược phổ biến bao gồm:

1. **Theo người dùng (per user):** Áp dụng hạn ngạch (ví dụ: N request/giây) cho từng người dùng đã xác thực. Phương pháp này giúp ngăn chặn lạm dụng hoặc khai thác tài nguyên quá mức từ một tài khoản.
2. **Theo IP (per IP):** Giới hạn lưu lượng từ một địa chỉ IP duy nhất. Chiến lược này giúp ngăn chặn tấn công DDoS thô sơ và các bot cào dữ liệu (scraping).
3. **Theo route (per route):** Thiết lập giới hạn riêng cho từng API. Kỹ thuật này thường áp dụng để bảo vệ các API tiêu tốn nhiều năng lực tính toán hoặc I/O (ví dụ: xuất báo cáo thống kê).
4. **Toàn cục (global):** Giới hạn tổng lượng yêu cầu truy cập đi vào hệ thống từ mọi nguồn. Quy tắc này hoạt động như chiếc cầu chì an toàn cuối cùng, đảm bảo hạ tầng mạng bên trong không bị quá tải khi đối mặt với lưu lượng đột biến.

Thiết kế rate limiting với Redis:

Spring Cloud Gateway sử dụng **thuật toán token bucket** (Redis-backed) — mỗi người dùng có một "bucket" chứa thẻ bài (token), mỗi yêu cầu tiêu thụ một thẻ bài, và các thẻ bài này được bổ sung theo tốc độ `replenishRate`:

__Listing 8.4:__ Cấu hình rate limiting cho endpoint nộp bài

```yaml
# application.yml — Rate limiting cho endpoint nộp bài
spring:
  cloud:
    gateway:
      routes:
        # Đặt TRƯỚC route kbm-core ở listing route (route được xét theo thứ tự);
        # hai listing thuộc cùng một danh sách routes
        - id: kbm-core-rate-limited
          uri: lb://kbm-core
          predicates:
            - Path=/api/core/submissions/**
          filters:
            - StripPrefix=2
            - name: RequestRateLimiter
              args:
                redis-rate-limiter.replenishRate: 5    # 5 request/giây
                redis-rate-limiter.burstCapacity: 10   # Burst tối đa 10
                redis-rate-limiter.requestedTokens: 1  # 1 token/request
                key-resolver: "#{@userKeyResolver}"    # Rate limit theo user

  data:
    redis:
      host: localhost
      port: 6379
```

Trong cấu hình trên, bộ lọc `RequestRateLimiter` được áp dụng vào route nộp bài tập với các thông số về tốc độ bổ sung và dung lượng bucket. Tuy nhiên, Gateway cần một `KeyResolver` để xác định

định danh (khóa) cho từng yêu cầu nhằm cấp phát hạn ngạch phù hợp. Đoạn mã dưới đây triển khai `KeyResolver` dùng `X-User-Id` (do bộ lọc xác thực cung cấp), kết hợp fallback an toàn để tránh `NullPointerException` đối với các API ẩn danh:

Listing 8.5: `KeyResolver` tránh `NullPointerException` — khóa rate limit theo `userId`

```
@Configuration
public class RateLimitConfig {

    // Địa chỉ các proxy biên được phép ghi X-Forwarded-For.
    // Đưa ra cấu hình (application.yml) thay vì hardcode; danh sách rỗng
    // nghĩa là không tin header nào — mặc định an toàn.
    @Value("${kbm.gateway.trusted-proxies:}")
    private Set<String> trustedProxies = Set.of();

    @Bean
    public KeyResolver userKeyResolver() {
        return exchange -> {
            // Lấy userId từ header đã được JWT filter inject
            String userId = exchange.getRequest().getHeaders()
                .getFirst("X-User-Id");
            if (userId != null) {
                return Mono.just(userId);            // Rate limit per user
            }
            // Fallback: rate limit per IP cho anonymous requests
            // CHÚ Ý: client tự đặt được X-Forwarded-For — chỉ tin header này khi
            // proxy biên (trusted proxy) đã ghi đè/sanitize nó,
            // nếu không kẻ tấn công đổi header để né rate limit;
            // không có proxy tin cậy → dùng remoteAddress
            java.net.InetSocketAddress remoteAddress =
                    exchange.getRequest().getRemoteAddress();
            String peer =
                (remoteAddress != null && remoteAddress.getAddress() != null)
                ? remoteAddress.getAddress().getHostAddress() : null;

            // Chỉ đọc X-Forwarded-For khi request THỰC SỰ đến từ proxy biên
            // đã đăng ký.
            // Thiếu điều kiện này, client tự gửi header và tự chọn key rate limit.
            String forwardedFor = exchange.getRequest().getHeaders()
                    .getFirst("X-Forwarded-For");
            if (forwardedFor != null && peer != null
                    && trustedProxies.contains(peer)) {
                // Lấy phần tử CUỐI (IP do proxy biên ghi nhận) — phần tử trái nhất
                // là dữ liệu client tự khai, kẻ tấn công đổi tùy ý để né rate limit
                String[] hops = forwardedFor.split(",");
                return Mono.just(hops[hops.length - 1].trim());
            }
            // Không có proxy tin cậy → chỉ tin địa chỉ TCP thật, không tin header
            return Mono.just(peer != null ? peer : "anonymous");
        };
    }
}
```

Khi người dùng vượt quá hạn ngạch, Gateway tự động trả về mã lỗi **HTTP 429 Too Many Requests**, qua đó thông báo trực tiếp cho client mà không cần chuyển tiếp yêu cầu xuống các dịch vụ phía sau. Về bảo mật, `X-Forwarded-For` chỉ đáng tin khi request thực sự đi qua proxy biên đã được cấu hình ghi đè header này; khi chuỗi có nhiều hop, hãy chọn IP theo số proxy tin cậy (đếm từ phải sang) thay vì lấy phần tử trái nhất do client kiểm soát. Một lưu ý vận hành: mọi khóa dùng chung

— nhánh `"anonymous"` cuối cùng, hay thực tế hơn là cả phòng máy đi ra ngoài qua một IP NAT — đều gom nhiều người dùng vào một bucket. Nếu áp chính sách cá nhân (5 request/giây) cho bucket đó, hàng trăm sinh viên sau cùng một NAT sẽ chia nhau đúng 5 request/giây và nhận 429 hàng loạt. Route công khai cần chính sách riêng rộng hơn, hoặc đứng ngoài lớp rate limit chặt này.

Rate limit cho chế độ thi

Trong thời gian diễn ra kỳ thi, tần suất nộp bài cao hơn bình thường (hàng trăm sinh viên nộp bài cùng lúc). Cần lưu ý các điểm sau: (1) đo burst nộp bài của từng người để chọn hạn ngạch cá nhân: bucket tính theo người dùng, nên số thí sinh tăng không tự làm tăng hạn ngạch của mỗi người; (2) khi tổng số thí sinh tăng, kiểm tra năng lực xử lý của cả hệ thống, rồi thêm **giới hạn theo từng route (per-route rate limit)** dành riêng cho các API thi cử hoặc hàng đợi phù hợp; (3) tận dụng các thao tác nguyên tử của Redis để đảm bảo đếm chính xác ngay cả khi có lượng lớn request đồng thời.

8.4.5. Logging & Tracing

Gateway là chốt biên HTTP để chuẩn hóa `X-Request-ID` — mã định danh cho từng yêu cầu đi vào hệ thống, đóng vai trò mã tham chiếu hỗ trợ (support handle) khi tra cứu log hoặc đối chiếu phản hồi lỗi. Cần phân biệt rõ ba loại mã định danh (chi tiết tại Chương 11): (1) `X-Request-ID` (`requestId` trong log) định danh một HTTP request đơn lẻ ở biên; (2) `traceparent` / `traceId` (chuẩn W3C Trace Context) liên kết các span trong một vết gọi phân tán; và (3) `correlationId` đặt trong event envelope để gom nhóm các bước của cùng một luồng nghiệp vụ bất đồng bộ qua Kafka (có thể trải qua nhiều request và nhiều trace).

Tại biên mạng, Gateway *không được* chép nguyên văn giá trị `X-Request-ID` thô từ client vào log (tránh lỗ hổng *log injection* hoặc chuỗi quá dài): bộ lọc chỉ tái sử dụng giá trị hợp lệ khớp biểu thức chính quy an toàn, ngược lại sinh mã mới mang tiền tố `req-`, rồi gắn đồng thời vào cả request chuyển xuống dịch vụ phía sau lẫn response trả về client:

Listing 8.6: GlobalFilter chuẩn hóa `X-Request-ID` tại biên Gateway

```
@Component
public class RequestIdFilter implements GlobalFilter, Ordered {
    private static final String HEADER = "X-Request-ID";
    private static final java.util.regex.Pattern SAFE_ID =
        java.util.regex.Pattern.compile("^[A-Za-z0-9._-]{1,64}$");

    @Override
    public Mono<Void> filter(
            ServerWebExchange exchange, GatewayFilterChain chain) {
        String incoming = exchange.getRequest().getHeaders().getFirst(HEADER);
        String requestId = (incoming != null && SAFE_ID.matcher(incoming).matches())
            ? incoming
            : "req-" + java.util.UUID.randomUUID();

        ServerHttpRequest mutatedRequest = exchange.getRequest().mutate()
            .headers(h -> h.set(HEADER, requestId))
            .build();
        exchange.getResponse().getHeaders().set(HEADER, requestId);

        return chain.filter(
            exchange.mutate().request(mutatedRequest).build()
        );
```

```
    }

    @Override
    public int getOrder() {
        // Chạy đầu tiên, trước cả JwtAuthGlobalFilter (-1)
        return Ordered.HIGHEST_PRECEDENCE;
    }
}
```

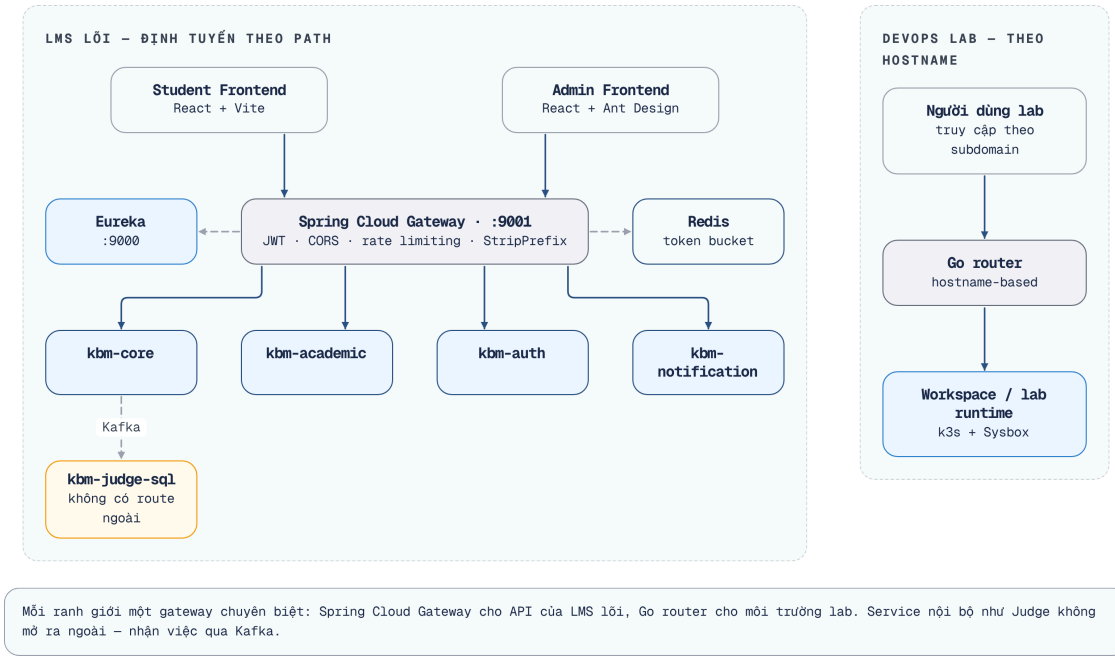
Gateway cũng là vị trí lý tưởng để ghi nhận **access log tập trung**, bao gồm: method, đường dẫn (path), mã trạng thái, độ trễ, `requestId` và định danh người dùng dẫn xuất. Thông tin này giúp hệ thống dễ dàng phát hiện endpoint nào có độ trễ bất thường, người dùng nào thường xuyên gặp lỗi và sự thay đổi của mẫu lưu lượng (traffic pattern) trước và sau khi deploy. Nhờ đó, không phải chèn thêm mã nguồn vào từng dịch vụ riêng lẻ — một `GlobalFilter` duy nhất đủ cho lớp biên HTTP.

Chi tiết triển khai structured logging, log aggregation và distributed tracing xem Chương 11 (Observability).

8.5. Case study: KBM

8.5.1. Kiến trúc tổng thể

Để hiểu rõ cách cấu hình API Gateway trong thực tế, hãy xem xét kiến trúc của hệ thống KBM (Hình 8.7). Gateway hoạt động như điểm vào duy nhất, làm nhiệm vụ định tuyến và che giấu sự phức tạp của các dịch vụ mạng nội bộ:



Hình 8.7: *Kiến trúc tổng thể KBM — Gateway là điểm vào duy nhất*

Hệ thống KBM có hai loại gateway/router cần được phân biệt rõ:

- **Spring Cloud Gateway**: dành cho LMS core, chịu trách nhiệm định tuyến API theo đường dẫn, xác minh JWT, cấu hình CORS, giới hạn lưu lượng và gắn mã yêu cầu ở biên (`X-Request-ID`).

- **Go hostname router**: router theo tên miền, viết bằng Go dành cho DevOps Lab, thực hiện định tuyến môi trường làm việc (workspace/lab runtime) dựa trên tên miền hoặc tên miền phụ (subdomain) nội bộ, đồng thời tích hợp trực tiếp với k3s/Sysbox ở lớp hạ tầng.

8.5.2. Phân tích cấu hình

Bảng 8.1 đối chiếu cấu hình Gateway của KBM tại thời điểm khảo sát (đầu năm 2026) với giải pháp khuyến nghị và chỉ ra khoảng trống.

__Bảng 8.1:__ Phân tích cấu hình Gateway trong KBM

Khía cạnh	KBM tại thời điểm khảo sát	Giải pháp khuyến nghị	Khoảng trống
Routing	`lb://` URI qua Eureka cho các dịch vụ Java; hostname routing cho DevOps Lab	✓ Đúng theo từng ranh giới	Cần tài liệu hóa quyền sở hữu route
Xác minh danh tính	Gateway xác minh JWT; downstream có nơi tin header, có nơi tự parse token	Xác minh ở biên và ở resource service theo trust tier	Trôi lệch triển khai; cần kiểm chứng việc chặn truy cập trực tiếp
CORS	`allowedOrigins: "*"`	× Nên giới hạn	Liệt kê các origin cụ thể
Rate Limiting	Đã có: token bucket (Redis) ở Gateway	✓ Phù hợp	Chính sách riêng cho kỳ thi
SSL/TLS	Kết thúc ở lớp biên (reverse proxy hoặc Gateway)	! Tùy cách triển khai	Ghi rõ điểm kết thúc TLS trong tài liệu triển khai
Ngữ cảnh truy vết	Có nền tảng (`X-Request-ID`), chưa chuẩn hóa đầu cuối	! Nên chuẩn hóa	Chuẩn hóa `X-Request-ID` ở HTTP, `traceparent` và truyền `correlationId` qua Kafka (Chương 11)
Path Rewriting	Bộ lọc `StripPrefix`	✓ Đúng	—
WebSocket	Route tới notification service	✓ Đúng	—
Health Check	Các endpoint Actuator	✓ Đúng	—

8.5.3. Vấn đề không đồng nhất phiên bản thư viện JWT

Một vấn đề cần lưu ý trong KBM tại thời điểm khảo sát: Gateway và các dịch vụ khác dùng hai phiên bản JJWT với API khác nhau (`parserBuilder()` và `parser()`). Vấn đề không nằm ở việc resource service xác minh bằng chứng, mà ở chỗ mỗi nơi tự triển khai một chính sách khác nhau. Khi nâng cấp hoặc bổ sung RS256, sự sai lệch phiên bản (version mismatch) có thể khiến Gateway chấp nhận token nhưng service phía sau từ chối, hoặc nguy hiểm hơn là hai nơi kiểm tra issuer, audience và thuật toán không đồng nhất.

Không đồng nhất phiên bản thư viện JWT

Với thuật toán khóa đối xứng đơn giản (HS256), hai phiên bản này có thể tương thích trong luồng xử lý chuẩn (happy path). Tuy nhiên, khi nâng cấp hoặc bổ sung các tính năng phức tạp (như RS256, token refresh), sự sai lệch phiên bản sẽ gây ra các lỗi tiềm ẩn khó gỡ lỗi (debug). **Lộ trình chuyển đổi**: (1) trước mắt, đồng bộ thư viện và hợp đồng xác minh (`alg`, `iss`, `aud`, `exp`, `nbf`) để các bản triển khai không trôi lệch nhau; (2) chuyển sang Spring Security OAuth2 Resource Server thay cho `JwtUtil` tự viết; (3) dùng RS256/JWKS để verifier chỉ nhận public key và hỗ trợ luân phiên khóa; (4) khi token ngoài không đúng audience của service phía sau, Gateway thực hiện token exchange sang token nội bộ có chữ ký, không thay bằng các header văn bản thuần.

8.5.4. Lộ trình cải tiến

Giai đoạn 1 — Tăng cường bảo mật (ưu tiên cao, chi phí triển khai thấp):

- Hạn chế chặt chẽ các origin trong CORS (liệt kê các URL frontend cụ thể)

- Gỡ bỏ mọi `X-User-*` do client gửi, giữ `Authorization` cho tới khi có signed downstream token thay thế
- Thống nhất thư viện và hợp đồng xác minh `alg`/issuer/audience/expiry/not-before
- Xác minh bằng cấu hình triển khai rằng chỉ Gateway và workload được cấp phép mới gọi được service nội bộ
- Tài liệu hóa ranh giới giữa Spring Cloud Gateway và Go hostname router

Giai đoạn 2 — Observability (ưu tiên trung bình):

- Chuẩn hóa `X-Request-ID` tại Gateway (`GlobalFilter`), ngữ cảnh truy vết W3C (`traceparent`) và truyền `correlationId` qua Kafka
- Ghi log request/response tại Gateway

Giai đoạn 3 — Bảo vệ và phân tầng tin cậy (ưu tiên trung bình):

- Áp chính sách rate limiting riêng cho chế độ thi (per-route limit cho `/submissions/**`)
- Giới hạn kích thước payload cho các endpoint tải tệp lên (file upload)
- Chuyển sang RS256/JWKS và signed identity propagation đúng audience; bổ sung workload identity/mTLS cho luồng nhạy cảm

⚠ Sai lầm thường gặp

- **Đưa business logic vào Gateway**: Nếu Gateway thực hiện biến đổi dữ liệu nghiệp vụ, kiểm tra tính hợp lệ theo quy tắc nghiệp vụ (validation rule), hoặc điều phối lời gọi giữa các dịch vụ — hậu quả là Gateway sẽ biến thành một monolith mới, mọi thay đổi về business logic đều đòi hỏi phải triển khai lại toàn bộ Gateway. *Phòng tránh*: Gateway chỉ nên đảm nhiệm các nghiệp vụ cắt ngang (xác thực ở biên, CORS, routing, rate limiting); business logic thuộc về các dịch vụ phía sau.
- **Mỗi service tự xác minh JWT theo một cách khác nhau**: Nếu mỗi dịch vụ dùng một phiên bản thư viện, tập thuật toán và quy tắc issuer/audience riêng, hệ thống sẽ có lỗ hổng do các nơi kiểm tra không đồng nhất, dù cùng mang tên "JWT validation". *Phòng tránh*: Gateway xác thực ở biên; resource service xác minh bằng chứng có chữ ký bằng cấu hình hoặc middleware chuẩn hóa; mọi service thực hiện phân quyền cục bộ. Không thay bước xác minh chữ ký bằng việc tin một header văn bản thuần chỉ vì request đến từ mạng nội bộ.
- **Không có phương án dự phòng (fallback) khi Gateway ngừng hoạt động (SPOF)**: Gateway là điểm vào duy nhất nên cũng có nguy cơ trở thành điểm lỗi duy nhất (single point of failure) — khi Gateway gặp sự cố, *toàn bộ* hệ thống bị gián đoạn và không thể truy cập. *Phòng tránh*: triển khai Gateway theo mô hình tính sẵn sàng cao (High Availability — HA) bằng cách chạy nhiều instance sau một Load Balancer (ví dụ: NGINX/ALB). Đồng thời, Gateway phải tránh lỗi dây chuyền (cascading failure). Nếu Redis (dùng cho rate limiting) gặp sự cố, `RedisRateLimiter` mặc định cho request đi qua (fail-open): hệ thống vẫn chạy nhưng tạm thời không còn rate limit, nên cần giám sát và cảnh báo lỗi Redis. Ở phía client, cần triển khai cơ chế Exponential Backoff khi nhận HTTP 429.
- **CORS `allowAll` trong production**: Cho phép mọi origin đọc phản hồi API là chính sách quá rộng nếu API không có chủ đích phục vụ truy cập cross-origin công khai. Cần phân biệt hai điều: với chế độ gửi kèm credential, trình duyệt vốn đã từ chối tổ hợp wildcard `*` + credential, nên muốn xác thực bằng cookie hoạt động xuyên origin thì bắt buộc phải allowlist origin cụ thể; còn CSRF là mối đe dọa riêng, không do CORS xử lý. *Phòng tránh*: Liệt kê rõ ràng các

origin hợp lệ, và với xác thực dựa trên cookie phải bổ sung `SameSite`, CSRF token hoặc kiểm tra Origin (Chương 9).

Bổ sung bảo vệ tại Gateway: Sau khi xử lý các sai lầm trên, cần thêm các lớp bảo vệ sau:

- **JWKS (JSON Web Key Set)**: Gateway không gắn cố định public key để xác minh chữ ký JWT. Thay vào đó, Gateway tự động lấy khóa từ endpoint `/.well-known/jwks.json` của Auth Service để cho phép luân phiên khóa (key rotation) định kỳ, kết hợp cùng bộ đệm nội bộ (in-memory cache) nhằm giảm tải I/O.
- **Redis Blacklist**: Do xác minh chữ ký JWT là thao tác không trạng thái, cần đặt thời gian sống (TTL) của access token ngắn (15–60 phút, §9.2) và kết hợp Redis Blacklist cho các trường hợp thu hồi khẩn cấp (như đình chỉ tài khoản gian lận trong kỳ thi). Khi Redis gặp sự cố, cần quy định rõ đánh đổi giữa hai chế độ: *fail-open* (tạm thời chỉ dựa trên chữ ký JWT có TTL ngắn để duy trì tính sẵn sàng cho các API thông thường) và *fail-closed* (từ chối yêu cầu ở các đường dẫn nhạy cảm như nộp bài thi chính thức cho tới khi kiểm tra được trạng thái thu hồi).
- **Giới hạn kích thước (Payload Size Limit)**: Áp dụng giới hạn cứng để chặn tấn công DoS hoặc payload quá lớn trước khi chuyển tiếp request.
- **HttpOnly Cookie ở biên**: Khi Gateway hoạt động như BFF, giữ token trong *HttpOnly Cookie* thay vì đưa Bearer token cho JavaScript; Gateway chuyển cookie thành credential nội bộ khi route và vẫn áp dụng biện pháp chống CSRF phù hợp (Chương 9).

Các pattern trong chương có minh họa tương tác đi kèm tại companion web:

Interactive Demo

- **Cơ chế định tuyến của API Gateway**: api-gateway-routing.html
- **Service Discovery & Registry**: service-discovery.html
- **Rate Limiting**: rate-limiting.html

8.6. Tổng kết

API Gateway giải quyết một thách thức cơ bản khi client tương tác với hệ thống microservices: thay vì phải biết địa chỉ của từng service, client chỉ cần biết một endpoint duy nhất. Gateway đảm nhận routing và các nghiệp vụ cắt ngang như xác thực ở biên, CORS; service phía sau tập trung vào business logic nhưng vẫn sở hữu phân quyền cục bộ và xác minh bằng chứng danh tính có chữ ký theo tầng tin cậy.

Spring Cloud Gateway (WebFlux) là lựa chọn hiện đại cho hệ sinh thái Java — reactive, non-blocking, tích hợp sâu với Spring Cloud (Eureka, Circuit Breaker). Cấu hình định tuyến với `lb://` URI kết hợp tự động với service discovery và load balancing. Nhờ đó, các service có thể mở rộng mà không cần thay đổi cấu hình tại Gateway.

Tập trung xử lý nghiệp vụ cắt ngang tại Gateway — như kiểm tra token ở biên, CORS, rate limiting và gắn `X-Request-ID` — là khoản đầu tư giúp hệ thống dễ bảo vệ, debug và vận hành hơn. Nguyên tắc cốt lõi: Gateway kiểm soát ingress; bằng chứng có chữ ký đi qua trust boundary; service đích kiểm tra bằng chứng và quyết định phân quyền cho nghiệp vụ mình sở hữu.

Phân tích KBM tại thời điểm khảo sát cho thấy cấu trúc Gateway cơ bản là phù hợp (Spring Cloud Gateway + Eureka + JWT cùng rate limiting với Redis cho LMS core, Go hostname router cho

DevOps Lab), nhưng cấu hình CORS còn quá rộng và việc truyền định danh chưa thống nhất. Lộ trình cải tiến ưu tiên làm sạch request ở biên (ingress sanitation) và một hợp đồng xác minh chung, tiếp đến RS256/JWKS cùng signed downstream identity, rồi bổ sung workload identity cho các tầng tin cậy nhạy cảm.

Ở Chương 9, chúng ta sẽ tìm hiểu sâu về **bảo mật microservices** — cấu trúc JWT, tích hợp OAuth 2.0, chiến lược luân phiên khóa hai bí mật (dual-secret key rotation), và RBAC trong KBM.

8.7. Đọc thêm

Sách tham khảo chính:

1. [2a] Chris Richardson, *Microservices Patterns*, 1st Ed. — Ch.8: External API Patterns (API Gateway, BFF)
2. [4a] Sam Newman, *Building Microservices* — Ch.4: Integration, Smart Endpoints and Dumb Pipes
3. [1] Thomas Erl, *Service-Oriented Architecture: Analysis and Design for Services and Microservices* — API Gateway trong bối cảnh SOA doanh nghiệp

Sách bổ trợ:

4. [2b] Chris Richardson, *Microservices Patterns*, 2nd Ed. — Ch.8: External API Patterns (bản cập nhật)
5. [5] Hugo Rocha, *Practical Event-Driven Microservices Architecture* — Ch.9: UI in EDA, BFF pattern

Nguồn trực tuyến:

- Spring Cloud Gateway reference — docs.spring.io/spring-cloud-gateway
- Netflix Zuul → Spring Cloud Gateway migration guide
- OWASP API Security Top 10 — owasp.org/www-project-api-security
- NIST SP 800-207 — Zero Trust Architecture
- RFC 8725 và RFC 9068 — JWT Best Current Practices và JWT Access Token Profile

Bài tập: 8-0, 8-1, 8-2 ở phần Bài tập & case study cuối sách.

CHƯƠNG 9

Bảo mật Microservices — OAuth 2.0, mTLS & Zero Trust

> "Some organizations attempt to ensure security at the perimeter of their networks, and therefore assume they don't need to do anything else when two services are talking together. However, should an attacker penetrate your network, you will have little protection against a typical man-in-the-middle attack."
> — Sam Newman, *Building Microservices* [4a, Ch.9]

MỤC TIÊU HỌC TẬP

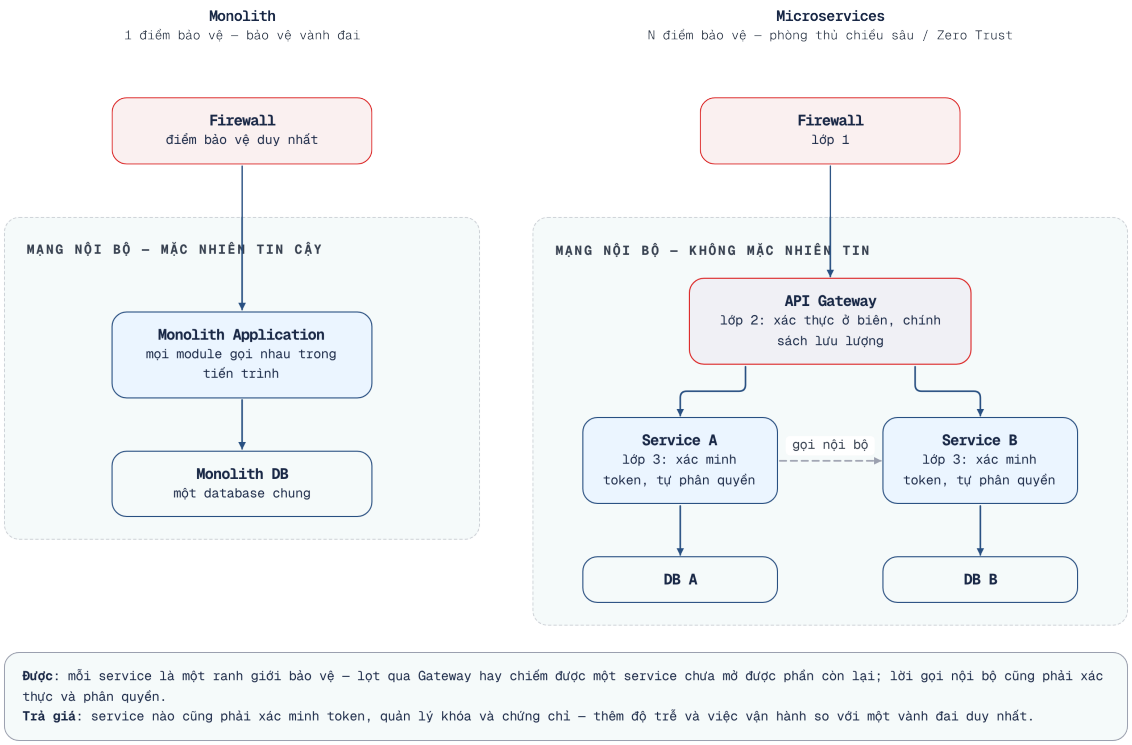
- Hiểu các thách thức bảo mật đặc thù của kiến trúc microservices
- Nắm vững cấu trúc JWT (JSON Web Token) dạng compact JWS: header, payload, signature — và phân biệt với dạng JWE năm phần khi cần mã hóa
- Phân biệt kiểm tra ở biên, xác minh tại service đích và đối chiếu trạng thái tại cơ sở dữ liệu; áp dụng tầng tin cậy (trust tier) theo mức rủi ro
- Hiểu OAuth2/OIDC (OpenID Connect) flow và tích hợp nhà cung cấp danh tính bên ngoài (Microsoft/Google)
- Thiết kế RBAC (Role-Based Access Control) cho hệ thống đa vai trò
- Tách biệt user subject, calling workload/actor và delegation trong giao tiếp service-to-service
- Phân tích kiến trúc bảo mật của KBM

9.1. Các thách thức bảo mật trong kiến trúc microservices

Việc bảo mật một hệ thống nguyên khối giống như bảo vệ một pháo đài duy nhất, nhưng với microservices, bài toán này biến thành việc phải bảo vệ từng tòa nhà, từng giảng đường nằm rải rác khắp khuôn viên đại học.

9.1.1. Vấn đề: bề mặt tấn công (attack surface) mở rộng

Trong kiến trúc nguyên khối (monolith), bảo mật tập trung tại một điểm: một ứng dụng, một cơ sở dữ liệu, một tầng xác thực (authentication layer). Khi chuyển sang microservices, bề mặt tấn công mở rộng tỷ lệ thuận với số lượng dịch vụ:



Hình 9.1: *Monolith (1 điểm bảo vệ) vs Microservices (N điểm bảo vệ)*

Tổng hợp từ *Building Microservices* của Sam Newman, có thể nêu năm thách thức bảo mật đặc thù khi chuyển dịch sang hệ thống phân tán, nơi sự tin tưởng mặc định không còn tồn tại và mỗi thành phần đều phải tự thiết lập cơ chế bảo vệ:

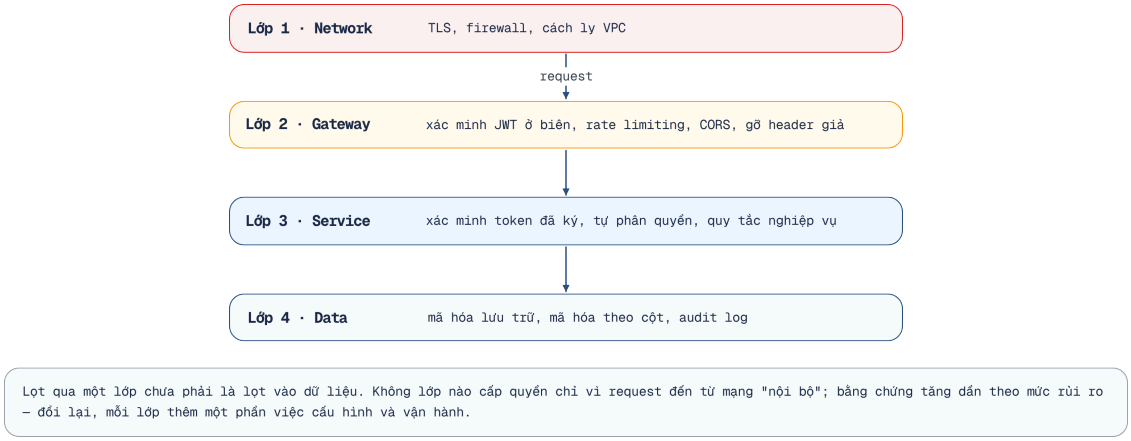
Bảng 9.1: *Năm thách thức bảo mật đặc thù khi chuyển từ monolith sang microservices*

#	Thách thức	Monolith	Microservices
1	**Xác thực**	Một session store	Gateway kiểm tra credential ở biên; resource service xác minh bằng chứng (proof) phù hợp với trust tier
2	**Phân quyền**	Kiểm tra cục bộ (in-process)	Mỗi dịch vụ cần phải kiểm tra quyền truy cập — người dùng được phép thực hiện những hành động nào?
3	**Tin cậy service-to-service**	Không có (in-process)	Dịch vụ B phải phân biệt workload A đang gọi, user subject (nếu có) và quyền delegation
4	**Dữ liệu trên đường truyền**	Nội bộ (in-process)	Lời gọi mạng — cần mã hóa (TLS)

#	Thách thức	Monolith	Microservices
5	**Secret Management**	Một file cấu hình	N dịch vụ kết hợp với M thông tin bí mật (secret) = bài toán quản lý phức tạp

9.1.2. Nguyên tắc phòng thủ chiều sâu

Để đối phó với các thách thức trên, bảo mật trong môi trường phân tán phải dựa trên nguyên tắc **phòng thủ chiều sâu** (defense in depth). Thay vì chỉ phụ thuộc vào một tường lửa duy nhất ở biên mạng, chúng ta phải xây dựng nhiều rào chắn bảo vệ nối tiếp nhau từ tầng mạng cho đến tận tầng dữ liệu:



Hình 9.2: *Phòng thủ chiều sâu — bảo vệ nhiều lớp từ tầng mạng đến tầng dữ liệu*

Trong thực tế, mức độ bảo vệ tùy thuộc vào **ranh giới tin cậy (trust boundary)**, nhưng vị trí trong private network/VPC không tự biến request thành đáng tin cậy. Gateway tạo một ranh giới vào (ingress) quan trọng; mỗi service đích vẫn là một ranh giới tài nguyên (resource boundary) vì service có dữ liệu và quy tắc phân quyền riêng.

Tin cậy phân tầng (Tiered Trust), không chỉ tin cậy ở biên

Zero Trust không có nghĩa là lặp lại truy vấn database ở mọi hop; nó yêu cầu không cấp quyền chỉ vì request đến từ một vị trí mạng được xem là "nội bộ". Thiết kế thực dụng là tăng bằng chứng theo mức rủi ro: làm sạch request ở biên (ingress sanitation), bằng chứng có chữ ký (signed proof) đúng audience, phân quyền cục bộ (local authorization), rồi bổ sung workload identity hoặc delegation khi cần.

Sách sử dụng bốn tầng tin cậy (trust tier) thống nhất. **Tier 0** là request ngoài chưa được tin cậy: Gateway phải làm sạch header và xác minh token. **Tier 1** là luồng nội bộ thông thường: resource service xác minh bằng chứng danh tính có chữ ký và tự phân quyền. **Tier 2** áp dụng cho nghiệp vụ nhạy cảm: ngoài user subject còn phải nhận diện calling workload/actor và delegation. **Tier 3** dành cho job hoặc command bất đồng bộ: consumer xác minh workload cùng quyền thực thi của job/command, không giả định rằng message broker đã tạo ra sự tin cậy.

9.1.3. mTLS, Secret Management và OAuth2 scope

Phần nâng cao (có thể lướt qua)

Ba chủ đề dưới đây (*mTLS*, *Secret Management*, *OAuth2 scope*) mở rộng cho các hệ thống có mức rủi ro cao hoặc tích hợp đối tác bên ngoài. Trong lần đọc đầu, người đọc có thể đọc lướt để nắm khái niệm và quay lại đối chiếu khi xem phần phân tích KBM ở §9.6.

Khi hệ thống mở rộng quy mô hoặc mức độ rủi ro tăng lên (ví dụ: bổ sung phân hệ thanh toán, xử lý dữ liệu cá nhân), cần nâng cấp các biện pháp bảo mật:

mTLS (Mutual TLS) — Trong TLS thông thường, chỉ có máy khách xác minh máy chủ (ví dụ trình duyệt xác minh chứng chỉ HTTPS). Trong mTLS, **cả hai bên đều xác minh chứng chỉ lẫn nhau**: khi dịch vụ A gọi dịch vụ B, B kiểm tra chứng chỉ của A và A kiểm tra chứng chỉ của B. Cơ chế này bảo vệ kênh truyền và xác thực workload ở hai đầu kết nối.

Khác với giao tiếp nội bộ qua HTTP không mã hóa, mTLS giảm nguy cơ nghe lén và giả mạo workload. Tuy nhiên, chứng chỉ của service A không chứng minh rằng claim `userId` do A chuyển tiếp là đúng; user subject vẫn cần signed proof hoặc delegation có thể xác minh. Cái giá phải đánh đổi là độ phức tạp khi cấp phát, luân phiên và thu hồi chứng chỉ; việc này thường cần công cụ như cert-manager hoặc service mesh.

Tại thời điểm khảo sát (đầu năm 2026), mTLS chưa được xác nhận là đã triển khai trong KBM. Đây là khoảng trống được chấp nhận tạm thời của một hệ thống học thuật, không phải lý do để coi mạng nội bộ là bằng chứng danh tính. Khi mức độ rủi ro tăng, mTLS cùng NetworkPolicy giúp thu hẹp khả năng di chuyển ngang (lateral movement); bằng chứng danh tính người dùng có chữ ký và phân quyền cục bộ vẫn là các lớp độc lập.

Secret Management — Các thông tin bí mật (secret như mật khẩu cơ sở dữ liệu, khóa API, khóa bí mật JWT) không nên được lưu trữ trực tiếp trong mã nguồn hoặc image Docker.

Trong việc quản lý thông tin nhạy cảm, gắn cố định (hardcode) secret trực tiếp vào mã nguồn (như `password: "abc123"`) là mức rủi ro cao nhất: một khi đã commit, secret nằm lại vĩnh viễn trong lịch sử Git. Cấp độ bảo mật trung bình là sử dụng biến môi trường (environment variable) qua file `.env`: phương án này tách bạch mã nguồn khỏi cấu hình, nhưng secret vẫn dễ bị đánh cắp nếu kẻ tấn công rà quét tiến trình chạy container. Giải pháp tiêu chuẩn cho hệ thống production là dùng công cụ quản lý secret chuyên dụng (Secret Manager) như HashiCorp Vault, cung cấp cơ chế mã hóa khi lưu trữ (encryption at rest), tự động luân phiên khóa và kiểm toán truy cập.

Tại thời điểm khảo sát, KBM dùng biến môi trường cho một phần cấu hình, còn một phần secret vẫn gắn cố định trong tệp cấu hình (§9.6). Dù ở hệ thống nào, khóa ký JWT cũng cần nằm ngoài mã nguồn và được luân phiên (rotation) định kỳ.

OAuth2 scope — Hiện tại RBAC của KBM dùng các vai trò (`STUDENT`, `LECTURER`, `ADMIN`). OAuth2 scope mở rộng thêm chiều kiểm soát: thay vì "người dùng có vai trò `ADMIN`", phạm vi quyền (scope) cho phép chỉ định "ứng dụng máy khách X có quyền đọc bài nộp nhưng không có quyền ghi điểm". Cơ chế này đặc biệt phù hợp khi KBM cung cấp API cho các bên thứ ba (như ứng dụng di động độc lập hoặc tích hợp với các hệ thống khác).

9.2. JWT — cấu trúc và cơ chế

9.2.1. Vấn đề: giới hạn khả năng mở rộng của phiên làm việc (session) trong microservices

Trong kiến trúc nguyên khối, quá trình xác thực thường sử dụng cơ chế phiên làm việc phía máy chủ (**server-side session**): người dùng đăng nhập, máy chủ tạo một phiên làm việc, lưu trữ trong bộ nhớ hoặc Redis, sau đó trả về mã định danh phiên (session ID) dưới dạng cookie. Các yêu cầu tiếp theo sẽ gửi kèm cookie này để máy chủ tra cứu và nhận diện người dùng.

Trong kiến trúc microservices, kho lưu trữ phiên (session store) tạo ra một điểm phụ thuộc tập trung (**centralized dependency**): việc mọi dịch vụ phải truy vấn cùng một cơ sở dữ liệu phiên sẽ gây ra điểm nghẽn và tạo thành điểm lỗi duy nhất (single point of failure). Ngược lại, nếu mỗi dịch vụ có một kho lưu trữ phiên riêng, người dùng sẽ phải đăng nhập lại mỗi khi chuyển đổi giữa các dịch vụ.

9.2.2. JWT (JSON Web Token)

Thi giấy vs thi máy tính

- **Session (Thi giấy)**: Giám thị phải mang theo một danh sách điểm danh. Mỗi khi có sinh viên nộp bài, giám thị phải tra đúng dòng, đúng tên trong danh sách (tra cứu session store). Nếu giám thị chuyển danh sách đi phòng khác, hệ thống sẽ rối loạn.
- **JWT (Thi máy tính)**: Giống như một **mã QR định danh** do hệ thống của Phòng Khảo thí cấp khi đăng nhập. Sinh viên cầm mã QR này nộp bài. Hệ thống chấm bài (service) chỉ cần quét xem mã QR có chữ ký điện tử hợp lệ của Phòng Khảo thí hay không và đọc (decode) được mã sinh viên mà không cần truy vấn cơ sở dữ liệu.

JWT giải quyết vấn đề này bằng cách đóng gói thông tin định danh *vào chính token* kèm chữ ký số — stateless, không cần kho lưu trữ phiên [2a, Ch.11]:

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.            ← Header
eyJ1c2VySWQiOiJ1c2VyLTEyMyIsInJvbGVzIjoiU1R       ← Payload
VREVOVCIsImV4cCI6MTg5MzQ1NjAwMH0.
SflKxwRJSMeKKF2QT4fwpMeJf36POk6yJV_adQssw5c       ← Signature
```

Ba phần của một JWT dạng compact JWS:

Hình 9.3: *Cấu trúc JWT dạng compact JWS — Header, Payload, Signature*

Về mặt cấu trúc, JWT minh họa ở đây là *compact JWS* — dạng phổ biến nhất — gồm ba phần được encode Base64URL riêng biệt (JWT dạng JWE mã hóa có năm phần). Phần đầu tiên là **Header**

chứa thông tin về thuật toán ký và loại token (ví dụ: `{"alg": "HS256"}`). Phần thứ hai là **Payload**, mang theo các dữ liệu định danh (claim) quan trọng của người dùng như `userId` hay `roles`. Thành phần cuối cùng là **Signature**, bảo vệ tính toàn vẹn của token bằng cách ký lên Header và Payload — HS256 dùng HMAC với khóa bí mật, RS256 ký bằng private key (bảng so sánh bên dưới) — giúp ngăn chặn việc chỉnh sửa dữ liệu trái phép.

Cần lưu ý rằng Base64URL chỉ là *encoding*, không phải mã hóa: bất kỳ ai cầm token đều đọc được toàn bộ payload. Vì vậy tuyệt đối không đặt mật khẩu hay dữ liệu nhạy cảm trong claim; khi thật sự cần giữ bí mật nội dung, hãy dùng JWE (JSON Web Encryption), lược bỏ claim nhạy cảm khỏi payload, hoặc chuyển sang opaque token — tùy mô hình mối đe dọa (threat model).

So sánh HS256 và RS256: Hai thuật toán phổ biến này khác nhau chủ yếu ở cách ký và cách xác minh chữ ký:

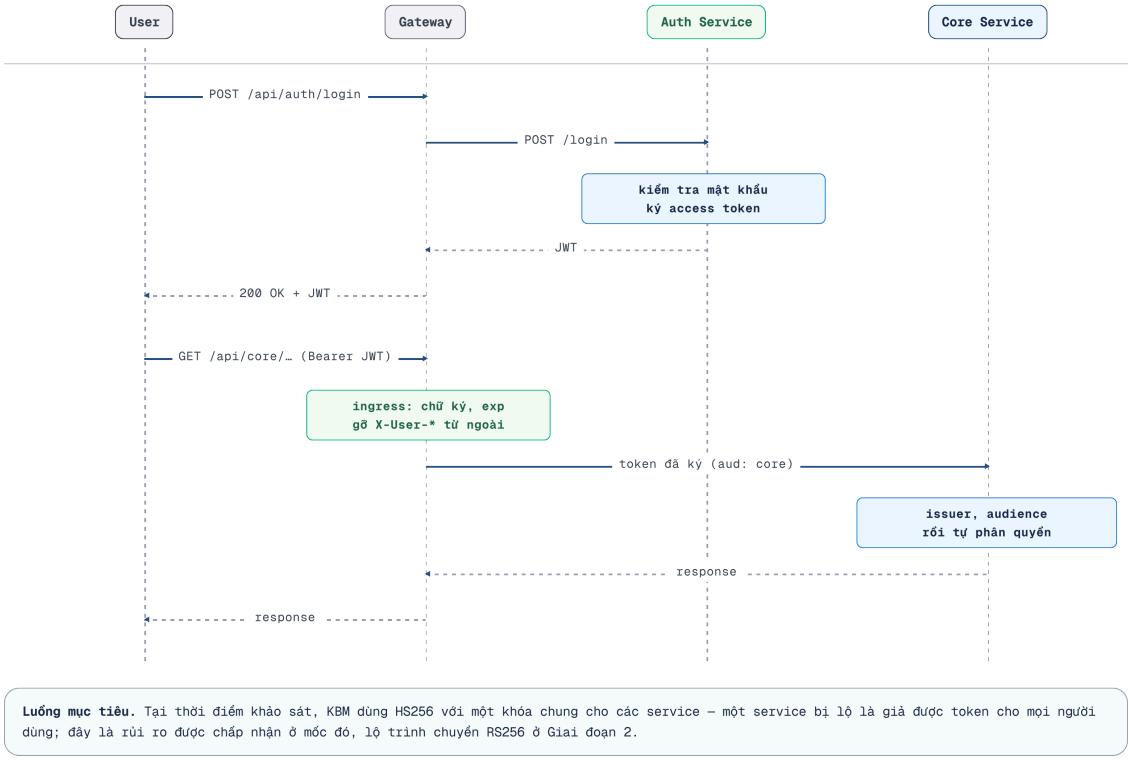
Bảng 9.2: *HS256 (đối xứng) vs RS256 (bất đối xứng)*

	HS256 (đối xứng)	RS256 (bất đối xứng)
Khóa	Khóa bí mật dùng chung (shared secret key)	Khóa riêng tư (ký) + Khóa công khai (xác minh)
Ai ký?	Auth Service (có khóa bí mật)	Auth Service (có khóa riêng tư)
Ai xác minh?	Bất kỳ dịch vụ nào có khóa bí mật	Bất kỳ dịch vụ nào có khóa công khai
Bảo mật	Nếu khóa bí mật (secret key) bị lộ, cả quá trình ký và xác minh đều bị xâm phạm	Khóa công khai phân phối rộng rãi không gây rủi ro; nhưng nếu khóa riêng tư (private key) bị lộ, kẻ tấn công ký được token giả mà mọi dịch vụ vẫn xác minh "hợp lệ" — phải thu hồi và luân phiên khóa khẩn cấp
Luân phiên khóa	Phải phân phối an toàn secret mới cho mọi bên xác minh (verifier)	Auth Service công bố khóa công khai mới qua JWKS; verifier tự làm mới cache
Phù hợp khi	Phạm vi hẹp, ít verifier và kiểm soát secret chặt	Nhiều resource service/verifier; chỉ issuer giữ private key

Dù dùng HS256 hay RS256, resource service không được suy ra tính hợp lệ chỉ từ việc decode thành công. Hợp đồng xác minh tối thiểu phải cố định thuật toán được phép, kiểm tra chữ ký, đối chiếu chính xác issuer và audience, rồi kiểm tra `exp` cùng `nbf` trước khi dùng claim để phân quyền. Token dành cho Gateway không mặc nhiên hợp lệ ở mọi dịch vụ tuyến sau (downstream service); nếu audience khác nhau, cần token exchange hoặc internal token riêng.

9.2.3. Luồng JWT mục tiêu và KBM tại thời điểm khảo sát

Hình 9.4 là luồng JWT mục tiêu: Gateway xác minh token ở biên, service đích nhận bằng chứng có chữ ký và tự phân quyền cục bộ. KBM tại thời điểm khảo sát còn cách mục tiêu này một khoảng; phần phân tích nằm ngay sau hình.



Hình 9.4: Luồng JWT mục tiêu — kiểm tra ở ingress, bằng chứng có chữ ký và phân quyền cục bộ

HS256 với shared secret trong KBM

Tại thời điểm khảo sát, KBM dùng HS256 (đối xứng) — các dịch vụ chia sẻ cùng một khóa bí mật. Đây là rủi ro đáng kể: nếu bất kỳ dịch vụ nào bị xâm nhập, kẻ tấn công có thể tạo lập chuỗi JWT giả mạo cho **bất kỳ người dùng nào trên toàn hệ thống** (không giới hạn ở dịch vụ bị tấn công). Với KBM học thuật, đây là **rủi ro được chấp nhận (accepted risk)** ở mốc đó, được ghi rõ, không phải mô hình mục tiêu cho production. **Lộ trình chuyển đổi** được xếp vào Giai đoạn 2 vì cần hạ tầng quản lý khóa: (1) Auth Service chuyển sang RS256 và là nơi duy nhất giữ private key; (2) các resource service dùng Spring Security OAuth2 Resource Server để xác minh theo cùng issuer/audience; (3) khóa công khai được luân phiên qua JWKS; (4) Gateway phát hành hoặc trao đổi sang internal token đúng audience khi không thể chuyển tiếp external token nguyên trạng.

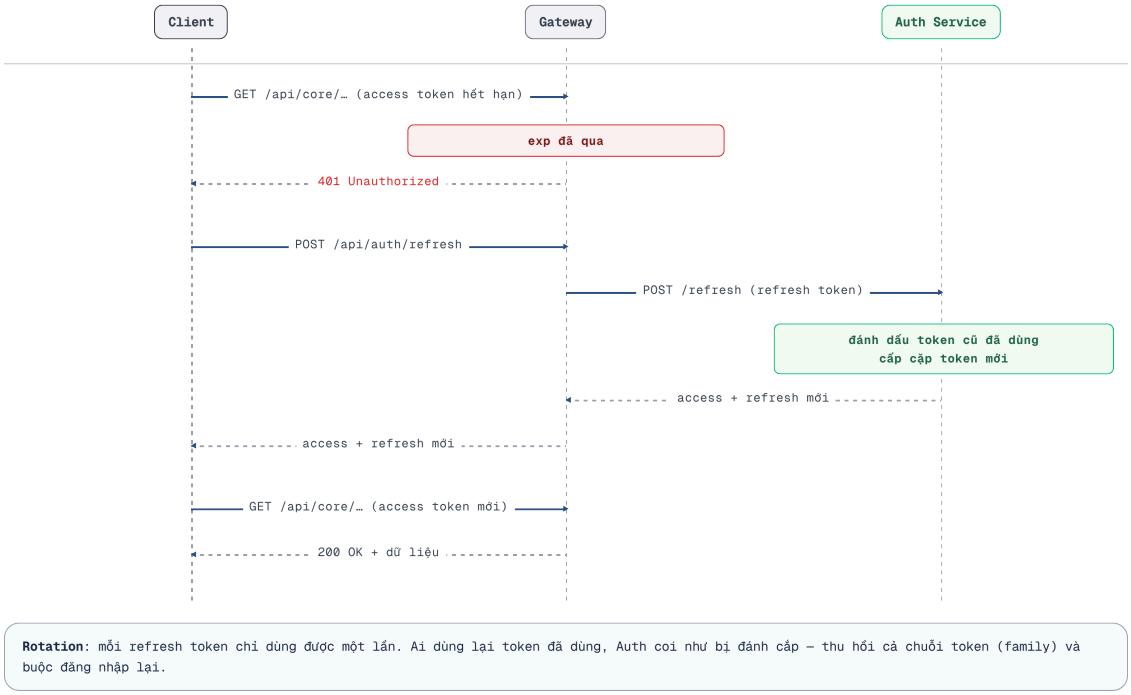
Việc luân phiên khóa (key rotation) trong thực tế vướng một rào cản kỹ thuật cụ thể.

Cửa sổ dual-secret khi luân phiên khóa HS256

Luân phiên khóa bí mật của HS256 không thể thực hiện tức thời: nếu Auth Service chuyển sang khóa bí mật mới trong khi các dịch vụ khác vẫn giữ khóa bí mật cũ, các token mới ký bằng khóa mới sẽ bị các dịch vụ đó từ chối; còn nếu gỡ khóa cũ quá sớm, các token cũ còn hạn sẽ bị từ chối. Cách an toàn là duy trì một **cửa sổ dual-secret**: trong giai đoạn chuyển tiếp, khâu *xác minh* của các dịch vụ sẽ chấp nhận **cả khóa cũ lẫn khóa mới** (`current` + `previous`), còn Auth Service sẽ *ký* bằng khóa mới; chỉ gỡ bỏ khóa cũ sau khi mọi token được ký bằng nó đã hoàn toàn hết hạn. Sự phức tạp này là một lý do nữa để chuyển sang cơ chế RS256/JWKS — khi đó chỉ cần công bố

thêm khóa công khai mới qua endpoint JWKS, không cần phải đồng bộ khóa bí mật qua tất cả các dịch vụ.

JWT có thời hạn (`exp`). Khi access token hết hạn, người dùng phải **xác thực lại (re-authenticate)** — điều này làm trải nghiệm kém đi ở các phiên làm việc kéo dài (ví dụ: giảng viên sử dụng hệ thống liên tục suốt một buổi giảng). **Refresh token** giải quyết bài toán này: access token có thời hạn ngắn (15–60 phút), còn refresh token có thời hạn dài hơn (7–30 ngày). Khi access token hết hạn, client dùng refresh token để nhận access token mới — không cần đăng nhập lại.



Hình 9.5: Luồng làm mới và luân phiên token — access token mới + refresh token mới

Token rotation là chiến lược luân phiên refresh token: mỗi lần refresh token được sử dụng, **bản thân nó cũng được thay mới** và bản cũ bị vô hiệu hóa. Lợi ích của cơ chế này là: nếu refresh token bị đánh cắp và dùng lại trái phép, Auth Service sẽ phát hiện token đó đã qua sử dụng. Khi đó, hệ thống sẽ tự động vô hiệu hóa toàn bộ chuỗi token liên quan và buộc người dùng phải đăng nhập lại từ đầu (RFC 9700).

Đăng ký vé dùng máy tính thư viện

- **Access token**: tương tự *vé sử dụng máy tính thư viện 1 giờ* (ngắn hạn để nếu lộ thông tin vé, kẻ gian chỉ dùng được tối đa trong 1 giờ).
- **Refresh token**: tương tự *thẻ sinh viên gốc*, dùng để xuất trình cho thủ thư cấp vé 1 giờ mới khi hết hạn.
- **Token rotation**: mỗi lần xin vé mới, thủ thư dán *tem gia hạn mới* lên thẻ và hủy hiệu lực tem cũ — refresh token vì thế đổi mới sau mỗi lần dùng. Nếu kẻ gian mang tem cũ đã hủy ra dùng lại, hệ thống phát hiện ngay và **thu hồi toàn bộ chuỗi tem** của thẻ đó.

Listing 9.1: *Endpoint làm mới token có luân phiên*

```
// Auth Service — Token refresh với rotation
// @Transactional: đánh dấu token cũ đã dùng + phát hành token mới
// trong CÙNG transaction.
// noRollbackFor: lệnh thu hồi family phải được COMMIT ngay cả khi
// chúng ta chủ động ném TokenReuseException để từ chối request
// (mặc định Spring rollback theo RuntimeException)
@Transactional(noRollbackFor = TokenReuseException.class)
@PostMapping("/refresh")
public TokenResponse refreshToken(@RequestBody RefreshRequest request) {
    RefreshToken stored = refreshTokenRepository
        .findByToken(request.getRefreshToken())
        .orElseThrow(() -> new AuthException("Invalid refresh token"));

    // Kiểm tra thu hồi và dùng lại TRƯỚC hạn dùng: token cũ đã hết hạn
    // mà bị dùng lại vẫn phải kích hoạt thu hồi cả chuỗi (RFC 9700)
    if (stored.isRevoked()) {
        throw new AuthException("Refresh token revoked");
    }
    if (stored.isUsed()) {
        refreshTokenRepository.invalidateFamily(stored.getFamily());
        throw new TokenReuseException("Token reuse detected — please login again");
    }

    // Kiểm tra hết hạn
    if (stored.isExpired()) {
        throw new AuthException("Refresh token expired");
    }

    // Cập nhật atomic, bắt trường hợp hai request dùng cùng token
    // UPDATE refresh_tokens SET used = true
    // WHERE id = ? AND used = false AND revoked = false
    int updatedRows = refreshTokenRepository.markAsUsedAtomic(stored.getId());
    if (updatedRows == 0) {
        // Request khác vừa dùng token này: coi là dùng lại,
        // vô hiệu hóa toàn bộ chuỗi token
        refreshTokenRepository.invalidateFamily(stored.getFamily());
        throw new TokenReuseException("Token reuse detected — please login again");
    }

    // Phát hành cặp token mới thuộc cùng một chuỗi (family)
    User user = stored.getUser();
    String newAccessToken = jwtUtil.generateAccessToken(user);  // 30 phút
    String newRefreshToken = createRefreshToken(user, stored.getFamily()); // 7 ngày

    return new TokenResponse(newAccessToken, newRefreshToken);
}
```

Có ba chi tiết triển khai dễ bị bỏ sót. Thứ nhất, ví dụ trên lưu token thô và nhận refresh token qua JSON body cho dễ đọc. Ở production nên lưu *hash* của refresh token (tương tự cách lưu mật khẩu) và so khớp qua hash; với client web giữ refresh token trong cookie `HttpOnly` (Bảng 9.3), endpoint đọc token bằng `@CookieValue` và trả token mới qua header `Set-Cookie`, vì JavaScript không đọc được cookie này.

Thứ hai, cặp thao tác "đánh dấu token cũ đã dùng — phát hành token mới" phải nằm trong cùng một transaction. Riêng nhánh phát hiện token bị dùng lại thì lệnh thu hồi token family phải được

commit chứ không bị rollback theo exception từ chối request — đó là lý do có `noRollbackFor` trong ví dụ; phương án khác là tách việc thu hồi vào một transaction `REQUIRES_NEW`.

Thứ ba, khi hai yêu cầu làm mới hợp lệ chạy song song, yêu cầu đến sau sẽ kích hoạt nhầm cảnh báo reuse. Hệ thống cần một chính sách rõ ràng cho tình huống này, chẳng hạn một khoảng ân hạn ngắn, để không vô hiệu hóa nhầm token vừa cấp.

Token rotation yêu cầu đặt thời gian sống (TTL) khác nhau cho từng loại token:

Bảng 9.3: *Access token, refresh token và token family — thời hạn, nơi lưu trữ và lý do thiết kế*

Token	Thời hạn	Lưu ở đâu	Lý do
Access token	15–60 phút	Bộ nhớ client/cookie	Thời gian sống ngắn giúp giảm thiểu rủi ro nếu token bị rò rỉ
Refresh token	7–30 ngày	HttpOnly cookie + DB	Thời gian sống dài duy trì trải nghiệm liền mạch, lưu ở DB để có thể thu hồi
Token family	Theo refresh token	DB (family ID)	Giúp phát hiện việc tái sử dụng trái phép và thu hồi toàn bộ chuỗi

9.3. Tiered Trust và xác minh theo tài nguyên

Bài toán không phải chọn một trong hai cực "mọi service đều truy vấn cơ sở dữ liệu" hoặc "chỉ Gateway kiểm tra token". Cần tách ba thao tác: Gateway kiểm tra credential ở ingress; resource service xác minh bằng chứng có chữ ký dành cho mình; chỉ những luồng cần trạng thái mới truy vấn Auth Service hoặc kho thu hồi (revocation store).

Cổng trường, giấy giới thiệu và phòng chức năng

- **Gateway**: giống trạm kiểm soát cổng trường — kiểm tra credential ngoài, loại bỏ giấy tờ do người vào tự sửa và chỉ cho request hợp lệ đi tiếp.
- **Resource service**: giống phòng chức năng — kiểm tra con dấu cùng tên phòng nhận trên giấy giới thiệu, rồi tự quyết định người đó có quyền xem bảng điểm, sửa đề hay duyệt kết quả hay không. Với nghiệp vụ nhạy cảm, phòng còn kiểm tra người mang giấy đang đại diện cho ai.

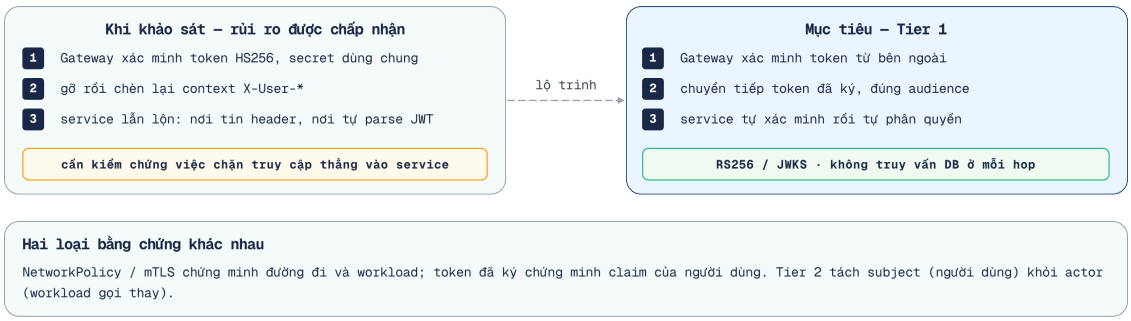
9.3.1. Vấn đề: ba phép kiểm tra thường bị gộp làm một

Xác minh mật mã (cryptographic verification) kiểm tra thuật toán, chữ ký, issuer, audience và thời hạn; thao tác này stateless và không đòi hỏi truy vấn cơ sở dữ liệu. **Kiểm tra trạng thái (state check)** hỏi tài khoản còn hoạt động hay token đã bị thu hồi; nó chỉ cần ở login, refresh hoặc thao tác nhạy cảm theo chính sách. **Phân quyền (authorization)** là quyết định nghiệp vụ trên tài nguyên cụ thể và luôn thuộc service sở hữu tài nguyên.

Nếu chỉ Gateway kiểm tra rồi thay token bằng `X-User-*`, service đích không còn bằng chứng mật mã khi có route bypass hoặc workload bị chiếm quyền. Ngược lại, nếu mỗi service tự viết verifier và tự truy vấn cơ sở dữ liệu người dùng ở mọi request, logic vừa trùng lặp vừa tạo coupling. Tiered Trust giữ bước xác minh tại resource service nhưng chuẩn hóa bước đó qua framework, issuer metadata và JWKS.

9.3.2. KBM tại thời điểm khảo sát và mục tiêu

Hình 9.6 đặt luồng định danh của KBM tại thời điểm khảo sát, với các rủi ro được chấp nhận khi đó, cạnh mục tiêu Tier 1.



Hình 9.6: *Luồng định danh KBM — rủi ro được chấp nhận tại thời điểm khảo sát và mục tiêu Tier 1*

Cả luồng tại thời điểm khảo sát lẫn mục tiêu đều được đánh giá theo bốn tầng tin cậy tóm tắt ở Bảng 9.4.

Bảng 9.4: *Bốn tầng tin cậy dùng xuyên suốt sách*

Tầng	Ranh giới	Bằng chứng	Quyết định tại service đích
T0	Client ngoài → Gateway	External token; làm sạch mọi header định danh	Gateway kiểm tra `alg`, chữ ký, `iss`, `aud`, `exp`, `nbf`
T1	Gateway/service → resource service	Signed token hoặc metadata đúng audience	Xác minh bằng chứng rồi phân quyền cục bộ
T2	Luồng nhạy cảm service-to-service	User subject + workload/actor + delegation; mTLS khi phù hợp	Giới hạn cả quyền người dùng lẫn quyền workload
T3	Producer/job → broker → consumer	Workload identity + quyền job/command + độ mới (freshness)	Consumer xác minh ngữ cảnh và phân quyền cho command

Thời điểm khảo sát — rủi ro được chấp nhận: Gateway xác minh HS256, loại bỏ rồi chèn lại header định danh; các service còn dùng lẫn lộn hai cách, tin vào header và tự phân tích JWT, nên chưa đủ căn cứ để coi mọi đường gọi nội bộ đã được kiểm soát. Vì vậy sách không gọi thiết kế này là Zero Trust hay phòng thủ chiều sâu.

Mục tiêu Tier 1: Gateway xác minh token ngoài, rồi giữ bằng chứng có chữ ký trong giai đoạn chuyển tiếp, hoặc dùng token exchange để đổi sang token nội bộ đúng audience. Mỗi resource service dùng verifier chuẩn hóa để kiểm tra bằng chứng và tự phân quyền. Auth Service chỉ kiểm tra trạng thái qua cơ sở dữ liệu khi đăng nhập, làm mới token, thu hồi token hoặc khi có thao tác nhạy cảm; không bắt buộc truy vấn cơ sở dữ liệu ở mọi hop.

⚠ Bảo mật header và truyền định danh (Identity Propagation)

Gateway bắt buộc phải loại bỏ mọi `X-User-*` do client gửi trước khi dẫn xuất context mới. Tuy nhiên, việc làm sạch header chỉ chặn spoofing tại ingress; một header văn bản thuần vẫn không phải bằng chứng danh tính. Service đích chỉ được ra quyết định dựa trên signed token/metadata đã kiểm tra issuer, audience và độ mới. NetworkPolicy giới hạn đường đi; mTLS xác thực workload; hai lớp này không thay thế chữ ký cho claim của người dùng.

Với machine-to-machine, Client Credentials đại diện cho **calling workload/actor**, không tự tạo ra user subject. Nếu service gọi thay mặt người dùng, bằng chứng phải mang subject và actor/delegation tách biệt để service đích giới hạn đồng thời quyền của cả hai. Job không có người dùng thì không được tự gắn `X-User-Id` giả để mượn quyền người dùng.

Gateway chịu trách nhiệm xác thực ở ingress nhưng không sở hữu toàn bộ quyết định tin cậy. Resource service xác minh bằng chứng tại ranh giới của mình và chịu trách nhiệm phân quyền nghiệp vụ. Sự phân công này giữ các quy tắc nghiệp vụ ngoài Gateway mà không buộc service tin mù quáng vào vị trí mạng hoặc định dạng header.

9.4. OAuth2 — nhà cung cấp danh tính bên ngoài

Ủy quyền xác thực (delegated authentication) cho các nhà cung cấp danh tính (identity provider) bên ngoài giúp giảm thiểu rủi ro bảo mật từ việc tự quản lý mật khẩu, đồng thời mang lại trải nghiệm đăng nhập một lần (SSO) liền mạch cho người dùng.

Vấn đề quản lý thông tin đăng nhập (credential): Tự quản lý thông tin đăng nhập đòi hỏi phải xử lý nhiều việc phức tạp: băm mật khẩu, cấp lại mật khẩu đã quên, xác thực đa yếu tố (2FA), chống tấn công dò mật khẩu (brute-force protection) và tuân thủ các quy chuẩn bảo mật. Với đội ngũ nhỏ, **ủy quyền xác thực** cho nhà cung cấp danh tính chuyên nghiệp (Google, Microsoft, GitHub) giúp giảm đáng kể công sức và rủi ro.

9.4.1. OAuth2 Authorization Code Flow

Đăng ký ký túc xá bằng tài khoản Cổng thông tin đào tạo

Hãy tưởng tượng Ban quản lý Ký túc xá cần xác nhận một sinh viên đang theo học trước khi xếp phòng. Ký túc xá (client) không được phép hỏi mật khẩu Cổng thông tin đào tạo của sinh viên (rất nguy hiểm). Thay vào đó, Ký túc xá chuyển hướng sinh viên sang Cổng thông tin đào tạo của trường để tự đăng nhập. Sau khi đăng nhập, trường cấp cho sinh viên một "Phiếu xác nhận" (Authorization Code) chỉ dùng được 1 lần. Hệ thống backend của Ký túc xá mang phiếu đó đến thẳng Phòng Đào tạo để đổi lấy "Giấy phép tra cứu hồ sơ" (Token) — có thời hạn và chỉ cho xem đúng những thông tin cần để xếp phòng.

Cần phân biệt hai lớp giao thức thường đi đôi với nhau: **OAuth 2.0** vốn là khung *ủy quyền truy cập* (cấp `Access Token` để gọi API thay mặt người dùng), còn **OIDC (OpenID Connect)** là lớp *xác thực danh tính* xây chồng lên OAuth 2.0, bổ sung thêm `ID Token` (một JWT có chữ ký của nhà cung cấp danh tính) để cho ứng dụng biết chính xác người dùng vừa đăng nhập là ai.

KBM tích hợp OAuth2/OIDC để thực hiện đăng nhập thông qua nhà cung cấp danh tính của trường (ví dụ: Microsoft Entra ID hoặc Google Workspace). Quy trình hoạt động như sau: người dùng được chuyển hướng tới nhà cung cấp danh tính để đăng nhập. Khi đăng nhập thành công, KBM nhận về authorization code qua callback. Tiếp đó, Auth Service sử dụng mã này để đổi lấy `ID Token` và `Access Token` thông qua giao tiếp trực tiếp giữa các máy chủ (server-to-server). Cuối cùng, dịch vụ xác minh `ID Token`, tìm kiếm hoặc tạo mới tài khoản tương ứng và phát hành JWT nội bộ của KBM. Với client không giữ được bí mật như SPA hay ứng dụng di động, **PKCE** bảo vệ authorization code: client tạo một chuỗi ngẫu nhiên, gửi bản băm của nó khi xin mã, rồi gửi chuỗi gốc khi đổi mã lấy token. Kẻ chặn được mã không có chuỗi gốc nên không đổi được token.

Điểm quan trọng: KBM *không sử dụng* trực tiếp token của bên thứ ba. Sau khi xác thực thành công, Auth Service sẽ tạo ra một **JWT nội bộ riêng của KBM** — các dịch vụ tuyến sau hoàn toàn không cần biết người dùng đã đăng nhập bằng Microsoft, Google, hệ thống Quản lý Đào tạo hay qua mật khẩu truyền thống. Cách làm này gần với ý tưởng token exchange — đổi danh tính bên ngoài lấy token nội bộ — dù không dùng giao thức RFC 8693.

9.4.2. Nhiều phương thức xác thực

KBM hỗ trợ nhiều phương thức xác thực nhưng hội tụ về cùng một token nội bộ:

__Bảng 9.5:__ Các phương thức xác thực trong KBM

Phương thức	Luồng	Khi nào
Tên đăng nhập/mật khẩu	Truyền thống, hash BCrypt	Mặc định cho mọi người dùng
Microsoft/Google OAuth2/OIDC	Authorization Code Flow + PKCE (kèm `state` chống CSRF và `nonce` chống phát lại `ID Token`)	SSO bằng tài khoản trường
Cổng QLĐT (Quản lý Đào tạo)	Tích hợp tùy biến	Đăng nhập bằng tài khoản quản lý đào tạo
Xác minh email	Xác minh quyền sở hữu email trước khi kích hoạt/khôi phục	Giảm tài khoản giả, hỗ trợ cấp lại mật khẩu
Turnstile	Challenge chống bot ở biểu mẫu nhạy cảm	Bảo vệ đăng nhập, đăng ký và cấp lại mật khẩu khỏi các công cụ tự động

Tất cả các phương thức đăng nhập này đều hội tụ về một đầu ra duy nhất: **JWT nội bộ của KBM** (chứa `userId`, `roles` và `exp`). Auth Service cung cấp hàm `generateTokens(user)` — tiếp nhận đối tượng người dùng từ bất kỳ phương thức đăng nhập nào và trả về cặp `{accessToken, refreshToken}`. Các dịch vụ ở tuyến sau (downstream service) không cần phân biệt nguồn gốc xác thực. Đây là cơ sở cho cơ chế đăng nhập một lần (SSO) đa nền tảng: các module LMS core, Network Lab và DevOps Lab có thể chia sẻ danh tính nội bộ mà không buộc từng module hiểu mọi nhà cung cấp bên ngoài.

9.4.3. Service-to-service: Client Credentials Flow

Lưu ý: luồng Authorization Code dành cho tương tác có mặt người dùng (user-facing). Khi một dịch vụ chạy nền tự gọi dịch vụ khác (machine-to-machine, ví dụ Job Scheduler gọi Judge Service), nó sử dụng **Client Credentials Flow** với credential riêng để nhận access token. Token này nhận diện workload/actor và các scope được cấp cho workload; nó *không được* tự mang user subject. Nếu tác vụ thực sự thực hiện thay mặt người dùng, hệ thống cần một cơ chế delegation hoặc token exchange ghi rõ cả subject lẫn actor, đúng audience của service đích và có thời hạn ngắn.

9.4.4. Không có trình duyệt: Device Authorization Grant

Authorization Code giả định client có thể đưa người dùng qua trình duyệt và nhận callback; Client Credentials không cần trình duyệt nhưng chỉ đại diện cho ứng dụng. Một công cụ dòng lệnh cho quản trị viên, một script chạy trên máy chủ không giao diện, hay một ứng dụng trên thiết bị chỉ có bàn phím số không mở được trình duyệt, nhưng vẫn cần đăng nhập *với tư cách một người dùng cụ thể* chứ không phải với tư cách một service.

Device Authorization Grant (RFC 8628) tách đôi quá trình: thiết bị không có trình duyệt hiển thị một mã ngắn kèm một URL; người dùng mở URL đó trên *thiết bị khác* (điện thoại, máy tính), đăng nhập và nhập mã để chấp thuận; trong lúc đó thiết bị ban đầu hỏi vòng (poll) authorization server cho tới khi nhận được token. Nhờ vậy thông tin đăng nhập không bao giờ đi qua thiết bị hạn chế.

Điểm cần nhớ khi chọn luồng: câu hỏi quyết định không phải "client thuộc loại nào" mà là hai câu — *có người dùng thật đứng sau không?* và *client có mở được trình duyệt không?* Có người dùng và có

trình duyệt thì dùng Authorization Code kèm PKCE (bắt buộc với client công khai, khuyến nghị cả với client có máy chủ theo RFC 9700); có người dùng nhưng không có trình duyệt thì dùng Device Authorization Grant; không có người dùng nào cả thì đó là Client Credentials.

9.5. RBAC — Role-Based Access Control

Phân quyền giống như việc cấp thẻ từ cho các khu vực khác nhau trong trường: thẻ của sinh viên chỉ mở được cửa thư viện và giảng đường, trong khi thẻ của giảng viên có thể mở thêm được cửa phòng thí nghiệm chuyên sâu. Việc quản lý truy cập theo vai trò giúp hệ thống dễ dàng kiểm soát ai được phép làm gì.

Trong phép so sánh này, xác thực (Authentication — AuthN) là trình thẻ để chứng minh "Tôi là sinh viên Nguyễn Văn A"; phân quyền (Authorization — AuthZ) là ổ khóa điện tử kiểm tra xem thẻ của Nguyễn Văn A có vai trò `LECTURER` để mở cửa phòng nghỉ giảng viên hay không.

9.5.1. Vấn đề: ai được làm gì?

Xác thực trả lời "người dùng là ai?". Phân quyền trả lời "người dùng được làm gì?". Về nghiệp vụ, ba vai trò của KBM lồng nhau: **STUDENT** (Sinh viên) nộp bài, xem kết quả và tham gia kỳ thi; **LECTURER** (Giảng viên) cần thêm quyền tạo bài tập, khởi tạo kỳ thi và xem báo cáo thống kê; **ADMIN** thêm quyền quản lý tài khoản và cấu hình hệ thống. Tại thời điểm khảo sát, KBM chưa khai báo quan hệ kế thừa này trong Spring Security (xem phân tích bên dưới).

9.5.2. Triển khai RBAC trong KBM

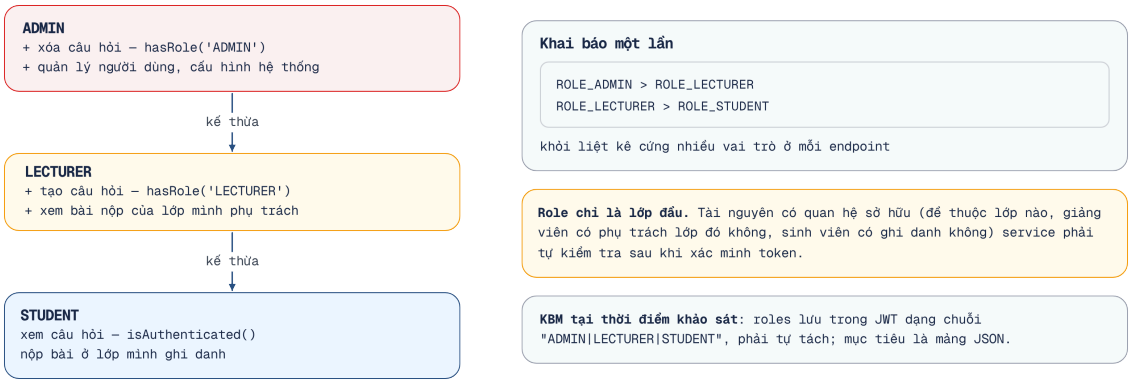
KBM dùng annotation `@PreAuthorize` của Spring Security với các vai trò lưu trong JWT:

Listing 9.2: Spring Security `@PreAuthorize` — RBAC dạng khai báo (declarative)

```
// Spring Security @PreAuthorize — declarative RBAC
@PreAuthorize("isAuthenticated()")
@GetMapping("/questions")
public List<QuestionResponse> getQuestions() { ... }  // Mọi user authenticated

@PreAuthorize("hasAnyRole('LECTURER', 'ADMIN')")
@PostMapping("/questions")
public QuestionResponse create(@RequestBody CreateQuestionRequest r) { ... }

@PreAuthorize("hasRole('ADMIN')")
@DeleteMapping("/questions/{id}")
public void delete(@PathVariable UUID id) { ... }
```

Kiểm tra vai trò (role check) chỉ là lớp đầu của phân quyền cục bộ. Với tài nguyên có quan hệ sở hữu (ownership) — chẳng hạn đề thi thuộc lớp nào, giảng viên có phụ trách lớp đó không, sinh viên có ghi danh hay không — service phải kiểm tra quan hệ với chính tài nguyên đó sau khi đã xác minh bằng chứng danh tính có chữ ký. Gateway không có đủ ngữ cảnh miền nghiệp vụ để quyết định thay service.

Phân cấp vai trò (role hierarchy): Sơ đồ dưới đây minh họa sự kế thừa quyền hạn giữa các vai trò:



Hình 9.7: *Phân cấp vai trò — ADMIN kế thừa LECTURER, LECTURER kế thừa STUDENT*

Để cấu hình `RoleHierarchy` cho `@PreAuthorize` và tránh phải kiểm tra cứng nhiều vai trò, chúng ta khai báo một `@Bean` (Listing 9.3). Method security chỉ tự dùng bean này từ Spring Security 6.3 (Spring Boot 3.3) trở đi, khi bật `@EnableMethodSecurity`:

Listing 9.3: *Cấu hình phân cấp vai trò `RoleHierarchy` trong Spring Security*

```
@Bean
public RoleHierarchy roleHierarchy() {
    return RoleHierarchyImpl.fromHierarchy(
            "ROLE_ADMIN > ROLE_LECTURER\nROLE_LECTURER > ROLE_STUDENT");
}
```

Tại thời điểm khảo sát, KBM lưu danh sách vai trò dạng **pipe-delimited** trong JWT: `"ADMIN|LECTURER|STUDENT"`; Gateway còn dẫn xuất `X-User-Roles` và Core Service phân tích chuỗi này. Ở mô hình mục tiêu, service lấy vai trò/quyền hạn từ token hoặc metadata có chữ ký đã được xác minh; header dẫn xuất chỉ là context, không phải nguồn quyền độc lập.

9.5.3. Điều khiển hiển thị route từ API quyền (frontend)

Frontend của KBM sử dụng một pattern bổ trợ: **danh sách quyền hiển thị route được truy xuất động từ API** (`GET /api/auth/me/permissions`) thay vì gắn cố định — response chứa danh sách route và thao tác mà người dùng được phép thấy trên giao diện. Ưu điểm của cách làm này là khi điều chỉnh phân quyền ở phía máy chủ (backend), thanh điều hướng và giao diện máy khách tự động thích ứng mà không cần triển khai lại frontend. Tuy nhiên, việc ẩn/hiện route trên giao diện chỉ phục vụ trải nghiệm người dùng (UX), hoàn toàn *không phải* rào chắn bảo mật: người dùng vẫn có thể gọi thẳng HTTP API, nên mọi kiểm tra phân quyền bắt buộc phải được thực thi độc lập tại backend.

RBAC trong KBM

Cách triển khai của KBM tại thời điểm khảo sát có một số hạn chế: (1) Các vai trò (role) được lưu dưới dạng chuỗi phân tách bằng dấu gạch đứng (pipe-delimited string) thay vì cấu trúc mảng. Điều này buộc hệ thống phải tự phân tích chuỗi, dễ phát sinh lỗi nếu tên vai trò có chứa ký tự phân cách. (2) Các annotation `@PreAuthorize` nằm rải rác ở từng controller — gây khó khăn cho quá trình kiểm toán (audit) quyền hạn của một vai trò cụ thể trên toàn bộ các endpoint. (3) KBM thiếu một hệ thống phân cấp vai trò (role hierarchy) rõ ràng ở tầng Spring Security — dẫn đến

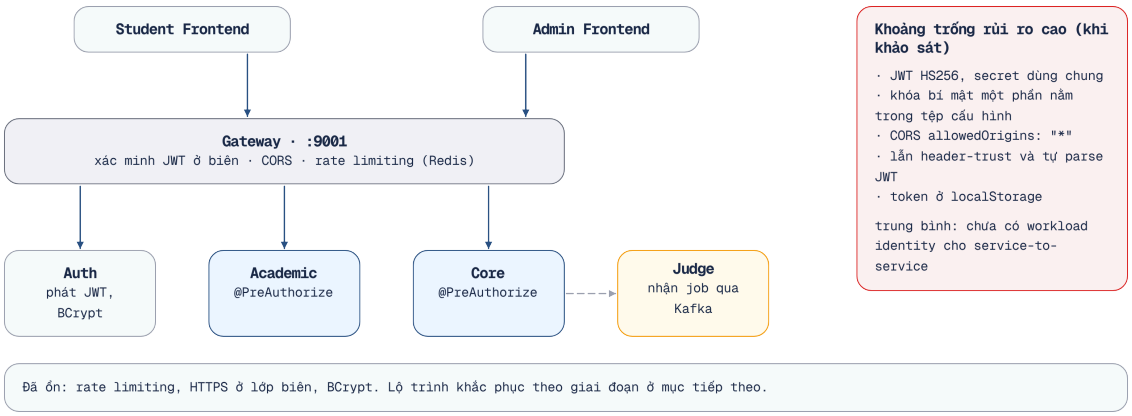
việc vai trò ADMIN phải liệt kê lại toàn bộ các quyền của LECTURER và STUDENT. (4) Phân hệ DevOps Lab cần bổ sung các chính sách kiểm soát tại tầng thực thi (runtime layer) để xác định rõ người dùng nào được phép khởi tạo không gian làm việc, vùng mạng cách ly, container hoặc các công việc đòi hỏi đặc quyền cao.

Lộ trình chuyển đổi: (1) chuyển cách lưu vai trò sang mảng JSON bên trong JWT payload, (2) thiết lập cấu hình `RoleHierarchy` thống nhất trong Spring Security, và (3) cân nhắc gom quy tắc phân quyền cấp endpoint về một chỗ (`authorizeHttpRequests` hoặc một `AuthorizationManager` dùng chung trong Spring Security), hoặc tách sang một công cụ chính sách chuyên dụng như Open Policy Agent khi số quy tắc lớn.

9.6. Case study: KBM

Phần này tổng hợp đánh giá kiến trúc bảo mật của KBM tại thời điểm khảo sát (đầu năm 2026), nhận diện các điểm yếu và đề xuất lộ trình khắc phục theo từng giai đoạn. Các điểm nêu ra mô tả mốc đó để rút bài học, không dùng để suy ra cấu hình đang vận hành.

Hình 9.8 tóm tắt kiến trúc bảo mật tổng thể của KBM.



Hình 9.8: *Kiến trúc bảo mật tổng thể của KBM*

9.6.1. Phân tích tổng hợp

Bảng dưới đây tổng hợp các khía cạnh bảo mật, rủi ro tương ứng và đề xuất biện pháp phòng ngừa:

Bảng 9.6: *Phân tích bảo mật KBM*

Khía cạnh	Tại thời điểm khảo sát	Mức rủi ro	Giải pháp khuyến nghị
Thuật toán JWT	HS256 (đối xứng, shared secret)	● Cao	RS256 + JWKS; chỉ issuer giữ private key
Lưu trữ token	Client-side (`localStorage`)	● Cao	HttpOnly cookie hạn chế JavaScript lấy token; vẫn phải phòng XSS và CSRF
Secret Management	Một phần gắn cố định trong tệp cấu hình	● Cao	Vault, AWS Secrets Manager
CORS	`allowedOrigins: "*"`	● Cao	Chỉ cho phép các origin cụ thể
Truyền định danh	Lẫn header-trust và tự phân tích JWT rời rạc	● Cao	Bằng chứng có chữ ký đúng audience + phân quyền cục bộ

Khía cạnh	Tại thời điểm khảo sát	Mức rủi ro	Giải pháp khuyến nghị
Xác thực service-to-service	Chưa có workload identity thống nhất	● Trung bình	Workload token/mTLS; delegation khi có user subject
Rate Limiting	Đã có: token bucket (Redis) ở Gateway (Chương 8)	● Thấp	Chính sách riêng cho kỳ thi
Xác minh email / Chống bot	Có/đang hoàn thiện theo module	● Trung bình	Xác minh email + Turnstile cho biểu mẫu nhạy cảm
Cô lập DevOps Lab	Runtime isolation riêng	● Cao nếu cấu hình sai	Sysbox/k3s + NetworkPolicy/RBAC
Kiểm tra tính hợp lệ dữ liệu đầu vào	Một phần	● Trung bình	Sử dụng Bean Validation (`@Valid`)
HTTPS	Kết thúc TLS ở lớp biên (reverse proxy hoặc Gateway)	● Thấp	Ghi rõ điểm kết thúc TLS trong tài liệu triển khai; cân nhắc mTLS nội bộ khi rủi ro tăng
Băm mật khẩu	BCrypt	● Thấp	—

Với HS256 mà KBM dùng tại thời điểm khảo sát, ràng buộc về độ dài khóa thường bị bỏ qua.

⚠ Độ dài khóa HS256

Với thuật toán HS256, khóa bí mật bắt buộc phải là một chuỗi ngẫu nhiên có độ dài tối thiểu 256 bit (32 byte). Các thư viện JWT hiện đại (như `jjwt` bản mới) sẽ ném ra `WeakKeyException` lúc khởi động nếu khóa quá ngắn.

9.6.2. Lộ trình cải tiến

Giai đoạn 1 — Các cải tiến nhanh (Quick Wins) (nỗ lực triển khai thấp, hiệu quả cao):

- Giới hạn origin của CORS (lưu ý: nếu kích hoạt HttpOnly cookie kéo theo `allowCredentials=true`, hệ thống tuyệt đối không được sử dụng `allowedOrigins("*")` — đặc tả CORS buộc trình duyệt chặn tổ hợp wildcard + credential, đây là quy tắc bảo mật, không phải vấn đề tương thích).
- Di chuyển secret ra khỏi tệp cấu hình sang biến môi trường (mức tối thiểu).
- Lưu trữ token trong HttpOnly cookie thay vì `localStorage`.
- Kích hoạt xác minh email và Turnstile cho đăng nhập, đăng ký, cấp lại mật khẩu ở các luồng giao tiếp công khai.
- Gỡ bỏ mọi `X-User-*` từ ingress, giữ bằng chứng có chữ ký cho dịch vụ tuyến sau và xác minh cấu hình chặn truy cập trực tiếp tới service.
- Chuẩn hóa ngay hợp đồng xác minh: thuật toán được phép (`alg`), issuer, audience, `exp`, `nbf`; không để mỗi service tự chọn quy tắc.

Giai đoạn 2 — Tăng cường xác thực (Authentication Hardening) (nỗ lực triển khai trung bình):

- Chuyển đổi thuật toán ký từ HS256 sang RS256 và công bố khóa công khai qua JWKS.
- Dùng Spring Security OAuth2 Resource Server hoặc middleware tương đương tại Gateway và các resource service.
- Triển khai luồng làm mới token kết hợp luân phiên refresh token (token rotation — mỗi lần làm mới sẽ vô hiệu hóa refresh token cũ).
- Chuẩn hóa SSO theo mô hình đổi token nội bộ có issuer/audience rõ ràng và hợp đồng claim chung.

Giai đoạn 3 — Workload Identity và Delegation (nỗ lực triển khai cao, áp dụng khi mở rộng quy mô):

- Xác thực service-to-service bằng workload token và/hoặc mTLS.
- Tách subject khỏi actor; chỉ cho phép on-behalf-of khi có delegation rõ ràng, đúng audience và thời hạn ngắn.
- Với job bất đồng bộ, ký hoặc ràng buộc command context để consumer xác minh quyền thực thi (Tier 3).
- Quản lý secret tập trung (HashiCorp Vault).
- Chính sách phân quyền ở mức API (Open Policy Agent).
- Rào chắn runtime cho DevOps Lab: Kubernetes RBAC, NetworkPolicy, resource quota và Sysbox isolation.

9.6.3. Secret Management trong production

Bảng 9.6 xếp Secret Management ở mức rủi ro cao. Đặt khóa bí mật trong tệp cấu hình đi cùng mã nguồn là lỗi phổ biến trong các dự án microservices và cần được khắc phục ở mọi môi trường production.

Bảng 9.7 so sánh ba cấp độ quản lý secret, từ biến môi trường đến giải pháp cho quy mô doanh nghiệp.

Bảng 9.7: *Chiến lược Secret Management — từ cơ bản đến quy mô doanh nghiệp*

Cấp độ	Cách tiếp cận	Ưu điểm	Nhược điểm	Phù hợp
Cấp 1	Biến môi trường	Đơn giản, tách secret khỏi code	Secret ở dạng văn bản thuần trên server, không có nhật ký kiểm toán (audit trail)	Dev/Staging
Cấp 2	Cấu hình mã hóa (Encrypted Config: `{cipher}` của Spring Cloud Config, Mozilla SOPS hoặc Jasypt)	Secret mã hóa trong repo, giải mã lúc runtime	Vẫn cần quản lý khóa mã hóa	Production quy mô nhỏ
Cấp 3	Secret Manager (HashiCorp Vault, AWS Secrets Manager, GCP Secret Manager)	Tự động luân phiên, nhật ký kiểm toán, chính sách truy cập, secret động (dynamic secret)	Chi phí hạ tầng, độ phức tạp vận hành	Production quy mô lớn

Cấp 1 — Biến môi trường (mức tối thiểu, nên áp dụng ngay):

Thay vì:

```
# SAI: application.yml — KHÔNG BAO GIỜ commit thông tin nhạy cảm vào Git
jwt:
  secretKey: "gia-tri-bi-mat-viet-truc-tiep"
  expiration: 86400000
```

Sử dụng:

```
# ĐÚNG: application.yml — tham chiếu biến môi trường
jwt:
  secretKey: ${JWT_SECRET_KEY}
  expiration: ${JWT_EXPIRATION:1800000}   # 30 phút
```

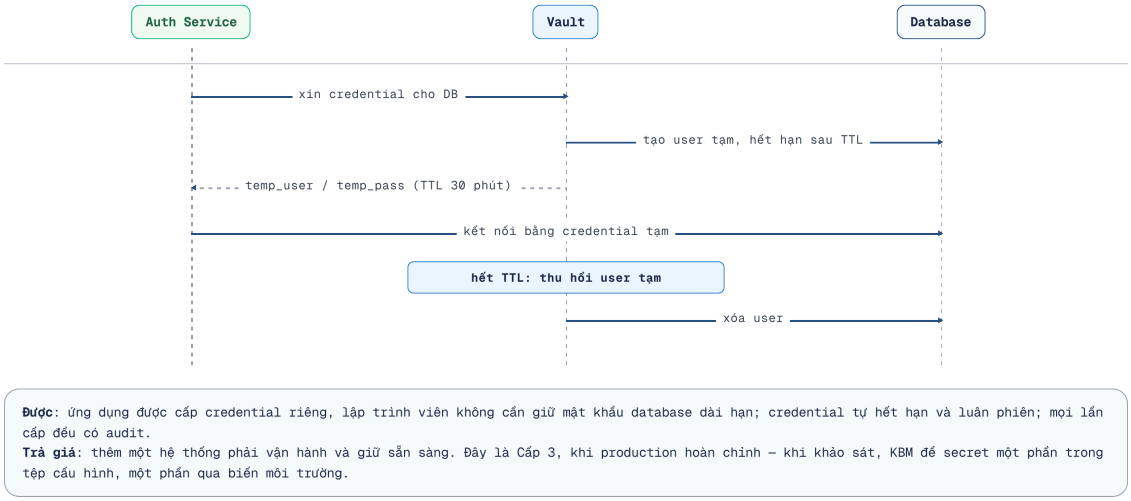
```
# Docker Compose — nạp từ file .env (không commit .env)
services:
  kbm-auth:
    environment:
```

```
      - JWT_SECRET_KEY=${JWT_SECRET_KEY}
      - DB_PASSWORD=${DB_PASSWORD}
```

Bước đầu tiên và quan trọng nhất là thêm `.env` vào `.gitignore`, đồng thời tạo `.env.example` không chứa giá trị thực để các thành viên trong nhóm phát triển biết những biến cần thiết lập. Đây là cải tiến bảo mật không tốn chi phí mà mọi dự án đều nên áp dụng ngay.

Cấp 3 — HashiCorp Vault (khi hệ thống production hoàn chỉnh):

Vault cung cấp **secret động (dynamic secret)** — thông tin đăng nhập cơ sở dữ liệu được tạo tự động, có thời hạn và tự động luân phiên. Ứng dụng được cấp credential riêng; lập trình viên không cần giữ mật khẩu database dài hạn. Khả năng đọc credential vẫn phụ thuộc chính sách truy cập Vault.



Hình 9.9: Vault Dynamic Secret — thông tin đăng nhập tạm thời, tự động luân phiên

⚠ Các lỗ hổng và sai lầm bảo mật thường gặp

- **Lưu JWT trong localStorage**: Bất kỳ đoạn mã JavaScript nào chạy trên trang cũng đều có thể đọc được dữ liệu này (tấn công XSS). Hậu quả: nếu tồn tại lỗ hổng XSS, kẻ tấn công có thể đánh cắp token và mạo danh người dùng. *Phòng tránh*: lưu trữ JWT trong HttpOnly cookie (JavaScript không đọc được), kết hợp `Secure`, `SameSite=Lax/Strict` và CSRF token hoặc kiểm tra Origin cho các yêu cầu thay đổi trạng thái — vì trình duyệt tự động đính kèm cookie, HttpOnly chỉ bảo vệ tính bí mật của token chứ không tự ngăn được tấn công CSRF.
- **Gắn cố định secret trong mã nguồn**: Đặt khóa bí mật trực tiếp trong `application.yml` và đưa lên Git. Hậu quả: ai có quyền truy cập repo đều có thể tạo JWT giả mạo. *Phòng tránh*: dùng biến môi trường (tối thiểu), hoặc trình quản lý secret chuyên dụng (Vault, AWS Secrets Manager).
- **Không đặt thời hạn hoặc đặt thời hạn access token quá dài**: Access token không có `exp` hoặc có thời hạn quá dài mà không đi kèm cơ chế làm mới hay luân phiên. Hậu quả: nếu token bị lộ, kẻ tấn công có thể sử dụng trong thời gian dài (thậm chí vĩnh viễn) mà hệ thống rất khó thu hồi. *Phòng tránh*: dùng access token ngắn hạn (15–60 phút), kết hợp refresh

token có thời hạn dài hơn (7–30 ngày) đi kèm cơ chế luân phiên refresh token và giám sát tái sử dụng trái phép để vô hiệu hóa toàn bộ chuỗi token.

- **Dùng HS256 shared secret cho toàn bộ môi trường production**: Tất cả các dịch vụ chia sẻ chung một khóa bí mật để ký và xác minh JWT. Hậu quả: nếu chỉ một dịch vụ bị xâm nhập, kẻ tấn công có thể tạo token giả mạo cho bất kỳ người dùng nào trên toàn hệ thống. *Phòng tránh*: chuyển sang thuật toán bất đối xứng RS256/JWKS — chỉ Auth Service giữ khóa riêng tư để ký, còn các dịch vụ khác chỉ dùng khóa công khai để xác minh.
- **Xác minh JWT ở mỗi dịch vụ theo cách khác nhau**: Mỗi dịch vụ sử dụng một phiên bản thư viện JWT và tập quy tắc riêng. Hậu quả không chỉ là Gateway chấp nhận nhưng service từ chối; một bộ xác minh lỏng hơn còn có thể chấp nhận sai thuật toán, issuer hoặc audience. *Phòng tránh*: chuẩn hóa bước xác minh tại resource service bằng cùng framework, issuer metadata/JWKS và kiểm thử hợp đồng (contract test). Gateway kiểm tra ở ingress; service đích vẫn xác minh bằng chứng có chữ ký và thực hiện phân quyền cục bộ (§9.3).
- **Tin tưởng dữ liệu định danh thô do client gửi**: Đọc thẳng header như `X-User-Id` hoặc tin claim chỉ vì request đi qua mạng nội bộ. Hậu quả: leo thang đặc quyền và tấn công confused deputy. *Phòng tránh*: Gateway gỡ bỏ các header định danh từ client; bằng chứng có chữ ký phải được service đích kiểm tra chữ ký, issuer, audience và thời hạn trước khi phân quyền. Header dẫn xuất không được trở thành nguồn quyền độc lập.
- **Confused Deputy Problem**: Dịch vụ A gọi dịch vụ B thay mặt người dùng, nhưng B không biết A đang "đại diện" cho người dùng hay hành động với đặc quyền riêng của A. Newman trong [4a, Ch.9] gọi đây là *confused deputy problem*. Hậu quả: người dùng chỉ có quyền `STUDENT` nhưng A lại gọi B bằng credential hệ thống toàn quyền. *Phòng tránh*: bằng chứng thay mặt người dùng (on-behalf-of proof) phải tách danh tính người dùng (*subject*) khỏi định danh tác nhân gọi (*actor/workload*), ghi rõ phạm vi ủy quyền (delegation), audience và thời hạn. B xác minh bằng chứng rồi giới hạn quyền theo cả subject lẫn actor; header văn bản thuần `X-User-*` không đủ để biểu diễn quan hệ này.

9.6.4. Cô lập thực thi và phong tỏa (lockdown) thi cử — hai bài học phòng thủ

Bảo mật không dừng ở JWT và RBAC. Hai nghiệp vụ chấm code và thi phòng máy của KBM cho thấy hai mặt khác của phòng thủ: **cô lập tài nguyên** khi phải chạy mã không tin cậy, và **phòng thủ chiều sâu** khi chống gian lận.

Chạy mã sinh viên: rủi ro không có sandbox và lời giải cô lập

Bộ chấm code thế hệ đầu (`kbm-judge-code` Gen 1) chạy bài nộp C/C++ và Java **trực tiếp trong tiến trình máy chủ, không có sandbox**. Đây là lỗ hổng nghiêm trọng: mã sinh viên là *dữ liệu đầu vào không đáng tin cậy có khả năng thực thi* — nó có thể đọc/ghi file hệ thống, thiết lập kết nối mạng trái phép ra bên ngoài, tiêu thụ vô hạn CPU/RAM, hoặc cố ý gây ngưng trệ dịch vụ. Khác với chấm SQL (chạy trong sandbox cơ sở dữ liệu cách ly mỗi lần nộp bài) hay DevOps Lab (pod Sysbox cách ly), Code Judge Gen 1 bỏ qua hoàn toàn lớp cách ly — một dạng đánh đổi "tự viết để kiểm soát tối đa" nhưng vô tình mở rộng bề mặt tấn công ngay bên trong ranh giới tin cậy của dịch vụ.

⚠ Không bao giờ chạy mã không tin cậy (untrusted code) chung tiến trình

Mã do người dùng nộp lên (bài nộp cho bộ chấm, plugin, công thức tùy biến...) phải được xem như **thù địch cho đến khi được chứng minh ngược lại**. Chạy nó chung tiến trình/máy chủ với

service đồng nghĩa trao cho kẻ tấn công chính quyền hạn của service đó: truy cập file, mạng, secret trong biến môi trường, và tài nguyên không giới hạn. Lỗ hổng này không phải lỗi logic mà là *thiếu một lớp cách ly*.

Thế hệ thứ hai (Gen 2) chuyển sang tích hợp **DOMjudge**, chạy mỗi bài nộp trong **sandbox cách ly** dựa trên cgroup/namespace của Linux: giới hạn CPU, bộ nhớ, thời gian thực thi, và cắt quyền truy cập tài nguyên ngoài. Nguyên tắc bảo mật cốt lõi: **không bao giờ chạy mã không tin cậy trong cùng ranh giới tin cậy với dịch vụ**. Ở đây, sandbox đi song hành với quyết định xây hay mua (build vs buy) (Chương 3, Chương 4). Việc lựa chọn một hệ thống chấm điểm mã nguồn mở (OSS judge) đã được kiểm chứng qua nhiều kỳ thi ICPC vừa giải quyết bài toán đa ngôn ngữ, vừa khắc phục lỗ hổng thiếu cách ly mà phiên bản tự phát triển trước đó gặp phải.

Phong tỏa thi cử và phòng thủ chiều sâu chống gian lận

Với kỳ thi tại phòng máy, KBM dùng `kbm-exam-client` — một desktop client kiểm soát chặt môi trường thi: bắt buộc toàn màn hình, chặn Alt-Tab/chuyển ứng dụng, vô hiệu hóa copy-paste và chuột phải, chặn mở trình duyệt/tài liệu ngoài, và chỉ cho kết nối từ **danh sách IP được phép (IP whitelist)** của phòng thi. Client còn thu tín hiệu giám sát (rời cửa sổ, mất focus, cắm/rút thiết bị) và đẩy về server làm dữ liệu chống gian lận (anti-cheat). Client này dùng cùng JWT của `kbm-auth` (SSO) và là một client của Bounded Context Examination & Proctoring.

Tuy nhiên, việc phong tỏa (lockdown) ở phía máy khách **không bao giờ được coi là lớp phòng thủ duy nhất**. Một thí sinh có kỹ thuật có thể vá, giả lập hoặc bỏ qua client desktop — mọi thứ chạy trên máy người dùng đều nằm dưới quyền kiểm soát của người dùng. Vì vậy server (logic kỳ thi và chống gian lận, hiện trong `kbm-core`) phải **độc lập** phát hiện bất thường: đổi IP hoặc User-Agent giữa ca thi, tần suất nộp bài bất thường, số lần vi phạm vượt ngưỡng cấu hình. Đây chính là **phòng thủ chiều sâu** (§9.1) áp dụng cho anti-cheat: tín hiệu được thu cả ở client (lockdown) lẫn ở server (phát hiện bất thường), không phía nào là điểm tin cậy tuyệt đối.

Lockdown của `kbm-exam-client` chỉ làm *tăng chi phí gian lận*, không *loại bỏ* khả năng gian lận. Cũng theo nguyên lý Gateway gỡ bỏ header `X-User-*` từ Internet (§9.3), mọi tín hiệu bảo mật do client gửi lên đều phải được server kiểm chứng hoặc đối chiếu độc lập. Client đảm nhiệm trải nghiệm người dùng và là lớp ngăn chặn đầu tiên; server nắm quyền quyết định cuối cùng và là căn cứ tin cậy duy nhất cho anti-cheat.

Các pattern trong chương có minh họa tương tác đi kèm tại companion web:

Interactive Demo

- **Luồng xác thực OAuth2 & JWT**: oauth2-jwt-flow.html
- **Role-Based Access Control (RBAC)**: rbac-authorization.html

9.7. Tổng kết

Bảo mật microservices phức tạp hơn bảo mật hệ thống nguyên khối (monolith) vì bề mặt tấn công lớn hơn — mỗi dịch vụ, mỗi lời gọi mạng, mỗi cơ sở dữ liệu đều là điểm tiềm ẩn rủi ro. Phòng thủ chiều sâu (defense in depth) — bảo vệ nhiều lớp thay vì dựa vào một lớp duy nhất — là nguyên tắc nền tảng.

JWT cung cấp các claim có chữ ký số theo cách stateless, không cần kho lưu trữ phiên chung cho mỗi request. HS256 (ký bằng khóa đối xứng) đơn giản nhưng mọi bên xác minh giữ shared secret cũng có khả năng tự ký token — chỉ nên dùng trong phạm vi hẹp và được kiểm soát chặt. RS256 (ký bằng cặp khóa bất đối xứng) tách bên ký khỏi bên xác minh: Auth Service giữ khóa riêng tư (private key), còn các resource service lấy khóa công khai (public key) qua JWKS. Dù dùng thuật toán nào, bên xác minh vẫn phải cố định thuật toán được phép, đối chiếu issuer và audience, rồi kiểm tra thời hạn token.

Mô hình tin cậy phân tầng (Tiered Trust) tách ba lớp kiểm tra: kiểm tra ở biên (Gateway xác minh token tại ingress), xác minh tại dịch vụ đích (service đích kiểm tra chữ ký và quyền truy cập) và đối chiếu trạng thái tại cơ sở dữ liệu. Tier 1 yêu cầu bằng chứng có chữ ký cùng phân quyền cục bộ tại service sở hữu nghiệp vụ; Tier 2 bổ sung workload identity, actor và ủy quyền (delegation); Tier 3 bảo vệ command bất đồng bộ. NetworkPolicy và mTLS thu hẹp đường đi mạng, nhưng không thay thế bằng chứng cho danh tính người dùng (user subject). OAuth 2.0 và OIDC cho phép giao việc xác thực người dùng cho nhà cung cấp danh tính chuyên dụng; Client Credentials chỉ đại diện cho workload (dịch vụ gọi dịch vụ) nếu không có delegation rõ ràng.

RBAC (phân quyền theo vai trò) là điểm khởi đầu phổ biến cho bài toán phân quyền. KBM triển khai RBAC qua `@PreAuthorize`, nhưng kiểm tra vai trò phải đi cùng kiểm tra quyền sở hữu tài nguyên (ownership) và quan hệ nghiệp vụ tại service đích — ví dụ, sinh viên chỉ xem được bài nộp của chính mình dù cùng vai trò `STUDENT`. Tại thời điểm khảo sát, hệ thống còn thiếu phân cấp vai trò (role hierarchy) rõ ràng và một cách khai báo chính sách phân quyền dễ kiểm toán.

Phân tích KBM tại thời điểm khảo sát cho thấy hệ thống đã có JWT, Gateway, RBAC và SSO, nhưng truyền định danh còn là rủi ro được chấp nhận có kiểm soát. Lộ trình mục tiêu là làm sạch request ở biên, thống nhất hợp đồng xác minh, chuyển sang RS256/JWKS cùng bằng chứng danh tính có chữ ký cho chặng nội bộ, rồi bổ sung workload identity/delegation theo tầng tin cậy. Các khoản nợ kỹ thuật khác — CORS, quản lý secret, lưu trữ token và cô lập thực thi — vẫn phải được xử lý song song.

Ở Chương 10, chúng ta sẽ vận dụng kiến thức từ Chương 1–9 vào **một bài toán thực tế**: chuyển đổi từ monolith sang microservices — khi nào nên chuyển, Strangler Fig pattern, tách database, và lộ trình chuyển đổi cho KBM.

9.8. Đọc thêm

Sách tham khảo chính:

1. [4a] Sam Newman, *Building Microservices* — Ch.9: Security
2. [2a] Chris Richardson, *Microservices Patterns*, 1st Ed. — Ch.11: Developing production-ready services (§11.1: Developing secure services, Access Token pattern)
3. [7] Martin Kleppmann, *Designing Data-Intensive Applications* — Ch.4: Encoding and Evolution (data formats, schema evolution — nền tảng để hiểu binary/JSON encoding trong token và thông điệp API)

Nguồn trực tuyến:

- JWT.io — jwt.io (interactive JWT decoder)
- OWASP API Security Top 10 — owasp.org/www-project-api-security
- Spring Security OAuth2 Resource Server — docs.spring.io/spring-security
- RFC 7519 — JSON Web Token (JWT) specification
- NIST SP 800-207 — Zero Trust Architecture

- RFC 8725 — JSON Web Token Best Current Practices
- RFC 9068 — JWT Profile for OAuth 2.0 Access Tokens
- RFC 9700 — Best Current Practice for OAuth 2.0 Security

Bài tập: 9-0, 9-1, 9-2 ở phần Bài tập & case study cuối sách.

CHƯƠNG 10

Chuyển đổi thực tế — từ Monolith đến Microservices

> "If you can't build a well-structured monolith, what makes you think microservices is the answer?"
> — Simon Brown

Lưu ý

Chuyển đổi (Migration) là một *kỹ năng riêng*: áp dụng các pattern đã học ở Phần I–II vào một hệ thống *đang vận hành thực tế*, nơi không thể viết lại từ đầu. Chương này thêm yếu tố *phán đoán* — khi nào nên và khi nào *không* nên chuyển sang microservices — và là nơi trình bày **lộ trình chuyển đổi chi tiết cho KBM** (Quick Wins → Observability → Resilience → Database Decomposition).

Mục tiêu: chuyển đổi nền tảng LMS mà không làm gián đoạn việc học. Chương này giúp người đọc:

MỤC TIÊU HỌC TẬP

- Hiểu khi nào nên và **khi nào không nên** chuyển sang microservices
- Nắm vững **Strangler Fig Pattern** — chiến lược chuyển đổi tăng dần phổ biến nhất
- Áp dụng **5 chiến lược tách database** từ góc nhìn chuyển đổi thực tế
- Hiểu **Anti-Corruption Layer** và các pattern chuyển đổi bảo vệ ranh giới hệ thống
- Thiết kế **lộ trình chuyển đổi** khả thi, ưu tiên theo tác động và nỗ lực (Impact × Effort)
- Phân tích lộ trình cho KBM, từ khoảng trống ở Chương 2–9 đến điều kiện tiên quyết ở Chương 11–12

10.1. Thời điểm thích hợp để chuyển đổi

10.1.1. Vấn đề: microservices không phải đích đến mặc định

Sau 9 chương phân tích các pattern kiến trúc, cơ chế giao tiếp, quản lý dữ liệu, API Gateway và bảo mật, độc giả có thể có ấn tượng rằng microservices là kiến trúc mọi hệ thống nên hướng tới. **Đây là một sai lầm phổ biến.**

Martin Fowler — người đồng tác giả bài viết gốc "Microservices" (2014) — cảnh báo rõ ràng [W2]:

Monolith First

> *"Almost all the cases where I've heard of a system that was built as a microservice system from scratch, it has ended up in serious trouble... You shouldn't start a new project with microservices, even if you're sure your application will be big enough to make it worthwhile."*
>
> *— Martin Fowler [W2]*

Newman trong [4b, Ch.1] bổ sung: microservices là **một lựa chọn kiến trúc** không phải bước tiến hóa bắt buộc. Monolith không phải "kiến trúc sai" — monolith là kiến trúc đúng cho phần lớn hệ thống ở giai đoạn đầu.

10.1.2. Ma trận quyết định — khi nào cân nhắc chuyển đổi

Bảng 10.1 tóm tắt các tiêu chí giúp quyết định khi nào monolith vẫn đủ tốt và khi nào nên cân nhắc microservices.

Bảng 10.1: *Ma trận quyết định (Decision Matrix) — khi nào cân nhắc chuyển đổi*

Tiêu chí	Monolith đủ tốt	Cân nhắc Microservices
Quy mô đội ngũ	Ước lượng ≤ 8 người	Ước lượng > 15 người, nhiều nhóm
Tần suất triển khai	Hàng tuần/tháng	Triển khai hàng ngày theo nhóm
Yêu cầu mở rộng	Mở rộng toàn bộ là đủ	Mở rộng từng thành phần độc lập
Đa dạng công nghệ	Một bộ công nghệ (tech stack) là đủ	Cần đa ngôn ngữ (polyglot: Java + Python + Go)
Độ phức tạp nghiệp vụ	Ranh giới chưa rõ	Bounded Context đã xác định rõ (Chương 2)
Mức độ trưởng thành của tổ chức	Chưa có DevOps, CI/CD	Đã có CI/CD, container hóa, giám sát

Martin Fowler gọi đây là **Microservice Premium** — chi phí phải trả khi chuyển sang microservices: độ phức tạp của hệ phân tán, lỗi mạng, eventual consistency, chi phí vận hành. Khoản phí này (premium) chỉ đáng trả khi lợi ích (như triển khai độc lập, mở rộng linh hoạt, sự tự chủ của đội ngũ) vượt qua chi phí.

10.1.3. KBM: kiến trúc tiến hóa (Evolutionary Architecture), không phải Big Bang

KBM không bắt đầu từ một bản thiết kế microservices hoàn chỉnh định sẵn. Hệ thống đã đi qua nhiều thế hệ: các judge monolith riêng cho bài lập trình, Network Judge, SQL Judge mới, rồi dần hình thành các service cho auth, academic, judge, notification và DevOps Lab. Vì vậy câu hỏi đúng không phải là "có nên microservices-first không?", mà là: *mỗi bước tiến hóa có đang giải quyết đúng vấn đề của thời điểm đó không?*

Microservice Premium trong KBM

KBM chọn hướng microservices vì hai lý do cụ thể: (1) mục đích **giáo dục** — bản thân hệ thống là tài liệu học tập cho sinh viên, (2) bảy core Bounded Context đã đủ rõ sau nhiều năm vận hành các hệ thống chấm điểm (judge system): Identity & Access, Academic Management, SQL Practice, Code Practice, Network Practice, DevOps Practice và Examination & Proctoring.

Tuy nhiên, sự đánh đổi (trade-off) hiện rõ: (1) cơ sở dữ liệu dùng chung (shared database) giữa Core Service và Academic Service — sự phụ thuộc (coupling) vẫn tồn tại dù dịch vụ đã được tách, (2) thư viện dùng chung (shared library) chứa quá nhiều logic — rủi ro nguyên khối phân tán (distributed monolith), (3) môi trường đa ngôn ngữ (polyglot) Java+Go làm tăng chi phí biên dịch, kiểm thử và triển khai. Đây là ví dụ thực tế của Microservice Premium: **lợi ích giáo dục và nhu cầu cô lập hệ thống chấm điểm đủ để chứng minh tính hợp lý của chi phí**, nhưng vẫn cần một lộ trình giảm thiểu sự phụ thuộc (coupling) có kiểm soát.

Sự khác biệt giữa các phân hệ chấm điểm cũng minh họa rõ việc chọn chiến lược theo từng ngữ cảnh. Network Practice được thiết kế độc lập để đánh giá giao tiếp qua mạng nên tách rất tự nhiên theo giao thức. Trong khi đó, với SQL Practice, bài toán nằm ở ranh giới giữa bộ điều phối `kbm-judge-sql` và các worker `kbm-sandbox-*` chạy trong môi trường cách ly theo từng hệ quản trị cơ sở dữ liệu (`kbm-sandbox-mysql`, `kbm-sandbox-postgres`, `kbm-sandbox-sqlserver`, ...). Chuyển đổi đúng hướng là làm rõ cơ chế định tuyến theo quyền sở hữu (ownership routing), chuẩn hóa hợp đồng công việc và kết quả, bổ sung idempotency dựa trên `judgeRunId`, sau đó mới mở rộng theo chiều ngang cho từng nhóm worker dựa trên tải thực tế của từng hệ quản trị cơ sở dữ liệu.

Monolith First — tổng hợp

Bắt đầu bằng monolith. Chỉ chuyển sang microservices khi monolith thực sự trở thành vấn đề và đã biết mình đang giải quyết vấn đề nào. Những hệ thống microservices tệ nhất thường do các nhóm bắt đầu bằng microservices *trước khi* hiểu ranh giới domain.

— Tổng hợp từ Martin Fowler [W2] và Sam Newman [4b, Ch.1]

Monolith First không phủ nhận những gì đã học — mà là nguyên tắc lựa chọn thời điểm và cách áp dụng đúng ngữ cảnh.

10.1.4. Bài học thực tế — khi chuyển đổi đi sai hướng (anti-pattern)

Dù quyết định chuyển đổi là đúng đắn, *cách thức* chuyển đổi vẫn có thể gây thất bại cho dự án. Richardson trong [2b, Ch.2] phân tích bốn anti-pattern phổ biến nhất khi chuyển từ monolith sang microservices — chúng ta minh họa qua các sai lầm *tiềm năng* nếu chia tách sai trong KBM:

Bảng 10.2: *Các anti-pattern khi chuyển đổi sang microservices (minh họa với KBM)*

Anti-pattern	Ví dụ sai lầm tiềm năng trong KBM	Hậu quả
Data Service *(Dịch vụ bao bọc CRUD)*	Tách riêng bảng `questions` thành một `QuestionDataService` chỉ làm CRUD — Core Service phải gọi HTTP mỗi lần cần đọc câu hỏi	Tăng độ trễ mạng, mất lợi ích giao dịch (transaction) nội bộ
Fine-grained Service *(Chia quá nhỏ)*	Chia Auth thành 3 service: `UserProfile`, `UserCredential`, `UserSession`	Mỗi lần đăng nhập phải điều phối (orchestrate) 3 dịch vụ, khó gỡ lỗi (debug)

Anti-pattern	Ví dụ sai lầm tiềm năng trong KBM	Hậu quả
Microservices-first	Chia KBM thành nhiều service ngay từ đầu, khi ranh giới miền chưa rõ, nên tách theo tầng kỹ thuật (technical layer) — mỗi thay đổi chấm điểm ảnh hưởng nhiều service cùng lúc	Phát sinh "Distributed Monolith" (xem thêm §10.6)
QA Gate đầu cuối *(Chờ kiểm thử tích hợp toàn cục)*	Triển khai độc lập nhưng đội kiểm thử (QA) phải chờ tích hợp tất cả để kiểm thử	Triệt tiêu khả năng triển khai độc lập *(Independent Deployability)*

Các sai lầm chuyển đổi khác (Big Bang Rewrite, Lift-and-Shift, Over-engineering) sẽ được phân tích chi tiết tại §10.6.

10.2. Strangler Fig Pattern — chuyển đổi tăng dần

10.2.1. Vấn đề: chuyển đổi tức thì ("Big Bang") so với chuyển đổi tăng dần (incremental migration)

Có hai cách thức chính để dịch chuyển từ monolith sang microservices: **Big Bang** (viết lại toàn bộ hệ thống mới và chuyển đổi cùng lúc) mang rủi ro rất cao và thường dẫn đến thất bại; hoặc **Tăng dần (Strangler Fig)** (tách dần từng phần, cho chạy song song) với rủi ro thấp hơn nhiều do có thể dễ dàng hoàn tác (rollback) từng phần.

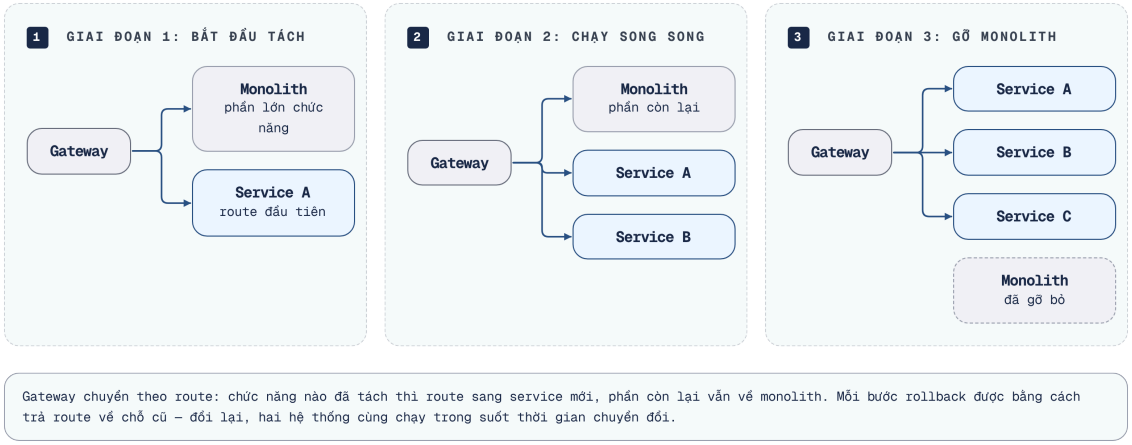
Newman trong [4b, Ch.2] và Richardson trong [2a, Ch.13] đều cảnh báo: **viết lại kiểu Big Bang rất rủi ro và thường thất bại**. Khi viết lại từ con số không, đội ngũ phát triển mất toàn bộ kiến thức nghiệp vụ (domain knowledge) được nhúng trong mã nguồn cũ, thời hạn (deadline) liên tục trễ, và khách hàng phải chờ đợi *tất cả* tính năng trước khi nhận *bất kỳ* giá trị nào.

10.2.2. Strangler Fig Pattern

Xây thư viện số hiện đại

Hãy tưởng tượng trường đại học muốn thay thư viện vật lý cũ bằng thư viện số. Tòa nhà cũ không bị đập bỏ ngay lập tức (Big Bang); thay vào đó, thư viện số được xây ngay bên cạnh (Strangler Fig). Ban đầu, khi sinh viên đến cổng trường (API Gateway) để xin "Mượn sách điện tử", bảo vệ chỉ họ sang tòa nhà mới. Nếu họ xin "Mượn sách giấy", bảo vệ vẫn chỉ vào tòa nhà cũ. Từng khu vực (tạp chí, giáo trình...) lần lượt được số hóa và sinh viên được chuyển hướng sang tòa nhà mới, cho đến khi tòa nhà cũ không còn ai vào nữa và bị phá dỡ hoàn toàn.

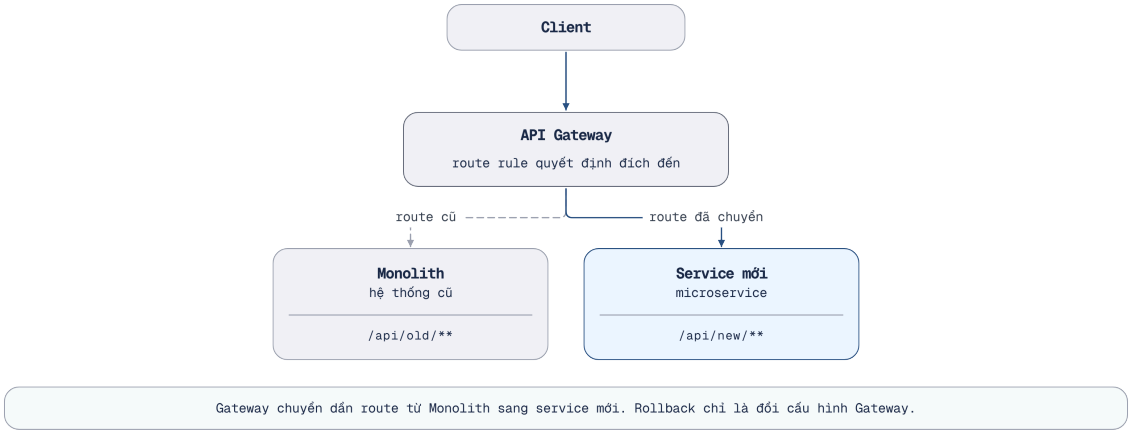
Strangler Fig — pattern do Martin Fowler đặt tên (2004), lấy cảm hứng từ cây đa bóp nghẹt (strangler fig) bao quanh cây chủ, dần thay thế cho đến khi cây chủ biến mất [W8].



Hình 10.1: *Strangler Fig Pattern — dần thay thế monolith qua 3 giai đoạn*

10.2.3. API Gateway — điểm tiếp nối (seam) cho Strangler Fig

Để áp dụng mô hình này, chúng ta cần khả năng định tuyến giao tiếp tốt. API Gateway (đã học ở Chương 8) hoạt động như một điểm tiếp nối (seam) cho Strangler Fig, giúp che giấu sự phức tạp của quá trình chuyển đổi khỏi người dùng cuối:



Hình 10.2: *API Gateway làm điểm tiếp nối cho Strangler Fig — client không biết route nào đến monolith, route nào đến service mới*

Tùy thuộc vào kiến trúc hiện tại, chúng ta có ba chiến lược triển khai Strangler Fig chính: **Route-based** (dùng Gateway định tuyến khác nhau đến monolith hoặc các service mới) rất phù hợp cho các hệ thống định hướng API (API-driven); **Event Interception** (service mới lắng nghe sự kiện từ monolith để dần thay thế logic) lý tưởng cho hệ thống định hướng sự kiện (event-driven); và **Asset Capture** (chuyển dần quyền sở hữu từng đối tượng nghiệp vụ cốt lõi — như một loại hồ sơ — sang hệ thống mới) thích hợp khi ranh giới dữ liệu rõ [W8].

Với KBM — nơi đã có API Gateway (Spring Cloud Gateway, Chương 8) — **route-based Strangler Fig** là lựa chọn tự nhiên nhất cho LMS chính. Với DevOps Lab, routing lại dựa trên hostname/wildcard DNS ở mức khái quát. Hai kiểu gateway này cùng tồn tại, cho thấy việc chuyển đổi không nhất thiết dùng một pattern duy nhất cho mọi Bounded Context.

Listing 10.1: *Cấu hình Spring Cloud Gateway áp dụng Strangler Fig (đường dẫn thi chuyển sang service mới, còn lại về monolith)*

```yaml
spring:
  cloud:
    gateway:
      routes:
        - id: new_exam_service_route
          uri: lb://kbm-exam
          order: 1
          predicates:
            - Path=/api/exams/**
        - id: legacy_monolith_fallback_route
          uri: http://kbm-legacy-monolith:8080
          order: 100
          predicates:
            - Path=/**
```

Chuyển đổi tăng dần

"If you do a big-bang rewrite, the only thing you're guaranteed of is a big bang."

— Martin Fowler, trích trong Sam Newman, Monolith to Microservices [4b, Ch.2]

10.2.4. KBM: Strangler Fig thực tế — từ `kbm-legacy-monolith` đến `kbm-*`

Trạng thái ban đầu (Before) của KBM (đã giới thiệu ở §1.8) là mô hình *independent silo monoliths* — ba khối MVC rời rạc chạy cạnh nhau mà không chia sẻ dữ liệu hay danh tính: phần quản lý đào tạo là ứng dụng **ASP.NET MVC** nguyên khối (`kbm-legacy-monolith`), còn các bộ chấm là những monolith MVC **tách rời** do nhóm tự viết — một code judge (C/C++ và Java, **không sandbox**) và một network judge (Java). Vì mỗi hệ là một khối riêng, chi phí bảo trì nhân lên theo *số hệ thống* chứ không được chia nhỏ.

Thay vì viết lại cả ba khối cùng một lúc ("Big Bang"), KBM tiến hóa tăng dần (2024–2025): các capability mới như SQL Practice (`kbm-judge-sql`), xác thực tập trung (`kbm-auth`), quản lý học vụ (`kbm-academic`), chấm mạng thế hệ mới (`kbm-judge-network`) lần lượt được phát triển thành các dịch vụ `kbm-*` đứng sau API Gateway (Spring Cloud Gateway, Chương 8). Cùng cơ chế điểm tiếp nối (seam) ở Gateway theo hướng định tuyến (`route-based` như Listing 10.1, với route dự phòng trỏ về `kbm-core` thay vì monolith cũ) tiếp tục được áp dụng khi cần bóc tách các phân hệ còn nằm trong `kbm-core` (chẳng hạn tách `kbm-exam` thành dịch vụ độc lập ở kiến trúc mục tiêu) — làm rõ ranh giới Bounded Context trước (Chương 2), rồi mới chuyển hướng định tuyến, đảm bảo mỗi bước đều nhỏ và có thể hoàn tác (rollback).

Xây → Mua: thay bộ chấm code tự viết bằng DOMjudge

Một bước Strangler Fig điển hình là `kbm-judge-code`. Bộ chấm mã nguồn (code judge) **Gen 1** (di sản, một monolith MVC riêng) do nhóm tự viết, chỉ hỗ trợ C/C++ và Java, và **chạy bài nộp trực tiếp trên môi trường OS của máy chủ — không có môi trường cách ly (sandbox)**: mã sinh viên có thể truy cập tài nguyên máy chủ, không giới hạn CPU/RAM. Đây vừa là lỗ hổng bảo mật (Chương 9) vừa là gánh nặng bảo trì.

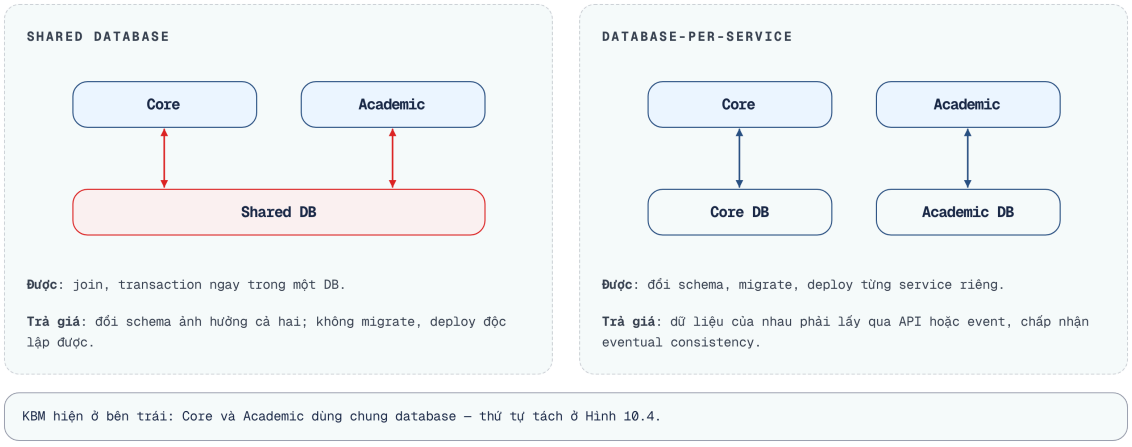
Thay vì tiếp tục chắp vá một bộ chấm tự xây, đội ngũ chọn **xây → mua/tích hợp (build → buy/adopt)**: `kbm-judge-code` **Gen 2** tích hợp **DOMjudge** — online judge mã nguồn mở chuẩn ICPC, đã

được tích hợp sẵn môi trường cách ly (sandbox bằng cgroup/namespace) và đa ngôn ngữ. Quyết định này **chuyển một lớp rủi ro ra khỏi nhóm**: cách ly sandbox và bảo trì compiler/runner được chuyển cho cộng đồng mã nguồn mở đã kiểm chứng; đội ngũ vẫn chịu trách nhiệm cấu hình, cập nhật và kiểm chứng môi trường judgehost khi tích hợp và vận hành. Đây là minh họa rằng việc chuyển đổi không phải lúc nào cũng là "tách ra rồi tự viết lại" — đôi khi bước hợp lý là **thay một thành phần tự xây bằng một sản phẩm đã được kiểm chứng**, loại bỏ một gánh nặng thay vì chỉ di dời nó.

10.3. Tách database — thách thức lớn

10.3.1. Vấn đề: database coupling nguy hiểm hơn code coupling

Tách mã nguồn (code) thành các dịch vụ (service) không quá khó — tách API, triển khai độc lập. Nhưng khi hai dịch vụ **chia sẻ cùng cơ sở dữ liệu** chúng vẫn bị ràng buộc (coupled) ở tầng sâu nhất:



Hình 10.3: Shared Database (coupled) vs Database-per-Service (decoupled)

Newman trong [4b, Ch.1] viết: **"One of the things I see people having the hardest time with is the idea that microservices should not share databases"** — và LMS là ví dụ trực tiếp: Core Service và Academic Service (`kbm-academic`) chia sẻ cùng database, những thay đổi về lược đồ (schema) ảnh hưởng đến cả hai, và hệ thống không thể triển khai các bản cập nhật cấu trúc cơ sở dữ liệu (database migration) một cách độc lập.

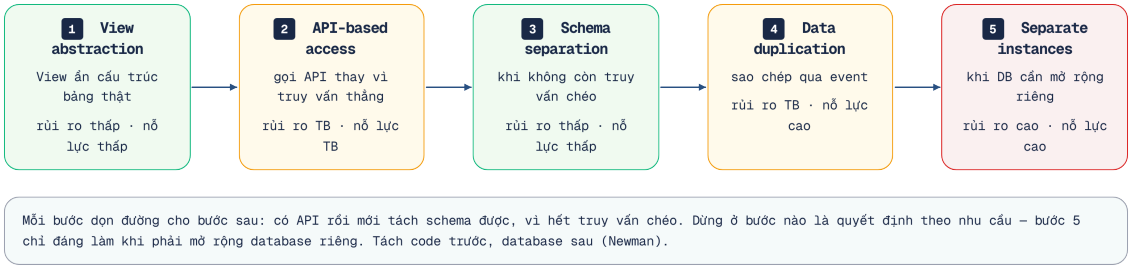
Từ Chương 7 (Database-per-Service), chúng ta đã biết danh mục năm chiến lược tách dữ liệu. Giờ hãy nhìn lại từ góc độ chuyển đổi — **thứ tự thực hiện** (kèm tên tương ứng đã học ở §7.2):

Bảng 10.3: 5 chiến lược tách database — từ góc độ chuyển đổi

#	Chiến lược (Strategy)	Mô tả	Rủi ro (Risk)	Nỗ lực (Effort)	Khi nào
1	**View abstraction** *(Database View, Ch.7)*	Tạo View ẩn đi cấu trúc bảng thực tế	● Thấp	Thấp	Bước đầu tiên — khi service B cần dữ liệu của service A
2	**API-based access** *(Database Wrapping Service, Ch.7)*	Service B gọi API service A thay vì truy vấn trực tiếp	● Trung bình	Trung bình	Khi thay thế việc gọi qua View/DB
3	**Schema separation** *(Separate Schema, Ch.7)*	Tách lược đồ (schema) trong cùng DB instance	● Thấp	Thấp	Khi không còn truy vấn chéo (nhờ API)

#	Chiến lược (Strategy)	Mô tả	Rủi ro (Risk)	Nỗ lực (Effort)	Khi nào
4	**Data duplication** *(Data Transfer, Ch.7)*	Sao chép dữ liệu thông qua các sự kiện (eventual consistency)	● Trung bình	Cao	Khi truy vấn chéo giữa các dịch vụ (cross-service query) trở nên phức tạp
5	**Separate instances** *(Full Database Split, Ch.7)*	Database instance riêng cho mỗi service	● Cao	Cao	Bước cuối — khi cần mở rộng cơ sở dữ liệu riêng

Sự dịch chuyển qua năm chiến lược trên được mô tả trong sơ đồ:



Hình 10.4: *Thứ tự thực hiện 5 chiến lược tách database*

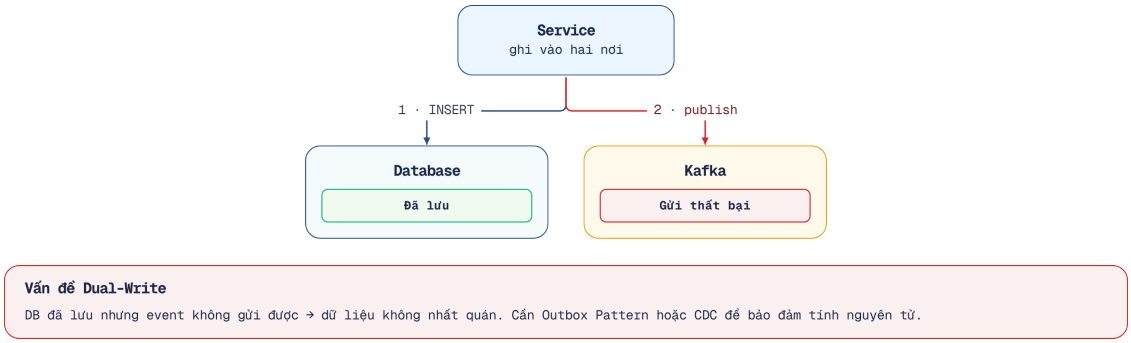
Tiến trình năm bước trên giúp giảm thiểu rủi ro và cho phép đội ngũ phát triển làm quen dần với độ phức tạp của dữ liệu phân tán. Đây là thứ tự tổng quát theo hướng *API trước*: gỡ truy vấn chéo rồi mới tách schema. Chương 7 đặt nó cạnh hướng *schema trước* — dựng ranh giới rẻ trong cùng DB server, tạm cho đọc chéo qua view rồi thay dần bằng API — và giải thích vì sao KBM, với hai service dùng chung một PostgreSQL và một nhóm nhỏ, chọn hướng sau (mục "Áp dụng cho KBM" ở §7.2). Lộ trình Giai đoạn 4 ở cuối chương đi theo lựa chọn đó.

Tách database tăng dần

> "We split both the code and data at once. [...] I strongly suggest avoiding this approach, and instead splitting either the schema or application tier first."
>
> — *Sam Newman, Monolith to Microservices [4b, Ch.4]*

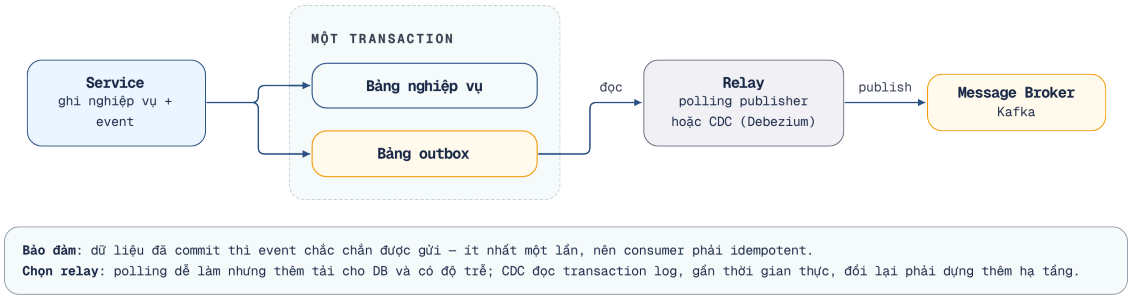
10.3.2. Cạm bẫy ghi kép (dual-write) và Outbox Pattern

Khi tách database, một anti-pattern phổ biến là **dual write**: dịch vụ vừa ghi vào cơ sở dữ liệu vừa phát tán sự kiện (publish event) — nếu một trong hai thất bại, dữ liệu sẽ không nhất quán.



Hình 10.5: *Cạm bẫy ghi kép — DB thành công nhưng event thất bại*

Trạng thái không nhất quán này đặc biệt nguy hiểm với việc nộp bài thi hoặc ghi nhận điểm số. Cách xử lý phổ biến là dùng chính database nội bộ làm điểm trung chuyển, tách thao tác ghi nghiệp vụ khỏi thao tác đẩy dữ liệu qua mạng. **Outbox Pattern** giải quyết vấn đề bằng cách ghi sự kiện vào bảng outbox *cùng một giao dịch* với dữ liệu nghiệp vụ chính; sau đó, một tiến trình ngầm sẽ đọc bảng này và đẩy thông điệp lên message broker:



Hình 10.6: *Outbox Pattern — sự kiện được ghi cùng giao dịch với dữ liệu nghiệp vụ*

Có hai cách để tiến trình ngầm đọc và đẩy sự kiện từ bảng outbox. Phương pháp thứ nhất là **Polling publisher**, sử dụng một luồng chạy ngầm để liên tục kiểm tra bảng outbox; cách này dễ triển khai nhưng tạo thêm tải cho cơ sở dữ liệu và có độ trễ nhất định. Phương pháp thứ hai tiên tiến hơn là sử dụng **CDC (Change Data Capture)** như Debezium để đọc trực tiếp từ transaction log; cách này cho độ trễ thấp và giảm tải polling (độ trễ thực tế phụ thuộc tải và vận hành, cần đo lag), nhưng đòi hỏi phải thiết lập hạ tầng phức tạp hơn.

Outbox Pattern

Giống như sinh viên nộp bài thi (business data) và ký tên vào danh sách điểm danh (event) *cùng một lúc* tại bàn giáo viên. Sau đó, một giám thị khác (tiến trình ngầm) mới cầm danh sách điểm danh đó báo cáo cho Phòng Đào tạo (Message Broker).

Để bài làm của sinh viên không bị mất khi sự cố mạng xảy ra giữa lúc ghi dữ liệu và phát sự kiện, dưới đây là cách cài đặt Outbox Pattern vào hệ thống nộp bài:

Listing 10.2: *Mã nguồn Transactional Outbox Pattern (lưu cả hai thực thể trong một giao dịch)*

```
@Service
public class SubmissionService {
    private final SubmissionRepository submissionRepo;
    private final OutboxRepository outboxRepo;

    public SubmissionService(SubmissionRepository submissionRepo,
                             OutboxRepository outboxRepo) {
        this.submissionRepo = submissionRepo;
        this.outboxRepo = outboxRepo;
    }

    @Transactional
    public void submitExam(Submission submission) {
        // 1. Lưu business data
        submissionRepo.save(submission);
```

```
        // 2. Lưu event vào bảng Outbox (cùng transaction)
        OutboxEvent event = new OutboxEvent(
            "SubmissionCreated",
            toJson(submission)
        );
        outboxRepo.save(event);
    }

    // toJson(...): hàm phụ chuyển Submission sang JSON, lược bớt
}
```

KBM chưa có Outbox Pattern

KBM hiện dùng **dual write**: Core Service lưu bài nộp vào DB rồi gọi `submitProducer.sendSubmission()` đến Kafka. Nếu Kafka không khả dụng (unavailable) tại thời điểm đó, bài nộp (submission) đã lưu nhưng Judge Service không nhận được bài — sinh viên thấy "đã nộp" nhưng không nhận kết quả chấm.

Lộ trình chuyển đổi: (1) Ngắn hạn — cơ chế thử lại (retry logic) cho Kafka producer (đã có nhưng cần được kiểm chứng lại một cách kỹ lưỡng), (2) Trung hạn — bảng outbox + polling publisher, (3) Dài hạn — Debezium CDC. Với lưu lượng truy cập (traffic) thấp của KBM hiện tại, polling publisher (trung hạn) đủ tốt — Debezium chỉ cần khi mở rộng quy mô trên môi trường thực tế lớn.

10.3.3. CDC cho chuyển đổi dữ liệu

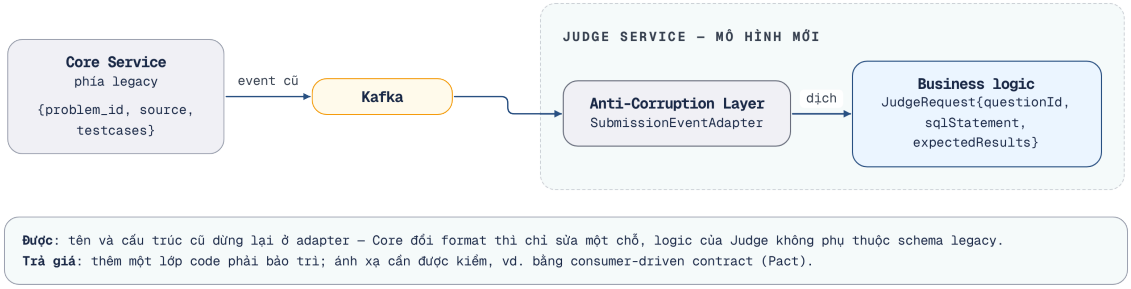
Lưu ý quan trọng: Trong chương này, chúng ta mới nhắc CDC trong ngữ cảnh Outbox Pattern. Tuy nhiên, CDC (như Debezium) thực chất là công cụ lõi để **đồng bộ dữ liệu thời gian thực (Real-time Sync)** khi phân tách cơ sở dữ liệu. Monolith vẫn tiếp tục ghi vào DB cũ, CDC sẽ nắm bắt (capture) các thay đổi đó và đẩy sang DB của Microservice mới liên tục. Nhờ vậy, cửa sổ gián đoạn khi chuyển đổi (cut-over) được rút xuống rất nhỏ (near-zero downtime): chỉ cần chặn thao tác ghi vài giây để CDC đuổi kịp độ trễ sao chép (replication) rồi mới chuyển hướng. Zero downtime tuyệt đối đòi hỏi thêm dual-write và cơ chế xử lý xung đột — chi phí thường không tương xứng ở quy mô KBM.

10.4. Anti-Corruption Layer và các pattern chuyển đổi

Khi dịch vụ mới bắt buộc phải gọi đến hệ thống di sản (legacy), chúng ta cần một lớp đệm trung gian để chuyển đổi mô hình dữ liệu, ngăn chặn thiết kế lạc hậu của hệ thống cũ ảnh hưởng đến kiến trúc mới.

10.4.1. Anti-Corruption Layer (ACL) — bảo vệ ranh giới

Khi tách dịch vụ mới ra khỏi monolith, dịch vụ mới cần giao tiếp với mã nguồn di sản (legacy) — nhưng **không nên để mô hình (model) cũ ảnh hưởng tiêu cực đến mã nguồn mới**. Eric Evans trong [6] định nghĩa **Anti-Corruption Layer** (ACL): một lớp dịch ngôn ngữ giữa hai Bounded Context có mô hình (model) khác nhau.



Hình 10.7: *Anti-Corruption Layer — lớp dịch giữa service mới và legacy*

Ví dụ minh họa với KBM: Judge Service nhận các bài nộp (submission) từ Core Service qua Kafka. Giả sử Core Service còn truyền dữ liệu theo định dạng cũ `{problem_id, source, testcases}`. Judge Service có thể: (1) × dùng trực tiếp tên trường dữ liệu di sản (legacy field name) trong mã nguồn — bị ràng buộc (coupling) với các quyết định của hệ thống di sản, hoặc (2) √ tạo adapter dịch sang mô hình nội bộ `JudgeRequest{questionId, sqlStatement, expectedResults}` — ranh giới rõ ràng (clean boundary).

ACL hoạt động như một "Thông dịch viên" hoặc hệ thống quy đổi tín chỉ: nó giúp dịch vụ mới tích hợp với hệ thống cũ (Monolith) mà không bị xung đột dữ liệu vì khác biệt mô hình miền (Domain Model).

Đoạn mã dưới đây minh họa cách một Adapter thực hiện chuyển đổi luồng dữ liệu một cách trong suốt:

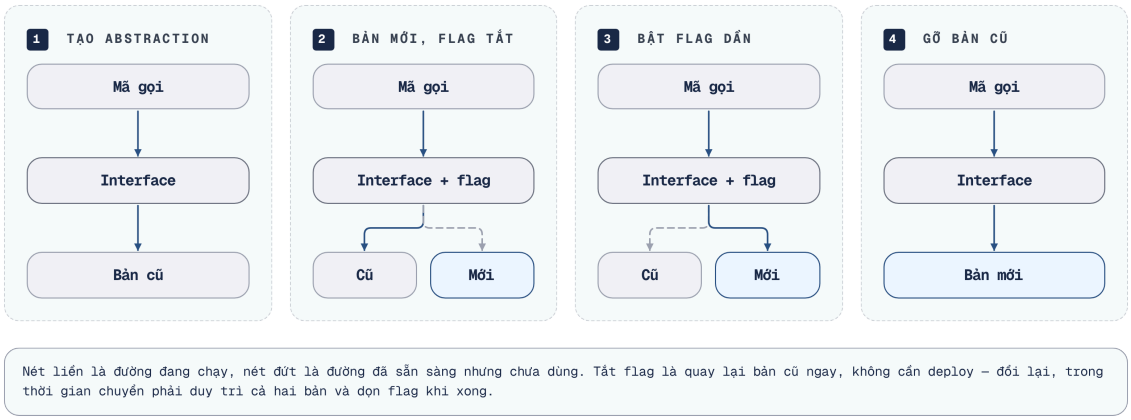
Listing 10.3: *Adapter trong Anti-Corruption Layer dịch sự kiện cũ (legacy event) sang mô hình mới*

```
@Component
public class SubmissionEventAdapter {
    // Dịch từ model cũ (Monolith) sang model mới (Judge Service)
    public JudgeRequest translate(LegacySubmissionEvent legacyEvent) {
        return new JudgeRequest(
            legacyEvent.getProblemId(),
            legacyEvent.getSource(),
            parseTestcases(legacyEvent.getTestcases()) // Phân tích định dạng cũ
        );
    }

    // parseTestcases(...): hàm phụ phân tích định dạng cũ, lược bớt
}
```

Khi bóc tách service, để đảm bảo việc thay đổi ở service mới không vô tình phá vỡ monolith cũ đang gọi nó, chúng ta nên áp dụng **Consumer-Driven Contracts** (ví dụ framework Pact) để tự động hóa việc kiểm tra tính tương thích API giữa hai hệ thống.

10.4.2. Branch by Abstraction

Newman trong [4b, Ch.3] đề xuất **Branch by Abstraction** cho việc chuyển đổi an toàn:



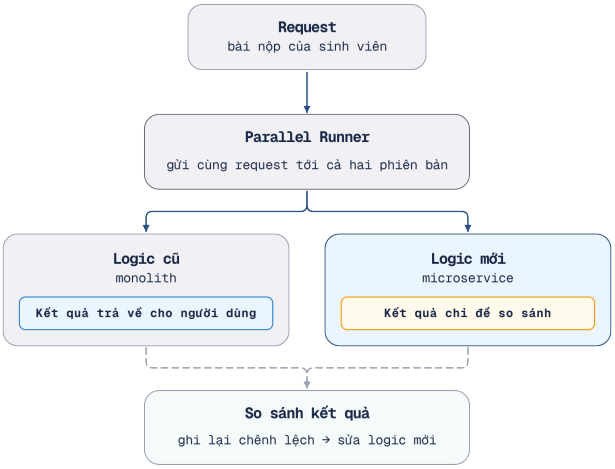
Hình 10.8: Branch by Abstraction — chuyển đổi an toàn qua feature flag

1. **Tạo lớp trừu tượng** (abstraction layer, interface) che chắn cho phần hiện thực cũ
2. **Hiện thực hóa phiên bản mới** phía sau cùng interface, kiểm soát bằng feature flag
3. **Chuyển lưu lượng** dần sang phiên bản mới và kiểm chứng
4. **Gỡ bỏ** phiên bản cũ khi đã chắc chắn về tính ổn định

Để quản lý Feature Flag chuyên nghiệp trong Java, thay vì dùng `if-else` cứng và cấu hình properties, chúng ta nên dùng các thư viện chuyên dụng như **Togglz**, **Unleash**, hoặc **FF4J** / **OpenFeature**. Các thư viện này cho phép bật/tắt luồng logic ngay tại runtime mà không cần khởi động lại dịch vụ.

10.4.3. Parallel Run — kiểm chứng trước khi chuyển đổi

Parallel Run — chiến lược cho phép **chạy đồng thời** cả phiên bản hiện thực cũ và mới, sau đó đối chiếu kết quả đầu ra. Ở cấp độ hạ tầng, API Gateway có thể áp dụng **Traffic Shadowing (Dark Launching)**: nhân bản âm thầm (shadow) một luồng lưu lượng (traffic) thực tế và chuyển hướng đến dịch vụ mới để kiểm tra tải và kiểm tra logic mà người dùng không hề hay biết.



Hình 10.9: Parallel Run — chạy song song logic cũ và mới, so sánh kết quả

- Phiên bản cũ **trả kết quả (response)** cho người dùng (đảm bảo an toàn — production-safe)

- Phiên bản mới chạy **song song** — kết quả trả về bị loại bỏ (discard)
- So sánh hai kết quả — ghi nhận (log) các điểm sai lệch (mismatch)
- Khi tỷ lệ sai lệch (mismatch rate) ≈ 0% → chuyển đổi (switch) sang phiên bản mới

⚠ Parallel Run chỉ an toàn với thao tác đọc

Với thao tác ghi (INSERT, UPDATE), chạy song song sẽ tạo tác dụng phụ kép (ghi hai lần). Để kiểm chứng an toàn, hãy dùng kỹ thuật **Shadow Database** hoặc **Discard Writes** — loại bỏ mọi thao tác ghi của phiên bản mới.

Áp dụng cho KBM: nếu cần viết lại `CompareUtil` (logic so sánh kết quả SQL — quan trọng nhất trong hệ thống chấm điểm), Parallel Run cho phép chạy phiên bản `CompareUtil` cũ và mới song song trên mỗi bài nộp, so sánh kết quả để đảm bảo logic chấm điểm mới hoàn toàn khớp trước khi chính thức chuyển đổi. Tuy nhiên, hệ thống phải hủy bỏ (discard) các thao tác ghi (write/side-effect) của phiên bản mới để tránh làm hỏng dữ liệu thật.

Chiến lược Parallel Run giúp xác minh tính chính xác của phiên bản mới bằng dữ liệu thực tế (production data) trước khi chuyển hướng hoàn toàn lưu lượng truy cập (cut-over).

Chuyển đổi phải hoàn tác được

Mỗi bước chuyển đổi phải hoàn tác được. Nếu không quay lui dễ dàng, nghĩa là đang nhận quá nhiều rủi ro một lần. Feature flag, parallel run và định tuyến tăng dần đều phục vụ cùng một mục đích: *làm cho việc thất bại trở nên an toàn.*

— Theo tinh thần Sam Newman, Monolith to Microservices [4b, Ch.2–3]

10.5. Quá trình chuyển đổi kiến trúc tại KBM

Phần này tổng hợp các khoảng trống đã phân tích từ Chương 2–9, đồng thời xem trước (preview) hai điều kiện tiên quyết về khả năng quan sát (Chương 11) và tự động hóa triển khai (Chương 12) thành một danh mục duy nhất, rồi sắp xếp thành lộ trình chuyển đổi bốn giai đoạn ưu tiên theo Impact × Effort.

10.5.1. Tổng hợp các phân tích khoảng trống

Từ Chương 2–9, chúng ta đã phân tích khoảng trống (gap) giữa hiện trạng KBM tại thời điểm khảo sát (đầu năm 2026) và các giải pháp khuyến nghị. Tuy nhiên, thực tế chuyển đổi cho thấy không thể tách database an toàn nếu hệ thống chưa có khả năng quan sát và quy trình triển khai tự động — hai chủ đề sẽ được học chi tiết ngay ở Chương 11 và Chương 12. Vì vậy, Bảng 10.4 gom các khoảng trống từ Chương 2–9 kèm hai điều kiện tiên quyết xem trước của Chương 11–12 (bức tranh tích hợp toàn cục sẽ được nhìn lại ở phần Kết luận):

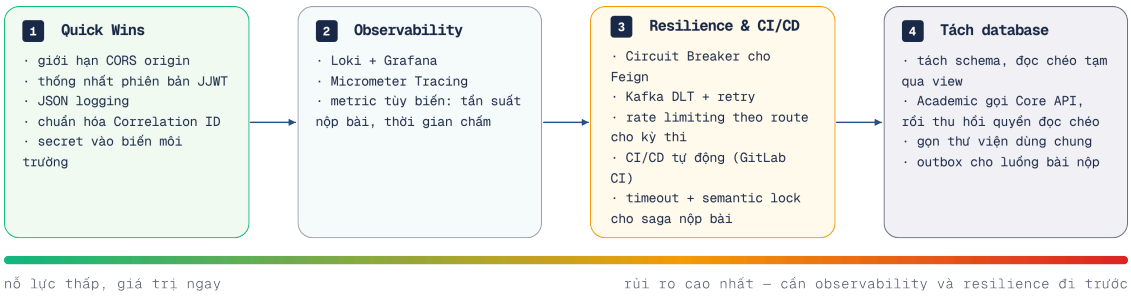
Bảng 10.4: *Tổng hợp khoảng trống của KBM tại thời điểm khảo sát (Chương 2–9) và hai điều kiện tiên quyết xem trước (Chương 11–12)*

Chương	Khoảng trống	Mức độ	Tham chiếu
Ch.2	7 core Bounded Context rõ hơn nhưng thư viện dùng chung vẫn gây coupling	● Trung bình	§2.5
Ch.3	Đặt tên/phiên bản API chưa nhất quán; contract worker của SQL Judge cần chuẩn hóa	● Trung bình	§3.9

Chương	Khoảng trống	Mức độ	Tham chiếu
Ch.4	SQL Judge coordinator/worker và tích hợp bên ngoài cần ACL/resilience rõ hơn	● Trung bình	§4.5
Ch.5	Kafka pipeline cần DLT/idempotency; SQL Judge dispatch và DevOps Lab queue cần chọn broker theo khối lượng công việc (workload)	● Trung bình	§5.9
Ch.6	Saga nộp bài ngầm định: thiếu timeout, compensation, semantic lock	● Trung bình	§6.6
Ch.7	**Shared database** (Core ↔ Academic)	● Cao	§7.6
Ch.8	Spring Cloud Gateway + Go router cần thống nhất chính sách ở biên	● Cao	§8.5
Ch.9	HS256 shared secret, lưu trữ token, quản lý secret, SSO đa nền tảng	● Cao	§9.6
Ch.11 *(xem trước)*	Observability có Actuator/Prometheus một phần, chưa có truy vết đầu cuối và log tập trung	● Cao	§11.9
Ch.12 *(xem trước)*	LMS chính dùng Docker Compose; DevOps Lab dùng k3s/Sysbox; CI/CD và rollback chưa đồng đều	● Cao	§12.7

10.5.2. Lộ trình chuyển đổi — 4 giai đoạn

Nguyên tắc ưu tiên: **Quick Wins trước, các thay đổi rủi ro cao (High-Risk) thực hiện sau**. Mỗi giai đoạn mang lại giá trị ngay lập tức — không cần đợi hoàn thành toàn bộ.



Hình 10.10: *Lộ trình chuyển đổi 4 giai đoạn — Quick Wins → Observability → Resilience → Decomposition*

Việc chia nhỏ thành bốn giai đoạn giúp đội ngũ ưu tiên giải quyết các vấn đề có tác động cao nhưng đòi hỏi nỗ lực thấp (High Impact / Low Effort) trước, và hoãn lại các thay đổi cơ sở dữ liệu rủi ro cao cho đến khi có đủ cơ chế giám sát.

Giai đoạn 1 — Các cải tiến nhanh (Quick Wins): nỗ lực thấp, tác động cao

Giai đoạn mở đầu tập trung vào những thay đổi nhỏ, tốn ít công sức nhưng mang lại hiệu quả bảo mật và vận hành đáng kể, không đòi hỏi thay đổi kiến trúc cốt lõi.

Bảng 10.5: *Giai đoạn 1 — Quick Wins*

#	Hành động	Khoảng trống	Nỗ lực	Tác động
1	Giới hạn CORS origin	Ch.8	30 phút	Thu hẹp chính sách CORS quá rộng
2	Thống nhất phiên bản JJWT trong parent POM	Ch.8	1 giờ	Ngăn lỗi sai lệch phiên bản
3	Chuyển sang JSON logging (Logback encoder)	Ch.11	2 giờ	Log có thể tìm kiếm

#	Hành động	Khoảng trống	Nỗ lực	Tác động
4	Chuẩn hóa ngữ cảnh truy vết: `X-Request-ID` ở HTTP, `traceparent` và `correlationId` qua Kafka	Ch.8, Ch.11	4 giờ	Gỡ lỗi xuyên dịch vụ và luồng bất đồng bộ
5	Đưa thông tin nhạy cảm ra khỏi tệp cấu hình, nạp qua biến môi trường hoặc kho secret; luân phiên secret định kỳ	Ch.9	2 giờ	Giảm nguy cơ lộ secret

Triết lý Giai đoạn 1: Không thay đổi kiến trúc tổng thể, không can thiệp sâu vào logic cốt lõi — chỉ tập trung **chuẩn hóa cấu hình và tăng cường bảo mật**. Một kỹ sư duy nhất cũng đủ sức hoàn thành nhóm công việc này trong một đến hai sprint — và đây là bước đệm cần thiết trước khi bắt tay vào các phần phân tách phức tạp hơn.

Giai đoạn 2 — Nền tảng Observability

Sau Giai đoạn 1, hệ thống cần được trang bị khả năng giám sát đầy đủ trước khi tiến hành phân tách các thành phần phức tạp.

Bảng 10.6: *Giai đoạn 2 — Nền tảng Observability*

#	Hành động	Khoảng trống	Nỗ lực	Tác động
1	Triển khai Loki + Grafana	Ch.11	1–2 ngày	Tra cứu log tập trung
2	Thêm Micrometer Tracing	Ch.11	1 ngày	Biết nút thắt cổ chai (bottleneck) ở đâu
3	Metric tùy biến (tần suất nộp bài, thời gian chấm)	Ch.11	2 ngày	Giám sát chủ động (proactive monitoring)

Triết lý Giai đoạn 2: Trước khi thực hiện các thay đổi kiến trúc phức tạp ở Giai đoạn 3 và 4, hệ thống cần có cơ chế giám sát tốt. Dashboard và hệ thống log tập trung là công cụ quan trọng để gỡ lỗi.

Giai đoạn 3 — Resilience & CI/CD

Với khả năng giám sát đã sẵn sàng, bước tiếp theo là xây dựng cơ chế phòng thủ để ứng dụng chịu được lỗi và tự động hóa quy trình phân phối.

Bảng 10.7: *Giai đoạn 3 — Resilience & CI/CD*

#	Hành động	Khoảng trống	Nỗ lực	Tác động
1	Resilience4j Circuit Breaker cho Feign client	Ch.4	2–3 ngày	Ngăn chặn lỗi dây chuyền (cascading failure)
2	Kafka Dead Letter Topic (DLT) + retry	Ch.5	2 ngày	Giảm thất lạc message đã vào broker; khoảng trống DB–broker cần Outbox (§10.3)
3	Áp chính sách rate limiting theo route cho kỳ thi (trên token bucket Redis đã có)	Ch.8	1–2 ngày	Bảo vệ hệ thống khi có lưu lượng lạm dụng
4	Hoàn thiện CI/CD triển khai tự động (GitLab CI)	Ch.12	3–5 ngày	Tự động hóa biên dịch và triển khai
5	Timeout + semantic lock cho saga nộp bài	Ch.6	2–3 ngày	Bài nộp không treo ở PENDING

Triết lý Giai đoạn 3: Hệ thống phải chứng minh được **tính bền bỉ trước khi bị xẻ nhỏ**. Việc phân tách cơ sở dữ liệu khi chưa có cơ chế ngắt mạch (circuit breaker) hay hệ thống cảnh báo sẽ dễ gây ra lỗi dây chuyền một khi có bất kỳ dịch vụ nào gặp sự cố nghẽn mạng.

Giai đoạn 4 — Phân tách cơ sở dữ liệu (database decomposition): thận trọng nhất
Bước cuối cùng — tháo gỡ sự phụ thuộc ở tầng dữ liệu — là thử thách lớn, và đòi hỏi sự ổn định từ ba giai đoạn trước.

__Bảng 10.8:__ Giai đoạn 4 — Phân tách cơ sở dữ liệu

#	Hành động	Khoảng trống	Nỗ lực	Tác động
1	Schema separation (`core_schema`, `academic_schema`); truy vấn chéo còn lại tạm đọc qua view	Ch.7	1–2 tuần	Ranh giới thấy được, đếm được truy vấn chéo
2	API-based access (Academic gọi Core API); hết truy vấn chéo thì thu hồi quyền đọc chéo	Ch.7	2–3 tuần	Loại bỏ truy cập DB trực tiếp
3	Refactor thư viện dùng chung (chỉ giữ DTO, ngoại lệ)	Ch.2	2–3 tuần	Giảm thiểu sự phụ thuộc (coupling)
4	Outbox pattern cho luồng xử lý bài nộp	Ch.5, Ch.6	1–2 tuần	Giao tiếp tin cậy (reliable messaging)

Triết lý Giai đoạn 4: Đây là tập hợp những thay đổi chứa đựng **nhiều rủi ro nhất** trong toàn bộ lộ trình. Chúng ta dựa vào nền tảng quan sát (Giai đoạn 2) và tính bền bỉ (Giai đoạn 3) để thực hiện bước này. Mỗi thay đổi cần được triển khai từ từ, theo dõi qua dashboard, khi ổn định mới tiến hành bước tiếp theo.

10.5.3. Ma trận quyết định — ưu tiên theo tác động × nỗ lực

Để chọn việc làm trước trong lộ trình, Hình 10.11 xếp các hạng mục theo hai trục: tác động và nỗ lực.

__Hình 10.11:__ Ma trận quyết định — ưu tiên theo tác động × nỗ lực

Chuyển đổi là cuộc chạy đường dài

Đừng cố gắng khắc phục mọi khoảng trống cùng lúc. Trong mỗi chu kỳ phát triển (sprint), chỉ nên ưu tiên 1–2 hạng mục thuộc nhóm “Tác động cao, nỗ lực thấp (High Impact, Low Effort)” trước. Khi đã hết các thành quả nhanh (quick wins), mới chuyển sang nhóm “Tác động cao, nỗ lực cao” — và luôn phải đảm bảo khả năng quan sát (observability) trước khi thực hiện các thay đổi lớn.

10.6. Sai lầm thường gặp khi chuyển đổi

Ngoài 4 anti-pattern ở Bảng 10.2 (§10.1), quá trình chuyển đổi còn có những rủi ro cần được nhận diện và xử lý sớm:

- **Distributed Monolith**: Tách các dịch vụ (service) nhưng vẫn dùng chung cơ sở dữ liệu mà không làm rõ ranh giới sở hữu, dẫn đến quy trình triển khai phải đồng bộ. Newman trong [4b, Ch.1] nhận xét loại hệ thống này "have all the disadvantages of a distributed system, and the disadvantages of a single-process monolith, without having enough upsides of either". *Phòng tránh* bằng cách xác lập rõ quyền sở hữu từng bảng (logical ownership), tách schema/quyền truy cập và thay truy vấn chéo bằng API trước khi tách cơ sở dữ liệu vật lý theo nhu cầu mở rộng.
- **Big Bang Rewrite**: Viết lại toàn bộ từ đầu (from scratch), ngày chuyển đổi ("switch") không bao giờ đến. *Phòng tránh* bằng cách sử dụng Strangler Fig (§10.2) — mỗi chu kỳ phát triển (sprint) tách một phần, hệ thống luôn ở trạng thái sẵn sàng triển khai (deployable).
- **Bỏ qua Conway's Law**: Phân tách thành các dịch vụ độc lập nhưng cấu trúc đội ngũ phát triển vẫn như cũ. *Phòng tránh* bằng cách tổ chức lại đội ngũ theo các Bounded Context (Chương 2) — mỗi nhóm chỉ nên sở hữu 1–2 dịch vụ.
- **"Lift and Shift" không thiết kế lại**: Sao chép nguyên bản mã nguồn vào các container, và ngộ nhận đó là kiến trúc microservices. *Phòng tránh* bằng cách sử dụng Anti-Corruption Layer (§10.4) ngăn chặn mô hình dữ liệu cũ (legacy model) xâm nhập vào kiến trúc mới.
- **Hạ tầng phức tạp quá mức cần thiết**: Triển khai K8s + Istio cho 3 dịch vụ với 100 yêu cầu/phút. *Phòng tránh* bằng cách bắt đầu đơn giản (Docker Compose, xem trước Chương 12), chỉ mở rộng quy mô khi lưu lượng truy cập thực sự đòi hỏi.

Các pattern trong chương có minh họa tương tác đi kèm tại companion web:

Interactive Demo

- **Chiến lược Strangler Fig Pattern**: strangler-fig-migration.html
- **Transactional Outbox**: outbox-pattern.html
- **CDC và Outbox**: cdc-outbox.html

10.7. Tổng kết

Chuyển đổi từ monolith sang microservices không phải quyết định kỹ thuật thuần túy — đây là sự kết hợp giữa **tổ chức đội ngũ (team organization)** (Conway's Law, Chương 2), **hiểu biết về miền nghiệp vụ (domain understanding)** (Bounded Context, Chương 2), **các mô hình giao tiếp (communication pattern)** (Chương 3–6), **chiến lược dữ liệu (data strategy)** (Chương 7), **sự sẵn sàng của hạ tầng biên và bảo mật** (Chương 8–9), và **mức độ trưởng thành trong vận hành** (sẽ học ở Chương 11–12).

Bài học quan trọng nhất: **"Monolith First"** — bắt đầu đơn giản, chỉ chuyển đổi (migrate) khi có lý do cụ thể, thực hiện chuyển đổi tăng dần (Strangler Fig), và mỗi bước đều phải có khả năng hoàn tác (reversible). Việc đập đi xây lại toàn bộ (Big Bang rewrite) gần như luôn thất bại; Strangler Fig giảm rủi ro đáng kể — vì nó cho phép chấp nhận các rủi ro ở quy mô nhỏ, rút kinh nghiệm nhanh chóng và liên tục mang lại giá trị thực tiễn.

Tách database là một trong những thách thức lớn nhất — sự phụ thuộc về mã nguồn có thể giải quyết qua tái cấu trúc (refactoring), nhưng sự phụ thuộc về dữ liệu đòi hỏi một chiến lược chuyển

đổi thận trọng: đi theo tiến trình tổng quát *API trước* (view abstraction → API-based access → schema separation → data duplication → separate instances) hoặc lộ trình *schema trước* khi nhóm nhỏ dùng chung một DB server như KBM (tách schema để lộ truy vấn chéo, thay dần bằng API rồi chỉ tách DB vật lý khi thực sự cần mở rộng riêng). Outbox Pattern giải quyết bài toán giao tiếp tin cậy (reliable messaging) trong quá trình chuyển đổi — giúp tránh cái bẫy ghi dữ liệu kép (dual-write pitfall).

Anti-Corruption Layer, Branch by Abstraction, và Parallel Run là bộ công cụ cho việc chuyển đổi an toàn — mỗi pattern phục vụ mục đích khác nhau nhưng chia sẻ nguyên tắc chung: **làm cho quá trình chuyển đổi có thể đảo ngược, chia nhỏ từng bước tiến hành, và đảm bảo hệ thống luôn trong trạng thái sẵn sàng triển khai**.

Lộ trình chuyển đổi cho KBM — tổng hợp từ các khoảng trống ở Chương 2–9 cùng hai điều kiện tiên quyết xem trước ở Chương 11–12 — cho thấy: các Quick Wins như CORS, JWT, log có cấu trúc (structured logging) có thể hoàn thành trong 1–2 tuần, mang lại giá trị ngay. Phân tách cơ sở dữ liệu (Database decomposition) — thay đổi lớn nhất — cần được thiết kế dựa trên nền tảng về khả năng quan sát (observability) và tính bền bỉ (resilience). DevOps Lab cho thấy cùng một hệ thống có thể cần chiến lược khác nhau theo Bounded Context: Docker Compose đủ tốt cho LMS chính, trong khi việc cô lập phòng thí nghiệm cần k3s/Sysbox. Mỗi quyết định thiết kế đều là một sự đánh đổi — không có phương án "đúng tuyệt đối", chỉ có phương án "phù hợp nhất với bối cảnh hệ thống tại thời điểm đó".

Ở Chương 11, chúng ta sẽ xem cách **quan sát và giám sát** hệ thống sau khi chuyển đổi — logging, metric, tracing — ba kỹ thuật trọng tâm giúp phát hiện và xử lý vấn đề trước khi người dùng bị ảnh hưởng.

10.8. Đọc thêm

Sách tham khảo chính:

1. [4b] Sam Newman, *Monolith to Microservices* — Toàn bộ sách: decomposition patterns, database splitting, migration strategies
2. [2a] Chris Richardson, *Microservices Patterns* 1st Ed. — Ch.13: Refactoring to Microservices — Strangler Fig, Anti-Corruption Layer
3. [4a] Sam Newman, *Building Microservices* — Ch.3: How to Model Services (decomposition), Ch.5: Splitting the Monolith

Sách bổ trợ:

4. [6] Eric Evans, *Domain-Driven Design* — Ch.14: Anti-Corruption Layer, Context Mapping
5. [3] Ronnie Mitra & Irakli Nadareishvili, *Microservices: Up and Running* — Ch.3: Designing Microservices (SEED(S))
6. [2b] Chris Richardson, *Microservices Patterns* 2nd Ed. — Ch.2: Migration Anti-patterns; Ch.21: Refactoring a monolith to microservices (Strangler Fig, cập nhật)

Nguồn trực tuyến:

- [W2] Martin Fowler, "MonolithFirst" — martinfowler.com/bliki/MonolithFirst.html
- [W8] Martin Fowler, "StranglerFigApplication" — martinfowler.com/bliki/StranglerFigApplication.html
- Martin Fowler, "MicroservicePremium" — martinfowler.com/bliki/MicroservicePremium.html

- Sam Newman, “Breaking Apart the Monolith” — samnewman.io
- Debezium (CDC) — debezium.io
- Martin Fowler, “BranchByAbstraction” — martinfowler.com/bliki/BranchByAbstraction.html

Bài tập: 10-0, 10-1, 10-2, 10-3 ở phần Bài tập & case study cuối sách.

CHƯƠNG 11

Quan sát hệ thống — Observability & Distributed Tracing

> Observability không chỉ là giám sát, mà là khả năng đặt câu hỏi mới về hành vi của hệ thống mà không phải triển khai mã mới.
> — Theo tinh thần Charity Majors

MỤC TIÊU HỌC TẬP

- Hiểu tại sao observability là **điều kiện tiên quyết** để vận hành microservices — không chỉ là thứ "có thì tốt"
- Phân biệt **observability và monitoring** và khi nào cần gì
- Nắm vững ba trụ cột: **Logs**, **Metrics**, **Traces** và cách chúng bổ trợ nhau
- Thiết kế chiến lược **log tập trung** (centralized logging) với log có cấu trúc (structured log), request ID và correlation ID
- Hiểu **truy vết phân tán** (distributed tracing) và các khái niệm trace, span, context propagation
- Nắm framework đo lường: **SLI/SLO/SLA** và chiến lược cảnh báo (alerting)
- Xây dựng **chiến lược xử lý lỗi** nhất quán giữa các service
- Phân tích các khoảng trống observability trong KBM và đề xuất lộ trình chuyển đổi

Phạm vi chương

Chương này tập trung vào nguyên tắc và thiết kế observability độc lập với hạ tầng triển khai cụ thể. Sau khi đọc Chương 12 (Triển khai & DevOps), người đọc sẽ thấy rõ hơn cách các công cụ observability được tích hợp vào CI/CD pipeline và môi trường chạy container.

Để vận hành hệ thống phân tán, các nhóm phát triển cần chuyển từ tư duy giám sát thụ động (monitoring: hệ thống có đang hoạt động bình thường không?) sang khả năng quan sát chủ động (observability: tại sao hệ thống chạy chậm?).

11.1. Ba trụ cột của observability

Để đảm bảo hệ thống LMS hoạt động trơn tru, chúng ta cần xây dựng cơ chế giám sát dựa trên ba trụ cột cốt lõi, giúp định vị chính xác mọi sự cố.

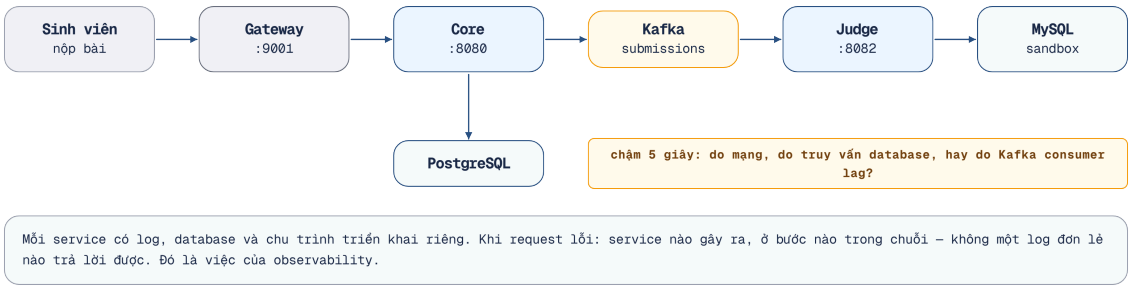
11.1.1. Vấn đề: "hệ thống lỗi nhưng không biết lỗi ở đâu"

Trong monolith, khi xảy ra lỗi: chỉ cần mở log file duy nhất, tìm ngoại lệ (exception) và đọc stack trace là có thể định vị được vấn đề. Toàn bộ request chạy trong cùng tiến trình, cùng log file, cùng database.

Trong kiến trúc microservices, một yêu cầu từ người dùng có thể đi qua nhiều dịch vụ, mỗi dịch vụ có nhật ký hệ thống, cơ sở dữ liệu và chu trình triển khai độc lập. Khi một yêu cầu gặp lỗi:

- Service nào gây lỗi? Gateway? Core? Judge? Auth?
- Lỗi xảy ra ở bước nào trong chuỗi gọi?
- Độ trễ 5 giây — do mạng, do truy vấn database, hay do Kafka consumer lag?

Hình 11.1 cho thấy một request đi qua hơn năm service — lỗi có thể nằm ở bất kỳ chặng nào.

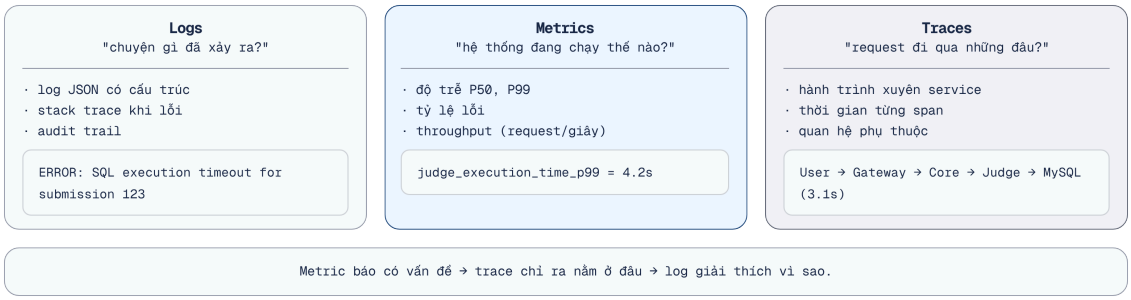


Hình 11.1: *Luồng request qua 5+ service — lỗi xảy ra ở đâu?*

Newman trong [4a, Ch.8] gọi đây là một thách thức lớn khi vận hành microservices: **sự phức tạp không nằm ở code, mà nằm ở tương tác giữa các service**. Trong monolith, lỗi thường có stack trace rõ ràng — một file, một method. Trong microservices, lỗi có thể xuất hiện ở dịch vụ B do dịch vụ A gửi dữ liệu sai, hoặc do message Kafka bị trễ — lúc này, không có stack trace nào ghi lại được nguyên nhân thật sự.

11.1.2. Ba trụ cột

Cộng đồng observability thường mô tả lĩnh vực này qua **ba trụ cột** (xem Sridharan, 2018); các kỹ thuật tương ứng có ở [2a, Ch.11] và [4a, Ch.8]:



Hình 11.2: Ba trụ cột của observability — Logs, Metrics, Traces

Hình 11.2 minh họa ba trụ cột ở mức tổng quan: mỗi trụ cột phục vụ một loại câu hỏi khác nhau khi gỡ lỗi hệ thống phân tán. Bảng dưới đây mô tả chi tiết vai trò, ví dụ cụ thể trong KBM, và các công cụ phổ biến cho từng trụ cột.

Bảng 11.1: Vai trò của ba trụ cột observability

Trụ cột	Trả lời câu hỏi	Ví dụ trong KBM	Công cụ phổ biến
Logs	"Chuyện gì đã xảy ra?"	`ERROR: SQL execution timeout for submission 123`	ELK Stack, Loki, Fluentd
Metrics	"Hệ thống đang hoạt động thế nào?"	`judge_execution_time_p99 = 4.2s`	Prometheus, Grafana, Micrometer
Traces	"Request đi qua những đâu?"	`User → Gateway → Core → Judge → MySQL (total: 3.1s)`	Jaeger, Grafana Tempo, Zipkin (dùng chuẩn OpenTelemetry OTLP)

M.E.L.T và hai nghĩa của Event

Trong công nghiệp, nhiều nền tảng giám sát (New Relic, Datadog) dùng thuật ngữ **M.E.L.T**, tách chữ *E* — *Events* thành yếu tố thứ tư. Cần phân biệt rõ: *Event* trong M.E.L.T là các **sự kiện vận hành/hạ tầng rời rạc** (triển khai phiên bản mới, khởi động lại container, sự kiện OOMKill, thay đổi cấu hình) giúp đối chiếu mốc thay đổi trên biểu đồ giám sát — khác với **Domain Event** mang ngữ nghĩa nghiệp vụ đi qua Message Broker (Kafka) đã học ở Chương 5–6.

Ba trụ cột **kết hợp với nhau:** metric cho biết *hệ thống đang có vấn đề* (thời gian phản hồi tăng), trace chỉ ra *vấn đề nằm ở đâu* (Judge Service chiếm 90% thời gian xử lý), log giải thích nguyên nhân *tại sao* (truy vấn SQL bị quá hạn do thiếu chỉ mục). Thiếu bất kỳ trụ cột nào, hai thành phần còn lại cũng mất đáng kể giá trị.

11.1.3. Observability và monitoring

Bảng 11.2 phân biệt hai khái niệm thường bị dùng lẫn này.

Bảng 11.2: Phân biệt monitoring và observability

	Monitoring	Observability
Mục đích	Phát hiện vấn đề đã biết	Khám phá vấn đề chưa biết
Cách tiếp cận	Đặt cảnh báo cho metric cụ thể	Khám phá dữ liệu để hiểu hành vi hệ thống
Ví dụ	"CPU > 80% → cảnh báo"	"Tại sao độ trễ tăng gấp 3 lúc 2 giờ chiều mỗi thứ Ba?"
Dữ liệu	Dashboard định sẵn	Logs + Metrics + Traces (kết hợp tùy biến theo nhu cầu)
Khi nào đủ	Hệ thống đơn giản, các kiểu lỗi (failure mode) biết trước	Hệ thống phân tán, các kiểu lỗi không dự đoán được

Monitoring trả lời: *"Có vấn đề không?"* Observability trả lời: *"Tại sao có vấn đề này, và vấn đề này khác gì so với lần trước?"* Với monolith, monitoring thường đủ. Với microservices — nơi các kiểu lỗi phức tạp và không dự đoán được — observability là yêu cầu bắt buộc.

Observability ≠ Monitoring

Monitoring cho biết *có* điều gì đó sai. Observability cho phép hỏi *vì sao*. Monitoring là một phần của observability: cần nhưng chưa đủ.

— Tổng hợp từ Charity Majors và cộng đồng observability

Camera giám sát và hệ thống quản trị học vụ

- **Monitoring**: giống như bác bảo vệ nhìn vào màn hình **camera giám sát** và thấy sinh viên đang ùn tắc trước thang máy nhà A. Bác chỉ biết: *hệ thống đang có vấn đề*.
- **Observability**: bác bảo vệ có hệ thống liên kết camera với lịch học và thẻ quẹt sinh viên. Tra cứu nhanh, bác phát hiện ra: *có 2 lớp học bù đột xuất được xếp trùng giờ ở cùng tầng 9*. Bác biết chính xác: *tại sao có vấn đề và nguyên nhân gốc rễ là gì*.

11.1.4. Mô hình trưởng thành observability (observability maturity model)

Không phải hệ thống nào cũng cần đầy đủ bộ công cụ observability ngay từ đầu. Mô hình trưởng thành giúp xác định *đang ở đâu* và *cần đến đâu*.

Bảng 11.3: *Mô hình trưởng thành observability*

Cấp độ	Mô tả	Đặc điểm	Phù hợp với
1 — Reactive	Gỡ lỗi khi người dùng báo lỗi	Console log, không có cảnh báo tự động, điều tra thủ công	Dự án cá nhân, bản mẫu thử nghiệm
2 — Basic monitoring	Dashboard cơ bản, cảnh báo thô	Health check, CPU/memory metric, log tập trung	Nhóm nhỏ, hệ thống ít dịch vụ
3 — Proactive	Phát hiện vấn đề trước khi người dùng gặp phải	Log có cấu trúc, request/correlation ID, metric tùy biến, cảnh báo	Nhóm quy mô vừa, nhiều dịch vụ
4 — Observability	Hỏi câu hỏi mới không cần triển khai code mới	Distributed tracing, SLI/SLO, dữ liệu cardinality cao	Nhóm quy mô lớn, hệ thống phức tạp
5 — Predictive	Dự đoán vấn đề trước khi xảy ra	Phát hiện bất thường dựa trên ML, hoạch định năng lực	Doanh nghiệp lớn, hệ thống trọng yếu

Nguồn: Tổng hợp từ Richardson [2a, Ch.11], Newman [4a, Ch.8], và Charity Majors, Liz Fong-Jones, George Miranda, Observability Engineering *(O'Reilly, 2022). Các cấp độ trưởng thành phản ánh kiến thức cộng đồng observability — không theo một framework độc quyền nào.*

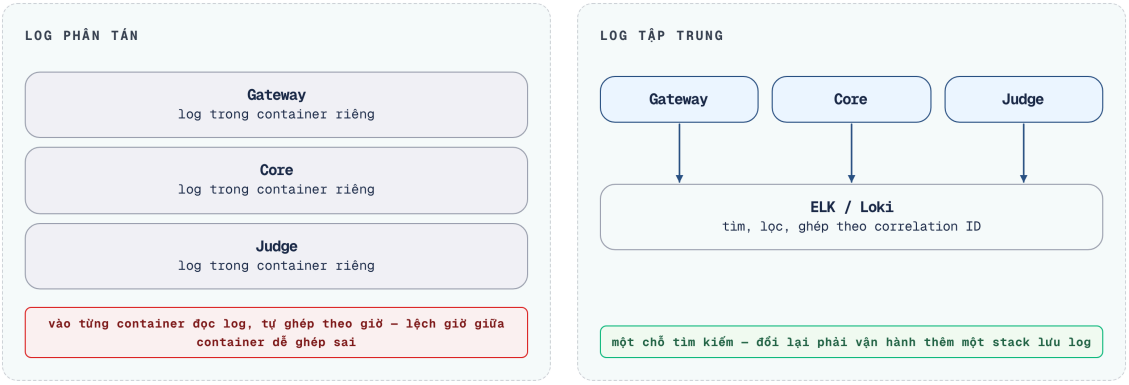
Với LMS — hệ thống giáo dục quy mô trung bình — mục tiêu hợp lý là **Cấp 3** (Proactive), với khả năng mở rộng lên Cấp 4 khi cần.

11.2. Logging — từ console đến gom log tập trung

Trong kiến trúc microservices, một request từ người dùng đi qua nhiều dịch vụ. Nếu log nằm rải rác trên nhiều máy chủ khác nhau, việc truy vết lỗi trở nên tốn kém và dễ bỏ sót.

11.2.1. Vấn đề: log phân tán, không có ngữ cảnh

Khi hệ thống có 7+ service chạy đồng thời, mỗi service ghi log vào container riêng. Để truy vết một yêu cầu thất bại, lập trình viên phải kết nối (SSH) vào từng container, đọc nhật ký và chắp vá thời gian — rất khó với các request đồng thời và lệch đồng hồ (clock skew) giữa các container.



Hình 11.3: *Log phân tán (gỡ lỗi thủ công) vs Log tập trung — tìm kiếm tại một nơi*

11.2.2. Log có cấu trúc — từ văn bản thô sang JSON

Bước đầu tiên: chuyển từ **log dạng văn bản** (text log) sang **log có cấu trúc** (JSON). Log dạng văn bản dễ đọc bằng mắt thường nhưng *rất khó phân tích tự động*.

Listing 11.1: *Ví dụ log có cấu trúc (JSON) theo chuẩn trường dữ liệu thống nhất*

```
// SAI: Text log — khó parse, khó filter, khó aggregate
2026-01-15 10:03:02 ERROR [kbm-core] - SQL execution failed for submission 8a3f
    user 12ab, Timeout after 30s

// ĐÚNG: Structured log (JSON) — dễ query, filter, aggregate
{"timestamp":"2026-01-15T10:03:02Z","severity":"ERROR","service.name":"kbm-core",
 "message":"SQL execution failed","submissionId":"8a3f","requestId":"req-8a3f12",
 "duration_ms":30000,"errorCode":"JUDGE_TIMEOUT",
 "error.type":"QueryTimeoutException",
 "traceId":"4bf92f3577b34da6a3ce929d0e0e4736","spanId":"00f067aa0ba902b7"}
```

Với log có cấu trúc, hệ thống có thể truy vấn: *"Tìm tất cả các lỗi trong 24h qua có* `error.type = QueryTimeoutException`*"* hoặc *"Thời gian xử lý trung bình của các lượt nộp bài theo giờ"* — điều không thể làm tin cậy với log dạng văn bản tự do.

Listing 11.2: *Khai báo Logback xuất JSON chứa ngữ cảnh truy vết và định danh (*`logback-spring.xml`*)*

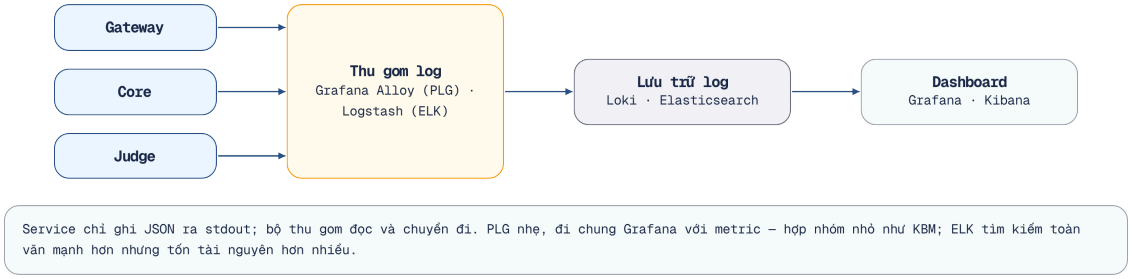
```
<configuration>
    <appender name="CONSOLE" class="ch.qos.logback.core.ConsoleAppender">
        <encoder class="net.logstash.logback.encoder.LogstashEncoder">
            <!— Tự động trích xuất ngữ cảnh từ MDC
                 (Micrometer Tracing + Request/Workflow Filter) —>
            <includeMdcKeyName>traceId</includeMdcKeyName>
            <includeMdcKeyName>spanId</includeMdcKeyName>
            <includeMdcKeyName>requestId</includeMdcKeyName>
            <includeMdcKeyName>correlationId</includeMdcKeyName>
        </encoder>
    </appender>
```

```
    <root level="INFO"><appender-ref ref="CONSOLE" /></root>
</configuration>
```

Từ Spring Boot 3.4 có thể bật log JSON bằng thuộc tính `logging.structured.format.console`, không cần thêm encoder.

11.2.3. Gom log tập trung (centralized log aggregation)

Khi log đã có cấu trúc, bước tiếp theo là **gom log tập trung** — thu gom log từ tất cả service về một nơi. Newman trong [4a, Ch.8] nhấn mạnh: *log tập trung là yêu cầu bắt buộc* cho microservices, không phải tùy chọn.



Hình 11.4: *Pipeline gom log tập trung — từ service đến nơi lưu trữ*

Để triển khai một pipeline thu thập log tập trung hiệu quả, các tổ chức thường đánh đổi giữa khả năng tìm kiếm toàn văn (full-text search) chuyên sâu và mức tiêu thụ tài nguyên phần cứng. Trong hệ sinh thái cloud-native, nổi bật nhất là hai stack tiêu chuẩn dưới đây:

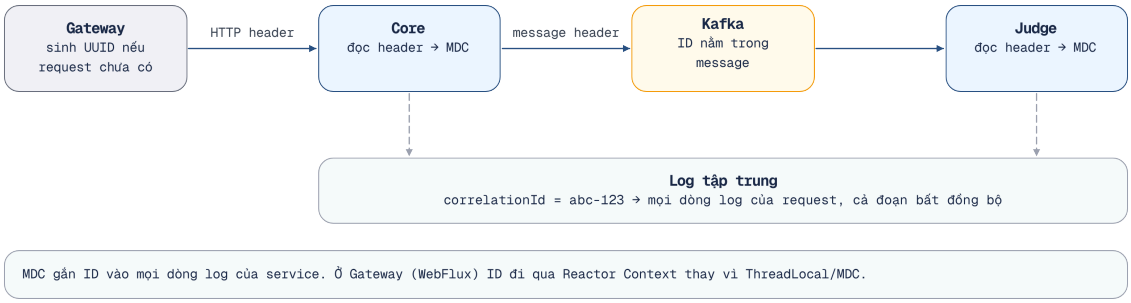
Bảng 11.4: *So sánh hai stack ELK và PLG*

Stack	Thành phần	Ưu điểm	Nhược điểm	Phù hợp
ELK	Elasticsearch + Logstash + Kibana	Trưởng thành, tìm kiếm toàn văn mạnh	Tốn tài nguyên (tối thiểu ~4 GB RAM)	Nhóm quy mô lớn, nhiều log
PLG	Promtail (đã hết vòng đời 03/2026, thay bằng Grafana Alloy) + Loki + Grafana	Nhẹ, tích hợp metric	Truy vấn kém linh hoạt hơn ELK	Nhóm nhỏ và vừa, KBM

11.2.4. Định danh xuyên suốt: Request ID và Correlation ID

Dù đã có hệ thống gom log tập trung, việc phân tích nguyên nhân gốc rễ (root cause analysis) vẫn rất khó nếu các dòng log của cùng một thao tác bị rời rạc. Newman trong [4a, Ch.8] mô tả **Correlation ID** là kỹ thuật nền tảng cho việc gỡ lỗi microservices: gắn định danh xuyên suốt từ điểm vào hệ thống qua các lời gọi HTTP và hàng đợi thông điệp. Như đã chuẩn hóa ở Chương 8 (§8.4), chúng ta phân biệt rõ vai trò của ba định danh để tránh dùng lẫn:

- `X-Request-ID` (trường log `requestId`): định danh từng HTTP request tại biên (Gateway kiểm tra định dạng hoặc sinh mới, chuyển tiếp xuống downstream và trả lại trong response header cho client).
- `correlationId`: định danh luồng nghiệp vụ đa bước (ví dụ một lượt nộp bài từ lúc HTTP `POST /submissions` tới các sự kiện Kafka và kết quả chấm), có thể bao phủ nhiều request và nhiều trace.
- `traceparent` / `traceId`: ngữ cảnh truy vết phân tán theo chuẩn W3C Trace Context (sẽ phân tích ở §11.3).



Hình 11.5: *Truyền định danh xuyên suốt request HTTP và luồng sự kiện Kafka*

Phiếu luân chuyển hồ sơ sinh viên

Hãy tưởng tượng một sinh viên nộp hồ sơ xin xét tốt nghiệp. Hồ sơ này được dán một mã vạch (`correlationId`) duy nhất cho toàn bộ quy trình, còn mỗi lần nộp bổ sung giấy tờ tại quầy một cửa có thêm số phiếu tiếp nhận (`X-Request-ID`). Bất kể hồ sơ đi qua Phòng Đào tạo (Core Service), Phòng Tài chính (Finance Service), hay Thư viện (Library Service), mọi phòng ban đều quét mã này và ghi vào sổ nhật ký. Nếu hồ sơ bị kẹt, Ban Giám hiệu chỉ cần tra mã là biết chính xác nó đang nằm ở khâu nào.

Cơ chế phối hợp đầu cuối:

1. **Gateway** kiểm tra định dạng `X-Request-ID` từ client (nếu thiếu hoặc sai định dạng thì sinh mới để chống log injection), gắn vào request chuyển tiếp và trả lại trong HTTP response header (Chương 8).
2. **Downstream service** đọc `X-Request-ID` ghi vào **MDC** (`requestId`), đồng thời gắn `correlationId` của luồng nghiệp vụ (chẳng hạn `submissionId`) vào MDC — logging framework tự động đính kèm vào mọi dòng log (Listing 11.2).
3. **Kafka producer/consumer** đặt `correlationId` (cùng `eventId`, `causationId` như đã học ở Chương 5) trong envelope của thông điệp để nối liền chặng bất đồng bộ.
4. **Khi gỡ lỗi**: tra `requestId` để xem một lượt gọi HTTP cụ thể, hoặc tra `correlationId` để xem toàn bộ vòng đời xử lý bài nộp qua nhiều dịch vụ.

Trên ứng dụng reactive như Spring Cloud Gateway (WebFlux), không ghi trực tiếp vào `ThreadLocal`/MDC trong thân filter vì một request có thể nhảy qua nhiều event-loop thread. Thay vào đó, filter ghi định danh đã làm sạch vào Reactor `Context` (Listing 11.3) và bật `Hooks.enableAutomaticContextPropagation()` (thư viện `micrometer-context-propagation`). Khóa `requestId` chỉ được đồng bộ sang MDC mỗi khi ghi log nếu ứng dụng đăng ký một `ThreadLocalAccessor` cho khóa MDC đó trong `ContextRegistry` (một lần, lúc khởi động):

Listing 11.3: *Chuẩn hóa `X-Request-ID` và truyền qua Reactor `Context` trong WebFlux*

```
@Component
public class RequestIdFilter implements GlobalFilter, Ordered {
    private static final String HEADER = "X-Request-ID";
    private static final Pattern SAFE_ID =
            Pattern.compile("^[A-Za-z0-9._-]{1,64}$");

    @Override
```

```
    public Mono<Void> filter(
            ServerWebExchange exchange, GatewayFilterChain chain) {
        String incoming = exchange.getRequest().getHeaders().getFirst(HEADER);
        String requestId = (incoming != null && SAFE_ID.matcher(incoming).matches())
            ? incoming
            : "req-" + UUID.randomUUID();

        // 1. Gắn vào request downstream và trả về response cho client
        ServerHttpRequest mutatedRequest = exchange.getRequest().mutate()
            .headers(h -> h.set(HEADER, requestId))
            .build();
        exchange.getResponse().getHeaders().set(HEADER, requestId);

        // 2. Lưu vào Reactor Context
        // (cầu nối sang MDC qua micrometer-context-propagation)
        return chain.filter(exchange.mutate().request(mutatedRequest).build())
            .contextWrite(Context.of("requestId", requestId));
    }

    @Override
    public int getOrder() { return Ordered.HIGHEST_PRECEDENCE; }
}
```

Mỗi request đều có một định danh xuyên suốt

> "When the first call is made, you generate a GUID for the call. This is then passed along to all subsequent calls [...] and can be put into your logs in a structured way."
>
> — *Sam Newman, Building Microservices [4a, Ch.8]*

11.3. Chi tiết về Distributed Tracing

Khi sinh viên phàn nàn hệ thống nộp bài quá chậm, thật khó để biết độ trễ nằm ở đâu nếu không có cơ chế theo dõi dấu vết. Distributed Tracing giải quyết bài toán này bằng cách vẽ lại chính xác đường đi của mỗi yêu cầu.

11.3.1. Vấn đề: request chậm — chậm ở đâu?

Các định danh tương quan (`requestId`, `correlationId`) giúp nối các dòng log lại — nhưng không cho biết **mỗi bước mất bao lâu**. Khi sinh viên nộp bài và thời gian phản hồi vọt lên 8 giây (so với khoảng 1 giây thường thấy ở truy vấn đơn giản), lập trình viên biết yêu cầu đã đi qua Gateway → Core → Judge → MySQL, nhưng không thể xác định thời gian dư thừa phát sinh ở bước nào nếu không dò timestamp trong log một cách thủ công.

11.3.2. Distributed Tracing — tự động đo thời gian

Distributed tracing ghi lại toàn bộ vòng đời của một request qua tất cả service, bao gồm thời gian mỗi bước (**span**):

Hình 11.6: Biểu đồ Gantt của một trace — thời gian của từng span

Trên một lượt chấm truy vấn đơn giản (~810 ms ở Hình 11.6), **Execute SQL on sandbox** chiếm 570 ms — tương đương ~70% tổng thời gian. Khi một bài nộp bất ngờ vọt lên 8 giây, cùng biểu đồ phân rã span này chỉ thẳng khâu bị kéo giãn (chẳng hạn hàng đợi sandbox hay khóa bảng MySQL) thay vì buộc lập trình viên phỏng đoán hoặc dò thủ công từng timestamp trong log.

11.3.3. Các khái niệm: Trace, Span, Context Propagation

Quy trình nhập học

- **Trace**: Là tổng thời gian từ lúc tân sinh viên bước vào trường đến lúc cầm thẻ sinh viên ra về (ví dụ: 120 phút).
- **Span**: Là thời gian sinh viên dừng lại ở từng trạm. Trạm "Đóng học phí" là một **span cha** (parent span, 30 phút).
- **Span con**: Bên trong trạm đóng học phí có 2 khâu nhỏ: "Xếp hàng chờ" (25 phút) và "Quẹt thẻ ngân hàng" (5 phút). Nhờ chia nhỏ thành span con (child span), chúng ta biết ngay "cổ chai" (bottleneck) không phải do hệ thống ngân hàng chậm, mà do thời gian chờ đợi xếp hàng quá lâu.

Ẩn dụ trên giúp hình dung trực quan quy trình tracing. Bảng dưới đây hệ thống hóa bốn khái niệm cốt lõi — Trace, Span, quan hệ cha-con, và Context propagation — cùng ví dụ cụ thể trong luồng nộp bài của KBM.

Bảng 11.5: Các khái niệm cơ bản trong Distributed Tracing

Khái niệm	Mô tả	Ví dụ
Trace	Toàn bộ vòng đời xử lý của một request	Nộp bài từ trình duyệt → trả điểm về
Span	Một bước trong trace (một đơn vị công việc)	"Core Service: save submission" (40 ms)
Cha-con (parent-child)	Span có thể chứa các span con	"Judge Service" span chứa "Execute SQL" và "Compare"
Context propagation	Truyền trace context qua ranh giới service	HTTP header `traceparent`, Kafka message header

11.3.4. Chiến lược lấy mẫu (Sampling)

Trên production, việc truy vết **100% request** sẽ tiêu tốn rất nhiều dung lượng lưu trữ và năng lực xử lý. Do đó, cần có chiến lược lấy mẫu (sampling strategy):

Bảng 11.6: Các chiến lược lấy mẫu cho tracing

Chiến lược	Mô tả	Khi nào dùng
Head-based	Quyết định lấy mẫu tại điểm vào (Gateway)	Đơn giản, dễ triển khai
Tail-based	Quyết định sau khi trace hoàn thành (giữ trace lỗi, bỏ trace thành công)	Ưu tiên giữ trace lỗi đã thu nhận, giảm dung lượng lưu trữ
Probabilistic (theo tỷ lệ)	Lấy mẫu N% request	Production lưu lượng cao (1–10%)
Adaptive	Tự động điều chỉnh tỷ lệ lấy mẫu theo lưu lượng	Doanh nghiệp lớn, tải biến động

Với KBM, cả trong chế độ thi tải vẫn ở mức hàng trăm lượt mỗi phút, nên có thể giữ 100% để không mất trace của bài nộp bị chậm; khi chi phí lưu trữ trở thành vấn đề thì chuyển sang tail-based — nhưng đây là đặc thù hệ thống nhỏ; với production có lưu lượng lớn, cần chọn chiến lược sampling phù hợp (probabilistic, tail-based) dựa trên tải thực tế và chi phí lưu trữ.

11.3.5. Tracing trong Spring Boot 3.x

Spring Boot 3.x tích hợp sẵn **Micrometer Tracing** — lớp trừu tượng (abstraction layer) cho distributed tracing. **OpenTelemetry (OTel)** hiện nay đã trở thành chuẩn mực công nghiệp thống nhất cho cả 3 trụ cột (Logs, Metrics, Traces), định nghĩa chuẩn truyền tải OTLP.

Để bật tracing và xuất dữ liệu ra OTel Collector, chúng ta chỉ cần khai báo các thư viện phụ thuộc (dependency) và cấu hình trong `application.yml` mà không phải sửa code logic:

Listing 11.4: Khai báo thư viện phụ thuộc cho tracing (`pom.xml`)

```
<!-- Spring Boot 3 Actuator -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
<!-- Micrometer OTel Bridge -->
<dependency>
    <groupId>io.micrometer</groupId>
    <artifactId>micrometer-tracing-bridge-otel</artifactId>
</dependency>
<!-- OTLP Exporter -->
<dependency>
    <groupId>io.opentelemetry</groupId>
    <artifactId>opentelemetry-exporter-otlp</artifactId>
</dependency>
```

Tuy nhiên, Distributed Tracing chỉ phát huy tối đa hiệu quả khi được kết hợp với Logs. Tracing giúp định vị "nút thắt" (bottleneck), còn Logs giải thích nguyên nhân gốc rễ (root cause).

Tracing Complements Logging

"Tracing xác định đường đi và thời gian xử lý của request, trong khi Logging giải thích nguyên nhân gây ra vấn đề. Sử dụng trace để nhận diện điểm nghẽn (bottleneck) và dùng log để hiểu nguyên nhân gốc rễ (root cause). Hai công cụ này bổ trợ cho nhau và không thể thay thế lẫn nhau."

— Tổng hợp từ Richardson [2a, Ch.11] và Newman [4a, Ch.8]

Sau phần thư viện phụ thuộc, việc còn lại là khai báo nơi xuất dữ liệu (OTLP exporter) và tỉ lệ lấy mẫu trong `application.yml` (Listing 11.5):

Listing 11.5: Cấu hình OTLP Exporter và Sampling (`application.yml`)

```yaml
management:
  endpoints:
    web:
      exposure:
        include: health,prometheus  # Expose metrics
  tracing:
    sampling:
      probability: 1.0  # Sample 100% (cho môi trường dev/staging)
  otlp:
    tracing:
      endpoint: http://otel-collector:4318/v1/traces
```

Với cấu hình này, Spring Boot **tự động** truyền trace context qua HTTP header theo chuẩn **W3C Trace Context** (header `traceparent` dạng `00-<traceId 32 hex>-<parentId 16 hex>-<trace-flags 2 hex>`, thay cho chuẩn B3 cũ). Để truyền qua Kafka, cần bật observation ở *cả hai phía*: `spring.kafka.template.observation-enabled=true` cho producer (`KafkaTemplate`) và `spring.kafka.listener.observation-enabled=true` cho consumer — thiếu phía producer thì trace sẽ đứt đoạn tại broker. Với `RestTemplate`, `RestClient` và `WebClient`, cần dựng từ builder do Spring Boot cung cấp thì context mới được truyền tự động; khi đó không cần viết code tracing thủ công cho phần lớn bài toán.

11.3.6. Kiến trúc triển khai Observability Pipeline

Phần nâng cao (có thể lướt qua)

Mục này dành cho hệ thống lớn. KBM ở quy mô hiện tại chưa cần Collector phân tầng.

Khi hệ thống lớn lên, ứng dụng không nên gửi trực tiếp dữ liệu đến Jaeger/Prometheus. Có 3 mô hình triển khai:

1. **Direct SDK**: Ứng dụng gửi trực tiếp dữ liệu đến hệ thống lưu trữ (tight coupling, dễ gây nghẽn mạng).
2. **Local Agent**: Ứng dụng gửi dữ liệu nội bộ tới agent chạy dưới dạng DaemonSet; agent tự quản lý bộ đệm và gom lô (batching).
3. **Centralized Collector**: Tất cả Agent gửi về một cụm Gateway Collector trung gian để che dữ liệu cá nhân (PII), lọc và định tuyến trước khi lưu trữ.

Nút thắt OTel Collector

Trong mô hình Centralized Collector, cụm Collector là điểm xử lý tập trung dễ trở thành nút thắt:

- **OOM (Out of Memory)**: Rất dễ xảy ra nếu dùng **Tail-based sampling**, vì Collector phải giữ tạm toàn bộ span của một trace trên RAM để đợi quyết định lưu trữ hay loại bỏ ở bước cuối cùng.
- **CPU Bound & Network Saturation**: Do phải phân tích (parse), lọc và biến đổi lượng dữ liệu log/trace lớn.

Biện pháp giảm thiểu: Triển khai kiến trúc phân tầng (Agent ở biên gom lô trước khi đẩy về cụm Gateway Collector), cấu hình bộ xử lý `memory_limiter` để chặn tràn RAM, dùng `loadbalancing`

`exporter` định tuyến mọi span cùng `traceId` về đúng một instance Collector khi lấy mẫu tail-based, và đặt Apache Kafka làm bộ đệm hấp thụ các đợt lưu lượng tăng đột biến. Ở tầng lưu trữ phía sau, áp thêm **chính sách vòng đời dữ liệu** (ví dụ chỉ giữ log nóng 7–14 ngày trên Loki rồi chuyển sang lưu trữ lạnh trên S3) để kiểm soát chi phí.

11.4. Metrics, SLI/SLO và chiến lược cảnh báo

Hệ thống cần các chỉ tiêu chất lượng rõ ràng để cảnh báo đội ngũ kỹ thuật kịp thời — trước khi sự cố kịp làm gián đoạn việc học của sinh viên.

11.4.1. Metrics — đo lường hành vi theo thời gian

Logs ghi lại *sự kiện đã xảy ra*. Metrics đo lường *xu hướng theo thời gian* — quan trọng cho việc **phát hiện sớm** (early detection) vấn đề trước khi người dùng báo lỗi.

Từ các observability pattern ở [2a, Ch.11], giáo trình tổng hợp bốn nhóm cần triển khai:

Bảng 11.7: *Bốn observability pattern cần triển khai*

Pattern	Mô tả	Ví dụ KBM
Health Check API	Endpoint `/health` cho hệ thống điều phối (orchestrator) biết dịch vụ đang hoạt động hay đã ngừng (up/down)	Eureka công bố trạng thái từ health check (khi bật tích hợp healthcheck, §11.6); LoadBalancer dựa vào đó để chọn instance
Application metrics	Metric ở mức nghiệp vụ	Tần suất nộp bài, thời gian chấm, tỷ lệ lỗi
Infrastructure metrics	Metric ở mức hệ thống	CPU, memory, DB connections, Kafka consumer lag
Audit logging	Ghi lại hành vi người dùng để đáp ứng yêu cầu tuân thủ (compliance)	Đăng nhập, thay đổi vai trò, truy cập dữ liệu

11.4.2. Bốn loại metric cơ bản của Prometheus

Trước khi áp dụng RED/USE, lập trình viên cần hiểu 4 kiểu dữ liệu đo lường:

1. **Counter**: Bộ đếm chỉ tăng lên (ví dụ: tổng số request). Tốt để tính Rate.
2. **Gauge**: Giá trị có thể tăng/giảm (ví dụ: mức dùng CPU, số kết nối đang mở). Tốt để tính Saturation.
3. **Histogram**: Nhóm dữ liệu vào các "bucket" (ví dụ: số request xong trong < 100 ms, < 500 ms). Nên dùng cho phân vị độ trễ (P95, P99) vì phía server tính lại và *gộp* được số liệu qua nhiều instance.
4. **Summary**: Ước lượng sẵn quantile ở phía client, theo sai số cấu hình, nhưng không gộp được giữa các replica, nên ít phù hợp với hệ phân tán nhiều bản sao.

Lưu ý rằng với độ trễ, các chỉ số vận hành luôn dùng **phân vị** (như P95, P99 tính từ Histogram) thay vì trung bình cộng (`average`). Trung bình cộng dễ che khuất đuôi trễ: nếu 99 sinh viên nhận kết quả trong 100 ms còn 1 sinh viên phải chờ 20 giây, giá trị trung bình chỉ khoảng ~300 ms — trông hoàn toàn bình thường dù 1% người dùng đang gặp trải nghiệm rất tệ.

⚠ Cardinality Explosion

Một sai lầm kinh điển của các nhóm mới làm quen với microservices là đưa `userId` hoặc `submissionId` vào làm nhãn (`label`) cho metric của Prometheus. Metric được thiết kế cho **Low Cardinality** (số lượng giá trị nhãn hữu hạn, ví dụ: `status=200`, `method=GET`). Nếu nhóm phát triển

gắn `userId` làm nhãn, cơ sở dữ liệu chuỗi thời gian (TSDB) sẽ sinh ra hàng triệu time-series khác nhau, ngốn hết bộ nhớ và làm sập hệ thống giám sát. Để tra cứu theo `userId`, hãy dùng Logs hoặc Traces.

Exemplars: cầu nối Metrics & Traces

Công nghệ hiện đại cho phép nhúng trực tiếp một `traceId` đại diện vào trong các bucket của Prometheus Histogram (gọi là **Exemplars**). Khi quan sát thấy một đỉnh trễ bất thường trên Grafana, kỹ sư có thể nhấp trực tiếp vào điểm dữ liệu đó để chuyển hướng đến Jaeger và phân tích chính xác trace gây ra sự chậm trễ.

Để quy chuẩn hóa việc đo lường trạng thái hệ thống, ngành công nghiệp thường sử dụng hai khuôn mẫu tiêu chuẩn: RED (cho dịch vụ) và USE (cho tài nguyên).

11.4.3. Hai khung đo lường: RED và USE

- **RED (dùng cho service phục vụ request)**: Bao gồm **R**ate, **E**rrors, và **D**uration. Ví dụ: tần suất nộp bài là 50 req/min, tỷ lệ lỗi là 2%, độ trễ P99 là 3.2 s.
- **USE (dùng cho tài nguyên hạ tầng)**: Bao gồm **U**tilization, **S**aturation, và **E**rrors. Ví dụ: CPU là 45%, DB connection pool là 8/10 active, Disk I/O errors là 0.

Kết hợp RED (cho service) + USE (cho tài nguyên) để giám sát cả hai khía cạnh của hệ thống. Metrics phát hiện "độ trễ nộp bài tăng" (RED), điều tra cho thấy "DB connection pool bị bão hòa" (USE) → nguyên nhân gốc rễ: rò rỉ kết nối (connection leak).

11.4.4. SLI/SLO/SLA — từ metric đến cam kết chất lượng

Metrics chỉ hữu ích khi biết **ngưỡng nào là chấp nhận được**. Khung SLI/SLO/SLA cung cấp ngôn ngữ chung:

Thư viện trường học

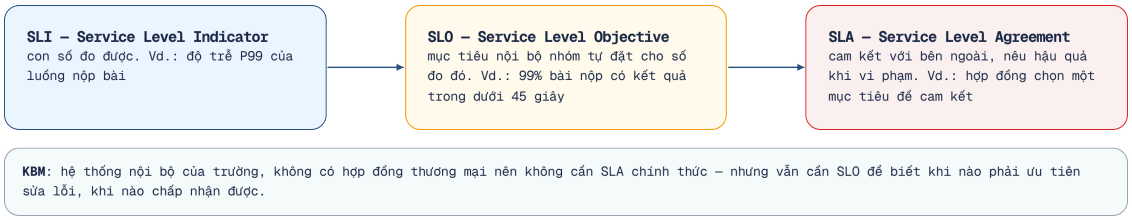
- **SLI (Chỉ số thực tế)**: Thời gian thực tế thủ thư dùng để xếp một cuốn sách trả lại lên kệ (ví dụ: 1.5 giờ).
- **SLO (Mục tiêu nội bộ)**: KPI mà giám đốc Thư viện đặt ra cho nhân viên: "99% sách trả lại phải được xếp lên kệ trong vòng 2 giờ".
- **SLA (Cam kết đền bù)**: Cam kết trong Sổ tay sinh viên: "Nếu sách người mượn trả chưa được hệ thống cập nhật trong ngày, nhà trường cam kết sẽ miễn trừ mọi khoản phạt trễ hạn tự động phát sinh".

Để đảm bảo chất lượng vận hành, hệ sinh thái đo lường được chia thành ba khái niệm chính:

- **SLI (Service Level Indicator)**: Là metric cụ thể được hệ thống đo lường trực tiếp để đánh giá chất lượng dịch vụ trong thực tế. Trong KBM, ví dụ tiêu biểu là đo "Độ trễ P99 của luồng nộp bài".
- **SLO (Service Level Objective)**: Là mục tiêu nội bộ mà đội ngũ kỹ thuật tự đặt ra đối với một SLI nhất định. Ví dụ: "99% bài nộp được trả kết quả chấm trong vòng 45 giây" (kịch bản chất lượng §1.5).

- **SLA (Service Level Agreement)**: Là những cam kết chính thức với khách hàng, có đi kèm với các hình phạt tài chính hoặc hậu quả pháp lý rõ ràng nếu hệ thống bị vi phạm. Ví dụ: "Hệ thống cam kết tính sẵn sàng (availability) đạt mức 99.5% tổng thời gian trong một học kỳ".

Hình dưới minh họa quan hệ giữa ba khái niệm: SLI là số đo để đối chiếu với mục tiêu SLO; SLA, nếu có, chọn các mục tiêu cam kết và nêu hậu quả khi vi phạm. Phần chênh lệch cho phép `100% - SLO` được gọi là **ngân sách lỗi** (error budget): chừng nào ngân sách lỗi trong kỳ đo còn dương, nhóm có thể ưu tiên phát hành tính năng mới; khi ngân sách lỗi cạn dần hoặc âm, mọi thay đổi rủi ro phải nhường chỗ cho việc gia cố độ tin cậy.



Hình 11.7: *Mối quan hệ giữa SLI, SLO và SLA*

Với KBM — hệ thống giáo dục nội bộ phục vụ sinh viên của trường — nhóm phát triển không cần SLA chính thức vì không có hợp đồng thương mại. Tuy nhiên, SLO nội bộ vẫn rất quan trọng: chúng đặt ngưỡng để nhóm phát triển biết khi nào cần ưu tiên khắc phục lỗi, khi nào chấp nhận được. Bảng dưới đề xuất năm SLO phù hợp với bối cảnh KBM:

Bảng 11.8: *SLO đề xuất cho KBM*

SLI	SLO	Lý giải
Thời gian tiếp nhận bài nộp P99	< 500 ms	Sinh viên cần biết ngay bài đã vào hàng đợi
Thời gian trả kết quả chấm P99	< 45 giây	Sinh viên chờ quá lâu dễ bấm nộp lặp hoặc bỏ dở phiên làm bài
Tính sẵn sàng trong ca thi	99.9% theo khung giờ ca thi (~5.4 giây gián đoạn / ca 90 phút)	Ca thi có thời gian giới hạn nghiêm ngặt; ngoài kỳ thi có thể áp mục tiêu 99.5% theo học kỳ
Tỷ lệ lượt chấm hoàn tất không lỗi hệ thống	≥ 99.9%	Tách bạch với tính đúng đắn tất định của bộ chấm (bất biến nghiệp vụ kiểm chứng bằng kiểm thử tự động); SLI vận hành đo tỷ lệ bài nộp không bị lỗi hạ tầng/sandbox
Tỷ lệ lỗi API (5xx)	< 1%	Tần suất lỗi máy chủ cao làm giảm trực tiếp độ tin cậy của hệ thống

11.4.5. Chiến lược cảnh báo

Tuy nhiên, nếu đội ngũ kỹ sư cấu hình cảnh báo quá nhạy cảm hoặc sai cách, họ sẽ nhanh chóng rơi vào trạng thái "mệt mỏi vì cảnh báo" (**alert fatigue**). Hiện tượng này xảy ra khi có quá nhiều cảnh báo giả (false positive) liên tục phát sinh, khiến các kỹ sư on-call dần sinh ra tâm lý chủ quan và bỏ qua ngay cả những sự cố thực sự nghiêm trọng. Để hạn chế tình trạng này, một cơ chế quan trọng cần áp dụng là **gom nhóm cảnh báo** — thay vì gửi 100 thông báo riêng lẻ khi một dịch vụ ngừng hoạt động (down) kéo theo hàng loạt lỗi liên quan, hệ thống sẽ gom chúng lại thành một thông báo duy nhất theo nguyên nhân gốc rễ.

Bảng 11.9: Chiến lược cảnh báo theo mức độ nghiêm trọng

Mức độ	Khi nào	Kênh	Hành động	Ví dụ
Nghiêm trọng (critical)	SLO bị vi phạm, ảnh hưởng đến người dùng ngay lập tức	PagerDuty, gọi điện	Kỹ sư trực hệ thống (on-call) phải xử lý ngay	Dịch vụ Judge ngừng hoạt động, nộp bài thất bại
Cảnh báo (warning)	Gần vi phạm SLO, cần chú ý	Slack, email	Xem xét trong giờ làm việc	Độ trễ P99 đạt 80% ngưỡng
Thông tin (info)	Bất thường nhưng không ảnh hưởng người dùng	Chỉ hiển thị trên dashboard	Xem xét sau hoặc phân tích định kỳ	Kafka consumer lag tăng nhẹ

Ba tầng cảnh báo ở trên giúp tránh tình trạng "mọi sự cố đều được coi là critical". Khi áp dụng thực tiễn, việc xây dựng tiêu chí cảnh báo cần gắn kết trực tiếp với trải nghiệm người dùng (thông qua SLO), thay vì dựa trên chỉ số kỹ thuật thuần túy. Chẳng hạn, nếu CPU tăng cao nhưng độ trễ vẫn trong giới hạn và người dùng không bị ảnh hưởng, thì không cần đánh thức kỹ sư on-call vào lúc 3 giờ sáng.

Cảnh báo theo triệu chứng, không theo nguyên nhân

"Cảnh báo khi SLO có nguy cơ bị vi phạm, không phải khi một metric thay đổi. CPU 90% nhưng độ trễ bình thường? Không cần cảnh báo. CPU 50% nhưng độ trễ vượt SLO? Cảnh báo ngay."

— Nguyên tắc SRE, dựa trên Google SRE book

11.5. Chiến lược xử lý lỗi — nhất quán giữa các service

Một hệ thống nhiều thành phần thường gặp tình trạng mỗi dịch vụ trả về một định dạng báo lỗi khác nhau. Để tránh gây nhầm lẫn cho ứng dụng frontend và sinh viên, hệ thống cần chuẩn hóa cách xử lý lỗi.

11.5.1. Vấn đề: mỗi service trả về định dạng lỗi khác nhau

Khi hệ thống bao gồm nhiều dịch vụ do các nhóm khác nhau phát triển, định dạng phản hồi lỗi (error response) rất dễ mất tính nhất quán. Dịch vụ A trả về lỗi trong trường `description`, Dịch vụ B lại sử dụng trường `message`, Gateway trả định dạng hoàn toàn khác. Ứng dụng khách (client/frontend) phải xử lý nhiều định dạng — mã nguồn phình phức tạp và trải nghiệm người dùng (UX) kém đi. Do đó cần thiết lập nguyên tắc chuẩn hóa.

11.5.2. Nguyên tắc thiết kế xử lý lỗi

Richardson trong [2a, Ch.11] và Newman trong [4a, Ch.8] đề xuất các nguyên tắc xử lý lỗi trong microservices:

1. Bộ xử lý ngoại lệ tập trung (centralized exception handler) — Mỗi service có một handler tập trung (trong Spring Boot: `@ControllerAdvice`) xử lý mọi ngoại lệ → chuyển thành định dạng chuẩn. Đặt trong thư viện dùng chung để mọi service cùng sử dụng.

2. Phân loại mã lỗi (error code taxonomy) — Phân loại lỗi theo namespace để client và hệ thống giám sát hiểu nhanh:

Bảng 11.10: Phân loại hệ thống mã lỗi (`errorCode` dạng chuỗi có namespace)

Namespace	Nhóm	Ví dụ `errorCode`
AUTH_*	Xác thực & phân quyền	AUTH_UNAUTHORIZED, AUTH_TOKEN_EXPIRED

Namespace	Nhóm	Ví dụ `errorCode`
NOT_FOUND_*	Không tìm thấy tài nguyên	NOT_FOUND_QUESTION, NOT_FOUND_CONTEST
BIZ_*	Vi phạm quy tắc nghiệp vụ	BIZ_CONTEST_NOT_STARTED, BIZ_SUBMIT_LIMIT
INFRA_*	Hạ tầng / phụ thuộc	JUDGE_UNAVAILABLE, KAFKA_SEND_FAILED (giữ tên phụ thuộc cho dễ đọc)
INTERNAL_*	Lỗi nội bộ	INTERNAL_UNEXPECTED

3. Problem Details (RFC 9457) — Mọi phản hồi lỗi dùng cùng cấu trúc Problem Details for HTTP APIs (RFC 9457), với các extension nội bộ KBM (`errorCode`, `traceId`):

__Listing 11.6:__ Error response theo RFC 9457 Problem Details

```
HTTP/1.1 503 Service Unavailable
Content-Type: application/problem+json
X-Request-ID: req-a1b2c3

{
  "type": "urn:kbm:problem:judge-unavailable",
  "title": "Judge service unavailable",
  "status": 503,
  "detail": "Judge service did not respond within timeout.",
  "instance": "urn:kbm:problem-instance:req-a1b2c3",
  "errorCode": "JUDGE_UNAVAILABLE",
  "traceId": "0af7651916cd43dd8448eb211c80319c"
}
```

4. Không để lộ chi tiết nội bộ — Stack trace, câu truy vấn SQL, đường dẫn nội bộ không bao giờ xuất hiện trong phản hồi lỗi. Ghi log chi tiết ở phía server (kèm `traceId` và `requestId` để đối chiếu), chỉ trả thông báo chung cho client.

11.5.3. Mối liên hệ giữa xử lý lỗi và observability

Trong thực tế, các mã lỗi tập trung không chỉ phục vụ client — chúng là **nguồn dữ liệu quan trọng cho metric và cảnh báo**:

- Đếm lỗi theo mã → phát hiện xu hướng: `JUDGE_UNAVAILABLE` tăng đột biến → cảnh báo
- Gom lỗi theo nhóm → dashboard: lỗi xác thực, lỗi nghiệp vụ, lỗi hạ tầng
- Tỷ lệ lỗi theo service → SLI cho độ tin cậy

Định dạng lỗi không nhất quán trong KBM

KBM sử dụng `@ControllerAdvice` với `ErrorCode` enum trong thư viện dùng chung — đúng pattern. Tuy nhiên, **Gateway** (Spring Cloud Gateway dùng WebFlux) xử lý lỗi JWT *bên ngoài* `@ControllerAdvice`, vì lỗi ở tầng filter nằm ngoài phạm vi controller advice. Kết quả: Gateway trả phản hồi lỗi dạng `{ "message": "..." }` trong khi các service trả `{ "description": "..." }` — không theo RFC 9457 và tên trường khác nhau. Ứng dụng giao diện (frontend) phải xử lý đồng thời cả hai định dạng này.

Lộ trình chuyển đổi: (1) Tạo `GatewayExceptionHandler` riêng cho WebFlux, chuẩn hóa phản hồi theo `application/problem+json` (RFC 9457), (2) Đảm bảo mọi phản hồi lỗi — kể cả từ Gateway — dùng cùng schema Problem Details với `errorCode` (chuỗi) và `traceId`, (3) Loại bỏ các trường không thuộc hợp đồng truyền dữ liệu chuẩn (`timestamp`, `description`, `message`).

11.6. Health check và Service Discovery

Service Discovery và API Gateway chỉ có thể tự động ngắt định tuyến khỏi dịch vụ gặp sự cố khi có căn cứ: đó là vai trò của health check.

11.6.1. Health check — nền tảng cho tự động hóa

Richardson trong [2a, Ch.11] mô tả **Health Check API** là observability pattern cơ bản nhất: mỗi dịch vụ cần cung cấp (expose) một endpoint `/health` để các công cụ điều phối nhận biết trạng thái hoạt động của nó.

Health check phân loại theo mục đích:

Bảng 11.11: *Ba loại health check*

Loại	Câu hỏi trả lời	Ý nghĩa cho orchestrator	Ví dụ KBM
Liveness	"Tiến trình còn chạy?"	Khởi động lại container nếu tiến trình không phản hồi	JVM còn hoạt động, không bế tắc (deadlock)
Readiness	"Sẵn sàng nhận traffic?"	Ngừng chuyển traffic nếu dịch vụ chưa sẵn sàng	Đã kết nối cơ sở dữ liệu và Kafka
Startup	"Đã khởi tạo xong?"	Chờ quá trình khởi động hoàn tất mới kiểm tra liveness	Đã chạy migration schema, đã làm nóng cache

Trong kiến trúc dùng Eureka như KBM, health check chỉ thực sự tham gia vào định tuyến khi nối liền chuỗi trách nhiệm: instance Core Service báo `DOWN` qua endpoint health → **Eureka** (Service Registry) cập nhật trạng thái trong danh bạ (khi bật tích hợp healthcheck) → Spring Cloud LoadBalancer ở Gateway loại instance đó khỏi danh sách lựa chọn và chuyển traffic sang instance khỏe mạnh. DevOps Lab bổ sung health/readiness ở tầng runtime để router chỉ chuyển traffic đến workspace sẵn sàng. Còn việc *khởi động lại* instance hỏng thuộc về tầng điều phối/giám sát tiến trình (chính sách restart của Docker, Kubernetes) — kết hợp cả hai tầng mới tạo thành cơ chế **tự phục hồi** (self-healing) hoàn chỉnh không cần can thiệp thủ công.

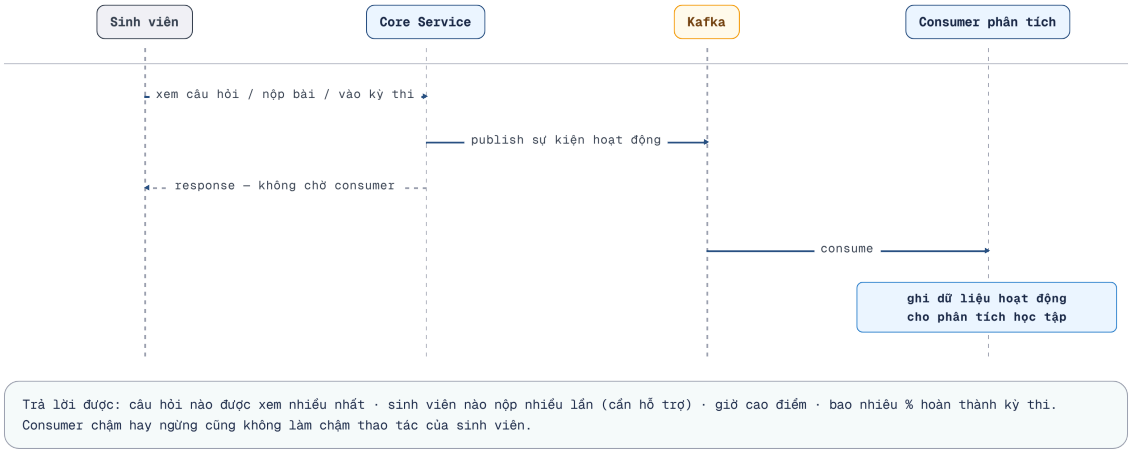
⚠ Eureka không tự suy ra trạng thái health của service

Một cấu hình dễ bỏ sót: mặc định Eureka client chỉ gửi *heartbeat* để báo tiến trình còn sống, nên một service vẫn được công bố `UP` dù endpoint health đã `DOWN` (chẳng hạn mất kết nối cơ sở dữ liệu) và tiếp tục nhận traffic lỗi. Để trạng thái từ Spring Boot Actuator được truyền lên registry, cấu hình client cần bật `eureka.client.healthcheck.enabled=true`. Khi bật cờ này, cũng cần cân nhắc kỹ mô hình kiểm tra sức khỏe: nếu endpoint health kiểm tra quá sâu vào các phụ thuộc bên ngoài, một sự cố nhất thời có thể khiến hàng loạt instance bị rút khỏi danh bạ cùng lúc (xem thêm phần phân biệt liveness và readiness ở cuối chương).

11.7. User Activity Tracking — observability ở mức nghiệp vụ

Ba trụ cột (log, metric, trace) giúp quan sát **hệ thống kỹ thuật**. Với một LMS như KBM, chúng ta còn cần quan sát **hành vi học tập**: sinh viên đang học gì, bài nào gây lúng túng nhất, khung giờ nào hoạt động sôi nổi nhất?

KBM triển khai **User Activity Tracking** qua đường ống Kafka: khi sinh viên xem câu hỏi, nộp bài hoặc tham gia kỳ thi, Core Service phát sự kiện theo dõi lên topic tương ứng. Consumer tiếp nhận và ghi nhận bất đồng bộ, không chặn (non-blocking) luồng xử lý chính của ứng dụng.



Hình 11.8: *User Activity Tracking pipeline qua Kafka*

Dữ liệu thu được từ pipeline này phục vụ nhiều mục đích, tóm tắt ở Bảng 11.12.

Bảng 11.12: *Ứng dụng của User Activity Tracking*

Dữ liệu	Câu hỏi trả lời được	Ứng dụng
Mẫu xem bài	"Câu hỏi nào được xem nhiều nhất?"	Đề xuất bài tập phù hợp
Mẫu nộp bài	"Sinh viên nào nộp bài nhiều lần?"	Phát hiện sinh viên cần hỗ trợ
Mẫu thời gian	"Giờ cao điểm là khi nào?"	Mở rộng hạ tầng phù hợp
Hành vi trong kỳ thi	"Bao nhiêu % sinh viên hoàn thành kỳ thi?"	Đánh giá hiệu quả giảng dạy

Đây là điểm giao giữa observability kỹ thuật và **phân tích đặc thù miền** (domain-specific analytics).

11.8. Chaos Engineering — kiểm tra khả năng phục hồi (resilience) chủ động

Observability cho phép **phát hiện** sự cố. Nhưng làm sao biết hệ thống sẽ phản ứng thế nào *trước khi* sự cố thật xảy ra? **Chaos Engineering** — phương pháp được Netflix phổ biến qua Chaos Monkey (2010) — trả lời câu hỏi này bằng cách **chủ động tiêm lỗi** (inject failures) vào môi trường staging/production và quan sát phản ứng. Đây là kỹ thuật ở giai đoạn trưởng thành cao (từ Cấp 4 trở lên); với quy mô như KBM, nhóm nên bắt đầu bằng các phép giả lập lỗi thủ công đơn giản trên staging.

Nguyên tắc (theo *Principles of Chaos Engineering* — principlesofchaos.org):

1. **Xác định trạng thái ổn định (steady state)** — hệ thống hoạt động bình thường trông như thế nào? (độ trễ < 200 ms, tỷ lệ lỗi < 0.1%, throughput > 100 req/s)
2. **Đặt giả thuyết** — "Nếu Judge Service ngừng hoạt động (down), bài nộp vẫn được xếp hàng và xử lý khi Judge khởi động lại"
3. **Tiêm lỗi** — Dừng đột ngột (kill) container Judge Service
4. **Quan sát** — Trạng thái ổn định có bị phá vỡ không? Hệ thống phục hồi trong bao lâu?
5. **Khắc phục điểm yếu** — Nếu trạng thái ổn định bị phá vỡ, cần sửa lỗi trên hệ thống (fix system) thay vì điều chỉnh lại kịch bản kiểm thử (fix test)

Diễn tập phòng cháy chữa cháy

Chaos Engineering giống như nhà trường cố tình bấm còi báo "Diễn tập phòng cháy chữa cháy" đột xuất ở Ký túc xá lúc 9 giờ tối. Mục đích là chủ động xem sinh viên có thoát hiểm kịp trong 5 phút không, cửa thoát hiểm có mở không — thay vì đợi đến khi có hỏa hoạn thật mới phát hiện ra cửa đã bị rỉ sét không mở được.

Tương tự như việc ban quản lý ký túc xá chuẩn bị nhiều kịch bản diễn tập khác nhau (chập điện, rò rỉ nước, hỏa hoạn), kỹ sư hệ thống cũng có các nhóm phép tiêm lỗi đa dạng để kiểm thử độ bền bỉ của nền tảng:

__Bảng 11.13:__ Phân loại kỹ thuật tiêm lỗi trong Chaos Engineering

Loại lỗi được tiêm	Ví dụ	Công cụ
Lỗi tiến trình	Kill container/service	Chaos Monkey, `docker stop`
Lỗi mạng	Delay, packet loss, partition	Toxiproxy, Gremlin
Cạn tài nguyên	CPU 100%, disk full, memory leak	stress-ng, Gremlin
Lỗi phụ thuộc	Database down, Kafka unavailable	Litmus, thủ công

Bốn loại lỗi được tiêm trên phủ gần hết các miền lỗi (failure domain) trong hệ thống phân tán. Nhóm phát triển không cần chạy tất cả cùng lúc — nên bắt đầu từ ba thí nghiệm đơn giản nhất, vừa tầm với hạ tầng Docker Compose của KBM và đủ kiểm chứng các mẫu chịu lỗi đã thiết kế ở các chương trước:

- **Kill Judge Service**: Giả thuyết đặt ra là các bài nộp chế độ thi của sinh viên sẽ tự động được xếp hàng an toàn trong Kafka và xử lý tiếp khi service khởi động lại. Kết quả mong đợi là cơ chế lưu giữ của Kafka giữ toàn bộ thông điệp và Judge bắt kịp tiến độ ngay khi dịch vụ phục hồi.
- **Tiêm độ trễ mạng 6000 ms (vượt `readTimeout: 3500` và `timeoutDuration: 5s` của cấu hình Resilience4j ở §4.4.4) giữa Core↔Judge**: Giả thuyết đặt ra là khi luồng chấm đồng bộ phản hồi quá ngưỡng cho phép, cơ chế bảo vệ sẽ ngắt nhanh (fail-fast) để không làm treo nhóm luồng (thread pool) của Core Service. Kết quả mong đợi (khi đã triển khai timeout và circuit breaker ở Giai đoạn 3, Chương 10) là yêu cầu chấm quá hạn trả lỗi có kiểm soát thay vì kéo sập các API học vụ khác trên Core; chạy thí nghiệm này sớm hơn sẽ bộc lộ đúng khoảng trống đó.
- **PostgreSQL restart**: Giả thuyết là Core Service có khả năng tự động kết nối lại cơ sở dữ liệu. Kết quả mong đợi là connection pool (HikariCP) tự động khôi phục kết nối mà không yêu cầu khởi động lại ứng dụng.

Chaos Engineering từ staging

Không cần Chaos Monkey hay công cụ phức tạp cho những bước đầu tiên: chỉ với Docker CLI (`docker stop kbm-judge-sql` trên staging → quan sát log và metric → kiểm tra khả năng phục hồi), nhóm đã thực hành đúng tinh thần Chaos Engineering. Khi quy trình đã chín, mới tự động hóa thành bước kiểm thử trong pipeline CI/CD.

11.8.1. Case study thực tế: Netflix Simian Army

Netflix là người tiên phong của Chaos Engineering. Năm 2010, sau khi chuyển từ trung tâm dữ liệu sang AWS, Netflix tạo **Chaos Monkey** — công cụ tự động tắt ngẫu nhiên các EC2 instance trong production. Mục đích: nếu hệ thống chịu được các sự cố ngẫu nhiên hằng ngày, nó sẽ chịu được sự cố thật.

Chaos Monkey phát triển thành **Simian Army** — tập hợp các công cụ chuyên biệt:

Bảng 11.14: Netflix Simian Army

Công cụ	Chức năng	Bài học
Chaos Monkey	Kill ngẫu nhiên các instance	Các service phải stateless, tự khởi động lại được
Chaos Kong	Giả lập sự cố toàn bộ region	Cơ chế chuyển đổi dự phòng đa vùng (multi-region failover) phải hoạt động
Latency Monkey	Tiêm độ trễ mạng (network delay)	Timeout + circuit breaker phải được cấu hình đúng
Conformity Monkey	Kiểm tra instance tuân thủ các giải pháp khuyến nghị	Tự động hóa thay cho kiểm tra thủ công

Kết quả: Netflix duy trì độ sẵn sàng cao (mục tiêu ~99.99%) khi phục vụ hơn 200 triệu người đăng ký (2021). Casey Rosenthal — người dẫn dắt nhóm Chaos Engineering của Netflix, sau này đồng sáng lập Verica — cùng Nora Jones là đồng tác giả *Chaos Engineering: System Resiliency in Practice* (O'Reilly, 2020).

Bài học cho hệ thống nhỏ: LMS không cần Simian Army. Nhưng nguyên tắc cốt lõi áp dụng: *nếu đội ngũ phát triển e ngại việc vô hiệu hóa đột ngột (kill) một dịch vụ, đó là dấu hiệu hệ thống thiếu khả năng phục hồi*. Bắt đầu thủ công (`docker stop`), tiến tới tự động (CI/CD chaos stage).

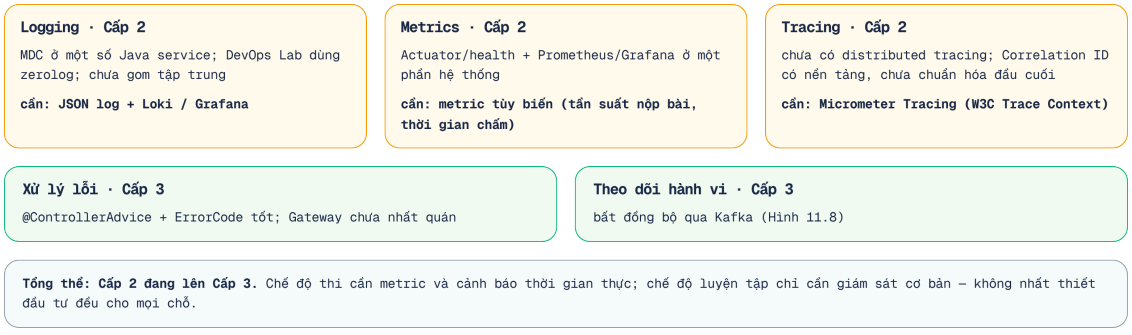
Nguồn: Netflix Technology Blog, "The Netflix Simian Army," 2011 (netflixtechblog.com). Casey Rosenthal et al., Chaos Engineering: System Resiliency in Practice, O'Reilly, 2020.

11.9. Case study: KBM

Phần này phân tích hiện trạng giám sát của KBM và vạch lộ trình nâng cấp tương ứng.

11.9.1. Hiện trạng — đánh giá theo mô hình trưởng thành

Hình 11.9 đánh giá hiện trạng observability của KBM theo từng thành phần.



Hình 11.9: Đánh giá hiện trạng observability của KBM

Đánh giá tổng thể: Cấp 2 đang tiến lên Cấp 3 — KBM đã có Actuator/health ở một số service và nền tảng Prometheus/Grafana cho một phần hệ thống, nhưng vẫn cần chuẩn hóa định danh tương quan, quy trình thu thập log và truy vết (tracing) xuyên suốt ranh giới các dịch vụ. Khi yêu cầu nộp bài phản hồi chậm hoặc thất bại, lập trình viên không nên phải mở nhật ký của từng container và ghép timestamp thủ công.

Bảng 11.15: Mức độ trưởng thành observability phân theo thành phần

Thành phần	Hiện trạng	Cấp độ trưởng thành
Logging	Các dịch vụ Java có log qua MDC ở một số nơi; DevOps Lab dùng zerolog; cần gom tập trung	● Cấp 2
Metrics	Actuator/health + Prometheus/Grafana ở một phần hệ thống	● Cấp 2
Tracing	Distributed tracing (W3C Trace Context) chưa triển khai; `X-Request-ID` và `correlationId` có nền tảng nhưng chưa chuẩn hóa đầu cuối	● Cấp 2
Xử lý lỗi	`@ControllerAdvice` + `ErrorCode` tốt, Gateway chưa nhất quán	● Cấp 3
Theo dõi hành vi người dùng	Bất đồng bộ qua Kafka — tốt	● Cấp 3

11.9.2. Phân tích theo bối cảnh nghiệp vụ

KBM phục vụ sinh viên học SQL, mạng, DevOps và các hoạt động LMS. Điểm đặc biệt là hệ thống có nhiều chế độ hoạt động khác nhau — mỗi chế độ có mẫu lưu lượng, yêu cầu độ trễ và mức chịu lỗi riêng. Chiến lược observability không thể áp dụng một khuôn cho tất cả: chế độ thi cần cảnh báo thời gian thực, chế độ luyện tập chỉ cần giám sát cơ bản. Bảng dưới phân tích chi tiết từng chế độ:

Bảng 11.16: Các chế độ hoạt động chính của LMS

Chế độ	Đặc điểm	SLO cần thiết	Observability cần
Chế độ luyện tập	Sinh viên tự luyện tập, traffic thấp, không có hạn chót	Độ trễ < 10 s là được, downtime chấp nhận được	Giám sát cơ bản là đủ
Chế độ thi	khoảng 200 sinh viên nộp bài đồng thời, có hạn chót, bảng xếp hạng thời gian thực	Tiếp nhận P99 < 500 ms, trả kết quả P99 < 45 giây, tính sẵn sàng ca thi 99.9%	Metrics + cảnh báo bắt buộc
DevOps Lab runtime	Runtime của workspace/container/ mạng biến động	Readiness chính xác, cô lập lỗi	Grafana Alloy/Loki/Grafana + runtime metric

Trong chế độ thi, **không có observability = thiếu tầm nhìn hệ thống (flying blind)**:

- Judge Service quá tải → bài nộp dồn hàng đợi → bảng xếp hạng lỗi thời → sinh viên phàn nàn
- Không có metric tùy biến → không biết thời gian chấm tăng cho đến khi sinh viên báo
- Không có tracing → không biết nút thắt ở Judge hay ở Kafka consumer

ⓘ Observability chưa chuẩn hóa đầu cuối

KBM đã có một số mảnh observability đúng hướng: Actuator/Prometheus cho các dịch vụ Java, `X-Request-ID`/MDC ở thư viện dùng chung, zerolog ở các dịch vụ Go, và bộ PLG (Promtail/Loki/Grafana) cho DevOps Lab. Tuy nhiên, hệ thống vẫn thiếu sự đồng nhất trên toàn chuỗi giao dịch.

Khoảng trống đáng kể hiện tại là **chuẩn hóa đầu cuối**: cùng một yêu cầu hoặc lô chấm bài đi qua Gateway → service → hàng đợi Kafka/DB → Judge/runtime cần giữ nguyên bộ định danh (`X-Request-ID` ở biên HTTP, `correlationId` cho luồng nghiệp vụ đa bước, `traceId` cho truy vết phân tán). Thiếu sự liền mạch này khiến việc gỡ rối bị đứt quãng giữa các tầng.

Để khắc phục, chúng ta cần một **lộ trình chuyển đổi** (tăng dần — mỗi giai đoạn đều mang lại giá trị ngay; mục tiêu là đạt đủ Cấp 3, riêng tracing là một phần của Cấp 4) như sau:

- **Giai đoạn 1 — Log có cấu trúc + Request ID** (chi phí triển khai thấp, tác động cao): Chuẩn hóa JSON logging cho các dịch vụ Java và tên trường zerolog cho Go; chuẩn hóa `X-Request-ID` lan truyền qua HTTP header (có kiểm tra đầu vào và gắn trả lại response) đồng thời truyền

`correlationId` trong message envelope trên Kafka. Giá trị nhận được ngay là khả năng tra cứu chéo dịch vụ chỉ bằng một định danh.

- **Giai đoạn 2 — Gom log tập trung** (chi phí triển khai trung bình): Mở rộng stack log đang chạy ở DevOps Lab (Loki + Grafana) sang LMS core, thu gom bằng Grafana Alloy vì Promtail đã hết vòng đời từ 03/2026. Việc cấu hình pipeline thu thập log tập trung sẽ chấm dứt ngay thói quen SSH vào từng container để đọc log thủ công.
- **Giai đoạn 3 — Distributed Tracing** (chi phí triển khai trung bình): Tích hợp Micrometer Tracing và OpenTelemetry vào mỗi dịch vụ, cùng với Jaeger làm backend, cho phép chúng ta nhìn rõ chính xác điểm nghẽn — liệu sự chậm trễ đến từ Judge Service, Kafka hay cơ sở dữ liệu.
- **Giai đoạn 4 — Metric nghiệp vụ + Cảnh báo** (chi phí triển khai trung bình): Mở rộng Prometheus/Grafana hiện có ra mọi service, thêm metric nghiệp vụ (tần suất nộp bài, thời gian chấm), từ đó thiết lập các cảnh báo dựa trên SLO thực tế như độ trễ nộp bài. Các cảnh báo sẽ được phân cấp hợp lý để gửi thông báo tức thời nếu Judge Service ngừng hoạt động, và phân mức cảnh báo độ trễ theo SLO cùng mức ảnh hưởng đến người dùng, không mặc định mọi bất thường độ trễ là cảnh báo nhẹ.

11.9.3. Những cạm bẫy về observability

Hệ thống giám sát thiết lập sai cách sẽ tự sinh nhiễu (noise), lãng phí tài nguyên và làm giảm độ nhạy bén của đội ngũ vận hành — độ chi tiết của dữ liệu phải được cân với chi phí. Những sai lầm phổ biến nhất:

1. **Log không cấu trúc, thiếu request/correlation ID** — Mỗi service ghi log dạng văn bản tự do riêng, không có ID xuyên suốt một request; khớp log bằng timestamp sai lệch ngay khi có xử lý đồng thời → debug như "mò kim đáy bể". *Phòng tránh*: log có cấu trúc (JSON) + `requestId` (HTTP ingress) và `correlationId` (luồng nghiệp vụ đa bước) lan truyền xuyên suốt mọi service.
2. **Logging quá nhiều hoặc quá ít** — Ghi toàn bộ request body làm lộ dữ liệu nhạy cảm và cạn kiệt lưu trữ; chỉ ghi log lỗi lại mất bối cảnh khi debug. *Phòng tránh*: INFO cho sự kiện nghiệp vụ, DEBUG cho chi tiết kỹ thuật; tuyệt đối không ghi mật khẩu hay PII.
3. **Cảnh báo theo nguyên nhân kỹ thuật thay vì theo SLO** — Cảnh báo cho mọi metric kỹ thuật rời rạc (CPU, một lỗi lẻ) sinh hàng chục thông báo mỗi ngày. Hậu quả: mệt mỏi vì cảnh báo (§11.4) — đội ngũ "miễn nhiễm" với cảnh báo, bỏ lỡ sự cố thật. *Phòng tránh*: cảnh báo dựa trên SLO và ngân sách lỗi (error budget) gắn với trải nghiệm người dùng, phân loại mức độ nghiêm trọng rõ ràng.
4. **Cardinality explosion trong metric** — Gắn giá trị biến thiên cao (`userId`, `submissionId`, `email`) làm nhãn cho Prometheus. Hậu quả: số time-series bùng nổ → quá tải hệ thống metric, tốn chi phí, truy vấn chậm. *Phòng tránh*: chỉ dùng nhãn có cardinality thấp (`status`, `method`, `service`); để chi tiết cho log/trace.
5. **Dùng tracing thay cho logging** — Tracing cho biết request đi đâu và tốn bao lâu, nhưng không giải thích *tại sao* thất bại. *Phòng tránh*: kết hợp cả hai — tracing cho cái nhìn tổng quan, logging làm rõ chi tiết.
6. **Liveness probe "đào" quá sâu** — Health check của liveness gọi tới DB hoặc downstream. Hậu quả: khi cơ sở dữ liệu xử lý chậm, Kubernetes nhận định sai rằng pod đã chết và kill tiến trình hàng loạt → sự cố lan rộng. *Phòng tránh*: liveness chỉ kiểm tra bản thân tiến trình (shallow); đưa kiểm tra phụ thuộc vào readiness.
7. **Bỏ qua observability vì "nhóm nhỏ"** — Ban đầu `docker logs` là đủ, nhưng khi hệ thống mở rộng, đọc log thủ công từ hàng chục container tiềm ẩn nhiều rủi ro vận hành. *Phòng tránh*:

bắt đầu ngay với log có cấu trúc + định danh tương quan — bước tốn ít công sức nhất nhưng giá trị nhất.

Các pattern trong chương có minh họa tương tác đi kèm tại companion web:

Interactive Demo

- **Truy vết phân tán (Distributed Tracing)**: distributed-tracing.html

11.10. Tổng kết

Observability là **điều kiện tiên quyết** để vận hành microservices — không phải tính năng bổ sung. Trong monolith, một tệp log và một trình gỡ lỗi (debugger) đủ để tìm lỗi. Trong microservices, yêu cầu đi qua nhiều service với các kiểu hỏng hóc khó dự đoán trước — cần khả năng *đặt câu hỏi mới* về hành vi hệ thống mà không phải triển khai bản mã mới.

Ba trụ cột — Logs, Metrics, Traces — bổ trợ đa chiều: metric phát hiện vấn đề (độ trễ tăng vọt), trace khoanh vùng vị trí (Judge Service), log chỉ ra nguyên nhân (SQL timeout). Mô hình trưởng thành giúp nhóm xác định *mức độ vừa sức*: với LMS như KBM, Cấp 3 (Proactive) là mục tiêu hợp lý.

Log có cấu trúc cùng bộ định danh (`requestId`, `correlationId`, `traceId`) là nền tảng có chi phí thấp nhưng tác động cao nhất. Trên nền đó, SLI/SLO và ngân sách lỗi chuyển trọng tâm từ "theo dõi chỉ số máy chủ" sang "bảo vệ trải nghiệm học tập", phản hồi lỗi chuẩn hóa theo RFC 9457 phục vụ đồng thời cả ứng dụng khách lẫn hệ thống cảnh báo, còn đường ống theo dõi hành vi qua Kafka mở rộng khả năng quan sát từ hạ tầng sang phân tích học tập.

Phân tích KBM cho thấy các mảnh ghép observability có sẵn cần được nối thành hợp đồng đầu cuối theo lộ trình tăng dần: log có cấu trúc + định danh tương quan → gom log tập trung → tracing → metric nghiệp vụ và cảnh báo theo SLO. Mỗi bước đều mang lại giá trị ngay lập tức.

Ở Chương 12, chúng ta sẽ chuyển sang **Triển khai và DevOps** — containerization, Docker Compose, pipeline CI/CD và cách triển khai hệ thống microservices hoàn chỉnh.

11.11. Đọc thêm

Sách tham khảo chính:

1. [2a] Chris Richardson, *Microservices Patterns* 1st Ed. — Ch.11: Developing Production-Ready Services — Health check API, Log aggregation, Distributed tracing, Application metrics
2. [4a] Sam Newman, *Building Microservices* — Ch.8: Monitoring — Logs, Metric tracking, Synthetic monitoring, Correlation IDs

Sách bổ trợ:

3. [5] Hugo Rocha, *Practical Event-Driven Microservices Architecture* — Distributed Tracing, Domain Segregation
4. [3] Ronnie Mitra & Irakli Nadareishvili, *Microservices: Up and Running* — Ch.9: Monitoring infrastructure
5. [4b] Sam Newman, *Monolith to Microservices* — Reporting, Monitoring and Troubleshooting during migration

Nguồn trực tuyến:

- OpenTelemetry official docs — opentelemetry.io

- Micrometer docs — micrometer.io (Spring Boot observability)
- Grafana Loki — grafana.com/oss/loki (lightweight log aggregation)
- Jaeger — jaegertracing.io (distributed tracing)
- Grafana Tempo — grafana.com/oss/tempo (tracing backend trong hệ sinh thái Grafana, một lựa chọn thay cho Jaeger)
- Brendan Gregg, "USE Method" — brendangregg.com/usemethod.html
- Tom Wilkie, "RED Method" — grafana.com/blog/2018/08/02/the-red-method-how-to-instrument-your-services/
- Google SRE Book, "Service Level Objectives" — sre.google/sre-book/service-level-objectives

Bài tập: 11-0, 11-1, 11-2 ở phần Bài tập & case study cuối sách.

CHƯƠNG 12

Triển khai & DevOps

> Nếu triển khai thấy đau, hãy triển khai thường xuyên hơn; tự động hóa biến một việc đáng sợ thành việc bình thường.
> — Theo tinh thần Jez Humble & David Farley, *Continuous Delivery* (2010)

MỤC TIÊU HỌC TẬP

- Hiểu vì sao triển khai microservices phức tạp hơn monolith và cần **tự động hóa**
- Nắm vững **container hóa** (containerization) với Docker — từ Dockerfile đến image registry
- Sử dụng **Docker Compose** để điều phối việc triển khai nhiều service trên một máy
- Thiết kế **pipeline CI/CD** cho microservices — build, kiểm thử và triển khai tự động
- So sánh các **chiến lược triển khai** (deployment strategies) Rolling, Blue/Green, Canary và Shadow
- Hiểu **Infrastructure as Code** (IaC) và vai trò của các nền tảng điều phối triển khai
- Phân tích kiến trúc triển khai của KBM và đề xuất cải thiện

12.1. Thách thức triển khai microservices

Triển khai monolith chỉ cần sao chép một artifact duy nhất, nhưng với microservices, đó là bài toán điều phối hàng chục dịch vụ độc lập. Sự phức tạp này đòi hỏi các luồng tự động hóa chuyên biệt.

12.1.1. Vấn đề: từ "triển khai một tệp WAR" đến "điều phối N service độc lập"

Trong monolith, triển khai nghĩa là: build một artifact (JAR/WAR), sao chép lên máy chủ và khởi động lại. Toàn bộ quy trình từ build, kiểm thử đến triển khai chỉ xoay quanh một tệp duy nhất.

Trong microservices, triển khai mang ý nghĩa khác hẳn. Thay vì đóng gói một ứng dụng duy nhất lên một máy chủ, hệ thống phải quản lý hàng chục artifact riêng lẻ có nhịp phát hành độc lập. Cơ sở dữ liệu cũng vậy: monolith chỉ cần một luồng cập nhật schema thống nhất, còn mỗi microservice sở hữu schema riêng và phải nâng cấp độc lập mà vẫn giữ tương thích với các dịch vụ xung quanh.

Quá trình khởi động và khôi phục hệ thống cũng phức tạp hơn: các tiến trình cần cơ chế chịu lỗi khi phụ thuộc chưa sẵn sàng, và khi có sự cố cần quay lui (rollback), việc đưa một service về phiên bản cũ mà không phá vỡ hợp đồng giao tiếp với các service khác đòi hỏi kỷ luật tương thích ngược chặt chẽ. Ngay cả kiểm thử tích hợp cũng cần dựng kèm các thành phần liên quan hoặc kiểm thử hợp đồng thay thế.

Newman trong [4a, Ch.6] nhấn mạnh: khả năng **triển khai độc lập** (independent deployment) là lợi ích cốt lõi của microservices — nhưng cũng là thách thức lớn. Nếu không tự động hóa, việc triển khai thủ công sẽ trở thành "nỗi ám ảnh chiều thứ Sáu" khi số dịch vụ tăng lên.

Triển khai độc lập

Khả năng triển khai từng service độc lập là lý do cốt lõi để chọn microservices — nhưng không có CI/CD, kỷ luật quản lý phiên bản và hạ tầng bất biến (immutable infrastructure), lợi ích này biến thành gánh nặng. Mỗi lần phát hành thủ công là rủi ro: bỏ sót bước, sai thứ tự migration, thiếu kế hoạch rollback. Tự động hóa DevOps không phải tùy chọn trong microservices — đó là **điều kiện bắt buộc** để lợi ích độc lập trở thành thực tế.

12.1.2. Từ thủ công đến tự động hóa — tư duy DevOps

Theo Mitra [3, Ch.6], DevOps cho microservices có ba nguyên tắc cốt lõi. Nguyên tắc đầu tiên là **Immutable Infrastructure** (hạ tầng bất biến): tuyệt đối không can thiệp trực tiếp vào máy chủ đang chạy mà phải khởi tạo phiên bản mới và loại bỏ phiên bản cũ, nhằm đảm bảo khả năng tái tạo và triệt tiêu sự sai lệch cấu hình (configuration drift). Kế tiếp là **Infrastructure as Code** (IaC), biến mọi thao tác cấu hình máy chủ, mạng hay cơ sở dữ liệu thành mã nguồn lưu trữ trên Git, giúp dễ dàng rà soát và quay lui. Cuối cùng, **Continuous Delivery** (phân phối liên tục) đảm bảo mã nguồn luôn ở trạng thái sẵn sàng phát hành chỉ với một thao tác kích hoạt, qua đó chia nhỏ rủi ro và tăng tốc vòng lặp phản hồi.

Hình 12.1: *Triển khai thủ công và tự động*

Mô hình triển khai tự động giúp loại bỏ các bước thủ công dễ sinh lỗi và giảm rủi ro khi phát hành phần mềm. Quá trình phân phối tự động phản ánh triết lý vận hành cốt lõi của DevOps: biến khó khăn thành động lực tự động hóa (if it hurts, do it more often).

If It Hurts, Do It More Often

Nếu triển khai thấy đau, hãy triển khai thường xuyên hơn. Chỗ đau cho biết cần tự động hóa điều gì.

— Theo tinh thần Jez Humble & David Farley, Continuous Delivery

12.2. Containerization với Docker

12.2.1. Vấn đề: “Works on my machine”

Sự không nhất quán giữa môi trường phát triển (máy cá nhân) và môi trường production thường xuyên gây ra lỗi “Works on my machine”. Container hóa giải quyết bài toán này.

12.2.2. Container — cô lập gọn nhẹ

Container (Docker) đóng gói ứng dụng cùng với các thư viện phụ thuộc (dependency) và môi trường thực thi (runtime) thành một **image** — chạy giống nhau trên mọi môi trường. Khác với máy ảo (Virtual Machine), container chia sẻ kernel với hệ điều hành máy chủ nên giảm đáng kể dung lượng và khởi động tính bằng giây thay vì tính bằng phút.

Hình 12.2: So sánh kiến trúc máy ảo và container

Khác biệt về kiến trúc giữa hai công nghệ này kéo theo khác biệt đáng kể về hiệu năng và cách vận hành. Về thời gian khởi động, máy ảo thường mất hàng phút do phải nạp lại toàn bộ hệ điều hành khách, trong khi container chỉ mất vài giây vì dùng chung kernel với máy chủ. Về tài nguyên, một máy ảo đòi hỏi dung lượng lưu trữ lên đến hàng gigabyte, trong khi container tối ưu hóa đáng kể tài nguyên do chỉ bao gồm ứng dụng và các thư viện cần thiết.

Đổi lại, mức độ cô lập của hai công nghệ khác nhau: máy ảo cô lập ở mức phần cứng nên phù hợp cho môi trường chia sẻ đa khách hàng (multi-tenant), còn container cô lập ở mức tiến trình của hệ điều hành.

Phòng thí nghiệm riêng biệt và buồng thực hành dùng chung hạ tầng

Để hình dung sự khác biệt về bản chất trong bối cảnh trường đại học:

- **Máy ảo (Virtual Machine)**: Giống như xây cho mỗi nhóm nghiên cứu một phòng thí nghiệm độc lập với hệ thống điện dự phòng, điều hòa và tủ thiết bị riêng hoàn toàn (hệ điều hành khách riêng). Mức cô lập rất cao, nhưng tốn diện tích và mất nhiều thời gian chuẩn bị trước mỗi ca học.
- **Container**: Giống như bố trí cho mỗi nhóm một buồng thực hành cô lập bên trong Trung tâm Thí nghiệm chung: mỗi nhóm có bộ dụng cụ và không gian thao tác riêng (user space), nhưng dùng chung điện lưới, mạng nội bộ và hệ thống thông gió của tòa nhà (OS kernel). Tài nguyên được tận dụng tối ưu và buồng thực hành sẵn sàng chỉ sau vài giây.

Richardson trong [2a, Ch.12] mô tả bốn mô hình triển khai (deployment pattern), trong đó **Service as Container** là phổ biến nhất cho microservices — cân bằng giữa mức độ cô lập, tốc độ khởi tạo và hiệu suất sử dụng tài nguyên.

12.2.3. Các mô hình triển khai — từ đặc thù ngôn ngữ đến Serverless

Bốn mô hình triển khai của Richardson trải dài từ đơn giản đến phức tạp. Cách tiếp cận nguyên sơ nhất là đóng gói ứng dụng theo định dạng đặc thù của ngôn ngữ (như JAR, WAR) rồi triển khai trực tiếp lên application server; dù đơn giản nhưng lại tiềm ẩn nguy cơ cao xảy ra lỗi xung đột thư viện do thiếu tính cô lập. Kế tiếp, việc đặt mỗi dịch vụ lên một máy ảo riêng giải quyết được bài toán cô lập, nhưng tốn nhiều tài nguyên và khởi tạo chậm.

Mô hình mỗi dịch vụ một container (như Docker) cân bằng được tính mỏng nhẹ, khả năng di động và sự độc lập, dù vẫn cần công cụ điều phối ở quy mô lớn. Khi cần tối giản vận hành, kiến trúc Serverless giao toàn bộ hạ tầng cho đám mây để đạt khả năng mở rộng tức thì, chấp nhận độ trễ khởi động và sự phụ thuộc nhà cung cấp. Với các hệ thống lớn, mô hình container thường chạy trên Kubernetes, nền tảng điều phối và tự phục hồi tiêu chuẩn, dù đòi hỏi nhiều công sức tiếp cận ban đầu.

12.2.4. Dockerfile — định nghĩa container image

Dockerfile mô tả cách đóng gói container image cho một service. Mẫu phổ biến cho microservice viết bằng Spring Boot:

__Listing 12.1:__ Dockerfile sử dụng multi-stage build

```
# Multi-stage build — tách build environment khỏi runtime
FROM eclipse-temurin:21-jdk AS build
WORKDIR /app
COPY pom.xml mvnw ./
COPY .mvn .mvn
RUN ./mvnw dependency:go-offline -B    # Cache dependencies layer
COPY src src
RUN ./mvnw package -DskipTests        # Build JAR

# Runtime image nhỏ hơn JDK
FROM eclipse-temurin:21-jre
WORKDIR /app

# Tạo non-root user (giải pháp khuyến nghị về bảo mật)
RUN useradd -m appuser
USER appuser

COPY --from=build /app/target/*.jar app.jar
EXPOSE 8080
# Khởi động ứng dụng (Spring Boot tự quản lý Graceful Shutdown qua cấu hình)
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Hai kỹ thuật quan trọng:

- **Build đa giai đoạn (multi-stage build)**: giai đoạn 1 (JDK) biên dịch tệp JAR, giai đoạn 2 (JRE) chỉ chứa môi trường thực thi, giúp giảm đáng kể dung lượng image
- **Cache theo lớp (layer caching)**: sao chép tệp cấu hình trước mã nguồn (`COPY pom.xml` trước `COPY src`), nhờ đó các thư viện phụ thuộc chỉ được tải lại khi cấu hình thay đổi, thay vì mỗi lần cập nhật mã

Mỗi service một container

Mỗi microservice chạy trong một container riêng. Không đóng gói nhiều dịch vụ vào chung một container — cách làm đó đánh mất tính cô lập, khó mở rộng và khó gỡ lỗi.

— Tổng hợp nguyên tắc container hóa từ Richardson [2a, Ch.12] và Newman [4a, Ch.6]

12.3. Docker Compose — điều phối trên một máy

12.3.1. Vấn đề: khởi chạy thủ công nhiều dịch vụ, cơ sở dữ liệu và Kafka

Nền tảng KBM yêu cầu khởi động nhiều dịch vụ, cơ sở dữ liệu và message broker theo đúng thứ tự. Việc chạy thủ công từng thành phần làm tốn thời gian và dễ dẫn đến sai lệch cấu hình mạng.

__Listing 12.2:__ Khởi động thủ công các service (anti-pattern)

```
# SAI: khởi động thủ công bằng 10+ lệnh docker run,
# đúng thứ tự, đúng network, đúng env vars
docker run -d postgres:15 ...
docker run -d zookeeper:3.8 ...
docker run -d kafka:3.5 ...
docker run -d eureka-server ...
docker run -d kbm-gateway --eureka-url=... ...
docker run -d kbm-core --db-url=... --kafka=... ...
# ... còn 5 services nữa
```

12.3.2. Docker Compose — khai báo nhiều service

Docker Compose định nghĩa **toàn bộ stack** trong một tệp YAML — các service, mạng (network), ổ lưu trữ (volume) và quan hệ phụ thuộc. Một lệnh `docker compose up` khởi động toàn bộ hệ thống.

Cấu trúc Docker Compose cho KBM LMS chính (đơn giản hóa):

__Listing 12.3:__ Cấu hình Docker Compose cho KBM LMS chính

```
# docker-compose.yml — KBM LMS chính
services:
  # --- Infrastructure ---
  postgres:
    image: postgres:15
    restart: unless-stopped
    environment:
      POSTGRES_DB: core_db
      POSTGRES_PASSWORD: ${DB_PASSWORD}
    volumes:
      - pgdata:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres"]
      interval: 10s
      timeout: 5s
      retries: 5

  # Kafka chạy KRaft mode (production-ready từ 3.3, mặc định từ 4.0):
  # broker tự quản metadata, không cần Zookeeper.
  # Cụm dựa trên Zookeeper nay là legacy (gỡ bỏ ở Kafka 4.0).
  kafka:
    image: confluentinc/cp-kafka:7.5.0
    restart: unless-stopped
    environment:
      # bắt buộc ở KRaft: format volume metadata
      CLUSTER_ID: "MkU3OEVBNTcwNTJENDM2Qk"
      KAFKA_PROCESS_ROLES: "broker,controller"
      KAFKA_NODE_ID: 1
      KAFKA_CONTROLLER_QUORUM_VOTERS: "1@kafka:9093"
      KAFKA_LISTENERS: "PLAINTEXT://:9092,CONTROLLER://:9093"
      # Listener tên CONTROLLER không trùng tên security protocol nào, nên phải
      # khai báo map tường minh; thiếu nó broker abort ngay khi validate config.
```

```
    KAFKA_LISTENER_SECURITY_PROTOCOL_MAP:
      "PLAINTEXT:PLAINTEXT,CONTROLLER:PLAINTEXT"
    KAFKA_CONTROLLER_LISTENER_NAMES: "CONTROLLER"
    KAFKA_INTER_BROKER_LISTENER_NAME: "PLAINTEXT"
    # Địa chỉ client nhận về sau bootstrap — thiếu thì service không kết nối được
    KAFKA_ADVERTISED_LISTENERS: "PLAINTEXT://kafka:9092"
    KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1   # single-node
  volumes:
    # giữ log và metadata KRaft qua các lần down/up
    - kafka-data:/var/lib/kafka/data
  healthcheck:
    test: ["CMD", "kafka-cluster", "cluster-id",
           "--bootstrap-server", "localhost:9092"]
    interval: 10s
    timeout: 5s
    retries: 5

# --- Platform Services ---
kbm-registry:
  image: ghcr.io/kbm/kbm-registry
  ports: ["9000:9000"]

kbm-gateway:
  image: ghcr.io/kbm/kbm-gateway
  ports: ["9001:9001"]
  depends_on: [kbm-registry]

# --- Application Services ---
kbm-core:
  # nhận tag từ pipeline; thiếu biến này thì luôn kéo latest
  image: ghcr.io/kbm/kbm-core:${IMAGE_TAG:-latest}
  restart: unless-stopped
  depends_on:
    postgres:
      condition: service_healthy
    kafka:
      condition: service_healthy
    kbm-registry:
      condition: service_started
  environment:
    SPRING_DATASOURCE_URL: jdbc:postgresql://postgres:5432/core_db
  # --wait ở pipeline chỉ đợi healthy khi có khối này;
  # image cần có wget hoặc curl
  healthcheck:
    test: ["CMD-SHELL",
           "wget -qO- http://localhost:8080/actuator/health || exit 1"]
    interval: 10s
    retries: 12
  deploy:
    resources:
      limits:
        cpus: '1.0'
        memory: 512M

# kbm-academic, kbm-auth, kbm-notification khai báo tương tự kbm-core
# (lược bớt cho gọn)
kbm-judge-sql:
  image: ghcr.io/kbm/kbm-judge-sql
```

```
    depends_on:
      kafka:
        condition: service_healthy
      kbm-registry:
        condition: service_started

volumes:
  pgdata:
  kafka-data:
```

12.3.3. Docker Compose — điểm mạnh và giới hạn

Bốn khía cạnh dưới đây nêu điểm mạnh kèm giới hạn tương ứng của Docker Compose:

- **Về tính khai báo (Declarative)**: Docker Compose định nghĩa toàn bộ stack trong một tệp cấu hình duy nhất, giúp dễ hình dung tổng thể hệ thống; đổi lại, công cụ này *chỉ điều phối trên một máy chủ duy nhất (single host)*.
- **Về tính tái tạo (Reproducible)**: Chỉ với lệnh `docker compose up` là có thể dựng lại môi trường nhất quán trên máy phát triển hay staging. Tuy nhiên, ngoài chính sách khởi động lại tiến trình cơ bản (`restart: unless-stopped`), Compose *không có cơ chế tự phục hồi cấp cụm* (như dời workload sang máy chủ khác khi nút phần cứng gặp sự cố).
- **Về quản lý phụ thuộc (Dependencies)**: Chỉ thị `depends_on` kết hợp `condition: service_healthy` đợi lệnh kiểm tra sức khỏe (ví dụ `pg_isready`) báo thành công trước khi bật container kế tiếp. Dù vậy, lệnh kiểm tra thành công chưa đảm bảo tiến trình bên trong đã hoàn tất mọi bước khởi tạo nghiệp vụ, nên ứng dụng vẫn cần cơ chế thử lại kèm thời gian chờ tăng dần (retry/backoff) khi kết nối tới phụ thuộc. Ngoài ra, Compose *không tích hợp sẵn bộ cân bằng tải nội bộ* giữa nhiều bản sao, đòi hỏi cấu hình thêm reverse proxy nếu muốn nhân bản tiến trình.
- **Về mạng nội bộ và cập nhật (Networking, Updates)**: Mạng nội bộ cho phép các dịch vụ gọi nhau thuận tiện qua tên service. Tuy nhiên, khi cập nhật phiên bản, Compose *thiếu cơ chế cập nhật cuốn chiếu (rolling update) không gián đoạn* dựa trên readiness probe.

Docker Compose **phù hợp cho** môi trường phát triển, staging, kiểm thử tự động và **hệ thống production quy mô nhỏ và vừa** (như KBM LMS chính). Phân hệ DevOps Lab là ngoại lệ có chủ đích: phân hệ này dùng cụm k3s nhỏ kèm Sysbox để cô lập không gian thực hành của sinh viên, chứ không phải vì toàn bộ hệ thống đã đòi hỏi hạ tầng Kubernetes quy mô lớn.

KBM: Docker Compose đủ cho production quy mô nhỏ–vừa

KBM LMS chính dùng Docker Compose thay vì Kubernetes — đây là lựa chọn **hợp lý cho giai đoạn hiện tại**. Docker Compose giảm đáng kể chi phí vận hành (không quản lý etcd, kube-apiserver, node pool) và đủ cho việc triển khai trên một máy chủ với tải vừa phải. Chọn Kubernetes trước khi thực sự cần mở rộng đa máy chủ hay tự phục hồi cấp cụm là thiết kế quá mức cần thiết — không phải giải pháp khuyến nghị.

12.4. Pipeline CI/CD cho microservices

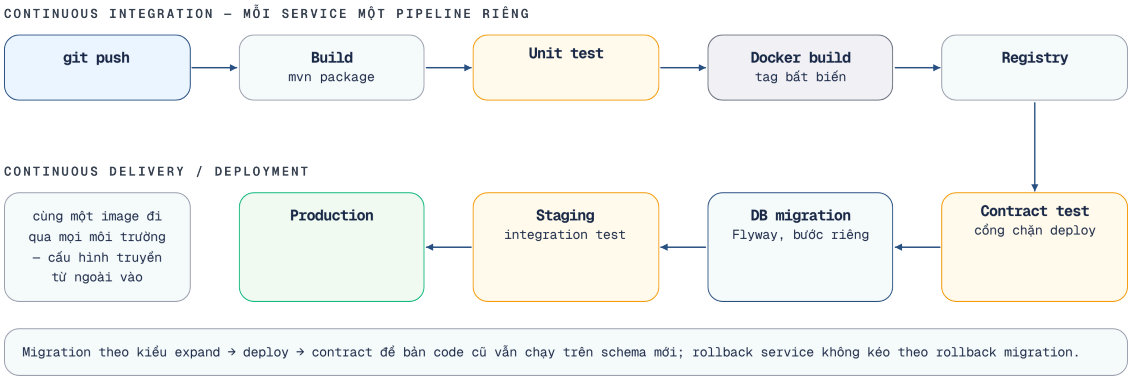
12.4.1. Vấn đề: N service × M giai đoạn = độ phức tạp nhân bội

Trong kiến trúc monolith, pipeline CI/CD thường chỉ là một luồng duy nhất. Với microservices, mỗi dịch vụ sở hữu một vòng đời riêng, đòi hỏi pipeline tự động hóa phải xử lý được các ràng buộc phức tạp:

- Việc triển khai phiên bản mới của Service A **không được phép làm gián đoạn** Service B (API versioning, Chương 3)
- Database migration chạy **trước** bước triển khai service mới
- Rollback service **không** được kéo theo rollback migration: schema phải tương thích ngược (expand → deploy → contract, xem mục Database Migration) để phiên bản mã cũ vẫn chạy được trên schema đã mở rộng — down migration chỉ dùng khi đã chứng minh được tính khả đảo và có bản sao lưu

12.4.2. Kiến trúc CI/CD cho microservices

Hình 12.3 là pipeline CI/CD chuẩn cho microservices, đáp ứng các yêu cầu vừa nêu.



Hình 12.3: *Pipeline CI/CD chuẩn cho microservices*

Pipeline CI/CD và đồ án tốt nghiệp

Toàn bộ pipeline CI/CD có thể ví như quá trình nộp đồ án:

- **Build**: Đóng quyển đồ án và dán mã số sinh viên bất biến (tương tự image tag).
- **Kiểm thử tự động**: Phòng Đào tạo tự động chạy hệ thống quét đạo văn và kiểm tra thể thức văn bản.
- **Deploy Staging**: Sinh viên bảo vệ thử đồ án trước Hội đồng Bộ môn.
- **Deploy Production**: Sau khi duyệt, quyển đồ án chính thức được đưa lên kệ Thư viện trường.

Đặc biệt, tính **bất biến** (immutable) nghĩa là: khi phát hiện lỗi, sinh viên *không được chỉnh sửa trực tiếp trên bản đồ án đã nộp lưu chiểu* (không SSH vào máy chủ sửa mã). Mọi thay đổi phải đi từ bản nháp, đóng quyển mới và làm lại toàn bộ quy trình nộp duyệt.

12.4.3. Mono-repo và Poly-repo — ảnh hưởng đến CI/CD

Câu hỏi đầu tiên khi thiết kế CI/CD là lưu trữ mã nguồn ở đâu. Giới công nghệ chia làm hai trường phái: **Mono-repo** (như cách Google, Meta làm) tích hợp tất cả dịch vụ vào chung một kho lưu trữ, giúp chia sẻ mã nguồn và thực hiện các thay đổi xuyên service rất tiện lợi. Đổi lại, pipeline CI/CD cần bộ lọc đường dẫn để chỉ build những dịch vụ thực sự có thay đổi. Ngược lại, **Poly-repo** (được Netflix, Amazon ưa chuộng) phân tách mỗi dịch vụ thành một kho lưu trữ độc lập: thiết lập N pipeline độc lập rất tự nhiên, nhưng việc chia sẻ thư viện nội bộ tốn thêm bước phát hành phiên bản qua kho gói (artifact repository).

Mitra trong [3, Ch.6] khuyến nghị: đối với đội ngũ quy mô vừa và nhỏ, **mono-repo là lựa chọn đơn giản hơn** — giúp tránh chi phí quản lý (overhead) cho nhiều kho lưu trữ và luồng pipeline. Khi đội ngũ lớn (>20 lập trình viên), poly-repo mang lại mức độ tự chủ cao hơn cho các nhóm.

12.4.4. Tối ưu hóa pipeline

Các giải pháp khuyến nghị trong việc thiết kế pipeline CI/CD bao gồm:

- **Build một lần, triển khai mọi nơi (build once, deploy everywhere)**: Artifact (ví dụ Docker image) chỉ được build một lần duy nhất, sau đó lần lượt được chuyển tiếp (promote) qua các môi trường staging và production. Cách làm này dùng chung một artifact duy nhất, triệt tiêu rủi ro "thử nghiệm chạy tốt nhưng production lỗi" do mã nguồn bị build lại.
- **Cấu hình bên ngoài (externalized config)**: Toàn bộ cấu hình đặc thù của môi trường (như URL cơ sở dữ liệu, khóa API) phải được truyền vào ứng dụng thông qua biến môi trường. Container image phải luôn đảm bảo tính bất biến (immutability) bất kể nó đang được triển khai ở đâu.
- **Pipeline song song (parallel pipeline)**: Mỗi service phải đóng gói, kiểm thử và phân phối được hoàn toàn độc lập. Việc nâng cấp Service A không được trở thành điểm nghẽn (bottleneck) cản trở quá trình phát hành của Service B.
- **Chạy cập nhật schema cơ sở dữ liệu như một bước riêng**: Công cụ quản lý cập nhật schema như Flyway hay Liquibase cần được thực thi như một bước độc lập trước khi khởi chạy ứng dụng mới. Việc tách thay đổi cấu trúc lưu trữ khỏi thay đổi logic mã nguồn giúp quá trình rollback rõ ràng và an toàn hơn.
- **Cổng kiểm thử hợp đồng (contract test gate)**: Trong hệ sinh thái giao tiếp chéo, pipeline chỉ được phép triển khai khi toàn bộ kiểm thử hợp đồng (contract test) đều đạt. Đây là chốt chặn ngăn các thay đổi phá vỡ tương thích (breaking change) lọt lên production.

Build Once, Deploy Everywhere

Build artifact một lần duy nhất, rồi chuyển tiếp chính artifact đó qua các môi trường (dev → staging → production). Nếu mỗi môi trường build lại một lần, thứ được kiểm thử ở staging không còn là artifact chạy trên production.

— Tổng hợp nguyên tắc CI/CD từ Newman [4a, Ch.6] và Mitra [3, Ch.10]

Đảm bảo tính bất biến của artifact qua các môi trường là cơ sở để xây dựng hệ thống tự động hóa đáng tin cậy.

Triển khai thủ công khó duy trì khi số service tăng

Với N service độc lập, mỗi service có bước cập nhật schema riêng, quản lý phiên bản riêng và kế hoạch rollback riêng — việc triển khai thủ công rất chậm và **khó bảo trì khi mở rộng quy mô**. Mỗi thao tác thủ công đều tiềm ẩn rủi ro: sai thứ tự cập nhật schema dữ liệu, thiếu kịch bản rollback, hay lỗi do con người. Tự động hóa CI/CD biến việc triển khai thành một quy trình thường quy, và nghiên cứu DORA cho thấy triển khai thường xuyên hơn lại **giúp giảm tỷ lệ lỗi** (xem Chương 1, DORA Metrics).

12.4.5. Ví dụ thực tế: pipeline CI/CD cho KBM

KBM thực tế tách mỗi dịch vụ thành một kho lưu trữ độc lập (poly-repo) và vận hành trên **GitLab CI** kèm GitLab Container Registry. Để minh họa kỹ thuật lọc theo đường dẫn (`paths` filter) trên một

nền tảng quen thuộc với đa số người đọc, ví dụ dưới đây giả định cấu trúc mono-repo trên **GitHub Actions** cho `kbm-academic` — các bước (build → test → scan → push → deploy) ánh xạ 1:1 sang `.gitlab-ci.yml` (stages/jobs). Mỗi service có workflow riêng và chỉ kích hoạt khi có thay đổi trong thư mục của service đó.

Listing 12.4: *Pipeline GitHub Actions cho `kbm-academic` — build → test → scan → push → deploy*

```yaml
# .github/workflows/kbm-academic.yml
name: kbm-academic CI/CD

on:
  push:
    branches: [main]
    paths: [kbm-academic/**]   # chỉ trigger khi service này thay đổi

permissions:
  contents: read
  # GITHUB_TOKEN mặc định chỉ đọc; cần quyền này để push lên ghcr.io
  packages: write

env:
  REGISTRY: ghcr.io
  IMAGE: ${{ github.repository }}/kbm-academic

jobs:
  build-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Set up JDK 21
        uses: actions/setup-java@v4
        with:
          java-version: '21'
          distribution: 'temurin'
          cache: 'maven'

      - name: Build and test
        run: mvn -f kbm-academic/pom.xml verify

      - name: Log in to registry
        uses: docker/login-action@v3
        with:
          registry: ${{ env.REGISTRY }}
          username: ${{ github.actor }}
          password: ${{ secrets.GITHUB_TOKEN }}   # tự động từ GitHub Actions

      - name: Build image (local)
        uses: docker/build-push-action@v5
        with:
          context: kbm-academic
          load: true
          tags: ${{ env.REGISTRY }}/${{ env.IMAGE }}:${{ github.sha }}

      - name: Scan vulnerability
        # ghim theo commit, không dùng nhánh
        uses: aquasecurity/trivy-action@<commit-sha>
        with:
          image-ref: '${{ env.REGISTRY }}/${{ env.IMAGE }}:${{ github.sha }}'
```

```
          format: 'table'
          exit-code: '1'
          severity: 'CRITICAL,HIGH'

      - name: Push image
        run: |
          # Push ĐÚNG image đã scan ở bước trước (không build lại)
          # — giữ nguyên tắc "build once"
          docker tag ${{ env.REGISTRY }}/${{ env.IMAGE }}:${{ github.sha }} \
            ${{ env.REGISTRY }}/${{ env.IMAGE }}:latest
          docker push ${{ env.REGISTRY }}/${{ env.IMAGE }}:${{ github.sha }}
          docker push ${{ env.REGISTRY }}/${{ env.IMAGE }}:latest

  deploy-staging:
    needs: build-test
    runs-on: ubuntu-latest
    environment: staging
    steps:
      - name: Run DB migration
        uses: appleboy/ssh-action@v1
        with:
          host: ${{ secrets.STAGING_HOST }}
          username: ${{ secrets.STAGING_USER }}
          key: ${{ secrets.STAGING_SSH_KEY }}
          script: |
            # BẢO MẬT: Inject qua file .env thay vì command line
            # để tránh lộ secret trong `ps aux`
            # dừng ngay khi có lệnh lỗi — exit code thất bại lan về pipeline
            set -Eeuo pipefail
            # luôn dọn file secret, kể cả khi migration thất bại
            trap 'rm -f .env.migration' EXIT
            echo "SPRING_DATASOURCE_URL=${{ secrets.STAGING_DB_URL }}" \
              > .env.migration
            IMAGE_TAG=${{ github.sha }} \
              docker compose -f /app/docker-compose.yml \
              run --rm --no-deps --env-from-file .env.migration kbm-academic \
                --spring.flyway.enabled=true \
                --spring.kafka.listener.auto-startup=false \
                --spring.main.web-application-type=none

      - name: Deploy service
        uses: appleboy/ssh-action@v1
        with:
          host: ${{ secrets.STAGING_HOST }}
          username: ${{ secrets.STAGING_USER }}
          key: ${{ secrets.STAGING_SSH_KEY }}
          script: |
            # chặn pipeline đến khi healthcheck báo healthy
            IMAGE_TAG=${{ github.sha }} \
              docker compose -f /app/docker-compose.yml \
              up -d --no-deps --wait kbm-academic

  deploy-production:
    needs: deploy-staging
    runs-on: ubuntu-latest
    environment: production    # yêu cầu manual approval trong GitHub Environments
    steps:
      - name: Run DB migration
```

```
        uses: appleboy/ssh-action@v1
        with:
          host: ${{ secrets.PROD_HOST }}
          username: ${{ secrets.PROD_USER }}
          key: ${{ secrets.PROD_SSH_KEY }}
          script: |
            set -Eeuo pipefail
            trap 'rm -f .env.migration' EXIT
            echo "SPRING_DATASOURCE_URL=${{ secrets.PROD_DB_URL }}" \
              > .env.migration
            IMAGE_TAG=${{ github.sha }} \
              docker compose -f /app/docker-compose.yml \
              run --rm --no-deps --env-from-file .env.migration kbm-academic \
                --spring.flyway.enabled=true \
                --spring.kafka.listener.auto-startup=false \
                --spring.main.web-application-type=none

      - name: Deploy to production
        uses: appleboy/ssh-action@v1
        with:
          host: ${{ secrets.PROD_HOST }}
          username: ${{ secrets.PROD_USER }}
          key: ${{ secrets.PROD_SSH_KEY }}
          script: |
            IMAGE_TAG=${{ github.sha }} \
              docker compose -f /app/docker-compose.yml \
              up -d --no-deps --wait kbm-academic
```

Một số điểm cần chú ý:

- `paths: [kbm-academic/**]` — trong cấu trúc mono-repo, bộ lọc đường dẫn (path filter) giúp tránh build lại toàn bộ kho mã khi chỉ một dịch vụ thay đổi (với cấu trúc poly-repo như KBM, mỗi kho lưu trữ vốn đã kích hoạt pipeline riêng).
- Bước cập nhật schema (`Run DB migration`) chạy **trước** khi triển khai service ở cả `staging` lẫn `production`, và chạy qua `docker compose ... run --rm --no-deps` để vào đúng mạng nội bộ của cơ sở dữ liệu. Thứ tự này là bắt buộc — xem mục Database Migration trong pipeline CI/CD bên dưới.
- `environment: production` kết hợp GitHub Environments cho phép đặt người duyệt bắt buộc (required reviewers): mỗi lần triển khai lên production cần con người phê duyệt.
- `github.sha` làm image tag thay vì `latest` — đảm bảo nguyên tắc "build once, deploy everywhere": cùng một artifact được chuyển tiếp từ staging lên production, không build lại.

12.4.6. Quản lý secret

Workflow trên dùng nhiều secret (`STAGING_HOST`, `PROD_SSH_KEY`, `STAGING_DB_URL`...). Nguyên tắc cơ bản:

> ⚠ **Không gắn cứng thông tin xác thực**
>
> Đặt thông tin xác thực (credentials) trực tiếp trong tệp `.yml` là lỗi nghiêm trọng — tệp workflow được lưu vào lịch sử Git và có thể bị lộ công khai. Dùng GitHub Secrets (Settings → Secrets and variables → Actions) để lưu trữ. `${{ secrets.GITHUB_TOKEN }}` là token tự động — không cần tạo thủ công.

Cụ thể, các secret thiết yếu của pipeline ví dụ ở trên được phân theo phạm vi (scope) như sau:

- `GITHUB_TOKEN`: Dùng để tự động xác thực với registry. GitHub Actions tự sinh token này và quản lý vòng đời của nó ở phạm vi Repository, không cần thao tác thủ công.

- `STAGING_SSH_KEY` và `PROD_SSH_KEY`: Là các khóa riêng tư SSH dùng để kết nối vào các máy chủ tương ứng. Chúng được giới hạn quyền truy cập ở phạm vi Environment (lần lượt cho môi trường staging và production).
- `STAGING_DB_URL` và `PROD_DB_URL`: Lưu trữ chuỗi kết nối (JDBC URL) đến các cụm cơ sở dữ liệu. Các thông số này cũng được cấu hình nghiêm ngặt theo từng phạm vi Environment tương ứng để loại trừ rủi ro truy cập chéo giữa các môi trường.

Lưu ý: khi cần cô lập, gắn secret với **Environment** (staging/production) thay vì ở mức Repository; khi đó secret của staging không thể dùng trong job của production và ngược lại. Đây là cách GitHub Environments đảm bảo cô lập giữa các môi trường triển khai.

12.4.7. Database Migration trong pipeline CI/CD

Trong microservices, thay đổi schema và thay đổi mã nguồn là hai artifact độc lập — nhưng phải được phối hợp theo thứ tự chính xác. Sai thứ tự dẫn đến gián đoạn dịch vụ hoặc hỏng dữ liệu.

Vấn đề cốt lõi: mã cũ đọc schema mới

Trong quá trình cập nhật cuốn chiếu, luôn tồn tại một khoảng thời gian mà **cả phiên bản cũ và mới** của service cùng hoạt động. Nếu migration tạo schema không tương thích ngược:

- Dịch vụ cũ v1 (đang xử lý request) đọc cột vừa bị đổi tên sẽ dẫn đến lỗi.
- Dịch vụ mới v2 (vừa được triển khai) đọc cột mới sẽ hoạt động bình thường.
- Kết quả: một phần request thành công, một phần lỗi — khó gỡ lỗi, khó quay lui.

Mẫu Expand/Contract — migration tương thích ngược

Giải pháp chuẩn mực cho bài toán này là chia nhỏ thay đổi schema thành chuỗi bước tương thích ngược theo mẫu Expand/Contract. Trước hết ở bước **Expand & ghi song song hai cột**, chúng ta thêm cột/bảng mới cho phép `NULL` mà không đụng tới cột cũ, đồng thời triển khai phiên bản chuyển tiếp ghi đồng bộ vào *cả cột cũ lẫn cột mới* (đường đọc dùng `COALESCE(cột_mới, cột_cũ)`). Kế đến ở bước **Bù dữ liệu lịch sử** (backfill), tác vụ nền chép dữ liệu theo từng lô nhỏ cho các bản ghi cũ còn `NULL` ở cột mới — vì bước ghi song song đã chạy trước, không có bản ghi phát sinh mới nào bị rơi rớt trong lúc backfill. Sau khi toàn bộ dữ liệu đã đồng bộ, ứng dụng chuyển sang đọc/ghi thuần cột mới và ngừng hoàn toàn phiên bản cũ; cuối cùng mới thực hiện bước **Contract (Thu hẹp)** để xóa cột cũ.

Ví dụ minh họa trong `kbm-academic`: đổi tên cột `score` thành `score_10` (thang 10):

Listing 12.5: *Trình tự Expand (kèm ghi song song) → Backfill → Switch → Contract khi đổi tên cột*

```
-- Bước 1 (Expand): thêm cột mới cho phép NULL, GIỮ cột cũ (score)
ALTER TABLE grades ADD COLUMN score_10 DECIMAL(4,2);
-- Triển khai bản chuyển tiếp v1.1: ghi đồng thời cả score và score_10;
-- đường đọc dùng COALESCE(score_10, score) (hoặc gắn DB trigger đồng bộ hai chiều).

-- Bước 2 (Backfill): KHÔNG chạy UPDATE toàn bảng đồng bộ trong migration CI/CD
-- (khóa bảng lớn sẽ treo pipeline và phá vỡ triển khai không gián đoạn).
-- Chạy bằng batch job nền theo từng lô nhỏ cho các dòng lịch sử chưa có score_10:
-- lặp theo lô: ... WHERE id IN (SELECT id FROM grades
--                               WHERE score_10 IS NULL LIMIT 1000)
UPDATE grades SET score_10 = score WHERE score_10 IS NULL;

-- Bước 3 (Switch): khi không còn dòng nào có score_10 IS NULL,
-- triển khai v2 chỉ đọc/ghi score_10 và đợi mọi instance cũ dừng hẳn.
```

```
-- Bước 4 (Contract): xóa cột cũ ở kỳ phát hành kế tiếp
ALTER TABLE grades DROP COLUMN score;
```

Thứ tự thực hiện trong pipeline

⚠ Migration trước khi triển khai

Thứ tự bắt buộc: **migration tương thích ngược → triển khai service mới → (sau khi ổn định) dọn schema cũ**. Nếu triển khai dịch vụ mới trước khi mở rộng schema, ứng dụng mới gặp schema cũ sẽ lỗi ngay khi khởi động hoặc xử lý truy vấn. Flyway và Liquibase kiểm tra checksum và phiên bản schema trước khi service nhận lưu lượng — đây là lý do bước migration trong Listing 12.4 được tách thành một bước riêng thay vì trộn vào quá trình khởi động thông thường.

Trong Listing 12.4, bước migration chạy bằng `docker compose ... run --rm --no-deps` kèm cờ `--spring.main.web-application-type=none`: container tham gia đúng mạng nội bộ để kết nối tới PostgreSQL, thực thi Flyway rồi thoát mà không mở cổng HTTP. Nếu service còn luồng nền, như Kafka listener, phải tắt chúng cho lần chạy này, hoặc dùng image Flyway riêng cho bước migration. Chỉ khi bước này trả mã thoát `0`, pipeline mới cập nhật container dịch vụ thật. Nếu tập lệnh SQL gặp lỗi, `set -Eeuo pipefail` dừng ngay pipeline để kỹ sư kiểm tra thay vì đẩy bản mã mới lên một schema dở dang.

12.5. Các chiến lược triển khai

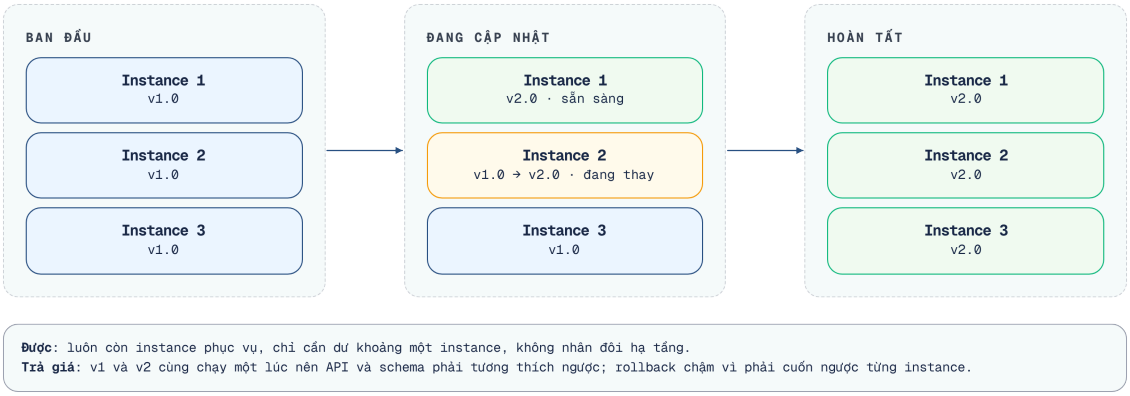
12.5.1. Vấn đề: triển khai phiên bản mới mà không gây gián đoạn

Khi triển khai phiên bản mới của một service, làm thế nào để đảm bảo người dùng không bị gián đoạn? Nếu phiên bản mới tồn tại lỗi, làm thế nào để khôi phục hệ thống nhanh chóng?

Bốn chiến lược được trình bày dưới đây: ba cách phát hành (Rolling, Blue/Green, Canary) và một kỹ thuật kiểm chứng trước khi phát hành (Shadow).

12.5.2. Rolling Update

Thay thế lần lượt **từng instance** — chuyển đổi dần dần từ phiên bản cũ sang phiên bản mới mà không yêu cầu tăng gấp đôi tài nguyên hạ tầng.



Hình 12.4: *Quá trình Rolling Update*

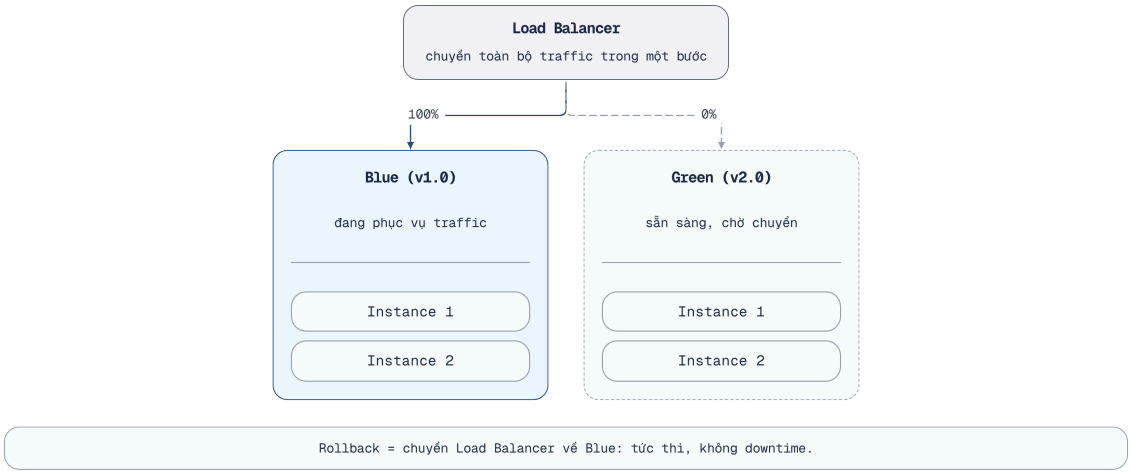
12.5.3. Blue/Green Deployment

Chạy **hai bản hoàn chỉnh** song song — "Blue" (hiện tại) và "Green" (mới). Router chuyển traffic một lần. Để rollback, chỉ cần chuyển router ngược lại; dữ liệu Green đã ghi không tự quay lui, nên Blue phải còn tương thích với schema và dữ liệu mới. Trong Kubernetes, mô hình này thường được thực hiện bằng cách thay đổi `selector` của `Service` để trỏ từ phiên bản Blue sang Green. Quá trình này diễn ra tức thời mà không cần cấu hình tỷ lệ traffic.

__Listing 12.6:__ Chuyển traffic lập tức bằng cách patch Service trong Blue/Green

```
# Triển khai phiên bản Green (đứng chờ, chưa có traffic)
kubectl apply -f deployment-green.yaml

# Khi test trên Green thành công, tiến hành switch traffic lập tức
# bằng cách patch Service
kubectl patch service kbm-academic-svc \
  -p '{"spec":{"selector":{"version":"green"}}}'

# Rollback ngay lập tức nếu có lỗi:
kubectl patch service kbm-academic-svc -p '{"spec":{"selector":{"version":"blue"}}}'
```

Việc thay đổi luồng định tuyến (routing) ở cấp độ mạng giúp hệ thống ngay lập tức phục vụ phiên bản mới mà không tốn thời gian khởi động hoặc làm gián đoạn các kết nối hiện tại.



__Hình 12.5:__ Kiến trúc Blue/Green Deployment

12.5.4. Canary Release

Triển khai phiên bản mới cho **một phần nhỏ lưu lượng truy cập**, sau đó giám sát các chỉ số đo lường (metric) và tiến hành mở rộng dần nếu hệ thống hoạt động ổn định. Trong môi trường Kubernetes sử dụng Ingress NGINX, chiến lược Canary có thể được thực hiện bằng cách cấu hình annotation điều hướng một tỷ lệ phần trăm lưu lượng (ví dụ 10%) sang phiên bản mới trong khi 90% vẫn vào phiên bản ổn định.

__Listing 12.7:__ Cấu hình Canary Ingress phân luồng 10% traffic

```
# canary-ingress.yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
```

```yaml
metadata:
  name: kbm-academic-canary-ingress
  annotations:
    nginx.ingress.kubernetes.io/canary: "true"
    # Định tuyến 10% traffic vào Canary
    nginx.ingress.kubernetes.io/canary-weight: "10"
spec:
  rules:
  - host: api.example.edu
    http:
      paths:
      - path: /academic(/|$)(.*)
        pathType: ImplementationSpecific
        backend:
          service:
            name: kbm-academic-canary-svc
            port:
              number: 80
```

Listing này dùng annotation của Ingress NGINX để minh họa cơ chế chia lưu lượng theo trọng số. Dự án Ingress NGINX đã ngừng bảo trì từ tháng 3/2026, nên hệ thống mới nên dùng Gateway API, với trọng số khai báo trong `HTTPRoute`.

12.5.5. Shadow Deployment

Gửi **một bản sao** của traffic thật sang phiên bản mới, trong khi người dùng vẫn chỉ nhận kết quả từ phiên bản cũ. Phiên bản mới xử lý request thật nhưng phản hồi của nó bị loại bỏ, nên lỗi ở phiên bản này không ảnh hưởng tới người dùng. Kỹ thuật này còn được gọi là *traffic mirroring* hoặc *dark launching*.

Điểm mạnh của kỹ thuật này là trả lời được câu hỏi mà staging không trả lời được: phiên bản mới có ổn định dưới tải *thật* hay không. Đổi lại, đây là chiến lược có chi phí cao nhất trong bốn chiến lược: phải chạy song song đủ tài nguyên cho toàn bộ traffic, và mọi **tác dụng phụ** (side effect) phải được cô lập. Nếu bản shadow vẫn ghi vào cùng database hay vẫn gửi email, hậu quả sẽ nhân đôi thay vì chỉ được mô phỏng.

Với KBM, ứng dụng hợp lý nhất là bộ chấm bài: cho `kbm-judge-sql` phiên bản mới chấm song song cùng bộ bài nộp, so kết quả với bản cũ, nhưng chỉ bản cũ mới được ghi điểm cho sinh viên.

12.5.6. So sánh

So sánh bốn chiến lược triển khai: **Rolling Update** giúp đạt zero-downtime với chi phí tài nguyên cực thấp (chỉ cần dư một instance) nhưng có nhược điểm là quá trình rollback chậm. **Blue/Green** cung cấp khả năng rollback tức thời chỉ thông qua việc điều hướng lại router, phù hợp cho các đợt phát hành rủi ro cao nhưng yêu cầu chi phí vận hành gấp đôi. **Canary Release** có cơ chế rollback tức thời và chi phí tài nguyên vừa phải, đồng thời cho phép thử nghiệm trên một nhóm nhỏ người dùng trước khi mở rộng (cơ chế chia lưu lượng giống A/B testing, nhưng mục đích là giảm rủi ro).

Shadow Deployment không chuyển người dùng sang bản mới, nên nó không phải là một cách phát hành mà là một cách *kiểm chứng trước khi* phát hành, và thường đứng trước một trong ba chiến lược kia.

Newman trong [4a, Ch.6] khuyến nghị: chọn chiến lược dựa trên **mức độ rủi ro** của thay đổi. Các bản vá lỗi nhỏ gọn nên áp dụng Rolling Update. Các thay đổi lớn có tầm ảnh hưởng diện rộng cần sử dụng Blue/Green. Những tính năng mới mang tính thử nghiệm nên được triển khai theo mô

hình Canary. Khi rủi ro nằm ở *hiệu năng dưới tải thật* chứ không ở tính đúng đắn của logic, Shadow Deployment là cách đo trước mà không bắt người dùng gánh rủi ro.

12.6. Infrastructure as Code và nền tảng điều phối triển khai

Việc cấu hình máy chủ thủ công qua SSH dễ gây ra tình trạng sai lệch cấu hình giữa các môi trường, khiến chúng không thể tái tạo được (reproducible). Infrastructure as Code (IaC) và các nền tảng điều phối container giải quyết tận gốc bài toán này.

12.6.1. Vấn đề: "Ai cấu hình máy chủ? Cấu hình nằm ở đâu?"

Khi hệ thống microservices chạy trên nhiều máy chủ và container, cấu hình thủ công dẫn đến sai lệch cấu hình nghiêm trọng — một máy chủ có thể bị thay đổi mà không được đồng bộ với máy chủ khác. Tình trạng này khiến môi trường không thể được tái tạo chính xác vì không ai nhớ rõ các bước thiết lập ban đầu. Khi xảy ra lỗi, hệ thống mất khả năng quay lui vì không lưu giữ được trạng thái cấu hình ổn định trước đó.

12.6.2. IaC — hạ tầng dưới dạng mã nguồn

Infrastructure as Code (IaC) nghĩa là mọi thành phần hạ tầng — máy chủ, mạng, cơ sở dữ liệu, message broker — đều được **định nghĩa bằng mã** (thường ở dạng khai báo YAML/HCL), được quản lý phiên bản và áp dụng tự động.

Bên cạnh pipeline CI/CD truyền thống, hệ sinh thái cloud-native mở rộng sang ba hướng bổ trợ:

- **GitOps (Argo CD, Flux)**: Lấy kho Git làm nguồn sự thật duy nhất (single source of truth) và tự động đồng bộ trạng thái hạ tầng, thay cho việc áp cấu hình thủ công.
- **Multi-cluster (Rancher, Karmada)**: Quản lý nhiều cụm ở nhiều vùng cho các tổ chức quy mô toàn cầu, đổi lại là bài toán kết nối mạng liên cụm.
- **Supply chain security**: Các khung kiểm định như SLSA và công cụ ký artifact như Sigstore, đi kèm SBOM (Software Bill of Materials) — bản kê thành phần phần mềm — đang dần trở thành yêu cầu chuẩn.

Các xu hướng này đều *bổ sung* — không thay thế — pipeline CI/CD đã trình bày trong chương này.

Mitra trong [3, Ch.6–7] mô tả IaC là nền tảng cho việc triển khai microservices thông qua một hệ sinh thái công cụ đa dạng, phục vụ cho từng tầng khác nhau của kiến trúc:

- **Tầng Provisioning (cung cấp tài nguyên)**: Bao gồm các công cụ tiêu biểu như Terraform, Pulumi hay CloudFormation. Chúng đảm nhận nhiệm vụ khởi tạo hạ tầng gốc như hệ thống máy ảo (VM), mạng nội bộ (VPC) hay cơ sở dữ liệu dạng dịch vụ được quản lý (managed database).
- **Tầng Container Orchestration (điều phối container)**: Sử dụng các mô tả dạng Kubernetes manifest hay Helm chart để định nghĩa cách thức điều phối ứng dụng, quy tắc scaling và định tuyến lưu lượng.
- **Tầng Configuration Management (quản lý cấu hình)**: Các công cụ như Ansible, Chef hay Puppet hỗ trợ cấu hình hệ điều hành cơ sở và cài đặt thư viện hệ thống mức máy chủ vật lý/ảo.
- **Tầng Service Deployment (triển khai dịch vụ)**: Những nền tảng như Docker Compose, Kubernetes hay Argo CD trực tiếp quản lý vòng đời của các dịch vụ, đảm bảo toàn bộ stack microservices luôn khớp với trạng thái đã khai báo.

Thay vì lặp lại các tệp YAML thô cho từng môi trường, các dự án Kubernetes thực tế thường dùng **Helm** làm trình quản lý gói (package manager) để tham số hóa manifest thành các khuôn mẫu (template) tái sử dụng:

Listing 12.8: Quản lý Kubernetes IaC với Helm

```
# Cài đặt hoặc nâng cấp kbm-academic service với custom values cho production
helm upgrade --install kbm-academic ./kbm-service \
  --namespace kbm-prod \
  --set image.tag=v2.0.1 \
  --set replicaCount=5 \
  --set resources.limits.cpu=1000m
```

12.6.3. Docker Compose như một dạng IaC

Docker Compose — dù gọn nhẹ — đã là một dạng IaC cơ bản: cơ sở dữ liệu, message broker và các service đều được khai báo trong tệp YAML có kiểm soát phiên bản. Nhìn theo lộ trình trưởng thành, các tổ chức thường đi qua ba cấp: **Cấp 1** dùng *Docker Compose* cho môi trường phát triển hoặc triển khai đơn máy chủ (như KBM LMS chính hiện tại); **Cấp 2** dùng *Kubernetes và Helm* khi cần điều phối đa máy chủ, tự phục hồi cấp cụm và tự động mở rộng (DevOps Lab mới áp dụng cục bộ một cụm k3s nhỏ để cô lập bài thực hành); và **Cấp 3** kết hợp *Terraform, Kubernetes và Argo CD* thành mô hình *GitOps* toàn diện.

Infrastructure as Code

Nếu không thể tái tạo toàn bộ hạ tầng từ con số không chỉ bằng mã nguồn, chúng ta chưa thực sự làm chủ hạ tầng đó — mà chỉ đang phụ thuộc vào trí nhớ của người thiết lập nó lần cuối.

— Tổng hợp nguyên tắc IaC từ Mitra [3, Ch.6]

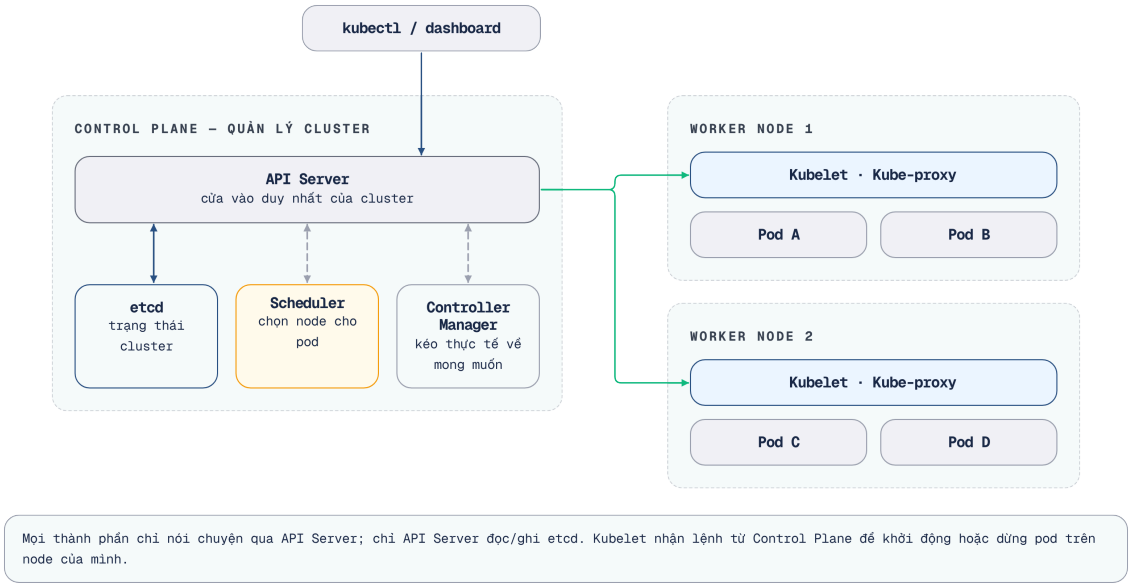
12.6.4. Kubernetes — điều phối container cho production

Phần nâng cao (có thể lướt qua)

Từ đây đến hết mục 12.6 dành cho hệ thống vượt quy mô một máy chủ. Người đọc lần đầu chỉ cần nắm khi nào nên chuyển lên Kubernetes.

Khi hệ thống vượt ngưỡng đơn máy chủ của Docker Compose và đòi hỏi **triển khai đa nút, tự động mở rộng và tự phục hồi**, Kubernetes (K8s) trở thành nền tảng tiêu chuẩn.

Kiến trúc một cụm Kubernetes gồm hai phần chính (Hình 12.6):



Hình 12.6: Kiến trúc Kubernetes — Control Plane và Worker Node

Trong đó, **Control Plane** quản lý toàn bộ cụm: API Server nhận yêu cầu (từ `kubectl` hoặc hệ thống GitOps), etcd lưu trạng thái cụm, Scheduler quyết định pod đặt lên nút nào, còn Controller Manager liên tục đối chiếu để đưa trạng thái thực tế về khớp trạng thái khai báo. Phía **Worker Node** trực tiếp chạy tải công việc (workload): tiến trình Kubelet trên mỗi nút nhận chỉ thị từ Control Plane để khởi tạo hoặc dừng các pod tương ứng.

Các khái niệm cốt lõi

- **Pod**: Đơn vị tính toán nhỏ nhất trong Kubernetes, chứa một hoặc nhiều container dùng chung không gian mạng (network namespace) và ổ lưu trữ gắn kèm.
- **Deployment**: Đối tượng quản lý vòng đời của Pod, duy trì đúng số bản sao (replica) mong muốn và điều khiển cập nhật cuốn chiếu không gián đoạn. Khả năng điều phối động theo trạng thái khai báo này là khác biệt cốt lõi so với Docker Compose.
- **Service**: Cung cấp endpoint mạng ổn định và cân bằng tải nội bộ, che đi việc địa chỉ IP của các Pod thay đổi mỗi lần tái tạo.
- **Ingress**: Bộ định tuyến HTTP/HTTPS ở rìa cụm dựa trên tên miền và đường dẫn URI, thay cho việc cấu hình thủ công từng máy chủ reverse proxy.
- **ConfigMap / Secret**: Tách cấu hình khỏi mã nguồn ứng dụng. Secret API tách dữ liệu nhạy cảm khỏi manifest và kiểm soát truy cập bằng RBAC, nhưng mặc định chỉ encode Base64, chưa phải mã hóa — muốn mã hóa thực sự trên đĩa trong etcd phải bật `encryption at rest` hoặc tích hợp trình quản lý secret bên ngoài. Ở phía Compose, tại thời điểm khảo sát (đầu năm 2026), KBM truyền secret qua các tệp đi cùng triển khai (Chương 9), dù Compose cũng hỗ trợ cơ chế `secrets` gắn vào `/run/secrets`.
- **HPA (Horizontal Pod Autoscaler)**: Tự động tăng/giảm số lượng Pod theo tải thực tế (CPU, bộ nhớ hoặc metric tùy biến).

Dưới đây là tệp YAML hoàn chỉnh cho `kbm-academic` (gồm Deployment, Service, Ingress và HPA), áp dụng cổng chuẩn `:8088` cùng bộ ba *Startup/Liveness/Readiness Probe* và giới hạn tài nguyên (*Resource Limits*):

Listing 12.9: *Cấu hình Kubernetes chuẩn mực cho* `kbm-academic`

```
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: kbm-academic
spec:
  # Không khai báo replicas khi đã có HPA bên dưới: mỗi lần apply/GitOps sync
  # sẽ ép số Pod về giá trị cứng này rồi HPA lại kéo về, gây flapping.
  # replicas do HPA sở hữu.
  selector:
    matchLabels:
      app: kbm-academic
  template:
    metadata:
      labels:
        app: kbm-academic
    spec:
      containers:
      - name: kbm-academic
        image: ghcr.io/kbm/kbm-academic:v1.2.0
        ports:
        - containerPort: 8088
        resources:
          requests:
            cpu: "200m"
            memory: "512Mi"
          limits:
            cpu: "500m"
            memory: "1Gi"
        # Bảo vệ JVM trong lúc khởi động nguội (tối đa 12 * 5s = 60s)
        # trước khi bật livenessProbe
        startupProbe:
          httpGet:
            path: /actuator/health/liveness
            port: 8088
          periodSeconds: 5
          failureThreshold: 12
        livenessProbe:
          httpGet:
            path: /actuator/health/liveness
            port: 8088
          periodSeconds: 10
        # Không có readinessProbe thì kubelet đánh dấu Pod Ready ngay khi container
        # start; Service route traffic vào JVM chưa khởi tạo xong → 502 ở mỗi lần
        # rolling update. Đây chính là cơ chế tạo ra zero-downtime (xem §11.6).
        readinessProbe:
          httpGet:
            path: /actuator/health/readiness
            port: 8088
          periodSeconds: 5
          failureThreshold: 3
---
apiVersion: v1
kind: Service
metadata:
  name: kbm-academic-svc
spec:
```

```
  selector:
    app: kbm-academic
  ports:
    - protocol: TCP
      port: 80
      targetPort: 8088
---
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: kbm-academic-ingress
  annotations:
    nginx.ingress.kubernetes.io/use-regex: "true"
    # /$2 giữ lại phần URI sau prefix — rewrite-target: / sẽ cắt cụt mọi path con
    nginx.ingress.kubernetes.io/rewrite-target: /$2
spec:
  rules:
  - host: api.example.edu
    http:
      paths:
      - path: /academic(/|$)(.*)
        pathType: ImplementationSpecific
        backend:
          service:
            name: kbm-academic-svc
            port:
              number: 80
---
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: kbm-academic-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: kbm-academic
  minReplicas: 2
  maxReplicas: 10
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 70
```

So sánh chi tiết Docker Compose và Kubernetes

Bảng 12.1 so sánh năng lực của hai nền tảng.

__Bảng 12.1:__ So sánh chi tiết Docker Compose và Kubernetes

Năng lực	Docker Compose	Kubernetes
Multi-host	× Chỉ một máy chủ	✓ Cluster nhiều node
Auto-scaling	× Thủ công `--scale`	✓ HPA dựa trên metric
Self-healing	! `restart: unless-stopped` (cơ bản)	✓ Liveness/readiness probe, tự thay thế
Rolling update	× Gián đoạn khi khởi động lại	✓ Zero-downtime rolling update

Năng lực	Docker Compose	Kubernetes
Service discovery	✓ DNS theo tên service	✓ DNS + load balancing
Secret management	! Có `secrets` gắn vào `/run/secrets`; KBM tại thời điểm khảo sát dùng `.env` và tệp cấu hình	✓ Secret API + RBAC (base64; bật encryption-at-rest để mã hóa thật)
Giới hạn tài nguyên	! Cơ bản (`deploy.resources`)	✓ Request + limit theo từng container
Độ phức tạp thiết lập	Thấp (1 tệp YAML)	Cao (nhiều tệp YAML, thiết lập cluster)
Thời gian học	1–2 ngày	2–4 tuần
Quy mô đội ngũ cần thiết	1–3 người	3–5+ người (hoặc managed K8s)

Kịch bản chuyển đổi của KBM: từ Compose sang Kubernetes

Nếu KBM cần mở rộng hơn (nhiều đơn vị đào tạo, nhiều sinh viên đồng thời):

__Listing 12.10:__ Kịch bản mở rộng KBM từ Compose lên Kubernetes

```
Bước 1 (Hiện tại): Docker Compose cho LMS chính
├── Phù hợp hạ tầng nhỏ và workload vừa phải
├── Triển khai: docker compose up -d
└── Chi phí vận hành thấp, thao tác đơn giản

Bước 2 (Mở rộng): Managed Kubernetes (GKE/EKS/AKS)
├── Nhiều đơn vị đào tạo hoặc lưu lượng kỳ thi lớn
├── kbm-judge-sql: HPA mở rộng 1→10 pod trong kỳ thi
├── Core Service: 2-3 replica cho độ sẵn sàng cao
├── PostgreSQL: managed service (Cloud SQL/RDS)
└── Chi phí và độ phức tạp tăng đáng kể
```

Khi nào chuyển lên Kubernetes?

Không chuyển lên K8s vì "Netflix dùng". Chuyển khi có **ít nhất 2 tiêu chí**: (1) cần triển khai đa nút hoặc cô lập workload tốt hơn, (2) cần auto-scaling (tải biến động: kỳ thi → bình thường), (3) cần triển khai zero-downtime (SLA yêu cầu), (4) đội ngũ đủ năng lực vận hành K8s. Managed Kubernetes (GKE, EKS, AKS) giảm đáng kể độ phức tạp — không cần tự dựng/bảo trì control plane.

12.6.5. Triển khai serverless — khi nào phù hợp?

Scale-to-Zero trong Kubernetes

Bên cạnh các dịch vụ serverless thuần túy từ nhà cung cấp đám mây, trên môi trường Kubernetes, xu hướng hiện nay là dùng **KEDA** (Kubernetes Event-driven Autoscaling) hoặc **Knative** để mang lại trải nghiệm tương tự serverless: cho phép giảm số pod xuống 0 khi không có tải (Scale-to-Zero), giúp tiết kiệm đáng kể chi phí cho các dịch vụ chạy nền.

Richardson trong [2a, Ch.12] liệt kê **Serverless** (AWS Lambda, Google Cloud Functions, Azure Functions) là một mô hình triển khai cho microservices. Thay vì quản lý container, chúng ta triển khai các *function* — nhà cung cấp đám mây quản lý hạ tầng, tự động mở rộng và tính phí theo lượt gọi.

Bảng 12.2: So sánh container (Docker/K8s) và serverless

Khía cạnh	Container (Docker/K8s)	Serverless
Quản lý máy chủ	Tự quản lý (hoặc managed K8s)	Nhà cung cấp đám mây quản lý hoàn toàn
Mở rộng	Cấu hình quy tắc auto-scaling	Tự động mở rộng (0 → N)
Mô hình chi phí	Trả theo máy chủ/giờ (dù có chạy hay không)	Trả theo lượt gọi (không dùng = không trả)
Cold start	Thường không (replica đã chạy sẵn); vẫn có khi khởi tạo hoặc scale-out	Có (100 ms–3 s khởi động lần đầu)
Stateful	Có thể (volume, session)	Chỉ stateless
Giới hạn thời gian chạy	Không giới hạn	Thường tối đa 15 phút mỗi lượt gọi
Phụ thuộc nhà cung cấp (vendor lock-in)	Thấp (Docker dễ di chuyển)	Cao (API riêng mỗi đám mây)

Khi nào serverless phù hợp cho microservices?

- **Khối lượng công việc mang tính đột biến và hướng sự kiện (event-driven, bursty workload)**: xử lý tải lên tệp, thay đổi kích thước ảnh, gửi thông báo — không yêu cầu máy chủ duy trì hoạt động liên tục
- **Hàm kết nối (glue function)**: kết nối các service, biến đổi dữ liệu, kích hoạt luồng xử lý
- **Làm bản mẫu (prototype/MVP)**: triển khai nhanh chóng mà không đòi hỏi thiết lập hạ tầng phức tạp

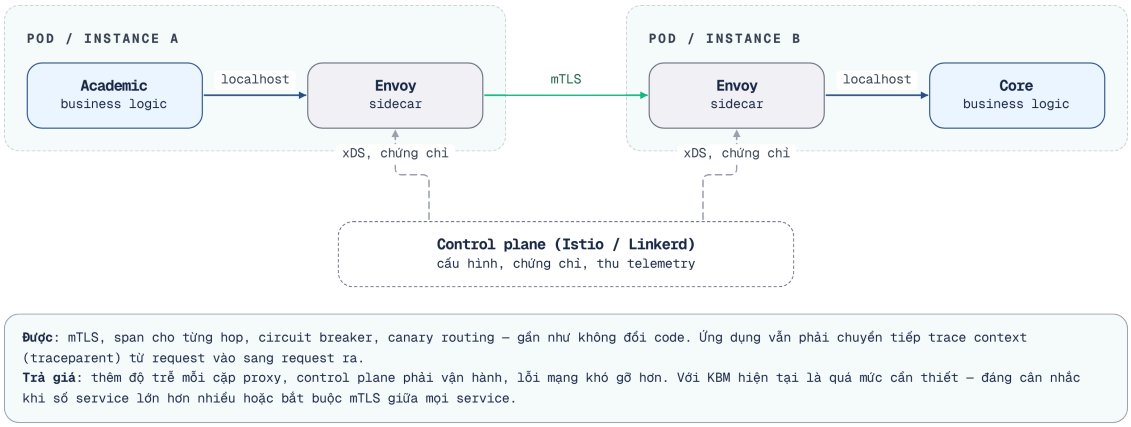
Khi nào không phù hợp?

- **Các luồng xử lý thiết yếu yêu cầu độ trễ thấp**: thời gian khởi động nguội không phù hợp cho các giao tiếp API đồng bộ (luồng nộp bài của LMS)
- **Tiến trình dài hoặc giữ tài nguyên cố định**: `kbm-judge-sql` giữ kết nối tới các sandbox DBMS và mỗi bài chạy 5–30 giây, cộng hàng đợi dồn trong kỳ thi — không hợp mô hình hàm ngắn hạn tính tiền theo lượt gọi
- **Các stateful service**: yêu cầu duy trì kết nối cơ sở dữ liệu hoặc bộ nhớ đệm, trong khi function chỉ tái sử dụng được kết nối lúc môi trường thực thi còn ấm, nên phải tự xử lý kết nối lại, concurrency và giới hạn số kết nối của database

Trong LMS, phần **gửi email/push** khi có sự kiện là ứng viên tốt cho serverless: hướng sự kiện (nhận sự kiện từ Kafka), đột biến theo kỳ thi, không giữ trạng thái. Kênh WebSocket của Notification Service thì không — nó giữ kết nối lâu với trình duyệt. `kbm-core`, `kbm-judge-sql` và `kbm-gateway` phù hợp với container hơn.

12.6.6. Sidecar Pattern và Service Mesh

Khi hệ thống microservices lớn (20+ service), mỗi service cần tự triển khai cùng các nghiệp vụ cắt ngang (cross-cutting concern): mTLS, logging, tracing, circuit breaker, rate limiting. **Sidecar Pattern** giải quyết bằng cách đặt một **tiến trình proxy bên cạnh mỗi instance của service** — proxy xử lý các nghiệp vụ cắt ngang, service chỉ tập trung vào business logic.



Hình 12.7: Sidecar Pattern — proxy xử lý các nghiệp vụ cắt ngang (mTLS, tracing)

Service Mesh (Istio, Linkerd) gồm sidecar proxy trên mọi service và control plane quản lý tập trung. Nó tự động cung cấp mTLS giữa các service, distributed tracing, traffic management (canary routing) và circuit breaking, **gần như không cần thay đổi code** (ngoại lệ quan trọng của tracing được nói rõ bên dưới). Xu hướng gần đây là **sidecarless / ambient mesh** (Istio Ambient, Cilium Service Mesh dựa trên eBPF): mTLS và định tuyến L4 được xử lý ở tầng node/kernel thay vì chạy một proxy riêng cho mỗi pod, nhờ đó cắt bớt chi phí phụ trội (overhead) trên mỗi hop và chi phí vận hành sidecar — đáng cân nhắc nếu đội ngũ đánh giá service mesh cho hệ thống mới.

So với việc cấu hình riêng lẻ trong từng ứng dụng, Service Mesh tập trung hóa các bài toán hạ tầng mạng:

- **Bảo mật luồng giao tiếp (mTLS)**: Thay vì buộc các lập trình viên phải tự triển khai logic mã hóa và quản lý chứng chỉ, các sidecar proxy của Service Mesh tự động thực hiện handshake và thiết lập kết nối mTLS giữa các microservice, trong suốt với ứng dụng.
- **Truy vết phân tán (Tracing)**: Sidecar tự sinh span cho từng hop mạng và export về backend, giảm đáng kể phần hạ tầng phải tự dựng. Tuy nhiên, ứng dụng vẫn **bắt buộc** phải chuyển tiếp trace context (`traceparent` hoặc B3 header) từ request đi vào sang request gửi ra. Proxy không thể tự suy ra outbound request nào bắt nguồn từ inbound request nào; thiếu bước này, các span sẽ không ghép được thành một trace hoàn chỉnh.
- **Cơ chế tự bảo vệ (Circuit Breaker)**: Thay vì phải viết các cấu hình ngắt mạch qua Resilience4j lồng ghép vào business logic, mọi quy tắc đều được khai báo tập trung ở lớp Envoy proxy, qua đó loại bỏ logic xử lý lỗi kết nối khỏi ứng dụng.
- **Định tuyến nâng cao (Canary routing)**: Quá trình phân luồng lưu lượng được thực hiện thông qua các tệp cấu hình khai báo (declarative), thay cho thao tác thủ công trên Load Balancer.
- **Đánh đổi về hiệu năng (Overhead)**: Mặc dù giảm độ phức tạp trong mã nguồn, Service Mesh làm tăng độ trễ mạng do request phải đi qua các lớp proxy. Benchmark chính thức của Istio ghi nhận khoảng 2–3 ms ở P90 cho một cặp proxy; con số thực tế phụ thuộc cấu hình telemetry và tải, nên phải đo trên chính hệ thống của mình.

Với KBM hiện tại, việc áp dụng Service Mesh vẫn mang tính thiết kế quá mức (over-engineering) cho hệ thống LMS chính. Mặc dù việc có thêm DevOps Lab khiến hệ thống trở thành đa ngôn ngữ (polyglot Java+Go), nhu cầu thiết yếu trước mắt vẫn là hoàn thiện CI/CD, tính khả quan sát (observability) và quản lý cấu hình bảo mật. Service Mesh phù hợp khi số lượng service lớn hơn

nhiều, triển khai nhiều máy chủ trở thành mặc định, hoặc yêu cầu bảo mật cao đến mức mTLS bắt buộc giữa mọi service.

⚠ Độ phức tạp khi vận hành và xử lý sự cố

Service Mesh giải quyết nhiều nghiệp vụ cắt ngang, nhưng đổi lại là độ phức tạp vận hành lớn. Vận hành Control Plane (như Istio) và gỡ lỗi mạng ở tầng proxy/eBPF gây khó khăn đáng kể cho việc bảo trì nếu đội ngũ không có chuyên gia mạng. Việc đưa Service Mesh vào khi chưa thực sự cần thiết tiềm ẩn rủi ro gián đoạn dịch vụ.

12.7. Case study: KBM

Phần này đối chiếu các nguyên lý tự động hóa và triển khai với hiện trạng hệ thống KBM, từ đó đề xuất lộ trình nâng cấp phù hợp với quy mô vận hành thực tế.

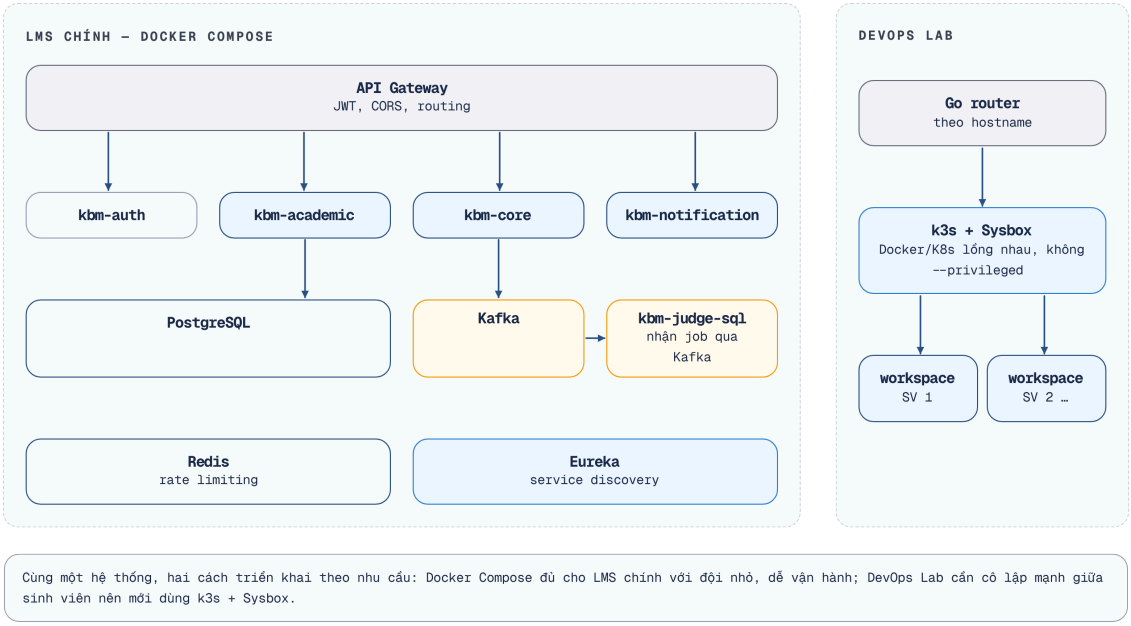
12.7.1. Hiện trạng

KBM hiện có hai kiểu triển khai song song ở mức khái quát:

1. **LMS chính**: các service Java/Spring Boot, database, Kafka, Gateway và frontend triển khai bằng Docker Compose trên hạ tầng nhỏ. Đây là lựa chọn thực dụng cho đội ngũ phát triển quy mô nhỏ, đảm bảo tính dễ vận hành mà không đòi hỏi thiết lập cụm (cluster) phức tạp.
2. **DevOps Lab**: phần thực hành Docker/Kubernetes dùng Go, k3s và Sysbox để tạo môi trường lab cô lập. Đây là nhu cầu đặc thù của miền nghiệp vụ: sinh viên cần chạy container/lab riêng trong môi trường an toàn.

Điều đáng chú ý là **sự đánh đổi trong triển khai**: cùng một hệ thống có thể dùng Docker Compose cho phần LMS chính và k3s/Sysbox cho Bounded Context cần cô lập.

Bài toán ở DevOps Lab là sinh viên cần chạy Docker/Kubernetes *bên trong* container thực hành. Cách làm phổ biến — Docker-in-Docker (DinD) — đòi hỏi cấp cờ `--privileged`, tức là mở gần như toàn quyền của host cho container, một rủi ro không chấp nhận được trong môi trường nhiều người dùng. Sysbox giải quyết đúng chỗ đánh đổi này: nó cho phép chạy Docker/K8s lồng nhau mà không cần đặc quyền, nhờ tận dụng user namespace và ảo hóa một phần `procfs`/`sysfs`.



Hình 12.8: Kiến trúc triển khai của KBM ở mức khái quát

12.7.2. Phân tích theo mức độ trưởng thành triển khai

Bảng 12.3 đối chiếu hiện trạng triển khai của KBM theo từng khía cạnh kỹ thuật để xác định các điểm nghẽn cần ưu tiên cải thiện.

Bảng 12.3: Phân tích mức độ trưởng thành triển khai của KBM

Khía cạnh	Hiện trạng	Mức trưởng thành	Nhận xét
Containerization	Docker image cho tất cả service	● Tốt	Multi-stage build, container registry (GitLab)
Orchestration	Docker Compose cho LMS chính; k3s/Sysbox cho DevOps Lab	● Phù hợp theo ngữ cảnh	Không ép một nền tảng cho mọi phần
CI/CD	GitLab CI đã phủ các phần chính; một số phần còn build + push thủ công	● Chưa đồng đều	Phần triển khai thủ công còn lại vẫn tạo rủi ro lệch phiên bản (version drift)
Chiến lược triển khai	All-at-once (stop → deploy → start)	● Có downtime	Sinh viên bị gián đoạn khi triển khai
IaC	Các tệp Docker Compose được quản lý phiên bản	● IaC cơ bản	Tái tạo được nhưng thiếu tự động hóa
Quản lý cấu hình	`application.yml` trong container	● Mới tách một phần	Một số cấu hình còn gắn cứng, chưa tách hoàn toàn ra ngoài
Cô lập lab	k3s + Sysbox cho các pod thực hành	● Đúng bài toán	Cần resource quota, RBAC, NetworkPolicy rõ ràng

12.7.3. Phân tích bối cảnh nghiệp vụ

KBM phục vụ sinh viên, do đó khung thời gian triển khai (**deployment window**) đóng vai trò then chốt:

Khi đã có hạ tầng cho nhiều chiến lược (Giai đoạn 3 bên dưới), có thể chọn theo thời điểm. Ngoài cửa sổ đóng băng thay đổi của kỳ thi, các đợt phát hành cần rollback nhanh có thể chọn **Blue/**

Green trong cửa sổ đó chỉ xử lý thay đổi khẩn cấp theo quy trình đã định. Trong những ngày **giữa tuần**, tải trọng trung bình cho phép áp dụng **Rolling Update** nếu đã có orchestrator. Các đợt bảo trì lớn vào kỳ **nghỉ hè** thường không chịu áp lực downtime, nên cách triển khai toàn bộ cùng lúc (**All-at-once**) là hoàn toàn chấp nhận được. Cuối cùng, với những bản vá khẩn cấp (**hotfix**), nên đi qua một đợt **Canary** ngắn (nếu hạ tầng cho phép) để thử nghiệm thận trọng trên một nhóm nhỏ trước khi triển khai trên toàn hệ thống.

CI/CD chưa đồng đều giữa các phân hệ

KBM đã container hóa và vận hành GitLab CI cho các phần chính (§12.4), nhưng một số phần vẫn **triển khai thủ công**: lập trình viên build image trên máy cá nhân, đẩy lên registry, sau đó truy cập từ xa (SSH) vào hạ tầng và thực thi lệnh triển khai. Ở những phần này, kiểm thử và chuyển giao chưa được tự động hóa.

Thực trạng này khiến kiểm thử tự động trước khi triển khai chưa phủ đều, cơ chế rollback chưa nhất quán, và mức độ CI/CD giữa LMS chính với DevOps Lab chênh lệch. Hệ quả là khi có lỗi trên production: (1) lập trình viên phát hiện qua báo cáo của người dùng, (2) quy trình sửa mã nguồn, đóng gói, đẩy lên registry và triển khai lại được thực hiện hoàn toàn thủ công, (3) nếu sửa sai, phải lặp lại toàn bộ quy trình đó. Các yếu tố này làm tăng MTTR (Mean Time To Recovery) và gây ra rủi ro triển khai nhầm phiên bản.

Lộ trình cải thiện có thể chia thành bốn giai đoạn theo thứ tự ưu tiên về chi phí và tác động:

- **Giai đoạn 1 — Phủ đều CI/CD (nỗ lực thấp, tác động cao)**: Mở rộng quy trình GitLab CI hiện có sang toàn bộ service, tự động chạy kiểm thử và build Docker image sau mỗi lần đẩy mã ở mọi phân hệ. Mục tiêu là không còn đoạn mã nào chưa qua kiểm thử lên được production, và mọi phiên bản đều truy vết được.
- **Giai đoạn 2 — Triển khai tự động (nỗ lực trung bình)**: Thiết lập triển khai tự động lên staging, kèm smoke test và một bước xét duyệt thủ công trước khi lên production. Kèm health check cho mỗi dịch vụ, việc triển khai chỉ cần một nút bấm, lịch sử kiểm toán được lưu lại và không ai phải SSH vào máy chủ.
- **Giai đoạn 3 — Triển khai không gián đoạn (zero-downtime) (nỗ lực trung bình–cao)**: Để sinh viên không gặp gián đoạn khi hệ thống cập nhật, dùng Blue/Green sau reverse proxy (hai bộ container, chuyển upstream), vì Compose không có rolling update đúng nghĩa; rolling update chỉ có sau khi chuyển sang orchestrator ở Giai đoạn 4.
- **Giai đoạn 4 — Chuyển sang Kubernetes (nỗ lực cao)**: Chỉ nên thực hiện khi nhu cầu tải thực sự vượt quá khả năng của Docker Compose. Kubernetes mang lại khả năng tự phục hồi, nhưng áp dụng khi chưa cần thiết sẽ phải gánh một khối lượng vận hành không tương xứng.

Rủi ro trong quá trình triển khai

Việc triển khai microservices phức tạp hơn đáng kể so với monolith. Một quyết định sai lầm trong quá trình này có thể làm đình trệ toàn bộ hệ thống hoặc gây ra gián đoạn kéo dài.

1. **Triển khai vào chiều thứ Sáu**: Khi đội ngũ phát hành tính năng mới vào cuối tuần, các lỗi phát sinh thường không được phát hiện do thiếu nhân sự trực. Hậu quả là thời gian gián đoạn có thể kéo dài đến đầu tuần sau. *Phòng tránh*: ưu tiên triển khai vào các buổi sáng giữa tuần khi đội ngũ sẵn sàng xử lý sự cố. Với hệ thống KBM, cần tuyệt đối tránh triển khai trước hoặc trong kỳ thi và buổi thực hành.

2. **Dùng Kubernetes cho hệ thống 3 service**: Tâm lý "Netflix dùng Kubernetes, chúng ta cũng nên" sẽ dẫn đến gánh nặng quản trị phức tạp (cluster management, RBAC, networking) cho một đội ngũ nhỏ. *Phòng tránh*: Docker Compose là đủ cho các hệ thống nhỏ. Việc chuyển đổi sang Kubernetes chỉ thực sự cần thiết khi có nhu cầu chạy trên nhiều máy chủ, yêu cầu mức độ cô lập cao hoặc tự động mở rộng.
3. **Build image khác nhau cho mỗi môi trường**: Việc build riêng cho môi trường phát triển, staging và production sẽ tạo ra các artifact khác nhau. Hậu quả là ứng dụng hoạt động tốt trên staging nhưng lại gặp lỗi trên production. *Phòng tránh*: build một lần, triển khai mọi nơi — đưa toàn bộ cấu hình phụ thuộc môi trường ra ngoài qua biến môi trường.
4. **Thiếu kế hoạch rollback**: Khi phiên bản mới gặp lỗi nhưng không có quy trình rollback sẵn sàng, việc can thiệp thủ công trong tình huống khẩn cấp dễ gây ra lỗi phát sinh nghiêm trọng. *Phòng tránh*: mọi quy trình triển khai phải đi kèm tài liệu hướng dẫn rollback, quy định rõ ràng các bước kỹ thuật để đưa hệ thống về trạng thái ổn định.
5. **Shared database giữa các service**: Chương 7 đã phân tích khía cạnh dữ liệu; ở khía cạnh triển khai, nếu Service A và B dùng chung database, bước cập nhật lược đồ (schema) của A có thể làm hỏng B ngay lúc phát hành. *Phòng tránh*: mỗi service sở hữu database schema riêng để cập nhật schema độc lập.

Các pattern trong chương có minh họa tương tác đi kèm tại companion web:

Interactive Demo

- **Các chiến lược triển khai (Blue/Green, Canary)**: deployment-strategies.html
- **Cấu hình bên ngoài (Externalized Config)**: config-server.html

12.8. Tổng kết

Triển khai microservices phức tạp hơn monolith do có nhiều dịch vụ cần được build, kiểm thử, triển khai và khôi phục độc lập nhưng vẫn phải đảm bảo tính tương thích chéo. **Containerization** (Docker) giải quyết vấn đề "works on my machine" — đóng gói service và các thư viện phụ thuộc thành một image có thể chạy ở mọi nơi. **Docker Compose** cho phép điều phối nhiều service trên hạ tầng nhỏ — phù hợp cho đội ngũ quy mô nhỏ và vừa cùng môi trường phát triển/staging.

Pipeline CI/CD là nền tảng cốt lõi: code push → auto build → auto test → auto deploy. Thiếu CI/CD, việc triển khai microservices trở thành một chuỗi thao tác thủ công: chậm, nhiều rủi ro và không tái tạo được (reproducible). Ba chiến lược triển khai chính — Rolling, Blue/Green, Canary — cần được lựa chọn theo mức độ rủi ro: các bản vá lỗi nhỏ áp dụng Rolling Update, thay đổi cấu trúc lớn sử dụng Blue/Green, trong khi các tính năng mới nên được thử nghiệm qua Canary. Shadow là kỹ thuật kiểm chứng trước khi phát hành, dùng khi rủi ro nằm ở hiệu năng dưới tải thật.

Infrastructure as Code đảm bảo hạ tầng tái tạo được và được quản lý phiên bản — tệp Docker Compose đã là IaC cơ bản. Khi mở rộng, Terraform + Kubernetes + Argo CD tạo thành một pipeline GitOps hoàn chỉnh — nhưng cần tránh việc thiết kế quá mức (over-engineering): hãy chọn mức điều phối (orchestration) theo từng Bounded Context.

Phân tích KBM cho thấy khâu container hóa đã tốt (Docker image, registry) và DevOps Lab đã có nhu cầu k3s/Sysbox riêng, nhưng CI/CD chưa phủ đều và mỗi lần triển khai vẫn gây gián đoạn

(all-at-once). Lộ trình cải thiện: CI/CD cơ bản → triển khai tự động → chiến lược zero-downtime → chuyển sang Kubernetes khi thật sự cần.

Qua 12 chương, chúng ta đã đi từ nền tảng SOA/Microservices (Chương 1–2), qua các mẫu giao tiếp (Chương 3–6), quản lý dữ liệu (Chương 7), hạ tầng (Chương 8–9), chuyển đổi thực tế (Chương 10), observability (Chương 11), đến triển khai (Chương 12). Chuyển đổi từ monolith sang microservices không phải là một lần "Big Bang" mà là **chuỗi quyết định nhỏ, mỗi quyết định mang lại giá trị thực tiễn**, kèm theo hiểu biết về sự đánh đổi (trade-off) của từng mẫu thiết kế.

12.9. Đọc thêm

Sách tham khảo chính:

1. [2a] Chris Richardson, *Microservices Patterns* 1st Ed. — Ch.12: Deploying Microservices — deployment patterns (language-specific, VM, container, serverless)
2. [4a] Sam Newman, *Building Microservices* — Ch.6: Deployment — CI, build pipelines, CD, Docker, service-to-host mapping và nền tảng điều phối container
3. [3] Ronnie Mitra & Irakli Nadareishvili, *Microservices: Up and Running* — Ch.6–7: Infrastructure Pipeline & Infrastructure — IaC, Terraform, Kubernetes; Ch.10: Releasing — Docker, Helm, Argo CD; Ch.11: Managing Change — deployment patterns

Sách bổ trợ:

4. [2b] Chris Richardson, *Microservices Patterns* 2nd Ed. — Ch.18–19: Deploying Microservices & on Kubernetes (updated)
5. [4b] Sam Newman, *Monolith to Microservices* — Ch.3: Splitting the Monolith — deployment considerations during migration

Nguồn trực tuyến:

- Docker official docs — docs.docker.com
- Docker Compose documentation — docs.docker.com/compose
- Kubernetes official docs — kubernetes.io/docs
- Martin Fowler, "Continuous Delivery" — martinfowler.com/bliki/ContinuousDelivery.html
- Martin Fowler, "BlueGreenDeployment" — martinfowler.com/bliki/BlueGreenDeployment.html
- Terraform by HashiCorp — terraform.io
- Argo CD — argo-cd.readthedocs.io (GitOps for Kubernetes)

Bài tập: 12-0, 12-1, 12-2 ở phần Bài tập & case study cuối sách.

Kết luận

> Kiến trúc không phải đích đến, mà là hành trình liên tục của những quyết định có cân nhắc và những đánh đổi có ý thức.
> — Tổng hợp từ Richardson [2a] và Newman [4a]

Sau 12 chương, chúng ta đã đi qua toàn bộ quá trình chuyển đổi của KBM LMS — từ ba monolith ban đầu đến kiến trúc microservices đang tiếp tục hoàn thiện. Chương kết này không giới thiệu pattern mới, mà tổng hợp các quyết định đã phân tích thành một **lộ trình thực tế có thứ tự ưu tiên**.

Tổng quan quá trình chuyển đổi

Nhìn lại, KBM đã áp dụng các nguyên lý của kiến trúc hướng dịch vụ (SOA) như tính tự chủ (Autonomy) và hợp đồng dịch vụ chuẩn hóa (Standardized Service Contract), nhưng tránh được điểm yếu của SOA truyền thống. Thay vì dùng Enterprise Service Bus (ESB) nặng nề, hệ thống đi theo nguyên tắc "smart endpoints, dumb pipes" của microservices (sử dụng API Gateway và Kafka).

Kiến trúc phần mềm giống như việc quy hoạch một khuôn viên trường đại học. Ban đầu, mọi thứ (phòng học, thư viện, giáo vụ) có thể xếp chung trong một tòa nhà đa năng (monolith). Nhưng khi quy mô tăng lên, chúng ta phải xây tách rời thành khu giảng đường, thư viện, ký túc xá (microservices), và cần quy hoạch hệ thống đường nội bộ, biển chỉ dẫn (API Gateway/Event Bus) để liên kết chúng.

Mỗi chương giải quyết một vấn đề cụ thể của KBM trong quá trình "quy hoạch" đó:

Bảng K.1: *Quá trình chuyển đổi của KBM qua 12 chương — vấn đề và giải pháp*

Chương	Vấn đề KBM gặp phải	Giải pháp (bài học của chương)
1	Ba monolith rời rạc: logic trùng lặp, danh tính không liên thông, chi phí bảo trì nhân lên	Quyết định chuyển đổi có cơ sở (khung ra quyết định)
2	Không rõ ranh giới service, thiết kế theo hướng thực thể (entity-driven design)	DDD + Event Storming → 7 Bounded Context core
3	API không nhất quán, breaking change thường xuyên	REST + OpenAPI + chiến lược versioning
4	Một service lỗi gây hiệu ứng dây chuyền (cascading failure)	Circuit Breaker + retry + timeout (Resilience4j)
5	Chế độ thi: hàng trăm lượt nộp mỗi phút, cần giữ thứ tự (ordering) và phát lại (replay)	Kafka event streaming + idempotent consumer

Chương	Vấn đề KBM gặp phải	Giải pháp (bài học của chương)
6	Luồng nộp bài treo ở PENDING khi `kbm-judge-sql` dừng đột ngột, cần tính nguyên tử (atomicity)	Choreography-based Saga + compensation + semantic lock + timeout
7	Database dùng chung giữa Core và Academic; bảng xếp hạng đọc nhiều	Database-per-service + CQRS cho bảng xếp hạng
8	Mỗi client, mỗi service tự kiểm tra danh tính — trùng lặp, dễ lệch chính sách	API Gateway tập trung: routing, xác minh JWT ở biên, giới hạn lưu lượng
9	Tại thời điểm khảo sát (đầu năm 2026): JWT HS256 dùng shared secret, token tồn tại quá lâu	`kbm-auth`: HS256 → RS256 + luân phiên refresh token
10	Không thể viết lại toàn bộ một lần (Big Bang rewrite) — hệ thống đang vận hành	Strangler Fig — tách dần từng service, mang lại giá trị tức thì
11	Lỗi xảy ra nhưng không biết tại đâu, tại sao	Structured logging + distributed tracing + cảnh báo theo SLO
12	Triển khai thủ công N service vừa rủi ro vừa chậm	Docker Compose + CI/CD tự động (GitLab CI) + kế hoạch rollback

Lộ trình chuyển đổi thực tế

Biết các pattern là chưa đủ; còn phải áp dụng đúng **thứ tự**. Không phải tất cả pattern đều có giá trị như nhau ở từng giai đoạn. Nguyên tắc xuyên suốt: **các cải tiến nhanh (Quick Wins) trước, thay đổi rủi ro cao sau** — mỗi giai đoạn phải mang lại giá trị ngay, không cần đợi hoàn thành toàn bộ.

Lộ trình chi tiết cho KBM — bốn giai đoạn *các cải tiến nhanh (Quick Wins) → nền tảng Observability → Resilience & CI/CD → phân tách cơ sở dữ liệu*, kèm bảng hành động theo tác động × nỗ lực và các sai lầm thường gặp — đã được trình bày đầy đủ ở **Chương 10** (§10.5, *Lộ trình chuyển đổi — 4 giai đoạn* sai lầm thường gặp ở §10.6). Điểm mấu chốt của thứ tự này: phải *nhìn thấy* được hệ thống (observability) và đảm bảo khả năng chịu lỗi (resilience) *trước khi* tách database — bước rủi ro nhất.

Giá trị trước, pattern sau

Không nên áp dụng CQRS chỉ vì nó hấp dẫn về mặt kỹ thuật. Hãy áp dụng khi truy vấn bảng xếp hạng làm chậm database dùng chung của Core và Academic. Mọi pattern trong sách này đều nhằm giải quyết một vấn đề cụ thể, không phải để bản vẽ kiến trúc trông có vẻ hoàn chỉnh.

Khi nào áp dụng — heuristic thực tế

Không phải hệ thống nào cũng cần mọi pattern trong 12 chương. Bảng dưới đây tóm tắt các tín hiệu thực tế (heuristic) để ra quyết định:

Bảng K.2: *Heuristic — khi nào áp dụng pattern nào*

Tín hiệu	Pattern cần xem xét	Chương tham khảo
Nhiều nhóm phải deploy chung và chờ nhau (Ch.1 §1.6, ma trận Ch.10)	Tách service theo Bounded Context	Ch.1, 10
Thời gian phản hồi tăng khi tải tăng đột biến	Circuit Breaker + tách rời bất đồng bộ	Ch.4, 5
Nhiều service cần cùng dữ liệu (JOIN xuyên service)	API Composition / CQRS	Ch.7
Giao dịch xuyên 2+ service	Saga + compensation	Ch.6
Không biết lỗi xảy ra ở đâu	Distributed tracing (`traceId`) + `requestId` / `correlationId`	Ch.11
Triển khai mất hơn 30 phút hoặc cần nhiều người	Tự động hóa CI/CD	Ch.12

Tín hiệu	Pattern cần xem xét	Chương tham khảo
Token bị lộ nhưng không thể thu hồi	JWT thời hạn ngắn + luân phiên refresh token	Ch.9

Điều không thay đổi

Công cụ thay đổi — Spring Boot hôm nay có thể khác với Spring Boot năm năm sau. Nhưng các nguyên tắc cốt lõi được học trong sách này không phụ thuộc vào framework cụ thể:

- **Kiến trúc hướng dịch vụ (SOA) là nền tảng**: dù công nghệ thay đổi (từ SOAP sang REST/gRPC, từ ESB sang Kafka), triết lý định nghĩa ranh giới dịch vụ và hợp đồng dịch vụ chuẩn hóa vẫn không đổi.
- **Conway's Law**: cấu trúc hệ thống phản ánh cấu trúc tổ chức — thay đổi bên này phải đi kèm thay đổi bên kia.
- **Failure as first-class citizen**: microservices không loại bỏ lỗi mà làm lỗi trở nên tường minh, vì vậy cần thiết kế cho tình huống lỗi ngay từ đầu.
- **Eventual consistency là trade-off, không phải khuyết điểm**: cần phân biệt khi nào hệ thống đòi hỏi strong consistency (điểm số, kết quả chấm bài) và khi nào có thể chấp nhận eventual consistency (bảng xếp hạng).
- **Observability trước tối ưu hóa**: không thể cải thiện những gì không đo được. Tracing và metric là công cụ tư duy, không chỉ là công cụ debug.
- **Monolith First khi phù hợp**: bộ công cụ đã học không phải lúc nào cũng nên áp dụng — sự phán đoán về *khi nào* quan trọng hơn kỹ năng *làm thế nào*.

KBM sau lộ trình: một kịch bản

Nếu KBM đi đúng lộ trình ở Chương 10: sau các cải tiến nhanh và nền tảng observability (Giai đoạn 1–2), nhóm phát triển nhìn thấy lỗi nằm ở đâu thay vì đoán qua log từng container. Khi có cơ chế chịu lỗi, dead letter topic và timeout cho saga nộp bài (Giai đoạn 3), không còn bài nộp nào treo ở PENDING. Đến khi tách schema và thay truy vấn chéo bằng API (Giai đoạn 4), Academic Service mới triển khai được độc lập với Core. Trong kỳ thi, dashboard cho thấy độ trễ nộp bài nằm trong SLO của chế độ thi (Chương 11). Đây là kết quả có thể đạt được khi áp dụng có trình tự và có phán đoán.

Quá trình học không kết thúc ở chương này. Lĩnh vực này tiếp tục phát triển — service mesh, platform engineering, AI-assisted observability — nhưng các công nghệ mới vẫn dựa trên nền tảng kiến trúc hướng dịch vụ và microservices đã học trong cuốn sách này: Bounded Context, cô lập lỗi và tự động hóa triển khai.

PHỤ LỤC

Tra cứu & luyện tập

Phụ lục A Bảng thuật ngữ
Phụ lục B Công cụ & tài nguyên
Phụ lục C Pattern Catalog
Phụ lục D Anti-pattern Catalog
Phụ lục E Case study KBM
Bài tập Bài tập & case study
Phụ lục F Gợi ý hướng giải
Phụ lục G Chiến lược kiểm thử & Testcontainers
Tài liệu tham khảo

PHỤ LỤC A

Bảng thuật ngữ

Thuật ngữ kỹ thuật dùng trong sách, xếp theo bảng chữ cái. Cột đầu ghi thuật ngữ tiếng Anh; nếu sách dùng dạng tiếng Việt, tên tiếng Việt ghi trong ngoặc. Cột "Chương" ghi chương trình bày chính của thuật ngữ. Trong các chương, thuật ngữ được chú giải ở lần xuất hiện đầu tiên; các thuật ngữ chuẩn quốc tế (API Gateway, microservices, endpoint...) giữ tiếng Anh.

Thuật ngữ	Giải thích ngắn	Chương
Accepted Risk (rủi ro được chấp nhận)	Rủi ro đã được nhận diện và tạm thời chấp nhận kèm điều kiện kiểm soát; phải ghi rõ và không được coi là kiến trúc mục tiêu.	Ch.8
Actor / Subject / Delegation	Trong phân quyền (authorization): subject là người dùng được đại diện; actor là workload đang gọi; delegation là quyền để actor hành động thay subject. Ba phần phải tách biệt để tránh confused deputy.	Ch.9
Aggregate	Cụm entity/value object được xem như một đơn vị nhất quán, truy cập qua một "gốc" (aggregate root) để bảo vệ ràng buộc nghiệp vụ.	Ch.2
Alert Fatigue (mệt mỏi vì cảnh báo)	Tình trạng đội vận hành nhận quá nhiều cảnh báo giả hoặc kém quan trọng đến mức bỏ qua cả sự cố thật; khắc phục bằng cảnh báo theo SLO và gom nhóm cảnh báo.	Ch.11
Anti-Corruption Layer	ACL — lớp dịch trung gian giúp một service không bị "nhiễm" mô hình dữ liệu của hệ thống ngoài/legacy khi tích hợp.	Ch.10
API Composition	Gộp dữ liệu từ nhiều service bằng cách gọi các service sở hữu dữ liệu rồi ghép kết quả ở tầng gọi (thay cho join trong database).	Ch.7
API Gateway	Lớp xử lý request tại biên, tập trung các nghiệp vụ cắt ngang (cross-cutting concern) như routing, xác minh token ở biên và rate limiting; không nắm quyền phân quyền nghiệp vụ của từng service.	Ch.8
Backend for Frontend	BFF — một gateway riêng "may đo" cho từng loại client (web, mobile), trả về dữ liệu vừa vặn nhu cầu client đó.	Ch.8
BASE	Basically Available, Soft state, Eventually consistent — triết lý dữ liệu phân tán đối trọng với ACID: ưu tiên tính sẵn sàng, chấp nhận trạng thái trung gian và nhất quán cuối cùng.	Ch.6
Blue/Green Deployment	Chạy song song hai môi trường (cũ "blue" và mới "green"), chuyển traffic sang bản mới khi sẵn sàng và có thể chuyển lại nếu phiên bản dữ liệu/API vẫn tương thích.	Ch.12
Bottleneck (điểm nghẽn, nút thắt cổ chai)	Thành phần chậm nhất giới hạn hiệu năng của toàn hệ thống.	Ch.1

Thuật ngữ	Giải thích ngắn	Chương
Bounded Context	Phạm vi mà một mô hình domain, bộ thuật ngữ và quy tắc có hiệu lực. Đây là ranh giới mô hình; không mặc nhiên ánh xạ một-một sang một service triển khai độc lập.	Ch.2
Branch by Abstraction	Kỹ thuật chuyển đổi: đặt một lớp trừu tượng trước phần hiện thực cũ, xây phiên bản mới phía sau cùng interface (bật/tắt bằng feature flag), chuyển dần lưu lượng rồi gỡ bản cũ.	Ch.10
Build	Biên dịch mã nguồn và đóng gói thành artifact chạy được.	Ch.12
Bulkhead	Cô lập tài nguyên (thread pool, kết nối) theo nhóm để một phần quá tải không gây đình trệ phần còn lại. Ẩn dụ: khoang kín của tàu thủy.	Ch.4
Business Logic	Phần mã nguồn cốt lõi thực thi các quy tắc, tính toán và luồng nghiệp vụ đặc thù của miền.	Ch.1
Canary Deployment	Phát hành dần cho một nhóm nhỏ người dùng trước, theo dõi, rồi mới mở rộng toàn bộ.	Ch.12
CAP Theorem	Định lý CAP — khi xảy ra network partition (phân vùng mạng), hệ phân tán buộc phải chọn giữa Consistency (nhất quán) và Availability (sẵn sàng); không phải "chọn 2 trong 3" (xem Chương 7).	Ch.7
Cardinality Explosion	Số time-series của metric tăng bùng nổ khi gắn nhãn có giá trị biến thiên cao (userId, submissionId), làm quá tải hệ thống metric; chi tiết theo từng đối tượng nên để cho log/trace.	Ch.11
Change Data Capture	CDC — bắt các thay đổi trong database (insert/update/delete) thành luồng sự kiện cho hệ thống khác tiêu thụ.	Ch.10
Chaos Engineering	Chủ động gây lỗi có kiểm soát trên hệ thống thật để kiểm chứng khả năng chịu lỗi.	Ch.11
Choreography	Cách phối hợp saga không có "nhạc trưởng": mỗi service tự phản ứng với event và phát event tiếp theo.	Ch.6
CI/CD	Continuous Integration / Continuous Delivery — tự động tích hợp, build, test và chuẩn bị bản sẵn sàng phát hành; triển khai production có thể cần phê duyệt. Continuous Deployment thì tự động triển khai mọi thay đổi vượt qua các cổng kiểm tra.	Ch.12
Circuit Breaker	Cơ chế "ngắt mạch" tạm dừng gọi tới một service đang lỗi để tránh lan truyền lỗi và giúp service đó có thời gian hồi phục. Ẩn dụ: cầu dao điện.	Ch.4
Cluster	Nhóm nhiều máy/node hoạt động như một hệ thống thống nhất.	Ch.12
Compensating Transaction (giao dịch bù trừ)	Thao tác "làm ngược lại" để hoàn tác hệ quả của một bước đã thực hiện trong saga (vì không thể rollback như giao dịch database).	Ch.6
Configuration Drift (sai lệch cấu hình)	Cấu hình các máy chủ/môi trường dần khác nhau do chỉnh sửa thủ công, khiến môi trường không tái tạo được; IaC và hạ tầng bất biến là cách phòng tránh.	Ch.12
Confused Deputy	Lỗ hổng khi service A gọi service B thay mặt người dùng nhưng B không phân biệt được A đang dùng quyền của người dùng hay đặc quyền riêng của A, dẫn đến leo thang đặc quyền.	Ch.9
Consistency Window	Khoảng thời gian từ lúc một sự kiện xảy ra cho đến khi mọi dịch vụ liên quan phản ánh trạng thái mới đó.	Ch.6
Consumer-Driven Contract Testing	CDCT — consumer công bố kỳ vọng về request/response, provider xác minh contract đó trong CI trước khi deploy (ví dụ: Pact). Sách dành `CDC` cho Change Data Capture.	Ch.3
Container	Gói đóng kín ứng dụng cùng mọi phụ thuộc để chạy giống nhau ở mọi môi trường.	Ch.12
Context Map	Bản đồ quan hệ giữa các Bounded Context, ghi rõ kiểu tích hợp giữa từng cặp.	Ch.2
Contract Testing (kiểm thử hợp đồng)	Kiểm thử xác nhận hai service tuân thủ đúng "hợp đồng" API đã thỏa thuận, không cần chạy cả hai cùng lúc.	Ch.3
Conway's Law	Cấu trúc hệ thống có xu hướng phản chiếu cấu trúc giao tiếp của tổ chức tạo ra nó.	Ch.2
Core Domain	Subdomain tạo khác biệt chiến lược và đáng nhận đầu tư thiết kế cao nhất; không phải tên gọi chung cho mọi Bounded Context nghiệp vụ.	Ch.2

Thuật ngữ	Giải thích ngắn	Chương
Correlation ID	Mã định danh gom nhóm một tiến trình nghiệp vụ qua nhiều request, trace và thông điệp bất đồng bộ (ví dụ trường `correlationId` trong event envelope); phân biệt với `requestId` (`X-Request-ID` cho từng HTTP request ở biên) và `traceId` (`traceparent` của W3C Trace Context).	Ch.5, 11
CQRS	Command Query Responsibility Segregation — tách mô hình ghi (command) và mô hình đọc (query) thành hai đường riêng để tối ưu độc lập.	Ch.7
Credential (thông tin xác thực)	Dữ liệu chứng minh danh tính khi xác thực, như mật khẩu, token, khóa API hoặc chứng chỉ.	Ch.9
Cross-cutting Concern (nghiệp vụ cắt ngang)	Các nghiệp vụ cắt ngang toàn bộ hệ thống (như bảo mật, logging, rate limiting) không thuộc về một domain cụ thể nào.	Ch.8
DB Queue	Dùng một bảng database làm hàng đợi công việc — đơn giản hơn message broker cho nhu cầu nhỏ.	Ch.5
Dead Letter Topic / Queue (DLT/ DLQ)	Nơi chứa message xử lý thất bại sau nhiều lần thử để đối soát và xử lý riêng thay vì retry vô hạn; Kafka dùng Dead Letter Topic (DLT), còn hàng đợi/outbox ngoài Kafka dùng Dead Letter Queue (DLQ).	Ch.5
Debug	Tìm và sửa nguyên nhân lỗi trong phần mềm.	Ch.1
Decision Matrix (ma trận quyết định)	Bảng đối chiếu tiêu chí để ra quyết định kiến trúc, ví dụ khi nào nên chuyển sang microservices hoặc ưu tiên công việc theo tác động × nỗ lực.	Ch.10
Defense in Depth (phòng thủ chiều sâu)	Bảo vệ nhiều lớp nối tiếp (mạng, ingress, service, dữ liệu) để khi một lớp bị vượt qua, các lớp còn lại vẫn đứng vững.	Ch.9
Distributed Monolith	Anti-pattern: đã có ranh giới service về hình thức nhưng dữ liệu, thư viện hoặc việc triển khai vẫn ràng buộc chặt, buộc phải thay đổi đồng loạt (lockstep), nên các service không thể tiến hóa độc lập.	Ch.1
Distributed Tracing (truy vết phân tán)	Theo dõi một request xuyên qua nhiều service bằng trace/span có đo thời gian từng bước — trả lời câu hỏi "chậm ở đâu" trong hệ phân tán.	Ch.11
Docker Compose	Công cụ định nghĩa và chạy nhiều container cùng lúc bằng một file YAML.	Ch.12
Domain-Driven Design	DDD — phương pháp thiết kế lấy mô hình nghiệp vụ (domain) làm trung tâm, dùng ngôn ngữ chung giữa lập trình viên và chuyên gia nghiệp vụ.	Ch.2
Downtime	Khoảng thời gian hệ thống không phục vụ được.	Ch.4
DTO	Data Transfer Object — đối tượng chỉ để truyền dữ liệu qua ranh giới (API), tách biệt với entity nội bộ.	Ch.3
Dual-write (ghi kép)	Ghi vào database rồi phát sự kiện bằng hai thao tác tách rời; nếu một bước thất bại, dữ liệu và sự kiện không còn nhất quán. Outbox Pattern là cách khắc phục.	Ch.10
Effectively-once	Đảm bảo "như thể xử lý đúng một lần" bằng at-least-once + idempotency (vì exactly-once tuyệt đối xuyên suốt các hệ thống là bất khả thi).	Ch.5
Endpoint	Điểm cuối giao tiếp mạng (thường là một URL cụ thể) nơi API tiếp nhận và trả về dữ liệu cho client.	Ch.3
Error Budget (ngân sách lỗi)	Phần chênh lệch cho phép `100% - SLO` trong một kỳ đo; còn ngân sách thì ưu tiên tính năng mới, cạn thì ưu tiên độ tin cậy.	Ch.4, 11
Event Sourcing	Lưu trạng thái dưới dạng chuỗi sự kiện bất biến (thay vì chỉ trạng thái hiện tại); trạng thái được tái dựng bằng cách "phát lại" sự kiện. Ẩn dụ: sổ sao kê ngân hàng.	Ch.7
Event Storming	Workshop dùng sticky note để cùng khám phá domain qua các sự kiện nghiệp vụ.	Ch.2
Event-driven (hướng sự kiện)	Kiến trúc trong đó các service giao tiếp gián tiếp bằng cách phát và nghe sự kiện.	Ch.5
Eventual Consistency	Dữ liệu giữa các service cuối cùng sẽ đồng bộ, nhưng có một khoảng trễ được chấp nhận — không có cận trên cam kết (đối lập với strong consistency).	Ch.6

Thuật ngữ	Giải thích ngắn	Chương
Expand/Contract	Kỹ thuật cập nhật schema tương thích ngược theo ba bước: mở rộng (thêm cột/bảng mới), chuyển dữ liệu, rồi thu hẹp (gỡ phần cũ) khi phiên bản code cũ đã ngừng chạy.	Ch.12
Flyway / Liquibase	Công cụ quản lý phiên bản schema database (versioned migration), chạy như một bước riêng trước khi deploy service.	Ch.12
GitOps	Quản lý hạ tầng/triển khai bằng Git làm nguồn sự thật duy nhất; mọi thay đổi qua commit/PR.	Ch.12
GraphQL	Ngôn ngữ truy vấn API cho phép client yêu cầu đúng dữ liệu cần, tránh over/under-fetching.	Ch.3
gRPC	Framework RPC hiệu năng cao của Google chạy trên HTTP/2 + Protobuf, hợp cho giao tiếp nội bộ service-to-service.	Ch.4
HATEOAS	Mức "trưởng thành" của REST trong đó response kèm các link điều hướng hành động tiếp theo.	Ch.3
Head-of-Line Blocking	Một message xử lý chậm ở đầu partition chặn mọi message phía sau trong cùng partition.	Ch.5
Header	Tiêu đề của một gói tin hoặc yêu cầu HTTP, dùng để truyền tải các siêu dữ liệu (metadata) như access token hoặc mã định danh yêu cầu (`X-Request-ID`).	Ch.8
Health Check	Endpoint (thường là `/health`) cho biết service còn chạy và sẵn sàng nhận request; được registry, load balancer và orchestrator dùng để định tuyến và tự phục hồi. Gồm liveness, readiness và startup.	Ch.11
Hexagonal Architecture	Ports & Adapters — tách lõi nghiệp vụ khỏi hạ tầng qua "cổng" và "bộ chuyển đổi", giúp dễ test và thay công nghệ.	Ch.3
Idempotency (tính lũy đẳng)	Tính chất của một thao tác: thực hiện nhiều lần cho kết quả giống như thực hiện một lần duy nhất. Ẩn dụ: bấm nút gọi thang máy 5 lần, thang vẫn chỉ đến một lần.	Ch.3, 5
Immutable Infrastructure (hạ tầng bất biến)	Không sửa trực tiếp máy chủ đang chạy; mọi thay đổi được thực hiện bằng cách tạo phiên bản mới và thay thế phiên bản cũ.	Ch.12
Implementation Drift (sự trôi lệch triển khai)	Các service hiện thực cùng một chính sách theo những cách khác nhau (thư viện, phiên bản, quy tắc kiểm tra), dẫn đến hành vi không nhất quán dù cùng tên gọi.	Ch.8
Independent Silo Monoliths	Nhiều ứng dụng monolith tách rời, mỗi ứng dụng có codebase/deployment riêng nhưng thiếu kiến trúc và contract thống nhất; khác Distributed Monolith.	Ch.1
Infrastructure as Code	IaC — mô tả hạ tầng bằng mã/khai báo để tạo lại tự động và nhất quán.	Ch.12
JWKS	JSON Web Key Set — tập khóa công khai (public key) do issuer công bố để resource server xác minh token và tự làm mới khóa khi luân phiên khóa (rotation).	Ch.9
JWT	JSON Web Token — định dạng token; sách chủ yếu dùng compact JWS có chữ ký để resource server xác minh claim mà không cần tra database ở mỗi request.	Ch.9
k3s	Bản Kubernetes gọn nhẹ, hợp cho edge và môi trường tài nguyên thấp.	Ch.12
Load Balancing (cân bằng tải)	Phân phối request đến nhiều instance để tránh quá tải một instance.	Ch.4
Message Broker	Trung gian nhận/lưu/chuyển message giữa producer và consumer, cho phép giao tiếp bất đồng bộ. Ẩn dụ: bưu điện giữ thư hộ.	Ch.5
Microservice	Dịch vụ nhỏ, tự chủ, triển khai độc lập, sở hữu dữ liệu riêng và giao tiếp với các dịch vụ khác qua mạng.	Ch.1
Migration (chuyển đổi) / Migration Path (lộ trình chuyển đổi)	Quá trình đưa một hệ thống đang vận hành từ kiến trúc hiện tại sang kiến trúc mục tiêu; lộ trình chuyển đổi là chuỗi bước tăng dần, mỗi bước mang lại giá trị và có thể hoàn tác.	Ch.10
Modular Monolith	Ứng dụng nguyên khối nhưng được chia thành các module có ranh giới rõ ràng bên trong — một lựa chọn hợp lệ trước khi tách microservices.	Ch.1
Monolith (nguyên khối)	Kiến trúc nguyên khối: ứng dụng được đóng gói và triển khai như một khối duy nhất.	Ch.1

Thuật ngữ	Giải thích ngắn	Chương
mTLS	Mutual TLS — hai phía xác thực certificate/workload identity và thiết lập kênh mã hóa. mTLS không tự chứng minh nội dung claim của người dùng hoặc thay cho phân quyền nghiệp vụ.	Ch.9
MTTF / MTBF / MTTR	Thời gian trung bình tới khi hỏng, giữa hai lần hỏng, và để khắc phục; `MTBF = MTTF + MTTR`.	Ch.4
OAuth2	Chuẩn ủy quyền cho phép cấp quyền truy cập tài nguyên mà không chia sẻ mật khẩu.	Ch.9
Observability	Khả năng hiểu trạng thái bên trong hệ thống từ dữ liệu nó phát ra (log, metric, trace) — đủ để đặt câu hỏi mới mà không cần triển khai lại.	Ch.11
Observability Maturity Model (mô hình trưởng thành)	Thang 5 cấp (Reactive → Predictive) giúp xác định năng lực observability hiện tại và mục tiêu phù hợp; khác với Richardson Maturity Model của REST.	Ch.11
OIDC	OpenID Connect — lớp xác thực (authentication) xây trên OAuth2, bổ sung danh tính người dùng qua ID token.	Ch.9
OpenAPI	OpenAPI / Swagger — chuẩn mô tả REST API dạng máy đọc được, dùng để sinh tài liệu và code.	Ch.3
OpenFeign	Thư viện Spring cho phép gọi REST service khác qua khai báo interface (declarative HTTP client).	Ch.4
Orchestration	Cách phối hợp saga có "nhạc trưởng" trung tâm điều khiển từng bước.	Ch.6
Outbox Pattern	Ghi sự kiện vào một bảng "outbox" cùng transaction với dữ liệu nghiệp vụ, rồi phát sang message broker ở bước riêng — tránh mất sự kiện do ghi kép (dual-write); relay có thể phát trùng nên consumer vẫn cần idempotent.	Ch.10
Overhead	Chi phí tài nguyên/độ trễ phát sinh thêm ngoài công việc chính.	Ch.1
Pact	Công cụ contract testing theo hướng consumer-driven.	Ch.3
Parallel Run	Chạy song song logic cũ và mới trên dữ liệu thật, chỉ trả kết quả của bản cũ và so sánh để phát hiện sai lệch trước khi chuyển hẳn sang bản mới.	Ch.10
Pipeline	Chuỗi bước tự động (build, test, deploy…) nối tiếp nhau.	Ch.12
Pivot Transaction (giao dịch bản lề)	Bước trong saga mà sau đó không thể quay đầu: các bước trước có thể bù trừ, các bước sau phải thử lại đến khi thành công.	Ch.6
PKCE	Proof Key for Code Exchange — cơ chế bảo vệ luồng OAuth2 Authorization Code cho client công khai (mobile/SPA) chống đánh cắp mã.	Ch.9
Poison Pill	Message lỗi/không xử lý được làm consumer kẹt vòng lặp retry; cần tách sang DLT.	Ch.5
Polyglot Persistence	Mỗi service tự chọn loại database phù hợp nhất với nhu cầu của nó (SQL, NoSQL, cache…).	Ch.7
Rate Limiting	Giới hạn số request trong một khoảng thời gian để bảo vệ hệ thống khỏi quá tải/lạm dụng.	Ch.8
RBAC	Role-Based Access Control — phân quyền theo vai trò được gán cho người dùng.	Ch.9
Refactor	Cải thiện cấu trúc mã mà không thay đổi hành vi.	Ch.1
Resilience4j	Thư viện Java hiện thực các resilience pattern (Circuit Breaker, Retry, Timeout, Bulkhead) qua annotation/config.	Ch.4
Resource Server	Service/API sở hữu tài nguyên được bảo vệ; nó xác minh access token dành cho mình và tự phân quyền cục bộ (local authorization).	Ch.9
REST	Kiểu kiến trúc API dựa trên resource và HTTP verbs (GET/POST/PUT/DELETE).	Ch.3
Retry (thử lại)	Thử lại request khi gặp lỗi tạm thời.	Ch.4
Rolling Update	Thay lần lượt từng instance từ phiên bản cũ sang phiên bản mới mà không cần gấp đôi tài nguyên; trong quá trình này cả hai phiên bản cùng chạy.	Ch.12
Routing (định tuyến)	Quá trình định tuyến, điều hướng request từ nguồn đến đúng đích dựa trên các quy tắc (path, header).	Ch.8

Thuật ngữ	Giải thích ngắn	Chương
Saga Pattern	Phối hợp chuỗi giao dịch cục bộ (local transaction); khi bước sau thất bại, hành động bù trừ (compensating action) đưa nghiệp vụ về trạng thái chấp nhận được, không nhất thiết khôi phục bit-for-bit như rollback database.	Ch.6
Scaling (mở rộng)	Mở rộng khả năng phục vụ bằng cách thêm tài nguyên (scale up) hoặc thêm instance (scale out).	Ch.1
Semantic Lock	Cờ trạng thái ở tầng ứng dụng đánh dấu bản ghi đang được một saga xử lý, để các saga khác không can thiệp.	Ch.6
Service	Một thành phần phần mềm tự chủ cung cấp một nhóm chức năng qua giao diện rõ ràng.	Ch.1
Service Discovery	Cơ chế để service tìm địa chỉ động của service khác mà không hard-code IP.	Ch.4
Service Mesh	Lớp hạ tầng (thường qua sidecar) lo giao tiếp service-to-service: routing, mTLS, retry, observability.	Ch.12
Service Registry	Danh bạ trung tâm lưu địa chỉ các instance đang hoạt động; service tự đăng ký (register) và bên gọi tra cứu để định tuyến (ví dụ: Netflix Eureka).	Ch.4
Shadow Deployment	Nhân bản lưu lượng thật sang phiên bản mới nhưng bỏ kết quả của nó, để kiểm chứng dưới tải thật trước khi phát hành; mọi tác dụng phụ phải được cô lập.	Ch.12
Sidecar	Sidecar pattern — đặt một container phụ chạy cạnh service chính để đảm nhận các tác vụ chung (proxy, logging) mà không phải sửa đổi mã nguồn của service đó.	Ch.12
Signed Identity Propagation	Truyền token/metadata có chữ ký qua ranh giới tin cậy (trust boundary); service đích kiểm tra issuer, audience, signature và freshness trước khi dùng claim.	Ch.8, 9
SLI/SLO/SLA	Chỉ số đo chất lượng dịch vụ (SLI), mục tiêu nội bộ (SLO) và cam kết với khách hàng (SLA).	Ch.11
Strangler Fig	Strangler Fig pattern — thay thế dần hệ cũ bằng cách bọc và chuyển từng phần chức năng sang hệ mới, đến khi hệ thống cũ bị thay thế hoàn toàn.	Ch.10
Strong Consistency (nhất quán mạnh)	Mọi thao tác đọc luôn thấy lần ghi mới nhất, không có khoảng tạm thời đọc phải dữ liệu cũ (stale).	Ch.7
Structured Logging (log có cấu trúc)	Ghi log ở dạng có cấu trúc (thường là JSON) với các trường như timestamp, service, traceId để truy vấn, lọc và tổng hợp tự động.	Ch.11
Sysbox	Runtime container cho phép chạy Docker/Kubernetes lồng bên trong container an toàn hơn.	Ch.12
Team	Nhóm/đội phát triển.	Ch.1
Tiered Trust (tin cậy phân tầng)	Bốn tầng tin cậy (trust tier) tăng bằng chứng theo rủi ro: T0 request từ bên ngoài chưa tin cậy; T1 bằng chứng người dùng có chữ ký + phân quyền cục bộ; T2 thêm workload/actor/delegation; T3 bảo vệ job/command bất đồng bộ.	Ch.9
Timeout	Bên gọi ngừng chờ sau một thời hạn; công việc phía remote có thể vẫn tiếp tục, nên luồng nhạy cảm cần cancellation, idempotency hoặc fencing phù hợp.	Ch.4
Token Rotation (luân phiên token)	Mỗi lần làm mới cấp một refresh token mới và vô hiệu hóa token cũ, nhờ đó phát hiện được việc dùng lại token đã lộ.	Ch.9
Traffic	Lưu lượng truy cập mạng hoặc khối lượng dữ liệu trao đổi giữa các thành phần hệ thống.	Ch.8
Turnstile	Cloudflare Turnstile — dịch vụ challenge chống bot (thay thế CAPTCHA), bảo vệ các form nhạy cảm như đăng nhập/đăng ký/khôi phục mật khẩu.	Ch.9
Two-Phase Commit	2PC — giao thức commit phân tán hai pha (prepare rồi commit); đảm bảo nhất quán nhưng khóa tài nguyên và kém chịu lỗi, ít dùng trong microservices.	Ch.6

Thuật ngữ	Giải thích ngắn	Chương
Ubiquitous Language (ngôn ngữ chung)	Bộ thuật ngữ thống nhất giữa lập trình viên và chuyên gia nghiệp vụ trong một Bounded Context, dùng cả trong trao đổi lẫn trong mã.	Ch.2
Virtual Threads	Luồng nhẹ do JVM quản lý từ Java 21, giúp các lệnh gọi blocking I/O ít chiếm luồng nền tảng.	Ch.4
WebSocket / SSE	Hai cơ chế server đẩy dữ liệu real-time về client: WebSocket (hai chiều, full-duplex) và Server-Sent Events (một chiều server→client).	Ch.5
Workload Identity	Danh tính mật mã của service/job đang chạy, dùng để xác thực giữa các service (service-to-service); khác với user subject mà workload có thể đại diện.	Ch.9
Zero Trust	Không mặc nhiên tin cậy chỉ vì vị trí mạng; mỗi lần truy cập tài nguyên cần xác thực và phân quyền phù hợp với chính sách.	Ch.9

PHỤ LỤC B

Công cụ & tài nguyên

Tổng hợp các công cụ và tài nguyên chính được dùng trong sách, nhóm theo chức năng. Các ví dụ mã giả định Spring Boot 3.x (Jakarta EE 10+, Java 17+); với Spring Boot 2.x, namespace `javax.*` thay cho `jakarta.*` và một số API cấu hình cũng khác. Phiên bản cụ thể của từng công cụ thay đổi theo thời gian; xem trang chính thức của công cụ để có thông tin mới nhất.

B.1. Framework & thư viện

Bảng B.1 liệt kê các framework và thư viện dùng trong sách.

Bảng B.1: *Framework và thư viện*

Công cụ	Mô tả	Chương	URL
Spring Boot	Framework Java cho microservices	Xuyên suốt	spring.io
Spring Cloud	Bộ công cụ cho hệ thống phân tán	Ch.4, 8	spring.io/cloud
Spring Modulith	Hỗ trợ xây dựng kiến trúc Modular Monolith cho Spring Boot — ranh giới module, kiểm thử tích hợp (integration testing) và phát hành sự kiện (event publication)	Ch.1	spring.io/projects/spring-modulith
Spring Cloud Gateway	API Gateway (dựa trên WebFlux)	Ch.8	docs.spring.io
Netflix Eureka	Service Discovery & Registry	Ch.4, 8	github.com/Netflix/eureka
OpenFeign	REST client dạng khai báo	Ch.4	github.com/OpenFeign
Resilience4j	Circuit Breaker, Retry, Bulkhead	Ch.4	resilience4j.readme.io
Spring Security	Xác thực và phân quyền	Ch.9	spring.io/security
JJWT	Sinh và xác minh JWT	Ch.9	github.com/jwtk/jjwt
MapStruct	Sinh mã ánh xạ DTO	Ch.3	mapstruct.org
Pact	Công cụ contract testing theo hướng consumer-driven	Ch.3, 10	pact.io
Chi	Bộ định tuyến HTTP (HTTP router) gọn nhẹ viết bằng Go	Ch.3	github.com/go-chi/chi

B.2. Messaging & dữ liệu

Bảng B.2 liệt kê các công cụ messaging và lưu trữ dữ liệu.

Bảng B.2: *Công cụ messaging và dữ liệu*

Công cụ	Mô tả	Chương	URL
Apache Kafka	Nền tảng event streaming phân tán	Ch.5, 6	kafka.apache.org
PostgreSQL	Cơ sở dữ liệu quan hệ	Ch.7	postgresql.org
Redis	Bộ nhớ đệm in-memory, dùng cả cho rate limiting	Ch.7, 8	redis.io
golang-migrate	Quản lý phiên bản schema cơ sở dữ liệu (database migration) cho các dịch vụ bằng Go	Ch.7	github.com/golang-migrate/migrate

B.3. Hạ tầng & DevOps

Bảng B.3 liệt kê các công cụ hạ tầng, container và CI/CD.

Bảng B.3: *Công cụ hạ tầng và DevOps*

Công cụ	Mô tả	Chương	URL
Docker	Nền tảng container hóa	Ch.12	docker.com
Docker Compose	Điều phối nhiều container trên một máy	Ch.12	docs.docker.com/compose
Kubernetes	Điều phối container (giới thiệu tổng quan)	Ch.12	kubernetes.io
k3s	Bản phân phối Kubernetes gọn nhẹ	Ch.12	k3s.io
Sysbox	Container runtime hỗ trợ sandbox/ container-in-container	Ch.9, 12	github.com/nestybox/sysbox
GitLab CI / GitHub Actions	Tự động hóa CI/CD	Ch.12	docs.gitlab.com/ci / docs.github.com/ actions
Helm	Trình quản lý gói (chart) cho Kubernetes	Ch.12	helm.sh
Argo CD	Công cụ GitOps đồng bộ trạng thái cluster từ Git	Ch.12	argo-cd.readthedocs.io
Terraform	Công cụ Infrastructure as Code khai báo hạ tầng	Ch.12	terraform.io

B.4. Chuyển đổi & quản lý phát hành

Bảng B.4 liệt kê các công cụ hỗ trợ chuyển đổi và phát hành.

Bảng B.4: *Công cụ chuyển đổi và quản lý phát hành*

Công cụ	Mô tả	Chương	URL
Debezium	Change Data Capture (CDC)	Ch.10	debezium.io
Flyway / Liquibase	Quản lý phiên bản schema cơ sở dữ liệu (database migration)	Ch.12	flywaydb.org / liquibase.org
Togglz / Unleash	Thư viện/nền tảng quản lý cờ tính năng (feature flag)	Ch.10	togglz.org / getunleash.io

B.5. Observability

Bảng B.5 liệt kê các công cụ observability cho log, metric và trace.

Bảng B.5: Công cụ observability

Công cụ	Mô tả	Chương	URL
Micrometer	Lớp trừu tượng thu thập metric cho JVM	Ch.11	micrometer.io
OpenTelemetry	Chuẩn thu thập telemetry (log, metric, trace)	Ch.11	opentelemetry.io
Jaeger	Hệ thống lưu trữ và trực quan hóa dữ liệu distributed tracing	Ch.11	jaegertracing.io
ELK Stack	Elasticsearch + Logstash + Kibana	Ch.11	elastic.co
Prometheus + Grafana	Thu thập và trực quan hóa metric	Ch.11	prometheus.io / grafana.com
Grafana Alloy + Loki	Bộ công cụ gom log tập trung gọn nhẹ	Ch.11	grafana.com/oss/loki
zerolog	Thư viện log có cấu trúc (structured logging) cho Go	Ch.11	github.com/rs/zerolog

B.6. Bảo mật & danh tính

Bảng B.6 liệt kê các công cụ bảo mật và định danh.

Bảng B.6: Công cụ bảo mật và định danh

Công cụ	Mô tả	Chương	URL
Microsoft Entra ID (OAuth2/OIDC)	Nhà cung cấp danh tính (identity provider) độc lập hỗ trợ đăng nhập một lần (SSO)	Ch.9	microsoft.com/entra
Cloudflare Turnstile	Thử thách chống bot	Ch.9	cloudflare.com/products/turnstile

B.7. Công cụ phát triển

Bảng B.7 liệt kê các công cụ phát triển thường dùng.

Bảng B.7: Công cụ phát triển

Công cụ	Mô tả	URL
IntelliJ IDEA	IDE cho Java	jetbrains.com/idea
Postman	Kiểm thử và tài liệu hóa API	postman.com
Swagger UI	Tài liệu API tương tác	swagger.io

PHỤ LỤC C

Pattern Catalog

Tổng hợp 58 pattern được trình bày trong sách, nhóm theo chủ đề. Mỗi dòng chỉ tóm tắt pattern; chi tiết, ví dụ và trade-off xem ở chương tương ứng.

C.1. Pattern giao tiếp

Bảng C.1 tóm tắt các pattern giao tiếp, kèm vị trí trong sách và tài liệu tham chiếu.

Bảng C.1: *Pattern giao tiếp*

Pattern	Mô tả ngắn	Chương	Tham chiếu
Request/Response	Máy khách (client) gửi yêu cầu và chờ đợi phản hồi	Ch.4 §4.1	[2a] Ch.3
Publish/Subscribe	Bên phát (producer) phát hành sự kiện, consumer đăng ký theo dõi	Ch.5 §5.1	[2a] Ch.3
One-way Notification	Gửi thông điệp một chiều (fire-and-forget)	Ch.5 §5.1	[2a] Ch.3
DB-backed Queue	Sử dụng bảng cơ sở dữ liệu làm hàng đợi cho các tác vụ cần giao dịch (transactional job)	Ch.5 §5.7	[5] Ch.3
API Composition	Gọi nhiều dịch vụ và tổng hợp kết quả trả về	Ch.7 §7.3	[2a] Ch.7
Dead Letter Topic / Queue (DLT/DLQ)	Định tuyến các thông điệp xử lý thất bại vào hàng đợi riêng để đối soát và xử lý thủ công	Ch.5 §5.9	[5] Ch.3

C.2. Pattern resilience

Bảng C.2 tóm tắt các pattern resilience (chịu lỗi), kèm vị trí trong sách và tài liệu tham chiếu.

Bảng C.2: *Pattern resilience*

Pattern	Mô tả ngắn	Chương	Tham chiếu
Circuit Breaker	Ngắt mạch khi dịch vụ hạ nguồn (downstream) liên tục gặp lỗi	Ch.4 §4.4	[5] Ch.7
Retry	Tự động thử lại khi gặp lỗi tạm thời	Ch.4 §4.4	[5] Ch.7
Timeout	Giới hạn thời gian chờ phản hồi	Ch.4 §4.4	[5] Ch.7
Bulkhead	Cô lập tài nguyên theo nhóm chức năng hoặc dịch vụ	Ch.4 §4.4	[5] Ch.7

Pattern	Mô tả ngắn	Chương	Tham chiếu
Idempotent Consumer	Đảm bảo consumer xử lý thông điệp trùng lặp mà không làm sai lệch trạng thái hệ thống	Ch.3 §3.1, Ch.5 §5.5	[2a] Ch.3

C.3. Pattern dữ liệu

Bảng C.3 tóm tắt các pattern quản lý dữ liệu, kèm vị trí trong sách và tài liệu tham chiếu.

Bảng C.3: *Pattern dữ liệu*

Pattern	Mô tả ngắn	Chương	Tham chiếu
Database-per-Service	Mỗi dịch vụ sở hữu dữ liệu riêng và kiểm soát truy cập; có thể dùng schema riêng trên cùng instance, không bắt buộc server riêng	Ch.7 §7.1	[4b] Ch.4
Orchestration-based Saga	Bộ điều phối trung tâm (central orchestrator) quản lý các giao dịch cục bộ	Ch.6 §6.2–6.3	[2a] Ch.4
Choreography-based Saga	Các dịch vụ tự phối hợp thông qua sự kiện	Ch.6 §6.2–6.3	[2a] Ch.4
CQRS	Tách biệt mô hình đọc (read model) và mô hình ghi (write model)	Ch.7 §7.4	[2a] Ch.7
Event Sourcing	Lưu trữ chuỗi sự kiện thay vì trạng thái hiện tại	Ch.7 §7.5	[2a] Ch.6
Data Duplication	Sao chép dữ liệu tham chiếu thông qua luồng sự kiện	Ch.7 §7.3	[4b] Ch.4
Shared Kernel	Các Bounded Context chia sẻ một mô hình chung	Ch.2 §2.5	[6]
Outbox Pattern	Đảm bảo gửi thông điệp tin cậy (reliable messaging) khi cập nhật cơ sở dữ liệu	Ch.10 §10.3	[2a] Ch.4
Change Data Capture (CDC)	Theo dõi các thay đổi dữ liệu từ transaction log của cơ sở dữ liệu để phát luồng sự kiện thời gian thực	Ch.10 §10.3	[7] Ch.11
Event-Carried State Transfer	Nhúng bản chụp trạng thái (state snapshot) vào thông điệp sự kiện, giúp consumer tự cập nhật bộ đệm cục bộ	Ch.7 §7.3	[5] Ch.3

C.4. Pattern hạ tầng

Bảng C.4 tóm tắt các pattern hạ tầng, kèm vị trí trong sách và tài liệu tham chiếu.

Bảng C.4: *Pattern hạ tầng*

Pattern	Mô tả ngắn	Chương	Tham chiếu
API Gateway	Điểm truy cập duy nhất (single entry point) cho mọi client	Ch.8 §8.1	[2a] Ch.8
Backend for Frontend (BFF)	Gateway được thiết kế riêng cho từng loại client	Ch.8 §8.1	[2a] Ch.8
Hostname-based Routing	Định tuyến yêu cầu theo tên miền tới từng không gian làm việc hoặc môi trường chạy	Ch.8 §8.5	[2a] Ch.8
Service Discovery	Sổ đăng ký (registry) để tìm kiếm các service instance đang hoạt động	Ch.4 §4.3	[2a] Ch.3
Client-side Discovery	Client tự truy vấn sổ đăng ký và tự cân bằng tải	Ch.4 §4.3	[2a] Ch.3
Server-side Discovery	Bộ cân bằng tải (load balancer) đảm nhiệm việc truy vấn sổ đăng ký	Ch.4 §4.3	[2a] Ch.3
Sidecar	Tiến trình phụ chạy kèm với mỗi instance của dịch vụ	Ch.12 §12.6	[3] Ch.8
Rate Limiting	Giới hạn tần suất gửi yêu cầu để bảo vệ dịch vụ khỏi nguy cơ quá tải và tấn công DoS	Ch.8 §8.4	[2a] Ch.8
Externalized Configuration	Tách biệt cấu hình ứng dụng khỏi mã nguồn và quản lý tập trung theo từng môi trường	Ch.12 §12.4	[2a] Ch.11

Pattern	Mô tả ngắn	Chương	Tham chiếu
Service Mesh	Lớp hạ tầng quản lý giao tiếp qua sidecar proxy; mTLS xác thực workload và bảo vệ kênh truyền, không thay thế bằng chứng danh tính người dùng	Ch.12 §12.6	[3] Ch.8

C.5. Pattern bảo mật

Bảng C.5 tóm tắt các pattern bảo mật, kèm vị trí trong sách và tài liệu tham chiếu.

Bảng C.5: *Pattern bảo mật*

Pattern	Mô tả ngắn	Chương	Tham chiếu
Token-based Auth (JWT)	Xác minh bằng chứng danh tính theo cách stateless, qua token có chữ ký	Ch.9 §9.2	[4a] Ch.9
Token Exchange	Chuyển đổi token định danh bên ngoài thành token nội bộ	Ch.9 §9.4	[4a] Ch.9
Signed Identity Propagation	Truyền token hoặc metadata có chữ ký, mang đúng audience; service đích xác minh bằng chứng rồi phân quyền cục bộ	Ch.9 §9.3	[2a] Ch.11
RBAC	Phân quyền truy cập dựa trên vai trò của người dùng	Ch.9 §9.5	[4a] Ch.9
Runtime Isolation	Cách ly môi trường chạy bằng runtime, chính sách mạng (NetworkPolicy) và hạn ngạch (quota)	Ch.9 §9.6	[4a] Ch.9

C.6. Pattern observability

Bảng C.6 tóm tắt các pattern observability, kèm vị trí trong sách và tài liệu tham chiếu.

Bảng C.6: *Pattern observability*

Pattern	Mô tả ngắn	Chương	Tham chiếu
Structured Logging	Ghi nhật ký định dạng JSON kèm theo các trường liên kết (correlation field)	Ch.11 §11.2	[2a] Ch.11
Distributed Tracing	Truy vết yêu cầu xuyên suốt qua nhiều dịch vụ	Ch.11 §11.3	[2a] Ch.11
Health Check	Kiểm tra trạng thái dịch vụ (liveness probe và readiness probe)	Ch.11 §11.6	[2a] Ch.11
Correlation ID	Mã định danh gom nhóm toàn bộ luồng nghiệp vụ qua nhiều dịch vụ và sự kiện bất đồng bộ	Ch.5 §5.6, Ch.11 §11.2	[4a] Ch.8
SLI/SLO/SLA	SLI: chỉ số chất lượng đo được; SLO: mục tiêu cho chỉ số; SLA: thỏa thuận chất lượng dịch vụ với khách hàng	Ch.11 §11.4	Beyer và cộng sự (2016)
Log Aggregation	Thu thập và hợp nhất dữ liệu nhật ký từ tất cả các dịch vụ phân tán về một kho lưu trữ tập trung	Ch.11 §11.2	[2a] Ch.11

C.7. Pattern triển khai

Bảng C.7 tóm tắt các pattern triển khai, kèm vị trí trong sách và tài liệu tham chiếu.

Bảng C.7: *Pattern triển khai*

Pattern	Mô tả ngắn	Chương	Tham chiếu
Rolling Update	Thay thế dần từng instance nhằm giảm gián đoạn dịch vụ; điều kiện là có đủ replica, có readiness probe và hai phiên bản tương thích với nhau	Ch.12 §12.5	[2a] Ch.12
Blue/Green	Duy trì hai môi trường song song và chuyển toàn bộ lưu lượng	Ch.12 §12.5	[2a] Ch.12

Pattern	Mô tả ngắn	Chương	Tham chiếu
Canary	Định tuyến một phần lưu lượng tới phiên bản mới để thử nghiệm	Ch.12 §12.5	[2a] Ch.12
Infrastructure as Code	Tự động hóa hạ tầng bằng mã khai báo (Docker Compose, Kubernetes)	Ch.12 §12.6	[3] Ch.6
CI/CD Pipeline	Tự động hóa toàn bộ chuỗi build, kiểm thử và triển khai	Ch.12 §12.4	[3] Ch.10

C.8. Pattern chuyển đổi

Bảng C.8 tóm tắt các pattern chuyển đổi, kèm vị trí trong sách và tài liệu tham chiếu.

Bảng C.8: *Pattern chuyển đổi*

Pattern	Mô tả ngắn	Chương	Tham chiếu
Strangler Fig	Chuyển đổi dần dần từ kiến trúc nguyên khối (monolith)	Ch.10 §10.2	[4b] Ch.3
Anti-Corruption Layer	Lớp dịch mô hình dữ liệu giữa hệ thống cũ và mới	Ch.10 §10.4	[6] Ch.14
Branch by Abstraction	Tạo lớp trừu tượng (abstraction) và chuyển đổi phần hiện thực (implementation)	Ch.10 §10.4	[4b] Ch.3
Parallel Run	Chạy song song hệ thống cũ và mới, so sánh kết quả đầu ra	Ch.10 §10.4	[4b] Ch.3

C.9. Pattern DDD

Bảng C.9 tóm tắt các pattern Domain-Driven Design, kèm vị trí trong sách và tài liệu tham chiếu.

Bảng C.9: *Pattern DDD*

Pattern	Mô tả ngắn	Chương	Tham chiếu
Bounded Context	Ranh giới hiệu lực của ngôn ngữ và mô hình nghiệp vụ	Ch.2 §2.3	[6] Ch.14
Context Map	Bản đồ thể hiện mối quan hệ giữa các Bounded Context	Ch.2 §2.3	[6] Ch.14
Aggregate	Cụm các thực thể (entity), là ranh giới nhất quán (consistency boundary)	Ch.2 §2.2	[6] Ch.5
Ubiquitous Language	Ngôn ngữ chung giữa đội ngũ phát triển, mã nguồn và chuyên gia nghiệp vụ	Ch.2 §2.2	[6] Ch.2
Event Storming	Workshop khám phá nghiệp vụ thông qua các sự kiện	Ch.2 §2.3	Brandolini (2013)

C.10. Pattern kiểm thử

Bảng C.10 tóm tắt các pattern kiểm thử, kèm vị trí trong sách và tài liệu tham chiếu.

Bảng C.10: *Pattern kiểm thử*

Pattern	Mô tả ngắn	Chương	Tham chiếu
Consumer-Driven Contract Testing	Consumer định nghĩa hợp đồng (contract), tức kỳ vọng về API; provider xác minh trong CI trước khi triển khai	Ch.3 §3.6, Phụ lục G	[2a] Ch.9
Chaos Engineering	Chủ động tiêm lỗi có kiểm soát vào hệ thống để kiểm chứng khả năng chịu lỗi	Ch.11 §11.8	principlesofchaos.org

PHỤ LỤC D

Anti-pattern Catalog

Tổng hợp 43 anti-pattern từ các mục sai lầm thường gặp, lỗ hổng, cạm bẫy và rủi ro của 12 chương, nhóm theo 7 chủ đề. Khi gặp một vấn đề trong hệ thống, tra bảng theo chủ đề, xem mức độ và cách phòng tránh, rồi đọc chương tương ứng để hiểu ngữ cảnh; ưu tiên các mục mức nghiêm trọng, nhất là mục liên quan đến mất dữ liệu và bảo mật.

Bốn mức độ nghiêm trọng:

- **NGHIÊM TRỌNG (critical, 8 mục)**: dữ liệu mất hoặc hệ thống sập.
- **CAO (high, 22 mục)**: ảnh hưởng nghiêm trọng đến tính đúng đắn hoặc khả năng vận hành.
- **TRUNG BÌNH (medium, 12 mục)**: tăng độ phức tạp hoặc nợ kỹ thuật (technical debt) đáng kể.
- **THẤP (low, 1 mục)**: bất tiện, dễ khắc phục.

D.1. Kiến trúc & thiết kế

Bảng D.1 tóm tắt các anti-pattern về kiến trúc và thiết kế, kèm mức độ, hậu quả và cách phòng tránh.

Bảng D.1: *Anti-pattern kiến trúc và thiết kế*

#	Anti-pattern	Mức độ	Hậu quả	Phòng tránh	Ch.
1	**Hype-Driven Microservices** — Lựa chọn kiến trúc microservices chạy theo trào lưu, không xuất phát từ bài toán thực tế của tổ chức	TRUNG BÌNH	Làm tăng độ phức tạp của hệ thống mà không mang lại lợi ích thực tiễn	Xuất phát từ vấn đề (Khung ra quyết định) thay vì áp đặt giải pháp từ đầu	1
2	**Big Bang Migration** — Chuyển đổi tổng thể một cách đột ngột từ kiến trúc nguyên khối (monolith) sang microservices, bỏ qua bước trung gian	CAO	Tạo ra Distributed Monolith — mang nhược điểm của cả hai mô hình	Tái cấu trúc (refactor) ranh giới các module rõ ràng *trước* khi tiến hành tách dịch vụ	1

#	Anti-pattern	Mức độ	Hậu quả	Phòng tránh	Ch.
3	**Premature Service Decomposition** — Phân rã dịch vụ trước khi tổ chức lại nhân sự (ví dụ: một đội ngũ phải ôm đồm 10 dịch vụ và triển khai chúng cùng lúc)	TRUNG BÌNH	Tăng gánh nặng vận hành (overhead) mà không đạt được sự tự chủ (autonomy) thực sự	Tổ chức lại đội ngũ nhân sự *cùng lúc* hoặc *trước* khi tách dịch vụ (định luật Conway)	1
4	**Entity-driven Design** — Thiết kế theo hướng thực thể dữ liệu (1 thực thể = 1 bảng, 1 dịch vụ = 1 nhóm bảng từ sơ đồ ERD)	CAO	Ranh giới dịch vụ bị phụ thuộc vào lược đồ dữ liệu (schema) thay vì miền nghiệp vụ (domain)	Thiết kế xuất phát từ hành vi và sự kiện nghiệp vụ (sử dụng Event Storming), không bắt đầu từ dữ liệu	2
5	**Nano-services** — Tách mỗi thực thể thành một dịch vụ siêu nhỏ mặc dù chúng luôn thay đổi cùng nhau	CAO	Độ trễ tăng cao, gây khó khăn lớn cho việc theo dõi và gỡ lỗi (debugging)	Phân rã theo Bounded Context, không phân rã theo từng thực thể	2
6	**Shared Library Abuse** — Lạm dụng thư viện dùng chung (shared library) vì sự tiện lợi, trộn lẫn logic nghiệp vụ với các nghiệp vụ cắt ngang	TRUNG BÌNH	Tạo ra sự phụ thuộc ngầm khi triển khai; thay đổi thư viện ảnh hưởng toàn bộ hệ thống	Phân định rõ ràng: nghiệp vụ cắt ngang (cross-cutting concern) thì dùng chung, logic nghiệp vụ thì tách biệt	2

D.2. Thiết kế API

Bảng D.2 tóm tắt các anti-pattern về thiết kế API, kèm mức độ, hậu quả và cách phòng tránh.

Bảng D.2: *Anti-pattern thiết kế API*

#	Anti-pattern	Mức độ	Hậu quả	Phòng tránh	Ch.
7	**Exposing Entities via API** — Trả trực tiếp thực thể dữ liệu (entity) qua API mà không qua đối tượng truyền dữ liệu (DTO), làm lộ cấu trúc nội bộ	CAO	Gây ra breaking change khi thay đổi cơ sở dữ liệu và tiềm ẩn rủi ro bảo mật	Luôn dùng DTO để tách biệt mô hình nội bộ (internal model) khỏi hợp đồng giao tiếp (external contract)	3
8	**Inconsistent Naming Convention** — Quy tắc đặt tên (naming convention) không đồng nhất, trộn lẫn số ít/số nhiều và kiểu viết (kebab-case/camelCase)	THẤP	Việc sửa đổi quy tắc đặt tên về sau sẽ gây ra breaking change trên diện rộng cho các consumer	Thống nhất một quy tắc chung và bắt buộc tuân thủ ngay từ những dòng mã đầu tiên	3
9	**Missing Standard Error Format** — Bỏ qua định dạng lỗi chuẩn hóa; mỗi dịch vụ trả về một cấu trúc báo lỗi riêng biệt	TRUNG BÌNH	Consumer phải xử lý quá nhiều định dạng, gây khó khăn cho việc gỡ lỗi	Dùng một bộ xử lý ngoại lệ tập trung, trả về định dạng lỗi thống nhất cho mọi dịch vụ	3

D.3. Giao tiếp

Bảng D.3 tóm tắt các anti-pattern về giao tiếp, kèm mức độ, hậu quả và cách phòng tránh.

Bảng D.3: *Anti-pattern giao tiếp*

#	Anti-pattern	Mức độ	Hậu quả	Phòng tránh	Ch.
10	**Synchronous Service Chain** — Thiết lập chuỗi gọi đồng bộ (A → B → C → D), mỗi dịch vụ phải chờ đợi dịch vụ tiếp theo phản hồi	CAO	Độ trễ cộng dồn, làm giảm tính khả dụng (ví dụ: xin chữ ký 4 phòng ban, một người đi vắng là tắc nghẽn toàn bộ)	Chuyển đổi sang giao tiếp bất đồng bộ (async) cho các luồng không trọng yếu, áp dụng API Composition	4

#	Anti-pattern	Mức độ	Hậu quả	Phòng tránh	Ch.
11	**Missing Circuit Breaker** — Thiếu cơ chế ngắt mạch (Circuit Breaker), tiếp tục gọi đến dịch vụ đang gặp sự cố mà không dừng lại	NGHIÊM TRỌNG	Gây ra hiệu ứng sụp đổ dây chuyền (cascading failure), làm tê liệt toàn bộ hệ thống	Áp dụng thư viện chịu lỗi (ví dụ Resilience4j) kết hợp ngắt mạch, thử lại (retry) và giới hạn thời gian (timeout)	4
12	**Retry Without Backoff** — Thử lại ngay lập tức khi gặp lỗi mà không có độ trễ tăng dần (exponential backoff)	CAO	Dịch vụ đang quá tải bị dồn thêm áp lực, nhanh chóng cạn kiệt tài nguyên và sụp đổ	Áp dụng độ trễ tăng dần kèm độ lệch ngẫu nhiên (exponential backoff + jitter): 1 s → 2 s → 4 s	4
13	**Premature Auto-commit** — Tự động xác nhận (auto-commit) đã nhận thông điệp trước khi tiến trình xử lý hoàn tất	NGHIÊM TRỌNG	Thông điệp bị mất vĩnh viễn nếu consumer gặp sự cố giữa chừng	Xác nhận thủ công (manual offset commit) — chỉ đánh dấu hoàn tất *sau khi* đã xử lý thành công	5
14	**Missing Dead Letter Topic** — Thiếu Dead Letter Topic (DLT) để chứa thông điệp lỗi, dẫn đến việc thử lại vô hạn hoặc gây tắc nghẽn luồng xử lý	CAO	Thông điệp lỗi (Poison Pill) chặn đứng toàn bộ các thông điệp xếp sau nó	Định tuyến các thông điệp lỗi sang một DLT riêng sau số lần thử lại nhất định	5
15	**Non-idempotent Consumer** — Consumer không bảo đảm idempotency, mặc định giả định mỗi thông điệp chỉ đến một lần	CAO	Thông điệp bị lặp (do hệ thống thử lại) dẫn đến sai lệch dữ liệu	Chống trùng nguyên tử theo eventId: unique constraint hoặc CAS, ghi dấu và cập nhật nghiệp vụ trong cùng transaction	5
16	**Familiarity-driven Broker Selection** — Lựa chọn message broker dựa trên thói quen thay vì đặc thù bài toán (ví dụ: dùng queue kiểu tiêu thụ rồi xóa cho bài toán cần phát lại sự kiện, chỉ vì đội ngũ đã quen thuộc)	TRUNG BÌNH	Thiếu tính năng cần dùng (ví dụ phát lại luồng sự kiện), buộc chuyển đổi tốn kém	Đánh giá dựa trên nhu cầu thực tế: lưu vết bền vững (Kafka) hay định tuyến thông minh (RabbitMQ)	5

D.4. Giao dịch & dữ liệu

Bảng D.4 tóm tắt các anti-pattern về giao dịch và dữ liệu, kèm mức độ, hậu quả và cách phòng tránh.

Bảng D.4: *Anti-pattern giao dịch và dữ liệu*

#	Anti-pattern	Mức độ	Hậu quả	Phòng tránh	Ch.
17	**Two-Phase Commit (2PC) in Microservices** — Áp dụng giao thức 2PC để cố gắng duy trì giao dịch phân tán (distributed transaction) xuyên suốt các dịch vụ	CAO	Gây tắc nghẽn (blocking), tạo điểm lỗi tập trung (SPOF) và làm giảm tính khả dụng	Chấp nhận tính nhất quán cuối cùng (eventual consistency) và dùng Saga Pattern	6
18	**Missing Compensating Action** — Bỏ qua việc định nghĩa các hành động bù trừ (compensation), chỉ tập trung vào kịch bản thành công	NGHIÊM TRỌNG	Dữ liệu không nhất quán, các bản ghi bị kẹt trạng thái vĩnh viễn	Mỗi bước có thể bù trừ (compensatable) của Saga phải có hành động bù trừ (compensating action) đã kiểm thử; xác định rõ giao dịch bản lề (pivot) và các bước có thể thử lại (retriable), không bắt mọi bước phải có bù trừ	6

#	Anti-pattern	Mức độ	Hậu quả	Phòng tránh	Ch.
19	**Saga Without Timeout** — Triển khai Saga nhưng thiếu cơ chế giới hạn thời gian (timeout) để xác định thời điểm giao dịch thất bại	CAO	Các yêu cầu xử lý bị kẹt ở trạng thái đang xử lý (`PENDING`) vô thời hạn	Thiết lập tiến trình nền (scheduled job) để kiểm tra quá hạn và tự động kích hoạt bù trừ	6
20	**Missing Semantic Lock** — Không sử dụng khóa ngữ nghĩa (Semantic Lock), cho phép can thiệp vào dữ liệu đang nằm trong một chuỗi Saga	CAO	Gây ra xung đột tương tranh (race condition), đọc dữ liệu bẩn (dirty read) và kết quả sai lệch	Thiết lập trạng thái khóa ngay khi Saga bắt đầu và chỉ gỡ khóa khi chuỗi giao dịch hoàn tất	6
21	**Premature Database Decomposition** — Phân tách cơ sở dữ liệu quá sớm khi chưa nắm rõ các mô hình truy cập dữ liệu (access pattern)	TRUNG BÌNH	Dẫn đến nhu cầu kết nối dữ liệu (JOIN) liên tục, buộc phải xây dựng các luồng sự kiện phức tạp	Bắt đầu bằng việc phân tách lược đồ (schema) trong cùng hệ quản trị, quan sát trước khi tách DB vật lý	7
22	**Overusing CQRS** — Lạm dụng kiến trúc CQRS cho mọi bài toán, kể cả các tác vụ thêm/đọc/sửa/xóa (CRUD) cơ bản	TRUNG BÌNH	Làm tăng độ phức tạp hệ thống lên gấp nhiều lần cho những dữ liệu ít khi thay đổi	Chỉ áp dụng CQRS khi luồng đọc thực sự phức tạp, lưu lượng lớn, hoặc cần kết hợp nhiều dịch vụ	7
23	**Data Duplication Without Source of Truth** — Nhân bản dữ liệu mà không xác định rõ nguồn sự thật duy nhất (Source of Truth)	CAO	Gây xung đột khi nhiều dịch vụ cùng chỉnh sửa; không thể xác định phiên bản dữ liệu nào là chuẩn xác	Mỗi thực thể dữ liệu chỉ được phép có một chủ sở hữu duy nhất — các bản sao đều phải ở dạng chỉ đọc	7
24	**Ignoring Replication Lag** — Bỏ qua độ trễ nhân bản dữ liệu (replication lag): vừa ghi xong chuyển sang đọc ngay lập tức dẫn đến thấy dữ liệu cũ	TRUNG BÌNH	Gây bối rối cho người dùng (ví dụ: sinh viên nộp bài thành công nhưng bảng điểm chưa cập nhật)	Tùy bài toán: "Cập nhật lạc quan" (Optimistic UI) cho luồng đơn giản; với dữ liệu phụ thuộc nhiều nguồn (leaderboard), dùng synchronous read model / read-your-writes	7

D.5. Bảo mật & Gateway

Bảng D.5 tóm tắt các anti-pattern về bảo mật và Gateway, kèm mức độ, hậu quả và cách phòng tránh.

Bảng D.5: *Anti-pattern bảo mật và Gateway*

#	Anti-pattern	Mức độ	Hậu quả	Phòng tránh	Ch.
25	**Bespoke JWT Validation** — Mỗi Gateway/service tự chọn thư viện, thuật toán và quy tắc issuer/audience khác nhau	CAO	Quy tắc xác minh trôi lệch giữa các nơi: nơi chấp nhận, nơi từ chối; bộ xác minh lỏng có thể chấp nhận token không dành cho mình	Chuẩn hóa bộ xác minh, JWKS và hợp đồng xác minh; Gateway kiểm tra ở biên (ingress), resource service xác minh bằng chứng có chữ ký rồi tự phân quyền cục bộ	8–9
26	**Missing Rate Limiting** — Bỏ qua cơ chế giới hạn lưu lượng (Rate Limiting) tại API Gateway	NGHIÊM TRỌNG	Dễ bị tấn công từ chối dịch vụ hoặc quá tải khi lưu lượng tăng đột biến (ví dụ: hàng nghìn sinh viên cùng lúc tải lại trang đăng ký tín chỉ khiến dịch vụ ngừng phản hồi)	Áp dụng cơ chế giới hạn lưu lượng tại Gateway (thuật toán Token Bucket với Redis)	8

#	Anti-pattern	Mức độ	Hậu quả	Phòng tránh	Ch.
27	**Overambitious API Gateway** — Nhồi nhét quá nhiều logic nghiệp vụ và logic biến đổi dữ liệu vào API Gateway	CAO	Gateway biến thành điểm nghẽn (bottleneck); mỗi lần thay đổi đều phải triển khai lại toàn bộ	Gateway chỉ nên đảm nhận các nghiệp vụ cắt ngang: định tuyến, xác minh token ở biên, giới hạn lưu lượng và ghi log	8
28	**Long-lived JWT** — Cấp phát token JWT không có thời hạn hoặc thời hạn quá dài (ví dụ: access token có hiệu lực 30 ngày)	NGHIÊM TRỌNG	Nếu token bị đánh cắp, tin tặc sẽ nắm giữ quyền truy cập hệ thống trong thời gian rất dài	Sử dụng access token có vòng đời ngắn (15–60 phút) kết hợp cùng refresh token để cấp lại	9
29	**Storing JWT in Local Storage** — Lưu trữ token JWT tại kho chứa cục bộ (localStorage) của trình duyệt trên máy khách	NGHIÊM TRỌNG	Nếu có XSS, mã độc JavaScript đọc và đánh cắp được token	HttpOnly cookie kèm Secure, SameSite và chống CSRF (§9.6); vẫn phải phòng XSS	9
30	**Hard-coded Roles** — Gắn cố định (hard-code) các vai trò trực tiếp vào mã nguồn (ví dụ: `if role == "ADMIN"`) rải rác khắp nơi	CAO	Mỗi khi bổ sung vai trò mới, lập trình viên phải sửa đổi mã nguồn ở nhiều nơi, dễ xảy ra sai sót	Dùng RBAC, đặt ánh xạ vai trò sang quyền hạn (permission) ở một chỗ, rồi kiểm tra theo quyền hạn thay vì so sánh tên vai trò rải rác trong mã	9

D.6. Chuyển đổi & observability

Bảng D.6 tóm tắt các anti-pattern về chuyển đổi và observability, kèm mức độ, hậu quả và cách phòng tránh.

Bảng D.6: *Anti-pattern chuyển đổi và observability*

#	Anti-pattern	Mức độ	Hậu quả	Phòng tránh	Ch.
31	**Distributed Monolith** — Phân rã dịch vụ nhưng vẫn dùng chung cơ sở dữ liệu, thư viện và phải triển khai đồng loạt	NGHIÊM TRỌNG	Gánh chịu toàn bộ độ phức tạp của microservices nhưng không thu được lợi ích (ví dụ: các khoa độc lập nhưng dùng chung một máy in)	Áp dụng cơ sở dữ liệu riêng cho từng dịch vụ và chỉ dùng chung các thư viện tối giản	10
32	**Big Bang Rewrite** — Đập bỏ xây mới: viết lại toàn bộ hệ thống từ đầu thay vì chuyển đổi dần dần từng phần	CAO	Tốn kém nguồn lực bảo trì cả hai hệ thống song song; thời điểm chuyển giao (switch day) liên tục bị trì hoãn	Áp dụng mẫu thiết kế Strangler Fig — bóc tách từng phần nhỏ và đưa vào sử dụng ngay	10
33	**Rushed Migration** — Chuyển đổi quá vội vã (ví dụ: tách 20 dịch vụ trong 2 tháng chỉ để thỏa mãn "quyết định kiến trúc")	CAO	Xác định sai ranh giới hệ thống, dẫn đến phải thiết kế, xây dựng lại và tiêu tốn gấp đôi công sức	Chuyển đổi từng dịch vụ một, bắt đầu từ những cấu phần mang lại giá trị nghiệp vụ cao nhất	10
34	**Lift-and-shift Without Redesign** — Chia các phần của monolith thành nhiều đơn vị triển khai qua mạng mà vẫn giữ coupling, không thiết kế lại (container hóa nguyên khối thì vẫn là bước chuyển tiếp hợp lệ)	CAO	Phát sinh độ trễ mạng và các rủi ro vận hành trong khi hệ thống vẫn bị phụ thuộc chặt chẽ vào nhau	Thiết kế lại mô hình nghiệp vụ cho từng dịch vụ và sử dụng Anti-Corruption Layer (lớp dịch mô hình)	10
35	**Infrastructure Over-engineering** — Phức tạp hóa hạ tầng quá mức cần thiết (ví dụ: dựng Kubernetes + Istio phức tạp chỉ cho 3 dịch vụ nhỏ)	TRUNG BÌNH	Đội ngũ tiêu tốn phần lớn thời gian để bảo trì hạ tầng thay vì tập trung phát triển các tính năng cốt lõi	Khởi đầu đơn giản với Docker Compose khi một máy đáp ứng được tải, và chỉ mở rộng quy mô hạ tầng khi thực sự cần	10

#	Anti-pattern	Mức độ	Hậu quả	Phòng tránh	Ch.
36	**Error-only Logging** — Chỉ ghi nhận nhật ký (log) khi có lỗi xảy ra, bỏ qua việc ghi nhận các luồng yêu cầu thông thường	CAO	Khi sự cố xảy ra, kỹ sư không có đủ dữ liệu bối cảnh (context) để tiến hành gỡ lỗi	Ghi nhật ký có cấu trúc (structured logging) cho mọi yêu cầu kèm các trường liên kết (`requestId`, `traceId`, `correlationId`)	11
37	**Metric Overload or Starvation** — Thu thập số liệu (metric) quá dư thừa (gây nhiễu) hoặc quá nghèo nàn (chỉ đo CPU/RAM)	TRUNG BÌNH	Dư thừa gây ra tình trạng nhiễu loạn cảnh báo. Nghèo nàn tạo ra các điểm mù trong hệ thống	Tập trung vào các chỉ số trọng tâm (SLI/SLO) như: độ trễ, tỷ lệ lỗi (error rate), thông lượng (throughput)	11
38	**Alert Fatigue** — Cảnh báo tràn lan, mọi exception đều kích hoạt thông báo gửi qua email hoặc Slack	TRUNG BÌNH	Gây ra tình trạng mệt mỏi vì cảnh báo (alert fatigue), khiến đội ngũ bỏ lỡ các sự cố nghiêm trọng thực sự	Chỉ cấu hình cảnh báo dựa trên các *triệu chứng* ảnh hưởng trực tiếp đến người dùng (vi phạm SLO)	11
39	**Broken Correlation ID** — Đứt gãy chuỗi định danh (Correlation ID), dữ liệu truy vết không được truyền qua các luồng bất đồng bộ	CAO	Bức tranh truy vết phân tán bị đứt đoạn, khiến việc theo dõi yêu cầu xuyên suốt hệ thống trở nên bất khả thi	Truyền `traceparent` qua Kafka header, `correlationId` trong event envelope, và đưa cả hai vào MDC khi xử lý	11

D.7. Triển khai & DevOps

Bảng D.7 tóm tắt các anti-pattern về triển khai và DevOps, kèm mức độ, hậu quả và cách phòng tránh.

Bảng D.7: *Anti-pattern triển khai và DevOps*

#	Anti-pattern	Mức độ	Hậu quả	Phòng tránh	Ch.
40	**Friday Deployment** — Thực hiện triển khai (deploy) vào lúc đội ngũ không sẵn sàng ứng cứu (ví dụ: cập nhật tính năng chiều thứ Sáu / cuối tuần)	CAO	Nếu xảy ra sự cố sập hệ thống (downtime) khi không có nhân sự trực, hậu quả sẽ kéo dài đến đầu tuần	Nên ưu tiên triển khai vào các buổi sáng giữa tuần, khi toàn bộ đội ngũ đang ở trạng thái sẵn sàng xử lý sự cố	12
41	**Kubernetes for Trivial Workloads** — Áp dụng Kubernetes cho các hệ thống quy mô nhỏ, đơn giản với tư duy "Các công ty lớn dùng thì chúng ta cũng nên dùng"	TRUNG BÌNH	Gây ra gánh nặng vận hành rất lớn cho các đội ngũ quy mô nhỏ (2–3 thành viên)	Sử dụng Docker Compose khi một máy đáp ứng được tải. Chỉ dùng Kubernetes khi *thực sự cần* phân tán đa máy chủ	12
42	**Environment-specific Image Builds** — Đóng gói các bản dựng (image) khác nhau cho từng môi trường riêng biệt (dev/staging/prod)	CAO	Gây ra lỗi môi trường (chạy tốt trên staging nhưng lỗi trên production) do sự sai khác về bản dựng	Xây dựng một lần, triển khai mọi nơi (Build once, deploy everywhere) — tách biệt cấu hình qua biến môi trường	12
43	**Missing Rollback Plan** — Thiếu phương án hoàn tác (rollback plan): khi phiên bản mới gặp lỗi nghiêm trọng, không có quy trình khôi phục rõ ràng	NGHIÊM TRỌNG	Gây hoảng loạn, dẫn đến việc chỉnh sửa trực tiếp trên production và sinh ra thêm lỗi mới	Mọi đợt triển khai bắt buộc phải có quy trình hoàn tác (rollback procedure) được ghi chép rõ ràng	12

PHỤ LỤC E

Case study KBM

Phụ lục này trình bày đầy đủ về **KBM** — case study xuyên suốt cuốn sách. Nội dung đi từ bối cảnh bài toán ban đầu, qua quá trình tiến hóa kiến trúc kéo dài gần một thập kỷ, đến kiến trúc microservices tại thời điểm biên soạn cùng những trade-off và sự cố đã xảy ra trong thực tế.

Mục đích không chỉ minh họa lý thuyết, mà cho thấy kiến trúc phần mềm là *kết quả của một chuỗi quyết định trong ràng buộc thực tế* — không phải bản thiết kế lý tưởng trên giấy. Mỗi chương trong sách đều liên hệ với ít nhất một khía cạnh của KBM; phụ lục này tập hợp toàn bộ bức tranh để người đọc có thể đối chiếu và rút ra bài học riêng.

E.1. Bối cảnh bài toán

Tại một trường đại học kỹ thuật, các môn học về cơ sở dữ liệu, lập trình mạng, và lập trình thuật toán đòi hỏi sinh viên thực hành với khối lượng lớn bài tập — hàng trăm bài SQL mỗi tuần, hàng chục bài lập trình mạng Java, và các kỳ thi lập trình trực tuyến. Quy trình truyền thống (sinh viên nộp file, giảng viên chấm thủ công) nhanh chóng trở thành nút thắt khi quy mô tăng lên hàng nghìn sinh viên mỗi học kỳ.

E.1.1. Sáu vấn đề cần giải quyết

1. **Chấm thủ công không khả thi.** Giảng viên không thể thực thi và kiểm tra kết quả hàng trăm bài SQL trên nhiều hệ quản trị cơ sở dữ liệu (DBMS) khác nhau trong thời gian hợp lý. Mỗi bài tập cần chạy trên MySQL, PostgreSQL, SQL Server, hoặc Oracle — mỗi hệ có cú pháp và hành vi riêng.
2. **Phản hồi chậm.** Sinh viên thường phải chờ vài ngày mới nhận được kết quả, làm giảm hiệu quả học tập. Vòng lặp *viết — nộp — chờ — sửa* kéo dài thay vì phản hồi tức thì.
3. **Đa dạng CSDL.** Chương trình đào tạo yêu cầu thực hành trên nhiều DBMS. Tại thời điểm đó, nhóm không tìm được giải pháp tự vận hành được (nhất là mã nguồn mở) hỗ trợ đủ MySQL, PostgreSQL, SQL Server và Oracle trong cùng một nền tảng cho nhu cầu giảng dạy — đây là khoảng trống KBM phải tự lấp.

4. **Thiếu môi trường sandbox.** Sinh viên cần chạy mã SQL trực tiếp trên cơ sở dữ liệu thật để kiểm tra kết quả, nhưng không thể cho phép truy cập tự do vào hệ thống production — cần cơ chế cách ly (sandbox) ở mức container.
5. **Rủi ro gian lận thi cử.** Các kỳ thi trực tuyến trên phòng máy dễ bị sao chép kết quả giữa các máy nếu không có cơ chế giám sát và khóa môi trường (lockdown) tại máy khách.
6. **Quản lý phân tán.** Thông tin khóa học, bài tập, điểm số, và lịch thi nằm rải rác trên email, bảng tính Excel, và Google Drive — không có hệ thống tập trung, dẫn đến mất đồng bộ và khó tra cứu, theo dõi.

E.1.2. Nhóm người dùng

KBM phục vụ ba nhóm chính: **sinh viên** (hàng nghìn mỗi học kỳ — thực hành, thi, nộp bài tập, đồ án), **giảng viên** (vài chục — tạo đề, tổ chức thi, chấm điểm, theo dõi tiến độ), và **quản trị viên** (vài người — quản lý tài khoản, cấu hình hệ thống, giám sát vận hành).

E.1.3. Ràng buộc thực tế

Bối cảnh phát triển KBM chịu những ràng buộc đáng kể — chính các ràng buộc này định hình mọi quyết định kiến trúc:

- **Nhân sự hạn chế:** Nhóm phát triển chính chỉ 1–2 người, cộng thêm sinh viên hỗ trợ theo từng đợt đồ án.
- **Hạ tầng giới hạn:** Chỉ có vài máy chủ ảo (VPS), ngân sách phần cứng hạn chế — không có môi trường cloud hay cluster lớn.
- **Phát triển song song với giảng dạy:** Hệ thống được xây dựng *trong khi vẫn phục vụ giảng dạy thực tế*, không có giai đoạn "đóng cửa để xây lại".
- **Tích hợp hệ thống cũ:** Phải kết nối với hệ thống quản lý đào tạo (QLĐT) của trường — một hệ thống legacy không có tài liệu API chính thức.

Ý nghĩa kiến trúc

Những ràng buộc này giải thích tại sao nhiều quyết định kiến trúc trong KBM là *trade-off có chủ đích* thay vì thiết kế lý tưởng. Ví dụ: dùng shared database thay vì database-per-service (xem §E.5) không phải vì thiếu hiểu biết, mà vì nhóm nhỏ chưa cần tách máy chủ database; nhóm vẫn cần quy ước chủ sở hữu ghi, migration owner và ranh giới schema trong cùng PostgreSQL (Chương 7). Đây là thực tế mà sách kiến trúc ít khi nói tới.

E.2. Bốn trụ cột nghiệp vụ

KBM được tổ chức quanh bốn trụ cột nghiệp vụ chính, mỗi trụ cột tương ứng với một nhóm Bounded Context riêng biệt. Ngoài ra, hệ thống còn hai thành phần bổ trợ quan trọng.

Bảng E.1: *Bốn trụ cột nghiệp vụ KBM và yêu cầu kiến trúc đặc trưng*

Trụ cột	Chức năng chính	Yêu cầu kiến trúc đặc trưng
SQL Lab	Chấm SQL tự động trên 4 DBMS, 2 chế độ (luyện tập đồng bộ + thi bất đồng bộ), phản hồi AI	Sandbox cách ly mỗi DBMS, xử lý bất đồng bộ qua message broker khi thi
Network Lab	Chấm lập trình mạng Java: 6 nhóm giao thức (TCP → gRPC), ~20 cổng	Standalone Java runtime, plugin system, khởi động nhanh
Course Management	Quản lý khóa học, bài tập, MCQ, đồ án, điểm danh, GitHub Classroom	CRUD + tích hợp bên ngoài (QLĐT, GitHub API)

Trụ cột	Chức năng chính	Yêu cầu kiến trúc đặc trưng
DevOps Lab	Thực hành K8s/Docker: container-in-container, web terminal, chấm tự động	Ngôn ngữ Go, k3s + Sysbox, wildcard DNS

E.2.1. SQL Lab — chấm SQL tự động

SQL Lab là trụ cột phức tạp nhất về mặt kiến trúc. Các online judge phổ biến được thiết kế cho ngôn ngữ lập trình truyền thống (compile → run → compare output), trong khi SQL không tuân theo mô hình này. KBM giải quyết bằng cách: *thực thi câu SQL trên sandbox CSDL thật, so sánh kết quả qua băm SHA-256 của result set*.

Hệ thống hỗ trợ hai chế độ vận hành:

- **Chế độ luyện tập (Practice Mode, đồng bộ):** Sinh viên gửi câu SQL → hệ thống thực thi trên sandbox tương ứng (MySQL, PostgreSQL, SQL Server, hoặc Oracle) → so sánh kết quả → trả phản hồi tức thì. Bài luyện tập nhỏ thường có kết quả trong vài giây; bài nặng hoặc đề chạy trên nhiều DBMS mất 5–30 giây (§4.1). Tích hợp phản hồi bằng AI (GPT-4o-mini) để giải thích lỗi logic.
- **Chế độ thi (Contest Mode, bất đồng bộ qua Kafka):** Trong kỳ thi, hàng trăm sinh viên gửi bài đồng thời. Hệ thống đẩy bài nộp vào Kafka topic → coordinator `kbm-judge-sql` tiêu thụ và phân bổ cho các worker `kbm-sandbox-*` chấm → kết quả đẩy lại qua WebSocket. Bước đưa vào hàng đợi (enqueue) rất nhanh so với thời gian chấm (mục tiêu P99 dưới 500 ms, §1.5), sinh viên không bị chặn chờ kết quả.

Kết quả chấm (verdict) bao gồm: AC (Accepted), WA (Wrong Answer), TLE (Time Limit Exceeded), RTE (Runtime Error), CE (Compilation Error) — cùng hệ thống phán định với các online judge truyền thống.

Cơ chế sandbox cách ly dữ liệu bằng tiền tố bảng (table prefix), kết hợp sắp xếp topo (topological sort) để thiết lập schema đúng thứ tự phụ thuộc.

E.2.2. Network Lab — chấm lập trình mạng

Network Lab phục vụ thực hành lập trình mạng Java, bao gồm 6 nhóm giao thức — từ socket cơ bản đến gọi thủ tục từ xa hiện đại:

1. **TCP** — 6 loại (echo, file transfer, multi-thread, v.v.)
2. **UDP** — 3 loại (datagram, multicast, broadcast)
3. **RMI** — 4 loại (registry lookup, callback, dynamic proxy, security policy)
4. **SOAP** — Web service cổ điển
5. **REST** — HTTP API
6. **gRPC** — Framework RPC hiệu năng cao

Điểm đặc biệt: sinh viên viết chương trình Java client trên IDE cá nhân, kết nối tới judge server qua giao thức tương ứng để nhận dữ liệu, xử lý và gửi đáp án. Judge là ứng dụng *standalone Java* (không dùng Spring Boot), dùng cơ chế *dynamic handler reloading* — một dạng plugin system cho phép tải lại handler của câu hỏi mà không cần khởi động lại judge service. Hệ thống phân bổ ~20 cổng mạng riêng biệt cho các nhóm giao thức.

E.2.3. Course Management — quản lý đào tạo

Trụ cột này bao quát các nghiệp vụ quản lý đào tạo truyền thống, nhưng với tích hợp sâu vào hệ sinh thái KBM:

- **Quản lý khóa học:** Tạo, cấu hình, phân quyền giảng viên.

- **Bài tập:** Hỗ trợ cả bài tập cá nhân và nhóm, định dạng rubric chấm điểm.
- **Thi trắc nghiệm (MCQ):** Ngân hàng câu hỏi, tạo đề ngẫu nhiên, chấm tự động.
- **Đồ án tốt nghiệp:** Quy trình 2 cấp duyệt → bảo vệ → chấm rubric trước hội đồng.
- **Điểm danh:** Theo buổi học (session-based), tích hợp lịch học.
- **Cơ cấu điểm (Grade Scheme) và CLO:** Cấu trúc điểm theo thành phần, ánh xạ chuẩn đầu ra (CLO — Course Learning Outcome).
- **GitHub Classroom:** Đồng bộ bài tập, liên kết danh sách lớp (roster), theo dõi tiến độ commit.

E.2.4. DevOps Lab — thực hành DevOps/Kubernetes

DevOps Lab — trụ cột mới nhất — được phát triển bằng **Go** thay vì Java, một quyết định có chủ đích:

- **Binary nhỏ gọn:** ~15 MB (Go binary) so với 200 MB+ (Spring Boot JAR + JRE).
- **Khởi động nhanh:** Mili-giây thay vì vài giây.
- **Goroutine:** Mô hình xử lý song song gọn nhẹ, giúp reverse proxy phục vụ hiệu quả nhiều kết nối đồng thời.
- **client-go SDK:** Tương tác native với Kubernetes API.

Kiến trúc sử dụng **mono-repo multi-binary**: một repository Go sinh ra 3 binary riêng biệt (`api`, `router`, `grader`). Hạ tầng dựa trên **k3s** (Kubernetes nhẹ) kết hợp **Sysbox** — runtime cho phép container-in-container an toàn mà không cần chế độ đặc quyền (privileged). Mỗi sinh viên nhận một *Lab Pod* cách ly với TTL (Time-To-Live) tự động thu hồi tài nguyên.

E.2.5. Thành phần bổ trợ

Ngoài bốn trụ cột chính, KBM có hai thành phần bổ trợ quan trọng:

Code Judge (`kbm-judge-code`) — Chấm thuật toán đa ngôn ngữ (C/C++/Java). Thành phần này thể hiện rõ quyết định "xây hay mua" (build vs buy): thế hệ đầu tiên (Gen 1) được phát triển nội bộ nhưng **không có sandbox** — mã sinh viên chạy trực tiếp trên server, tạo rủi ro bảo mật nghiêm trọng. Thế hệ thứ hai (Gen 2) chuyển sang tích hợp **DOMjudge** — hệ thống chấm mã nguồn mở chuẩn ICPC, cung cấp sandbox đầy đủ. Quyết định này minh họa bài học xây hay mua trong Chương 3 và Chương 10.

Desktop Exam Client (`kbm-exam-client`) — Ứng dụng máy tính để bàn dành cho phòng máy thi, thực hiện *lockdown* môi trường: chặn ứng dụng ngoài, giới hạn IP cho phép (whitelist), và giám sát (proctoring) tại máy khách. Client sử dụng chung JWT token với `kbm-auth`, đảm bảo nhất quán xác thực.

E.3. Quá trình tiến hóa — từ Monolith đến Microservices

KBM không được thiết kế microservices ngay từ đầu. Hệ thống hiện tại là kết quả của gần một thập kỷ tiến hóa, qua 5 giai đoạn (era) — mỗi giai đoạn bắt đầu bằng một vấn đề thúc đẩy (trigger) và kết thúc bằng một bài học kiến trúc.

E.3.1. Giai đoạn 1: Independent Silo Monoliths (2016–2022)

Vấn đề thúc đẩy: Nhu cầu ban đầu về chấm bài lập trình và quản lý đào tạo.

Giai đoạn đầu tiên không phải là một monolith duy nhất, mà là *ba hệ thống MVC rời rạc* — ba silo độc lập, mỗi silo là một monolith:

1. **Hệ thống quản lý đào tạo** — Monolith ASP.NET MVC, quản lý khóa học, bài tập, điểm số, tài khoản người dùng.

2. **Code Judge (Gen 1)** — Hệ thống chấm thuật toán C/C++ và Java, tự phát triển. Đặc điểm: *không có sandbox* — mã sinh viên chạy trực tiếp trên server.
3. **Network Judge** — Monolith MVC riêng biệt cho chấm lập trình mạng TCP/UDP/RMI.

Ba hệ thống hoạt động độc lập: không chia sẻ dữ liệu, không chia sẻ danh tính người dùng, không giao tiếp với nhau. Thomas Erl gọi đây là kiến trúc *silo-based* — mỗi ứng dụng là một "ốc đảo" với logic trùng lặp và chi phí bảo trì cao.

Bài học: Ba hệ thống rời rạc (independent silos) không phải distributed monolith — chúng không bị ràng buộc đồng loạt (lockstep) mà hoàn toàn cô lập. Tuy nhiên, sự cô lập này cũng không mang lại lợi ích của kiến trúc phân tán: logic trùng lặp, danh tính người dùng không liên thông, và chi phí bảo trì *nhân lên theo số hệ thống* thay vì được chia sẻ.

E.3.2. Giai đoạn 2: SQL Judge — sáng tạo từ ràng buộc (2023)

Vấn đề thúc đẩy: Nhu cầu chấm SQL tự động — nhóm không tìm được sản phẩm tự vận hành (self-hosted) nào đáp ứng đủ.

Đây là cột mốc quan trọng nhất trong lịch sử phát triển KBM. Các online judge phổ biến được thiết kế cho ngôn ngữ lập trình (compile → run → compare), nhưng SQL không tuân theo mô hình này — không có stdin/stdout, kết quả là *result set* phụ thuộc vào dữ liệu mẫu.

Giải pháp: thực thi câu SQL trên sandbox CSDL thật (mỗi DBMS một container riêng), so sánh result set bằng băm SHA-256. Cơ chế cách ly sử dụng tiền tố bảng kết hợp sắp xếp topo cho các phụ thuộc của schema. Hệ thống hỗ trợ đồng thời 4 DBMS.

Bài học: Đổi mới thường sinh ra từ ràng buộc. Khi không có giải pháp có sẵn, việc thiết kế từ đầu (greenfield) buộc phải suy nghĩ sâu về bài toán.

E.3.3. Giai đoạn 3: Thiết kế Microservices (2024)

Vấn đề thúc đẩy: Sự phức tạp gia tăng khi tích hợp SQL Judge vào hệ thống hiện có.

Kiến trúc microservices được áp dụng, nhưng không phải bằng một cuộc chuyển đổi Big Bang (viết lại toàn bộ). Thay vào đó, KBM phát triển theo kiểu *organic growth* (tăng trưởng hữu cơ): thiết kế microservices cho SQL Judge mới, rồi dần dần tách các thành phần cũ.

Dấu hiệu của sự phát triển hữu cơ: `spring.application.name = "app"` từng giống nhau giữa `kbm-core` và `kbm-academic` — bằng chứng rằng hai service này từng là một (Chương 4). Shared database `core_db` là *accepted transitional debt* (nợ kỹ thuật chuyển tiếp có chủ đích): đội ngũ nhỏ, cùng chu kỳ triển khai, ưu tiên tốc độ phát triển hơn tách biệt hoàn toàn.

Bài học: Microservices trong thực tế hiếm khi bắt đầu "sạch". Sự tiến hóa hữu cơ để lại dấu vết (architectural debt), và nhận diện được các dấu vết này là bước đầu tiên để cải thiện (xem chi tiết tại §E.5).

E.3.4. Giai đoạn 4: Chuyển đổi và mở rộng (2025)

Vấn đề thúc đẩy: Nhu cầu tách biệt nghiệp vụ đang chồng chéo và mở rộng tính năng mới.

Giai đoạn này đánh dấu loạt thay đổi lớn:

- **Tách service:** `kbm-academic` được tách ra từ `kbm-core`, tuy vẫn dùng chung database (shared DB, giai đoạn chuyển tiếp).
- **Kafka:** Triển khai message broker cho chế độ thi, giải quyết bài toán giờ cao điểm — hàng trăm lượt nộp mỗi phút vào cuối giờ thi.

- **Network Judge mở rộng:** TCP/UDP/RMI → thêm SOAP → REST → gRPC, phản ánh thay đổi chương trình đào tạo.
- **Xây → Mua:** Code Judge Gen 1 (tự phát triển, không sandbox) → Gen 2 (DOMjudge, mã nguồn mở chuẩn ICPC).
- **Desktop Exam Client:** Ứng dụng lockdown cho phòng máy thi.
- **OAuth2 mở rộng:** Thêm Microsoft OAuth2, xác thực email, Cloudflare Turnstile.
- **Observability:** Metric Prometheus, định danh tương quan / truy vết (chưa xuyên suốt — xem trade-off tại §E.5).

Bài học: Chuyển đổi không phải sự kiện một lần mà là quá trình liên tục. Mỗi bước chuyển đổi phải mang lại giá trị ngay lập tức, không chờ đến khi "chuyển đổi xong".

E.3.5. Giai đoạn 5: Polyglot — DevOps Lab (2025–2026)

Vấn đề thúc đẩy: Tài nguyên VPS hạn chế, Java quá nặng cho workload proxy/grading.

Quyết định sử dụng Go thay vì Java cho DevOps Lab là minh chứng cho nguyên tắc *right tool for the right job*. Với hạ tầng giới hạn, binary Go 15 MB khởi động trong mili-giây là lựa chọn thực dụng hơn Spring Boot JAR 200 MB+.

Kiến trúc mono-repo multi-binary (1 repo → 3 binary: `api`, `router`, `grader`) khác biệt hoàn toàn so với multi-repo microservices của LMS Java. Sự khác biệt này minh họa rằng "microservices" không đồng nghĩa với "mỗi service một repo" — cấu trúc repo phụ thuộc vào ngữ cảnh đội ngũ và ngôn ngữ.

Bài học: Polyglot architecture (đa ngôn ngữ) mang lại lợi thế kỹ thuật, nhưng đòi hỏi đội ngũ phải thành thạo nhiều ngôn ngữ — một chi phí phải được đánh giá rõ ràng (xem trade-off tại §E.5).

Dòng thời gian tổng quan

2016: Independent Silo Monoliths (3 MVC) → **2023:** SQL Judge (cột mốc sáng tạo) → **2024:** Microservices (organic growth) → **2025:** Chuyển đổi + Kafka + DOMjudge → **2025–2026:** Polyglot Go DevOps Lab. Tổng cộng khoảng 10 năm phát triển, microservices chính thức từ 2024.

E.4. Kiến trúc hiện tại

E.4.1. Bounded Context

Kiến trúc KBM tổ chức theo 9 Bounded Context (BC), gồm 7 core và 2 supporting:

1. **Identity & Access** (`kbm-auth`) — Quản lý danh tính, RBAC, JWT, OAuth2 (Google/Microsoft), SSO.
2. **SQL Practice** (`kbm-core` + `kbm-judge-sql` + các worker `kbm-sandbox-*`) — Chấm SQL trên 4 DBMS.
3. **Network Practice** (`kbm-judge-network`) — Chấm lập trình mạng 6 nhóm giao thức.
4. **Code Practice** (`kbm-judge-code`) — Chấm thuật toán, tích hợp DOMjudge.
5. **Academic Management** (`kbm-academic`) — Quản lý khóa học, bài tập, MCQ, đồ án, điểm danh.
6. **Examination & Proctoring** (logic trong `kbm-core`, mục tiêu tách thành `kbm-exam`) — Quản lý kỳ thi, chống gian lận, ứng dụng khóa môi trường thi (`kbm-exam-client`).
7. **DevOps Practice** (`kbm-devops-api`, `kbm-devops-infra`, `kbm-devops-web`) — Nền tảng thực hành DevOps bằng Go.
8. **Communication** (`kbm-notification`) — Supporting BC: đẩy thông báo thời gian thực qua WebSocket, tiêu thụ sự kiện từ Kafka.

9. **Platform / Infrastructure** (`kbm-gateway`, `kbm-registry`, `kbm-shared`) — Supporting BC: Gateway, service discovery, thư viện dùng chung (shared library).

⚠ God Service

`kbm-core` hiện chứa cả SQL Practice (BC #2) lẫn Examination (BC #6) — đây là ví dụ điển hình của anti-pattern *God Service* (xem §E.5). Lưu ý rằng `kbm-core` phát triển hữu cơ từ giai đoạn 3, khi ranh giới BC chưa được xác định rõ. Mục tiêu tương lai là tách phần thi cử thành `kbm-exam` riêng biệt.

E.4.2. Công nghệ sử dụng

KBM là hệ thống **polyglot** — hai nền tảng công nghệ khác nhau cho hai nhóm workload:

Bảng E.2: *So sánh tech stack giữa LMS chính (Java) và DevOps Lab (Go)*

Khía cạnh	LMS chính (Java)	DevOps Lab (Go)
Ngôn ngữ	Java 17 + Spring Boot 3.x	Go 1.x + Chi v5
Kiến trúc repo	Multi-repo microservices (hơn 10 repo)	Mono-repo multi-binary (1 repo → 3 binary)
Database	PostgreSQL + MySQL + SQL Server (JPA)	PostgreSQL (sqlx raw SQL)
Auth	Spring Security + JWT	Tại thời điểm khảo sát: shared JWT từ `kbm-auth` (HS256 shared secret)
Runtime	Docker Compose	k3s + Sysbox
Công cụ migration	JPA auto-DDL	golang-migrate
Frontend	2 ứng dụng (React + TS + Vite)	1 ứng dụng (kế thừa stack LMS)

Bảo mật của case study tại thời điểm khảo sát (đầu năm 2026): KBM dùng Gateway edge validation, `X-User-*` dẫn xuất và HS256 shared secret; một số service tự phân tích JWT. Đây là rủi ro được chấp nhận (accepted risk) kèm trôi lệch triển khai (implementation drift), không phải kiến trúc mục tiêu. Mục tiêu Tier 1 là RS256/JWKS, signed proof đúng audience tại mỗi resource service và phân quyền cục bộ; luồng nhạy cảm bổ sung workload identity cùng subject/actor/delegation theo Tier 2.

E.4.3. Danh mục dịch vụ

Bảng E.3 liệt kê các dịch vụ của KBM theo Bounded Context.

Bảng E.3: *Danh sách dịch vụ KBM theo Bounded Context*

Tên service kỹ thuật	Bounded Context	Mô tả vai trò
`kbm-auth`	Identity & Access	Quản lý danh tính người dùng, đăng nhập, phân quyền (RBAC), cấp phát và xác minh JWT, OAuth2 (Google/Microsoft), SSO.
`kbm-core`	SQL Practice + Examination	Quản lý kỳ thi, bài tập SQL, bài nộp, chống gian lận và thống kê. Đây là dịch vụ ôm đồm (God Service) tạm thời chứa cả logic thi cử (mục tiêu tương lai là tách rời thành `kbm-exam`).
`kbm-judge-sql`	SQL Practice	Điều phối chấm SQL: nhận thông điệp từ Kafka, phân bổ cho các sandbox tương ứng với từng hệ quản trị CSDL, tích hợp tính năng phản hồi tự động bằng trí tuệ nhân tạo (AI feedback).
`kbm-sandbox-mysql`, `kbm-sandbox-postgres`, `kbm-sandbox-sqlserver`, `kbm-sandbox-oracle`	SQL Practice	Bốn worker chạy mã SQL trong môi trường cách ly trên từng container CSDL — mỗi DBMS một worker riêng.

Tên service kỹ thuật	Bounded Context	Mô tả vai trò
`kbm-judge-network`	Network Practice	Chấm điểm lập trình mạng Java (TCP/UDP/RMI/SOAP/REST/gRPC), hỗ trợ môi trường thực thi Java độc lập (standalone Java) và cơ chế tải lại động trình xử lý (dynamic handler reloading).
`kbm-judge-code`	Code Practice	Chấm điểm thuật toán đa ngôn ngữ (C/C++/Java…). Thế hệ đầu tiên được phát triển nội bộ không có môi trường cách ly (sandbox); thế hệ thứ hai tích hợp DOMjudge để cung cấp môi trường cách ly mã nguồn mở.
`kbm-exam-client`	Examination & Proctoring	Ứng dụng máy tính để bàn (desktop) dành cho phòng máy: khóa môi trường thi (lockdown), danh sách IP cho phép, giám sát thi (proctoring) tại máy khách; sử dụng chung JWT của `kbm-auth`.
`kbm-academic`	Academic Management	Quản lý khóa học, bài tập dạng rubric, điểm danh, MCQ, syllabus/CLO, GitHub Classroom, đồ án tốt nghiệp.
`kbm-notification`	Communication	Đẩy thông báo thời gian thực (kết quả chấm bài) qua WebSocket; là consumer của Kafka.
`kbm-gateway`	Platform / Infra	Gateway của LMS Java (DevOps Lab có reverse proxy riêng, định tuyến theo hostname): định tuyến (path-based), giới hạn lưu lượng (rate limiting), xác minh JWT ở tầng biên (edge validation), cấu hình CORS.
`kbm-registry`	Platform / Infra	Dịch vụ đăng ký (Service registry) sử dụng Netflix Eureka để hỗ trợ Service Discovery giữa các ứng dụng Java.
`kbm-shared`	Platform / Infra (Shared Library)	Các thành phần dùng chung: thực thể cơ sở, đối tượng truyền dữ liệu (DTO), các tiện ích JWT, xử lý ngoại lệ tập trung (xem đánh giá trade-off của việc chia sẻ mã nguồn ở §2.5).
`kbm-devops-api`, `kbm-devops-infra`, `kbm-devops-web`	DevOps Practice	Nền tảng thực hành DevOps/Kubernetes được phát triển bằng Go với cấu trúc kho lưu trữ đơn (mono-repo multi-binary), bao gồm: API, máy chủ proxy đảo ngược (reverse proxy), bộ chấm điểm (grader), k3s và Sysbox.

Quy ước

Xuyên suốt các chương, để đảm bảo tính thực tiễn khi ánh xạ từ sơ đồ kiến trúc xuống mã nguồn hoặc cấu hình triển khai (Docker/Kubernetes), sách sử dụng trực tiếp các định danh kỹ thuật có tiền tố `kbm-*` này. Khi tự xây dựng các hệ thống microservices, đội ngũ phát triển cũng nên duy trì một danh mục dịch vụ (service catalog) tương tự để giữ một ngôn ngữ chung (Ubiquitous Language) giữa các nhóm phát triển.

E.4.4. Tích hợp hệ thống bên ngoài

KBM tích hợp với nhiều hệ thống bên ngoài theo ba kiểu khác nhau, mỗi kiểu đặt ra thách thức kiến trúc riêng:

Bảng E.4: *Ba kiểu tích hợp hệ thống bên ngoài trong KBM*

Hệ thống	Kiểu tích hợp	Đặc điểm và thách thức
QLĐT của trường	Legacy — không có tài liệu API	Phải tự tìm hiểu API vì không có tài liệu. Cần Anti-Corruption Layer (ACL). Rủi ro cao: API có thể thay đổi không báo trước.
GitHub Classroom	API có tài liệu	REST API chuẩn, OAuth2 App. Tích hợp: đồng bộ bài tập, liên kết danh sách lớp (roster), theo dõi tiến độ.

Hệ thống	Kiểu tích hợp	Đặc điểm và thách thức
Google / Microsoft OAuth2	Giao thức chuẩn	OpenID Connect chuẩn. Thách thức: gộp danh tính (identity merge) khi cùng một người dùng đăng nhập qua nhiều provider (xem Sự cố 4, §E.6).

E.4.5. Mô hình triển khai

Hai cụm triển khai đảm nhận các thành phần cốt lõi:

- **Docker Compose đơn máy (single-host):** Chạy các Java microservices và các thành phần dùng chung. Đây là đánh đổi tính sẵn sàng cao (high availability) lấy sự đơn giản trong vận hành.
- **k3s cluster:** Chạy DevOps Lab (các service Go) và các lab pod của sinh viên. Cluster k3s sử dụng ResourceQuota và LimitRange để giới hạn tài nguyên mỗi lab pod.

E.5. Trade-off và khoảng trống — bài học từ thực tế

Mỗi hệ thống phần mềm thực tế đều chứa trade-off — những quyết định *chấp nhận hạn chế ở khía cạnh này để đạt được lợi ích ở khía cạnh khác*. Với KBM, các trade-off không phải điều xấu hổ cần che giấu, mà là chất liệu tốt cho việc học kiến trúc: chúng ta học được nhiều hơn từ một quyết định "đủ tốt trong bối cảnh" so với một thiết kế lý tưởng trên lý thuyết.

Dưới đây là 25 trade-off của KBM tại thời điểm khảo sát (đầu năm 2026), được phân loại theo 3 mức nghiêm trọng. Mỗi mục kèm theo chương tham chiếu trong sách — nơi vấn đề được thảo luận chi tiết.

E.5.1. Nghiêm trọng (Critical) — 8 mục (tính đúng đắn / bảo mật)

Bảng E.5 liệt kê các trade-off ảnh hưởng tới tính đúng đắn và bảo mật.

Bảng E.5: *Trade-off mức nghiêm trọng — ảnh hưởng tính đúng đắn và bảo mật*

Trade-off	Mô tả ngắn	Chương
Thiếu idempotency phía consumer	API nộp bài đã có idempotency key (Sự cố 1), consumer Kafka chưa có — message lặp có thể bị chấm hai lần	Ch.5–6
Plain-header trust / thiếu workload identity	`X-User-*` không phải signed proof; service-to-service chưa nhận diện actor thống nhất	Ch.8–9
Secret management thô sơ	Một phần secret gắn cố định trong tệp cấu hình	Ch.9
Token lưu localStorage	Nếu có XSS, mã độc đọc được token	Ch.9
Code Judge không có sandbox (Gen 1)	Mã sinh viên chạy trực tiếp trên server	Ch.9–10
CORS allow all origins	Mọi trang web đọc được phản hồi API không kèm credential	Ch.8
Shared Database giữa các service	Schema coupling → lỗi dây chuyền (cascading failure) tiềm ẩn	Ch.7
Bespoke JWT validation	Khác thư viện/quy tắc `alg`/issuer/audience → trôi lệch khi xác minh	Ch.8–9

E.5.2. Quan trọng (Major) — 11 mục (khả năng mở rộng / bảo trì)

Bảng E.6 liệt kê các trade-off ảnh hưởng tới khả năng mở rộng và bảo trì.

Bảng E.6: *Trade-off mức quan trọng — ảnh hưởng khả năng mở rộng và bảo trì*

Trade-off	Mô tả ngắn	Chương
Shared library coupling	`kbm-shared` tạo coupling compile-time giữa các service	Ch.2–4
Gọi đồng bộ thiếu khả năng chịu lỗi (fault tolerance)	Không có circuit breaker, chính sách retry	Ch.4
Thiếu API versioning	API thay đổi có thể phá vỡ consumer	Ch.3

Trade-off	Mô tả ngắn	Chương
God Service	`kbm-core` quá nhiều trách nhiệm	Ch.2
Entity trùng lặp giữa các service	Cùng khái niệm, khác model → trôi lệch dữ liệu	Ch.2–7
God Class ~1400 dòng	Class vi phạm Single Responsibility	Ch.7
Thiếu domain event	Không phát sự kiện khi trạng thái thay đổi → coupling ẩn	Ch.5
Thiếu database migration tool	JPA auto-DDL trong production → rủi ro schema drift	Ch.7–10
Kafka single-node	Không có replication → single point of failure	Ch.5
Correlation ID chưa xuyên suốt	Khó truy vết (trace) request qua nhiều service	Ch.11
Docker Compose single-host	Không có HA → downtime khi VPS gặp sự cố	Ch.12

E.5.3. Nhỏ (Minor) — 6 mục (cải thiện chất lượng)

Bảng E.7 liệt kê các điểm cải thiện chất lượng.

Bảng E.7: *Trade-off mức nhỏ — cải thiện chất lượng*

Trade-off	Mô tả ngắn	Chương
Thiếu caching layer	Mọi request đều truy vấn DB trực tiếp	Ch.7
Chưa tự động sao lưu (backup)	Rủi ro mất dữ liệu khi sự cố	Ch.12
Polyglot complexity	Java + Go → đội ngũ cần thành thạo 2 ngôn ngữ	Ch.1–12
Mono-repo rebuild	Thay đổi 1 binary → build lại cả 3	Ch.2
DB queue thay vì message broker	DevOps Lab dùng DB polling thay Kafka → không replay	Ch.5
Xây hay mua bộ chấm code	Quyết định chuyển từ tự phát triển sang mã nguồn mở	Ch.3–10

Tại sao công bố trade-off?

Việc liệt kê 25 trade-off — bao gồm cả những vấn đề nghiêm trọng như "không có sandbox" hay "quản lý secret thô sơ" — là có chủ đích. Mục tiêu giáo dục không phải trình bày một hệ thống lý tưởng (điều không tồn tại trong thực tế), mà cho thấy *mỗi quyết định đều có bối cảnh*. Trade-off Shared DB là ví dụ: với nhóm 1–2 người, nhóm chưa cần tách máy chủ database nhưng vẫn cần ranh giới schema và quyền truy cập — shared DB là lựa chọn thực dụng, kèm theo kế hoạch tách dần khi đủ điều kiện (xem chi tiết trong Chương 7).

E.6. Sự cố thực tế và bài học

Bốn sự cố dưới đây đã xảy ra trong quá trình vận hành KBM. Mỗi sự cố được trình bày theo cấu trúc: **Vấn đề** → **Nguyên nhân gốc** → **Cách xử lý** (hoặc hướng xử lý) → **Bài học kiến trúc**.

E.6.1. Sự cố 1: Bài nộp trùng lặp trong kỳ thi

Vấn đề. Trong một kỳ thi SQL online, một số bài nộp bị trùng lặp — cùng một sinh viên, cùng một đề bài, cùng nội dung, nhưng xuất hiện hai lần trong hệ thống. Kết quả chấm bị ảnh hưởng.

Nguyên nhân gốc. Thiếu idempotency key trong API nhận bài nộp. Khi mạng timeout, client tự động retry → server nhận cùng một request hai lần và xử lý cả hai như request mới.

Cách xử lý. Thêm idempotency key (UUID) cho tất cả thao tác ghi quan trọng. Server kiểm tra key trước khi xử lý: nếu đã tồn tại → trả lại kết quả cũ, không tạo bản ghi mới. Phép kiểm tra phải nguyên tử: key được ghi với ràng buộc `UNIQUE` trong cùng transaction với bản ghi bài nộp, để hai lần retry đến cùng lúc không cùng lọt qua.

Bài học kiến trúc. Idempotency không phải thứ "có thì tốt" — đây là yêu cầu bắt buộc cho mọi thao tác ghi trong hệ thống phân tán. Mạng *sẽ* timeout, client *sẽ* retry. Xem chi tiết tại Chương 5 (Idempotent Consumer) và Chương 6 (Saga Pattern).

E.6.2. Sự cố 2: Dữ liệu không nhất quán giữa các bảng

Vấn đề. Hai service (`kbm-core` và `kbm-academic`) sử dụng chung database nhưng tạo các bảng trùng lặp cho cùng một khái niệm nghiệp vụ. Ví dụ: `class_room` và `as_course` cùng lưu thông tin khóa học. Dữ liệu giữa hai nhóm bảng dần trôi lệch (drift) — cập nhật ở bảng này không phản ánh ở bảng kia, dẫn đến sinh viên thấy thông tin khóa học khác nhau tùy theo tính năng đang sử dụng.

Nguyên nhân gốc. Hai giai đoạn phát triển (thực tế là cùng tác giả nhưng ở hai thời điểm khác nhau) xây dựng entity mapping độc lập, tạo ra 9 điểm xung đột: 3 cặp bảng trùng lặp, 3 bảng cùng tên nhưng mapping khác, 3 bảng bị ghi từ nhiều service.

Hướng xử lý. Chuyển đổi 3 giai đoạn: (1) Chuẩn hóa entity dùng chung qua `@Immutable` + `ReadOnlyRepository` trong `kbm-shared`; (2) Gộp bảng trùng lặp (`class_room` + `as_course` → `sys_course`); (3) Thiết lập quy ước quyền sở hữu (ownership): tiền tố `sys_` cho bảng dùng chung, `as_` cho bảng của nghiệp vụ bài tập, và mỗi khái niệm chỉ có một bên ghi (single writer).

Bài học kiến trúc. Shared database không tự động gây vấn đề — vấn đề nằm ở *thiếu quy ước quyền sở hữu* rõ ràng. Khi nhiều service cùng ghi vào cùng một khái niệm mà không có "nguồn sự thật duy nhất" (single source of truth), trôi lệch dữ liệu là hệ quả tất yếu. Xem chi tiết tại Chương 7 (Quản lý dữ liệu) và Chương 2 (Bounded Context).

E.6.3. Sự cố 3: Multi-instance Judge — định tuyến sai

Vấn đề. Khi mở rộng Network Judge (`kbm-judge-network`) lên nhiều instance, giao thức chấm 2 pha gặp lỗi: Bước 1 (nhận đề bài, sinh request ID) được xử lý bởi Instance A; Bước 2 (gửi bài giải kèm request ID) được chuyển đến Instance B. Instance B không biết request ID từ Instance A → từ chối bài nộp.

Nguyên nhân gốc. Giao thức chấm *stateful* (lưu request ID trong bộ nhớ của instance) nhưng hạ tầng lại mở rộng theo kiểu *stateless* (load balancer phân bổ ngẫu nhiên). Hai mô hình xung đột nhau.

Hướng xử lý. Có ba hướng: session affinity (sticky session) tại load balancer, shared state qua Redis hoặc database, hoặc thiết kế lại giao thức thành stateless. Sticky session chỉ che triệu chứng: state vẫn nằm trong bộ nhớ của instance, instance dừng là mất. Shared state gỡ được state khỏi instance nhưng thêm một phụ thuộc. Hướng thứ ba giải quyết tận gốc: bước gửi bài giải tự mang đủ ngữ cảnh chấm (self-contained) dưới dạng một token do judge ký (ví dụ bằng HMAC) và có hạn dùng, nhờ đó bất kỳ instance nào cũng xác minh và xử lý được yêu cầu mà không cần nhớ request ID từ bước trước, còn sinh viên không sửa được ngữ cảnh.

Bài học kiến trúc. Stateful protocol + stateless scaling = xung đột. Khi thiết kế giao thức giao tiếp giữa các service, cần *giả định rằng bất kỳ instance nào cũng có thể xử lý bất kỳ request nào* — đây là nguyên tắc stateless design. Nếu buộc phải giữ state, cần đưa state ra một kho dùng chung (shared store).

E.6.4. Sự cố 4: Gộp danh tính (Identity Merge) — khủng hoảng danh tính

Vấn đề. Khi mở rộng OAuth2 (thêm Microsoft bên cạnh Google), nhóm phát triển phát hiện nhiều trường hợp một cá nhân có 2–3 tài khoản riêng biệt — đăng nhập qua Google tạo tài khoản A, qua

Microsoft tạo tài khoản B, qua email/password tạo tài khoản C. Dữ liệu học tập (điểm, bài nộp, lịch sử) bị phân tán qua nhiều danh tính.

Nguyên nhân gốc. Thiếu cơ chế *liên kết danh tính* (identity linking) — hệ thống tạo tài khoản mới cho mỗi provider mà không kiểm tra email trùng. Đây là vấn đề phổ biến khi thêm OAuth2 provider vào hệ thống đã có sẵn người dùng.

Hướng xử lý. Liên kết các tài khoản cùng email đã xác thực thành một danh tính duy nhất và trỏ khóa ngoại của dữ liệu liên quan về danh tính chính, kèm bước đối soát thủ công cho các trường hợp biên (ví dụ cùng một người nhưng dùng các email khác nhau). Email chỉ là tín hiệu để gợi ý liên kết, vì định danh ổn định của nhà cung cấp là `iss` + `sub`, nên cần xác minh người dùng kiểm soát cả hai tài khoản. Mục tiêu là không mất dữ liệu học tập (điểm, bài nộp, lịch sử) khi gộp.

Bài học kiến trúc. Quản lý danh tính trong hệ thống nhiều nhà cung cấp cần được thiết kế từ đầu với khái niệm *identity* tách biệt khỏi *credential*. Một người dùng = một identity, có thể liên kết nhiều credential (email, Google, Microsoft). Xem chi tiết tại Chương 2 (BC discovery cho Identity & Access) và Chương 9 (Bảo mật).

E.7. Ánh xạ giữa case study và các chương

Mỗi chương trong sách đều tham chiếu đến ít nhất một khía cạnh của KBM. Bảng dưới đây tổng hợp ánh xạ giữa nghiệp vụ, chương, pattern/bài học, và sự cố liên quan (nếu có).

Bảng E.8: *Ánh xạ nghiệp vụ KBM ↔ chương sách ↔ sự cố thực tế*

Chương	Nghiệp vụ KBM liên quan	Pattern / bài học chính	Sự cố
Ch.1	Tổng quan hệ thống, từ các monolith rời rạc → microservices	Monolith Hell, Fast Flow, Quality Attribute	—
Ch.2	Khám phá 7+2 BC, God Service, thực thể trùng lặp	DDD, Bounded Context, Context Map	#2, #4
Ch.3	Multi-protocol judge, DOMjudge ACL, exam-client multi-client	API Design, “Xây hay mua” (build vs buy), Versioning	—
Ch.4	Gọi `kbm-judge-sql` qua OpenFeign, load balancing, tích hợp QLĐT, khả năng chịu lỗi	Sync Communication, Circuit Breaker, ACL	#3
Ch.5	Kafka cho chế độ thi, DB queue DevOps Lab, idempotency	Async Messaging, Idempotent Consumer	#1
Ch.6	Submit-Judge implicit saga, bài nộp trùng lặp	Saga Pattern, Compensating Transaction	#1
Ch.7	Shared DB, JPA auto-DDL vs golang-migrate, data drift	Database per Service, Shared DB trade-off	#2
Ch.8	Spring Cloud Gateway + Go hostname-based routing, CORS	API Gateway, Rate Limiting, làm sạch request ở biên và signed identity propagation	—
Ch.9	SSO đa nền tảng, OAuth2, rủi ro thiếu sandbox, khóa môi trường thi	Tiered Trust, JWT/JWKS, RBAC, workload identity và Secret Management	#4
Ch.10	Lộ trình chuyển đổi, Strangler Fig, xây → mua DOMjudge	Monolith → Microservices, Strangler Fig	—
Ch.11	Correlation ID, Prometheus, Promtail/Alloy→Loki→Grafana	Observability, Distributed Tracing	—
Ch.12	Docker Compose vs k3s, CI/CD, RBAC, DevOps Lab infra	Triển khai, Container Orchestration, IaC	—

Bài tập & case study

> Kỹ sư giỏi không chỉ biết pattern, mà còn biết khi nào không dùng chúng.
> — Tổng hợp từ các cuộc trao đổi về phỏng vấn thiết kế hệ thống

Hướng dẫn sử dụng

Mỗi bài tập/case study được phân loại:

Ký hiệu	Loại bài	Mô tả
`DESIGN`	**System Design**	Thiết kế hệ thống từ yêu cầu — dạng phỏng vấn
`EVAL`	**Trade-off Analysis**	Phân tích trade-off giữa các phương án — dạng câu hỏi phỏng vấn
`CASE`	**Case study**	Phân tích hệ thống thực tế (Netflix, Amazon, Stripe…)
`DEBUG`	**Kịch bản gỡ lỗi (Debugging)**	Tìm nguyên nhân gốc rễ từ triệu chứng — dạng xử lý sự cố khi trực on-call
`ADR`	**Architecture Decision Record**	Viết ADR cho một quyết định thiết kế
`NOTE`	**Ôn tập**	Câu hỏi ôn tập kiến thức cốt lõi — kiểm tra hiểu bài

Mức độ: `L1` Cơ bản | `L2` Trung bình | `L3` Nâng cao (mức phỏng vấn)

Chương 1: Tổng quan SOA & Microservices

1-0 `NOTE` Ôn tập: Monolith vs Microservices `L1`

Ghép mỗi đặc điểm (cột trái) với kiến trúc phù hợp nhất (cột phải).

Đặc điểm	Kiến trúc
A. Deploy toàn bộ hệ thống mỗi lần phát hành	1. Microservices
B. Mỗi đội sở hữu và deploy service độc lập	2. Monolith
C. Mọi class phụ thuộc lẫn nhau, ranh giới module mờ nhạt	3. Modular Monolith
D. Ranh giới module rõ ràng, nhưng deploy cùng nhau	
E. Cần mức trưởng thành DevOps cao (CI/CD, giám sát)	
F. Đội nhỏ, domain chưa rõ, nhưng muốn giữ khả năng tách dần về sau	

Câu hỏi thêm:

1. Martin Fowler khuyên chiến lược gì khi bắt đầu dự án mới? Giải thích ngắn gọn lý do.
2. Kể tên 3 dấu hiệu cho thấy một monolith *đã sẵn sàng* để tách microservices (tham khảo Bảng 1.1).
3. Kể tên hai nguyên lý SOA mà microservices đẩy mạnh và hai cơ chế mà microservices giảm nhẹ (§1.2). Vì sao có sự chọn lọc này?
4. So sánh cơ chế mở rộng (scaling) giữa kiến trúc Monolith và Microservices. Giải thích tại sao Microservices tối ưu tài nguyên phần cứng hơn khi hệ thống phải xử lý tải trọng không đồng đều giữa các tính năng.

1-1 EVAL Khi nào không nên dùng Microservices? L2

Tình huống: Trong vai chuyên gia tư vấn kiến trúc, được ba công ty mời tư vấn "có nên chuyển sang microservices?"

Công ty	Mô tả	Đội ngũ	Lưu lượng	Vấn đề chính
A	Startup fintech, 8 tháng tuổi, chưa rõ product-market fit	4 lập trình viên	200 người dùng/ ngày	Ban giám đốc muốn mở rộng quy mô lớn
B	Thương mại điện tử 5 năm, monolith 500K dòng code, deploy mất 4 giờ	25 lập trình viên, 4 đội	50K đơn hàng/ ngày	Mỗi lần deploy các đội phải phối hợp với nhau, chu kỳ phát hành 2 tuần
C	Hệ thống ERP nội bộ ngân hàng, Java 8, Oracle DB	10 lập trình viên, 2 đội	500 người dùng nội bộ	Muốn "hiện đại hóa" vì đối tác tư vấn đề xuất Microservices

Câu hỏi phân tích:

1. Áp dụng Ma trận quyết định (Bảng 10.1, Chương 10 — đọc trước phần này) cho từng công ty. Với mỗi tiêu chí, công ty nghiêng về cột nào? Tổng hợp thành nhận định chung.
2. Tính **Microservice Premium** cho mỗi trường hợp: chi phí phải trả (độ phức tạp của hệ phân tán, mức trưởng thành DevOps cần có, thời gian học) so với lợi ích thực tế.
3. Đưa ra khuyến nghị cho mỗi công ty: **Yes / Not Yet / Never** — kèm lập luận cụ thể.
4. Với công ty B (ứng viên mạnh nhất): service nào nên tách *trước*? Vì sao? Áp dụng nguyên tắc "tách service mang lại nhiều giá trị nhất trước" (Richardson [2a, Ch.13]).
5. **Câu hỏi phỏng vấn:** Công ty A muốn hệ thống được thiết kế theo microservices "để sẵn cho tương lai". Thuyết phục họ thế nào rằng đây là sai lầm? Dùng trích dẫn nào của Martin Fowler?

1-2 CASE Case study: Shopify — Monolith tỷ đô L3

Bối cảnh: Shopify (2023) vận hành trên **Ruby on Rails monolith** phục vụ hàng triệu nhà bán hàng, xử lý khoảng $236 tỷ GMV. Thay vì chuyển sang microservices, Shopify chọn **Modular Monolith** — tách code thành các component có ranh giới rõ ràng nhưng deploy cùng nhau.

Tham khảo: Shopify Engineering Blog "Deconstructing the Monolith" (2019), "Under Deconstruction" (2020).

Câu hỏi phân tích:

1. Tại sao Shopify — với quy mô lớn hơn hầu hết các công ty — lại *không* chuyển sang microservices? Đây có mâu thuẫn với chiến lược "Monolith First" hay các bài học từ Netflix, Uber ở §1.4 không?
2. **Modular Monolith** khác gì **Microservices** về: (a) ranh giới triển khai, (b) cô lập dữ liệu, (c) chi phí giao tiếp (communication overhead), (d) quyền tự chủ của đội ngũ?

3. Shopify dùng khái niệm **Component Boundaries** thay vì **Service Boundaries**. So sánh hai khái niệm này — khi nào component boundary đủ, khi nào cần service boundary?
4. Nếu KBM trong sách áp dụng Modular Monolith thay vì Microservices, kiến trúc sẽ trông thế nào? Vẽ sơ đồ. Trade-off so với kiến trúc hiện tại?
5. **Tranh luận:** "Microservices là bắt buộc cho hệ thống lớn" — đồng ý hay phản đối? Dùng Shopify làm bằng chứng.

Chương 2: Phân tích hướng domain

2-0 `NOTE` Ôn tập: các khái niệm Domain-Driven Design `L1`

Chọn định nghĩa đúng cho mỗi khái niệm DDD:

1. **Bounded Context** là:
   (a) Một cơ sở dữ liệu riêng cho mỗi service
   (b) Ranh giới mà trong đó một mô hình domain có nghĩa nhất quán
   (c) Một design pattern để tách frontend và backend
2. **Ubiquitous Language** là:
   (a) Ngôn ngữ lập trình chung cho toàn dự án
   (b) Từ vựng thống nhất giữa lập trình viên và chuyên gia nghiệp vụ trong cùng một Bounded Context
   (c) Tài liệu API specification bằng OpenAPI
3. **Aggregate** là:
   (a) Một cụm entity được xử lý như một đơn vị nhất quán (consistency boundary)
   (b) Một bảng trong cơ sở dữ liệu quan hệ
   (c) Một microservice chứa nhiều API endpoint

Câu hỏi thêm:

1. Trong KBM, "bài nộp" (Submission) xuất hiện ở những Bounded Context nào, mỗi nơi mang nghĩa gì? "Tài khoản sinh viên" thuộc Context nào? Các Context này giao tiếp qua cơ chế gì?
2. Phân tích vai trò của chuyên gia nghiệp vụ (Domain Expert) trong việc xác định Bounded Context. Tại sao quá trình thiết kế hệ thống Microservices không nên chỉ do đội ngũ kỹ sư phần mềm thực hiện độc lập?

2-1 `DESIGN` Event Storming: thiết kế hệ thống đăng ký tín chỉ `L2`

Tình huống: Thiết kế hệ thống đăng ký tín chỉ (LMS Course Registration) cho học kỳ mới. Các bên liên quan mô tả:

Kịch bản

"Sinh viên đăng nhập hệ thống, tìm kiếm học phần mở trong kỳ. Hệ thống kiểm tra điều kiện tiên quyết. Sinh viên chọn lớp và nhấn đăng ký. Hệ thống kiểm tra sĩ số lớp, nếu còn chỗ sẽ ghi nhận đăng ký thành công và cập nhật sĩ số. Nếu lớp đầy, sinh viên được đưa vào danh sách chờ (waitlist). Khi đăng ký xong, hệ thống gửi thông tin sang Phòng Tài vụ để tính học phí. Sinh viên phải hoàn thành đóng học phí trong 7 ngày, nếu không hệ thống sẽ tự động hủy kết quả đăng ký."

Yêu cầu:

1. **Liệt kê các Domain Event** (≥12 sự kiện) theo trình tự: `RegistrationRequested → PrerequisiteChecked → SeatReserved → ...`
2. **Xác định các Aggregate:** nhóm các sự kiện → xác định ≥4 aggregate (Course? Registration? Waitlist? Payment? ...)
3. **Vẽ các Bounded Context:** xác định ≥4 context và quan hệ giữa chúng
4. **Câu hỏi trade-off:** Nên tách "Billing/Payment" thành Bounded Context riêng hay gộp vào "Registration Management"? Phân tích ưu nhược của cả hai cách. Nếu chọn gộp thì số context có thể ít hơn chỉ tiêu ở câu trên, miễn là giải thích được ranh giới.
5. Áp dụng phân loại Task/Entity/Utility của Erl cho cùng kịch bản. Kết quả có trùng các context tìm bằng Event Storming không? Khác ở đâu, vì sao?
6. **Conway's Law:** Nếu đội chỉ có 8 người, nên tổ chức đội thế nào để khớp với các Bounded Context?

2-2 `EVAL` Shared Library vs API — khi nào nên dùng gì? `L2`

Tình huống: Hai microservice (Order Service và Inventory Service) cần dùng chung logic kiểm tra định dạng mã sản phẩm. Đội ngũ tranh luận:

Phương án	Cách làm	Lập luận ủng hộ
Shared Library	Tạo `common-validation` JAR, cả hai service cùng import	Dễ tái sử dụng, theo nguyên tắc DRY
API Call	Tạo `Validation Service` riêng, gọi qua REST	Liên kết lỏng (loose coupling), triển khai độc lập
Duplicate Code	Sao chép logic kiểm tra vào mỗi service	Không coupling, nhưng bảo trì thế nào?

Câu hỏi phân tích:

1. Phân tích trade-off chi tiết cho từng phương án theo 5 tiêu chí: coupling, khả năng triển khai độc lập, hiệu năng, bảo trì, quyền tự chủ của đội ngũ.
2. KBM sử dụng shared library (`kbm-shared`) chứa DTO, ErrorCode, utils. Đây có phải anti-pattern? Khi nào shared library là hợp lý?
3. Value object `StudentId` nên đặt ở Shared Library hay Shared Kernel? Những ai phải đồng ý khi đổi nó?
4. Newman trong [4a, Ch.4] cảnh báo: mã dùng chung mà lộ ra ngoài ranh giới service, như một thư viện đối tượng miền dùng chung, sẽ tạo coupling; còn thư viện nội bộ không lộ ra ngoài, như logging, thì dùng chung được. KBM vẫn giữ DTO và contract chung trong `kbm-shared`. Từ hai góc nhìn này, phân loại nội dung `kbm-shared`: thành phần nào nên giữ, thành phần nào nên chuyển về từng service?
5. **Câu hỏi phỏng vấn:** Ở một đội vừa gia nhập, một shared library 50 class đang được 8 service sử dụng. Mỗi khi shared library thay đổi, tất cả service phải deploy cùng lúc. Đây là triệu chứng của vấn đề gì? Đề xuất giải pháp.

Chương 3: Thiết kế dịch vụ & API

3-0 `NOTE` Ôn tập: các nguyên tắc thiết kế API `L1`

Phân loại mỗi thay đổi API sau đây là **Breaking** hay **Non-breaking**:

Thay đổi	Breaking / Non-breaking?
A. Thêm trường mới `createdAt` vào response JSON	

Thay đổi	Breaking / Non-breaking?
B. Đổi tên trường `questionTitle` → `title`	
C. Thêm query parameter tùy chọn `?sort=asc`	
D. Thay đổi HTTP status từ `200` → `201` cho POST	
E. Xóa trường `legacyId` khỏi response	
F. Thêm trường bắt buộc `priority` vào request body	

Câu hỏi thêm:

1. Giải thích nguyên tắc **Tolerant Reader**. Tại sao nguyên tắc này quan trọng cho việc phát triển API theo thời gian (API evolution)?
2. REST API nên dùng chiến lược versioning nào: URL path (`/v2/users`) hay custom header (`X-API-Version: 2`)? Nêu 1 ưu và 1 nhược cho mỗi cách.
3. Trong Hexagonal Architecture (§3.5), `@RestController`, `QuestionRepository` và `JpaQuestionAdapter` giữ vai trò gì? Vì sao `JpaRepository` không phải outbound port?
4. Cho hai màn hình: quản lý câu hỏi (CRUD) và dashboard giảng viên tổng hợp từ ba service. Mỗi màn hình nên dùng REST hay GraphQL? Vì sao?
5. Đánh giá tầm quan trọng của việc duy trì tính tương thích ngược (backward compatibility) khi thiết kế API trong kiến trúc Microservices so với kiến trúc Monolith. Hậu quả của việc vi phạm nguyên tắc này là gì?

3-1 `CASE` Case study: Stripe API — tại sao được coi là "gold standard"? `L2`

Bối cảnh: Stripe API được cộng đồng lập trình viên đánh giá là **một trong những API được thiết kế tốt nhất**. Phân tích tại sao.

Dữ liệu: Truy cập `stripe.com/docs/api` (hoặc dựa trên kiến thức).

Câu hỏi phân tích:

1. Stripe dùng **date-based versioning** (`2024-12-18`), chọn qua header `Stripe-Version`, song song với namespace `/v1`, `/v2` trong URL. So sánh với URL path versioning — ưu/nhược điểm gì? Khi nào date-based tốt hơn?
2. Stripe API trả response dạng **envelope pattern**: `{ "object": "list", "data": [...], "has_more": true }`. So sánh với dạng `{ "data": [...], "pagination": {...} }` ở Listing 3.1 — cái nào tốt hơn? Tại sao?
3. Stripe sử dụng **Idempotency Key** cho mọi POST request (tránh trừ tiền khách hàng 2 lần). Thiết kế cơ chế tương tự cho `POST /api/submissions` của LMS — sinh viên nộp bài 2 lần do mạng chậm → chỉ chấm 1 lần.
4. Stripe luôn trả **error object nhất quán** — mọi lỗi đều nằm dưới khóa `error`, kèm các trường như `type`, `code` và `message` (ví dụ `type: "card_error"`, `code: "expired_card"`). So sánh với định dạng lỗi hiện tại của LMS — khoảng trống ở đâu?
5. **Thiết kế:** Thiết kế API cho tính năng mới "Giảng viên tạo kỳ thi → sinh viên đăng ký → nộp bài → bảng xếp hạng": viết danh sách endpoint theo phong cách của Stripe.

3-2 `DEBUG` Gỡ lỗi: sự cố breaking change của API `L3`

Tình huống sự cố:

Kịch bản

Giả định KBM đã có ứng dụng mobile. 9:00 AM — Đội ngũ deploy Core Service v2.3: đổi trường `questionTitle` → `title` trong response (cho gọn tên). 9:15 AM — Ứng dụng mobile (v1.2, chưa cập nhật) crash khi tải danh sách câu hỏi. 200 sinh viên đang luyện tập bị ảnh hưởng. 9:30 AM — Đội vận hành phát hiện qua dashboard, rollback Core Service về v2.2. 10:00 AM — Bắt đầu họp rút kinh nghiệm (postmortem).

Câu hỏi phân tích:

1. **Nguyên nhân gốc rễ:** Đây là loại breaking change nào? (đổi tên, xóa, đổi kiểu dữ liệu?) Tại sao đổi tên trường tương đương "xóa cũ + thêm mới"?
2. **Phòng ngừa:** Liệt kê ≥3 cơ chế ngăn sự cố này tái diễn (contract testing, kiểm tra schema, chính sách versioning...)
3. **Thiết kế giải pháp:** Nếu buộc phải đổi tên trường, làm thế nào để không gây breaking change? Mô tả quy trình 3 bước.
4. **Consumer-Driven Contract Testing:** Giải thích khái niệm này. Nếu ứng dụng mobile có contract test, sự cố này có được phát hiện trước khi lên production không?
5. **ADR:** Viết Architecture Decision Record cho quyết định: "Từ nay response của API chỉ được *thêm* trường, không được *xóa* hoặc *đổi tên* trường". Dùng mẫu ADR ở bài 8-2: bối cảnh, tiêu chí quyết định, các phương án, quyết định, hệ quả và theo dõi sau triển khai.

Chương 4: Giao tiếp đồng bộ

4-0 `NOTE` Ôn tập: các resilience pattern `L1`

Ghép mỗi resilience pattern với mô tả đúng:

Pattern	Mô tả
A. Circuit Breaker	1. Giới hạn số request đồng thời đến một service
B. Retry with Backoff	2. Ngắt kết nối khi service downstream liên tục lỗi, tránh lãng phí tài nguyên
C. Bulkhead	3. Trả về giá trị mặc định khi service chính không phản hồi
D. Timeout	4. Thử lại request thất bại với khoảng cách tăng dần
E. Fallback	5. Đặt giới hạn thời gian chờ để tránh thread bị chặn vô hạn

Câu hỏi thêm:

1. Circuit Breaker có 3 trạng thái. Kể tên và mô tả ngắn gọn điều kiện chuyển đổi giữa chúng.
2. Tại sao timeout ở tầng Gateway phải **lớn hơn** timeout ở tầng service bên trong? Nếu ngược lại sẽ xảy ra vấn đề gì?
3. Một OpenFeign client khai báo `@FeignClient(name = "kbm-academic")` — không host, không port. Hãy mô tả chuỗi bước từ cái tên đó tới một địa chỉ IP thật: Eureka giữ vai trò gì, và chuyện gì xảy ra khi service đích đang chạy ba instance?
4. Nếu Eureka registry sập trong 5 phút nhưng các service vẫn chạy, những lời gọi Feign đang có còn hoạt động không? Vì sao? Điều đó nói lên gì về vị trí của registry trong sơ đồ phụ thuộc?

4-1 DEBUG Gỡ lỗi: Cascading Failure — "Tại sao cả hệ thống ngừng hoạt động?" L3

Tình huống sự cố (mô phỏng từ thực tế):

Lịch trình

- 14:00 — Database server của Service Tài vụ (kiểm tra công nợ) gặp sự cố phần cứng, mọi truy vấn đều timeout (30 s)
- 14:02 — Service Đăng ký tín chỉ gọi Service Tài vụ → timeout 30 s → thread pool cạn kiệt
- 14:05 — Service Cổng thông tin gọi Service Đăng ký tín chỉ → cũng timeout → thread pool cạn kiệt
- 14:08 — API Gateway gọi Cổng thông tin → timeout → Gateway thread pool đầy
- 14:10 — **Toàn bộ hệ thống** sập. Sinh viên không xem được cả lịch học. Health check thất bại.
- 14:25 — Kỹ sư trực on-call khởi động lại tất cả service. Hệ thống phục hồi sau 5 phút.
- **Tổng downtime**: 20 phút (14:10–14:30). Ảnh hưởng: 15.000 người dùng.

Câu hỏi phân tích:

1. Vẽ **sơ đồ diễn biến theo thời gian** (kiểu sequence diagram) cho cascading failure. Service nào lỗi trước? Hiệu ứng domino lan thế nào?
2. **Nguyên nhân gốc rễ và tác nhân kích hoạt:** Lỗi phần cứng database là tác nhân kích hoạt (trigger). Nguyên nhân gốc rễ thực sự là gì? (gợi ý: thiếu pattern nào?)
3. Nếu hệ thống có **Circuit Breaker** (Resilience4j) giữa mỗi service call, lịch trình sẽ thay đổi thế nào? Viết lại lịch trình.
4. Nếu có Circuit Breaker *và* **Bulkhead**, kết quả khác gì so với chỉ có Circuit Breaker?
5. Thiết kế **cấu hình resilience** cho hệ thống 4 tầng này: timeout, retry, circuit breaker, bulkhead cho mỗi tầng. Giải thích tại sao timeout tầng ngoài phải > timeout tầng trong.
6. **Câu hỏi phỏng vấn:** Netflix xử lý hàng tỷ request mỗi ngày. Thiếu circuit breaker ở *một* service có thể làm sập toàn bộ nền tảng. Netflix giải quyết bằng cách nào? (Gợi ý: Hystrix; khi Hystrix ngừng phát triển, hệ sinh thái Spring chuyển sang Resilience4j)

Gợi ý

Nhớ lại khái niệm Circuit Breaker và Bulkhead ở Chương 4.

4-2 EVAL REST vs gRPC vs GraphQL — chọn công nghệ nào cho kịch bản tương ứng? L2

Tình huống: Thiết kế giao tiếp giữa các microservice cho hệ thống thương mại điện tử:

Giao tiếp	Đặc điểm
A: Mobile app ↔ BFF (Backend for Frontend)	Băng thông hạn chế, cần truy vấn linh hoạt
B: Order Service ↔ Inventory Service (internal)	Thông lượng cao, độ trễ thấp, schema chặt chẽ
C: Admin Dashboard ↔ Product Service	Cần truy vấn dữ liệu lồng nhau (product + reviews + inventory)
D: API công khai cho các nhà bán hàng bên thứ ba	Ổn định, có tài liệu, có versioning

Câu hỏi phân tích:

1. Với mỗi kịch bản (A–D), chọn REST / gRPC / GraphQL. Giải thích trade-off.
2. Có kịch bản nào mà 2 giao thức phù hợp như nhau? Tiêu chí quyết định cuối cùng (tiebreaker) là gì?

3. Netflix dùng gRPC cho giao tiếp giữa các service, GraphQL cho mobile BFF. Spotify dùng gRPC nội bộ + REST cho API công khai. Phân tích: tại sao các công ty lớn đều hướng gRPC cho nội bộ nhưng giữ REST/GraphQL cho bên ngoài?
4. Trong KBM, khi `kbm-sandbox-mysql` ngừng hoạt động, `kbm-sandbox-sqlserver` vẫn phải chấm được. Đặt Circuit Breaker ở tầng nào để đạt điều đó, và vì sao không đặt ở Core?
5. KBM hiện dùng REST cho LMS core, nhưng Network Lab đã có TCP/UDP/RMI/SOAP/REST/gRPC và DevOps Lab dùng Go/Chi cho một phần API. Nếu phải chọn giao thức cho một ranh giới nội bộ mới — chọn gì? Phân tích chi phí và lợi ích.

Chương 5: Giao tiếp bất đồng bộ

5-0 `NOTE` Ôn tập: các khái niệm Kafka `L1`

Điền vào chỗ trống bằng thuật ngữ đúng: *Topic, Partition, Consumer Group, Offset, Producer, Broker*.

1. _____ là đơn vị logic để phân loại message (ví dụ: `submissions`, `judge-results`).
2. Mỗi topic được chia thành nhiều _____ để xử lý song song.
3. _____ là vị trí (số thứ tự) của message trong partition — consumer dùng nó để theo dõi đã đọc đến đâu.
4. Nhiều consumer instance cùng thuộc một _____ sẽ chia nhau các partition — mỗi partition chỉ được 1 consumer trong group xử lý.
5. _____ là thành phần gửi message vào topic.
6. _____ là server Kafka lưu trữ và phục vụ message.

Câu hỏi thêm:

1. Giải thích sự khác biệt giữa **at-most-once**, **at-least-once**, và **exactly-once**. Luồng xử lý bài nộp (submission pipeline) của KBM nên dùng loại nào? Tại sao?
2. Khi nào nên tăng số partition của một topic? Có rủi ro gì khi tăng quá nhiều?
3. Xếp các message sau vào đúng loại (Command, Event, Document, Query): `JudgeSubmission`, `SubmissionJudged`, `UserDocument`. Message nào có thể bị từ chối, vì sao?
4. Vẽ đường đi của một message từ exchange kiểu Topic tới hai queue, chỉ rõ binding.
5. Kết quả chấm bài cần hiện lên màn hình sinh viên ngay khi có. So sánh ba cách đẩy kết quả tới trình duyệt — **polling**, **Server-Sent Events** và **WebSocket** — theo bốn tiêu chí: độ trễ, tải lên server, chiều giao tiếp, và độ phức tạp khi service chạy nhiều instance. Chọn một phương án cho KBM và nêu điều kiện nào sẽ khiến phương án đó không còn phù hợp.

5-1 `EVAL` Kafka vs RabbitMQ — quyết định dựa trên gì? `L2`

Tình huống: 4 kịch bản khác nhau, mỗi kịch bản cần một message broker:

Kịch bản	Mô tả	Yêu cầu đặc biệt
A: Thông báo học phí	Sinh viên nộp tiền → Gửi email nhắc/xác nhận	Đảm bảo gửi đúng 1 lần, thứ tự không quan trọng
B: Lịch sử giao dịch	Mọi giao dịch đóng/rút học phí = event	Thứ tự đặc biệt quan trọng, lưu giữ (retention) vĩnh viễn
C: Điểm danh quẹt thẻ	Hàng chục ngàn sinh viên quẹt thẻ cổng trường	Thông lượng cực cao, mất vài message vẫn chấp nhận được
D: Hàng đợi xử lý file	Báo cáo đồ án: tải lên → đưa vào queue → kiểm tra đạo văn	Phân phối round-robin, consumer xác nhận (acknowledgment)

Câu hỏi phân tích:

1. Với mỗi kịch bản: chọn Kafka hay RabbitMQ? Giải thích bằng khác biệt kiến trúc (log-based vs queue-based).
2. Kleppmann trong [7, Ch.11] giải thích: Kafka = "distributed commit log" (chỉ ghi nối, consumer tự quản lý offset). RabbitMQ = "message queue" (broker đẩy message, message bị xóa sau khi ack). Mô hình tư duy (mental model) này giúp ra quyết định thế nào?
3. LinkedIn (nơi tạo ra Kafka) xử lý 7 nghìn tỷ message/ngày trên 7 triệu partition (số liệu 2019). Họ *cần* Kafka. KBM có nhiều loại queue khác nhau: submission pipeline, SQL Judge worker dispatch (`kbm-judge-sql` → `kbm-sandbox-*`), các tác vụ (job) của DevOps Lab. Luồng nào thật sự cần Kafka, luồng nào chỉ cần DB queue? Đánh đổi giữa "học đúng công cụ" và "dùng công cụ phù hợp"?
4. **Exactly-once delivery:** Kafka hỗ trợ ngữ nghĩa exactly-once theo giao dịch từ bản 0.11. RabbitMQ không hỗ trợ exactly-once sẵn có. Giải thích tại sao exactly-once khó và tại sao "at-least-once + idempotent consumer" thường đủ.
5. KBM chọn Kafka cho luồng xử lý bài nộp. Nếu dùng RabbitMQ hoặc DB queue thay thế, kiến trúc thay đổi thế nào? Tính năng nào bị mất?

5-2 `DEBUG` Gỡ lỗi: "Thông điệp biến mất" — Dead Letter & Poison Pill `L3`

Tình huống sự cố:

Triệu chứng

Kỳ kiểm tra cuối kỳ — 5 sinh viên báo "nộp bài nhưng không nhận kết quả". Giảng viên kiểm tra DB: bản ghi bài nộp vẫn tồn tại, nhưng không có điểm.

Nhật ký điều tra:

- Log của Core Service: ✓ sự kiện `SubmissionCreated` được publish thành công
- Kafka topic `submissions`: ✓ Message vẫn tồn tại (đã kiểm tra bằng `kafka-console-consumer`)
- Log của Judge Service: 5 message ném `JsonParseException` → consumer crash và khởi động lại → message được tiêu thụ lại → crash lại → **vòng lặp vô hạn**
- Nguyên nhân gốc rễ: 5 bài nộp chứa dữ liệu sai encoding → giải tuần tự hóa (deserialization) thất bại

Câu hỏi phân tích:

1. Đây là ví dụ về **message "Poison Pill"**. Định nghĩa poison pill. Tại sao nó nguy hiểm hơn một message xử lý lỗi thông thường?
2. Không có DLT (Dead Letter Topic): consumer crash → khởi động lại → tiêu thụ lại chính message đó → crash → lặp vô hạn. **45 message khác** (của các sinh viên không gặp lỗi) cũng bị kẹt phía sau poison pill. Giải thích cơ chế (offset của Kafka consumer không được commit khi xử lý lỗi)
3. Thiết kế giải pháp **3 tầng**:
   - Tầng 1: Retry cho lỗi tạm thời (bao nhiêu lần? chiến lược backoff?); lỗi giải tuần tự hóa có nên thử lại không?
   - Tầng 2: Dead Letter Topic (định dạng message? cơ chế cảnh báo?)
   - Tầng 3: Dashboard rà soát thủ công (admin xem các message trong DLT, xử lý lại hoặc loại bỏ)

4. Sau khi triển khai DLT, luồng xử lý mới sẽ như thế nào? Vẽ sơ đồ. 5 message lỗi đi đâu? 45 message còn lại có bị ảnh hưởng?
5. **Phòng ngừa:** Làm sao ngăn poison pill ngay từ đầu? (kiểm tra schema, contract testing ở producer)

Gợi ý

Xem lại khái niệm Dead Letter Topic ở Chương 5 và cấu hình `@RetryableTopic`.

Chương 6: Giao dịch phân tán

6-0 NOTE Ôn tập: nền tảng Saga L1

Cho bốn bước của một Saga "Sinh viên nộp bài" trong KBM:

Các bước (chưa đúng thứ tự):

- Ghi nhận điểm vào Gradebook
- Judge Service chấm bài (chạy SQL trong sandbox)
- Core Service tạo bản ghi Submission
- Ghi bền vững lệnh thông báo kết quả vào outbox

Yêu cầu:

1. Sắp xếp lại 4 bước theo thứ tự đúng và phân loại từng bước: Compensatable, Pivot hay Retriable.
2. Nếu bước chấm bài thất bại vì sandbox timeout, bước nào trước đó cần bù trừ, và hành động bù trừ cụ thể là gì?
3. Choreography Saga và Orchestration Saga khác nhau ở điểm nào? Nêu 1 ưu điểm chính của mỗi loại.
4. **Đúng hay Sai:** "Saga đảm bảo ACID transaction xuyên suốt nhiều service." Giải thích câu trả lời.

6-1 DESIGN Thiết kế: Uber Ride Saga — Choreography hay Orchestration? L3

Tình huống: Một chuyến xe Uber trải qua các bước:

```
1. Hành khách đặt chuyến
2. Ghép tài xế (tìm tài xế gần nhất)
3. Tài xế nhận chuyến
4. Bắt đầu chuyến đi (bật theo dõi GPS)
5. Kết thúc chuyến đi (tính quãng đường)
6. Tính cước (hệ số giờ cao điểm, giảm giá)
7. Thu tiền qua phương thức thanh toán của hành khách
8. Trả tiền cho tài xế (sau khi trừ chiết khấu)
9. Gửi biên lai
```

Nếu bước 7 thất bại (thẻ hết tiền):

```
Bù trừ: Hủy giao dịch thanh toán → Thông báo hành khách "thêm phương thức thanh
toán"
        → Giữ bản ghi chuyến đi (chờ thử lại)
Lưu ý: Không thể hoàn tác chuyến đi (xe đã chạy) → đây là hành động không thể bù trừ
(non-compensatable action)
```

Câu hỏi phân tích:

1. Xác định **Compensatable**, **Pivot**, và **Retriable** transaction trong 9 bước. Bước nào là điểm không thể quay lui (point of no return)?
2. Uber (thực tế) dùng **Orchestration** cho ride saga — tại sao? So sánh nếu dùng Choreography: cần bao nhiêu sự kiện? Ai theo dõi trạng thái tổng thể?
3. **Vấn đề isolation:** Khách A đặt xe, tài xế X được ghép → trong lúc tài xế X đang trên đường đến → khách B đặt xe, hệ thống cũng ghép tài xế X. Đây là anomaly gì? (Lost update? Dirty read?) Biện pháp đối phó (countermeasure) nào cho kịch bản này?
4. **Thiết kế timeout:** Tài xế không nhận chuyến trong 30 giây → timeout → tìm tài xế khác. Nhưng do mạng chậm, tài xế nhận chuyến ở giây thứ 31 (message đến trễ). Thiết kế cơ chế xử lý "nhận chuyến muộn" (late acceptance).
5. **Liên hệ KBM:** Hãy thiết kế Saga cho **Quy trình công bố điểm** (Chương 6, "Workflow học vụ cũng là saga"): tính điểm dự kiến → giảng viên xác nhận → công bố bảng điểm → đồng bộ read model → thông báo sinh viên. Bước nào là pivot? Nếu bước đồng bộ read model lỗi sau khi đã công bố, xử lý thế nào? Vẽ sơ đồ.

6-2 `EVAL` Trade-off: Choreography vs Orchestration `L2`

Bài tập: Cho 4 kịch bản saga, phân tích nên dùng Choreography hay Orchestration:

Saga	Các bước	Đặc điểm
A: Đăng ký người dùng	Create account → Send verification email → Assign default role	3 bước, đơn giản, tỷ lệ lỗi thấp
B: Đơn hàng thương mại điện tử	Reserve stock → Process payment → Ship → Update loyalty points	4 bước, thanh toán là then chốt, cần giám sát
C: Yêu cầu bồi thường bảo hiểm	Submit claim → Fraud detection → Adjuster review → Payout → Notify	5 bước, có con người tham gia (human-in-the-loop), quy trình kéo dài nhiều ngày
D: Đặt vé máy bay (nhiều chặng)	Book leg 1 → Book leg 2 → Book hotel → Book car → Payment	5 bước, gọi API bên ngoài, thường gặp lỗi một phần

Câu hỏi phân tích:

1. Với mỗi saga: chọn Choreography hay Orchestration? Lập luận dựa trên độ phức tạp, số bước, xác suất lỗi, mức độ tham gia của con người.
2. Chương 6 dùng ngưỡng "hơn 4 bước hoặc có phân nhánh → cân nhắc orchestration" (theo Rocha [5, §4.4]). Ngưỡng này có hợp lý không? Có ngoại lệ không?
3. **Saga D (Đặt vé máy bay)** đặc biệt khó — các API bên ngoài (hãng bay, khách sạn) có thể timeout hoặc trả kết quả mơ hồ. Thiết kế bù trừ cho tình huống: "Đặt chặng 1 thành công, đặt chặng 2 bị timeout — không biết thành công hay thất bại". Đây là "trạng thái không chắc chắn" (uncertain state) — xử lý thế nào?
4. Netflix tạo **Conductor**, Uber tạo **Cadence** (về sau có nhánh phát triển là Temporal): đều là engine orchestration mã nguồn mở. Tại sao các công ty lớn tự xây engine orchestration riêng thay vì dùng choreography?

Chương 7: Quản lý dữ liệu

7-0 `NOTE` Ôn tập: các pattern quản lý dữ liệu `L1`

Ghép mỗi pattern/khái niệm với bài toán phù hợp nhất:

Pattern / khái niệm	Bài toán
A. Database per Service	1. Cần lưu vết kiểm toán (audit trail) đầy đủ mọi thay đổi trạng thái (ví dụ: giao dịch ngân hàng)
B. CQRS	2. Tải đọc và tải ghi có yêu cầu rất khác nhau (ví dụ: bảng xếp hạng)
C. Event Sourcing	3. Đảm bảo mỗi service có thể phát triển schema độc lập
D. Shared Database	4. Đội nhỏ, cần đơn giản, chấp nhận coupling giữa các service

Câu hỏi thêm:

1. Giải thích ngắn gọn **CAP Theorem**. Tại sao nói "chọn 2 trong 3" là cách hiểu không chính xác?
2. KBM hiện dùng Shared Database. Nêu 2 vấn đề cụ thể mà kiến trúc này gây ra khi muốn tách microservices.

7-1 `EVAL` CAP Theorem trong thực tế — "Pick Two" có đúng không? `L3`

Tình huống: Đội ngũ tranh luận về chiến lược database:

Kịch bản

"CAP theorem nói chỉ chọn được 2 trong 3: Consistency, Availability, Partition Tolerance. Vậy nếu partition xảy ra, ta phải chọn giữa C và A. Nhưng hệ thống của ta cần CẢ HAI..."

Câu hỏi phân tích:

1. Martin Kleppmann phản bác cách hiểu "pick two". Ý của ông: chỉ khi có phân vùng mạng mới phải chọn giữa consistency và availability; không có phân vùng thì có thể có cả hai. Giải thích — khác biệt thực tế so với "pick two" là gì?
2. Phân loại các database theo CAP. Với mỗi database, ghi rõ giả định về cấu hình sao chép, mức xác nhận và thao tác đọc/ghi trước khi phân loại:

Database	Phân loại	Hành vi khi có network partition
PostgreSQL (single master)	?	?
Cassandra	?	?
MongoDB (replica set)	?	?
Redis Cluster	?	?

3. KBM dùng PostgreSQL cho nhiều dữ liệu giao dịch. Khi primary DB ngừng hoạt động một khoảng thời gian, hệ thống có thể không khả dụng cho các luồng ghi. Nếu dùng PostgreSQL với streaming replication, trade-off thay đổi thế nào?
4. **Kịch bản trong trường học:** Giỏ đăng ký môn học dự kiến vs Thanh toán học phí — yêu cầu C/A khác nhau:
   - Giỏ môn học: ưu tiên availability hơn consistency (chấp nhận lưu tạm khi mất kết nối mạng, sinh viên có thể thêm bớt sau)
   - Thanh toán học phí: ưu tiên consistency hơn availability (không chấp nhận trừ tiền hai lần)
   - Thiết kế: dùng chiến lược database nào cho từng tính năng?
5. **Câu hỏi phỏng vấn:** Bài báo Dynamo của Amazon (SOSP 2007) nêu yêu cầu cho dịch vụ giỏ hàng: "customers should be able to view and add items to their shopping cart even if disks are failing, network routes are flapping, or data centers are being destroyed by tornados." Vì vậy giỏ hàng dùng kho lưu trữ eventually consistent (Dynamo) và chấp nhận có lúc phải hợp nhất các phiên bản giỏ hàng xung đột. Trade-off này có hợp lý không? Giải thích bằng CAP.

7-2 DESIGN Thiết kế: chiến lược chuyển đổi database cho KBM L2

Tình huống: KBM hiện có shared database — Core Service và Academic Service (`kbm-academic`) dùng chung PostgreSQL. Nhiệm vụ: tách database.

Ràng buộc:

- Zero downtime (sinh viên đang dùng hệ thống)
- Đội ngũ 2 lập trình viên
- Thời gian: 3 tháng
- Không được mất dữ liệu

Các bảng:

```
Core domain:     questions, submissions, scores, contests
Academic domain: assignments, assignment_questions, student_assignments
Shared:          users (dữ liệu gốc ở Auth Service;
                 cả hai service đều cần tên người dùng)
Cross-reference: assignment_questions.question_id → questions.id
```

Câu hỏi phân tích:

1. Áp dụng 5 chiến lược tách database (§7.2, Hình 7.5): thiết kế lộ trình 3 tháng — tháng 1, 2, 3 làm gì?
2. Cả hai service đều cần tên người dùng từ Auth Service. 3 phương án: (a) gọi API của Auth mỗi khi cần, (b) nhân bản qua sự kiện `UserUpdated`, (c) API Composition ở Gateway hoặc BFF. Phân tích trade-off của từng phương án.
3. `assignment_questions` tham chiếu `questions.id` — đây là một phép join xuyên miền (cross-domain join). Sau khi tách DB, join này không còn hoạt động. Thiết kế giải pháp: API composition? Data duplication? CQRS?
4. **Rủi ro ghi kép (dual write):** Trong quá trình chuyển đổi, có một giai đoạn code cũ (truy cập DB trực tiếp) và code mới (qua API) cùng tồn tại. Làm sao tránh dữ liệu không nhất quán? (Feature flag? Shadow write?)
5. **Kế hoạch rollback:** Nếu tháng 2 phát hiện vấn đề hiệu năng nghiêm trọng — làm cách nào để rollback về shared database an toàn?

Chương 8: API Gateway & BFF Pattern

8-0 NOTE Ôn tập: nghiệp vụ cắt ngang ở API Gateway L1

Từ danh sách sau, đánh dấu những chức năng nào thuộc trách nhiệm của API Gateway (có thể chọn nhiều):

1. ☐ Xác minh JWT ở biên (edge token validation / auth)
2. ☐ Business logic xử lý đơn hàng
3. ☐ Rate Limiting (giới hạn số request/giây)
4. ☐ Request routing đến đúng downstream service
5. ☐ Database migration
6. ☐ SSL/TLS termination
7. ☐ Load balancing
8. ☐ Response caching
9. ☐ Tính toán điểm bài tập (grading logic)

Câu hỏi thêm:

1. Giải thích sự khác biệt giữa việc Gateway **định tuyến** request và việc Gateway **xử lý thay** các nghiệp vụ cắt ngang (offloading). Cho ví dụ mỗi loại.
2. Tại sao không nên đặt business logic vào API Gateway? Hậu quả nếu vi phạm là gì?

8-1 `CASE` Case study: Netflix Zuul 1, Zuul 2 và Spring Cloud Gateway `L2`

Bối cảnh: Netflix viết lại API Gateway của mình từ blocking sang non-blocking; hệ sinh thái Spring đi một hướng riêng:

- **Zuul 1**: Mã nguồn mở năm 2013. Blocking I/O, dựa trên servlet, mỗi request chiếm một thread.
- **Zuul 2**: Netflix giới thiệu năm 2016, mã nguồn mở năm 2018. Non-blocking, dựa trên Netty. Năm 2018, Netflix vận hành hơn 80 cluster Zuul 2, xử lý hơn 1 triệu request/giây, làm cửa ngõ cho mọi request vào hạ tầng cloud của Netflix.
- **Spring Cloud Gateway**: Bản chính thức đầu tiên năm 2018, trong Spring Cloud Finchley. Dự án của Spring, không phải của Netflix; reactive trên Spring WebFlux và Reactor Netty; được Spring giới thiệu là gateway thế hệ mới thay thế Spring Cloud Netflix Zuul.

Câu hỏi phân tích:

1. Chuyển từ Zuul 1 sang Zuul 2 nghĩa là viết lại gateway theo mô hình bất đồng bộ. Cái giá của việc này là gì (độ phức tạp khi lập trình, khi gỡ lỗi, các thư viện đi kèm cũng phải bất đồng bộ)? Vì sao một hệ thống ở quy mô Netflix vẫn chấp nhận trả cái giá đó? (Gợi ý: đọc bài "Zuul 2: The Netflix Journey to Asynchronous, Non-Blocking Systems", Netflix TechBlog, 2016)
2. **Blocking vs Non-blocking Gateway:** Zuul 1 dùng 1 thread/request (blocking). Spring Cloud Gateway dùng event loop (non-blocking). Với 10K kết nối đồng thời, Zuul 1 cần bao nhiêu thread? Spring Cloud Gateway cần bao nhiêu? Hệ quả với bộ nhớ?
3. KBM dùng Spring Cloud Gateway cho LMS core và Go hostname-based router cho DevOps Lab. Với lưu lượng học thuật vừa phải, blocking hay non-blocking có khác biệt? Khi nào KBM *cần* non-blocking gateway/router?
4. **Gateway là điểm lỗi duy nhất (single point of failure):** Nếu Gateway sập, toàn bộ hệ thống ngừng hoạt động. Thiết kế tính sẵn sàng cao cho Gateway thế nào? (Nhiều instance? Load balancer phía trước?)
5. **BFF Pattern (Backend for Frontend):** Netflix có gateway riêng cho ứng dụng mobile (băng thông ít) và ứng dụng web (cần nhiều dữ liệu hơn). Nếu KBM cần hỗ trợ ứng dụng mobile, BFF nên được thiết kế thế nào?

8-2 `ADR` ADR: Rate Limiting Strategy `L2`

Tình huống: Hệ thống LMS bị lạm dụng:

- Một sinh viên viết script tự động nộp bài 1000 lần/phút (dò đáp án SQL kiểu brute-force)
- Judge Service quá tải → ảnh hưởng kỳ thi của 200 sinh viên khác

Yêu cầu: Viết Architecture Decision Record cho Rate Limiting.

Mẫu ADR:

1. **Tiêu đề:** Chiến lược rate limiting cho API Gateway của LMS
2. **Bối cảnh:** Mô tả vấn đề (hành vi lạm dụng, tác động).
3. **Tiêu chí quyết định:** Bảo mật, sử dụng công bằng, hiệu năng, chi phí triển khai.
4. **Các phương án:**

- Phương án 1: Rate limit theo người dùng — cấu hình như Listing 8.4 (5 request/giây). Mức này có đủ để chặn script nộp 1000 lần/phút không? Nếu cần giới hạn theo phút, phải đổi tham số thế nào?
- Phương án 2: Rate limit theo IP
- Phương án 3: Rate limit theo route + cửa sổ thời gian (endpoint thi nghiêm ngặt hơn)
- Phương án 4: Rate limiting thích ứng (tự điều chỉnh theo tổng tải)

5. **Quyết định:** Chọn phương án nào? Tại sao?
6. **Hệ quả:** Tích cực, tiêu cực, rủi ro.
7. **Theo dõi sau triển khai:** Chiến lược giám sát? Cảnh báo khi người dùng bị rate limit?

Chương 9: Bảo mật Microservices

9-0 `NOTE` Ôn tập: Authentication vs Authorization `L1`

Phân loại mỗi tình huống sau là **Authentication** (AuthN) hay **Authorization** (AuthZ):

Tình huống	AuthN / AuthZ?
A. Sinh viên đăng nhập bằng username/password	
B. Hệ thống kiểm tra sinh viên có quyền xem bài tập của lớp này không	
C. Gateway xác minh JWT chưa hết hạn	
D. Service kiểm tra role “ADMIN” trước khi cho phép xóa người dùng	
E. OAuth2 server cấp access token sau khi người dùng đồng ý cấp quyền (consent)	
F. API kiểm tra `scope` trong token có bao gồm `submissions:write` không	

Câu hỏi thêm:

1. JWT gồm 3 phần. Kể tên và mô tả ngắn gọn nội dung mỗi phần.
2. Tại sao sách khuyến nghị dùng RS256 thay vì HS256 cho microservices? Nêu 1 lý do chính.
3. Với KBM, vẽ phân cấp ba vai trò và nêu một quyền chỉ kiểm tra được bằng quan hệ sở hữu (ownership), không kiểm tra được chỉ bằng vai trò.

9-1 `DEBUG` Sự cố bảo mật: đánh cắp JWT `L3`

Tình huống sự cố:

Báo cáo

Sinh viên A phát hiện điểm bài tập thay đổi — bài chưa nộp bỗng có điểm 100. Log cho thấy các bài nộp được tạo bằng JWT của sinh viên A, nhưng từ một địa chỉ IP khác.

Điều tra: Sinh viên B (cùng phòng) sao chép JWT từ DevTools trên trình duyệt của A khi A quên đăng xuất trên máy dùng chung. Token dùng HS256, thời hạn 24 giờ, không có cơ chế làm mới token (refresh).

Câu hỏi phân tích:

1. **Tại sao JWT stateless lại nguy hiểm trong tình huống này?** Token bị đánh cắp nhưng server không biết — không có cơ chế thu hồi (revoke). So sánh với xác thực dựa trên session (session ID có thể thu hồi ngay lập tức).
2. **Các tầng giảm thiểu rủi ro** — thiết kế 4 tầng bảo vệ:
   - Tầng 1: Thời hạn sống của token (TTL): rút ngắn thời hạn → bao nhiêu phút là hợp lý?

- Tầng 2: Access token ngắn hạn kết hợp làm mới và luân phiên refresh token (Chương 9, Listing 9.1) — giải thích luân phiên phát hiện việc tái sử dụng refresh token ra sao, và vì sao nó không tự phát hiện access token bị sao chép trong tình huống này
- Tầng 3: Nhận dạng thiết bị (device fingerprinting): gắn token với thiết bị/trình duyệt
- Tầng 4: Phát hiện bất thường (anomaly detection): nộp bài từ 2 IP khác nhau trong 5 phút → gắn cờ

3. **HS256 vs RS256:** (a) Sinh viên B đánh cắp được một JWT bearer token. B có thể dùng token này để truy cập hệ thống không? B có thể tự tạo token giả với vai trò `ADMIN` không? Trả lời cho cả hai trường hợp HS256 và RS256. (b) Nâng cao: Khác biệt nghiêm trọng hơn xuất hiện khi một service phía sau (downstream) bị xâm nhập — nếu service giữ khóa bí mật dùng chung (shared secret) của HS256 thì kẻ tấn công có thể tự ký token bất kỳ; nếu service chỉ giữ public key của RS256 thì sao? Phân tích phạm vi thiệt hại (blast radius) của từng trường hợp.
4. **Danh sách đen token (token blacklist):** Triển khai danh sách đen (Redis set) để thu hồi các token bị đánh cắp. Trade-off: mất ưu điểm "stateless" của JWT. Khi nào danh sách đen đáng để đánh đổi?
5. **Zero Trust và tin cậy phân tầng (Tiered Trust):** Tại sao mạng nội bộ (private network) không phải là bằng chứng danh tính? Đối chiếu rủi ro được chấp nhận (accepted risk) của KBM tại thời điểm khảo sát (đầu năm 2026) — tin vào header thuần (plain-header trust) và tự viết mã phân tích JWT — với mục tiêu Tier 1 (signed proof đúng audience + local authorization). Đề xuất thứ tự chuyển đổi không làm gián đoạn các request đang đi qua Gateway.

9-2 `EVAL` OAuth2 + OIDC Flow — chọn flow nào cho kịch bản nào? `L2`

Bài tập: 4 ứng dụng cần tích hợp với nhà cung cấp danh tính (identity provider) như Keycloak, Auth0:

Ứng dụng	Loại	Tương tác người dùng
A: Web app cho sinh viên (React SPA)	Ứng dụng trang đơn, chạy trên trình duyệt	Người dùng đăng nhập, phiên dài
B: Công cụ CLI cho quản trị viên	Không có trình duyệt, chạy trên terminal	Quản trị viên đăng nhập
C: Judge Service (server-to-server)	Không có người dùng, xử lý ở backend	Máy với máy (machine-to-machine)
D: Ứng dụng di động (React Native)	Ứng dụng native, màn hình đăng nhập riêng	Người dùng đăng nhập, sinh trắc học

Câu hỏi phân tích:

1. Với mỗi ứng dụng (A–D): chọn OAuth2 flow nào? (Authorization Code + PKCE, Client Credentials, Device Authorization Grant)
2. **Ứng dụng A (SPA):** Tại sao Authorization Code flow cần PKCE cho SPA? Không có PKCE → kiểu tấn công nào có thể xảy ra?
3. **Ứng dụng C (server-to-server):** Client Credentials flow chỉ định danh workload, không tạo subject là người dùng. Nếu Judge Service cần xử lý thay mặt sinh viên, hãy thiết kế cơ chế delegation có chữ ký, tách subject khỏi actor, giới hạn audience/thời hạn, và nêu cách service đích phân quyền cục bộ (local authorization).
4. KBM đổi danh tính từ nhà cung cấp danh tính bên ngoài/QLĐT/tên đăng nhập-mật khẩu lấy KBM JWT (gần với token exchange, Chương 9). Nếu chuyển sang Keycloak hoặc Microsoft Entra ID làm IdP trung tâm: những gì thay đổi? Lợi ích? Chi phí chuyển đổi?

Chương 10: Chuyển đổi thực tế

10-0 NOTE Ôn tập: các chiến lược chuyển đổi L1

Sắp xếp các bước sau đây của **Strangler Fig Pattern** theo thứ tự đúng:

Các bước (chưa đúng thứ tự):

- (A) Chuyển hướng traffic từ tính năng cũ sang service mới
- (B) Xác định tính năng ít rủi ro nhất để tách trước
- (C) Xây dựng service mới hiện thực tính năng đã chọn
- (D) Gỡ bỏ code cũ khỏi monolith sau khi service mới ổn định
- (E) Chạy shadow để so sánh kết quả cũ và mới trước khi chuyển traffic thật

Câu hỏi thêm:

1. Nêu 2 anti-pattern phổ biến khi chuyển đổi từ monolith sang microservices (tham khảo Phụ lục D).
2. **Đúng hay Sai:** "Nên tách tất cả service cùng lúc (Big Bang) để tiết kiệm thời gian." Giải thích.
3. Trong các phân hệ còn nằm trong `kbm-core`, KBM nên tách phân hệ nào trước? Tại sao? (Gợi ý: xem xét độ độc lập và giá trị nghiệp vụ)

10-1 CASE Case study: Amazon — từ monolith, SOA đến microservices (2002–2020) L3

Bối cảnh: Chỉ thị nổi tiếng được cho là của Jeff Bezos, ban hành khoảng năm 2002 (theo Steve Yegge kể lại trong "Stevey's Google Platforms Rant", 2011; trích một phần):

Trích dẫn

> "All teams will henceforth expose their data and functionality through service interfaces. ... There will be no other form of interprocess communication allowed ... Anyone who doesn't do this will be fired."

Câu hỏi phân tích:

1. Chỉ thị năm 2002 ra đời *trước* khi thuật ngữ "microservices" phổ biến (bài viết của Lewis và Fowler, 2014). Amazon gọi kiến trúc của họ là "Service-Oriented Architecture". Sự khác biệt giữa SOA của Amazon và microservices hiện đại là gì?
2. Amazon không làm "big bang" rewrite. Họ tách service dần dần qua nhiều năm. Có thể dùng Strangler Fig Pattern (Chương 10) để mô tả quá trình này không? Vẽ lịch trình 3 giai đoạn.
3. **Quy tắc "Two-Pizza Team"** — mỗi team 6–8 người, sở hữu 1–2 service. Đây có phải là Conway's Law trong thực tế? Liên hệ với Chương 2 (cấu trúc team ↔ ranh giới service)
4. Amazon hiện có **hàng nghìn** microservices. Vấn đề mới: số service phình to mất kiểm soát (service sprawl), quản lý phụ thuộc, mơ hồ về quyền sở hữu. Giải pháp? (Service mesh, service catalog, platform team)
5. **So sánh với KBM:** KBM có nhiều service/module cho một team nhỏ. Áp dụng quy tắc Two-Pizza: bao nhiêu service là hợp lý cho quy mô team này? KBM có rủi ro "quá nhiều service so với một team nhỏ" không?

10-2 EVAL Outbox Pattern vs Event Sourcing vs Change Data Capture L2

Tình huống: Team cần giải quyết vấn đề ghi kép (dual-write problem): vừa lưu DB vừa phát event. Có 3 cách tiếp cận:

Cách tiếp cận	Cơ chế	Ví dụ công cụ
Outbox Pattern	Ghi event vào một bảng trong DB (cùng transaction) → polling/CDC phát event	Custom code + cron/Debezium
Event Sourcing	Sự kiện (event) *là* nguồn sự thật (source of truth), trạng thái được suy ra từ chuỗi sự kiện	Axon Framework, KurrentDB
CDC (Change Data Capture)	Đọc transaction log của DB → tự động phát các thay đổi	Debezium + Kafka Connect

Câu hỏi phân tích:

1. So sánh 3 cách tiếp cận theo 6 tiêu chí: độ phức tạp, chi phí hạ tầng, độ trễ, độ tin cậy, thời gian học, tác động lên mô hình dữ liệu.
2. **Event Sourcing** thay đổi tận gốc mô hình dữ liệu (event là gốc, trạng thái được suy ra). Khi nào trade-off này đáng? Khi nào quá mức? Cho ví dụ domain phù hợp (ngân hàng) và không phù hợp (trang quản trị CRUD).
3. **CDC chạy bên ngoài mã ứng dụng** — không cần thay đổi code. Ưu điểm rõ ràng. Nhưng nhược điểm là gì? (phụ thuộc loại DB, phức tạp khi vận hành, nhạy cảm với thay đổi schema)
4. Chương 10 đề xuất cho KBM Outbox Pattern với polling publisher. Có hợp lý không? Nếu traffic tăng 100 lần, cách tiếp cận nào phù hợp hơn?
5. **Câu hỏi phỏng vấn:** Trong buổi phỏng vấn, người phỏng vấn hỏi: "Giải thích vấn đề ghi kép (dual-write) và 3 cách giải quyết, mỗi cách kèm một ví dụ thực tế." — Trả lời trong 3 phút.

10-3 `DESIGN` Thiết kế: đồng bộ dữ liệu và Strangler Fig `L3`

Tình huống: Nhóm phát triển đang áp dụng Strangler Fig Pattern để bóc tách tính năng "Điểm danh sinh viên" từ hệ thống Monolith cũ sang một Microservice mới. Tuy nhiên, tính năng "Tính điểm chuyên cần" vẫn nằm bên trong Monolith và cần dữ liệu điểm danh.

Câu hỏi phân tích:

1. Nếu Service Điểm danh ghi dữ liệu vào Database mới, làm thế nào Monolith lấy được dữ liệu này để tính điểm?
2. Phân tích trade-off của 3 phương án đồng bộ:
   - (A) Trigger ở Database mới đồng bộ ngược về DB Monolith.
   - (B) Tạo API trên Service mới để Monolith gọi đồng bộ (Sync API).
   - (C) Sử dụng Event Queue (Kafka/RabbitMQ) để Service mới gửi event `AttendanceRecorded` và Monolith hoạt động như Consumer lắng nghe.
3. Sắp 5 việc sau vào ma trận tác động × nỗ lực rồi chia thành 3 giai đoạn: thêm log có cấu trúc, tách schema, thêm Circuit Breaker, viết adapter ACL cho sự kiện cũ, đổi rate limit theo kỳ thi.
4. Kịch bản: Quá trình chuyển đổi kéo dài 6 tháng. Giữa chừng, trường thay đổi logic quy định thời gian điểm danh trễ. Việc duy trì logic này ở cả code Monolith lẫn Microservice tạo ra nguy cơ gì? Giải quyết thế nào?

Chương 11: Quan sát hệ thống

11-0 `NOTE` Ôn tập: ba thành phần Observability `L1`

Ghép mỗi câu hỏi vận hành với thành phần Observability phù hợp nhất: **Logs**, **Metrics**, hoặc **Traces**.

Câu hỏi vận hành	Logs / Metrics / Traces?
A. “Mức dùng CPU trung bình của Judge Service trong 1 giờ qua là bao nhiêu?”	
B. “Tại sao request nộp bài của sinh viên X mất 5 giây?”	
C. “Lỗi gì xảy ra lúc 3 giờ sáng khiến service bị crash?”	
D. “Tỷ lệ request thành công (success rate) là bao nhiêu %?”	
E. “Request đi qua những service nào và mỗi service mất bao lâu?”	
F. “Nội dung stack trace của ngoại lệ khi chấm bài thất bại là gì?”	

Câu hỏi thêm:

1. Phân biệt **SLI**, **SLO**, và **SLA**. Cho ví dụ mỗi loại trong bối cảnh KBM.
2. Tại sao cảnh báo nên dựa trên SLO (trải nghiệm người dùng) thay vì metric kỹ thuật (CPU, RAM)?
3. Gateway trả `{ "message": … }`, các service trả `{ "description": … }`. Viết lại một lỗi 503 theo RFC 9457 và nêu trường nào giúp đối chiếu với log.

11-1 `DEBUG` Gỡ lỗi: bí ẩn độ trễ tăng đột biến `L3`

Tình huống sự cố:

Cảnh báo

Dashboard Grafana báo độ trễ P99 của các lượt chấm truy vấn đơn giản tăng từ 2 s → 12 s. Xảy ra mỗi thứ Ba lúc 14:00.

Dữ liệu hiện có:

- Metrics: CPU của tất cả service < 30%. Bộ nhớ bình thường. Kafka consumer lag tăng lúc 14:00.
- Logs: Không có lỗi. Judge Service log: “Executing SQL...” — bình thường.
- Tracing: Một trace mẫu cho thấy: Gateway (5 ms) → Core (20 ms) → Kafka publish (10 ms) → Judge receive (delay 8000 ms!) → Judge execute (800 ms) → Core result (15 ms)

Manh mối: Thứ Ba 14:00 = đầu giờ “Cơ sở dữ liệu” — 120 sinh viên truy cập hệ thống cùng lúc.

Câu hỏi phân tích:

1. Từ dữ liệu trace, **điểm nghẽn ở đâu?** Kafka consumer delay 8000 ms — nguyên nhân có thể là gì? (Gợi ý: consumer lag, số partition, số consumer thread)
2. **RED analysis:** chỉ ra đại lượng RED nào xác định được từ dữ liệu đã cho; với Rate và Error rate, nêu số request, số lỗi và cửa sổ đo cần bổ sung. Phân biệt P99 của toàn hệ thống với độ trễ của trace mẫu.
3. Judge Service xử lý từng message tuần tự (chỉ một consumer thread). 120 bài nộp cùng lúc → dồn ứ trong hàng đợi. Thiết kế **chiến lược mở rộng**: (a) tăng số consumer thread/instance, (b) tăng số partition, (c) cả hai? Trade-off?
4. Thiết kế **SLO** cho độ trễ nộp bài. Hiện P99 bình thường là 2 s, lúc tăng đột biến là 12 s. SLO hợp lý: “99% bài nộp < X giây” — X bao nhiêu? Ngưỡng cảnh báo?
5. Trong vai kỹ sư trực (on-call) nhận cảnh báo lúc 14:05, mô tả **runbook** gồm 5 bước đầu tiên cần kiểm tra (từ tổng thể đến chi tiết).

11-2 `EVAL` Observability Stack: tự xây, mua hay dùng dịch vụ được quản lý (managed)? `L2`

Tình huống: CTO yêu cầu đề xuất observability stack cho hệ thống 15 microservices:

Phương án	Thành phần	Chi phí (ước tính/tháng)	Công sức thiết lập
A: Self-hosted OSS	Prometheus + Grafana + Loki + Jaeger	Chi phí hạ tầng tự quản	2–3 tuần
B: Managed OSS	Grafana Cloud (free tier → paid)	Theo mức sử dụng	2–3 ngày
C: Commercial	Datadog / New Relic / Dynatrace	Cao hơn, đổi lại có hỗ trợ	1 ngày (cài agent)
D: Cloud-native	AWS CloudWatch + X-Ray / GCP Cloud Monitoring	Theo cloud provider	1–2 ngày

Câu hỏi phân tích:

1. Với 3 hồ sơ team sau, phương án nào phù hợp nhất?
   - Startup 5 người, ngân sách $0–100/tháng
   - Scale-up 20 người, ngân sách $500/tháng, SLA 99.9%
   - Enterprise 100 người, ngành chịu quản lý chặt (ngân hàng)
2. KBM hiện đã có một số mảnh ở Cấp 2 (Actuator/Prometheus/Grafana, zerolog/PLG ở DevOps Lab) nhưng chưa chuẩn hóa đầu cuối. Lộ trình chuyển đổi đề xuất lên Cấp 3: chọn phương án nào? Tại sao?
3. **Vendor lock-in:** Ngôn ngữ truy vấn độc quyền của Datadog so với PromQL của Prometheus (chuẩn mở). Khi nào vendor lock-in chấp nhận được? Khi nào nguy hiểm?
4. OpenTelemetry (OTel) đang trở thành chuẩn chung — "instrument once, send to any backend". Chiến lược: dùng OTel ngay từ đầu → dễ đổi backend về sau. Trade-off so với dùng SDK của nhà cung cấp?

Chương 12: Triển khai & DevOps

12-0 `NOTE` Ôn tập: các chiến lược triển khai `L1`

Ghép mỗi chiến lược triển khai với đặc điểm chính của nó:

Chiến lược	Đặc điểm
A. Blue-Green Deployment	1. Triển khai cho một phần nhỏ người dùng, tăng dần nếu ổn
B. Canary Deployment	2. Duy trì 2 môi trường giống hệt nhau, chuyển traffic tức thì
C. Rolling Update	3. Gửi bản sao traffic đến phiên bản mới nhưng không ảnh hưởng người dùng
D. Shadow Deployment	4. Thay thế từng instance cũ bằng instance mới, từng cái một

Câu hỏi thêm:

1. Docker container khác gì virtual machine? Nêu 2 điểm khác biệt chính.
2. Giải thích nguyên tắc **Infrastructure as Code**. Tại sao không nên cấu hình server thủ công?
3. Với KBM (team nhỏ, Docker Compose), chiến lược triển khai nào phù hợp nhất để bắt đầu? Tại sao?

12-1 `EVAL` Docker Compose vs Kubernetes — ở quy mô nào? `L2`

Tình huống: 3 hệ thống ở các quy mô khác nhau:

Hệ thống	Số service	Traffic	Team	Yêu cầu
A: Startup MVP	3 service	500 req/ngày	2 lập trình viên	Ra mắt nhanh, ngân sách thấp
B: KBM LMS + các phân hệ lab	Nhiều service/ module	Traffic học thuật, tăng đột biến khi thi/lab	1–2 lập trình viên chính (+ sinh viên theo đợt)	Tin cậy, dễ vận hành

Hệ thống	Số service	Traffic	Team	Yêu cầu
C: E-commerce platform	25 service	1M req/ngày	15 lập trình viên, 4 team	Auto-scaling, zero-downtime, nhiều region

Câu hỏi phân tích:

1. Với mỗi hệ thống: Docker Compose hay Kubernetes? Hay kết hợp cả hai? Giải thích dựa trên các tiêu chí: năng lực team, chi phí vận hành, nhu cầu mở rộng.
2. **Anti-pattern:** Hệ thống A (3 service, 2 lập trình viên) muốn dùng Kubernetes "để sẵn cho tương lai". Tính **chi phí của Kubernetes** cho team 2 người: thời gian học, bảo trì cluster, độ phức tạp khi debug. Chi phí > lợi ích?
3. **Lộ trình chuyển đổi:** Hệ thống B (KBM) hiện dùng Docker Compose cho LMS core và k3s/Sysbox cho DevOps Lab. Khi nào *cần* mở rộng Kubernetes cho nhiều phần hơn? Xác định 3 tín hiệu kích hoạt (traffic, quy mô team, yêu cầu SLA)
4. **Managed vs Self-managed K8s:** GKE/EKS/AKS so với tự dựng cluster bằng kubeadm. So sánh chi phí và rủi ro cho hệ thống C.
5. **Câu hỏi phỏng vấn:** "How does Kubernetes ensure zero-downtime deployment?" — Trả lời: cơ chế Rolling Update, readiness probe, đủ replica sẵn sàng (PDB chỉ bảo vệ khi drain node, không giới hạn rolling update). Vẽ sơ đồ.

12-2 `DESIGN` Thiết kế: pipeline CI/CD cho microservices `L3`

Tình huống: Thiết kế CI/CD pipeline cho KBM (Java LMS core nhiều repo/service + DevOps Lab Go multi-binary):

Yêu cầu:

- Push code → tự động build → test → deploy lên staging → phê duyệt thủ công → deploy lên production
- Chỉ build/test các service bị thay đổi (không build lại toàn bộ)
- Migration cơ sở dữ liệu chạy *trước* khi deploy service
- Cơ chế rollback: quay lại phiên bản trước trong < 5 phút
- Quản lý secret: không hardcode mật khẩu trong cấu hình CI

Câu hỏi phân tích:

1. **Thiết kế pipeline:** vẽ sơ đồ CI/CD pipeline với đầy đủ các stage, gồm: phát hiện service thay đổi, build song song, cổng kiểm thử (test gate), cổng phê duyệt, triển khai.
2. **Phát hiện service thay đổi** (nếu gom các service vào một mono-repo): Git diff → xác định service nào thay đổi → chỉ build service đó. Thiết kế thuật toán. Trường hợp biên: thư viện dùng chung thay đổi → build lại *tất cả* service?
3. **Thứ tự migration cơ sở dữ liệu:** Core Service phụ thuộc vào các bảng dùng chung. Academic Service phụ thuộc vào các bảng của Core. Thứ tự migration có quan trọng không? Thiết kế đồ thị phụ thuộc (dependency graph) cho các migration.
4. **Chiến lược rollback:** Dùng canary deployment cho production. Nếu instance canary có tỷ lệ lỗi > 5% → tự động rollback. Thiết kế cơ chế (health check → metric → quyết định → rollback). Vẽ sơ đồ.
5. **GitOps vs push-based:** CI/CD truyền thống chủ động đẩy (push) bản triển khai; GitOps (Argo CD) kéo (pull) trạng thái mong muốn từ Git. So sánh trong bối cảnh LMS — nên dùng cách nào?

Capstone: Phỏng vấn thiết kế hệ thống — thiết kế "Online Judge" L3

Đề bài (dạng phỏng vấn 45 phút):

Kịch bản

"Thiết kế hệ thống Online Judge (như LeetCode/HackerRank) cho 10.000 người dùng đồng thời. Hệ thống cho phép: người dùng nộp code (Python, Java, SQL), hệ thống chạy code trong sandbox, so sánh output, trả kết quả. Có chế độ thi (1.000 người dùng nộp bài cùng lúc) với bảng xếp hạng thời gian thực."

Gợi ý cấu trúc câu trả lời (theo khung phỏng vấn thiết kế hệ thống):

1. **Làm rõ yêu cầu** (5 phút): Yêu cầu chức năng? Phi chức năng (độ trễ, thông lượng, tính sẵn sàng)? Quy mô?
2. **Thiết kế tổng quan** (10 phút): Vẽ sơ đồ kiến trúc. Xác định các service, database, message queue.
3. **Đi sâu** (20 phút): Chọn 2–3 thành phần quan trọng nhất:
   - **Thực thi trong sandbox**: Mỗi bài nộp một Docker container? Bảo mật? Giới hạn tài nguyên?
   - **Quản lý hàng đợi**: Chiến lược chia partition Kafka để lập lịch công bằng (chế độ thi)?
   - **Bảng xếp hạng**: CQRS + Redis sorted set? Tần suất cập nhật?
4. **Mở rộng và trade-off** (10 phút):
   - 1.000 bài nộp đồng thời → chiến lược mở rộng Judge Service?
   - Trade-off CAP cho bảng xếp hạng: khi có network partition, nên tiếp tục trả dữ liệu có thể cũ hay từ chối để giữ nhất quán mạnh? Khi mạng bình thường, yêu cầu độ mới và độ trễ là bao nhiêu?
   - Ước tính chi phí: mỗi bài nộp = 1 Docker container × 10 giây = tốn bao nhiêu tài nguyên tính toán?

Tiêu chí đánh giá:

- ☐ Kiến trúc có ≥4 service rõ ràng
- ☐ Kiểu giao tiếp hợp lý (đồng bộ hay bất đồng bộ, khi nào dùng gì)
- ☐ Chiến lược quản lý dữ liệu (database-per-service, CQRS cho bảng xếp hạng)
- ☐ Cơ chế chịu lỗi phù hợp từng ranh giới giao tiếp (circuit breaker khi có lời gọi đồng bộ; retry, timeout và DLT cho luồng message)
- ☐ Bảo mật (cách ly sandbox, xác thực bằng JWT)
- ☐ Phân tích khả năng mở rộng (mở rộng ngang Judge Service, chiến lược chia partition)
- ☐ Thảo luận trade-off (phân tích rõ ≥3 trade-off)

Ghi chú: Các bài tập case study dựa trên thông tin công khai từ engineering blog và tài liệu của các công ty (Netflix, Amazon, Shopify, Stripe). Các dữ kiện đã được đơn giản hóa cho mục đích giáo dục — nên tham khảo nguồn gốc để có bức tranh đầy đủ.

PHỤ LỤC F

Gợi ý hướng giải

Gợi ý hướng giải cho các bài tập trong phần "Bài tập & case study". Gợi ý chỉ định hướng, không phải đáp án đầy đủ: nhiều bài (đặc biệt mức `L2` và `L3`) là bài mở, không có một đáp án đúng duy nhất; giá trị nằm ở quá trình phân tích và lập luận.

F.1. Chương 1: Tổng quan SOA & Microservices

1-0 `NOTE` **Ôn tập** `L1`: A là 2, B là 1, C là 2, D là 3, E là 1, F là 3. Martin Fowler khuyên nên bắt đầu bằng monolith (Monolith First), chỉ tách nhỏ khi có đủ lý do xác đáng. Xem §1.6. Đối với khả năng mở rộng, kiến trúc monolith bắt buộc phải nhân bản toàn bộ hệ thống ngay cả khi chỉ một tính năng bị quá tải, trong khi kiến trúc microservices cho phép chỉ nhân bản dịch vụ chịu tải cao, giúp phân bổ tài nguyên hiệu quả hơn. Dấu hiệu monolith đã sẵn sàng để tách: mỗi thay đổi nhỏ phải phối hợp nhiều đội, chu kỳ phát hành kéo dài, không mở rộng riêng được phần chịu tải (Bảng 1.1). Về SOA: microservices đẩy mạnh Loose Coupling, Autonomy và giảm nhẹ Reusability ở cấp doanh nghiệp, điều phối qua ESB, vì độc lập được ưu tiên hơn tái sử dụng (§1.2).

1-1 `EVAL` `L2`: Công ty A: chưa nên (Not Yet) do đang là startup, chưa đạt độ phù hợp sản phẩm-thị trường (product-market fit). Công ty B: nên chuyển đổi (Yes) vì đã có đủ đội ngũ và điểm nghẽn (pain point) rõ ràng. Công ty C: không bao giờ hoặc chưa nên (Never/Not Yet) vì đây chỉ là hệ thống ERP nội bộ với lượng người dùng rất nhỏ (500 người dùng). Xem Bảng 10.1 về ma trận quyết định (Decision Matrix).

1-2 `CASE` `L3`: Shopify lựa chọn kiến trúc Modular Monolith vì ranh giới giữa các thành phần (component boundary) đã đáp ứng đủ cho mức độ mở rộng (scale) của họ. Xem §1.6.

F.2. Chương 2: Phân tích hướng domain

2-0 `NOTE` **Ôn tập** `L1`: Bounded Context tương ứng với (b); Ubiquitous Language tương ứng với (b); Aggregate tương ứng với (a). "Bài nộp" có ba nghĩa: câu trả lời SQL (SQL Practice), nội dung nộp bài theo rubric (Academic Management), bài thi có hạn giờ (Examination & Proctoring). Xem §2.3. "Tài khoản" thuộc Identity & Access. Các Bounded Context giao tiếp với nhau qua API hoặc

sự kiện. Chuyên gia nghiệp vụ (Domain Expert) bảo đảm mô hình phần mềm phản ánh đúng quy trình thực tế; nếu kỹ sư tự thiết kế một mình, ranh giới Bounded Context và Ubiquitous Language dễ bị xác định sai.

2-1 `DESIGN` `L2`: Luồng sự kiện: yêu cầu đăng ký (RegistrationRequested), kiểm tra điều kiện tiên quyết (PrerequisiteChecked), giữ chỗ thành công (SeatReserved) hoặc vào danh sách chờ (WaitlistJoined), khởi tạo thanh toán (BillingInitiated), thanh toán hoàn tất (PaymentCompleted) hoặc hết hạn thanh toán (PaymentExpired), xác nhận đăng ký (RegistrationConfirmed) hoặc hủy đăng ký (RegistrationCancelled). Các Bounded Context gồm: danh mục môn học, đăng ký, thanh toán và hồ sơ sinh viên. Theo Erl: Task là điều phối đăng ký; Entity là học phần, sinh viên, hóa đơn; Utility là thông báo. Erl đi theo quy trình, DDD phát hiện khác biệt ngôn ngữ (§2.3).

2-2 `EVAL` `L2`: Trước hết phân biệt kiểm tra định dạng ổn định dùng chung với quy tắc nghiệp vụ do một context sở hữu: quy tắc nghiệp vụ (domain logic) cần được giữ lại bên trong dịch vụ, còn thư viện kiểm tra trung lập có thể dùng chung khi lợi ích vượt coupling và có quản lý version. Thư viện dùng chung (shared library) chỉ nên chứa các đối tượng truyền dữ liệu (DTO), mã lỗi (error code) và các nghiệp vụ cắt ngang (cross-cutting concern) như ghi nhật ký (logging). `StudentId` là mô hình miền nên thuộc Shared Kernel, các context đồng sở hữu phải cùng đồng ý (§2.5). Xem Newman [4a].

F.3. Chương 3: Thiết kế dịch vụ & API

3-0 `NOTE` **Ôn tập** `L1`: A là non-breaking (thêm trường dữ liệu); B là breaking (đổi tên đồng nghĩa với xóa trường cũ và thêm trường mới); C là non-breaking (tham số tùy chọn); D là breaking (thay đổi mã trạng thái HTTP); E là breaking (xóa trường); F là breaking (thêm trường bắt buộc). Tolerant Reader là nguyên tắc trong đó consumer tự bỏ qua các trường dữ liệu không nhận diện được. Xem §3.9. Tính tương thích ngược đặc biệt quan trọng trong microservices vì các dịch vụ được triển khai độc lập; phá vỡ hợp đồng dữ liệu sẽ gây lỗi trực tiếp cho các consumer chưa kịp cập nhật. URL path rõ ràng, dễ định tuyến nhưng phải duy trì nhiều URL; custom header giữ URL sạch nhưng khó gọi thử và debug (§3.2). `@RestController` là inbound adapter, `QuestionRepository` là outbound port, `JpaQuestionAdapter` là outbound adapter; `JpaRepository` mang kiểu của Spring Data nên thuộc adapter (§3.5). CRUD dùng REST; dashboard hợp GraphQL ở tầng BFF, cần DataLoader (§3.7).

3-1 `CASE` `L2`: Stripe sử dụng định dạng phiên bản theo ngày (date-based versioning) để đảm bảo tính tương thích ngược dài hạn (backward compatibility). Idempotency Key là một chuỗi định danh (UUID) do client sinh ra và đính kèm vào header của request. Xem §3.1 (Idempotency Key), §3.2 (versioning).

3-2 `DEBUG` `L3`: Nguyên nhân gốc rễ: việc đổi tên trường dữ liệu (tương đương xóa trường cũ và thêm trường mới) khiến các consumer phiên bản cũ gặp lỗi. Phòng ngừa: kiểm thử hợp đồng (contract testing) và áp dụng mẫu Expand-and-Contract (mở rộng rồi thu hẹp). Xem §3.2, §3.6.

F.4. Chương 4: Giao tiếp đồng bộ

4-0 `NOTE` **Ôn tập** `L1`: A là 2, B là 4, C là 1, D là 5, E là 3. Circuit Breaker chuyển từ trạng thái Closed sang Open khi lỗi vượt ngưỡng, sau một khoảng thời gian thì sang Half-Open để thử lại, và quay lại Closed nếu thành công. Xem §4.4. Câu 2: tầng ngoài hết hạn trước thì kết quả của tầng trong bị bỏ phí. Câu 3: tên service được tra trong danh sách instance lấy từ Eureka, rồi client tự chọn một instance. Câu 4: lời gọi vẫn chạy nhờ danh sách đã lưu ở client, nên registry không nằm trên đường đi của mỗi request (§4.3).

4-1 `DEBUG` `L3`: Nguyên nhân gốc rễ: thiếu Circuit Breaker và đặt timeout quá dài (30 giây). Khi có Circuit Breaker, dịch vụ báo lỗi nhanh (fail-fast) thay vì giữ luồng chờ đến hết thời gian timeout. Kỹ thuật Bulkhead giúp giới hạn phạm vi ảnh hưởng (blast radius). Xem §4.4, §4.5.

4-2 `EVAL` `L2`: A chọn GraphQL vì phù hợp cho các truy vấn linh hoạt và băng thông hạn chế; B chọn gRPC vì đáp ứng thông lượng cao và schema dữ liệu nghiêm ngặt; C chọn GraphQL vì phù hợp cho cấu trúc dữ liệu lồng nhau; D chọn REST để đảm bảo tính ổn định và dễ dàng lập tài liệu. Xem §4.1 (và §3.7 cho GraphQL). Về Circuit Breaker trong KBM: đặt theo từng worker ở `kbm-judge-sql`, vì Core chỉ làm việc với một coordinator và không phân biệt được worker nào lỗi (§4.5).

F.5. Chương 5: Giao tiếp bất đồng bộ

5-0 `NOTE` **Ôn tập** `L1`: Thứ tự điền: (1) Topic, (2) Partition, (3) Offset, (4) Consumer Group, (5) Producer, (6) Broker. Luồng xử lý bài nộp (submission pipeline) của KBM nên áp dụng chiến lược chuyển giao ít nhất một lần (at-least-once) kết hợp với idempotent consumer. Xem §5.3, §5.5. `JudgeSubmission` là Command, có thể bị từ chối; `SubmissionJudged` là Event, đã xảy ra nên không từ chối được; `UserDocument` là Document (§5.6). Exchange kiểu Topic so khớp routing key với binding của từng queue (§5.2).

5-1 `EVAL` `L2`: A chọn RabbitMQ vì phù hợp cho định tuyến đơn giản và cơ chế xác nhận; B chọn Kafka vì đảm bảo thứ tự và lưu giữ luồng sự kiện; C chọn Kafka vì đáp ứng thông lượng cao; D chọn RabbitMQ vì điều phối vòng tròn và cơ chế xác nhận. Xem §5.2.

5-2 `DEBUG` `L3`: Poison Pill là thông điệp không thể giải tuần tự hóa (deserialize), khiến consumer rơi vào vòng lặp lỗi (crash loop). Giải pháp: dùng `ErrorHandlingDeserializer` để lỗi giải tuần tự hóa đi thẳng sang Dead Letter Topic, vì thử lại không sửa được dữ liệu hỏng; retry có backoff chỉ dành cho lỗi tạm thời; sau đó rà soát thủ công. Xem §5.9.

F.6. Chương 6: Giao dịch phân tán

6-0 `NOTE` **Ôn tập** `L1`: Thứ tự thực hiện: (1) Core tạo Submission, (2) Judge chấm bài, (3) Ghi điểm Gradebook, (4) Ghi bền vững lệnh thông báo vào outbox; notification worker gửi email ngoài saga với retry giới hạn và đối soát khi thất bại. Phân loại: bước 1 là Compensatable, bước 2 là Pivot, bước 3 và 4 là Retriable (§6.2). Compensation cho bước 1 là đánh dấu Submission là ERROR (C1 — Mark ERROR). Saga *không* đảm bảo ACID, nó đảm bảo ACD (tính nguyên tử qua compensation, tính nhất quán giữ bởi quy tắc nghiệp vụ trong mỗi local transaction, và tính bền vững — durability). Lưu ý: "nguyên tử qua compensation" là nguyên tử *nghiệp vụ* (hoàn tất toàn chuỗi hoặc bù trừ các bước đã làm), không phải rollback ACID — trạng thái trung gian vẫn có thể bị quan sát, và mỗi local transaction bên trong vẫn là ACID. Xem §6.2, §6.4.

6-1 `DESIGN` `L3`: Bước 1–3 (đặt chuyến, ghép tài xế, nhận chuyến) là Compensatable. Bước 4 (bắt đầu chuyến) là Pivot: chuyến đã chạy thì không hoàn tác được, như đề đã nêu. Bước 5–9 là Retriable: bước 7 thất bại thì giữ hồ sơ chuyến và chờ thử lại, không bù trừ chuyến. Orchestration phù hợp hơn vì luồng nghiệp vụ phức tạp và đòi hỏi nhiều kịch bản bù trừ (compensation). Xem §6.3.

6-2 `EVAL` `L2`: Mô hình Choreography đơn giản và có liên kết lỏng (loose coupling) nhưng khó truy vết (trace). Mô hình Orchestration cung cấp quản lý tập trung và dễ truy vết hơn, nhưng phải vận hành thêm một thành phần trung tâm và có nguy cơ gom quá nhiều nghiệp vụ vào đó. Hướng lập luận: A đơn giản nên choreography là đủ; B cần giám sát thanh toán nên nghiêng về orchestration;

C có con người tham gia, kéo dài nhiều ngày, và D có trạng thái không chắc chắn với API ngoài, nên đều cần orchestrator giữ trạng thái. Xem §6.3.

F.7. Chương 7: Quản lý dữ liệu

7-0 `NOTE` **Ôn tập** `L1`: A là 3, B là 2, C là 1, D là 4. Theo định lý CAP, chỉ khi xảy ra network partition (phân vùng mạng), hệ thống mới phải đánh đổi giữa tính nhất quán (Consistency) và tính sẵn sàng (Availability); khi không có partition, hệ thống có thể bảo đảm cả C lẫn A. Vấn đề khi KBM dùng chung cơ sở dữ liệu là các dịch vụ bị ràng buộc về schema (schema coupling), không thể mở rộng hay triển khai độc lập. Xem §7.1.

7-1 `EVAL` `L3`: Cách nói "chọn 2 trong 3" của định lý CAP là sự hiểu lầm, vì khả năng chịu phân vùng mạng (partition tolerance — ký hiệu P) không phải là một tùy chọn: network partition luôn có thể xảy ra trong hệ thống phân tán. Chỉ khi network partition thực sự xảy ra, hệ thống mới phải chọn giữa C và A. Xem §7.1.

7-2 `DESIGN` `L2`: Bước 1 là tách schema (`core_schema`, `academic_schema`), truy vấn chéo còn lại tạm đọc qua view. Bước 2 là thay truy vấn chéo bằng API rồi thu hồi quyền đọc chéo. Bước 3 là nhân bản dữ liệu tham chiếu qua sự kiện. Tách hẳn database server chỉ khi cần mở rộng độc lập. Xem §7.2 (Áp dụng cho KBM).

F.8. Chương 8: API Gateway & BFF Pattern

8-0 `NOTE` **Ôn tập** `L1`: Thuộc Gateway gồm xác minh JWT ở biên (edge token validation / auth), rate limiting, routing, SSL termination, load balancing và response caching. Không thuộc Gateway gồm business logic, database migration và tính điểm bài tập (grading logic). Xem §8.4.

8-1 `CASE` `L2`: Netflix chuyển từ Zuul 1 (blocking, mỗi request một thread) sang Zuul 2 (non-blocking, dựa trên Netty) và vận hành Zuul 2 ở quy mô lớn. Spring Cloud Gateway là dự án riêng của Spring, xây trên WebFlux (non-blocking), thay thế Spring Cloud Netflix Zuul. Cái giá của non-blocking: luồng xử lý khó theo dõi, gỡ lỗi khó hơn, và mọi thành phần trên đường đi của request đều phải bất đồng bộ. Xem §8.2.

8-2 `ADR` `L2`: Phương án 1 nhắm đúng người lạm dụng nhờ định danh đã xác thực. Phương án 2 dễ chặn nhầm cả phòng máy đi ra qua một IP NAT (§8.4.4). Phương án 3 bảo vệ đúng endpoint thi. Phương án 4 phức tạp so với nhóm nhỏ. Có thể chọn kết hợp 1 và 3. Xem §8.4.

F.9. Chương 9: Bảo mật Microservices

9-0 `NOTE` **Ôn tập** `L1`: A là AuthN; B là AuthZ; C là AuthN; D là AuthZ; E là AuthZ (OAuth 2.0 cấp quyền truy cập, không xác thực danh tính); F là AuthZ. JWT gồm Header (algorithm), Payload (claim), và Signature. RS256 cho phép xác minh (verify) mà không cần bí mật chung (shared secret), điều này quan trọng khi có nhiều dịch vụ. Xem §9.2, §9.5. Phân cấp vai trò là ADMIN > LECTURER > STUDENT; quyền "sửa bài nộp của chính mình" cần quan hệ sở hữu, chỉ xét vai trò thì không đủ (§9.5).

9-1 `DEBUG` `L3`: Khi phát hiện token bị đánh cắp (token theft), hệ thống cần áp dụng cơ chế danh sách đen (token blacklist), sử dụng access token có vòng đời ngắn (short-lived access token) kết hợp với cơ chế luân phiên (rotation) cho refresh token. Xem §9.2.

9-2 `EVAL` `L2`: A (React SPA) và D (ứng dụng di động React Native) dùng luồng Authorization Code kèm PKCE, vì cả hai là client công khai, không giữ được client secret an toàn. B (công cụ CLI không có trình duyệt, quản trị viên đăng nhập) dùng Device Authorization Grant. C (Judge Service, giao tiếp máy với máy, không có người dùng) dùng luồng Client Credentials. Ứng dụng web có backend riêng trên máy chủ, không có trong đề, dùng luồng Authorization Code chuẩn. Xem §9.4.

F.10. Chương 10: Chuyển đổi thực tế

10-0 `NOTE` **Ôn tập** `L1`: Thứ tự là B, C, E, A, D. Các anti-pattern gồm Big Bang rewrite (viết lại toàn bộ một lần) và Distributed Monolith. Chiến lược đập bỏ xây mới (Big Bang) là sai lầm do rủi ro quá cao; thay vào đó nên áp dụng mẫu thiết kế Strangler Fig để bóc tách từng phần. Với KBM, ứng viên là phân hệ thi (`kbm-exam`), vì logic Examination đã có ranh giới nghiệp vụ rõ; cần làm rõ Bounded Context trước rồi mới chuyển hướng định tuyến. Xem §10.2, Phụ lục D.

10-1 `CASE` `L3`: Sự kiện API Mandate tại Amazon năm 2002 bắt buộc mọi nhóm phát triển phải bộc lộ chức năng thông qua API (expose service API). Quá trình chuyển đổi này thực tế đã kéo dài hơn 10 năm. Câu 2: dùng các bước ở §10.2. Câu 3: xem Conway's Law ở Chương 2.

10-2 `EVAL` `L2`: Outbox Pattern đơn giản vì dùng giao dịch cục bộ của cơ sở dữ liệu; Event Sourcing tạo ra lưu vết lịch sử (audit trail) nhưng có độ phức tạp cao hơn; mô hình CDC can thiệp gián tiếp (không xâm lấn mã nguồn, non-invasive) nhưng yêu cầu công cụ bên thứ ba như Debezium. Xem §10.3.

10-3 `DESIGN` `L3`: Về ma trận: log có cấu trúc và Circuit Breaker có tác động cao, nỗ lực thấp nên làm trước; tách schema và adapter ACL tốn nỗ lực hơn; rate limit theo kỳ thi làm khi cần cho chế độ thi. Về đồng bộ: (A) khó cấu hình các trigger trên cơ sở dữ liệu và làm ẩn đi logic nghiệp vụ; (B) dễ lập trình nhưng gây liên kết chặt (tight coupling) vì monolith liên tục gọi đồng bộ đến API sẽ tạo ra điểm nghẽn; (C) hàng đợi sự kiện (Event Queue) là phương án có liên kết lỏng nhất (loose coupling) vì monolith chỉ cần lắng nghe sự kiện AttendanceRecorded để tự động cập nhật điểm. Rủi ro chung là phải duy trì logic ở hai bộ mã nguồn, do đó cần đóng băng phần logic cũ trong monolith sớm nhất có thể.

F.11. Chương 11: Quan sát hệ thống

11-0 `NOTE` **Ôn tập** `L1`: A là Metrics; B là Traces; C là Logs; D là Metrics; E là Traces; F là Logs. SLI là chỉ số đo lường thực tế (ví dụ độ trễ P99); SLO là mục tiêu nội bộ hệ thống cần đạt (ví dụ P99 dưới 500 ms); SLA là bản cam kết pháp lý với khách hàng. Lỗi 503 theo RFC 9457 có `type`, `title`, `status`, `detail`; thêm `traceId` để đối chiếu với log (§11.5). Cảnh báo nên được thiết lập dựa trên SLO, vì nếu chỉ số phần cứng (CPU) cao nhưng trải nghiệm người dùng chưa bị ảnh hưởng thì chưa cần thiết phải báo động. Xem §11.1, §11.4.

11-1 `DEBUG` `L3`: 8000 ms nằm giữa Kafka publish và Judge receive: message đang chờ trong hàng đợi, Judge không chậm khi đã nhận bài. Một consumer thread xử lý tuần tự 120 bài nên lag tăng. Thêm instance chỉ có tác dụng khi số partition tăng tương ứng (§5.9). Xem §11.3, §11.4.

11-2 `EVAL` `L2`: Tự vận hành mã nguồn mở (A, Self-hosted OSS) linh hoạt nhưng tốn công sức vận hành. Dịch vụ được quản lý cho mã nguồn mở (B, Managed OSS, như Grafana Cloud) giảm gánh nặng hạ tầng, chi phí theo mức sử dụng. Giải pháp thương mại (C, như Datadog hoặc New Relic) thiết lập nhanh, có hỗ trợ, nhưng chi phí cao. Công cụ gốc của nhà cung cấp đám mây (D, như

AWS CloudWatch) tiện lợi khi hệ thống đã chạy trên đám mây đó, nhưng có rủi ro phụ thuộc nhà cung cấp (vendor lock-in). Xem §11.1–11.3, §11.9.

F.12. Chương 12: Triển khai & DevOps

12-0 `NOTE` **Ôn tập** `L1`: A là 2 (Blue-Green), B là 1 (Canary), C là 4 (Rolling), D là 3 (Shadow). Container chia sẻ nhân hệ điều hành (OS kernel) nên khởi động nhanh; máy ảo (VM) có hệ điều hành riêng nên cô lập chặt chẽ hơn. Infrastructure as Code (IaC) đảm bảo mọi cấu hình đều được viết bằng mã, lưu trữ phiên bản và có tính lặp lại (reproducible). Đối với hệ thống KBM triển khai bằng Docker Compose đơn instance, thực chất chỉ có chiến lược All-at-once (Recreate; chấp nhận gián đoạn ngắn, chọn khung giờ thấp điểm) hoặc Blue-Green thủ công sau reverse proxy nếu cần triển khai không gián đoạn (zero-downtime); Rolling Update đúng nghĩa đòi hỏi orchestrator điều phối từng replica (Swarm/Kubernetes). Xem §12.2, §12.5.

12-1 `EVAL` `L2`: Docker Compose phù hợp cho môi trường phát triển, môi trường staging hoặc các hệ thống quy mô nhỏ. Kubernetes là lựa chọn hợp lý khi hệ thống cần tự động mở rộng (auto-scaling), tự phục hồi (self-healing) và có nhiều dịch vụ. Xem §12.6.

12-2 `DESIGN` `L3`: Pipeline CI/CD gồm các bước: gửi mã nguồn (commit), build, kiểm thử, triển khai lên staging, phê duyệt thủ công, phát hành canary lên production và phát hành toàn bộ (full rollout). Việc phát hiện service có thay đổi kết hợp kiểm tra khác biệt mã nguồn (`git diff`) với đồ thị phụ thuộc (dependency graph). Xem §12.4.

F.13. Capstone: Thiết kế “Online Judge”

Capstone `L3`: Đề xuất 4 dịch vụ tối thiểu: API Gateway, Submission Service (nộp bài), Judge Service (chấm bài) và Leaderboard Service (bảng xếp hạng). Giao tiếp: sử dụng đồng bộ (REST hoặc gRPC) cho hành động nộp bài và bất đồng bộ (Kafka) cho hàng đợi chấm điểm. Dữ liệu: áp dụng kiến trúc CQRS cho bảng xếp hạng (sử dụng Redis Sorted Set cho mô hình đọc). Tính chịu lỗi (resilience): nêu rõ ranh giới lỗi của từng luồng — Circuit Breaker khi có lời gọi đồng bộ cần bảo vệ; retry, timeout và Dead Letter Topic (DLT) cho luồng message tới dịch vụ chấm bài. Khả năng mở rộng (scale): nhân bản theo chiều ngang (horizontal scale) dịch vụ chấm bài, đồng thời chia partition Kafka theo `userId` để giữ thứ tự theo sinh viên (§5.9) — tránh key theo ID kỳ thi vì sẽ dồn cả kỳ thi vào một partition.

PHỤ LỤC G

Chiến lược kiểm thử & Testcontainers

Kiểm thử trong kiến trúc phân tán không dừng ở việc xác minh logic nội bộ, mà còn phải chứng minh khả năng chịu lỗi của hệ thống trước độ trễ và sự cố mạng. Phụ lục này trình bày cách điều chỉnh chiến lược kiểm thử khi chuyển từ monolith sang microservices: từ việc tái cân bằng tháp kiểm thử (Test Pyramid), kiểm thử thành phần với Testcontainers, đến kiểm thử hợp đồng (Contract Testing) giữa các dịch vụ độc lập.

G.1. Chiến lược kiểm thử trong microservices

Trong kiến trúc nguyên khối (monolith), quá trình kiểm thử thường tập trung vào logic nghiệp vụ nội bộ và giao tiếp trực tiếp với cơ sở dữ liệu dùng chung. Tuy nhiên, khi chuyển đổi sang kiến trúc microservices, bài toán kiểm thử trở nên phức tạp hơn nhiều. Các dịch vụ không còn chia sẻ cùng một không gian bộ nhớ; thay vào đó, chúng giao tiếp thông qua hệ thống mạng với độ trễ và tỷ lệ lỗi không thể dự đoán trước. Việc kiểm thử hệ thống phân tán đòi hỏi cách tiếp cận mới: ưu tiên cô lập môi trường kiểm thử của từng dịch vụ và đảm bảo các bên tuân thủ nhất quán hợp đồng API (API contract).

G.2. Tháp kiểm thử (Test Pyramid) trong hệ thống phân tán

Khái niệm tháp kiểm thử được Mike Cohn giới thiệu nhằm tối ưu hóa tỷ lệ giữa chi phí và tốc độ kiểm thử. Tuy nhiên, áp dụng mô hình này vào microservices đòi hỏi sự điều chỉnh chiến lược đáng kể.

G.2.1. Giới hạn của kiểm thử E2E truyền thống

Trong mô hình nguyên khối, kiểm thử đầu cuối (E2E) hoạt động như cơ chế kiểm soát chất lượng cuối cùng. Nhưng với microservices, việc khởi động toàn bộ cụm dịch vụ (ví dụ: 10 dịch vụ, 3 cơ sở dữ liệu, 1 cụm Kafka) để thực thi bộ kiểm thử E2E là một quy trình tốn kém về tài nguyên và thời gian. Hơn thế nữa, các bài kiểm thử E2E trong môi trường phân tán thường xuyên gặp hiện

tượng kết quả không đồng nhất (flaky test) do độ trễ mạng hoặc thứ tự khởi động dịch vụ. Điều này làm giảm đáng kể sự tin cậy của quy trình tích hợp liên tục (CI).

G.2.2. Dịch chuyển trọng tâm sang kiểm thử thành phần và kiểm thử hợp đồng

Thay vì phụ thuộc vào kiểm thử E2E, chiến lược kiểm thử hiện đại cho microservices tập trung mở rộng hai tầng giữa của tháp kiểm thử: **Component Testing** (kiểm thử thành phần) và **Contract Testing**. Component Testing cho phép khởi chạy và kiểm thử một dịch vụ duy nhất trong môi trường cô lập (isolated) bằng cách cung cấp các thành phần hạ tầng (như cơ sở dữ liệu riêng biệt) một cách tạm thời. Song song đó, Contract Testing đảm bảo rằng thông điệp giao tiếp giữa các dịch vụ (ví dụ: giữa API Gateway và Core Service) luôn tuân thủ đúng định dạng đã thỏa thuận mà không cần thiết lập kết nối mạng thực tế giữa chúng trong lúc kiểm thử.

G.3. Thay thế mock bằng Testcontainers

Component Testing đối mặt với một thách thức lớn: làm thế nào để cung cấp cơ sở dữ liệu và message broker cho dịch vụ khi chạy kiểm thử tự động?

G.3.1. Điểm yếu của cơ sở dữ liệu in-memory và mock

Theo truyền thống, lập trình viên thường sử dụng cơ sở dữ liệu trong bộ nhớ (in-memory database) như H2 để thay thế cho PostgreSQL, hoặc sử dụng các thư viện giả lập (mock) để thay thế Kafka. Tuy nhiên, chiến lược này tiềm ẩn nhiều rủi ro. Các truy vấn SQL đặc thù của PostgreSQL (như `JSONB` hoặc tính năng khóa bản ghi `FOR UPDATE`) không thể hoạt động chính xác trên H2. Tương tự, việc giả lập hoạt động của Kafka không thể tái hiện các kịch bản thực tế như tái cân bằng nhóm tiêu thụ (consumer group rebalancing) hay xử lý thông điệp trùng lặp. Sự sai lệch này dẫn đến tình trạng "thành công trong môi trường kiểm thử nhưng thất bại khi triển khai thực tế".

G.3.2. Testcontainers: môi trường kiểm thử cô lập

Testcontainers là một thư viện mã nguồn mở giúp lập trình viên quản lý các container Docker trực tiếp từ mã kiểm thử (ví dụ: JUnit trong Java). Thay vì sử dụng cơ sở dữ liệu giả lập, Testcontainers giao tiếp với Docker API để khởi động một container PostgreSQL thực tế và một container Kafka thực tế ngay trước khi bộ kiểm thử bắt đầu, và tự động giải phóng tài nguyên (teardown) sau khi kết thúc.

Việc áp dụng Testcontainers mang lại lợi ích kép: đảm bảo độ tin cậy cao do sử dụng cùng một công nghệ hạ tầng với production, đồng thời giữ được tính tự động hóa và cô lập cần thiết cho quy trình CI/CD. Một bài kiểm thử thành công với Testcontainers mang lại độ tin cậy tương đương với kiểm thử tích hợp thực tế.

G.4. Case study 1: Kiểm thử idempotency với Kafka

Idempotency là một yêu cầu kỹ thuật bắt buộc trong giao tiếp bất đồng bộ, nhằm đảm bảo hệ thống không xử lý sai lệch trạng thái khi một thông điệp bị gửi trùng lặp. Logic chống xử lý trùng có thể kiểm thử bằng unit test gọi handler hai lần với cùng `eventId`. Mock thông thường không phản ánh cơ chế xác nhận thông điệp (offset acknowledgment) của Kafka, nên để kiểm thử các kịch bản đó cần một broker thật: `@EmbeddedKafka` trong JVM, hoặc Testcontainers khi muốn sát hạ tầng production hơn.

Dưới đây là một minh họa rút gọn, dùng Testcontainers để kiểm thử idempotency theo bối cảnh KBM; tên lớp sự kiện được đặt riêng cho ví dụ. Kịch bản gửi cùng một sự kiện `SubmissionCreatedEvent`

(mang cùng một `eventId`) vào Kafka hai lần liên tiếp, mô phỏng tình huống kết nối mạng không ổn định khiến bên phát (producer) phải gửi lại dữ liệu.

Listing 7.1: *Cấu hình và thực thi kiểm thử idempotency với Kafka Testcontainers*

```
@SpringBootTest
@Testcontainers
class SubmissionConsumerIdempotencyTest {

    @Container
    static PostgreSQLContainer<?> postgres =
            new PostgreSQLContainer<>("postgres:15-alpine");

    @Container
    static ConfluentKafkaContainer kafka =
            new ConfluentKafkaContainer(
                    DockerImageName.parse("confluentinc/cp-kafka:7.5.0"));

    @Autowired
    private KafkaTemplate<String, Object> kafkaTemplate;

    @Autowired
    private SubmissionRepository submissionRepository;

    @DynamicPropertySource
    static void overrideProperties(DynamicPropertyRegistry registry) {
        registry.add("spring.kafka.bootstrap-servers", kafka::getBootstrapServers);
        registry.add("spring.datasource.url", postgres::getJdbcUrl);
        registry.add("spring.datasource.username", postgres::getUsername);
        registry.add("spring.datasource.password", postgres::getPassword);
    }

    @Test
    void shouldProcessEventOnlyOnce_WhenDuplicateMessagesArrive() throws Exception {
        // Configure message with Idempotency Key (eventId)
        String eventId = UUID.randomUUID().toString();
        SubmissionCreatedEvent event =
                new SubmissionCreatedEvent(eventId, "student123", "question456");

        // Send message the first time
        kafkaTemplate.send("submissions", event).get();

        // Send the EXACT SAME MESSAGE a second time (simulate duplicate)
        kafkaTemplate.send("submissions", event).get();

        // Wait for Consumer to process (using Awaitility)
        Awaitility.await()
            .atMost(Duration.ofSeconds(30))
            // Ensure count remains 1 for at least 2 seconds
            .during(Duration.ofSeconds(2))
            .untilAsserted(() -> {
                // Verify database: data is inserted exactly once
                long count = submissionRepository.countByEventId(eventId);
                assertThat(count).isEqualTo(1);
            });
    }
}
```

Bài kiểm thử này xác nhận số bản ghi giữ nguyên bằng 1 trong suốt cửa sổ chờ: ràng buộc duy nhất (unique constraint) trên cột `eventId` ở cơ sở dữ liệu ngăn việc chèn bản ghi thứ hai. Để kết luận idempotency ở mức nghiệp vụ, còn cần xác nhận consumer đã xử lý cả hai thông điệp và kiểm tra rằng tác dụng phụ chấm bài chỉ xảy ra đúng một lần.

G.5. Case study 2: Kiểm thử chuyển trạng thái khi Saga gặp lỗi

Saga pattern xử lý các giao dịch phân tán bằng chuỗi các giao dịch cục bộ và các hành động bù trừ (compensation). Việc kiểm thử luồng bù trừ đòi hỏi sự phối hợp chặt chẽ giữa cơ sở dữ liệu lưu trạng thái và message broker. Kịch bản dưới đây kiểm thử *bước chuyển trạng thái thất bại* — phần dễ kiểm chứng nhất của luồng này: nếu Judge Service gặp sự cố (ví dụ: mã nguồn không hợp lệ), nó phải gửi một sự kiện thất bại về Core Service để Core Service chuyển bài nộp sang trạng thái lỗi. Một bài kiểm thử bù trừ đầy đủ hơn còn phải tạo ra tác dụng phụ (side effect) trước đó (ví dụ trừ lượt nộp/hạn mức) rồi xác nhận hành động hoàn trả thực sự diễn ra — kể cả khi sự kiện thất bại bị gửi trùng hoặc được thử lại (retry).

Listing 7.2: *Kiểm thử chuyển trạng thái khi nhận sự kiện thất bại trong Saga*

```
@Test
void shouldTransitionToError_WhenJudgeServiceEmitsFailureEvent() throws Exception {
    // 1. Core Service receives request and initializes status to JUDGING
    Submission submission =
            new Submission("sub_1", "student_1", SubmissionStatus.JUDGING);
    submissionRepository.save(submission);

    // 2. Simulate Judge Service emitting a JudgeFailedEvent via Kafka
    JudgeFailedEvent failureEvent =
            new JudgeFailedEvent("sub_1", "Compilation Error");
    kafkaTemplate.send("judge-results", failureEvent).get();

    // 3. Verify the failure-state transition at Core Service
    Awaitility.await().atMost(Duration.ofSeconds(30)).untilAsserted(() -> {
        Submission updatedSubmission =
                submissionRepository.findById("sub_1").orElseThrow();
        // Status must move to ERROR instead of stuck in JUDGING
        assertThat(updatedSubmission.getStatus()).isEqualTo(SubmissionStatus.ERROR);
        assertThat(updatedSubmission.getErrorMessage())
                .isEqualTo("Compilation Error");
    });
}
```

Bài kiểm thử này không phụ thuộc vào mã nguồn nội bộ của Judge Service. Nó mô phỏng kết quả đầu ra của Judge Service bằng cách trực tiếp phát thông điệp thất bại vào hệ thống Kafka (được Testcontainers cấp phát). Điều này cho phép đội ngũ phát triển Core Service tự tin rằng bài nộp không bị kẹt ở trạng thái trung gian khi Judge báo lỗi, mà không cần thiết lập hệ thống kiểm thử E2E phức tạp.

G.6. Consumer-Driven Contract Testing

Nếu Testcontainers giải quyết bài toán kiểm thử logic nghiệp vụ với hạ tầng thực tế, thì **Contract Testing** giải quyết bài toán giao tiếp giữa các dịch vụ độc lập. Trong microservices, khi dịch vụ A (bên gọi API, consumer) gọi dịch vụ B (bên cung cấp API, provider), bất kỳ sự thay đổi nhỏ nào trong cấu trúc JSON của dịch vụ B cũng có thể gây ra lỗi nghiêm trọng tại dịch vụ A.

Thay vì thực hiện các lời gọi mạng thực tế, Consumer-Driven Contract Testing (CDCT, với các công cụ như Pact hoặc Spring Cloud Contract) hoạt động theo nguyên tắc: Consumer định nghĩa trước một hợp đồng (contract) mô tả kỳ vọng về cấu trúc dữ liệu trả về. Hợp đồng này sau đó được tự động chia sẻ và chạy trên hệ thống CI của Provider. Nếu Provider thay đổi mã nguồn làm vi phạm hợp đồng (ví dụ: đổi tên trường `title` thành `name`), bộ kiểm thử của Provider sẽ thất bại ngay lập tức, ngăn phiên bản lỗi được triển khai lên production. Chiến lược này giảm tải đáng kể cho hệ thống kiểm thử E2E, đồng thời nâng cao tính tự chủ cho các nhóm phát triển độc lập.

Tài liệu tham khảo

Thân sách trích dẫn các sách tham khảo chính theo mã, ví dụ [2a, Ch.3]. Tài liệu bổ sung và tài nguyên trực tuyến riêng của từng chương được liệt kê ở mục "Đọc thêm" cuối chương đó.

Sách tham khảo chính

[1] Erl, T. (2016). *Service-Oriented Architecture: Analysis and Design for Services and Microservices* (2nd ed.). Prentice Hall.

[2a] Richardson, C. (2018). *Microservices Patterns: With Examples in Java*. Manning Publications.

[2b] Richardson, C. (2025). *Microservices Patterns* (2nd ed.; bản MEAP, truy cập 07/2026). Manning Publications.

[3] Mitra, R., & Nadareishvili, I. (2020). *Microservices: Up and Running: A Step-by-Step Guide to Building a Microservices Architecture*. O'Reilly Media.

[4a] Newman, S. (2015). *Building Microservices: Designing Fine-Grained Systems*. O'Reilly Media.

[4b] Newman, S. (2019). *Monolith to Microservices: Evolutionary Patterns to Transform Your Monolith*. O'Reilly Media.

[5] Rocha, H. F. O. (2022). *Practical Event-Driven Microservices Architecture: Building Sustainable and Highly Scalable Event-Driven Microservices*. Apress.

[6] Evans, E. (2003). *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley.

[7] Kleppmann, M. (2017). *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*. O'Reilly Media.

Tài liệu bổ sung

Bellemare, A. (2020). *Building Event-Driven Microservices*. O'Reilly Media.

Beyer, B., Jones, C., Petoff, J., & Murphy, N. R. (Eds.). (2016). *Site Reliability Engineering: How Google Runs Production Systems*. O'Reilly Media.

Brandolini, A. (2013). *Introducing EventStorming*. Leanpub.

Burns, B., Beda, J., & Hightower, K. (2019). *Kubernetes: Up and Running* (2nd ed.). O'Reilly Media.

Narkhede, N., Shapira, G., & Palino, T. (2017). *Kafka: The Definitive Guide*. O'Reilly Media.

Nygard, M. T. (2018). *Release It!* (2nd ed.). Pragmatic Bookshelf.

Sridharan, C. (2018). *Distributed Systems Observability*. O'Reilly Media.

Stopford, B. (2018). *Designing Event-Driven Systems*. O'Reilly Media.

Vernon, V. (2013). *Implementing Domain-Driven Design*. Addison-Wesley.

Bài báo và tài nguyên trực tuyến

Clemson, T. (2014). *Testing Strategies in a Microservice Architecture*. martinfowler.com/articles/microservice-testing

Garcia-Molina, H., & Salem, K. (1987). Sagas. *ACM SIGMOD Record*, 16(3), 249–259.

Lewis, J., & Fowler, M. (2014). *Microservices*. martinfowler.com/articles/microservices.html

Richardson, C. (n.d.). *Microservice Architecture*. microservices.io